

数理逻辑与软件工​​程导论

形式化方法原理和应用

2023 年 4 月 23 日

目录

第一章 正确的软件	6
1.1 形式化方法	6
1.1.1 什么是形式化方法	6
1.1.2 形式化方法的历史	7
1.2 软件工程	9
1.2.1 软件开发周期	9
1.2.2 软件开发和形式化方法	11
1.2.3 形式化软件验证: Verifiable C	12
1.2.4 Verifiable C 中的证明	13
1.2.5 验证 OpenSSL HMAC	19
1.2.6 示例讨论	22
1.3 本章小节	23
第二章 形式化基础	25
2.1 背景知识	25
2.1.1 集合的基本概念	26
2.1.2 集合的大小	28
2.1.3 对角论证法	29
2.1.4 数学认识论	30
2.2 命题逻辑	32
2.2.1 语法和词汇	33
2.2.2 文法和二义性	34

2.2.3	语义, 演绎规则	37
2.2.4	模型, 系统合理性	38
2.2.5	元逻辑概念	41
2.2.6	无限多的前提	41
2.2.7	逻辑联结词的功能完备性	43
2.2.8	范式	44
2.3	可计算性	45
2.3.1	类比证明	45
2.3.2	图灵机	46
2.3.3	结构化图灵机	49
2.3.4	μ -递归函数	54
2.3.5	编码图灵计算	57
2.3.6	通用图灵机	61
2.3.7	集合的可计算性定义	62
2.3.8	停机问题	66
2.3.9	莱斯定理	69
2.3.10	波斯特对应问题	71
2.4	本章小节	73
第三章	框架: 一阶逻辑	77
3.1	一阶逻辑	77
3.1.1	语法和词汇	77
3.1.2	语义	79
3.1.3	演绎规则	81
3.1.4	相继式演绎	85
3.1.5	可靠性和完备性	87
3.1.6	切消定理	90
3.1.7	切消定理的应用	94

3.1.8	演绎的实现	96
3.2	一阶算术	98
3.2.1	等于	98
3.2.2	算术语言	99
3.2.3	算术阶层	101
3.2.4	罗宾逊算术	105
3.2.5	哥德尔第一不完备定理	107
3.2.6	皮亚诺算术	111
3.2.7	哥德尔第二不完备定理	112
3.2.8	可证明条件的推导	114
3.2.9	更弱的算术	116
3.3	高阶逻辑	119
3.3.1	语法和语义	120
3.3.2	二阶皮亚诺算术	120
3.3.3	亨金语义	121
3.4	本章小节	123
第四章	集合论组件	125
4.1	公理集合论	125
4.1.1	朴素集合论	126
4.1.2	罗素悖论	127
4.1.3	ZFC 集合论	128
4.1.4	集合的基本运算	130
4.1.5	有序对	131
4.1.6	关系和函数	135
4.1.7	无限	136
4.2	数学对象	137
4.2.1	自然数	137

4.2.2	次序关系	139
4.2.3	递归函数	141
4.2.4	算术模型	142
4.2.5	实数	143
4.2.6	实数完全性	145
4.3	集合结构	149
4.3.1	序数和替换公理模式	149
4.3.2	良序和良基结构	150
4.3.3	序数定义	153
4.3.4	序数分类	156
4.3.5	序数运算	158
4.3.6	康托尔范式定理	160
4.3.7	哈托格数	161
4.3.8	基数	162
4.3.9	选择公理	164
4.4	本章小节	167
第五章	集合论与元逻辑	168
5.1	证明与模型	168
5.1.1	证明与模型的形式化	168
5.1.2	冯·诺伊曼全集	170
5.1.3	正则公理	172
5.2	内模型	174
5.2.1	可构造全集	174
5.2.2	ZF 的内模型	179
5.2.3	可构造性的绝对性	180
5.2.4	可构造全集的全局良序性	185
5.2.5	相对一致性的形式化	186

5.3	力迫法	189
5.3.1	力迫法构成	189
5.3.2	力迫关系	192
5.3.3	力迫关系的应用	195
5.3.4	可构造公理的独立性	197
5.3.5	序数可定义集合	198
5.3.6	迭代力迫	201
5.3.7	选择公理的独立性	204
5.3.8	力迫法的起源与发展	207
5.4	基础的基础	210
5.4.1	逆向数学	210
5.4.2	形式主义数学	210
5.4.3	非标分析	211
5.4.4	形式化数学数据库	215
5.5	本章小节	219

第一章 正确的软件

本章主要介绍形式化方法包括涉及的数理逻辑，以及软件工程的背景和概念。狭义的形式化方法一般指软件正确性验证中的特定方法，这边给出的是更广义的概念。1.1 节介绍形式化方法及其历史，并以此引入数理逻辑的相关概念。1.2 节介绍软件工程，主要涉及其中的开发部分和其与形式化的关系，并给出了两个 C 语言程序的形式化验证例子。会看到现主流软件工程指导原则的模糊性，这种模糊性也带来了软件工程尤其是质量控制的主要问题。同时也会看到现有形式化方法应用的复杂性，尽管这种复杂性源于其的严格精确性，但也意味着我们至今还未找到一种通用且易用的形式化处理办法。

1.1 形式化方法

可以认为，广义的形式化方法是数理逻辑的基础和主要研究对象之一，而通过这种方式方法认识理解数学又是数学哲学的主要流派之一。同时，数理逻辑中的一个重要分支，计算理论，又可认为是计算机科学的理论基础。

本节中，我们将首先介绍形式化方法的基本概念，并从其产生和发展历史的角度出发，引入数理逻辑的主要概念及其主要分支。

1.1.1 什么是形式化方法

形式化方法，顾名思义，就是基于形式而非内容的方法。比如三段论：

前提： 如果 A ，那么 B
 如果 B ，那么 C
结论： 如果 A ，那么 C

就是形式化的。因为我们不需要知道 A 、 B 、 C 的具体含义，就可以由前提得到结论。人类对自然语言的处理是非形式化的，因为我们很难只通过语法从一门陌生的语言中获得信息。而计算机对程序语言的处理是形式化的，因为通常认为计算机并不理解其所处理的内容。

剥离了内容理解也就摆脱了不同理解可能带来的歧义，这种客观性可以说是形式化方法的最大优势。而在没有内容的情况下，形式化方法的处理依赖的是形式匹配规则。形式逻辑可以看作是这种方法的核心框架，一般分为语法、语义和演绎规则三部分。其中广义的说演绎规则也可被视为语法上的概念，因其构成和执行均是基于形式的。而语义可以看作是内容的部分，更确切的说，是外部赋予的形式和内容的对应关系，形式化方法的执行不依赖语义，但其对外部世界的作用需要体现在形式所代表的内容上。

例如对于上面的三段论，语法是指 如果 ... 那么 ... 的格式，演绎规则指前提到结论的匹配规则。最

后通过赋予特定的内容，我们可以得到如下有意义的推断：¹

前提： 如果现在是雨季，那么天会下雨
 如果天会下雨，那么人们需要带伞
 结论： 如果现在是雨季，那么人们需要带伞

在很多其它定义中，形式化方法会被描述为基于数学模型的方法。本质上数学模型的严格性依赖于数学语言和证明过程的形式化特点。虽然大多常见的数学证明是由半自然语言的形式给出的，有时会为了简练，留有诸如“观察可得”、“以此类推”这样的空白，使未经训练的读者很难完全跟上证明的过程。但这样的证明都可以被严格的扩展到基于形式化的过程下面，实际上，不能形式化给出的证明往往是错误的²。不过需要指出的是，尽管证明是形式化的，但人类寻找证明的过程一般都会建立在对内容的理解上。后面我们会看到简单的机械化的证明过程搜索非常低效。

形式逻辑可以看作数学的基础，也可以看作计算机科学的基础。形式逻辑的具体形式有很多种，在其中一些的基础上构建的系统可以作为另一些的基础³，也有些可以互相描述。更具体的方法可以看作是基于数学模型的，也可以看作是非数学的，这些只是命名上的区别，本质上都是基于形式的。

最后是关于形式和内容的区别，唯物的看，一切内容都可以说是基于形式的，也就是没有超越物质的精神存在。不过我们至今还不知道智慧到底是什么，比如人工智能真的是智能吗，让机器去处理自然语言，我们可以说机器理解自然语言了吗。也许形式和内容的区别还是只是命名上的区别。不过在此之前，我们仍然还是有足够多的例子去说明这种程度上的区别所带来的变化。现在我们先看看历史。

1.1.2 形式化方法的历史^{4 5}

戈特洛布·弗雷格 (Gottlob Frege) 1879 年出版的《概念文字 (Begriffsschrift)》可以说是继亚里士多德之后逻辑学的又一个里程碑，它标志着形式逻辑的诞生，尽管它的重要性要在多年以后才被人们逐步的意识。为了避免直觉和主观判断对数学研究的影响⁶，弗雷格想要寻找一种精确并且严格的方法。用弗雷格自己的话说⁷，他的目标是：

“因此我们会问，一方面，我们如何逐步得到给定的命题。另一方面，我们如何最终给其以最严格的保证。”

“显然，最可靠的方法是通过纯粹的逻辑构建证明，不依赖客体具体的特点，而只依赖一切知识背后的法则。”

“为防止不经意间受到直觉的影响，我花了很大的努力使推理链中没有缝隙。”

“通过这个例子，我们看到通过纯粹的，不受先验观念或直觉影响的，仅依赖于自身构成形式的思维过程，同样可以得出那些初看之下仅能从直觉中得到的结论。”

这种以形式逻辑来研究数学，加上使用数学反过来研究形式逻辑特点的学科被称为数理逻辑。其中纯粹基于形式的研究之后发展出了数理逻辑中我们的第一个分支，证明论⁸。

¹更多关于形式逻辑的技术细节见第二章、第三章。

²数学符号的形式化定义和证明过程见第四章、第五章。

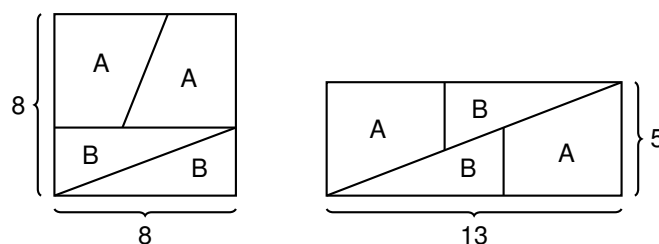
³见第五章。

⁵Wikipedia contributors. Church-Turing thesis — Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/w/index.php?title=Church%E2%80%93Turing_thesis&oldid=1042787335. 2021.

⁶图 1.1 给出了一个不严肃的问题示例，引自 Tim Doyle et al. “Proofs Without Words and Beyond - Why We Write Proofs”. In: Convergence (2014).

⁷选译自 Gottlob Frege. “Begriffsschrift”. In: From Frege to Gödel. Ed. by Jean Van Heijenoort. Cambridge: Harvard University Press, 1967, pp. 1–82.

⁸更多证明论相关的方法见第三章。

图 1.1: $64 = 65$? 一个糟糕的证明

在弗雷格建立起他的一整套形式系统后，1902 年，他遇到了逻辑学史上最著名的悖论——罗素悖论。形式逻辑在已有的前提下，通过特定的规则，得出新的结论。而随着前提和规则的复杂化，可以得出的结论也变得越来越复杂。当一套形式化系统可以同时得出相互矛盾的结论时，这套系统就不再符合真实世界了，或者说不再一致了。罗素悖论，除了对格奥尔格·康托尔（Georg Cantor）的朴素集合论的影响之外，也给了弗雷格的系统一个足以得出矛盾结论的实例。

通俗版的罗素悖论——理发师悖论：

- 一个理发师声称他要为所有不给自己理发的人理发，那理发师该给自己理发吗
- 如果接受理发师的声明，那他给不给自己理发都会得到矛盾。所以出问题的是理发师的声明，对理发师来说，满足他声明的客户的集合是不存在的。
- 对于罗素悖论来说，会产生矛盾的形式表述也不应该存在。

为了解决罗素悖论，提出了两种（之后成为主流的）方案。一是恩斯特·策梅洛（Ernst Zermelo）1908 年提出并在之后由亚伯拉罕·弗兰克尔（Abraham Fraenkel）、陶拉尔夫·斯科伦（Thoralf Skolem）和约翰·冯·诺伊曼（John von Neumann）完善的策梅洛-弗兰克尔（ZFC）集合论。另一个是由伯特兰·罗素（Bertrand Russell）1903 年首先提出，并在之后由里昂·奇维斯特克（Leon Chwistek）、弗兰克·拉姆齐（Frank Ramsey）简化，由鲁道夫·卡尔纳普（Rudolf Carnap）、威拉德·范奥曼·蒯因（Willard Van Orman Quine）和阿尔弗雷德·塔斯基（Alfred Tarski）公式化的类型论。¹

至此，我们都在努力将直觉从证明中，或者说将内容从形式中脱离出来。然而没有内容的形式对外部世界是没有意义的，接着我们来探索下形式和内容的关系。

纯粹形式上的不一致并不会给形式化系统本身带来困扰，形如 $A \text{ 且 } B$ 的语句不比 $A \text{ 且非 } A$ 更规范。不允许矛盾存在的是其所表达的内容。处理两者之间区别的关键在于形式到内容的对应关系上。对于一个不一致的形式描述，或者说形式理论，这种对应关系是不存在的。一个形式理论所对应的那部分内容被称为该理论的模型。这样，一个不一致的形式理论没有模型。同样的，一个形式理论可能对应多个本质上有区别的模型。这种形式理论和模型关系的研究被称为模型论。李奥帕德·勒文海姆（Leopold Löwenheim）在 1915 年第一次提出，此后由斯科伦完善的勒文海姆-斯科伦定理被认为是模型论的开端²。

当这种形式和内容的对应关系开始被更深入的研究，人们开始区分形式和内容上的概念。其中可以称得上最著名的结论是库尔特·哥德尔（Kurt Gödel）分别在 1929 年，1931 年发表的哥德尔完备性定理和哥德尔不完备定理。可以说第一条肯定了形式和内容上的对应关系，第二条又给出了它们之间的差异。这些差异也意味着大卫·希尔伯特（David Hilbert）在 1920 年提出的希尔伯特计划——为全部数学提供一个安全的形式化基础——破产。

在数理逻辑发展的同时，作为其中的另一个分支，计算理论（也称递归论）也开始进入了人们的视

¹更多关于罗素悖论和集合论的技术细节见第四章，类型论见??。

²关于模型的更严谨的定义、模型论的主要定理见第二章第三章。

野。计算理论的核心概念是可计算性，一个问题是可计算的是指可以通过有效手段解决的，一个可计算函数简单的说是指存在有效算法从输入得到相应的输出。而不同的计算模型指的就是计算所采用的不同形式的形式化步骤。

二十世纪三十年代，几种独立的计算模型几乎同时被提了出来，包括：

1. 1933 年，哥德尔和雅克·埃尔布朗 (Jacques Herbrand) 提出了 μ -递归函数计算模型。
2. 1936 年，阿隆佐·邱奇 (Alonzo Church) 提出了 λ 演算。
3. 同年，艾伦·图灵 (Alan Turing) 提出了图灵机。

根据邱奇-图灵猜想，任何可有效计算的问题都可由图灵机计算。这是一个假说，因为“可有效计算”的定义是模糊的。但所有已经提出的实际可操作的计算模型，包括 μ -递归函数和 λ 演算的计算能力都是和图灵机相同的。

计算理论和形式化的关系主要体现在以下几个方面¹：

1. 计算模型是形式化的。
2. 计算理论的理论极限，比如停机问题，和形式逻辑的理论极限，如不完备定理有着密切的关系。事实上，哥德尔在证明其的不完备定理时首次形式化了一系列 μ -递归函数计算过程。
3. 计算和证明有很高的相似性。包括之后由哈斯凯尔·柯里 (Haskell Curry) 和威廉·阿尔文·霍华德 (William Alvin Howard) 相继发现的柯里-霍华德同构。
4. 计算模型可以被视为解决形式化证明的工具，一个正确的形式化证明过程一定是可计算的。
5. 为了更容易地验证计算的正确性，人们提出了各种不同形式的形式逻辑和方法。

接下来，让我们关注更具体的计算，也是本书主要讨论的形式化方法应用目标——软件。

1.2 软件工程

软件工程是一门涉及软件产品所有方面的工程学科²。而软件产品的生命周期又包括软件开发周期和软件交付、运行、维护和回收等过程。同时，在一个软件生命周期中可能包含多个软件开发周期，以支持软件功能的更新。并且多个开发周期在时间上也并不一定是串行的，不同周期的不同阶段可能同时展开进行³。本书主要关注软件开发周期中涉及的方法与技术。

1.2.1 软件开发周期

软件开发周期一般包括需求分析、设计和实现、测试和维护等阶段。其中需求分析阶段主要通过和用户的沟通确定软件包含的功能，设计和实现阶段根据需求分析结果设计编码实现软件功能，测试和维护阶段验证软件正确性，保障软件功能符合用户需求。这样的阶段划分本身是模糊的，不同的软件开发实践中，不仅多个开发周期下的阶段可能会有重叠，同一个开发周期中的各个阶段之间也可能会有重合甚至倒置⁴。同时，不同软件开发在各个阶段采用的技术方法也不尽相同。这里例举两个著名软件开源项

¹其中 1 见第二章。2 见第三章。3 见??。4 5 见????。

²Ian Sommerville. Software Engineering. 10th ed. 2015.

³P. Bourque and eds. R.E. Fairley. Guide to the Software Engineering Body of Knowledge, Version 3.0. 2014.

⁴如测试驱动的开发模式会要求先编写测试用例。

目——Linux 内核¹和 LLVM²的开发周期特点。

简单的说，Linux 内核是一个操作系统内核。操作系统内核是负责管理整个系统，包括硬件、其它系统软件和应用软件的核心软件部分。Linux 内核项目可以说是目前最大且最活跃的开源软件项目之一，而从小到音乐播放器，大到超级计算机均可以看到 Linux 内核的身影。一个 Linux 内核的版本发布周期同样可以分为需求分析和设计、实现、测试和维护这几个部分。邮件是 Linux 内核社区成员，包括用户、开发者和维护者之间的主要沟通手段。Linux 内核开发人员在收到一个用户需求时，社区鼓励在尽早的阶段进行讨论以优化设计，包括如避免对已有模块的误用等。对于代码补丁提交前的准备，以下要求译自内核开发手册：

- 尽可能的测试代码。确保测试中使用了内核提供的测试工具，确保可以在各种合理的配置选项组合下通过内核编译，确保可以通过交叉编译等。
- 确保代码符合内核内核代码风格要求。
- 如果你的代码会对性能造成影响，需要在你的提交中包含相关的性能基准测试结果。
- 确保你和你的雇主有权提交这些代码，并同意通用公共许可证 (GPL)。

除了鼓励尽早沟通以外，其它关于代码设计和实现的讨论基本都在提交阶段以代码审核的形式进行。只有通过审核的代码才允许提交到公共分支，提交的代码会经过多个公共分支进入主干，每次进入更上层的分支都可能会进行更广泛的代码审核，并会有更多的社区成员开始使用你的代码。在进入主干之后，确定稳定版之前，仍会有多个小版本发布以确保代码的稳定性。Linux 内核一般每三个月左右发布一次稳定版本。对于文档的要求，除了必要的提交日志以外，对用户接口的修改，启动参数的修改，文档是必须的。对于内部接口和实现，鼓励以特定格式的注释形式和代码一起提交，这些注释可以通过特定的工具生成相应的文档。总的来说，文档没有特别严格的格式要求，例如对于提交日志：

- 日志需要包括解决的问题，解决的方式，参与者，任何对性能的影响，以及任何其它需要注意的部分。

而关于上面提到的测试工具，包括³：

- 额外的调试编译选项，如用以检查可能错误返回函数的返回值是否被忽略，对象的分配和释放是否乱序，以及锁是否误用的选项。
- 更重的，也就是对性能有更大影响的，记录所有获取释放动作的锁检查工具 *lockdep*。
- 模拟一些常见错误，如内存申请失败的错误注入工具。
- 代码模式检查和自动转换工具 *Coccinelle*。
- 静态代码检查工具 *sparse*。

最后对于所提交的代码对现有代码的影响，最重要的一点是不能影响现有功能，不能造成功能的回退⁴。

接着我们来看 LLVM 项目。LLVM 是一套编译框架，提供了一套通用的编译中间表示和一系列模块化、可重用组件以方便用户快速构建编译器。例如其中的 Clang 子项目提供了 C/C++/Objective-C 语言到中间表示的前端实现，和 LLVM Core 共同组成了支持多个常见体系平台的编译系统。以下主要关注 LLVM 和 Linux 内核项目开发上主要的不同点。其中最大的区别是 LLVM 的代码提交有对测试用例的要求。尽管 Linux 内核也有自动化测试的子项目，但并不要求随代码一起提交相应的测试用例。相对的，Linux 内核项目非常注重社区测试，同时通过不同的分支逐渐扩大使用群体来减少潜在问题的影响范围。对于 LLVM：

- 漏洞修复和新功能的添加需要包含相应的测试用例，包括单元测试、回归测试和可能的正确性和性能的覆盖测试等。
- 提交的代码需要通过现有的测试。
- 持续集成系统会执行提交后检查。

而对于代码分支，LLVM 一般只区分主干和发布分支。另外对于可信的提交者，LLVM 允许提交后的代

¹<https://www.kernel.org/doc/html/v5.15-rc1/process/development-process.html>。

²<https://releases.llvm.org/13.0.0/docs/GettingInvolved.html>。

³更多关于静态、动态检查工具的技术细节见??。

⁴回归测试目标就是避免出现功能的回退，这里的回归是来自英文单词 regression，其本身也是回退的意思。

码审核。最后，LLVM 的稳定版本一般每六个月发布一次。

通过以上两个例子，我们可以看到软件工程在方法选择和应用上有很大的灵活性，并没有一个统一的最佳实践方案，很多技术指导都是原则性的，具体的问题解决很大程度上依赖直觉和经验，或者说不太形式化。接下来我们来更具体的讨论软件开发和形式化方法的关系，并给出应用形式化软件验证技术的例子。

1.2.2 软件开发和形式化方法

软件需求分析的产出一般被称为软件规格说明。软件规格说明需要描述软件所包含的功能和使用方法，它是用户和开发者之间的约定。这里的用户并不一定指终端用户，比如对于操作系统内核来说，它的用户可以是应用程序，也可以是硬件驱动程序；对于服务提供商来说，它的用户可以是客户端软件；对于软件框架、库或者一个组件来说，它的用户可以是软件中的其它组件。不管是文档化的，还是口头的软件规格说明都将为之后的软件开发提供依据，它是设计和实现的目标，也是内部测试的准则。一份好的规格说明首先应该是没有矛盾的，这在工程实践中并没有看起来这么容易做到，尤其是在新的周期对软件功能进行迭代修改的时候，新功能需求可能造成旧功能回退。同时，一份好的规格说明应该是不易造成歧义的，歧义不仅会造成用户和开发者对最终产品的不同理解，也会使开发者和开发者，开发者和测试者之间对软件设计和实现产生不同理解。这种含义的精确性也不仅针对在规格说明中提到的内容，对于没有提到的内容的处理方式，用户和开发者也应该是有共识的。

除了软件规格说明的不精确性可能给软件产品带来问题以外，验证软件实现是否满足软件规格说明的不精确性同样，甚至更有可能带来软件问题。验证软件一般有两类方法，一是通过运行软件观察结果¹来确定软件的正确性，这种运行可能是有针对性的测试，也可能是用户的实际使用。这种运行可能是自动的，比如通过维护测试用例保证相对稳定的测试场景覆盖率。这种运行也可能运行在比实际场景更严格的环境下，比如在更大的压力下运行，或者有更严格的动态检查工具监督。如果在每一种可能的场景下，软件都能运行得到正确的结果，那我们就可以说软件是正确的。显然这里的问题是每一种，运行的具体场景是无限多的，即使在有自动化测试的情况下，对这些具体场景的分类在大多数时候也依赖的是开发或测试者的经验和直觉。并且即使有严格的分类方法，场景的类别数也可能是（近乎²）无限多的。然后，验证软件的另一种方法是证明，任何足够复杂的系统其设计、开发或审核者在开发过程中一定在不断尝试说服自己系统的正确性，不严格的说，不管有没有文档化，这也是一种证明过程。而更严格的，教材或者论文里（新）算法的介绍一定包含算法正确性的证明。我们可以认为通过正确证明的系统是正确的系统，那么这里的问题是如何确定证明的正确性。

至此，我们发现我们面临的是和弗雷格一样的问题，如何避免直觉和主观判断对软件正确性的影响。而我们将采用同样的解决方案——形式化。

需要指出的是软件产品本身是形式化的，不管是高级语言还是机器语言都是形式语言，一种基于严格语法的语言，它们的运行都严格按照一定的规则。一些时候，软件运行结果具有随机性，但这种随机性最终来源于外部输入的随机性。这种随机性可能来自于未定义的行为，也可能是某些输入，比如时间的天然属性³。因此，撇开软件产品本身，这里的形式化更多的是指形式化的描述用户需求，并形式化的检查软件实现是否符合这种需求描述，也就是形式化软件验证。

¹ 这种运行的方法可以看作是模型验证方法。更多关于模型验证和基于证明的方法定义见??。

² 严格的说，一个实际离散系统所包含的状态数总是有限的，但往往过大，比如一个 4G 的内存可能对应的状态数是 $2^{4G \times 8}$ 。

³ 更多关于软件随机性的说明见??。

1.2.3 形式化软件验证：Verifiable C

让我们来看一个基于证明的形式化软件验证的示例，我们将使用 Verified Software Toolchain (VST)¹ 证明以下 C 实现的（非最优的）素数（质数）检查函数符合注释中给出的要求。我们也将从中看到形式化软件验证的优势和挑战。

```
isPrime.c
1 | #include <stdint.h>
2 | int32_t isPrime(uint32_t a) { // 输入 a > 1; 输出非零值若 a 是素数，否则 0。
3 |     int32_t p = 1;
4 |     for (uint32_t i = 2; i < a; ++i)
5 |         if (a % i == 0) p = 0;
6 |     return p;
7 | }
```

VST 原理上采用的是霍尔逻辑²变体、分离逻辑，实现上是在 Coq³ 形式化开发和证明系统基础上建立的一套 C 语言的编译和证明系统，其中编译由编译器 CompCert⁴ 完成，证明在该编译的中间文件上完成。而 CompCert 本身由 Coq 开发并严格验证，意味着在证明了中间文件的正确性后，我们就可以得到正确的可执行文件⁵。分离逻辑加上中间文件（对应的 C 子集）共同构成了 Verifiable C 程序逻辑。这里⁶对于 isPrime.c，我们可以通过命令 `clightgen -normalize isPrime.c` 得到 Coq（源码）格式的中间文件 isPrime.v⁷，并通过 `coqc isPrime.v` 编译该文件。下面的介绍将忽略大多数技术细节，但仍会给出一个完整的规格说明和证明实现。

Coq 文件 `verif_isPrime.v` 包含了全部规格说明和证明实现。在执行完上面的命令后，我们可以通过 `coqc verif_isPrime.v` 编译（验证）这个文件。首先是一些准备工作⁸：

```
verif_isPrime.v
1 | Require Import VST.floyd.proofauto. (* 导入 VST 库 *)
2 | Require Import isPrime. (* 导入我们的中间文件 isPrime.v *)
```

定义素数谓词 Prime：

```
4 | (* a 为素数 (Prime a 为真) 是指 a 是大于 1 的整数，且若正整数 x 可以整除 a，则 x = 1 或 a *)
5 | Definition Prime a : Prop := a > 1 /\ (forall x, x > 0 /\ (x | a) -> x = 1 \/ x = a).
```

接着定义 isPrime 函数的规格说明 isPrime_spec。其中 `_isPrime` 是函数在编译中间文件中（自动）定义的标识符。这里一个函数的规格说明包括前置条件和后置条件两个部分，前置条件是指对输入的要

¹<https://vst.cs.princeton.edu/>. [Accessed: 2022-09-17].

²霍尔逻辑的细节见??。

³<https://coq.inria.fr/>. [Accessed: 2022-09-17].

⁴<https://compcert.org/>. [Last modified: 2022-07-19].

⁵确切的说是满足规格说明的汇编文件

⁶我们可以通过：`docker run -it coqorg/coq:8.13.2`
`opam install -y coq-vst.2.8`

安装运行环境，并将源文件放到当前目录下，同时记得保存 docker 镜像。

此外，还可以参考 Lennart Beringer Andrew W. Appel and Qinxian Cao. Verifiable C. ed. by Benjamin C. Pierce. Vol. 5. Software Foundations. Version 1.1.1, <http://softwarefoundations.cis.upenn.edu>. Electronic textbook, 2021 了解包括安装在内的更多 VST 技术细节，也可以通过整个系列了解 Coq 更多信息。

⁷.v 代表 Coq（曾经的，现已被统一到 Gallina 语言下）的命令语言 Vernacular。

⁸其中 (* *) 为注释部分，另外更多 Coq 相关技术细节见??。

求，后置条件是指对输出的要求，分别对应 PRE 和 POST 两部分：

```

7 | Definition isPrime_spec : ident * funspec :=
8 | DECLARE _isPrime WITH a : Z                                (* 有整数 a *)
9 |   PRE [ tuint ]                                           (* 前置条件 *)
10 |     PROP (1 < a <= Int.max_unsigned)
11 |     (* 函数的第一个参数值 uint32_t a 等同 Vint (Int.repr a), 等同的具体含义见下方正文 *)
12 |     PARAMS (Vint (Int.repr a))
13 |     SEP (TT)
14 |   POST [ tint ] EX p,                                    (* 后置条件, 存在整数 p *)
15 |     PROP (Int.min_signed <= p <= Int.max_signed
16 |           /\ (p <> 0 <=> Prime a))                          (* p 不等于 0 当且仅当 a 是素数 *)
17 |     RETURN (Vint (Int.repr p))                            (* 函数的返回值 *)
18 |     SEP (TT).

```

技术上，这里需要注意的是 a （其定义对应代码中的 $a : Z$ ）和 p 的类型是由 Coq 定义的整数 Z ，而前面素数谓词 `Prime` 的定义也是（隐式）关于 Z 的，这个整数可以简单理解为数学概念上的整数，它不需要考虑如 C 整型计算中可能的溢出问题。换句话说，比如对于 a 相关的标识符，Coq 中的 $a : Z$ 并不表示 C 代码中的参数 `uint32_t a`。和 `isPrime` 函数一样，`uint32_t a` 中的 a 标识符实际对应的是中间文件中定义的 `_a`。所谓的等同可以理解为函数输入参数对应的 `_a`（所代表）的值等于 `Vint (Int.repr a)`，即 $a : Z$ 转换到 32 位机器表示（括号内的部分）后对应的 C 表示。VST 通过这种等同关系在基于 Coq 整数的证明和 C 语句的执行之间建立联系。这种联系成立的基础包括比如前置条件中要求的 $a \leq \text{max_unsigned}$ ，和后置条件中要证明的 $\text{min_signed} \leq p \leq \text{max_signed}$ 。

从软件工程的角度来说，仅从这份规格说明中，我们至少可以看到以下两点优势：

- 形式化的给出了输入参数 $a > 1$ 的要求，这个要求仅从 C 的函数签名中我们是无法得到的。这个要求表明函数实现可以不考虑 $a \leq 1$ 时的处理方式。
- 形式化的说明了在 a 为素数时，返回值仅保证非 0。从 C 的函数实现中我们可以推出更强的保证，也就是 a 为素数时返回值为 1，但这可能并非是开发者希望提供的保证，可能随着版本的更新这个行为会发生变化。这种问题在更复杂的场景下更为常见，原则上，我们需要遵循的标准不应该是实现标准。

1.2.4 Verifiable C 中的证明

接着我们要开始证明了，首先证明两条辅助引理 L1 和 L2：

```

20 | Require Import micromega.Lia.
21 |
22 | (* 对整数 x, 若 0 <= x <= max_unsigned, 且 x 的 32 位机器表示等于 0 的机器表示, 则 x = 0 *)
23 | (* 注意 A -> B -> C 等价于 A /\ B -> C *)
24 | Lemma L1 x : 0 <= x <= Int.max_unsigned -> Int.repr x = Int.zero -> x = 0.
25 | Proof.
26 |   intros. assert (Int.unsigned (Int.repr x) = Int.unsigned Int.zero). auto with f_equal.
27 |   assert (A : Int.unsigned (Int.repr x) = x). apply Int.unsigned_repr. trivial.
28 |   rewrite <- A, <- Int.unsigned_zero. trivial.
29 | Qed.
30 |
31 | (* 若整数 a > 0, 则 a 不能被 i, 1 < i < a 整除等价于 a 是素数 *)
32 | Lemma L2 a : a > 1 -> (forall i, 1 < i < a -> ~ (i | a)) <-> Prime a.
33 | Proof.
34 |   intros. split.
35 |   - intro A. split; [ lia | intro i; specialize A with i; intro;
36 |     assert (i = 1 ∨ 1 < i < a ∨ i = a ∨ i > a) by lia;
37 |     assert (i > a -> ~ (i | a)) by (intro; assert (E : a mod i <> 0) by (
38 |       assert (a mod i = a) by (apply Z.mod_small; lia); lia);

```

```

39 |         contradict E; apply Zdivide_mod; trivial); tauto ].
40 | - intro A. destruct A. intro i. specialize H1 with i.
41 |     assert (1 < i < a -> i > 0 /\ i < 1 /\ i < a) by lia. tauto.
42 | Qed.

```

辅助引理的细节暂不展开，不过可以看到证明应用了一些更基础的定理，比如在 L2 的证明过程中应用了 `Zdivide_bounds`: $(a \mid b) \rightarrow b \neq 0 \rightarrow \text{Z.abs } a \leq \text{Z.abs } b$ (若 a 可以整除 b ，且 b 不等于 0，则 a 的绝对值小于等于 b 的绝对值)。

然后，我们准备证明这个示例的目标，函数实现符合规格说明——引理 `body_isPrime`:

```

44 | Definition Gprog := [isPrime_spec]. (* 涉及的函数接口 *)
45 | (* 收集涉及的 C 全局变量，此示例中没有 *)
46 | Definition Vprog : varspecs.mk_varspecs prog. Defined.
47 | (* 收集涉及的 C 复合类型，即 struct/union 定义，没有 *)
48 | Instance CompSpecs : compspecs.make_compspecs prog. Defined.
49 |
50 | (* f_isPrime 是中间文件中定义的函数体，可以理解为代表了函数实现，而 _isPrime 仅代表函数名 *)
51 | Lemma body_isPrime : semax_body Vprog Gprog f_isPrime isPrime_spec.

```

先简单的说明一下证明的结构。如图 1.2 所示，本质上我们从函数控制流入口（调用的前置条件）出发，跟随控制流的走向，证明流程图里特定节点上的属性，从而证明控制流出口的属性（返回的后置条件）：

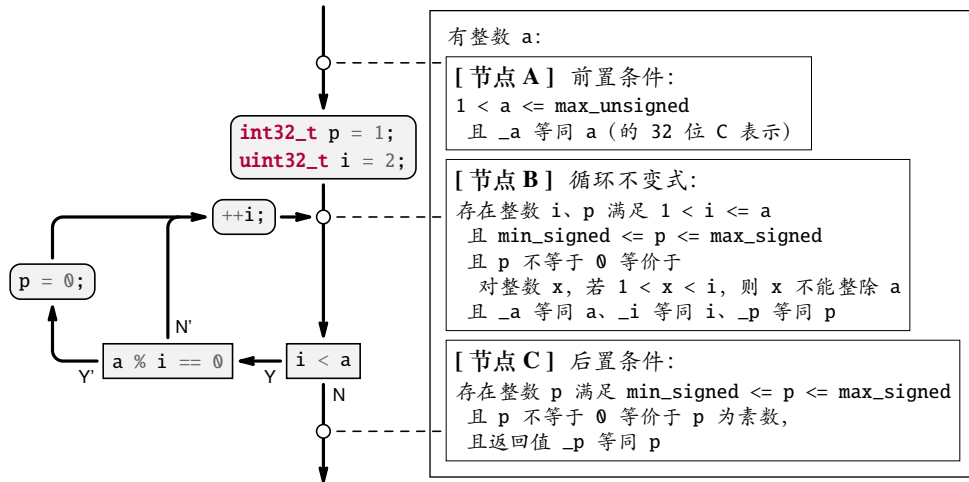


图 1.2: `isPrime` 的控制流程图和节点属性

图 1.2 中给出了程序运行到 A、B、C 三个节点需要满足的完整属性条件。在讨论节点的具体属性之前，我们先来看一下流程图。从图上我们可以清晰的看到证明函数满足规格说明等价于由条件 A 可以（经过函数执行）推导出 B。即等价于证明以下四条路径：

1. A 到 B;
2. B 经 Y、Y' 到 B;
3. B 经 Y、N' 到 B;
4. B 经 N 到 C。

在证明这些路径之前，我们来看下符号执行和循环不变式的定义：

```

4 Proof.
5 start_function. forward.
6 forward_for_simple_bound a ( (* a : Z 为循环计数器的最大值 *)
7 (* i : Z 为循环计数器, 其等同关系的建立如下面关于局部变量的注释所诉 *)
8 EX i, EX p,
9 PROP (1 < i (* forward_for_simple_bound 会自动添加 i <= a *)
10 /\ Int.min_signed <= p <= Int.max_signed
11 /\ (p <> 0 <-> forall x, 1 < x < i -> ~ (x | a)))
12 (* 建立局部变量 (如 _a 到 a) 的等同关系。会自动添加类似 _i 到 i 的等同关系,
13 这里加上“类似”的原因见下方正文更多对等同关系的说明及注释 *)
14 LOCAL (temp _a (Vint (Int.repr a)); temp _p (Vint (Int.repr p)))
15 SEP (TT)).

```

前面提到了 `coqc` 编译工具, 但 `Coq` 的可交互性来自于其提供的其它组件。更重要的是大多数时候我们很难通过仅阅读 `Coq` 证明代码掌握证明过程, 我们需要知道各个步骤下的上下文, 比如已知的条件和当下的 (子) 目标。这里主要介绍命令行交互工具 `coqtop`¹。我们可以通过 `coqtop -l verif_isPrime` 加载并进入交互界面。如果加载的文件只包含到目前为止 (1 - 15 行) 的内容, 运行以上命令后, 可以通过 `Show.` 指令查看当前 (全部子目标的) 上下文, 也可以输入新的操作 (如 16 行) 检查上下文的变化, 下面马上会看到这样的例子。

VST 库为我们提供了符号执行操作, 如这里的 `start_function` 和 `forward*`。符号执行是指输入并非实际运行中使用的具体数据, 而是符号的执行方式, 其执行结果可以用来表示输入和输出的关系²。比如对于 C 函数 `int f(int i) { return i + 1 + 1; }`, 其符号执行结果是对于输入参数 `i`, 输出返回值等于 `(i + /* C int 加 */ 2)`。

另一个需要注意的是这里为局部变量建立等同关系的方式, 如 `temp _a (Vint (Int.repr a))`。更精确的, 等同关系应理解为 `_a` 在符号执行到当前节点位置³时所代表的值等于 `Vint (Int.repr a)`。

我们首先证明路径 `I A` 到 `B`, 也是 `Coq` 在这里为我们分配的第一个子目标。

¹ 尽管图形化的工具会更友善, 但命令行更轻, 对环境的依赖更少, 交互方式也更统一。

² 更多符号执行细节见??。

³ 上面的注释中使用了类似是因为对于 `forward_for_simple_bound`, 在需要执行自增操作 `++i` 时, VST 会要求先证明循环不变式的成立, 再执行自增操作 (而如果在注释位置的 `LOCAL ( )` 中直接添加 `temp _i (Vint (Int.repr i))` 应表示先自增再证明)。当然先证明时对应的目标也会考虑自增效果。同样的, 在有自增操作时, 图 1.2 中的节点 B 位置也并不完全符合 VST 的实现。通过交互运行这个证明并观察输出, 我们会看到这个 (和其它一些) 细微的差别。不过这并不影响对基本原理的理解。

```

[demo] $ head -n 63 verif_isPrime.v >temp.v && coqtop -l temp
Welcome to Coq 8.13.2 (September 2021)

body_isPrime < Show 1.
subgoal 1 is:

  Espec : OracleKind
  a : Z
  Delta_specs : Maps.PTree.t funspec
  Delta := abbreviate : tycontext
  H : 1 < a <= Int.max_unsigned
  POSTCONDITION := abbreviate : ret_assert
  MORE_COMMANDS := abbreviate : statement
  =====
  ENTAIL Delta,
  PROP ( )
  LOCAL (temp_p (Vint (Int.repr 1)); temp_a (Vint (Int.repr a))) SEP (TT)
  |-- EX p : Z,
    PROP (1 < 2 /\
          Int.min_signed <= p <= Int.max_signed /\
          (p <> 0 <-> (forall x : Z, 1 < x < 2 -> ~ (x | a))))
    LOCAL (temp_a (Vint (Int.repr a)); temp_p (Vint (Int.repr p)))
    SEP (TT)

body_isPrime < █

```

图 1.3: coqtop 加载 1 到 15 行时相应的第一个子目标的上下文

图 1.3 是我们当前目标的上下文，这里简单解释一下图中的内容。一个（子）目标是指在已知的前提下，通过一系列证明操作，得到期望的结论。在这个图中，分割线（14 行）以上的部分是已知的前提，以下是期望的结论。在前提中我们看到有 $a : Z$ (8)，这个是在规格说明中通过 WITH 引入的，另外有 $H : 1 < a \leq \text{Int.max_unsigned}$ (11)，这个对应节点 A（函数）的前置条件，由 PRE 引入，H 是由 Coq 分配的标识符。

前提中其它的部分均为 VST 自动生成的和规格说明以及 C（中间文件）代码执行相关的条件，分别是 Espec 对应的 C 引用的外部接口的规格说明（我们的示例中同样没有），Delta_specs 引用的全局变量规格说明，Delta 变量类型上下文，POSTCONDITION 后置条件，MORE_COMMANDS 后续的语句。为获得更多细节，我们可以在 coqtop 中使用 `unfold abbreviate in Delta.` 指令查看 Delta 的内容，也可以相应的将 Delta 替换成 POSTCONDITION 或 MORE_COMMANDS。此外，这里总是可以尝试 VST 提供的 `hint.` 指令来获取下一步的建议。

对于期望的结论，尽管看起来非常复杂，它的核心结构是 `ENTAIL Delta, P |-- Q`，含义是在上下文 Delta 下，若满足 P，则满足 Q。这里 P 对应的是节点 A 加上符号执行了 A 到 B 之间语句之后对应的条件，Q 对应的是节点 B 的条件，循环计数器 `uint32_t i` 的值已经被 `forward_for_simple_bound` 带入到相关的条件之中。值得注意的是，这里 Q 的含义是存在 $p : Z$ 满足跟随其后的条件和等同关系（`PROP ( )` 和 `LOCAL ( )`），此处 p 的作用域仅限这部分条件和等同关系。VST 符号执行并不会因为这种等同关系而对 p 产生影响，因为符号执行时这里的 p 已经出了作用域范围而不再存在了，我们会在后面的证明中更清晰的看到这一点。

因此我们要证明的是 $P |-- Q$ ，也就是在 P 的条件下，我们要给出一个满足 Q 中条件的整数 p 。恰巧这里 1 就满足所需的条件（比如 2 在 `temp_p (Vint (Int.repr 1))` 的条件下不满足 `temp_p (Vint (Int.repr p))`）。所以我们通过 `Exists 1.` 指令告诉 VST 这点，然后通过 `entailer!` 指令看看 VST 能自动帮我们做到什么程度。

```
16 | - Exists 1. entailer!.
```

技术上，行首的 `-` 字符可以帮我们结构化整个证明过程，告诉 Coq 我们正在证明第一个子目标。然后让我们看看目标的变化。

```
1  Espec : OracleKind
2  a : Z
3  Delta_specs : Maps.PTree.t funspec
4  H : 1 < a <= Int.max_unsigned
5  =====
6  1 <> 0 <-> (forall x : Z, 1 < x < 2 -> ~ (x | a))
```

图 1.4: 16 行运行后的新目标

如图 1.4 所示，新的目标简化了很多，并且 `<->`（当且仅当）两侧都为真¹。我们可以通过以下指令证明这个（子）目标²：

```
17 | assert (1 <> 0) by lia. assert (forall x, 1 < x < 2 -> ~ (x | a)) by lia. tauto.
```

我们证明了第一条路径，下面将快速的看一下剩下三条路径的证明，仅从概念的层面考察最重要的几点步骤说明。首先这里 `forward_if` 包含了路径 2 和路径 3，但这里实际包含了 3 个子目标，`+` 字符标识了这 3 个嵌套的子目标。`forward_if` 除了引入了真假 (Y'/N') 路径对应的两个子目标以外，还因为 C 中 `a % i` 运算而要求证明 `i` 不等于 0。你可能注意到这里的证明采用了 `assert (i = 0)`，这是因为使用了反证法，即通过 `i` 等于 0 得到矛盾。我们可以通过交互运行检查这一点，将看到分割线下方的期望结果是 `False`。同时，这里应用了我们之前证明的辅助定理 L2。第二个 `+` 对应路径 2: B 经 Y 、 Y' 到 B 。和路径 1 类似，这里 0 是满足条件 $(p <> 0 \leftrightarrow \text{forall } x, 1 < x < i + 1 \rightarrow \sim (x | a))$ ³ 的 p 值，因为 `i` 就可以整除 `a`⁴。

```
18 | - forward_if.
19 |   + assert (i = 0) by (refine (L1 i _ H3); lia). lia. (* 证明 False *)
20 |   + forward. Exists 0. entailer!. (* a % i == 0 分支 *)
21 |     assert (a mod i = 0) by (refine (L1 (a mod i) _ H1);
22 |       assert (0 <= a mod i < i) by (apply Zmod_pos_bound; lia); lia).
23 |     assert (~ (forall x, 1 < x < i + 1 -> ~ (x | a))) by (
24 |       contradict H3 with i; pose proof (Zmod_divide a i);
25 |       assert (1 < i < i + 1 /\ i <> 0) by lia; tauto). tauto.
26 |   + forward. Exists p. entailer!. (* a % i != 0 分支 *)
```

第三个 `+` (26 行) 对应 3: B 经 Y 、 N' 到 B 。其中 `Exists p` 中满足条件的 `p` 对应的是已知前提中的 `p`，如上面对作用域范围的讨论所述，它和证明目标中的存在 `p` 只是名字相同。而前提中 `p` 的存在是因为 `forward_for_simple_bound` 再次帮我们做了简化并为其分配了标识符 `p`。我们会在路径 4 的证明中看到未被简化的上下文。代入 `p` 到存在，并执行 `entailer!`。后，图 1.5 是我们的新目标。

¹我们会在第二章探讨右侧为什么是真的。

²这里形如 `assert A by B`。的操作是指我们可以通过 `B` 指定的步骤证明 `A`，如果只有 `assert A`。那么 Coq 会建立一个 `A` 对应的新的子目标。

³注意这里自增操作还没执行，所以不变式被相应的修改了。

⁴证明中形如 `contradict A`。指令（根据上下文结构）的其中一种作用是将从前提 `A` 证明如结论 `~ B` 转换成从 `B` 证明 `~ A`。

```

1  Espec : OracleKind
2  a : Z
3  Delta_specs : Maps.PTree.t funspec
4  H : 1 < a <= Int.max_unsigned
5  i : Z
6  H0 : 2 <= i < a
7  p : Z
8  H1 : Int.repr (a mod i) <> Int.repr 0
9  H2 : 1 < i /\
10     Int.min_signed <= p <= Int.max_signed /\
11     (p <> 0 <=> (forall x : Z, 1 < x < i -> ~ (x | a)))
12     =====
13     p <> 0 <=> (forall x : Z, 1 < x < i + 1 -> ~ (x | a))

```

图 1.5: 26 行后的新目标

本质上，图 1.5 告诉我们有整数 a 、 i 、 p 满足 H 、 $H0$ 、 $H1$ 、 $H2$ ，要证明图中 14 行的结论。这里我们使用分类讨论和数学归纳的方法，即在 p 等于 0 和不同于 0 两种情况下，有在 i 时命题成立（前提条件），证明 $i + 1$ 时命题成立（期望结论）。细节上，在分类讨论之前我们需要证明我们的分类包含了所有情况，即 p 要么等于 0，要么不等于 0。之后这两种情况分别对应了 * 标识的两个新的子目标。

```

27  assert (A : p = 0 ∨ p <> 0) by lia. destruct A. (* p 要么等于 0, 要么不等于 0 *)
28  * assert (~ (forall x, 1 < x < i + 1 -> ~ (x | a))) by ( (* p 等于 0 *)
29      assert (B : ~ (forall x, 1 < x < i -> ~ (x | a))) by tauto;
30      contradict B; intro x; specialize B with x; auto with zarith). tauto.
31  * assert (forall x, 1 < x < i + 1 -> ~ (x | a)) by ( (* p 不等于 0 *)
32      intros x **; assert (1 < x < i -> ~ (x | a)) by (generalize x; tauto);
33      assert (B : 1 < x < i ∨ x = i) by lia; destruct B; [ auto | contradict H1;
34      f_equal; refine (Zdivide_mod a i _); rewrite <- H6; trivial ]). tauto.

```

终于我们来到了路径 4: B 经 N 到 C 。注意这条路径上没有实际的运算操作，因此我们主要关注两点。第一点是我们来看一下更规范（未调整简化）的目标上下文结构。

```

1  Espec : OracleKind
2  a : Z
3  Delta_specs : Maps.PTree.t funspec
4  Delta := abbreviate : tycontext
5  H : 1 < a <= Int.max_unsigned
6  POSTCONDITION := frame_ret_assert
7      (function_body_ret_assert tint
8        (EX p : Z,
9          PROP (Int.min_signed <= p <= Int.max_signed /\
10             (p <> 0 <=> Prime a))
11          RETURN (Vint (Int.repr p))
12          SEP (TT)%argsassert (stackframe_of f_isPrime)
13      : ret_assert
14  =====
15  semax Delta
16  (EX p : Z,
17    PROP (1 < a /\
18          Int.min_signed <= p <= Int.max_signed /\
19          (p <> 0 <=> (forall x : Z, 1 < x < a -> ~ (x | a))))
20  LOCAL (temp_i (Vint (Int.repr a)); temp_a (Vint (Int.repr a));
21  temp_p (Vint (Int.repr p))) SEP (TT) (return _p;) POSTCONDITION

```

图 1.6: 34 行后的新目标¹

如图 1.6 所示，除去表面上的复杂性，其期望结论的核心结构是 `semax Delta P C Q`，其中这里 `C` 是 `(return _p;)`。这是一个标准的霍尔逻辑会采用的格式，`P`、`Q` 分别代表语句 `C` 执行前后对应的条件，也就是路径中 `B`、`C` 两点要满足的条件。可以很清晰的看到我们要通过 16 行的存在 `p` 证明第 8 行 `POSTCONDITION` 展开中的存在 `p`，这也可以更清晰的看到这两个 `p` 的不同作用域范围。

要从存在一个整数满足 `P` 中的条件证明存在一个整数满足 `Q` 中的条件，我们先通过 `Intros p'`。具体化第一个整数，接着符号执行，接着告诉 `VST p'` 就满足 `Q` 中条件，接着化简¹，并应用 `L2`：

```
35 | - Intros p'. forward. Exists p'. entailer!. clear Espec Delta_specs H H1.
```

```
1 a, p' : Z
2 H0 : 1 < a
3 H2 : p' <> 0 <-> (forall x : Z, 1 < x < a -> ~ (x | a))
4 =====
5 p' <> 0 <-> Prime a
```

图 1.7: 35 行后的新目标

```
36 | pose proof (L2 a (Z.lt_gt 1 a H0)). tauto.
37 | Qed. (* 证毕 *)
```

至此证明结束。从这个示例里我们已经可以很明显的感受到 `VST` 的一些优缺点了，不过在讨论它们之前，让我们再看一个更贴近实际生产的示例。

1.2.5 验证 OpenSSL HMAC²

`OpenSSL` 是一个用于保障计算机网络中通信安全的软件库，它可以用于如防止通信内容窃取和远端身份认证。`HMAC`³ 是其采用的一种加密认证算法。在 `HMAC` 算法使用场景中，消息发送端和接收端共享会话密钥 `k`，当发送端发送消息 `m` 时会附加算法加密后的认证码 `s = HMAC(k, m)`。接收端在收到消息 `m` 和其附带的 `s` 时，会重新计算 `s' = HMAC(k, m)` 并比对 `s` 是否等于 `s'` 以确认发送端的身份。

因此，和其它加密算法一样，保证 `HMAC` 有效的核心在于保证密钥 `k` 的安全。而保证 `k` 安全的核心在于 `m` 和 `s` 之间的相关性，也就是要保证第三方难以直接通过观察（一系列）`m` 和 `s` 猜到 `k` 的值。为此，和很多其它加密算法一样，`HMAC` 在计算过程中调用了加密散列函数。这里我们关心的加密散列函数的两个特点是，这也是 `HMAC` 期望拥有的两个特点：

1. 它是一个散列函数，既对一个确定的输入，它会产生一个确定的输出。
2. 它是一个单向函数，也就是人们很难通过它的返回结果确定它的输入参数。在输入参数不同时，它的返回结果应该表现出随机性⁴。

¹通过 `- unfold abbreviate in POSTCONDITION` 指令而不是 `Show 1` 得到。

²这里使用 `p'` 只是为了区分，也可以用 `p`。另外清理了不必要的前提。

³Lennart Beringer et al. "Verified Correctness and Security of OpenSSL HMAC". in: 24th USENIX Security Symposium (USENIX Security 15). Washington, D.C.: USENIX Association, Aug. 2015, pp. 207–221. URL: <https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/beringer>.

⁴密钥散列消息码 (keyed-Hash Message Authentication Code)

⁵没有人（攻击者，也包括接收端）可以精确预测一个（真）随机函数的行为。

SHA 安全散列算法是加密散列函数的一类，而这个示例实现中采用的 SHA-256 是其中的一种。以下是具体的 HMAC 伪码实现：

$$HMAC(k, m) = SHA_{256}((k \oplus opad) \parallel SHA_{256}((k \oplus ipad) \parallel m))$$

其中 $\oplus$ 表示异或操作， $\parallel$ 表示字符串连结，*opad* 和 *ipad* 是特定的常量。

在此背景下，这里的规格说明，代码实现和证明目标分别是：

规格说明：HMAC 算法是一种安全的消息认证算法；

代码实现：*hmac.c* 及其调用的 *sha.c*；

证明目标：*hmac.s* 和 *sha.s*（中的汇编指令操作序列）满足规格说明。

其中形式化规格说明的关键在于对安全的定义，也就是对输出结果的（伪）随机性的定义。这里采用的方法¹是基于博弈论去比较加密函数和（真）随机函数²的输出结果。简单的说，在这里的博弈中，我们在加密和随机函数中（随机）选择并输出其计算结果，我们的对手需要猜测该结果的来源，猜对即获得（反比于猜测耗时的）收益。我们重复这个过程并期望我们对手的平均收益尽可能低。那么这里应该期望多低呢，对于加密算法，我们总是可以通过暴力破解的方式获得密钥³，因此，我们期望我们对手的平均收益应低于其采用暴力破解时所获得的收益（的一定比例）。可以看出尽管复杂，这个定义是可形式化的。下文中我们将（形式化后的）这个定义称为加密安全性。

为了证明上述目标，我们需要将其分解简化。在分解目标之前，让我们先从编程语言语义⁴的角度，对软件开发中的形式化方法做一个粗略的分类：

- 公理语义，即如示例 *isPrime* 中系统行为被描述为前置条件和后置条件。事实上，不仅每段路径上有对应的前后置条件，每个基本操作，如 `a % i == 0` 也都有对应的条件。且基本操作的前后置条件是由具体语言决定的，即语义（行为）是由语言决定的。因此基本操作的前后置条件可以看做是由外部提供的假设，也因此被称为公理。而我们的证明本质上是在这些基本操作对应的公理上推导这些操作的组合，也就是代码块符合其所对应的前后置条件描述。
- 操作语义，即系统行为被其它（一般情况下）更简单的计算模型（比如另一种语言或另一种算法）所描述。在已知两种模型基本操作的对应关系的情况下，我们需要证明对应代码块之间的等价性。比如对于编译过程，要求编译后的目标代码等价于源代码，也可以说目标代码的操作语义须符合源代码。这里的证明是由编译器开发人员（一般情况下非形式化的⁵）给出的。在这个 HMAC 验证示例中，我们将看到很大一部分证明会基于这种语义描述⁶。
- 指称语义，即用数学对象描述系统行为，证明将基于数学理论（如集合论中的数学对象）进行。注意计算模型中的概念并不能简单对应到数学对象上，比如计算中的函数并不等价于数学中的函数，数学中的函数要求自变量不能是函数本身（否则会带来矛盾），而计算中，比如递归调用中，将函数自身作为参数传给自己是一种很常见的做法⁷。

¹ Mihir Bellare and Phillip Rogaway. Code-Based Game-Playing Proofs and the Security of Triple Encryption. mihir@cs.ucsd.edu 13498 received 30 Nov 2004, last revised 16 Dec 2006. 2004. URL: <http://eprint.iacr.org/2004/331>.

² 真随机函数的随机性来源于系统随机池。系统通过记录（假定随机的）外部设备操作，比如键盘按键时刻，网络包到达时刻等建立随机池。

³ 或者说对于散列算法，我们总是可以通过暴力破解的方式获得散列实现。

⁴ 可理解为代码对应的具体行为。

⁵ 如前所述，CompCert 编译器的编译过程是经形式化验证的。

⁶ 前面给出的 HMAC 伪码实现就是一种操作语义描述。

⁷ 研究指称语义下对应数学对象的理论一般指域理论，我们会在??中介绍。

有了操作语义的概念，我们可以开始分解证明目标了。

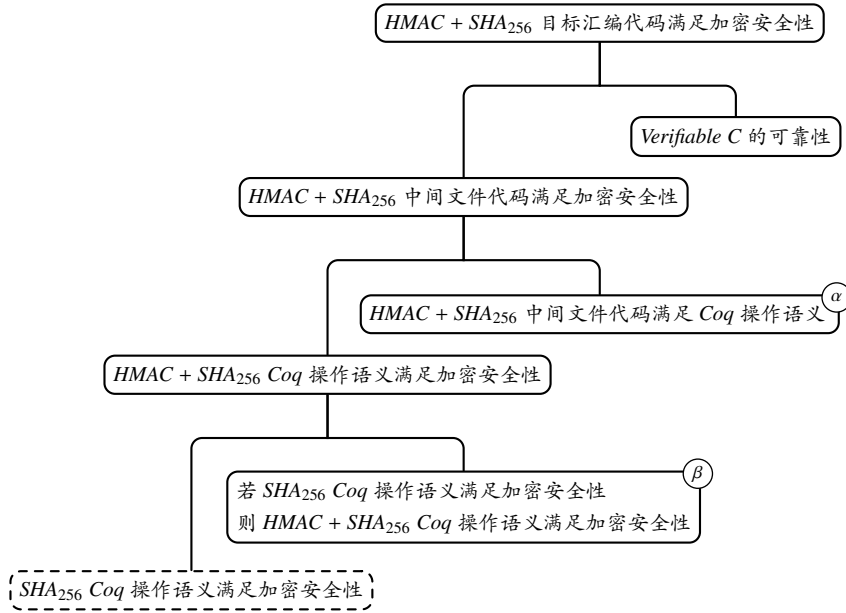


图 1.8: HMAC 验证自顶向下的目标分解树

图 1.8 给出了（简化后的）目标树，其中 Verifiable C 的可靠性是指若 CompCert 中间代码满足某条件，则其生成的汇编代码也保证满足该条件。而 Coq 操作语义是指由 Coq 语言编写的符合 FIPS¹ 的计算过程²。最后，虚线框表示目前为止还没有人知道如何证明这个目标（当然也没有证伪，这里假定它是正确的，但这不是一个安全的假设）。因此可以看到，在 Verifiable C 的可靠性已经由 VST 保证，而虚线框中的目标还没有完成理论证明的情况下，这里的主要工作是形式化证明标记为 α 和 β 的两个（子）目标，并串连起整个证明树。在 β 的证明中，这里使用了 Foundational Cryptography Framework (FCF)³，一个同样是基于 Coq 的处理涉概率编程逻辑的框架。简单的说，这里 β 目标要证明的就是 HMAC 实现完整保留了 SHA-256 算法的（伪）随机性。

此处我们略过具体的实现细节，仅引用其中整个验证过程涉及的代码行数对比，作为一个不精确的工作量评估指标。首先是（子）目标 α ，（C 对应的）中间文件满足 Coq 操作语义规格说明：

C 行数	Coq 行数	SHA-256 组件	C 行数	Coq 行数	HMAC 组件
	169	Coq 操作语义 (FIPS 180-4)		159	Coq 操作语义 (FIPS 198-1)
71	247	接口（规格）说明	25	374	接口（规格）说明
	1022	辅助引理	25	533	规格说明总计
10	229	addlength()		875	辅助引理
81	1650	sha256_block_data_order()	74	1530	HMAC_Init()
10	43	SHA256_Init()	7	101	HMAC_Update()
38	1682	SHA256_Update()	27	196	HMAC_Final()
31	1484	SHA256_Final()	5	31	HMAC_Cleanup()
7	58	SHA256()	21	99	HMAC()
248	6574	总计	134	2832	证明总计

¹联邦信息处理标准，其中的一些规格说明是以操作语义的方式给出的。比如在 FIPS 180-4 标准中指定了 SHA-256 算法的实现方式。在 FIPS 198-1 中指定了 HMAC 的实现方式，即对应前文中的伪码实现，其中 SHA-256 可替换为任意满足条件的加密散列函数。

²Coq 提供了函数式计算功能，见后面的讨论。

³Adam Petcher and Greg Morrisett. The Foundational Cryptography Framework. 2014. arXiv: [1410.3735](https://arxiv.org/abs/1410.3735) [cs.PL].

接着我们来看（子）目标 β 和证明树的串连。其中 HMAC 抽象规格说明是指在 β 的证明中使用的另一种（未在图 1.8 中显示，由 FCF 提供支持的）中间操作语义描述：

Coq 行数	HMAC 抽象规格说明保留（伪）随机性	Coq 行数	HMAC 规格说明等价性证明和串连
70	抽象规格说明	3017	HMAC 通用等价性定理及证明
25	定理定义	993	通用定理应用于 SHA-256
377	定理证明	4010	总计
472	总计		

1.2.6 示例讨论

以上两个示例在一定程度上展示了 VST 的特点，也部分说明了形式化软件验证的特征。这里除了继续看好的方面，还会对其中的问题做更多的讨论。

除了规格说明可以提供精确的接口描述外，从证明中我们也能看到整个过程的严格性，比如 `isPrime` 中对整型范围和取模运算的检查。这种严格性是形式化方法的一般特征，而这种严格性的缺失也是软件工程中大多数混乱的来源。

技术上，你可能已经注意到了 Coq 相关语言的独特性。事实上，Coq 提供了两种基础语言和非常强大的语法扩展能力。依靠这种能力，像 VST、FCF 这类工具（库）除定义了额外的类型和函数外，还提供了各自如 `PROP ( )` `LOCAL ( )` `SEP ( )` 这样的操作符。实际上，大多数 Coq 内建的操作符（如 `/\ ->`）也是通过同样的方式定义的。对于基础语言，Coq 提供了 Gallina（包括 Vernacular）和 Ltac 两种。在基本理念上，Coq 希望通过柯里-霍华德同构（所描述的计算和证明的相似关系）提供一个统一的（函数式¹）计算和证明框架。而 Gallina 就是这个框架提供的（核心）函数式语言，可以同时用来实现计算和验证证明。同时为了简化证明的编写，Coq 还提供了 Ltac 帮助实现像 `forward`、`entailer` 这样的证明策略。本质上，Ltac 是一种元编程语言，类似 C 提供的宏，它将 Ltac 代码转换成 Gallina，再由 Coq 的统一框架检查。

不同于 Coq 内建的面向函数式计算的软件验证，在 Coq 基础上搭建的 VST 提供了面向指令式语言 C 的通用验证手段。而类 C 语言又是编写在对性能有要求时的系统核心模块的首选语言。尽管在 `isPrime` 中并没有展示 VST 对比如（堆）内存（通过 `SEP ( )`）和外部接口（`Espec`）的处理，但我们可以在官方示例²中找到诸如指针、IO、内存分配和并发处理的例子。

规格说明和证明过程的严格性，以及 VST 面向 C 的通用性带来了 VST，也可以说是形式化软件验证的最大问题，复杂度。我们很容易从示例中看出这点，这里做进一步的分析。首先，对于 VST，指令式语言对验证更不友善。相比之下，函数式语言作为声明式语言的一种，由于更接近对问题（而非解决方法）的描述，相对要更容易验证（和优化）^{3 4}。

类似的，我们也可以通过限制问题的其它方面，比如表达能力、所需的计算能力，来获得更高效，甚至是全自动的解决方法。同时，我们也总是可以尝试寻找严格度和复杂性之间的平衡点。就像 HMAC 示例中看到的，我们可以因为难度，或者（低）重要性而假定（大）部分模块是正确的。甚至在没有证明的情况下，仅通过形式化的规格说明也可以显著提升开发者之间沟通的精确度。然而，不同的问题范围，不同的严格程度，也意味验证方法的多样性。在示例中，我们已经看到了不同的逻辑，不同的理论、不同的语言和不同的工具。

¹相对的，像 C 这样的语言可以认为是指令式的。函数式和指令式等编程范式的详细讨论见??。

²<https://github.com/PrincetonUniversity/VST/tree/v2.8>，示例在 `progs` 和 `progs64` 目录下。

³指令式接近解法意味着更接近机器语言，意味着一般情况下会有更好的性能表现。

⁴??会给出 `isPrime` 的 Coq 函数式实现和证明。

在进一步讨论多样性之前，有另一些值得注意的方面。其中的一个问题是我们可以证明实现是符合规格说明的，但我们如何保证规格说明的正确性呢。这里没有形式化的办法，尽管我们可以比如像操作语义那样，用一个严格的语言去描述另一个的行为，但我们只是推迟了形式到（最终的）内容——我们通过软件所解决的日常问题——之间对应关系的验证¹。更传统的说，在比如自动化的单元测试里，我们如何保证测试用例的正确性，一个可信的解释是我们只有在待测代码和测试中犯了同样的错误，我们才能通过测试而不发现这个错误，这种出错的概率会小很多。我们可以用同样的说法去解释这里规格说明的问题²。

此外，严格的证明也是优化的基础，优化又是编译的核心。从声明式语言到指令式的目标语言的编译过程可以看成是从问题域到解法域的转换，也就是寻找解决方法的过程。其中一个好的优化能力决定了所得到的解决方法的可行性。同时，形式化的证明过程为证明的搜索提供了精确的定义，为自动化证明提供了可能，也就为自动优化，自动编程提供了可能³。

最后回到方法的多样性上，这里有两个问题。一是如何掌握这么多不同的逻辑、理论、语言和工具。举个例子，就像如果我们已经熟练掌握了一门编程语言，相信就能很快的掌握其它相似的语言，因为这些语言之间框架是一样的，要解决的问题也是类似的。同样的，比如对于不同的逻辑和理论，它们的基本结构也是一样的，可以互相参考的。另一个更加重要的问题是我们如何保证方法的正确性。这（从形式而不涉及其对应内容的角度上来说）是有形式化的解决方法的。就像示例所示，有各种不同的工具都是基于 Coq 的，而 Coq 采用的证明系统基于的也不是（VST 所采用的）霍尔逻辑，而是类型论。我们可以在一个系统上构建另一个系统，我们也可以通过一个系统证明另一个系统。而通过比如用同一个底层系统证明各种不同的上层系统，上层系统之间的兼容性和相似性也就不难理解了。

下一章将会探讨形式化方法的核心框架——形式逻辑。

1.3 本章小节

-
- 形式化方法是严格基于形式的方法，通过基于形式避免了直觉和主观理解可能带来的歧义。形式逻辑是形式化方法的核心框架，它包括语法、语义和演绎规则三部分；或广义的也可认为语法包括演绎规则，即同为其中纯粹形式的部分。
 - 数学模型的严格性来源于其形式化特点。形式逻辑和数学的交叉学科被称为数理逻辑，其主要分支包括研究形式推导（演绎）的证明论，研究形式和内容对应关系的模型论，研究数学对象的集合论和研究可计算性的计算理论。
 - 罗素悖论指出了形式化系统不一致的可能。ZFC 集合论和类型论是解决罗素悖论的两种方案。
 - 软件产品的生命周期包括软件开发周期和软件交付、运行、维护和回收等过程。软件开发周期一般包括需求分析、设计实现和测试维护等阶段。传统软件开发方式依赖直觉和经验，也面临由直觉和经验带来的问题。

¹比如 CompCert 的可靠性只保证到（其给出的）汇编行为的正确性，但这并不能保证（其给出的汇编行为）和比如硬件厂商提供的行为的一致性。

²实际上，测试用例也可以被看作是一种（不严格的）规格说明。

³更多关于自动编程的讨论见??。

- 软件需求分析的产出被称为软件规格说明，其是软件功能和使用方法的说明书。形式化的规格说明是指由具有严格语法规则的形式语言描述的规格说明，具有更高的精确性。形式化软件验证是指用形式化方法验证软件实现符合规格说明。
- Coq 是基于类型论的形式化开发和证明系统，它通过柯里-霍华德同构为函数式计算和证明提供了统一的框架。Coq 提供了强大灵活的语法扩展能力。VST 是在 Coq 基础上构建的基于霍尔逻辑的 C 语言编译和证明系统。
- 编程语言语义一般分为基于操作前后置条件的公理语义，基于另一种计算模型的操作语义和基于域理论的指称语义。
- 函数式编程作为声明式编程相比于指令式编程更接近于对问题的描述，因此也更容易验证和优化，但同时性能也相对较差。
- 形式化证明为证明的自动搜索提供了可能，进一步为自动优化和自动编程提供了可能。
- 形式化软件验证的复杂性带来了理论和工具的多样性。这种多样性背后一般有着统一的结构，一种逻辑理论有时可以用来描述其它理论，不同理论之间有时可以互相描述。

第二章 形式化基础

本章主要介绍形式化系统和方法的基础。2.1 节主要介绍用于讨论形式逻辑系统行为性质的一些背景知识，包括（由半自然语言描述的）作为背景的集合论，后续论证中将经常使用的重要方法，以及会实际影响系统构成的不同角度的对数学的认识方式。2.2 节从相对简单的命题逻辑入手，并以此为例，介绍形式逻辑中的主要概念，同时指出其中具体的关于可（有效）计算性的问题。2.3 节介绍计算理论可计算性相关的主要理论，包括计算设备/模型，如图灵机、 μ -递归函数的定义，其计算能力相等的证明，以及由不可计算问题指出其计算能力的理论极限。

2.1 背景知识

图 2.1 给出了接下去四章的路线图。在本章中，我们主要讨论形式化相关的基础部分，包括非形式化的元逻辑概念、形式化的命题逻辑和可计算性问题。在下一章中主要讨论一阶逻辑。在第四章中介绍公理集合论（即形式化的集合论，如 ZFC 集合论），在第五章中介绍元逻辑。在某种意义上可以认为图中的箭头表示了形式化讨论的基础，如命题逻辑的形式化讨论为一阶逻辑提供了基础，继而为公理集合论提供了基础。而通过公理集合论，我们将更容易理解元逻辑的形式化含义。

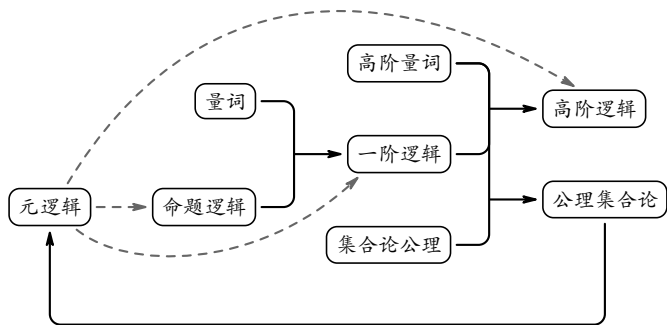


图 2.1: 路线图

如此，我们就看到了我们的第一个问题——路线图的主路径是一个环¹。具体的说，比如我们使用元逻辑定义和论证关于逻辑本身的性质。当我们没有完成元逻辑的形式化时，我们将没法比如精确的定义在对命题逻辑的性质论证中所采用的证明过程本身是什么，当我们完成证明时，我们的结论在何种意义上被认为是正确的。

当然，我们最终会完成这个闭环。但我们只能从非形式化的（半）自然语言开始，从图中虚线处入手，逐步完成整个形式化的过程。而在最开始，我们能依赖的只有直觉和经验。

¹注意当然这样的介绍方式是为了更容易直观的对各种概念进行理解。如在下面几个章节将看到的，这并不意味着整个过程中有任何循环定义或循环论证。

2.1.1 集合的基本概念

这里给出部分集合相关的基本概念, 我们需要这些概念来帮助我们描述后续给出的定义和论证。首先我们引入一种会经常用到的简写。对任意 x , 任意自然数 i, j , 写 $x_{i\dots j}$ 表示 $x_i, \dots, x_j$, 则 $y_{i\dots j}$ 表示 $y_i, \dots, y_j$, $z_{i\dots j}$ 表示 $z_i, \dots, z_j$ 等等:

- 集合: 一组不重复的无序物体。比如对于 2 3 5 7 四个数构成的集合我们写作 $\{2, 3, 5, 7\}$ 。并在不引起歧义的情况下使用省略号, 比如对数字 1 到 10, 我们写作 $\{1, 2, \dots, 10\}$, 或者自然数集合 $\{0, 1, 2, \dots\}$ 。对于不含任何物体的集合, 我们称其为空集, 写作 $\emptyset$ 。集合也是物体¹, 因此我们也可以构造集合的集合, 比如 $\{\emptyset, \{\emptyset\}, \{2, 3\}\}$ 。物体是不重复的意为如 $\{2, 3\} = \{2, 2, 3\}$, 无序为 $\{2, 3\} = \{3, 2\}$ 。

同时, 我们也会采用另一种集合表示法 $\{ _x, y, \dots _ : _x, y, \dots _ \}$ 意为同时满足 | 左右两侧条件的集合, 其最简单的形式, 如自然数集合 $\{x : x \text{ 是自然数} \}$ 。

- 属于: 物体 a 在集合 A 内, 我们称 a 属于 A , 或 a 是 A 的元素, 写作 $a \in A$, 反之 $a \notin A$ 。比如对于集合 $A = \{2, 3\}$, 我们有 $3 \in A$, $4 \notin A$ 。同时对任意物体 a , 我们有 $a \notin \emptyset$ 。
- 子集: 若集合 A 内的所有元素都属于 B , 我们称 A 是 B 的子集, 也称 B 包含 A 或 A 包含于 B , 写作 $A \subseteq B$ 。反之 $A \not\subseteq B$ 。我们有 $A \subseteq A$ 。对任意集合 A , 有 $\emptyset \subseteq A$ 。我们可以将集合 A 中满足一定条件的元素子集写作 $\{x \in A : x \text{ 满足} \dots\}$ 。同时, 若 $A \subseteq B$ 且 $A \neq B$, 我们称 A 是 B 的真子集, 也称 B 真包含 A 或 A 真包含于 B , 写作 $A \subset B$ 。反之 $A \not\subset B$ 。
- 幂集: 我们称集合 A 所有子集的集合为 A 的幂集 ($\{x : x \subseteq A\}$), 写作 $\wp(A)$ 。如 $\wp(\{2, 3\}) = \{\emptyset, \{2\}, \{3\}, \{2, 3\}\}$ 。对任意集合 A , 我们有 $\emptyset \in \wp(A)$, $A \in \wp(A)$ 。
- 并集: 对集合 A, B , 我们称 $\{x : x \in A \text{ 或 } x \in B\}$ 为 A 和 B 的并集, 写作 $A \cup B$ 。如 $\{2, 3\} \cup \{2, 4\} = \{2, 3, 4\}$ 。对集合 A_1 到 A_n 和 $A = \{A_1, \dots, A_n\}$ 我们定义 $\bigcup A = A_1 \cup A_2 \cup \dots \cup A_n$, 我们有 $\bigcup \wp(A) = A$ 。定义 $\bigcup _x, y, \dots _ = \bigcup \{ _x, y, \dots _ : _x, y, \dots _ \}$ 。我们同时定义 $A; b = A \cup \{b\}$ 。
- 交集: 对集合 A, B , 我们称 $\{x \in A : x \in B\}$ 为 A 和 B 的交集, 写作 $A \cap B$ 。如 $\{2, 3\} \cap \{2, 4\} = \{2\}$, $\{2\} \cap \{3\} = \emptyset$ 。我们以和并集类似的方式定义 $\cap$ 。

此外, 我们也已类似的方式扩展其它二元操作, 如 $+\{x, y, z\} = x + y + z$, $\times\{x, y, z\} = x \times y \times z$ 。同时, 对于这两个例子, 更常见的, 我们将 $+A$ 写作 ΣA , $\times A$ 写作 ΠA 。

- 差集: 对集合 A, B , 我们称 $\{x \in A : x \notin B\}$ 为 A 和 B 的差集, 写作 $A - B$ 。写 $A \setminus b$ 表示 $A - \{b\}$ 。我们有正整数集合等于 $\mathbb{N} \setminus 0$ 。同时, 若 $A \subseteq B$, 也称 $B - A$ 为 A 在全集 B 下的补集。在明确全集时, 可写 $\bar{A}$ 表示补集。
- 有序对: 对物体 x, y , 我们将其组成的有序对写作 $\langle x, y \rangle$ 。我们有 $\langle x, y \rangle \neq \langle y, x \rangle$ 。定义 $\langle x, y, z \rangle = \langle \langle x, y \rangle, z \rangle$, 称为三元组。类似的, 对 $n > 1$, $\langle x_1 \dots x_{n-1}, y \rangle = \langle \langle x_1 \dots x_{n-1} \rangle, y \rangle$, 称为 n 元组。我们可以定义 $\langle x, y \rangle = \{\{x\}, \{x, y\}\}$ 以满足有序对的要求²。此外, 我们定义一元组 $\langle x \rangle = x$ 。我们也称 n 元组 ($n \geq 1$) 为多元组。
- 关系: 我们将由有序对组成的集合称为关系。对关系 R , 我们称 $\{x : \text{存在 } y, \langle x, y \rangle \in R\}$ 为 R 的定义域, 写作 $\text{dom } R$, 类似的 $\{y : \text{存在 } x, \langle x, y \rangle \in R\}$ 为值域, 写作 $\text{ran } R$ 。称 $\text{dom } R \cup \text{ran } R$ 为 R 的域, 写作 $\text{fld } R$ 。同时对关系 R , 我们称 R 是 $\text{dom } R$ 到 $\text{ran } R$ 上的关系, 也称 R 是在 $\text{fld } R$ 上定义的关系。比如在 $A = \{0, 1, 2\}$ 上定义的小于关系为 $<_A = \{\langle 0, 1 \rangle, \langle 0, 2 \rangle, \langle 1, 2 \rangle\}$, 其中 $<_A = \{\langle x, y \rangle : x \in A \text{ 且 } y \in A \text{ 且 } x < y\}$, 有 $\text{dom } <_A = \{0, 1\}$, $\text{ran } <_A = \{1, 2\}$ 。

¹集合论中(几乎)一切概念都可以用集合表示, 更多相应的形式化定义见第四章。

²见章节 4.1.5。

对于关系 R ，我们称 $\{ \langle y, x \rangle : \langle x, y \rangle \in R \}$ 为 R 的逆关系，写作 R^{-1} 。我们有时也将 $\langle x, y \rangle \in R$ 写作 $x R y$ ，比如将 $\langle 0, 1 \rangle \in <$ 写作 $0 < 1$ 。此外，我们定义由一元组组成的关系为一元关系，可以看到若构成一元关系 R 的一元组 $\langle x \rangle$ 有 $x \in A$ ，则 $R \subseteq A$ 。对 $n > 1$ ，我们将由 n 元组组成的关系称为 n 元关系。

- 等价关系：称集合 A 上的（二元）关系 R 为等价关系若其满足：

- R 是自反的，即任意 $x \in A$ 有 $x R x$ 。
- R 是对称的，即任意 $x R y$ 有 $y R x$ 。
- R 是传递的，即任意 $x R y$ 且 $y R z$ 有 $x R z$ 。

例如当视有理数的不同写法（如 $\frac{1}{2}$ 和 $\frac{2}{4}$ ）为不同元素时，这些写法集合 $\mathbb{R}$ 上构成的值相等关系 $=_{\mathbb{R}}$ （如 $\frac{1}{2} =_{\mathbb{R}} \frac{2}{4}$ ）即为等价关系。

对任意 A 上的等价关系 R ，有 $A = \text{fld } R$ 。则对任意 $x \in A$ ，称 $\{ y : x R y \}$ 为 x 的等价类，写作 $[x]_R$ ¹。有上述例子中如 $\frac{2}{4} \in [\frac{1}{2}]_{=_{\mathbb{R}}}$ 。称集合 $\{ [x]_R : x \in A \}$ 为 A 由 R 等到的划分。则可视 $\mathbb{R}$ 由 $=_{\mathbb{R}}$ 为有理数集合 $\mathbb{R}$ 。

- （非严格）偏序关系：称集合 A 上的（二元）关系 R 为非严格偏序关系、或仅偏序关系若：

- R 是自反的。
- R 是反对称的，即任意 $x R y$ 且 $y R x$ 有 $x = y$ （ x, y 为同一元素）。
- R 是传递的。

可以看到 $\mathbb{N}$ 上的小于等于关系即为非严格偏序关系。

- 严格偏序关系：称集合 A 上的（二元）关系 R 为严格偏序关系若：

- R 是反自反的，即任意 x 没有 $x R x$ 。
- R 是传递的。

可以看到 $\mathbb{N}$ 上的小于关系即为严格偏序关系。同时注意由 R 的反自反和传递性有：

- R 是非对称的，即任意 $x R y$ 没有 $y R x$ 。

也有反自反且反对称性意味着非对称性。以及非对称性意味着反自反性。

- （非严格）全序关系：直观的说，全序关系要求在偏序关系的基础上集合内任意两元素是有序的。具体的，称集合 A 上的（二元）关系 R 为非严格全序关系、或仅全序关系若：

- R 是 A 上的非严格偏序关系。
- R 是强连通的，即任意 $x \in A$ 、 $y \in A$ 有 $x R y$ 或 $y R x$ 。

可以看到 $\mathbb{N}$ 上的小于等于关系亦为非严格全序关系。

- 严格全序关系：称集合 A 上的（二元）关系 R 为严格全序关系若：

- R 是 A 上的严格偏序关系。
- R 是连通的，即任意 $x \in A$ 、 $y \in A$ 有 $x R y$ 或 $y R x$ 或 $x = y$ 。

可以看到 $\mathbb{N}$ 上的小于关系亦为严格全序关系。同时如树中的父子关系为严格偏序关系但不一定是全序的，因可见对任意子树 a, b ， a 不为 b 的子树并不意味着 $a = b$ 或 b 为 a 的子树。直观的说，这是偏序和全序的主要区别。而全序的树即退化为（单向）链表。最后，也称全序关系为线性关系。

- 笛卡尔积：对集合 A, B ，我们称 $C = \{ \langle x, y \rangle : x \in A \text{ 且 } y \in B \}$ 为 A 和 B 的笛卡尔积，写作 $A \times B$ 。我们有 C 是 A 到 B 上的（二元）关系， $\text{dom } C = A$ ， $\text{ran } C = B$ 。此外对 $n > 1$ ，集合 A_1 到 A_n ，我们有 $A_1 \times A_2 \times \dots \times A_n = \{ \langle x_1, \dots, x_n \rangle : \text{对 } 1 \leq i \leq n, x_i \in A_i \}$ 。同时我们定义 $A^1 = A$ 。因此，我们有对

¹ 因 $A = \text{fld } R$ ，该写法不会产生歧义。

$n > 1$, $A^n = A^{n-1} \times A$ 。比如对 $A = \{0, 1\}$, 若我们将 $\langle a, b \rangle$ 写作 ab , 则 $A^3 = \{000, 001, 010, \dots, 111\}$, 即所有由字符 0 1 组成的长度为 3 的字符序列, 或者二进制的 0 到 7。这样, 我们有如 $\langle x \rangle \in \{x\}^1$, $\langle x, x \rangle \in \{x\}^2$, $\dots$ 。最后, 我们定义 $A^+ = \bigcup_{i \in \mathbb{N} \setminus 0} A^i$ 。即在上述例子中, A^+ 代表任意长度不为零的 0 1 字符序列集。

- 函数: 我们称一个关系 F 为函数若对其定义域中的每一个 x , 其值域中只有唯一的 y 构成 $\langle x, y \rangle \in F$, 或者说, 对任意 $x y z$, 若 $\langle x, y \rangle \in F$ 且 $\langle x, z \rangle \in F$ 则 $y = z$ 。若 $\text{dom } F \subseteq A$, $\text{ran } F \subseteq B$, 我们称 F 为 A 到 B 上的部分函数, 对 $x \in A - \text{dom } F$, 我们称 $F(x)$ 是未定义的。特别的, 若 $\text{dom } F = A$, 我们称 F 为 A 到 B 上的全函数, 或直接称 A 到 B 上的函数。注意部分函数包含全函数, 反之不成立, 即全函数集合是部分函数集合的真子集。此时 (即 $\text{dom } F = A$ 时), 我们又称 F 将 A 映射到 B , 写作 $F: A \mapsto B$ 。若还有 $\text{ran } F = B$, 我们称 F 将 A 满射到 B 。

同时, 对一个函数 F , 若对其值域中的每一个 y , 其定义域中只有唯一的 x 构成 $\langle x, y \rangle \in F$, 我们称 F 为单射函数, 或者一对一函数。对于单射函数 F , 我们有 F^{-1} 也是单射函数, 称为 F 的逆函数。若单射函数 F 由 A 满射到 B , 我们称 F 为 A 到 B 上的双射函数。

我们写 $F(x)$ 表示在 F 中 x 对应的 (唯一) y , 即 $y = F(x)$, 并称其中 x 为自变量, y 为因变量。此外, 对 $n \geq 1$, 我们将 $F(\langle x_1 \dots x_n \rangle)$ 写作 $F(x_1 \dots x_n)$, 并称其为 n 元函数, 将所有 n 元函数统称为多元函数。特别的若对 $0 < i \leq n$ 都有 $x_i \in A$, 称 F 为 A 上的 n 元函数。注意如若 F 为 A 上的二元函数, 则 F 也是 $A \times A$ 上的 (一元) 函数。

若有 $F: B \mapsto C$, $G: A \mapsto B$, 我们可以组成新的复合函数 $F \circ G: A \mapsto C$, 其中 $F \circ G = \{ \langle x, y \rangle : y = F(G(x)) \}$ 。

最后, 对函数 $F = \{ \langle x, y \rangle : y = _ _ x _ _ \}$, 我们也写作 $F(x) = _ _ x _ _$ 。对 $n > 1$, $F = \{ \langle \langle x_1 \dots x_n \rangle, y \rangle : y = _ _ x_1 \dots x_n _ _ \}$, 我们也写作 $F(x_1 \dots x_n) = _ _ x_1 \dots x_n _ _$ 。如我们可以将加法函数写作 $F(x, y) = x + y$ 。

2.1.2 集合的大小

- 基数: 我们将反映一个集合 A 的大小的量度称为其基数, 写作 $|A|$ 。对于元素个数有限的集合, 它的基数就是它的元素个数。两个集合间元素个数的大小关系就表达了集合的大小关系。比如对于 $A = \{2, 3, 5\}$ 有 $|A| = 3$ 。下面我们考虑元素个数无限的集合, 比如自然数集合。

初看之下无限集合是无法比较大小的, 这里的问题在于大小的定义上, 因为我们没有足够自然的对无限集合的大小定义方法。在给出大小的定义之前, 我们先来简单讨论一下为什么要比较集合的大小。就如前面所说, 集合可以用来表示各个具体的物体, 也可以用来表示它们之间的关系, 我们研究集合是为了通过集合的形式, 研究其对应物体之间的关系。那么, 集合的大小反映了其可研究的对应物体关系的复杂程度。也就是说, 我们可以通过研究较大的集合来得到较小的集合的属性, 但反过来是不行的。

比如说对于字符集 $A = \{0, 1, 2\}$, 对其可构成的长度为 2 的字符序列 $A^2 = \{00, 01, 02, 10, \dots\}$, 我们有逆序函数 $F: A^2 \mapsto A^2 = \{ \langle 00, 00 \rangle, \langle 01, 10 \rangle, \langle 02, 20 \rangle, \langle 10, 01 \rangle, \dots \}$ 。可以看到, 我们有比如 $F(01) = 10$ 。那么对 $B = \{0, 1\}$ 我们仍可以用关于 A^2 的 F 查询 B^2 元素的逆序结果。那如果对字符集 $C = \{a, b\}$, 我们只需要通过一个单射函数 $M: A \mapsto C$ 如 $\{ \langle a, 0 \rangle, \langle b, 1 \rangle \}$ 即可实现查询。反之则是不行的, 不管通过怎样的单射函数 M , A 中总会有一个在 B 或 C 中不存在对应的字符。

因此, 大小的定义的关键在于对应关系。更具体的说, 如果两个集合的元素之间有一对一的对应关系, 则我们称这两个集合的基数相等。或者更严格的, 对集合 $A B$, 若存在单射函数 $M: A \mapsto B$, 则 $|A| \leq |B|$,

若 $|A| \leq |B|$ 且 $|B| \leq |A|$ ，则 $|A| = |B|$ ^{注1}。

首先这个定义符合有限集合下对大小的直观理解。然后我们来比较几个无限集合的大小。比如自然数集合 $\mathbb{N}$ 和正整数集合 P 的基数是相等的，我们可以取 $M: \mathbb{N} \mapsto P$ 为 $M(n) = n + 1$ 。整数集合 $\mathbb{Z}$ 和偶数集合 E 基数相等，取 $M: \mathbb{Z} \mapsto E$ 为 $M(z) = 2 \times z$ 。

定义 $\aleph_0 = |\mathbb{N}|$ 。可以证明 $\mathbb{N}$ 的任意无限子集的基数等于 $\aleph_0$ ，因此 $\aleph_0$ 是最小的无限基数。事实上，整数、分数集合的基数都等于 $\aleph_0$ 。一般的说，如果我们能找到一个有效的方法（在任意长的时间内）逐一枚举（列举）集合 A 内的元素，我们就能构造一个具体的单射函数 $M: A \mapsto \mathbb{N}$ ，意味着 $|A| \leq \aleph_0$ 。若 A 是无限的，则 $|A| = \aleph_0$ 。注意反过来 $|A| = \aleph_0$ 并不意味着我们一定有有效方法枚举 A ，单射函数的存在不代表其可被确切的描述，我们会在章节 2.3.8 看到具体的例子 ²。回到整数和分数集合，我们可以通过比如 $0, 1, -1, 2, -2, \dots$ 的方式枚举整数集合。通过递增分子分母之和的方式枚举分数，比如和等于 2 时，有 $\frac{1}{1}$ ，3 时， $\frac{1}{2}, \frac{2}{1}$ ，4 时， $\frac{1}{3}, \frac{2}{2}, \frac{3}{1}$ ， $\dots$ 。

我们称基数小于等于 $\aleph_0$ 的集合为可数集合。称有有效枚举方式的集合为可枚举集合。

对 $n > 0$ ，定义 $\aleph_n$ 为比 $\aleph_{n-1}$ 大的最小基数。我们首先证明 $\aleph_1$ 的存在。我们证明 $|\wp(\mathbb{N})| > \aleph_0$ 。对于 $\mathbb{N}$ 的某个子集 A ，我们可以通过询问一系列（无限的）问题确定 A 的组成元素：0 是否属于 A ，1 是否属于 A ，2，3，4， $\dots$ ，依次对于肯定的答案我们记为字符 1，否则 0。这样每个子集即对应一串无限长的 01 字符序列，比如 $\emptyset$ 对应全 0 串， $\mathbb{N}$ 对应全 1 串。因此令 $S = \{s : s \text{ 为无限长的 01 字符序列}\}$ ，有 $|\wp(\mathbb{N})| = |S|$ 。

现在我们有了一个更容易理解的问题。你可以先尝试逐一枚举一下无限长的 01 字符序列 ³，或遍历一棵无限大的在每个节点都有两个子节点的二叉树，以对其数量/大小有一个直观的理解。然后，无法枚举并不代表 $|S| > \aleph_0$ ^{注4}，我们需要采用另一种（后面还会经常用到的）证明方法——对角论证法。

2.1.3 对角论证法 ⁵

一般而言，对角论证法会被用来处理无限场景中的问题。因为理论上来说，我们总是可以枚举有限场景中的所有情况，从而得到场景中的所有细节，同时我们也无法通过（有限次数的）集合操作从有限集合得到无限集合，比如有限集合的幂集也是有限的。但对角论证法仍然可以工作于有限场景中，这里我们使用该方法证明比如对 $|A| = 3$ ，有 $|\wp(A)| > 3$ 。首先结论显然是正确的，给定 A ，我们可以枚举出 $\wp(A)$ 的全部 8 个元素，或者说我们可以枚举出 $\wp(A)$ 对应的长度为 3 的 01 字符序列集合的全部 8 个元素。然后我们来看对角论证法的应用。

简单的说，对角论证法在这里的应用是指对长度 n （不管 n 等于 3 还是 $\aleph_0$ ），无论通过什么函数将基数为 n 的集合映射到长度为 n 的 01 字符序列，总是可以通过该函数构造出一个长度为 n 但不在值域中的序列，既不存在基数为 n 的集合和长度为 n 的 01 字符序列之间的双射函数。对于 $n = 3$ 的情况，有：

示例 2.1.1 令 S 等于长度为 3 的 01 字符序列集合，对任意函数 $f: \{0, 1, 2\} \mapsto S$ ，总是存在 $s \in S$ 且 $s \notin \text{ran } f$ 。

¹由第四章中给出的康托尔-伯恩斯坦-施罗德定理 4.3.44 可知这样定义的 $|A| = |B|$ 等价于存在由 A 到 B 的双射函数，即这里所谓的一对一对应关系。

²见推论 2.3.14.1。

³试想一个可以逐一输出无限长字符串的程序，第一个问题是如何（用有限的字符）表示这种字符串，事实上，这个问题的无解也直接导致了枚举的失败。

⁴即推论 2.3.14.1。

⁵1891，康托尔。

证明：令：

$$F = \begin{bmatrix} \mathbf{c}_{11} & c_{12} & c_{13} \\ c_{21} & \mathbf{c}_{22} & c_{23} \\ c_{31} & c_{32} & \mathbf{c}_{33} \end{bmatrix} \quad f' = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 1 \end{bmatrix}$$

其中 F 代表函数 f 的一般形式。每一个 c_{ij} 可以被赋为字符 0 或 1，每一行 s_i 代表一个对应的字符序列，即 $f(i) = s_i$ 。这样， c_{ij} 的赋值组合决定了 f 。比如对 $f' : \{0, 1, 2\} \mapsto S$ ， $f'(0) = 100$ ， $f'(1) = 001$ ， $f'(2) = 101$ 。接着，对每一个 c_{ii} （即为对角线上的字符）做取非操作（ $\sim$ ： $\sim 0 = 1$ ， $\sim 1 = 0$ ）得到的 $s = \sim c_{11} \sim c_{22} \sim c_{33}$ 即为符合 $s \in S$ 且 $s \notin \text{ran } f$ 的序列，因为对每个 i ， $s \neq s_i$ （因为 S 中的第 i 个字符不等于 S_i 中的第 i 个字符）。比如对于 f' ，010 即为不在 $\text{ran } f'$ 中的序列。□

以上证明可以扩展到 n 等于 $\aleph_0$ 的情况（相应的需要扩展字符序列长度的定义），比如 $n = \aleph_0$ 时，对应的：

$$F = \begin{bmatrix} \mathbf{c}_{11} & c_{12} & \cdots & c_{1u} & \cdots \\ c_{21} & \mathbf{c}_{22} & \cdots & c_{2u} & \cdots \\ \vdots & \vdots & \ddots & \vdots & \\ c_{u1} & c_{u2} & \cdots & \mathbf{c}_{uu} & \\ \vdots & \vdots & & & \ddots \end{bmatrix}$$

对应的 $s = \sim c_{11} \sim c_{22} \dots \sim c_{uu} \dots$ ，我们有对任意正整数 i ， $s \neq s_i$ 。

因此，我们有 $|\wp(\mathbb{N})| > \aleph_0$ 。相应的扩展幂运算，对任意 A ， $|\wp(A)| = 2^{|A|}$ ，我们有 $2^{\aleph_0} > \aleph_0$ 。类似的，对任意 n ， $2^{\aleph_n} > \aleph_n$ 。

自然的，我们来到了著名的连续统¹ 假设 $2^{\aleph_0} = \aleph_1$ 。事实上，在 ZFC 集合论中，连续统假设被证明既不能被证明，也不能被证伪。后面我们还会看到类似的情况。

同时注意这里指出了是在 ZFC 集合论背景下有连续统假设不可证明不可证伪。这是因为我们有各种不同的形式系统，我们也可以提出新的形式系统²，有时通过不同的形式系统可以得到不同的结论。

具体的说，我们可以认为一个形式系统是由其采用的形式逻辑和其假定正确的前提决定的。这里假定的前提又被称为（非逻辑）公理。因此不同的形式系统可以由不同的形式逻辑和不同的公理（集合）组合而成。我们来看一个具体的例子。

2.1.4 数学认识论

前文中我们很自然的提到了无限集合，比如自然数集 $\mathbb{N}$ ，并开始讨论它们的性质，但我们并没有思考无限集合是否真的存在。事实上，一些人拒绝接受无限的存在——比如他们接受任意大的自然数存在，但不接受 $\mathbb{N}$ 的存在——并在此基础上研究数学。这样的学派被称为有限主义数学³。

有限主义学派认为在现实生活中我们不可能通过执行无限次的操作来构造涉及无限的数学对象，因此无限的存在是没有意义的。而其中还有一些人拒绝接受大到一定规模的数学对象（比如非常大的自然

¹在集合论的语境下，实数集也被称为连续统。它的基数也等于 $2^{\aleph_0}$ 。

²第一章中的 VST 和 FCF 就可以认为是在 Coq 基础上提出的新的形式系统。

³Wikipedia contributors. Finitism — Wikipedia, The Free Encyclopedia. <https://en.wikipedia.org/w/index.php?title=Finitism&oldid=1009787110>. 2021。

数)的存在,因为他们认为现实中没有足够的资源执行那么多次的操作,这种学派被称为极端有限主义。

在一些分类法中,这种基于强调通过构造创建数学对象的学派被统称为构造主义数学¹。其中除了有限主义,还包括我们会主要在??讨论的直觉主义数学^{2 3}。这里面技术上最大的区别是直觉主义数学像主流的非构造主义数学(比如集合论)一样,作为公理接受无限的存在。认识上可以认为有限主义数学更倾向于用现实世界中的物体作为数学对象的参照,而直觉主义倾向于用人类思维中的概念对应数学对象,并因此接受无限概念的存在。

相对的,如柏拉图主义⁴这样的非构造主义数学认为数学对象客观且永恒存在于即非现实世界也非思维意识中的第三领域中,即其接受可构造的对象、不可构造但可证明存在的对象(如不可数集合),也接受不可证明存在对象存在的可能。

尽管不同学派的基础在于对数学认识方式的不同,我们仍然主要从技术层面出发考察不同学派解决实际问题的能力差异,比如说,有限主义和直觉主义同作为构造主义数学,都拒绝接受不通过构造得到的数学对象。比如他们都拒绝接受像反证法这样的非构造性证明,让我们来看一个更具体的例子,证明 $\sqrt{2}$ 是无理数⁵,即不存在整数 a, b 满足 $\sqrt{2} = \frac{a}{b}$ 。

示例 2.1.2 $\sqrt{2}$ 是无理数。

非构造性证明: 我们采用反证法。

1. 若存在整数 a, b 满足 $\sqrt{2} = \frac{a}{b}$, 则约分后存在 p, q 使 $\sqrt{2} = \frac{p}{q}$ 且 p, q 的最大公因数为 $\gcd(p, q) = 1$ 。
2. 有 $p^2 = 2q^2$ 。因为奇数的平方是奇数, 所以 p 是偶数, 即存在 k 使 $p = 2k$ 。
3. 有 $4k^2 = 2q^2$ 。同上 q 也是偶数, 则 p, q 有公因数2, 与1矛盾。因此不存在整数 a, b 满足 $\sqrt{2} = \frac{a}{b}$ 。□

构造性证明⁶: 简单的说, 我们要证明对任意 $\frac{a}{b}$, 我们都能构造出一个其与 $\sqrt{2}$ 不为0的差。

1. 对任意非零整数 a, b , 有 $a^2 \neq 2b^2$, 因为 a^2 和 $2b^2$ 质因子分解后前者2的个数为偶数, 后者为奇数。因此有 $|2b^2 - a^2| \geq 1$ 。
2. 当 $\frac{a}{b} \leq 3 - \sqrt{2}$, 有 $|\sqrt{2} - \frac{a}{b}| = \frac{|2b^2 - a^2|}{b^2(\sqrt{2} + \frac{a}{b})} \geq \frac{1}{b^2(\sqrt{2} + \frac{a}{b})} \geq \frac{1}{3b^2} > 0$ 。
3. 否则有 $|\sqrt{2} - \frac{a}{b}| > 3 - 2\sqrt{2} > 0$ 。□

本质上来说, 拒绝反证法是因为拒绝接受作为公理(或演绎规则)的排中律, 也就是拒绝接受对任意命题(陈述), 要么其本身是真的, 要么其反面是真的。需要注意的是这里不接受的是将排中律作为公理, 任何由其它公理推导出的排中结论都是可以也应该接受的, 比如上面构造性证明中用到的要么 $x \leq y$ 要么 $x > y$ 。

那么为什么像集合论这样的系统要包含无限的概念(以及它带来的麻烦)。首先我们需要明确的是这里形式系统的目标是为了提供给数学一个坚实的基础(ZFC集合论也是当前最流行的数学基础之一), 而

¹Wikipedia contributors. Constructivism (philosophy of mathematics) — Wikipedia, The Free Encyclopedia. [https://en.wikipedia.org/w/index.php?title=Constructivism_\(philosophy_of_mathematics\)&oldid=1039716465](https://en.wikipedia.org/w/index.php?title=Constructivism_(philosophy_of_mathematics)&oldid=1039716465). 2021。

²Wikipedia contributors. Intuitionism — Wikipedia, The Free Encyclopedia. <https://en.wikipedia.org/w/index.php?title=Intuitionism&oldid=1048235157>. 2021。

³事实上, Coq 基于的类型论是直觉主义类型论, 见??。

⁴Wikipedia contributors. Platonism — Wikipedia, The Free Encyclopedia. <https://en.wikipedia.org/w/index.php?title=Platonism&oldid=1047743193>. 2021。

⁵在比如有限主义数学中, 无理数可以通过它的计算过程进行定义。比如通过一个接受精度要求作为输入, 产生一个满足精度要求的结果作为输出的算法定义。

⁶证明自 Karin Usadi Katz and Mikhail G. Katz. Meaning in Classical Mathematics: Is it at Odds with Intuitionism? Section 2.3, footnote 15. 2011. arXiv: [1110.5456](https://arxiv.org/abs/1110.5456) [math.LO]。

无论在怎样的认识下，数学都被认为是研究其它学科（也包括元逻辑）的基本工具。

同时，也应注意到（部分程度上）为避免被认为是由无限带来的麻烦和悖论，古典数学一般避免涉及，甚至否定无限的存在。此外，物理学在形式系统发展的同时，也已经开始发现越来越多的证据表明现实世界中不存在无限小和无限大的实体¹。至此在有了明确的讨论目标和背景后，让我们来看看希尔伯特是怎么对无限的概念进行声辩的²：

“尽管可以认为在大小两个方向上，现实都是有限的。但无限仍可以是我们思考中合理且不可缺少的部分。”

“基于理想模型的研究方法为无限提供了新的不同定位。”

“是我们对推理的随意应用『而不是无限』导致了推理的失效。”

“有限主义式的命题给逻辑关系的表达带来了很大的局限性，这种局限性在其嵌套涉及‘全部’或者‘存在’的子命题时尤其明显。”

那么对于特定领域的研究（比如元逻辑），我们需要引入无限吗？事实上，即使我们不承认无限的存在，只承认存在任意的自然数，并接受最基本的运算关系，我们就足以构建出不满足某些看似合理要求的系统。比如哥德尔不完备定理就指出任何假设一致的形式系统，只要包含最基本的算术公理，就存在语句不可证明也不可证伪，这样的系统也无法证明自己的一致性。这点我们将在章节 3.2.5 和 3.2.7 详细讨论。因为这些仍不可避免的限制，对于如元逻辑来说，我们也很可能更倾向于接受我们的研究对象、也就是逻辑系统可以是任意复杂的。

同时，尽管不同的形式系统有某些相同的理论极限，但这并不妨碍它们在不同领域仍有实用性上的区别。我们在第一章中已经看到了一些例子，我们还将看到更多的例子。并且，我们也将看到在章节 5.4.3 看到在不带来一致性问题的情况下，允许新数学对象的存在的确会在一些问题的分析上为我们带来便利，或者说至少提供一个不一样的视角。

最后再来看一下极端有限主义。极端有限主义有一些很明显的问题，比如如果不承认过大的自然数的存在，那这个界限应该划分在哪。然而它也与计算理论，尤其是其中涉及计算复杂度的部分有不小的相似性，比如通过只考虑可接受的复杂度来对问题规模进行限制。不过基于这种认识论发展起来的形式系统的实际应用前景现在还未可知。

我们称直觉主义数学所采用的（不含公理排中律的）逻辑为直觉主义逻辑，相对的，称更常见的（含公理排中律的）逻辑为经典逻辑。

接着，有了这些背景，让我们先来讨论一个相对简单的形式逻辑——命题逻辑。

2.2 命题逻辑

命题逻辑有时也被称为语句逻辑，因为它可以用来处理语句间的逻辑关系。对于被处理的句子，更精确的，我们称其中的形式部分（字符序列）为语句，其所对应的内容（含义）为命题。换句话说，我们称语句是形式上的概念，命题是内容上的概念，后面我们将在各种场景区分形式和内容这两个类别，尽管在命题逻辑中一个概念在两个类别中对应的属性往往是一致的。

在命题逻辑的处理过程中，简单的说，我们通过有限的几个连词在简单语句的基础上构造复杂的表达式，并假定部分表达式是真的，并通过规则尝试确定其它表达式的真实性。比如我们有语句 A 、 B 和 C ，

¹ 比如普朗克提出的量子化能量否定了能量无限可分的可能，而爱因斯坦的相对论又表明了有限宇宙的可能。

² 选译自 David Hilbert. “On the infinite”. In: From Frege to Gödel. Ed. by Jean Van Heijenoort. Cambridge: Harvard University Press, 1967, pp. 367–392. [⋯] 为译注部分。

连词 且，我们假定 A 且 B 为真，则我们有 A 为真以及 B 为真，当然我们无法确定 C 是否是真的。其中 A 、 B 、 C 可以对应任何我们认为可以让 A 且 B 有意义的语句¹。

同时，命题逻辑也只能在语句的粒度上处理这些关系，这使它比较简单。但撇开技术上简单的特点，它仍包含了形式逻辑里面几乎所有的重要概念。如第一章所述，形式逻辑可以分为语法，语义和演绎规则三个部分，其中语法和语义（以及相应的词汇）构成了该系统的语言，且形式逻辑都有严格的语法规则，因此也是一种形式语言。这里，如第一章所述，语法是形式上的概念，语义是内容上的概念。

现在，我们先来看命题逻辑的语法。

2.2.1 语法和词汇

首先介绍一下词汇的通用概念。形式上的词汇被称为符号，可分为逻辑符号和非逻辑符号。尽管在不同的介绍中这些概念会有细微的差别，这里我们可以简单的认为逻辑符号是由形式逻辑本身提供的符号，非逻辑符号是形式逻辑的用户提供的符号。

命题逻辑中，逻辑符号包括逻辑联结词，可以是：否定² $\neg$ ，合取 $\wedge$ ，析取 $\vee$ ，双向条件 $\leftrightarrow$ ，条件 $\rightarrow$ 。此外还包括括号 $()$ 。

命题逻辑中，非逻辑符号又称为语句符号，可以包括任何不同于逻辑符号的符号，比如 ABC 等。可以认为语句符号是由使用命题逻辑的用户提供的。

可以认为，不同的符号集合（词汇表）构成了不同的语言，比如语句符号集 $\{A\}$ 和 $\{A, B\}$ 构成了不同的命题逻辑语言，同时我们也可以使用不同的（但不影响处理能力³的）逻辑联结词集合。而命题逻辑是使用这类满足特定条件的语言和演绎规则的形式逻辑的统称。

由符号组成的符号序列被称为表达式。其中（对命题逻辑来说）符合语法规则的被称为形式合法的公式，简称合式。

命题逻辑中，以上述的逻辑联结词集合为例，我们可以通过以下递归的语法规则构成合式：

1. 单个语句符号构成合式。
2. 若 α 为合式，则 $\neg \alpha$ 为合式。
3. 若 $\alpha \beta$ 为合式， $\square \in \{\wedge, \vee, \leftrightarrow, \rightarrow\}$ ，则 $(\alpha \square \beta)$ 为合式。

比如 $\neg(A \rightarrow B)$ ， $(\neg A \rightarrow B)$ 就是合式，可以看到这种规则下的合式可能会包含大量（可认为是不必要的）括号，尽管这影响的只是人类的阅读体验，我们仍将在稍后对括号使用的讨论中一并处理一下这个问题。

同时在命题逻辑中，合式也称语句。可以看出，为能构成语句，用户至少需要提供一个语句符号，因此下文我们默认语句符号集不为空。而对于语句符号集大小的上限，一般认为命题逻辑允许基数小于等于 $\aleph_0$ 的语句符号集，即允许可数的语句符号集。这里有三点需要注意。

一是允许基数等于 $\aleph_0$ ，也就是可数的无限，比如 $\{A_0, A_1, \dots\}$ 。尽管基数等于 $\aleph_0$ 的符号集（及语句集）在通过命题逻辑解决实际问题中并不常见，但对其我们需要采用不同的处理方式并会得到不同的性

¹ 章节 1.1.1 中的例子即是命题逻辑式的。

² 符号名，区别与语义上的概念。从严格的形式意义上说，我们可以忽略名称对应的含义。

³ 我们将在章节 2.2.7 定义这个概念。

质结果，我们会在后面的讨论中看到这一点，而这种讨论也为我们处理更多涉及无限且更具实际意义的问题提供了基础。

二是，尽管一般而言，可数集合不等于可枚举集合¹，但因为形式逻辑一般不要求所采用的符号集元素有特定性质，所以任意基数相等的符号集所构成的形式逻辑对问题的处理能力也是相同的，所以允许不可枚举但可数的符号集并不增加命题逻辑的处理能力。

最后，基数大于 $\aleph_0$ 的符号集在实际应用中意义不大，比如其中的大部分符号显然是不可显示和打印的。因此在大多数时候，比如在后面的一阶逻辑中，我们也主要讨论（可能无限的）可枚举符号集。

同时，任意集合 S 是可枚举的意味着存在可有效计算的双射函数 f 由 $\mathbb{N}$ 到 S 。这里将讨论范围限制在语句符号集内。即对任意语句符号集 $\mathcal{A}$ ，写 $A_0 = f(0)$ ， $A_1 = f(1)$ ， $\dots$ ，则 $\mathcal{A} = \{A_0, A_1, \dots\}$ 。注意，这样 $\mathcal{A}$ 中的每一个元素均可由有限符号集（即 $\{A, 0, \dots, 9\}$ ）构成的限长度符号序列表示。事实上，任意可枚举集合中的元素均可由有限符号集构成的有限符号序列表示。进一步，为方便讨论，我们写 $A = A_0$ ， $B = A_1$ ， $\dots$ 。

进一步，事实上我们基本不考虑不合法的表达式，因此也可直接称合式为式。最后回到括号的使用上。本质上，括号的使用是为了满足形式语言的最基本要求，即不能产生歧义。让我们来更严格的定义一下语法规则和歧义。

2.2.2 文法和二义性²

用于描述形式化语言语法结构的形式规则被称为文法，或者说文法以特定的规则生成符合语言语法规则的字符序列。如命题逻辑语言即作为一种形式化语言，将看到其可被文法描述。技术上，若我们令语言字符集等于有限集合 T_0 。并将对应的长度不为零的字符序列表示为多元组，即如 $abc = \langle a, b, c \rangle$ 。这样，文法 G 由以下四个部分组成：

- 终结符集 T ：有限集合， $T \subseteq T_0$ ； ϵ 。其中 ϵ 代表空字符，即如 ϵ 等于长度为零的字符序列，如 $a\epsilon b\epsilon = ab$ 。
- 非终结符集 N ：非空有限集合， $N \cap T = \emptyset$ 。
- 开始符号 S ： $S \in N$ 。
- 产生式规则集 P ：有限集合，在最一般的形式上为 $(T \cup N)^+$ 上的二元关系，其中的元素 $\langle \alpha, \beta \rangle$ ，这里也写作 $\alpha \mapsto \beta$ ，代表产生式规则，也即是 α 和 β 之间的对应关系。我们称这种最一般形式下的文法为无限制文法、或0-型文法。本书中主要关注上下文无关文法，即要求 $\alpha \in N$ 。同时，我们称 α 为左部， β 为右部。上下文无关文法也被称为2-型文法。

此外，一般还区分如对字符集 $\{a, b, c\}$ ，可生成语言 $\{a\}^n \times \{b\}^n \times \{c\}^n$ ($n \geq 1$) 的上下文有关文法，也称1-型文法；以及可生成基本正则表达式可识别语言的正则文法，也称0-型文法。0到3-型文法构成了乔姆斯基阶层，有0-型 $\subset$ 1-型 $\subset$ 2-型 $\subset$ 3-型。

这样，即有 $G = \langle T, N, S, P \rangle$ 。

文法通过产生式规则可以在两个方向上应用。一是从开始符号出发通过有限次的应用产生式规则，生成符合语法要求的（有限长度）字符序列，称为语句³。称这种语句的集合为文法 G 产生的语言 $\mathcal{L}(G)$ 。二

¹如前所述，我们会在章节2.3.8看到具体的例子。

²形式语言与自动机理论，21世纪大学本科计算机专业系列教材，清华大学出版社，2013。

³一般来说，文法中语句和形式逻辑中的语句是两个不同的概念，只是在命题逻辑中，其恰好是相等的。

是分析字符序列是否符合语法要求，并在肯定的情况下给出相应的语法结构。对一条产生式规则，在生成中，我们将其左部替换为右部，称为左部派生出右部。在分析中，我们将右部替换为左部，称为右部归约为左部。如此可以看到上下文无关文法中派生规则的选择不受非终结符上下文的影响。

再以包含两个语句符号 AB ¹ 和逻辑联结词 $\neg \rightarrow$ 的命题逻辑语言为例，我们有 $G_p = \langle T_p, N_p, S_p, P_p \rangle$ 。其中终结符集 $T_p = \{A, B, \neg, \rightarrow, (,)\}$ 。非终结符集 $N_p = \{F\}$ ，其中 F 代表合式²。开始符号 $S_p = F$ 。同时，对于上一小节中语法规则的第一条，有 $i: F \mapsto A$ 和 $ii: F \mapsto B$ ，即语句符号构成合式。第二条有 $iii: F \mapsto \neg F$ 。以及第三条有 $iv: F \mapsto (F \rightarrow F)$ 。即 $P_p = \{i, ii, iii, iv\}$ 。

例如我们可以从开始符号 F 出发通过规则 iv ，再对新派生出的两个非终结符 F 分别依次应用规则 iii 、 i 和规则 ii ，生成合式 $(\neg A \rightarrow B)$ 。同时我们也可以从 $(\neg A \rightarrow B)$ 中分析出生成它的规则序列，这些规则的应用也代表了该合式的语法结构。

上下文无关文法使我们可以以语法树的形式描述语法结构。在语法树中，根节点代表开始符号，中间节点代表非终结符，叶节点代表终结符。在产生式规则的应用中，我们将左部的（唯一）非终结符作为父节点，右部的符号序列作为子节点组。例如图 2.2 给出了 $(\neg A \rightarrow B)$ 的对应语法树。

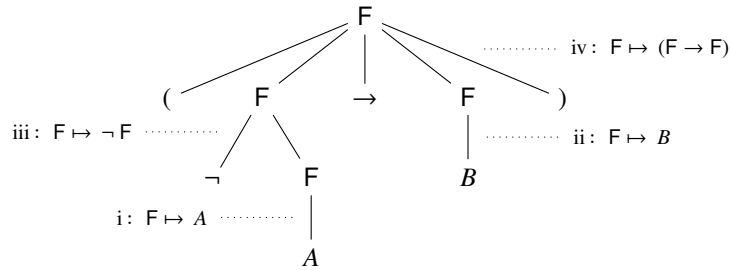
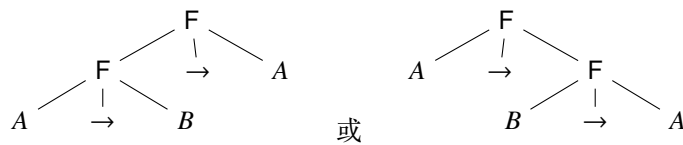


图 2.2: $(\neg A \rightarrow B)$

可以看到，叶节点的终结符序列构成了语句。先根遍历语法树构成了生成（派生）过程，后根遍历构成了分析（归约）过程。这种对语法结构描述的便利性使大多数形式语言采用了上下文无关文法。

对于上下文无关文法 G ，若由其生成的语言 $\mathcal{L}(G)$ 中存在可以以不同语法结构解释，即可以对应不同语法树的语句时，我们认为 $\mathcal{L}(G)$ 是有歧义的，称 G 是具有二义性的。比如文法 $G_p' = \langle \{A, B, \neg, \rightarrow\}, N_p, S_p, \{i, ii, iii, F \mapsto F \rightarrow F\} \rangle$ —— 从上面例子中的命题逻辑文法 G_p 中去掉了括号的使用 —— 就是具有二义性的。比如对语句 $A \rightarrow B \rightarrow A$ ，可以构造出两棵对应的语法树：



上下文无关文法的二义性判断已被证明是不可判定的，即我们不能通过通用的可有效计算的方法确定文法是否具有二义性³。但对于特定的文法，证明是可能存在的。例如，我们可以通过数学归纳法证明 G_p 是无二义性的。证明中我们称只对应一棵语法树的语句为无歧义的。

示例 2.2.1 G_p 是无二义性的。

¹严格的说，是 $A_0 A_1$ （即 $f(0) f(1)$ ）对应的语句符号，下同。

²我们用无衬线字体表示非终结符。

³我们将在章节 2.3.8 中更详细的讨论不可判定问题。

证明：我们对 $\mathcal{L}(G_p)$ 中的语句（合式）长度 n 做数学归纳。

1. 当 $n = 1$ 时，语句只有 A 和 B ，显然是无歧义的。
2. 对 $n > 1$ ，假设对任意 $0 < i < n$ ，任意长度等于 i 的语句都是无歧义的。
3. 则对任意长度等于 n 的语句 γ ，若 γ 形如 $\neg\alpha$ ，那它只能由规则 iii 生成，且 α 长度小于 n ，由假设可得 γ 是无歧义的。
4. 否则 γ 只能形如 $(\alpha \rightarrow \beta)$ ，由 iv 生成，将新生成的 $\rightarrow$ 称为 $\rightarrow_0$ ，我们需要在序列中确定 $\rightarrow_0$ 的位置。
5. 通过数学归纳法，可以得到任意合式中左右括号数相同，合式内 $\rightarrow$ 左侧的左括号数大于右括号数。
6. 可得在 γ 中， $\rightarrow_0$ 左侧左右括号数差等于 1，而由 α 和 β 引入的 $\rightarrow$ 左侧左右括号数差大于 1。由此可确定 $\rightarrow_0$ 的位置，由假设可得 γ 是无歧义的。

因此 $\mathcal{L}(G_p)$ 中的全部语句都是无歧义的，即 G_p 是无二义性的。 $\square$

同时注意即使上下文无关文法具有二义性，我们仍能通过额外的约定解决二义性。比如当语句存在不同语法树与之对应时，我们总是可以约定选择其中如符合某一特定模式的语法树。尽管这在一定程度上算是一种变通方法。

现在我们通过调整语法规则减少命题逻辑语句中的括号使用。首先引入一个简化版的文法表示法，我们将文法表示为多个形如 $\alpha \mapsto \beta_0 \mid \beta_1 \dots \mid \beta_n$ 的规则项，对应产生式规则 $\alpha \mapsto \beta_0, \alpha \mapsto \beta_1$ 到 $\alpha \mapsto \beta_n$ 。所有规则项中涉及的终结字符构成终结符集¹，非终结字符构成非终结符集，第一项的 α 为开始符号。则 G_p 可表示为 $F \mapsto A \mid B \mid \neg F \mid (F \rightarrow F)$ 。此外，《...》为注释。

对调整后的含逻辑联结词 $\neg \wedge \vee \leftrightarrow \rightarrow$ 的命题逻辑文法产生式规则，用自然语言来说，当语句同时含有 $\wedge \vee \leftrightarrow \rightarrow$ 且没有括号时，（按惯例）我们优先归约 $\wedge$ 两侧的内容，我们称 $\wedge$ 具有更高优先级，继而归约 $\vee$ ，继而 $\leftrightarrow$ ，最后 $\rightarrow$ 。如 $A \wedge B \vee C \wedge D \rightarrow E \leftrightarrow F = (((A \wedge B) \vee (C \wedge D)) \rightarrow (E \leftrightarrow F))$ 。在有多个相同的逻辑联结词时，我们从最右侧开始归约，我们称逻辑联结词是右结合的，反之称左结合的。如 $A \rightarrow B \rightarrow C = (A \rightarrow (B \rightarrow C))$ 。同时，括号仍是可加的。

相应的，我们使用 WFF（合式），Sentence（语句符号或括号及以内的子表达式，可认为对应最高优先级的归约操作），Negation[Ⓢ]（否定⁺，可认为是否定或否定以上归约优先级的子表达式），Conjunction[Ⓢ]（合取⁺），Disjunction[Ⓢ]（析取⁺），Biconditional[Ⓢ]（双向条件⁺），Conditional[Ⓢ]（条件⁺）作为非终结符。对比如含有两个语句符号 $A B$ 的命题逻辑语言，图 2.3 给出了相应的文法 G_P ²。

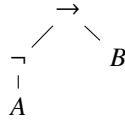
WFF	$\mapsto$ Conditional [Ⓢ]	《开始符号 WFF》
Conditional [Ⓢ]	$\mapsto$ Biconditional [Ⓢ] $\mid$ Biconditional [Ⓢ] $\rightarrow$ Conditional [Ⓢ]	《条件 ⁺ ，最低优先级，右结合》
Biconditional [Ⓢ]	$\mapsto$ Disjunction [Ⓢ] $\mid$ Disjunction [Ⓢ] $\leftrightarrow$ Biconditional [Ⓢ]	《双向条件 ⁺ 》
Disjunction [Ⓢ]	$\mapsto$ Conjunction [Ⓢ] $\mid$ Conjunction [Ⓢ] $\vee$ Disjunction [Ⓢ]	《析取 ⁺ 》
Conjunction [Ⓢ]	$\mapsto$ Negation [Ⓢ] $\mid$ Negation [Ⓢ] $\vee$ Conjunction [Ⓢ]	《合取 ⁺ 》
Negation [Ⓢ]	$\mapsto$ Sentence $\mid$ $\neg$ Negation [Ⓢ]	《否定 ⁺ 》
Sentence	$\mapsto$ A $\mid$ B $\mid$ (Conditional [Ⓢ])	《语句符号或括号子表达式》

图 2.3: 含两个语句符号 $A B$ 的命题逻辑文法 G_P

最后我们给出（不严格的）简化版语法树。以图 2.2 为例，简化后我们有：

¹我们将避免在终结符集中使用 $\mapsto \mid \langle \rangle \oplus$ 。

²更多关于文法及其分析工具，即语法分析器的介绍见??。



2.2.3 语义，演绎规则

在确定了语法之后，我们需要给语句填上内容。我们使用正体表示语句对应的内容。我们可以将五个逻辑联结词构成的语句对应到自然语言：

$\neg \alpha$	α 不符合事实	$\alpha \leftrightarrow \beta$	α 当且仅当 β
$\alpha \wedge \beta$	α 且 β	$\alpha \rightarrow \beta$	若 α ，则 β
$\alpha \vee \beta$	α 或 β		

比如语句符号 A 对应 正在下雨， B 对应 正在刮风，则 $\neg A$ 代表 正在下雨不符合事实，或者说现在没有下雨。 $A \wedge B$ 代表 正在下雨且正在刮风， $A \vee B$ 代表 正在下雨或正在刮风。 $\neg(A \wedge B)$ 代表 正在下雨且正在刮风不符合事实。

那么对于推导（演绎），即从已知的事实集确认新的事实，直观的说，比如对于上面的例子，我们需要从事实（集）正在下雨且正在刮风中确认事实 正在下雨和 正在刮风，也就是说我们的推导过程需要可以（纯形式的）从集合 $\{A \wedge B\}$ 确认 A 和 B 。我们将这种通过一定规则从语句集 Γ 确认语句 σ 的过程称为演绎，也称演算，将其肯定的结果，即从 Γ 确认了 σ ，称为 Γ 语法蕴含 σ ，也称 Γ 逻辑蕴含 σ ，写作 $\Gamma \vdash \sigma$ 。并在不会引起歧义的情况下，省略集合 Γ 表示中的花括号。类似的，将 $\emptyset \vdash \Sigma$ 写作 $\vdash \Sigma$ 。同时，对语句 $\gamma \sigma$ ，若 $\gamma \vdash \sigma$ 且 $\sigma \vdash \gamma$ ，我们称 γ 和 σ 是语法等价或逻辑等价的，写作 $\gamma \vdash \sigma$ 。如此，对上面的例子，我们会希望命题逻辑的演绎规则除了可以得到 $\{A \wedge B\} \vdash A$ （或 $A \wedge B \vdash A$ ）， $A \wedge B \vdash B$ ，还可以使我们有如 $A \vdash A$ ； $A \vdash A \vee B$ ； $A, B \vdash A \wedge B$ ； $\neg A, B \vdash \neg(A \wedge B)$ 以及 $\vdash A \vee \neg A$ 和 $A \wedge B \vdash \neg \neg B \wedge A$ 等。

同时，当我们有了像 $A \wedge B \vdash A$ 这样的演绎结果，直观的说，不管语句符号 AB 对应什么内容，我们都能从相应的已知事实中确认相应的新的事实。更严格的，这种直观上的正确性依赖所对应内容满足一定要求，并且这种要求是符合我们对自然语言的基本理解的。以对应到自然语言为例，要求：

- 对语句符号对应的自然语句，我们认为要么符合事实，称为真，要么不符合事实，称为假。
- 如果自然语句 α 为真，那么 α 不符合事实 必为假。如果 α 为假，那么 α 不符合事实 必为真。以真值表的形式表示：

α	α 不符合事实
真	假
假	真

- 类似的，要求：

α	β	α 且 β	α 或 β	α 当且仅当 β	若 α ，则 β
真	真	真	真	真	真
真	假	假	真	假	假
假	真	假	真	假	真
假	假	假	假	真	真

对于自然语言，这里可能唯一需要注意的是如果 α 是假的，那无论 β 是否为真，若 α 则 β 都应被认为是真的，这同样是符合解决问题的要求的。比如对于 若下雨，则降雨量不为零，在不下雨（ α 假）时，降雨量为零（ β 假），语句为真；而对于 若下雨，则大气中含有水蒸气，在 α 为假时， β 为真，语句仍然

为真。

同时，我们可以将命题逻辑语言（包括逻辑连词和语句符号）对应到任何符合上述要求的内容上。比如对于其中的第一点，我们要求语句对应的内容可以明确的分为两个类别，或称二值逻辑¹。

而对于内容，我们还可以注意到命题逻辑不关心语句符号对应的具体内容，而只关心其对应内容的类别（真假）。因此我们可以认为语句符号（以及语句）直接对应到真假上。更正式的，对语句符号集 S 我们有对应函数 $\mathfrak{U}: S \rightarrow \{\text{真}, \text{假}\}$ ，我们称 $\mathfrak{U}$ 为真值指派。显然，对于 S 对应的语言 $\mathcal{L}_S$ ，我们有 $2^{|S|}$ 种不同的真值指派。此外，我们将形式逻辑语言中像语句符号这样可以对应到不同内容上的符号称为该语言的参数。将对这些参数的内容赋予称为解释，所以真值指派也可被称为解释。

现在，我们回到讨论演绎规则。对于 $\Gamma \vdash \sigma$ ，我们可以将 Γ 中的语句称为前提、假设或者非逻辑公理^{2,3}，将 σ 称为推论，引理或定理。也称通过演绎规则由 Γ 确认 σ 为由 Γ 证明 σ ，因此也称 σ 是可证明的。对一个（非逻辑）公理集，我们可以通过演绎规则确认各种不同的推论或者定理，我们将所有这些推论或者定理构成的集合称为理论。显然，理论由其公理集及所使用的形式逻辑决定。

从纯形式上说，若不考虑演绎规则的合理性，就像允许语句 $A \wedge \neg A$ 一样，其只需要是一种可有效计算的方法，若当它在输入如 $\langle \Gamma, \sigma \rangle$ 时返回（最终停止），我们认为有 $\Gamma \vdash \sigma$ 。或者说对于演绎规则 F ，我们有 $\vdash$ 等于二元关系 $\{ \langle \Gamma, \sigma \rangle : F(\Gamma, \sigma) \text{ 返回} \}$ ⁴。不过我们仍区分一种特殊的情况，即若对前提集 Γ ，对任意语句 σ 有 $\Gamma \vdash \sigma$ ，即若我们可以由 Γ 演绎得到任意推论，我们称这种情况为语法上不一致的。注意此时造成不一致的原因可能是前提集 Γ 中包含矛盾，也可能是演绎规则中包含矛盾，即演绎规则是不合理的。特别的，对命题逻辑，我们可以通过检查是否存在 α 有 $\Gamma \vdash \alpha \wedge \neg \alpha$ 判断一致性。

另外需要注意的是，在这种最小要求的 F 下，我们并没有有效的办法确定如是否 $\Gamma \vdash \sigma$ ，即我们可以确定有 $\Gamma \vdash \sigma$ ，但不能确定有 $\Gamma \not\vdash \sigma$ 。因为可证明并没有通用的有效方法确定是否 F 返回，即著名的停机问题结论。特别的，我们一般不能通过尝试得到如 $\Gamma \vdash \neg \sigma$ 得出 $\Gamma \not\vdash \sigma$ 。比如我们令 F 返回当且仅当 $\sigma \in \Gamma$ ，如令 $\Gamma = \{\sigma, \neg \sigma\}$ ，有 $\Gamma \vdash \sigma$ 且 $\Gamma \vdash \neg \sigma$ 。又如在同样的 F 下，令 $\Gamma = \emptyset$ ，则对任意 σ ，我们既不能证明 σ ，也不能证明 $\neg \sigma$ 。

同时，对一个集合 A ，若我们有有效方法对任意 a 确定是否 $a \in A$ ，我们称 A 是可判定的。若我们有有效方法对任意 $a \in A$ ，（在任意长的时间内）给出肯定的结果，我们称 A 是半可判定的。这样，一般的说，二元关系集 $\vdash$ 是半可判定的。

最后，我们可以认为可以由可判定公理集 A 得到的理论为可公理化的⁵，当 A 有限时，为有限可公理化的。

2.2.4 模型，系统合理性

显然，我们关心的是合理的演绎规则，这可以分为可靠的和完备的两个方面。直观的说，对一个演绎规则，若前提对应为真（符合事实），则由该规则确认的任何推论对应也为真，我们称该演绎为可靠的。

¹ 存在语句对应三个或多个类别的形式逻辑，称为三值逻辑和多值逻辑。

² 从形式上来说，这几个称呼本质上是一样的，在本书中将选择更符合上下文的称呼。

³ 如章节 3.1.3 中将看到的，一些系统中还有和演绎规则一起给出的逻辑公理。为保证（将在下一小节讨论的）演绎规则的合理性，逻辑公理一般不能随意指定。下面大多数情况下我们将省略非逻辑。

⁴ 可有效计算的 F 并不是集合论意义上的函数，可以认为它是一段程序。对于可有效计算和接下去的停机问题、可判定等，见章节 2.3。

⁵ 更精确的，一般定义可以由（全）递归公理集得到的理论为可公理化的，但如我们将在章节 2.3.7 看到，可判定集合即为（全）递归集合。这里采用可判定以延后对（全）递归的定义。

反过来，若前提对应为真，我们可以通过该演绎规则确认任何对应为真的推论，我们称其为完备的。

我们以一个对有限语句集 Γ ，语句 σ ，能够可靠且完备的确定 $\Gamma \vdash \sigma$ 的命题逻辑演绎规则为目标，更严格的定义可靠性和完备性。

我们将真值表对应到形式上（即如 α 对应真时， $\neg \alpha$ 对应假），重述如下：

α	$\neg \alpha$
真	假
假	真

α	β	$\alpha \wedge \beta$	$\alpha \vee \beta$	$\alpha \leftrightarrow \beta$	$\alpha \rightarrow \beta$
真	真	真	真	真	真
真	假	假	真	假	假
假	真	假	真	假	真
假	假	假	假	真	真

则给定语句符号如 A, B ，我们可以递归的建立从 A, B 到其上任意复杂的语句的真值表，如：

A	B	$\neg A$	$A \vee \neg A$	$\neg(A \wedge B)$	$\neg A \vee \neg B$
真	真	假	真	假	假
真	假	假	真	真	真
假	真	真	真	真	真
假	假	真	真	真	真

反查真值表，我们可以得到如 $\neg(A \wedge B)$ 对应为真时， A, B 可以分别对应真和假。

同时，在形式逻辑中，我们一般称令语句集 Γ 内的元素均对应真的解释 $\mathfrak{S}$ 为 Γ 的模型¹，写作 $\models_{\mathfrak{S}} \Gamma$ ²。我们称 $\mathfrak{S}$ 是 Γ 内元素 γ 的模型，称 γ 在 $\mathfrak{S}$ 上为真。否则，我们有 γ 在 $\mathfrak{S}$ 上为假。如此，在上面的例子中，我们有如真值指派 $\{\langle A, \text{真} \rangle, \langle B, \text{假} \rangle\}$ 为 $\neg(A \wedge B)$ 的模型。最后，类似 $\vdash$ ，我们也可以省略花括号的使用。

对语句集 Γ ，语句 σ ，解释 $\mathfrak{U}$ ，写 $\Gamma \models_{\mathfrak{U}} \sigma$ 表示 $\mathfrak{U}$ 是 Γ 的模型意味着 $\mathfrak{U}$ 是 σ 的模型。则若对任意解释 $\mathfrak{S}$ 都用 $\Gamma \models_{\mathfrak{S}} \sigma$ ，则称 Γ 语义蕴含 σ ，也称 Γ 恒真蕴含 σ ，类似语法蕴含，写作 $\Gamma \models \sigma$ 。同时，称 σ 是 Γ 的结果。特别的，当 $\Gamma = \emptyset$ 时，写作 $\models \sigma$ ，并称 σ 是有效的，或称 σ 是恒真语句。当存在解释 $\mathfrak{U}$ 是 Γ 的模型时，我们称 Γ 是可满足的，称 $\mathfrak{U}$ 满足 Γ ；反之称 Γ 是不可满足的，或语义上不一致的。当 Γ 不可满足时，对任意 σ ， $\Gamma \models \sigma$ 成立（即若假则任意）。最后，同样类似的，对语句 γ, σ ，若 $\gamma \models \sigma$ 且 $\sigma \models \gamma$ ，我们称 γ 和 σ 是语义等价的，写作 $\gamma \models \sigma$ 。如此，对于上面的例子，我们有如 $\neg A \models \neg(A \wedge B)$ ； $\models A \vee \neg A$ 以及 $\neg(A \wedge B) \models \neg A \vee \neg B$ 。

现在我们定义可靠性和完备性。对一个特定的形式逻辑语言中的语句集 Γ ，语句 σ ，其演绎规则的可靠性和完备性定义如下：

- 可靠性：若 $\Gamma \vdash \sigma$ ，则 $\Gamma \models \sigma$ 。
- 完备性：若 $\Gamma \models \sigma$ ，则 $\Gamma \vdash \sigma$ 。

显然，对于可靠且完备的演绎规则，语法蕴含和语义蕴含是等价的，我们可以简称为蕴含。对语句 γ, σ ，语法等价 $\gamma \vdash \sigma$ 和语义等价 $\gamma \models \sigma$ 是等价的，因此我们也直接称 γ 和 σ 是等价的，写作 $\gamma \equiv \sigma$ 。同时，对于可靠且完备的演绎规则，语句集 Γ 不可满足当且仅当对任意语句 σ ，有 $\Gamma \vdash \sigma$ ，即语法上的一致性和语义上的一致性也是等价的。

¹类似于对其它如自然科学问题的建模，可以认为寻找（数学）模型帮助我们更好的理解纯形式上的对象和规则。

²有些人也写作 $\mathfrak{S} \models \Gamma$ 。我们采用下标的形式是为了和马上就要看到的 $\models$ 的另一种用法作区分，并令其与 $\vdash$ 保持一致。

在进一步给出有限场景下可靠且完备的命题逻辑演绎规则之前。我们首先引入逻辑符号 $\top$ 对应真，即在任何解释上 $\top$ 为真，相应的引入 $\perp$ 对应假。可定义 $\top$ 为任意的恒真语句，如 $A \rightarrow A$ ； $\perp$ 为任意的不可满足的语句，如 $A \wedge \neg A$ 。

这样，对有限语言符号集的命题逻辑中在输入有限语句集 Γ ，语句 σ 时，可以通过如下方法 $F(\Gamma, \sigma)$ 确定 $\Gamma \vdash \sigma$ ：

1. 令 $\phi = \bigwedge \Gamma$ ； $\top$ （增加 $\top$ 以处理 Γ 为空集的情况）。因为 Γ 是有限的，所以 ϕ 是语句。同时因为语句符号是有限的，所以我们可以构建有限的语句符号到 ϕ 的真值表 $\mathcal{T}$ 。
2. 从有限的 $\mathcal{T}$ 中我们可以在有限时间内选出所有令 ϕ 为真的真值指派，即所有 ϕ 的模型。
3. 对每一个 ϕ 的模型 $\mathfrak{A}$ ，我们可以通过到 σ 的真值表在有限时间内检查 $\mathfrak{A}$ 是否为 σ 的模型。
4. 因为只有有限个 ϕ 的模型，我们可以在有限的时间内检查其中每一个模型。若每一个 ϕ 的模型也是 σ 的模型，演绎规则 F 返回。

这里若在每个 ϕ 的模型也是 σ 的模型时，返回 1，否则返回 0，我们就可以用该方法，记作 F' ¹ 确定是否 $\Gamma \vdash \sigma$ ，也就是说我们可以使用该方法同样得到 $\Gamma \not\vdash \sigma$ 。这样，我们由 Γ 得到的理论 T 是可判定的（有有效方法确定语句是否在理论内）。同时，显然在 $\Gamma = \emptyset$ 时，我们得到了任意（命题逻辑）恒真语句的判定方法²。

可以看到，演绎规则 F 的实现完全依照了语义蕴含的定义，并且采用了完全对应语义要求的真值表，因此这里的可靠性和完备性是显而易见的。

而通过注意到有限的语句集包含的语句符号也是有限的，且语句的真值只受其所包含的语句符号影响。对有限的 Γ ，我们可以通过只构造语句符号集 $\{x : \text{存在语句属于 } \Gamma \text{ 包含 } x\}$ 到 Γ 和 σ 的真值表确定（即使在语句符号无限时）是否有 $\Gamma \vdash \sigma$ 。

这样，我们即得到了有限语句集下可靠且完备的命题逻辑演绎规则。当一个形式逻辑采用了可靠 / 完备的演绎规则时，我们称该系统是可靠 / 完备的。同时，当一个形式逻辑存在可靠且完备的演绎规则时，我们默认所讨论的演绎规则是可靠且完备的，默认该形式逻辑采用的是可靠且完备的演绎规则；类似的，当仅存在可靠的演绎规则时，我们默认讨论的是可靠的演绎规则。换句话说，我们将不再考虑任何明显不合理的演绎规则³。

最后，考虑模型和理论的关系，显然对一些理论，存在语句在部分模型下为真，部分模型下为假。如由 A, B 两个语句符号构成的前提集为 $\{A\}$ 的命题逻辑理论，语句 B 在部分模型下为真，部分模型下为假。则在合理的演绎规则下，这样的语句即不可被证明，也不可被证伪，称这样的语句为独立的，比如 B 相对于 $\{A\}$ 是独立的。但在很多时候，我们希望在我们的理论中不存在这样的语句，希望所有的语句都是可证明或可证伪。更严格的说，令理论 T 的前提集为 Γ ，若对任意语句 α 有 $\Gamma \vdash \alpha$ 或 $\Gamma \vdash \neg \alpha$ ，则我们称理论 T 是完备的。比如对上述仅由 A, B 两个语句符号构成的语言，由前提集 $\{A, \neg B\}$ 构成的理论是完备的。注意完备的理论不一定是一致的，即可能同时有 $\Gamma \vdash \alpha$ 和 $\Gamma \vdash \neg \alpha$ 。而一致且完备的理论有要么 $\Gamma \vdash \alpha$ 要么 $\Gamma \vdash \neg \alpha$ ，即有 $\Gamma \vdash \alpha$ 异或 $\Gamma \vdash \neg \alpha$ 。我们也称一致且完备的理论是最大一致的，因我们不能再继续向这样的理论中增加不可证明的语句而保持一致性。也可以看到，在理论一致且完备的情况下，我们可以通过尝试得到如 $\Gamma \vdash \neg \sigma$ 得到 $\Gamma \not\vdash \sigma$ 。且对于一致且完备的理论，其模型下语句的真假性

¹ 章节 2.2.6 中还将用到 F' 。

² 一些材料会侧重于对恒真语句的讨论。事实上，由 $\vdash$ 的定义可得对任意语句 γ, σ ， $\gamma \vdash \sigma$ 等价于 $\vdash \gamma \rightarrow \sigma$ 。因此，恒真语句演绎的可靠和完备性以及恒真语句的可判定性等价于这里演绎规则 F' 的可靠和完备性、理论 T 的可判定性，以及章节 2.2.6 中演绎规则 F'' 的可靠和完备性。

³ 当然，我们会证明演绎规则的可靠性和完备性。

也是一致的，或者说以理论所使用语言角度描述这些模型都是没有区别的¹，并可以通过演绎规则确定这些模型下语句的真假性。然而，一般的说，在形式逻辑中，我们并不一定能得到一致且完备的前提集（公理集）。事实上，我们将在章节 3.2.5 中看到，哥德尔第一不完备定律告诉我们当一阶逻辑的公理集蕴含某些最基本的算术定理时，其理论就一定不是可公理化且一致且完备的。而本质上连续统假设在 ZFC 集合论中不可证明也不可证伪即因为 ZFC 集合论不是完备的。

这样，我们即有了足够多的相关概念，且尽管简单，有限语句集下存在可靠且完备的命题逻辑演绎规则也可以算是我们第一个元逻辑下的重要定理。因此，是时候介绍一下元逻辑并梳理一下这些相关概念所在的位置了。

2.2.5 元逻辑概念

元逻辑是研究形式逻辑系统本身性质的逻辑分支。比如前文中对文法和二义性、演绎规则的可靠性完备性、以及公理集的一致性完备性的定义和论证以及后面对可有效计算的定义和论证都可认为²属于元逻辑的范畴。同时，元逻辑也是一种逻辑，其也可以并且应该被形式化。元逻辑也是一种逻辑也意味着我们可以用元逻辑研究元逻辑本身。

对于元逻辑的作用，更通俗的说，特定的形式逻辑帮助我们解决具体问题，而元逻辑帮助我们确定该特定形式逻辑可以正确的帮助解决我们的问题。

在元逻辑对特定形式逻辑的研究中，我们称被研究的形式逻辑语言为目标语言，称元逻辑语言为元语言。如在前文对命题逻辑的讨论中，命题逻辑语言为目标语言，（半）自然语言为元语言。具体的说，如命题逻辑逻辑联结符 $\neg \wedge \vee \leftrightarrow \rightarrow$ 以及语句符号 $A B$ 等是目标语言中的符号（词汇），如集合相关的符号如 $\in \emptyset$ 和蕴含符号 $\vdash$ 及等价符号如 $\equiv$ 等是元语言中的符号。

元逻辑又可被粗略的分为对形式部分的研究和对内容部分的研究，或者称对语法部分的研究和语义部分的研究。可认为其中对语法及其上的演绎规则的研究发展出了证明论；对语义，即解释和模型的研究发展出了模型论。在后续的章节中，我们将看到典型的证明论和模型论定理。

2.2.6 无限多的前提

接着，我们考虑对无限的语句集 Γ 和语句 σ 确定 $\Gamma \vdash \sigma$ 。我们将会马上看到 $\Gamma \models \sigma$ 当且仅当存在有限 $\Delta \subset \Gamma$ 有 $\Delta \vdash \sigma$ 。因此现在的问题是如何找到 Δ 。我们使用搜索。

这里我们在 F' 的基础上进行搜索来实现演绎规则 F'' 。回顾对任意有限语句集 Δ ，语句 σ ，当 $\Delta \vdash \sigma$ 时， $F'(\Delta, \sigma)$ 返回 1，否则返回 0。首先，我们需要有效方式表示无限的语句集，这种有效表示的最一般形式为可枚举集合。而这里的有效是指，类似演绎规则，当某个元素属于该集合时，我们有有效方法肯定该元素属于该集合^{3 4}。同时，需要注意可枚举集合的有限子集也是可枚举的⁵。这样，通过在输入可枚举语句集 Γ 和语句 σ 时，我们逐一枚举 Γ 的有限子集 Δ ，若 $F'(\Delta, \sigma)$ 返回 1，返回这样的方式我们可以实现处理无限多前提的演绎规则 F'' ，显然 F'' 不一定返回。

¹元逻辑的语言可能具有更强的表达能力，因此从元逻辑的角度看这些模型可以是不同的。而模型不同一般也指在元逻辑下不同。

²也可以理解为元逻辑借用了如形式语言理论和可计算理论中的相关内容。

³可以将可枚举集合在计算中的具体表示想象为生成器，而确认元素属于集合即是等待生成器生成该元素。

⁴显然 F'' 不能处理不可枚举的语句集。事实上，一般认为没有有效方法处理不可枚举集合。

⁵我们将在章节 2.3.7 给出证明。

接着，我们证明 F'' 的可靠性和完备性。我们首先证明由紧性定理可得 F'' 是可靠且完备的。然后证明紧性定理。

定理 2.2.2 (紧性定理) 在命题逻辑中，对任意语句集 Γ ，若 Γ 的每一个有限子语句集都是可满足的，则 Γ 是可满足的¹。

此外，我们还需要几条引理：

引理 2.2.3 命题逻辑中，对任意语句集 Γ Δ 和语句 σ ：

- a. 若 $\Delta \subseteq \Gamma$ 且 $\Delta \models \sigma$ ，则 $\Gamma \models \sigma$ 。
- b. $\Gamma \models \sigma$ 当且仅当 $\Gamma; \neg \sigma$ 是不可满足的。
- c. 若 Γ 是不可满足的，则 $\Gamma \models \sigma$ 。

证明：由语义蕴含的相关定义直接可得。 □

定理 2.2.4 F'' 是可靠且完备的。

证明：对任意可枚举语句集 Γ ，语句 σ 分别证明 $F''(\Gamma, \sigma)$ 的可靠性和完备性。

- 可靠性：若 $\Gamma \vdash \sigma$ ，则存在有限 $\Delta \subseteq \Gamma$ 且 $\Delta \vdash \sigma$ 。由 F' 的可靠性可得 $\Delta \models \sigma$ 。由引理 a 可得 $\Gamma \models \sigma$ 。
- 完备性：若 $\Gamma \models \sigma$ ，则由引理 b 可得 $\Gamma; \neg \sigma$ 是不可满足的。同时，由紧性定理可得 $\Gamma; \neg \sigma$ 不可满足意味着存在有限 $\Delta \subseteq \Gamma$ 不可满足。因此，若 $\neg \sigma \notin \Delta$ ，则由引理 c 可得 $\Delta \models \sigma$ ，否则由引理 a 同样可得 $\Delta \models \sigma$ 。由有限场景下 F' 的可靠性可得 $\Delta \vdash \sigma$ ，有 F'' 返回，即 $\Gamma \vdash \sigma$ 。 □

接着，我们证明紧性定理²。

引理 2.2.5 命题逻辑中，

- a. 对任意语句集 Γ ，若 Γ 的每一个有限子集都是可满足的，则对任意语句 α ， $\Gamma; \alpha$ 或 $\Gamma; \neg \alpha$ 的每一个有限子集也都是可满足的。
- b. 令语句集 Θ 满足 i：其每一个有限子集都是可满足的，ii：对任意语句 α 有 $\alpha \in \Theta$ 或 $\neg \alpha \in \Theta$ 。同时令真值指派 $\mathfrak{A}$ 为对任意语句符号 A ：

$$\mathfrak{A}(A) = \begin{cases} \text{真} & \text{若 } A \in \Theta \\ \text{假} & \text{若 } A \notin \Theta \end{cases}$$

则 iii：对任意语句 θ ， θ 在 $\mathfrak{A}$ 上为真当且仅当 $\theta \in \Theta$ 。

证明：对 a，若推论不成立，即 $\Gamma; \alpha$ 和 $\Gamma; \neg \alpha$ 都存在不可满足的有限子集。则存在语句集 $\Delta_1 \subseteq \Gamma$ 有 $\Delta_1; \alpha$ 不可满足，存在 $\Delta_2 \subseteq \Gamma$ 有 $\Delta_2; \neg \alpha$ 不可满足。令 $\Delta = \Delta_1 \cup \Delta_2$ ，有 $\Delta; \neg \alpha$ 不可满足。则 $\Delta \models \alpha$ ，与 Δ 可满足而 $\Delta_1; \alpha$ 不可满足矛盾。对 b，对 θ 的结构进行数学归纳，即 θ 由单个语句符号构成时满足推论。当 $\theta = \neg \theta_1$ 等时，假设 θ_1 等满足推论，由条件 i 和 ii 有 θ 满足推论。 □

紧性定理 2.2.2 的证明：我们将在 Γ 的基础上构造满足引理 b 中条件的 Θ ，并证明其对应的真值指派 $\mathfrak{A}$ 是 Γ 的模型。由语句是可枚举的，我们枚举所有语句 $\alpha_1 \alpha_2 \dots$ 。令 $\Theta_0 = \Gamma$ ，对 $n > 0$ ，

$$\Theta_n = \begin{cases} \Theta_{n-1}; \alpha_n & \text{若其每一个有限子集都是可满足的} \\ \Theta_{n-1}; \neg \alpha_n & \text{若其它} \end{cases}$$

¹ 紧性可直观理解为可通过论证有限场景下的性质得到无限场景下的性质。

² 证明自 Herbert B. Enderton. A Mathematical Introduction to Logic. New York: Academic Press, 1972, pp. 59–60.

令 $\Theta = \bigcup_n \Theta_n$ (即 $\Theta_{n \rightarrow \infty}$ ^{注1})，则 Θ 满足引理 b 中的条件 i (因对 Θ 的任意有限子集 Δ 均存在 Θ_n 满足 $\Delta \subseteq \Theta_n$)、ii。则存在 $\mathfrak{A}$ 满足 iii。又由 $\Gamma \subseteq \Theta$ 有 $\mathfrak{A}$ 是 Γ 的模型。 $\square$

最后，尽管这里 $\mathbf{F}''$ ，即搜索证明的过程是可能不返回的，即通过搜索确定是否存在有限语句集 $\Delta \subseteq \Gamma$ 有 $\Delta \vdash \sigma$ 是不可判定的。但往往我们仍希望验证证明的问题是可判定的，即如这里我们希望在给定某个 Δ 有 $\Delta \vdash \sigma$ 时，存在有效方法确定是否 $\Delta \subseteq \Gamma$ 。这等价于要求 Γ 是可判定的。尽管如此，如章节 2.3.8 将要看到的，给定一个任意可枚举集合，我们并没有通用的有效方法确定其是否是可判定的。事实上，我们无法确定一系列的可枚举集合性质，如该集合的大小，该集合是否包含某个特定元素等等。

2.2.7 逻辑联结词的功能完备性

本小节主要讨论关于逻辑联结词的两个问题，如对 $\neg \wedge \vee \leftrightarrow \rightarrow$ ，一是其中是否包含了冗余的联结词，二是其中是否包含了足够的联结词。我们先来看第一个问题，并给出一个用形式语言作为语义内容的例子。

通过观察真值表，我们可以很容易的发现对任意式 $\alpha\beta$ ， $\alpha \leftrightarrow \beta \equiv (\alpha \rightarrow \beta) \wedge (\beta \rightarrow \alpha)$ 。因此，可以认为 $\leftrightarrow$ 是冗余的，即 $\leftrightarrow$ 是可以被 $\rightarrow \wedge$ 定义的。这里，我们借此机会将其中被定义的概念放到形式语言语义的角度进行讨论。即在定义了新符号后，我们获得了一个新的语言，我们将原来的语言作为元语言，新语言作为目标语言，对新语言的性质进行讨论。尽管大多数时候我们不会以这样的方式看待新符号的定义，但由一种已知的形式语言系统描述新的形式语言系统是形式化讨论形式语言和逻辑系统的一般方法。

这里，对任意语句符号集 $\mathcal{S}$ ，令终结符集 $T_p = \mathcal{S} \cup \{\neg, \wedge, \vee, \rightarrow, (,)\}$ ，现在我们有命题逻辑语言 $\mathcal{L}_p = \mathcal{L}(T_p, \dots)$ ，以及在其基础上增加了 $\leftrightarrow$ 的目标逻辑语言 $\mathcal{L}_{p\leftrightarrow} = \mathcal{L}(T_p; \leftrightarrow, \dots)$ 。这里的元语言为 (半)自然语言和 $\mathcal{L}_p$ 的并集。我们要将 $\mathcal{L}_{p\leftrightarrow}$ 中的式对应到 $\mathcal{L}_p$ 上并给出作为元语言的 $\mathcal{L}_p$ 的真值定义。其中式的对应方式简单的说，对 $\leftrightarrow$ 以外的符号进行一一对应，对 $\leftrightarrow$ 进行展开。具体如下：

1. 单个语句符号如 A 对应到 A 。
2. 若 $\mathcal{L}_{p\leftrightarrow}$ 中的式 $\alpha\leftrightarrow$ 对应到 α ，则 $\neg\alpha\leftrightarrow$ 对应到 $\neg\alpha$ 。
3. 若 $\mathcal{L}_{p\leftrightarrow}$ 中的式 $\alpha\leftrightarrow$ 对应到 α ， $\beta\leftrightarrow$ 对应到 β ， $\square \in \{\wedge, \vee, \rightarrow\}$ ，则 $\alpha\leftrightarrow \square \beta\leftrightarrow$ 对应到 $\alpha \square \beta$ 。
4. 若 $\mathcal{L}_{p\leftrightarrow}$ 中的式 $\alpha\leftrightarrow$ 对应到 α ， $\beta\leftrightarrow$ 对应到 β ，则 $\alpha\leftrightarrow \leftrightarrow \beta\leftrightarrow$ 对应到 $(\alpha \rightarrow \beta) \wedge (\beta \rightarrow \alpha)$ 。

我们令式 $\alpha\leftrightarrow \in \mathcal{L}_{p\leftrightarrow}$ 对应真当且仅当其对应的 $\alpha \in \mathcal{L}_p$ 对应真。注意这里我们并没有对 $\alpha \in \mathcal{L}_p$ 中的真值作出要求。如此，我们就可以仅通过 $\mathcal{L}_p$ 对应的演绎规则对 $\mathcal{L}_{p\leftrightarrow}$ 对应的演绎规则进行论证。

除 $\leftrightarrow$ 以外，对式 $\alpha\beta$ ，有 $\alpha \wedge \beta \equiv \neg(\alpha \rightarrow \neg\beta)$ ， $\alpha \vee \beta \equiv \neg\alpha \rightarrow \beta$ ，即我们可以仅用 $\neg \rightarrow$ 定义其它逻辑联结符。类似的，我们也可以用如 $\neg \wedge$ 定义其它联结符，如 $\alpha \vee \beta \equiv \neg(\neg\alpha \wedge \neg\beta)$ 。同样，我们也可以定义新的二元或多元逻辑联结符，如定义异或 $\alpha \oplus \beta$ 为 $(\alpha \wedge \neg\beta) \vee (\neg\alpha \wedge \beta)$ 。

显然，进一步减少逻辑联结词使其只含 $\neg$ 是无法定义如 $\rightarrow$ 的。这自然引出了第二个问题，我们是否包含了足够的逻辑联结词。事实上，除了 $\neg$ 是不够的以外，我们还不能确定如 $\neg \wedge \vee \leftrightarrow \rightarrow$ 是否足够了，或者说是否是功能完备的。更严格的说，命题逻辑中，对逻辑联结词集合 C 和任意有限语句符号集 $\mathcal{S}$ ，令其构成的命题逻辑语言为 $\mathcal{L}_C$ ，令 $\mathcal{A} = \{\mathfrak{A} : \mathfrak{A} \text{ 为 } \mathcal{S} \text{ 的真值指派}\}$ ，我们称 C 是功能完备的，若对任意函数 $\mathcal{F} : \mathcal{A} \rightarrow \{\text{真}, \text{假}\}$ ，存在语句 $\alpha \in \mathcal{L}_C$ ，有 i：对任意 $\mathfrak{A} \in \mathcal{A}$ ， $\mathcal{F}(\mathfrak{A}) = \text{真}$ 当且仅当 $\mathfrak{A}$ 是 α 的模型。下面看一个功能不完备的例子。

¹不严格的我们认为 ∞ 为 $\aleph_0$ 。 $\Theta_{n \rightarrow \infty}$ 即为 Θ_n 当 n 趋向于 (可数) 无限时的结果。

示例 2.2.6 $\{\neg\}$ 不是功能完备的。

证明：对语句符号集 $\{A, B\}$ ，有如下真值指派到真值的函数 $\mathcal{F}$ ，其中可以看到如 $\mathcal{F}(\langle A, \text{真} \rangle, \langle B, \text{假} \rangle) = \text{假}$ ：

A	B	$\mathcal{F}(\mathfrak{A})$
真	真	真
真	假	假
假	真	假
假	假	假

显然语句 $A, B, \neg A, \neg B, \neg\neg A, \dots$ 都不满足条件 i。因此 $\{\neg\}$ 不是功能完备的。 $\square$

接着证明 $\{\neg, \wedge, \vee\}$ 是功能完备的。

定理 2.2.7 $\{\neg, \wedge, \vee\}$ 是功能完备的。

证明¹：我们对语句符号集 S 做数学归纳。

1. 对任意语句符号 A ，当 $S = \{A\}$ 时，语句 $A, \neg A, A \vee \neg A$ 和 $A \wedge \neg A$ 分别对应全部四种可能的 $\mathcal{F}$ ，条件 i 成立。
2. 对任意非空语句符号集 $S' = \{A'_1, \dots, A'_n\}$ 和 A ，当 $S = S'; A$ ，假设对 S' ，i 成立。则任意函数 $\mathcal{F} : \{\mathfrak{A} : \mathfrak{A} \text{ 为 } S \text{ 的真值指派}\} \rightarrow \{\text{真}, \text{假}\}$ 可表示为：

A	$A'_1 \cdots A'_n$	$\mathcal{F}(\mathfrak{A})$
真	$\mathfrak{A}'_1$	x_1
$\vdots$	$\vdots$	$\vdots$
真	$\mathfrak{A}'_{2^n}$	x_{2^n}
假	$\mathfrak{A}'_1$	y_1
$\vdots$	$\vdots$	$\vdots$
假	$\mathfrak{A}'_{2^n}$	y_{2^n}

其中对 $0 < i \leq 2^n$ ， $x_i \in \{\text{真}, \text{假}\}$ ， $y_i \in \{\text{真}, \text{假}\}$ 。且 $x_1 \dots x_{2^n}, y_1 \dots y_{2^n}$ 的取值决定了 $\mathcal{F}$ 。同时，由假设可得，对 $\mathcal{F}'_x = \{\langle \mathfrak{A}'_i, x_i \rangle : 0 < i \leq 2^n\}$ 和 $\mathcal{F}'_y = \{\langle \mathfrak{A}'_i, y_i \rangle : 0 < i \leq 2^n\}$ 我们有对应的语句 α'_x 和 α'_y 满足条件 i。则对 $\mathcal{F}$ ，有 $\alpha = (A \wedge \alpha'_x) \vee (\neg A \wedge \alpha'_y)$ 满足 i。 $\square$

推论 2.2.7.1 $\{\neg, \rightarrow\}$ 、 $\{\neg, \wedge\}$ 和 $\{\neg, \vee\}$ 都是功能完备的。

2.2.8 范式

最后一个在本节中将要介绍的重要的形式逻辑概念是范式。范式是指语言中符合某些特定语法规则并可用来代表其它语句的语句，一般范式有比其它语句更容易处理的性质。这里我们介绍析取范式和合取范式。

在命题逻辑中，称由任意语句符号 A 构成的式 A 和 $\neg A$ 为文字，对 $n > 0$ ，称由任意文字 $\alpha_1 \alpha_2 \cdots \alpha_n$ 构成的式 $\beta = \alpha_1 \wedge \alpha_2 \wedge \dots \wedge \alpha_n$ 为合取子式。对 $m > 0$ ，称由任意合取子式 $\beta_1 \beta_2 \cdots \beta_m$ 构成的语句 $\beta_1 \vee \beta_2 \vee \dots \vee \beta_m$ 为析取范式。如 $A, A \wedge \neg A, (A \wedge \neg A) \vee B, (A \wedge \neg A) \vee B \vee (\neg B \wedge C)$ 都是析取范式。

¹证明自 Emil Leon Post. “Introduction to a general theory of elementary propositions”. In: From Frege to Gödel. Ed. by Jean Van Heijenoort. Cambridge: Harvard University Press, 1967, pp. 264–283。

类似的，对 $n > 0$ ，称由任意文字 $\alpha_1 \alpha_2 \cdots \alpha_n$ 构成的式 $\beta = \alpha_1 \vee \alpha_2 \vee \cdots \vee \alpha_n$ 为析取子式。对 $m > 0$ ，称由任意析取子式 $\beta_1 \beta_2 \cdots \beta_m$ 构成的语句 $\beta_1 \wedge \beta_2 \wedge \cdots \wedge \beta_m$ 为合取范式。

接着，我们证明在命题逻辑中，析取范式和合取范式可以代表其它语句。或者严格的说，

定理 2.2.8 对任意命题逻辑中的语句 ρ ，存在析取范式 ϕ ，有 $\rho \equiv \phi$ 。

证明：对任意 ρ ，构造 ρ 中所含语句符号到 ρ 的真值表，对表中每一行 ρ 对应为真的 i ：

A_1	A_2	$\dots$	A_n	ρ
x_{i1}	x_{i2}	$\dots$	x_{in}	真

构造合取子式 $\beta_i = \bigwedge_{1 \leq j \leq n} (\text{若 } x_{ij} = \text{真}, A_j, \text{ 否则 } \neg A_j)$ 。则当存在 β_i 时 $\phi = \bigvee \beta_i$ 为符合要求的析取范式。否则可以令 $\phi = A_1 \wedge \neg A_1$ 。□

定理 2.2.9 对任意命题逻辑中的语句 ρ ，存在合取范式 φ ，有 $\rho \equiv \varphi$ 。

证明：对任意 ρ ，存在析取范式 ϕ 。同时有 $i: (\alpha \wedge \beta) \vee \gamma \equiv (\alpha \vee \gamma) \wedge (\beta \vee \gamma)$ 。重复应用 i 于 ϕ 可得符合要求的 φ 。□

由析取范式我们可以很容易的如确定其是否是可满足的（只要其中任一合取子式可满足）。而通过将语句的否定转化为析取范式再判断其是否可满足为我们提供了另一种判定恒真语句的方法，继而为有限场景提供了另一种（可靠且完备的）演绎规则¹。合取范式的优势会在后续章节看到。

至此，通过命题逻辑，我们介绍了形式逻辑中的大部分基本概念，包括形式上的语言、符号和演绎规则，内容上的解释和模型，分别对应的蕴含关系，互相之间的可靠性和完备性，以及公理、定理、理论和公理 / 理论的一致性完备性等。这其中的大部分概念都会和其它形式逻辑系统中被使用、类推和扩展。但在介绍新的形式逻辑之前，我们需要更好的理清已经多次涉及的可有效计算的确切含义。

2.3 可计算性

我们已经多次提到可有效计算这个概念，比如我们称存在可有效计算的方法由自然数对应到元素的集合为可枚举集合，存在可有效计算的方法确定元素是否在集合内的集合为可判定集合。又比如我们要求演绎规则为可有效计算的在语法蕴含成立时最终给出肯定结果的方法。同时，我们也提到了没有通用的可有效计算的解决方法的问题。比如给定一个可枚举集合，并没有通用的办法确定其大小。现在我们来精确的讨论可有效计算的概念和性质，即可计算性。

2.3.1 类比证明

上一节介绍的形式逻辑为其它形式系统的讨论提供了很好的参考对象。而计算本质上是另一种形式系统，这里将通过和形式逻辑，特别是其中的证明部分，作对比，引出关于计算的一些基本概念。

当我们想通过某种方法解决某个问题的时候，直觉上我们认为这种方法是能够解决这个问题的。当问题简单的时候，直觉往往是正确的，但当问题开始变得复杂起来，我们也会开始不那么确定所使用的

¹ 尽管从计算复杂度的角度来说，这种方法在最差情况下并不优于真值表。更多关于命题逻辑演绎规则计算复杂度的讨论会在后续章节给出。

方法是否总是能解决问题。又一次，我们遇到了和弗雷格一样的问题。当弗雷格和他的同行们想要给数学建立一个坚实的基础时，他们的目标中很大一部分就是为证明提供坚实的基础，让证明可以在不受直觉影响的框架下进行。类似的，计算的目标就是为解决问题提供一个不受直觉影响的框架。研究计算的学科，即计算理论不仅为机械性的解决问题提供了框架，还在相当可信的程度上指出了可解决问题的范围，以及解决各类问题所需要的资源，比如所需要的时间和空间。

类似的，第一步就是将解决问题的形式，也就是计算从内容中脱离出来。就如图灵所说¹：

“我们应该使用‘可计算方法’特指可以通过[[图灵或任何等价]]机械设备执行的方法，而使用‘可有效计算’指直觉上的不同于任何这些[[图灵或任何等价机械设备]]定义的概念。”

于是就有了一般被人们所接受的邱奇-图灵猜想，即任何可有效计算的方法都是可计算方法。这里的可有效计算并没有形式化的定义。

同样，我们有各种能力不一定相同的计算方法，或者更直观的说，有各种由不同类型的（理想化的）机械设备执行的方法，我们称其为计算模型。特别的，图灵在定义可计算方法时提到的图灵机就是一种计算模型，而邱奇-图灵猜想本质上是说任何（实际可行的）计算模型的计算能力最多和图灵（机）等价模型相等。同时，在一个计算模型上，我们同样需要形式语言描述计算的输入，输出和步骤。并将要解决的具体问题对应到输入和输出上，有时我们也称这个对应过程为编码和解码。称从输入到可能返回输出的步骤序列为方法²。对输入 a ，方法 F ，写作 $F(a)$ ，当该过程返回输出 b 时，写作 $b = F(a)$ 。最后这里我们称计算模型中具体的机械设备为计算设备。

当我们为某个或某类问题提供了方法之后，可以认为相当于提供了一套新的形式逻辑。即如对方法 F ，若 $b = F(a)$ ，可以认为 $a \vdash b$ ，而我们所要解决的问题要求 $a \models b$ 。于是对方法实现的论证即相当于形式逻辑中对演绎规则的可靠性和完备性论证。

尽管如此，计算和证明之间仍有不小的差别。比如形式上，方法对于一个输入只会产生至多一个输出；而在形式逻辑中，一个前提往往会有很多不同的推论。并且也许计算或计算中的一部分可以和形式逻辑中的某些概念完全对应起来³，但更实际的，计算和证明的侧重点也是显然不同的，比如计算往往是大量重复的，因此更注重效率。又比如我们考虑如何计算时采用的策略也往往完全不同于考虑如何证明。

现在我们来看图灵机。

2.3.2 图灵机

图灵机是一类符合特定特点的计算设备的统称，这里也称机械设备为机器，可以认为其中每一种机器可以用来解决某类特定的问题。如图 2.4 所示，一个具体的图灵机包括一根两端可任意扩展的纸带⁴，该纸带按固定间隔划分成多个单元格，每个单元格可填写一个符号，可选的符号由 s_i 表示（ i 属于由当前图灵机决定的 $\mathbb{N}$ 的有限子集，下同），如图中实现框表示纸带，其中包含符号 $s_1 s_0 s_1 s_1$ 。同时可以看到这里 $i \in \{0, 1\}$ 。新扩展的纸带单元格初始填写符号 s_0 ，如图中虚线框所示，也意味着我们至少需要符号 s_0 。

¹ 选译自 Alan Turing. Systems of logic based on ordinals. 1938，原文中两个可计算分别为 computable 和 calculable。

² 区别于集合论意义上的函数，同时书中主要使用无衬线字体表示计算方法相关的概念。

³ 一方面，从大的角度说计算和形式逻辑显然同在形式系统的框架内。另一方面，也有比如在后续章节会介绍的柯里-霍华德同构这样的特定计算模型和演绎规则之间的对应关系。不过目前在更多层面上更系统的对应关系还是未知的。

⁴ 一些材料中将纸带描述为无限长，尽管图灵机更多的是作为一种想象中的机器，但从图灵机实际可用的目标来说这是不精确的。具体的说，无限长的纸带在现实中是不存在的，而可任意扩展的纸带是可能的。这里面的一个重要区别在于任意大和无限的概念是不相同的，这个在其它地方也经常需要注意。比如有限主义数学允许任意大但不允许无限。又比如按文法构建语句时，我们允许任意多次的使用产生式规则，但不允许无限次的使用产生式规则（因此类原因我们没有无限长的语句）。

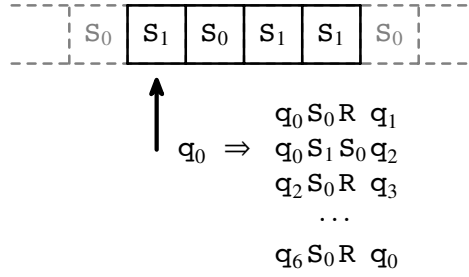


图 2.4: 图灵机示例

同时，机器包含一个指向单元格的指针，该指针可以读取和修改所指单元格中的符号，也可以左右移动。当指针移过纸带两端时，纸带自动进行扩展。指针如图中箭头所示。此外，机器记录一个当前状态，从可选状态 q_i 中选择，机器至少包含一个初始状态 q_0 。如图中箭头下标所示。

最后，机器包含有限条数的一组指令，每条指令为一个四元组 $q_i S_j A q_k$ ，表示当机器在状态 q_i 时若指针所指单元格中的符号为 S_j 则执行动作 A 并将状态置为 q_k 。其中 A 可以为任意符号，即代表将所指单元格中的符号改为 A 。或可以为 L 或 R ，分别表示指针左移或右移一格。如图中 $\Rightarrow$ 右侧为该示例图灵机的全部指令，其中第一条表示在状态 q_0 且读到的符号为 S_0 时，指针右移一格并跳转至状态 q_1 。可以看到示例中的当前状态、指针位置和纸带内容要求执行第二条指令，即修改符号为 S_0 并跳转至状态 q_2 。同时，显然的，要求指令由四元组中的前两位决定，即若指令 $q_{i_1} S_{j_1} A_1 q_{k_1}$ 、 $q_{i_2} S_{j_2} A_2 q_{k_2}$ 中 $i_1 = i_2$ 且 $j_1 = j_2$ 则 $A_1 = A_2$ 且 $k_1 = k_2$ ；即在任何情况下，总是只有至多一条指令满足执行条件。

我们使用图灵机解决一类问题。当我们需要解决该类问题中的一个具体问题时，我们使用输入描述这个具体问题的具体特点，或称参数。比如我们可以有一种图灵机专门解决自然数加法问题，则当我们想要解决 1 加 2 时，我们将 1 和 2 作为输入。我们将输入进行编码填写到纸带上特定于指针位置的多个单元格上，并从 q_0 状态开始运行图灵机，图灵机按照指令解释规则运行指令，即修改纸带内容，移动指针，扩展纸带，跳转状态。同时，显然运行中可能找不到满足当前机器状态的指令，比如当前状态为 q_1 而指令集中没有以 q_1 开始的指令。当出现这种情况时，机器停机，即返回，此时纸带上相对于指针位置的内容表示输出，即问题的答案。同样显然的，运行中可能总是能找到满足当前机器状态的指令，则机器永不停机，我们区分此状态作为另一种运行结果。然而，现实中，我们并不能通过这样的运行方式¹确定图灵机不会停机，也就是说我们并不能肯定出现了这种运行结果，所以我们也不能肯定问题以这种结果被解决了。而之所以我们仍区分这种状态，是因为对于一些问题，我们并没有其它采用完全可确定结果的解决方式。这一点，如章节 2.3.8 所见，在邱奇-图灵猜想下是可证明的。同样，我们也将看到这一点是本书中很多理论问题的极限所在。

接着，可以看到，图灵机的全部机器状态由纸带内容，指针位置和当前状态决定。我们可以将其写为 Mq_iN 。其中， M 代表指针左侧的纸带内容， N 代表指针所指及右侧的纸带内容，即 MN 共同构成了纸带。同时我们省略纸带两端全为 S_0 的部分以得到状态的规范形式²。 q_i 表示当前状态。比如我们可以将示例中的机器状态表示为 $q_0 S_1 S_0 S_1 S_1$ ，执行了一次指令后，我们有 $q_2 S_0 S_0 S_1 S_1$ 。我们称这种表示法为即时描述，特别的，我们称机器运行前的初始即时描述为输入描述，运行后的为输出描述。

现在来看输入输出的编解码问题。本质上，这里的编解码不是图灵机的一部分，是对图灵机所提供的（部分）形式化表示的解释，即和内容的对应关系。显然，任意图灵机在任意内容的纸带上都可以运行，因此，图灵机运行所对应的意义需要由用户保证。即在解决特定问题时，用户需要选择可解决该问

¹如章节 2.3.8 所述，事实上我们没有任何通用的方法确定图灵机是否会停机。

²即指当图灵机纸带两端（以符号 S_0 ）向外任意扩展时， MN 不变。

题的图灵机并将输入编码成所选图灵机约定的纸带内容。当然在输入和具体的图灵机间，约定可以有不同的方式。比如对于示例中的 $q_0 s_1 s_0 s_1 s_1$ ，可以认为代表四位二进制数 1011。或者表示由单个 s_0 分隔开的两个由 s_1 的个数决定的自然数，即 1 和 2。在第二种约定下， $q_0 s_0 s_1 s_1 s_1$ 表示输入 0 和 3，单个 q_0 表示 0 和 0（这里按要求省略了 s_0 ）。而 $q_0 s_1 s_0 s_0 s_1$ 则是没有实际意义的。

最后可以看到，图灵机由可选的符号，可能的状态以及包含的指令决定。同时，除 $s_0 q_0$ 外，其它运行中会涉及的符号和指令都可由包含的指令决定¹。因此我们可以简单的说指令集合决定了图灵机。

接着，我们需要确定图灵机确实能够解决一些问题，以自然数加法为例，并采用上述的第二种输入约定。我们有图灵机²：

$q_0 s_0 R q_1$	$q_3 s_1 R q_3$	$q_5 s_1 L q_5$
$q_0 s_1 s_0 q_2$	$q_3 s_0 R q_4$	$q_5 s_0 L q_6$
$q_2 s_0 R q_3$	$q_4 s_1 R q_4$	$q_6 s_1 L q_6$
	$q_4 s_0 s_1 q_5$	$q_6 s_0 R q_0$

直观的说，该图灵机将第一个参数中的 s_1 逐个剪切到第二个参数末尾。比如对于输入 1 和 2，最终有 $q_1 s_1 s_1 s_1$ ，以类似输入的按连续 s_1 的个数解码有输出 3。更具体的，图 2.5 给出了该图灵机的对应流程图。

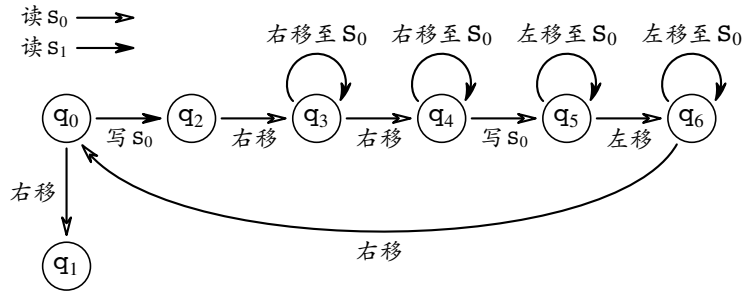


图 2.5: 自然数加法图灵机控制流程图

现在，我们证明含两个符号的图灵机和含三个符号的图灵机的计算能力是相同的。严格的说，我们称两个计算设备是等价的，若两个设备的输入之间、输出之间均有双射关系（即为严格一对一的），且对于对应的输入，两个设备要么同时返回，并结果也是符合对应关系的，要么同时不返回。当一个计算模型 $\mathcal{A}$ 中的所有计算设备都可以在另一个模型 $\mathcal{B}$ 中找到等价设备，则称 $\mathcal{A}$ 的计算能力小于等于 $\mathcal{B}$ 。当 $\mathcal{A}$ 的计算能力小于等于 $\mathcal{B}$ 且 $\mathcal{B}$ 的计算能力小于等于 $\mathcal{A}$ ，我们称 $\mathcal{A}$ 和 $\mathcal{B}$ 的计算能力是相同的，或 $\mathcal{A}$ 和 $\mathcal{B}$ 是等价的。特别的，我们称和图灵机等价的计算模型为图灵完备的。回到我们不同符号个数图灵机的例子，正向是显然的，这里证明反向。

示例 2.3.1 对任意含三个符号的图灵机 T_3 ，存在等价的含两个符号的图灵机 T_2 。

证明：直观的说，我们用 T_2 纸带上的两个单元格表示 T_3 中的一个单元格，并按照该对应方式将 T_3 中的指令转换成 T_2 中的指令。具体的，将 T_3 中 s_0 对应为 T_2 中的 $s_0 s_0$ ， s_1 对应为 $s_0 s_1$ ， s_2 对应为 $s_1 s_0$ 。比如，对于 T_3 中输入纸带，我们按此对应关系生成 T_2 中的纸带。同时，我们以相应的方式处理输出。对于 T_3 中的指令 $q_i s_j A q_k$ 。我们可以按如下方式对其进行转换，其中下标 $i, j = 2^{i+1}3^{j+1}$ ：

¹所有输入中包含但指令中不包含的符号的处理方式是统一的，即这样的符号不影响运行中指令的选择，当读到这样的符号时，图灵机停机。

²即图 2.4 所示图灵机的完整指令集合。

1. 若 j 等于 0, 则在 T_2 中添加 $q_{i,0} S_0 R q_{i,1}; q_{i,1} S_0 L q_{i,2}$ (读 S_0 、右移、读 S_0 、左移);
若等于 1, 则添加 $q_{i,0} S_0 R q_{i,1}; q_{i,1} S_1 L q_{i,2}$ (读 S_0 、右移、读 S_1 、左移);
否则, 添加 $q_{i,0} S_1 S_1 q_{i,2}$ (读 S_1)。
2. 若 A 为 L , 则添加 $q_{i,2} S_x L q_{i,3}; q_{i,3} S_x L q_{k,0}$ (左移、左移);
若为 R 添加 $q_{i,2} S_x R q_{i,3}; q_{i,3} S_x R q_{k,0}$ (右移、右移);
若为 S_0 添加 $q_{i,2} S_x S_0 q_{i,3}; q_{i,3} S_x R q_{i,4}; q_{i,4} S_x S_0 q_{i,5}; q_{i,5} S_0 L q_{k,0}$ (写 S_0 、右移、写 S_0 、左移);
若为 S_1 则添加 $q_{i,2} S_x S_0 q_{i,3}; q_{i,3} S_x R q_{i,4}; q_{i,4} S_x S_1 q_{i,5}; q_{i,5} S_1 L q_{k,0}$ (写 S_0 、右移、写 S_1 、左移);
否则, 添加 $q_{i,2} S_x S_1 q_{i,3}; q_{i,3} S_x R q_{i,4}; q_{i,4} S_x S_0 q_{i,5}; q_{i,5} S_0 L q_{k,0}$ (写 S_1 、右移、写 S_0 、左移)。其中都有 $x \in \{0, 1\}$, 即如 $q_u S_x A q_v$ 表示指令 $q_u S_0 A q_v; q_u S_1 A q_v$ 。

显然, 以此方式生成的 T_2 等价于 T_3 。

□

此为证明计算模型之间的等价性的一般方法。我们可以同样证明任意含两个以上符号的图灵机和仅含两个符号的图灵机计算能力相同; 也可以用此类方法证明向图灵机中增加任意数量纸带不增加计算能力。且在接下去的两小节中我们也将用此类方法证明图灵机和 μ -递归函数之间的等价性。

同时注意到这里 $x, y = 2^{x+1} \cdot 3^{y+1}$, 这样, $x_1, y_1 = x_2, y_2$ 当且仅当 $x_1 = x_2, y_1 = y_2$ 。更一般的, 我们可以扩展这个方式对任意复杂的自然数序列进行转换, 使得两个 (不一定等长的) 序列对应转换后的数字相等当且仅当这两个序列完全一致。具体的, 我们将转换函数表示为 gn , 则对 $n \geq 1$, 自然数 $x_1 \dots x_n$, $gn(x_1 \dots x_n) = 2^{x_1+1} \cdot 3^{x_2+1} \dots pr(n)^{x_n+1}$, 其中 $pr(n)$ 为第 n 个素数¹, 可以看到 gn 满足转换要求。更一般的, 通过将任意 (可枚举) 符号集——对应到自然数上, 我们可以对其上的任意符号序列进行类似转换²。如若 $L R S_0 S_1 q_0 q_1$ 分别对应 0 到 5, 则 $gn_s(q_0 S_0 L q_1) = 2^5 \cdot 3^3 \cdot 5^1 \cdot 7^6$ 。我们称此类序列到数的转换方式为哥德尔编码, 称转换后的数为被转换序列的哥德尔数。

最后, 由邱奇-图灵猜想有可由 (任意) 图灵机解决等于可有效计算。

2.3.3 结构化图灵机

尽管给出了自然数加法的例子, 但显然应用图灵机解决实际问题并不容易。这部分是因为图灵机提供的操作太过基础, 又没有框架性的指导, 而我们解决复杂问题的方式在于分解并逐个解决子问题。图灵机为我们提供了结构化的可能, 但并没有作出结构化的要求。这里, 我们讨论一部分满足特定结构的图灵机。作为示例, 我们给出第一部分要求:

1. 图灵机只使用符号 S_0 和 S_1 , 令 $q_0 = S_0, q_1 = S_1$ 。
2. 图灵机的输入输出均按照上述第二种约定编解码, 即由 q_0 分隔的多个 (长度可能为零) 的 q_1 序列, 每个序列的长度代表其所对应的自然数。

这样的讨论有两个目标, 一是提供一个更方便解决复杂问题的框架。更重要的是我们最终将证明这部分符合特定结构的图灵机的计算能力和没有结构要求的图灵机是一样的。同时, 在此过程中, 我们将引入更多的概念, 并在事实上得到另一种计算模型—— μ -递归函数, 以及一种可以用来运行任意图灵机的图灵机——通用图灵机。现在给出更多要求:

3. 图灵机至少要求一个输入。输入时, 图灵机的指针指向第一个 q_1 序列的左侧第一位。

¹ 由欧几里得定理有素数的个数是无限的。

² 可以通过统一添加如起始符号处理长度可能为零的符号序列集合。

4. 若图灵机产生输出，其只输出一个 $\perp$ 序列，即只对应一个自然数。指针同样指向该序列的左侧第一位，且该位置和输入时的初始指针位置相同。
5. 图灵机运行时不读取不修改初始指针位置左侧的内容，但要求初始位置左侧单元格全为 $\perp$ 。

例如，若输入 1, 0, 2, 输出 2, 对任意（不以 $\perp$ 开头的）符号序列 M ，输入描述为 $Mq_0 \perp \perp \perp \perp \perp$ ，输出如 $Mq_i \perp \perp \perp$ ；若输入 0, 1, 输出 0, 则 $Mq_0 \perp \perp \perp$ 到 Mq_i 。可以看出，我们的目标是在纸带上构建一个向右侧增长的栈，从而实现图灵机的组合。

写 $x_{i \dots j}$ 表示 $x_i, \dots, x_j$ 。同时，可以看到，这样的图灵机可以对应到自然数集上的多元部分函数。反过来说，对一个在自然数上的 n 元部分函数 f ，若存在图灵机 T ，对任意 $\langle x_{1 \dots n} \rangle \in \text{dom } f$ 且 $y = f(x_{1 \dots n})$ ，有 $y = T(x_{1 \dots n})$ ；对 $\langle x_{1 \dots n} \rangle \notin \text{dom } f$ 有 $T(x_{1 \dots n})$ 不返回，则我们称 f 是部分可计算的。当 f 为全函数时，称 f 是全可计算的（或简称可计算的）¹。需要注意的是类似全函数和部分函数之间的关系，全可计算函数集合是部分可计算函数集合的真子集。最后，这里以及后续类似的我们使用无衬线字体表示编码后的形式符号序列，即如这里 y 为纸带上 y 个 $\perp$ 组成的序列，又如 0 代表长度为零的 $\perp$ 序列。

在开始组合之前，我们需要实现一些具有基本功能的图灵机。具体的，我们需要：

- 后继 S ， $S(x) = x\perp$ ，即输入 x ，输出 $x + 1$ ^{注2}。
- 常量 C_c^n ，即对任意自然数 c ，正整数 n ， $C_c^n(x_{1 \dots n}) = c$ 。
- 投影 P_m^n ，即对任意正整数 n ， $m \leq n$ ， $P_m^n(x_{1 \dots n}) = x_m$ 。

为方便实现，令：

- $\text{movR}_{i,j}$ 表示指令 $q_i \xrightarrow{\perp} R q_{i,0}$ ； $q_{i,0} \xrightarrow{\perp} R q_{i,0}$ ； $q_{i,0} \xrightarrow{\perp} \perp q_j$ 。即指针移至当前位置右侧第一个 $\perp$ 处。其中 $\xrightarrow{\perp} \in \{\perp, \perp\}$ ，即如 $q_u \xrightarrow{\perp} A q_v$ 表示指令 $q_u \xrightarrow{\perp} A q_v$ ； $q_u \xrightarrow{\perp} A q_v$ ，下同。同时， i, j 分别对应入口和出口状态下标。且为方便嵌套， i, j 均为多元组，并对如 $i = \langle i_1, \dots, i_n \rangle$ ， $n \geq 1$ ，令状态下标 $i, k = gn(i_1 + 1, \dots, i_n + 1, k + 1)$ 。
- $\text{movL}_{i,j}$ 表示指令 $q_i \xrightarrow{\perp} L q_{i,0}$ ； $q_{i,0} \xrightarrow{\perp} L q_{i,0}$ ； $q_{i,0} \xrightarrow{\perp} \perp q_j$ 。即指针移至当前位置左侧第一个 $\perp$ 处。

如此，后继图灵机 S 即可通过指令集合 $\text{movR}_{0,1}$ ； $q_1 \xrightarrow{\perp} \perp q_2$ ； $\text{movL}_{2,3}$ （移至右侧第一个 $\perp$ ，写 $\perp$ ，移回）实现。注意像 $\text{movR}_{0,1} \text{movL}_{2,3}$ 这样的缩写所对应的（多条）指令在运行前就是完全确定的，运行时的图灵机仍是严格意义上的由四元组指令集合构成的图灵机³。我们称这类缩写为辅助方法。现在，我们定义后继图灵机 S 对应的辅助方法，令：

- $\text{nop}_{i,j}$ 表示指令 $q_i \xrightarrow{\perp} \perp q_j$ 。 enter_i 表示 $\text{nop}_{i,\langle i,0 \rangle}$ 。 $\text{leave}_{i,k,j}$ 表示 $\text{nop}_{\langle i,k \rangle,j}$ 。
- $S_{i,j}$ 表示指令 enter_i ； $\text{movR}_{\langle i,0 \rangle,\langle i,1 \rangle}$ ； $q_{i,1} \xrightarrow{\perp} \perp q_{i,2}$ ； $\text{movL}_{\langle i,2 \rangle,\langle i,3 \rangle}$ ； $\text{leave}_{i,3,j}$ 。

可以看到图灵机 S 等价于 $S_{0,1}$ 。而图灵机 $S_{0,1}$ ； $S_{1,2}$ 对输入 x 输出 $x\perp\perp$ 。接着，我们定义更多辅助方法，令：

- $\text{repeat}(code_k)_k^n$ 表示指令 $code_0$ ； $code_1$ ； $\dots$ ； $code_{n-1}$ ，其中如 $code_0$ 表示用 0 替换 $code$ 中（由 repeat 下标指定）的 k 。可以看到，当 $n = 0$ 时，对应为空。
- $\text{movR}_{i,j}^n$ 表示指令 enter_i ； $\text{repeat}(\text{movR}_{\langle i,k \rangle,\langle i,k+1 \rangle})_k^n$ ； $\text{leave}_{i,n,j}$ 。即移至右侧第 n 个 $\perp$ （ $n = 0$ 时不移动）。

¹尽管这里我们很多时候使用全可计算和后面的全递归以明确含义，但注意在很多类似相关术语命名中会省略全。

²如无特别说明，后续 S 均代表后继或相关概念。

³类似元编程，如 C 语言中的宏扩展，因此也可以认为这里的扩展是静态的。

- 以类似的方式定义 $\text{movL}_{i,j}^n$ 表示移至左侧第 n 个 $\perp$ 。
- $\text{resetR}_{i,j}$ 表示指令 $\mathbf{q}_i \bowtie \mathbf{R} \mathbf{q}_{i,0}; \mathbf{q}_{i,0} \perp \perp \mathbf{q}_{i,1}; \mathbf{q}_{i,1} \perp \mathbf{R} \mathbf{q}_{i,0}; \text{leave}_{i,0,j}$ 。即如令当前位置右侧第一个 $\perp$ 所在单元格为 p ，将当前指针位置右侧到 p 之间的单元格全写为 $\perp$ ，最终指针指向 p 。
- $\text{resetR}_{i,j}^n$ 表示指令 $\text{enter}_i; \text{repeat}(\text{resetR}_{\langle i,k \rangle, \langle i,k+1 \rangle}^n)_k^n; \text{leave}_{i,n,j}$ 。即如令当前位置右侧第 n 个 $\perp$ 所在单元格为 p ，将当前指针位置右侧到 p 之间的单元格全写为 $\perp$ ，最终指针指向 p 。
- 以类似方式定义 $\text{resetL}_{i,j}$ 、 $\text{resetL}_{i,j}^n$ 。

如此，我们定义常量图灵机对应的辅助方法，令：

- $\mathbf{C}_{c,i,j}^n$ 表示指令 $\text{enter}_i; \text{movR}_{\langle i,0 \rangle, \langle i,1 \rangle}^n; \text{resetL}_{\langle i,1 \rangle, \langle i,2 \rangle}^n; \text{repeat}(\mathbf{S}_{i,2+k})_k^c; \text{leave}_{i,2+c,j}$ 。即右移至末尾，向左写 $\perp$ 至初始指针位置，执行 c 次后继。

可以看到，常量图灵机 $\mathbf{C}_c^n$ 可以通过 $\mathbf{C}_{c,0,1}^n$ 实现。为实现投影，我们还需要：

- $\text{shiftL}_{i,j}$ 表示指令 $\mathbf{q}_i \perp \perp \mathbf{q}_{i,0}; \mathbf{q}_{i,0} \perp \perp \mathbf{q}_{i,1}; \mathbf{q}_{i,1} \perp \perp \mathbf{q}_j$ ，即对如 $\mathbf{M} \perp \mathbf{q}_0 \perp \mathbf{N}$ 执行 $\text{shiftL}_{0,1}$ 后有 $\mathbf{M} \mathbf{q}_1 \perp \perp \mathbf{N}$ ，可以看到指针所指的 $\perp$ 左移了一格。
- $\text{backL}_{i,j}$ 表示指令 $\mathbf{q}_i \perp \perp \mathbf{q}_{i,0}; \mathbf{q}_{i,0} \perp \perp \mathbf{q}_j$ ，即（从当前 $\perp$ 位置）左移并写 $\perp$ 。
- $\text{cut}_{i,j}^n$ 将当前位置右侧前 n 个输入对应为 $\perp$ 单元格清 $\perp$ ，并将第 $n+1$ 个输入剪切至当前且位置右侧，并最终将指针指回当前位置。同时 $\text{cut}_{i,j}^n$ 要求运行前指针所指单元格为 $\perp$ 。例如对输入 $\mathbf{q}_0 \perp \perp \perp \perp \perp \perp \perp \perp$ 经 $\text{cut}_{0,1}^2$ 后为 $\mathbf{q}_1 \perp \perp \perp \perp \perp \perp \perp \perp$ ；又如对输入 $\mathbf{q}_0 \perp \perp \perp \perp \perp \perp$ 经 $\text{cut}_{0,1}^1$ 后为 $\mathbf{q}_1 \perp \perp \perp \perp \perp \perp$ ¹。具体的，当 $n \neq 0$ 时， $\text{cut}_{i,j}^n$ 表示指令：

$$\text{enter}_i; \text{movR}_{\langle i,0 \rangle, \langle i,1 \rangle}^n; \text{repeat}(\text{movL}_{\langle i,1+2k \rangle, \langle i,2+2k \rangle}; \mathbf{q}_{i,2+2k} \perp \perp \mathbf{q}_{i,3+2k})_k^{n-1}; \text{movL}_{\langle i,2n-1 \rangle, \langle i,2n \rangle}; \quad (\text{i})$$

$$\mathbf{q}_{i,2n} \perp \mathbf{R} \mathbf{q}_{i,2n+1}; \mathbf{q}_{i,2n+1} \perp \perp \mathbf{q}_{i,2n+2}; \text{movR}_{\langle i,2n+2 \rangle, \langle i,2n+3 \rangle}; \text{shiftL}_{\langle i,2n+3 \rangle, \langle i,2n+4 \rangle}; \quad (\text{ii})$$

$$\text{movR}_{\langle i,2n+4 \rangle, \langle i,2n+5 \rangle}; \text{backL}_{\langle i,2n+5 \rangle, \langle i,2n+6 \rangle}; \text{movL}_{\langle i,2n+6 \rangle, \langle i,2n \rangle}^2;$$

$$\mathbf{q}_{i,2n+1} \perp \mathbf{R} \mathbf{q}_{i,2n+7}; \mathbf{q}_{i,2n+7} \perp \perp \mathbf{q}_{i,2n+8}; \text{movR}_{\langle i,2n+8 \rangle, \langle i,2n+9 \rangle}; \text{backL}_{\langle i,2n+9 \rangle, \langle i,2n+10 \rangle}; \quad (\text{iii})$$

$$\text{movL}_{\langle i,2n+10 \rangle, \langle i,2n+11 \rangle}; \mathbf{q}_{i,2n+11} \perp \perp \mathbf{q}_{i,2n+12}; \mathbf{q}_{i,2n+12} \perp \perp \mathbf{q}_j; \quad (\text{iv})$$

$$\mathbf{q}_{i,2n+7} \perp \perp \mathbf{q}_{i,2n+13}; \mathbf{q}_{i,2n+13} \perp \perp \mathbf{q}_j$$

其中 (i) 将当前指针右侧 $n-1$ 个 $\perp$ 写为 $\perp$ ，执行后如 $\mathbf{M} \mathbf{q}_{i,2n} \perp \mathbf{U} \perp \mathbf{V} \perp \mathbf{N}$ ，其中 $\mathbf{U}$ 对应前 n 个输入在其间分隔符 $\perp$ 被写为 $\perp$ 后的 $\perp$ 序列， $\mathbf{V}$ 对应第 $n+1$ 个输入， $\mathbf{U} \mathbf{V}$ 长度均可能为零。(ii) 处理 $\mathbf{U}$ 长度不为零的情况，为一个循环，一次执行后 $\mathbf{U}$ 长度减 1，循环结束后有 $\mathbf{M} \mathbf{q}_{i,2n} \perp \perp \mathbf{V} \perp \mathbf{N}$ 。(iii) 开始处理 $\mathbf{U}$ 长度为零，即循环结束后的情况，分别处理 $\mathbf{V}$ 长度不为零，和 (iv) 中 $\mathbf{V}$ 长度为零的情况。最后，当 $n=0$ 时， $\text{cut}_{i,j}^n$ 对应 $\text{nop}_{i,j}$ 。

如此，我们定义投影图灵机对应的辅助方法，令：

- $\mathbf{P}_{m,i,j}^n$ 表示指令 $\text{enter}_i; \text{movR}_{\langle i,0 \rangle, \langle i,1 \rangle}^m; \text{resetL}_{\langle i,1 \rangle, \langle i,2 \rangle}^{n-m}; \text{movL}_{\langle i,2 \rangle, \langle i,3 \rangle}^m; \text{cut}_{\langle i,3 \rangle, j}^{m-1}$ 。

可以看到，投影图灵机 $\mathbf{P}_m^n$ 可以通过 $\mathbf{P}_{m,0,1}^n$ 实现。

同时可以看到后继、常量和投影图灵机是符合前文提出的结构化要求的。进一步，我们通过定义结构化图灵机给出结构化的完整要求：

1. 后继、常量和投影图灵机为结构化图灵机。

¹事实上，在这里的应用中不会出现初始指针位置右侧有超过 $n+1$ 个输入的情况。

2. 以复合、原始递归和极小化方式组合结构化图灵机而成的图灵机亦为结构化图灵机。

具体的，这里的三种组合方式和结果分别要求，其中我们称要求 n 个输入的结构化图灵机为 n 元图灵机：

- 复合 $CN_m^n [H, G_1, \dots, G_m]$ ，表示对任意正整数 m, n ，由任意 m 元图灵机 H ， n 元图灵机 G_i ($1 \leq i \leq m$) 组合而成的 n 元图灵机。令 $F = CN_m^n [H, G_1, \dots, G_m]$ ，要求对任意 x_1 到 x_n ：

$$F(x_1 \dots x_n) = H(G_1(x_1 \dots x_n), \dots, G_m(x_1 \dots x_n))$$

- 原始递归 $PR^n [G, H]$ ，表示对任意 $n > 1$ ，由任意 $n-1$ 元图灵机 G ， $n+1$ 元图灵机 H 组合而成的 n 元图灵机。令 $F = PR^n [G, H]$ ，要求对任意 x_2 到 x_n ， y ：

$$F(0, x_2 \dots x_n) = G(x_2 \dots x_n)$$

$$F(S(y), x_2 \dots x_n) = H(y, F(y, x_2 \dots x_n), x_2 \dots x_n)$$

- 极小化 $MN^n [G]$ ，表示对任意正整数 n ，基于任意 $n+1$ 元图灵机 G 的 n 元图灵机。直观的说，极小化返回从零开始第一个满足由 G 指定要求的输出。具体的，令 $F = MN^n [G]$ ，要求对任意 x_1 到 x_n ， y, z ：

$$F(x_1 \dots x_n) = z \text{ 当前仅当 对任意 } 0 \leq y < z, G(y, x_1 \dots x_n) \neq 0$$

$$\text{且 } G(z, x_1 \dots x_n) = 0$$

我们将在下一小节给出这些组合的应用。现在我们先讨论结构化图灵机的返回情况，也就是其对应的部分可计算函数是否是全可计算的。显然，后继、常量和投影图灵机是一定返回输出的。复合图灵机和原始递归图灵机返回当且仅当组成它们的图灵机在运行时均返回。而对于极小化，当不存在满足要求的输出时，其也不会返回，即我们只能有（显然的）若极小化图灵机返回，组成它的图灵机在运行时返回。因此，结构化图灵机不返回的唯一最终来源是极小化组合方式。

接着我们考虑实现组合。首先要求所有组合而成的图灵机有对应的辅助方法¹，则所有结构化图灵机均有对应的辅助方法。对任意结构化图灵机 T ，其辅助方法写为 $T_{i,j}$ 。同时，该要求也意味着要实现组合而成的图灵机，我们只需给出对应的辅助方法。不过在此之前，我们仍需要：

- $\text{dup}_{i,j}^n$ 将位于当前位置右侧的第一个输入复制为第 $n+1$ 个输入，并将指针指回当前位置。要求当前位置为 0 ， $n \geq 1$ ，以及当前位置右侧仅有 n 个输入。即如 $q_0 0 1 1 0 1$ 经 $\text{dup}_{0,1}^2$ 后有 $q_1 0 1 1 0 1 0 1 1$ ，又如 $q_0 0 1$ 经 $\text{dup}_{0,1}^3$ 后有 $q_1 0 1 0 0 0 1$ 。具体的， $\text{dup}_{i,j}^n$ 表示指令：

$$\text{enter}_i; q_{i,0} \rightarrow R q_{i,1}; q_{i,1} \rightarrow 0 q_{i,2}; \text{movR}_{\langle i,2 \rangle, \langle i,3 \rangle}^{n+1}; q_{i,3} \rightarrow 0 q_{i,4}; \text{movL}_{\langle i,4 \rangle, \langle i,5 \rangle}^{n+1}; q_{i,5} \rightarrow 0 q_{i,0}; \quad (i)$$

$$q_{i,1} \rightarrow 0 q_{i,6}; \text{movL}_{\langle i,6 \rangle, j} \quad (ii)$$

其中 (i) 逐一循环复制 1 ，具体通过将被复制的 1 置为 0 作为标记实现。循环结束后，执行 (ii) 移动指针回初始位置。

- $\text{dup}_{i,j}^{m,n}$ 将当前位置右侧 m 个输入复制到右侧第 n 个输入后，并将指针指回当前位置。要求当前位置为 0 ， $n \geq m \geq 1$ ，以及当前位置右侧仅有 n 个输入。具体的，表示：

$$\text{enter}_i; \text{repeat} (\text{dup}_{\langle i,2k \rangle, \langle i,2k+1 \rangle}^n; \text{movR}_{\langle i,2k+1 \rangle, \langle i,2k+2 \rangle}^m)_k^m; \text{movL}_{\langle i,2m \rangle, j}^m$$

¹事实上，图灵机对应的辅助方法只是为了方便状态下标的组合进而方便嵌套，通过参数化下标和增加入口出口处的跳转指令，任意图灵机都可以构造出对应的辅助方法。但尽管如此，这里我们着重于直接通过组合下层辅助方法生成新图灵机对应的辅助方法。

现在，我们通过实现对应的辅助方法实现结构化图灵机组合：

- $CN_m^n [H, G_1, \dots, G_m]_{i,j}$ 表示：

$$\text{enter}_i; \text{repeat} (\text{dup}_{\langle i,4k \rangle, \langle i,4k+1 \rangle}^{n,n+k}; \text{movR}_{\langle i,4k+1 \rangle, \langle i,4k+2 \rangle}^{n+k}; G_{k+1} \langle i,4k+2 \rangle, \langle i,4k+3 \rangle; \text{movL}_{\langle i,4k+3 \rangle, \langle i,4k+4 \rangle}^{n+k})_k^m; \quad (i)$$

$$\text{movR}_{\langle i,m \rangle, \langle i,m+1 \rangle}^n; H_{\langle i,m+1 \rangle, \langle i,m+2 \rangle}; \text{movL}_{\langle i,m+2 \rangle, \langle i,m+3 \rangle}^n; \text{cut}_{\langle i,m+3 \rangle, j}^n \quad (ii)$$

其中 (i) 循环执行 G_i ($1 \leq i \leq m$)，(ii) 执行 H 。

- $PR^n [G, H]_{i,j}$ 表示：

$$\begin{aligned} &\text{enter}_i; \text{dup}_{\langle i,0 \rangle, \langle i,1 \rangle}^{n,n+1}; \text{movR}_{\langle i,1 \rangle, \langle i,2 \rangle}^{n+1}; q_{i,2} \sqcup R q_{i,3}; \\ &q_{i,3} \sqcup L q_{i,4}; \text{movR}_{\langle i,4 \rangle, \langle i,5 \rangle}^n; S_{\langle i,5 \rangle, \langle i,6 \rangle}; \text{movL}_{\langle i,6 \rangle, \langle i,7 \rangle}^n; \text{dup}_{\langle i,7 \rangle, \langle i,8 \rangle}^{n+1}; \text{movR}_{\langle i,8 \rangle, \langle i,9 \rangle}^{n+2}; \\ &\text{backL}_{\langle i,9 \rangle, \langle i,10 \rangle}; \text{movL}_{\langle i,10 \rangle, \langle i,11 \rangle}^{n+1}; \text{dup}_{\langle i,11 \rangle, \langle i,12 \rangle}^{n-1,n}; \text{movR}_{\langle i,12 \rangle, \langle i,2 \rangle}^{n-1}; \end{aligned} \quad (i)$$

$$\begin{aligned} &q_{i,3} \sqcup \sqcup q_{i,13}; G_{\langle i,13 \rangle, \langle i,14 \rangle}; q_{i,14} \sqcup L q_{i,15}; \text{cut}_{\langle i,15 \rangle, \langle i,16 \rangle}^1; q_{i,16} \sqcup L q_{i,17}; \\ &q_{i,17} \sqcup L q_{i,18}; \text{movL}_{\langle i,18 \rangle, \langle i,19 \rangle}^n; \text{dup}_{\langle i,19 \rangle, \langle i,20 \rangle}^{n+2}; \text{movR}_{\langle i,20 \rangle, \langle i,21 \rangle}^{n+3}; \text{backL}_{\langle i,21 \rangle, \langle i,22 \rangle}; \\ &\text{movL}_{\langle i,22 \rangle, \langle i,23 \rangle}; \text{dup}_{\langle i,23 \rangle, \langle i,24 \rangle}^2; \text{movL}_{\langle i,24 \rangle, \langle i,25 \rangle}^n; \text{dup}_{\langle i,25 \rangle, \langle i,26 \rangle}^{n-1,n+3}; \text{movR}_{\langle i,26 \rangle, \langle i,27 \rangle}^{n+1}; \\ &H_{\langle i,27 \rangle, \langle i,28 \rangle}; \text{movL}_{\langle i,28 \rangle, \langle i,29 \rangle}^{n+2}; \text{cut}_{\langle i,29 \rangle, \langle i,16 \rangle}^{n+2}; \end{aligned} \quad (ii)$$

$$q_{i,17} \sqcup L q_{i,30}; \text{movL}_{\langle i,30 \rangle, \langle i,31 \rangle}^n; \text{cut}_{\langle i,31 \rangle, j}^{n+1} \quad (iii)$$

为理清这组复杂指令的运行过程，我们引入一个更方便的状态描述方法。这里我们视纸带为由 $\sqcup$ 分隔的代表自然数的 $\sqcup$ 序列，因此我们直接以对应自然数表示其中的 $\sqcup$ 序列，并用 $\circ$ 或 $\circ$ 分隔。同时，我们写 $\rangle_k$ 表示 $q_{i,k}$ ，则如 $\sqcup \sqcup \sqcup \sqcup q_{i,4} \sqcup \sqcup \sqcup \sqcup \sqcup$ 可表示为 $2, 0, \rangle_4 1, 3$ 。这样，输入 $x_1 \dots x_n$ ，初始状态为 $\rangle_0 x_1 \dots x_n$ 。然后：

1. 我们通过 (i) 循环建立起由 x_1 递减到 0 的递归栈；即在跳出 (i) 时，我们有状态 $x_1 \dots x_n; \bullet, x_1, x_2 \dots x_n; \circ, x_1 - 1, x_2 \dots x_n; \circ, x_1 - 2, x_2 \dots x_n; \dots; \circ, 2, x_2 \dots x_n; \circ, 1, x_2 \dots x_n; \circ, 0, \rangle_3 x_2 \dots x_n$ ；其中 $\bullet = 0$ ，标记栈顶， $\circ = 1$ ，为对应占位符。
2. 在 (ii) 中，我们首先执行 G ，接着对 0 到 $x_1 - 1$ 循环逐一执行 H 。每次 H 执行前有状态 $x_1 \dots x_n; \bullet, x_1, x_2 \dots x_n; \circ, \dots, y + 1, x_2 \dots x_n; \circ, \rangle_{27} y, z, x_2 \dots x_n$ ，其中 $y < x_1$ ， $z = PR^n [G, H] (y, x_2 \dots x_n)$ ，即内层递归的返回结果。并在执行后将结果剪切至上层栈分隔符右侧。
3. 最后，通过 (iii) 将最终结果剪切至初始指针位置右侧。

- $MN^n [G]_{i,j}$ 表示：

$$\begin{aligned} &\text{enter}_i; \text{movR}_{\langle i,0 \rangle, \langle i,1 \rangle}^n; \text{dup}_{\langle i,1 \rangle, \langle i,2 \rangle}^1; \text{movL}_{\langle i,2 \rangle, \langle i,3 \rangle}^n; \text{dup}_{\langle i,3 \rangle, \langle i,4 \rangle}^{n,n+2}; \text{movR}_{\langle i,4 \rangle, \langle i,5 \rangle}^{n+1}; G_{\langle i,5 \rangle, \langle i,6 \rangle}; \\ &q_{i,6} \sqcup R q_{i,7}; q_{i,7} \sqcup \sqcup q_{i,8}; \text{movR}_{\langle i,8 \rangle, \langle i,9 \rangle}; \text{resetL}_{\langle i,9 \rangle, \langle i,10 \rangle}; \text{movL}_{\langle i,10 \rangle, \langle i,11 \rangle}; S_{\langle i,11 \rangle, \langle i,1 \rangle}; \end{aligned} \quad (i)$$

$$q_{i,7} \sqcup \sqcup q_{i,12}; \text{movL}_{\langle i,12 \rangle, \langle i,13 \rangle}^{n+2}; \text{cut}_{\langle i,13 \rangle, j}^n \quad (ii)$$

其中 (i) 循环递增执行 G 并检查结果是否为 0。(ii) 剪切。

至此，我们完成了全部结构化图灵机的定义。通过定义的过程，我们已经看到不少图灵机实现特定功能的例子。更重要的，我们定义的结构化图灵机和另一种计算模型， μ -递归函数有严格的对应关系。这种对应关系使我们有：

引理 2.3.2

a. 计算模型 μ -递归函数和结构化图灵机是等价的。

b. 任意 μ -递归函数下的计算设备¹均存在等价的图灵机。

这样，我们可以通过讨论 μ -递归函数的计算能力进一步考察图灵机的计算能力。

2.3.4 μ -递归函数²

类似图灵机， μ -递归函数即是计算模型的名称，也是该计算模型下具体计算设备的名称。同时，如上所述， μ -递归函数的定义和结构化图灵机有严格的对应关系。任意 μ -递归函数（计算设备）接受固定非零数量的自然数作为输入，可能不返回，当返回时，返回一个自然数作为输出。对接受 n 个输入的 μ -递归函数，我们称其为 n 元 μ -递归函数。在 μ -递归函数中，我们使用符号 0 对应自然数 0，并有基础函数：

- 后继 S ，一元 μ -递归函数。若 x 对应自然数 x ， $S(x)$ 对应 $x+1$ 。
- 常量 C_c^n ， n 元 μ -递归函数。即对任意自然数 c ，正整数 n ， $C_c^n(x_1 \dots x_n) = \overbrace{S \dots S}^c 0$ 。其中 $S \dots = S(\dots)$ ，即 $\overbrace{S \dots S}^c 0$ 对应自然数 c 。
- 投影 P_m^n ， n 元 μ -递归函数。即对任意正整数 n ，正整数 $m \leq n$ ， $P_m^n(x_1 \dots x_n) = x_m$ 。

显然，这里和结构化图灵机中相关概念的唯一区别在于如何编码自然数。而 μ -递归函数的组合方式和结构化图灵机更是完全一致的，重述如下：

- 复合 $CN_m^n[H, G_1, \dots, G_m]$ ，表示对任意正整数 m, n ，由任意 m 元 μ -递归函数 H ， n 元 μ -递归函数 G_i ($1 \leq i \leq m$) 组合而成的 n 元 μ -递归函数。令 $F = CN_m^n[H, G_1, \dots, G_m]$ ，要求对任意 x_1 到 x_n ：

$$F(x_1 \dots x_n) = H(G_1(x_1 \dots x_n), \dots, G_m(x_1 \dots x_n))$$

- 原始递归 $PR^n[G, H]$ ，表示对任意 $n \geq 2$ ，由任意 $n-1$ 元 μ -递归函数 G ， $n+1$ 元 μ -递归函数 H 组合而成的 n 元 μ -递归函数。令 $F = PR^n[G, H]$ ，要求对任意 x_2 到 x_n ， y ：

$$F(0, x_2 \dots x_n) = G(x_2 \dots x_n)$$

$$F(S(y), x_2 \dots x_n) = H(y, F(y, x_2 \dots x_n), x_2 \dots x_n)$$

- 极小化 $MN^n[G]$ ，表示对任意正整数 n ，基于任意 $n+1$ 元 μ -递归函数 G 的 n 元 μ -递归函数。令 $F = MN^n[G]$ ，要求对任意 x_1 到 x_n ， y ， z ：

$$F(x_1 \dots x_n) = z \text{ 当前仅当 } \text{对任意 } 0 \leq y < z, G(y, x_1 \dots x_n) \neq 0$$

$$\text{且 } G(z, x_1 \dots x_n) = 0$$

其中，在 μ -递归函数中，我们称 CN_m^n 、 PR^n 和 MN^n 为算子，特别的，我们又称极小化算子为 μ -算子。同时，为方便讨论，类似结构化图灵机，对任意自然数 n ，我们用无衬线字体 n 表示 $\overbrace{S \dots S}^n 0$ 。

同样类似结构化图灵机，每个 μ -递归函数对应一个自然数集上的多元部分函数。同样反过来，对一个在自然数集上的 n 元部分函数，若存在 μ -递归函数 F 满足 i ：对任意 $\langle x_1 \dots x_n \rangle \in \text{dom } f$ 且 $y = f(x_1 \dots x_n)$ ，有 $y = F(x_1 \dots x_n)$ ；对 $\langle x_1 \dots x_n \rangle \notin \text{dom } f$ 有 $F(x_1 \dots x_n)$ 不返回，则我们称 f 是部分递归的。当 f 为全函数时，称 f 是全递归的（或简称递归的）。特别的，由于不返回的唯一最终来源是极小化算子，对 f ，若存在构成中

¹更严格的定义见下一小节。

² μ -递归函数也称一般递归函数，不过本书中总是使用 μ -递归函数这一更具辨识度的名称。

不含极小化算子的 μ -递归函数 F 满足 i, 则称 f 是原始递归的, 也称 F 是原始递归的。显然, 此时 f 是全函数, 即 f 也是全递归的。但是, 反之是不成立的, 即存在不是原始递归的全递归函数, 我们将在章节 2.3.7 给出例子。

接着, 我们讨论 μ -递归函数的计算 (运行) 方法。我们称给定某 μ -递归函数的输入尝试得到输出的计算过程为对该 μ -递归函数的调用。计算中, 我们对 μ -递归函数调用的构成形式进行匹配, 并按定义重写, 直至最终写成 $\overbrace{S \dots S}^n 0$ 的形式, 即 n , 对应输出自然数 n 。这样, 对如实现自然数加法计算的 μ -递归函数:

$$+ = PR^2 [P_1^1, CN_1^3 [S, P_2^3]]$$

其计算如 $+(1, 3)$ 的过程如下:

$$\begin{aligned} +(1, 3) &= PR^2 [P_1^1, CN_1^3 [S, P_2^3]] (1, 3) \\ &= CN_1^3 [S, P_2^3] (0, PR^2 [P_1^1, CN_1^3 [S, P_2^3]] (0, 3), 3) \\ &= CN_1^3 [S, P_2^3] (0, P_1^1 (3), 3) = CN_1^3 [S, P_2^3] (0, 3, 3) \\ &= S (P_2^3 (0, 3, 3)) = S (3) \\ &= 4 \end{aligned}$$

而对于极小化, 在重写 $MN^n [G] (x_1 \dots x_n)$ 时, 我们从 $z = 0$ 起依次计算 $G(z, x_1 \dots x_n)$ 直至找到满足极小化定义的重写结果 z 。

现在, 让我们看看 μ -递归函数还能执行哪些计算, 同时也提供一个等价且更方便的实现描述方式:

1. 直观的说, 如对于一元 μ -递归函数 G , 我们可通过 $F = CN_1^2 [G, P_1^2]$ 对任意 x_1, x_2 实现如 $F(x_1, x_2) = G(x_1)$ 。具体的, 对 m 元 μ -递归函数 G , 对任意 $1 \leq i \leq m$, $t_i \geq 1$, 同时对 n 满足 $n \geq t_i$ ($1 \leq i \leq m$), 通过:

$$F = CN_m^n [G, P_{t_1}^n, \dots, P_{t_m}^n]$$

可实现, 对任意 x_1 到 x_n :

$$F(x_1 \dots x_n) = G(x_{t_1} \dots x_{t_m})$$

注意这里 n 不一定大于或等于 m 。同时这意味着当我们写 $G(x_{t_1} \dots x_{t_m})$, 存在满足上式的 F 和 n 可用于替换这部分内容, 即 $G(x_{t_1} \dots x_{t_m})$ 。

2. 这样, 对任意 $m \geq 1$, 对 m 元 μ -递归函数 H , 并对任意 $1 \leq i \leq m$, 任意 $n_i \geq 1$, G_i 为 n_i 元 μ -递归函数, 对任意 $1 \leq j \leq n_i$, 任意 $t_{i,j} \geq 1$, 同时对任意 n 满足 $n \geq t_{i,j}$ ($1 \leq i \leq m$ 且 $1 \leq j \leq n_i$), 对任意 x_1 到 x_n 写:

$$H(G_1(x_{t_{1,1}} \dots x_{t_{1,n_1}}), \dots, G_m(x_{t_{m,1}} \dots x_{t_{m,n_m}}))$$

意味着对任意 $1 \leq i \leq m$, 存在 n 元 μ -递归函数 G'_i , 有:

$$H(G'_1(x_1 \dots x_n), \dots, G'_m(x_1 \dots x_n)) = H(G_1(x_{t_{1,1}} \dots x_{t_{1,n_1}}), \dots, G_m(x_{t_{m,1}} \dots x_{t_{m,n_m}}))$$

即存在 n 元 μ -递归函数 $CN_m^n [H, G'_1, \dots, G'_m]$:

$$CN_m^n [H, G'_1, \dots, G'_m] (x_1 \dots x_n) = H(G_1(x_{t_{1,1}} \dots x_{t_{1,n_1}}), \dots, G_m(x_{t_{m,1}} \dots x_{t_{m,n_m}}))$$

类似的, 当我们写上式的右侧时, 可以用左侧替换, 则我们可以递归的替换嵌套调用, 即存在 μ 递归函数满足 (即实现) 含嵌套的右侧部分。或者说当 (可能含有嵌套的) 调用 A 含输入 $x \in \{x_1, \dots, x_n\}$ 时, 可实现 n 元 μ -递归函数 F 满足 $F(x_1 \dots x_n) = A$ 。如可实现 F 满足 $F(x_1, x_2, x_3) = +(+(x_1, +(x_1, +(x_2, x_3))), x_3)$ 。

3. 在讨论原始递归之前，定义在自然数上的前驱函数 $\dot{-}(x) = \begin{cases} x-1 & \text{若 } x \neq 0 \\ 0 & \text{若 } x = 0 \end{cases}$ 。可实现 μ -递归函数：

$$\dot{-} = \text{CN}_2^1 [\text{PR}^2 [\text{C}_0^1, \text{P}_1^3], \text{P}_1^1, \text{C}_0^1]$$

其中的主要复杂度来自于我们要求 PR^n 中的 $n \geq 2$ （而这个要求来自我们定义 μ -递归函数至少接受一个输入）。并令 $\dot{-}\mathbf{x} = \dot{-}(\mathbf{x})$ 。则有对任意 $\mathbf{y}$, $\mathbf{x} = \mathbf{S}(\mathbf{y})$, 有 $\mathbf{y} = \dot{-}\mathbf{x}$ 。

4. 现在, 我们讨论原始递归。对任意 $n \geq 2$, 任意 $\mathbf{x}_2$ 到 $\mathbf{x}_n$, 任意调用 G_A 只输入 $x \in \{\mathbf{x}_2, \dots, \mathbf{x}_n\}$, 有 $n-1$ 元 μ -递归函数 G 满足 $G(\mathbf{x}_{2\dots n}) = G_A$ 。任意 $\mathbf{y}$, 任意 $\mathbf{z}$, 任意调用 H_A 只输入 $x \in \{\mathbf{x}_2, \dots, \mathbf{x}_n, \mathbf{y}, \mathbf{z}\}$, 有 $n+1$ 元 μ -递归函数 H 满足 $H(\mathbf{y}, \mathbf{z}, \mathbf{x}_{2\dots n}) = H_A$ 。则存在 n 元 μ -递归函数 $F = \text{PR}^n [G, H]$, 满足当 $\mathbf{z} = F(\mathbf{y}, \mathbf{x}_{2\dots n})$ 时:

$$F(0, \mathbf{x}_{2\dots n}) = G_A$$

$$F(\mathbf{S}(\mathbf{y}), \mathbf{x}_{2\dots n}) = H_A$$

令 H'_A 等于替换 $\mathbf{y}$ 为 $\dot{-}\mathbf{x}_1$, 替换 $\mathbf{z}$ 为 $F(\dot{-}\mathbf{x}_1, \mathbf{x}_{2\dots n})$ 后的 H_A 。则有对任意 $\mathbf{x}_1$ 到 $\mathbf{x}_n$:

$$F(\mathbf{x}_{1\dots n}) = \begin{cases} G_A & \text{若 } \mathbf{x}_1 = 0 \\ H'_A & \text{若 } \mathbf{x}_1 \neq 0 \end{cases}$$

或写作 $F(\mathbf{x}_{1\dots n}) = (H'_A \text{ if } \mathbf{x}_1 \text{ else } G_A)$ 。同时由 1 有存在 F 满足 $F(\mathbf{x}_{1\dots n}) = (H'_A \text{ if } \mathbf{x}_i \text{ else } G_A)$, 其中 $1 \leq i \leq n$ 。

则可实现 + 满足¹:

- 加法: $+(a, b) = (\mathbf{S}(+(\mathbf{a}, \dot{-}\mathbf{b})), \mathbf{a}) \text{ if } \mathbf{b} \text{ else } \mathbf{a})$, 即 $a \cdot b = \begin{cases} a + (b-1) + 1 & \text{若 } b \neq 0 \\ a & \text{若 } b = 0 \end{cases}$ 。

最后, 我们令:

5. 在不作为输入输出时, n 表示 $\text{C}_n^1(\mathbf{x})$ 注²。

这样, 对自然数, 我们还可实现:

- 乘法: $\cdot(a, b) = (+(\cdot(\mathbf{a}, \dot{-}\mathbf{b})), \mathbf{a}) \text{ if } \mathbf{b} \text{ else } 0)$, 即 $a \cdot b = \begin{cases} a \cdot (b-1) + a & \text{若 } b \neq 0 \\ 0 & \text{若 } b = 0 \end{cases}$ 。
- 幂: $^{\wedge}(a, b) = (\cdot(^{\wedge}(\mathbf{a}, \dot{-}\mathbf{b})), \mathbf{a}) \text{ if } \mathbf{b} \text{ else } 1)$, 即 a^b , 其中采用了定义 $0^0 = 1$ 。
- 减法: $\dot{-}(a, b) = (\dot{-}(\dot{-}\mathbf{a}, \dot{-}\mathbf{b}) \text{ if } \mathbf{b} \text{ else } \mathbf{a})$, 其中 $a \dot{-} b = \begin{cases} a - b & \text{若 } a \geq b \\ 0 & \text{若 } a < b \end{cases}$ 。
- 非: $!(a) = (0 \text{ if } \mathbf{a} \text{ else } 1)$, 令 $!\mathbf{a} = !(\mathbf{a})$, 则 $!!\mathbf{a} = (1 \text{ if } \mathbf{a} \text{ else } 0)$ 。
- 比较: $\geq(a, b) = !!\dot{-}(a, b)$, 则 $((a, b) = !\geq(b, a))$, 类似的可实现 $\leq$ 和 $>$ 。
- 相等: $=?(a, b) = \cdot(\geq(a, b), \leq(b, a))$ 。

¹此处 + 定义对应的图灵机显然不与前文给出的相同。

²注意 $\mathbf{x}$ 是否出现在调用的其它位置并不对这个定义的有效性产生影响。

- 条件: $\text{cond}(c, a, b) = +(\cdot(!c, a), \cdot(!c, b))$, 即计算 $\begin{cases} a & \text{若 } c \neq 0 \text{ 注}^1 \\ b & \text{若 } c = 0 \end{cases}$ 。
- 除法: $/ (a, b) = /'(a, b, a)$ 。其中,

$$/'(a, b, q) = (\text{cond}(+(\text{=?}(a, \cdot(b, q)), \cdot(>(a, \cdot(b, q)), <(a, \cdot(b, S(q))))), \\ q, /'(a, b, \text{!}q)) \text{ if } q \text{ else } 0)$$

即 q 自 a 起向下逐个检查直到有 $a = b \cdot q$ 或 $a > b \cdot q$ 且 $a < b \cdot (q + 1)$ 。
- 取余: $\%(a, b) = \dot{-}(a, \cdot(/(a, b), b))$ 。注意当 $b = 0$, 即对应除法和取余未定义时, 上面 $/$ 返回 0 而这里 $\%$ 返回 a 。事实上, 我们会通过如乘 0 (即类似卫语句, 如 cond 的实现) 的方式保证计算不受未定义时返回结果的影响, 因此未定义时的返回结果可以是任意的, 我们采用方便实现的结果²。类似的, 如上所述, 我们的计算同样不会受定义 $0^0 = 1$ 的影响, 而我们(无特殊处理的)采用了 $0^0 = 1$ 。
- 整除: $| (b, a) = !\%(a, b)$ 。
- 素数判定: $\text{pr}_?(p) = \text{cond}(\geq(p, 2), \text{pr}'_?(p, p), 0)$ 。其中,

$$\text{pr}'_?(p, d) = (\cdot(!+(\text{=?}(d, 1), \text{=?}(d, p)), !|(d, p)), \text{pr}'_?(p, \text{!}d)) \text{ if } d \text{ else } 1)$$
- 素数: $\text{pr}(n) = \text{pr}'(n, \wedge(2, n))$, 用以计算第 n 个素数。其中,

$$\text{pr}'(n, p) = (\cdot(\text{pr}_?(p), \text{=?}(\text{pr}_\#(p), n)) \text{ if } p \text{ else } 0)$$

其中 $\text{pr}_\#(p)$ 用于计算小于等于 p 的自然数中素数的个数, 可知 $\text{pr}'(n, p)$ 自 $p = 2^n$ 起³ 向下逐个检查直到有 p 为素数, 且小于等于 p 的素数个数为 n 。具体的,

$$\text{pr}_\#(p) = (+(\text{pr}_?(p), \text{pr}_\#(\text{!}p)) \text{ if } p \text{ else } 0)$$

由 pr 的实现我们也可以看到当计算的结果有上界且有判定方法时, 我们可以通过原始递归而非极小化得到计算结果。同时注意我们到目前为止并没有使用极小化, 事实上在很多情况下我们不需要使用极小化, 并且不涉及极小化的计算有很多很好的性质, 比如这样的计算总是返回的。然而, 如我们将看到的, 对本书主要目标中大部分理论极限的讨论必须依赖极小化。因此, 我们仍定义:

6. 对于极小化算子, 对任意 $n \geq 2$, 任意 x_1 到 x_n , 任意调用 G_A 只输入 $x \in \{x_1, \dots, x_n\}$, 则写 $\text{Min}_{x_1} \{G_A\} = \text{MN}^{n-1} [G] (x_{2..n})$, 其中 $G(x_{1..n}) = !G_A$ 。如我们可实现 μ -递归函数 F 有 $F(x) = \text{Min}_y \{<(x, y)\}$, 即 $F(x)$ 返回第一个大于 x 的自然数, 即 $S(x)$ 。

我们有了一些可被 μ -递归函数计算的例子。现在让我们通过证明:

引理 2.3.3 任意图灵机均存在等价的 μ -递归函数。

得到:

定理 2.3.4 计算模型 μ -递归函数和图灵机是等价的。

同时, 显然以此我们也完成了结构化图灵机和图灵机计算能力相同的证明。

2.3.5 编码图灵计算

回顾图灵机的即时描述给出了指令执行前后全部机器状态的规范形式, 包括纸带内容、指针位置和当前状态。特别的, 输入描述为运行前即时描述, 输出描述为运行后即时描述。则显然对任意图灵机 T ,

¹概念和用法上都不同于 if else , 比如用法上这里的条件可以是任意复杂的调用。

²或者说, 当对应概念未定义时, 计算结果是由实现定义的。而我们的最终结果不应受实现定义的影响。

³由伯特兰-切比雪夫定理有对 $n \geq 1$, 有 $\text{pr}(n+1) < 2\text{pr}(n)$, 即 $\text{pr}(n) \leq 2^n$ 。

给定即时描述 d_i ，有要么有满足其描述状态的指令执行后可得即时描述 d_{i+1} ，要么其为输出描述。则给定任意输入描述 d_0 ，有要么存在自然数 n 有 d_n 为输出描述，要么图灵机 T 不返回。当存在 d_n 时，我们称 $\langle d_0, \dots, d_n \rangle$ 为 T 的一次运算描述，写 $d_{\dots n}$ 。

现对自然数 m, n ，考虑由符号集 $\{0, 1\}$ ^{注1}，状态集 $\{q'_i : 0 \leq i \leq m\}$ ^{注2} 和指令集合 $I = \{i'_i : 0 \leq i < n\}$ 构成的图灵机 T 。我们将自顶向下的给出构造等价于该图灵机的 μ -递归函数的方法。具体的，我们将构造一个 μ -递归函数 F ，对输入 x ，若 T 返回并输出 y ，则 $y = F(x)$ ，若 T 不返回，则 F 不返回。其中 $x^T y$ （若存在 y ）对应 xy 在 F 下的编码结果。同时，我们需要 $x^T y^T$ 表示 xy 在 T 的编码方式下的结果，事实上， $x^T y^T$ 即对应 T 此次运算的输入描述和输出描述（若存在输出描述），且在实际构造中，我们直接建立（且也只能直接建立） xy 和 $x^T y^T$ 之间的对应关系。此外，对任意概念 a ，在需要时， a^T 和 a 总是分别表示图灵机和 μ -递归函数下的编码结果（即符号序列），则写 $q_i^T = q'_i$ ($i \leq m$)， $i_i^T = i'_i$ ($i < n$)，且我们也将看到更多其它具体概念的例子³。

直观的说，我们尝试寻找运算输入描述为 x^T 的运算描述，并在成功时，从中得到输出描述 y^T 。具体的，我们对 T 中的指令和即时描述、运算描述进行哥德尔编码。首先要求 $x^T y$ （若存在 y ）分别等于 $x^T y^T$ 的哥德尔数，这样即确定了 F 的输入输出编解码方式。同时，在 F 中，所有即时描述、运算描述的表达式均通过其对应的哥德尔数，这样，在 F 中，我们称即时描述、运算描述即分别指其对应的哥德尔数。类似的，在 F 中，我们称指令指其对应的哥德尔数。然后，我们使用极小化算子实现寻找满足其输入描述等于 x 的运算描述，并在找到后，从中取出输出描述（即 y ），即：

- $F(x) = \text{out}(\text{Min}_c\{G(c, x)\})$ ，其中 $G(c, x) = 1$ 当且仅当 c 等于输入描述为 x 的运算描述。 out 从运算描述中取出输出描述。

为实现指令和即时描述、运算描述的哥德尔编码，我们以如下方式将构成指令和即时描述的元素一一对应到自然数上：

L^T	R^T	0	1	q_0^T	q_1^T	$\dots$	q_m^T
0	1	2	3	4	5	$\dots$	$m+4$

则写 $L = 0$ ， $R = 1$ 。对应 0 写 $\text{zero} = 2$ ，对应 1 写 $\text{one} = 3$ 。并对应 q_i^T 写 q_i ($i \leq m$)。

这样，如指令 $i^T = q_i^T 0 L q_j^T$ ($i \leq m$ 且 $j \leq m$)，其哥德尔数 $gn_s(i^T) = 2^{i+5} \cdot 3^3 \cdot 5^1 \cdot 7^{j+5}$ 。如此，对每一个指令 $i^T \in I$ ，都有一对一对应的哥德尔数 $gn_s(i^T)$ ，写 i_i 表示 $gn_s(i^T)$ 在 μ -递归函数中对应的符号序列。进而因图灵机由指令集合决定，我们可以用 $gn_I = gn(1, gn_s(i_0^T), \dots, gn_s(i_{n-1}^T))$ 代表 T ，注意 gn_I 仅由 T 决定（而不受运行时输入影响）⁴。写 i_{all} 表示 μ -递归函数中 gn_I 对应的符号序列。

而如即时描述 $d^T = 1 0 1 q_i^T 1 1$ ($i \leq m$)，其哥德尔数 $gn_s(d^T) = 2^4 \cdot 3^3 \cdot 5^4 \cdot 7^{i+5} \cdot 11^4 \cdot 13^4$ 。则每一个即时描述 d_i^T 同样有一对一对应的哥德尔数 $gn_s(d_i^T)$ ，写 d_i 表示 $gn_s(d_i^T)$ 在 μ -递归函数中的符号序列。进而对每一个运算描述 $d_{\dots i}^T$ 都有一对一对应的哥德尔数 $gn_c(d_{\dots i}^T) = gn(gn_s(d_0), \dots, gn_s(d_i))$ ，类似的 $d_{\dots i}$ 为其在 μ -递归函数中的符号序列。如此，我们便完成了所有所需的哥德尔编码定义。接着，我们分解用于极小化中条件判定的 μ -递归函数 G ：

- $G(c, x) = \cdot(C?(c), =?(in(c), x))$ 。其中 $C?(c)$ 用于判断 c 是否为运算描述， $in(c)$ 从运算描述 c 中取出输入描述。

¹如章节 2.3.2 中指出，增加符号不增加图灵机的计算能力。

²符号 q'_i 因如下述 q_i 用于表示 μ -递归函数下的编码结果。

³当以一个系统描述另一个系统，或一个系统讨论另一个系统时，普遍需要在两套系统的符号间建立对应关系，有时这种对应关系非常简单而显得可以直接在两套系统中使用相同的符号。但一旦讨论趋于复杂，这种符号在系统间混淆的可能性就会变得非常危险，所以，在不熟练的时候，总是建议明确符号所属的系统。

⁴同时注意由于集合的无序性（和严格的说可重复性）， gn_I 不是规范形式，即可能有不同的 gn_{I_i} 对应同一个图灵机。此外， $gn_I = gn(1, \dots)$ 中的 1 是为了处理 $n = 0$ 的情况。

- 具体的, $C_?(c) = \cdot(\text{gn}_?(c), \cdot(\text{descs}_?(c, \text{step}_\#(c)), C'_?(c, \perp \text{step}_\#(c))))$ 。其中 $\text{gn}_?(c)$ 检查 c 是否为哥德尔数。 $\text{step}_\#(c)$ 为 c 的长度 (注意 c 中元素下标从零开始)。 $\text{descs}_?$ 检查 c 是否对应即时描述序列。 $C'_?$ 递归的检查全部相邻即时描述之间的转换是否是可行的, 即若 c 中即时描述最大下标为 b 个即时描述组成, 对 $0 \leq i < b$, d_i 表示 c 中第 i 个即时描述, 检查对任意 d_i^T ($0 \leq i < b$) 对应的机器状态, 是否存在指令 i_j^T ($0 \leq j < n$) 满足执行条件, 并在执行后有机器状态对应 d_{i+1}^T 。
- 以 $\text{descs}_?(c, s) = (\text{desc}_?(\text{step}_\circ(c, \perp s), \text{step}_\circ(c, s)) \text{ if } s \text{ else } 1)$ 。其中 $\text{step}_\circ(c, k)$ 表示从 c 中取出下标为 k 的项, 符号 $\circ$ 标记下标从零开始 (下同)。 $\text{desc}_?(d)$ 检查 d 是否为即时描述。
- 及 $C'_?(c, k) = (\cdot(\text{do}_?(\text{step}_\circ(c, \perp k), \text{step}_\circ(c, k)), C'_?(c, \perp k)) \text{ if } k \text{ else } 1)$ 。其中 $\text{do}_?$ 检查 d_{k-1} 和 d_k 是否构成可行的步骤。则 $\text{do}_?(d, e) = \text{do}'_?(n, d, e)$, 回顾 n 为指令个数。
- $\text{do}'_?(k, d, e) = (!!+(\text{by}_?(\text{inst}_\circ(\perp k), d, e), \text{do}'_?(\perp k, d, e)) \text{ if } k \text{ else } 0)$ 。其中 $\text{inst}_\circ(k)$ 计算下标为 k ($k < n$) 的指令, 即 i_k 。 $\text{by}_?(k, d, e)$ 检查是否可以通过指令 k 由即时描述 d 到 e 。另¹。

接着, 我们实现 $\text{desc}_?$, 首先:

- $q_?(s) = \cdot(\geq(s, 4), \leq(s, +(m, 4)))$ 。即检查 s 是否对应状态符号 (回顾 m 为状态符号最大下标)。

并对任意待检查 (长度可能为零) 的符号序列 d^T :

- $\text{sym}_\#(d)$ 对应 d^T 的长度。
- $\text{sym}(d, k)$ 对应 d^T 的中第 k 位符号 (k 自 1 开始)。
- $\text{xs}_?(d, b, e) = (!!+(\geq(b, e), \cdot(+(\text{=?}(\text{sym}(d, \perp e), \text{zero}), \text{=?}(\text{sym}(d, \perp e), \text{one})), \text{xs}_?(d, b, \perp e))) \text{ if } e \text{ else } 1)$
即检查 d^T 中第 b 到 e (不包括 e) 位上的符号是否均为 $\underline{0}$ 或 $\underline{1}$ 。
- $\text{ptr}(d) = \text{ptr}'(d, \text{sym}_\#(d))$ 指出 d^T 中最后一个状态符号的位置。若不含状态符号, 返回 0。其中,
 $\text{ptr}'(d, e) = (\text{cond}(q_?(\text{sym}(d, e)), e, \text{ptr}'(d, \perp e)) \text{ if } e \text{ else } 0)$ 。
- $\text{desc}_?(d) = \cdot(\text{gn}_?(d), \cdot(\text{ptr}(d), \cdot(\text{xs}_?(d, 1, \text{ptr}(d)), \text{xs}_?(d, S(\text{ptr}(d)), S(\text{sym}_\#(d))))), \cdot(!\text{=?}(\text{sym}(d, 1), \text{zero}), \text{=?}(\text{sym}(d, \text{sym}_\#(d)), \text{zero})))$
即存在状态符号, 且 d^T 中其它位置均为 $\underline{0}$ 或 $\underline{1}$, 且首尾不为 $\underline{0}$ 。

接着, 我们实现 $\text{by}_?$, 对任意 (I 中的) 指令 $k^T = k_1^T k_2^T k_3^T k_4^T$; 任意自然数 u, v , 任意即时描述 $d^T = \underbrace{\underline{x} \dots \underline{x}}_u q_i^T \underbrace{\underline{x} \dots \underline{x}}_v$:

- $\text{by}_?(k, d, e) = \cdot(\text{prec}_?(k, d), \text{post}_?(k, d, e))$ 。其中 $\text{prec}_?(k, d)$ 检查 d^T 是否满足执行指令 k^T 的状态要求, 即指令的前置条件。 $\text{post}_?(k, d, e)$ 检查 d^T 在执行 k^T 后机器状态是否等于 e^T , 即 e^T 是否满足指令的后置条件。
- 要求 $1\text{st}(k) = k_1$, $2\text{nd}(k) = k_2$, $3\text{rd}(k) = k_3$, $4\text{th}(k) = k_4$ 。
- 则 $\text{prec}_?(k, d) = \cdot(\text{=?}(1\text{st}(k), \text{sym}(d, \text{ptr}(d))), \text{=?}(2\text{nd}(k), \text{cond}(<(\text{ptr}(d), \text{sym}_\#(d)), \text{sym}(d, S(\text{ptr}(d))), \text{zero})))$
即要求 $k_1^T = q_i^T$ 且若 $v = 0$, $k_2^T = \underline{0}$, 否则等于 v 段的第一个符号。
- 而 $\text{post}_?(k, d, e) = \cdot(\text{=?}(4\text{th}(k), \text{sym}(e, \text{ptr}(e))), \text{cond}(\text{=?}(3\text{rd}(k), \text{L}), \text{postL}_?(d, e), \text{cond}(\text{=?}(3\text{rd}(k), \text{R}), \text{postR}_?(d, e), \text{postS}_?(d, e, \text{!!=?}(3\text{rd}(k), \text{one}))))))$

¹其中 !! 仅为严格的处理构成指令集合 I 的序列 i_j^T 中可能有重复指令, 从而在加法计算后可能返回非 1 的正整数。

即要求 e^T 中的当前状态为 k_4^T ，并我们分类检查 k_3^T 等于 L^T ， R^T 和 $\underline{x}^T$ 时纸带和指针位置的情况。

- 具体的，首先需要 $\text{eqs}_?(d, b_d, e, b_e, s) =$

$$(\cdot(=?(\text{sym}(d, +(b_d, \perp s)), \text{sym}(e, +(b_e, \perp s))), \text{eqs}_?(d, b_d, e, b_e, \perp s)) \text{ if } s \text{ else } 1)$$

即检查 d^T 中自 b_d 和 e^T 中自 b_e 开始的 s 位符号相等 (b_d 和 b_e 自 1 开始)。

下面 $\text{postL}_?$ 的冗长实现源于对 d^T 中 uv 分别是否等于零的分类讨论，设 $e^T = \overbrace{\underline{x} \dots \underline{x}}^{u'} q_j^T \overbrace{\underline{x} \dots \underline{x}}^{v'}$:

- 令 $u(d) = \dot{+}(\text{ptr}(d), 1)$ ， $v(d) = \dot{+}(\text{sym}_{\#}(d), \text{ptr}(d))$ 。即分别用于计算运算描述 d 对应的 uv 。
- 则 $\text{postL}_?(d, e) = \text{cond}(\cdot(!u(d), !v(d)), p_{00}(e), \text{cond}(\cdot(!u(d), v(d)), p_{01}(d, e), \text{cond}(\cdot(u(d), !v(d)), p_{10}(d, e), p_{11}(d, e))))$

其中 p_{00} ， p_{01} ， p_{10} 和 p_{11} 分别对应 $u = 0, v = 0$ ， $u = 0, v \neq 0$ ， $u \neq 0, v = 0$ 和 $u \neq 0, v \neq 0$ 的情况。具体的：

$$- p_{00}(e) = ?(\text{sym}_{\#}(e), 1) \text{ 注}^1。$$

$$- p_{01}(d, e) = \cdot(!u(e), \cdot(=?(\text{sym}(e, 2), \text{zero}), \text{eqs}_?(e, 3, d, 2, v(d))))$$

即要求 $u' = 0$ 且 $v' = v + 1$ 且 v' 段首为 $\underline{0}$ 且后续位置符号和 v 段相同。

$$- p_{10}(d, e) = \cdot(\cdot(=?(\text{sym}(u(d)), \text{one}), \cdot(=?(\text{sym}(e, S(u(d))), \text{one}), !v(e))))$$

即要求 $u' = u - 1$ 且 u' 段符号和 u 段前 u' 个相同，且若 u 段末为 $\underline{1}$ 则 v' 段由一个 $\underline{1}$ 组成，否则 $v' = 0$ 。

$$- p_{11}(d, e) = \cdot(\cdot(=?(\text{sym}(u(d)), \text{one}), \cdot(=?(\text{sym}(e, S(u(d))), \text{one}), !v(e))))$$

$$\cdot(\text{eqs}_?(e, 1, d, 1, u(e)), \text{eqs}_?(e, +(u(d), 2), d, +(u(d), 2), v(d))),$$

$$=?(\text{sym}(e, +(u(e), 2)), \text{sym}(d, u(d))))$$

即 $u' = u - 1$ 且 $v' = v + 1$ ，且 u 段末符号等于 v' 段首符号，且对应段中其它位置的符号相同。

- 类似的，我们可以实现 $\text{postR}_?$ 。

对于 $\text{postS}_?(d, e, s)$ ，其中 s 代表指令所写的符号，我们检查 u 和 u' 段中的符号是否相同，并对 v 段的内容和 s 进行分类讨论。

- 即 $\text{postS}_?(d, e, s) = \cdot(\cdot(=?(\text{sym}(u(d)), \text{one}), \cdot(=?(\text{sym}(e, S(u(d))), \text{one}), !v(e))))$

其中 p_{00} ， p_{01} 和 p_{10} 分别处理 $v = 0, s^T = \underline{0}$ ， $v = 0, s^T = \underline{1}$ 和 $v = 1, s^T = \underline{0}$ 的情况， $p_{\star}$ 处理其它情况。具体的：

$$- p_{00}(e) = !v(e)。p_{01}(e) = ?(v(e), 1)。p_{10}(e) = !v(e)。$$

$$- p_{\star}(d, e, s) = \cdot(=?(\text{sym}(e, +(u(e), 2)), s), \text{eqs}_?(e, +(u(e), 3), d, +(u(d), 3), \dot{+}(\text{sym}_{\#}(e), +(u(d), 3))))$$

即 v' 段首为 s^T ，其它和 v 段中相同。

至此，我们只剩下检查是否为哥德尔数的 $\text{gn}_?$ 和对各类哥德尔数的基本操作没有实现，其中立即可得 $\text{in}(c) = \text{step}_{\circ}(c, 0)$ 和 $\text{out}(c) = \text{step}_{\circ}(c, \perp \text{step}_{\#}(c))$ 。这样，我们只剩 $\text{gn}_?$ ， $\text{step}_{\#}$ ， $\text{step}_{\circ}$ 。

¹这里没有直接比较 d 和 e 是因为 p_x 统一不检查其中的当前状态。

$inst_{\circ}$, $sym_{\#}$ 和 sym 需要实现, 我们可以基于以下通用方法实现:

- 哥德尔数判定: $gn_{\circ}(g) = \cdot(!!ge_{\#}(g), =_{\circ}(ge_{\#}(g), ge_c(g)))$ 。其中 $ge_{\#}(g)$ 返回最大可整除 g 的素数下标 (自 1 起), 当 g 为哥德尔数时, 其即为对应序列长度。 $ge_c(g)$ 返回可整除 g 的素数个数。
- 哥德尔数对应序列长度: $ge_{\#}(g) = ge'_{\#}(g, g)$ 。其中,

$$ge'_{\#}(g, n) = (cond(\mid (pr(n), g), n, ge'_{\#}(g, \frac{1}{2}n)) \text{ if } n \text{ else } 0)$$
- $ge_c(g) = ge'_c(g, g)$ 。其中,

$$ge'_c(g, n) = (cond(\mid (pr(n), g), S(ge'_c(g, \frac{1}{2}n)), ge'_c(g, \frac{1}{2}n)) \text{ if } n \text{ else } 0)$$
- 哥德尔数对应序列项: $ge(g, k) = \frac{1}{2}ge'(g, k, g)$ 。其中,

$$ge'(g, k, e) = (cond(\mid (^{pr(k)}, e), g, e, ge'(g, k, \frac{1}{2}e)) \text{ if } e \text{ else } 0)$$

则:

- $step_{\#} = ge_{\#}$ 。 $sym_{\#} = ge_{\#}$ 。 $sym = ge$ 。
- $step_{\circ}(c, k) = ge(c, S(k))$ 。
- $inst_{\circ}(k) = ge(i_{all}, +(k, 2))$ 。回顾 i_{all} 对应 gn_I , $gn_I = gn(1, gn_s(i_0^T), \dots, gn_s(i_{n-1}^T))$, 且在给定图灵机 T 时即可确定。

至此, 我们给出了对应图灵机 T 的 μ -递归函数 F 的全部实现。即完成了引理 2.3.3 的证明, 从而证明了图灵机和 μ -递归函数的计算能力是相同的。

2.3.6 通用图灵机¹

注意到上一小节中, 关于图灵机 T 的全部信息均被编码进了 gn_I , 我们可以通过扩展 μ -递归函数 F 的输入, 将 i_{all} 参数化, 使其可以模拟任意 (符号集等于 $\{0, 1\}$) 的图灵机 (在任意输入下) 的运行²。即扩展后有:

- ${}^uF(i_{all}, x) = out(\text{Min}_c\{{}^uG(c, i_{all}, x)\})$ 。
- ${}^uG(c, i_{all}, x) = \cdot({}^uC_{\circ}(i_{all}, c), =_{\circ}(in(c), x))$ 。
- ...
- ${}^u\text{do}_{\circ}(i_{all}, d, e) = {}^u\text{do}_{\circ}(i_{all}, \frac{1}{2}ge_{\#}(i_{all}), d, e)$ 。同时注意这里 n 被替换成了 $\frac{1}{2}ge_{\#}(i_{all})$ 。
- ${}^u\text{do}_{\circ}(i_{all}, k, d, e) =$

$$(!!+(by_{\circ}({}^uinst_{\circ}(i_{all}, \frac{1}{2}k), d, e), {}^u\text{do}_{\circ}(i_{all}, \frac{1}{2}k, d, e)) \text{ if } k \text{ else } 0)。$$
- ${}^uinst_{\circ}(i_{all}, k) = ge(i_{all}, +(k, 2))$ 。

由计算模型的等价性, 对应 uF , 我们有通用图灵机 uT 可以模拟任意图灵机 (在任意输入下) 的运行。

尽管理论上 uT 和 uF 是等价的, 但可以认为 uT 更具有现实意义。可以看到图灵机更接近实际机械设备, 其发明的目标之一也是实际可建造。其中唯一较难实现的部分是允许纸带的任意扩展, 但纸带长度和计算能力之间的关系是相对可知的, 计算理论中空间复杂度概念的提出可认为就是为专门研究这种关系。而通用图灵机在理论上保证了我们不需要为各类问题建造不同的图灵机, 其也被认为是冯·诺伊曼提出存储程序计算机的理论基础³, 而存储程序计算机的概念中就包括著名的冯·诺伊曼体系结构。

¹最早由艾伦·图灵在 1936-1937 年提出。

²在接下去的讨论中, 如无特别说明, 我们将只考虑符号集等于 $\{0, 1\}$ 的图灵机和即时描述

³不过显然这里给出的通用图灵机实现是低效的。

最后，我们定义直接模拟 n 元 μ -递归函数计算的通用 μ -递归函数 ${}^uF'_n(i, x_1, \dots, x_n) = d^T({}^uF(e_i^T(i), e_n^T(x_1, \dots, x_n)))$ 。其中 i 对应 μ -递归函数的哥德尔数。 e_i^T 执行 μ -递归函数的哥德尔数到图灵机的哥德尔数的转换，同时我们定义 d_i^T 做反向转换。 n 元 e_n^T 执行 μ -递归函数编码下的自然数多元组到结构化图灵机输入描述的转换，如 $\langle 2, 1 \rangle \Rightarrow q_0^T \perp \perp \perp \perp \Rightarrow gn_s(q_0^T \perp \perp \perp \perp) = gn(5, 4, 4, 3, 4) \Rightarrow 8135418060000$ ，即 $e_2^T(2, 1) = 8135418060000$ 。 d^T 做结构化图灵机输出到 μ -递归函数编码下自然数的转换。

2.3.7 集合的可计算性定义

现在，让我们将我们的讨论限制在集合的相关概念上，同时看到这样限制并不影响我们的讨论，并在此过程中给出问题可解的相关严格定义。

如上文所见，我们已经给出了（可能多元的）自然数函数部分可计算 / 全可计算，部分递归 / 全递归的定义。又由于图灵机和 μ -递归函数的等价性，这两类概念也是等价的。事实上，很多图灵机和 μ -递归函数涉及的概念之间总是可以找到这样的对应关系，因此，这里使用任一在上下文中更方便的计算模型对这些概念进行定义。

并且由于哥德尔编码的存在，任何多元的自然数函数都可以一对一对应到一元自然数函数上。事实上，由哥德尔编码，任何图灵机的输入输出都可以对应到自然数上^{1 2}。因此，当我们说一个问题图灵可解时，等于说其所对应的输入到输出函数进一步编码后得到的自然数函数 f 是部分可计算的，当自然数 x 不对应编码结果时， $x \notin \text{dom } f$ ，计算 f 的图灵机不返回。显然我们可以补全 f 的定义域使其成为一个全函数，即我们需要解决当 x 是原问题输入的编码结果时，返回原问题编码后的输出，否则返回任意（可由 x 决定的）结果。但部分可计算的 f 在补全后并不一定是全可计算的，我们已多次提到这种可能，并且将在下一小节中证明这种情况的确存在。

这样，在最一般的情况下，我们可以使用一元自然数函数对图灵机 / μ -递归函数的计算能力进行讨论。为完整性，同时重述可计算性相关的定义如下：

- 对任意 $A \subseteq \mathbb{N}$ ，函数 $f: A \mapsto \mathbb{N}$ 是部分可计算的若存在图灵机 T 对任意自然数 x ， $\mathbf{x}$ 为 x 的编码结果，当 $x \in \text{dom } f$ 时， $y = T(\mathbf{x})$ ，其中 y 为 $y = f(x)$ 的编码结果，当 $x \notin \text{dom } f$ 时， $T(\mathbf{x})$ 不返回。并称 T （或相应 μ -递归函数如 F ）计算 f 。同时也称 f 是部分递归的。
- 对任意自然数上的全函数 $f: \mathbb{N} \mapsto \mathbb{N}$ ，若其是部分可计算的，则 f 是全可计算（或可计算）的。同时也称 f 是全递归（或递归）的。
- 对任意自然数上的全函数 $f: \mathbb{N} \mapsto \mathbb{N}$ ，若原始 μ -递归函数 F 满足对任意 x ， $\mathbf{x}$ 为 x 的编码结果，有 $y = F(\mathbf{x})$ 且 y 为 $y = f(x)$ 中 y 的编码结果，则称 f 是原始递归的。

注意全递归和原始递归不是等价的。显然原始递归函数是全递归的，但：

定理 2.3.5 存在全递归函数不是原始递归的。

我们延后上述定理的证明至小节尾，以先完成集合相关的可计算性定义讨论。对任意集合 $A \subseteq \mathbb{N}$ ，并称函数 $c: \mathbb{N} \mapsto \{0, 1\}$ 满足 $c(x) = \begin{cases} 1 & x \in A \\ 0 & x \notin A \end{cases}$ 为 A 的特征函数：

¹ 因此也显然没有图灵机能用于解决输入集合大于 $\aleph_0$ 的问题。在 ZFC 集合论下，这类问题是存在的，如实数加法，但我们无法实际——采用基数小于等于 $\aleph_0$ 的语言，比如自然语言和本书介绍的所有形式语言——精确描述这类问题下的每一个具体问题，如果我们无法描述每一个实数。事实上，这样的具体问题可认为是非构造性的，这样的问题类别也是非构造性的。

² 也因此我们的可计算性讨论也不涉及不可数的集合。

- 若存在部分递归函数 f 为 $\{0, 1, \dots, |A| - 1\}$ 和 A 上的双射函数¹，我们称 A 是可枚举²的。直观的说，我们可以通过 $f(0), f(1), \dots, f(|A| - 1)$ ，不重复的枚举出 A 中的任意元素。同时，我们称用于计算 f 的机械设备 $\mathbf{f}$ 枚举 A 。
- 若特征函数 c 是全递归的，则 A 是可判定的。即总是存在图灵机对任意自然数可以判定其是否属于 A 。否则我们称其为不可判定的。当 A 是可判定的，也称其是全可计算（可计算），全递归（递归）的。若特征函数 c 是原始递归的，则也称 A 是原始递归的。
- 若函数 $c' : A \mapsto \{1\}$ 是部分递归的，则 A 是半可判定的。即存在图灵机对任意自然数 x ，若 $x \in A$ ，可以肯定 $x \in A$ 。注意可判定的集合也是半可判定的，而我们将在下一小节证明不可判定的集合可能是半可判定的。即 $\underbrace{\dots \text{可判定} \dots | \dots \text{不可判定} \dots}_{\wp(\mathbb{N})}$ 。同时，当 A 是半可判定的，也称其是半可计算，半递归的。

同时注意可枚举和半可判定是等价的，即任意集合 A 是可枚举的当且仅当其是半可判定的。即我们有

$$\underbrace{\dots \text{可判定} \dots | \dots \text{不可判定} \dots}_{\wp(\mathbb{N})} \text{。更一般的：}$$

定理 2.3.6 对任意集合 $A \subseteq \mathbb{N}$ ， A 是可枚举的当且仅当：

- 存在部分递归函数 f ，有 $A = \text{ran } f$ 。
- 存在部分递归函数 f ，有 $A = \text{dom } f$ 。
- $A = \emptyset$ ，或存在全递归函数 f ，有 $A = \text{ran } f$ 。
- $A = \emptyset$ ，或存在原始递归函数 f ，有 $A = \text{ran } f$ 。

证明：令 $D = \{0, 1, \dots, |A| - 1\}$ ， A 是可枚举的即指存在部分递归函数 f_0 有 $\text{dom } f_0 = D$ ， $\text{ran } f_0 = A$ 。同时，我们修改 $\mathcal{U}_F$ 为额外接受表示运算描述长度限制的输入 m 的 $\mathcal{U}_F(i_{\text{all}}, x, m)$ ，直观的说， $\mathcal{U}_F$ 模仿由 i_{all} 决定的图灵机 $F_{i_{\text{all}}}$ 在输入 x 时的运行直到要么有返回结果，要么总指令执行次数达到 m ，即可理解为超时。当超时 $\mathcal{U}_F$ 返回 0，否则若模拟运行返回 y ，返回 $y + 1$ 。技术上， m 为最外层最小化算子所搜索的运算描述提供了上限，因此可以用原始递归替换。这样 $\mathcal{U}_F$ 也是原始递归的，即 $\mathcal{U}_F$ 总是返回的。通过 $\mathcal{U}_F$ ，对于任意有限多个可能不返回的计算，我们可以通过逐步增加超时，以广度优先的方式搜索其中的第一个返回结果。在此基础上，我们构造对应 $\mathcal{U}_{F'_n}$ 的 $\mathcal{U}_{F'_n}$ 。接着我们应用这一方法逐一证明 $a b c d$ 。

- 显然有正方向。对反方向，令部分递归函数 f_1 有 $A = \text{ran } f_1$ ， $\mathbf{f}_1$ 为其对应的 μ -递归函数， $i_{\mathbf{f}_1}$ 为其对应用于输入 $\mathcal{U}_{F'_1}$ 的哥德尔数。构造 $\mathbf{f}_0$ 在输入 x 时，使用极小化算子搜索 i 满足 i 是 $\langle x_i, m_i \rangle$ 的哥德尔数，其中 $x_i m_i$ 表示任意由 i 决定的自然数（下同），且 $\mathcal{U}_{F'_1}(i_{\mathbf{f}_1}, x_i, m_i)$ 返回 $y_i + 1$ ，且对任意 $j < i$ ，若有 y_j ， $y_j \neq y_i$ 且由所有这样的 y_j 构成的集合基数等于 x 。当找到这样的 i 时，返回 y_i 。即直观的说，逐步增加 $\mathbf{f}_1$ 的输入和可执行步数（时间）直到找到 A 中的第 x 个元素。可以看到这样的 $\mathbf{f}_0$ 是可实现的，即 f_0 是部分递归的，且有 $\text{dom } f_0 = D$ ， $\text{ran } f_0 = A$ 。
- 对正方向，我们通过 μ -递归函数 $\mathbf{f}_0$ 构造 $\mathbf{f}_1$ 对应 f_1 有 $\text{dom } f_1 = A$ 。具体的，对 $\mathbf{f}_1$ 的输入 x ，我们使用极小化算子搜索 i 满足 i 对应 $\langle x_i, m_i \rangle$ ，且 $\mathcal{U}_{F'_1}(i_{\mathbf{f}_0}, x_i, m_i)$ 返回 $x + 1$ ，当找到时返回如 x_i （这样构成的 $f_1 = f_0^{-1}$ ）。对反方向，我们构造 $\mathbf{f}'_1$ 对输入 x ，若 $\mathbf{f}_1$ 返回，则返回 x ，则 f'_1 是部分递归的，且 $\text{ran } f'_1 = A$ ，由 a 可得 f_0 。
- $A = \emptyset$ 时在两个方向上都显然成立。当 $A \neq \emptyset$ 时，正方向，同样通过 $\mathbf{f}_0$ 构造 $\mathbf{f}_1$ 。具体的，对输入 x ，若 x 不对应 $\langle x_x, m_x \rangle$ ，返回 $\mathbf{f}_0(0)$ ，否则若 $\mathcal{U}_{F'_1}(i_{\mathbf{f}_0}, x_x, m_x)$ 返回 $y + 1$ ，返回 y ，否则返回 $\mathbf{f}_0(0)$ 。

¹这里当 $|A| = \aleph_0$ 时， $\{0, 1, \dots, |A| - 1\} = \mathbb{N}$ ，下同。

²对图灵机和 μ -递归函数，也分别称计算可枚举和递归可枚举。

反方向由 a 直接可得。

d. 对 $A \neq \emptyset$, 正方向, 可以看到我们在 c 中构造的 f_1 即是原始递归的。反方向同样由 a 直接可得。□

为方便讨论, 对可枚举集合 A , 称计算函数 $c' : A \mapsto \{1\}$ 的机械设备 $\mathbf{c}'$ 可枚举 A 。

最后, 当一个集合内元素可编码到自然数时, 我们也可以相应的称该集合是可枚举, 可判定的等。这样, 有图灵机, μ -递归函数和原始 μ -递归函数 (即原始递归函数对应的 μ -递归函数) 的集合, 以及相应的部分可计算 / 部分递归函数和原始递归函数的集合也都是可枚举的。同时, 不是原始递归的全递归函数存在也意味着我们不能通过枚举原始递归函数枚举全递归 (即全可计算) 函数。在下一小节中, 我们将进一步看到全递归函数是不可枚举的。

下面, 我们证明之前已经用到的结论和定理 2.3.5。

定理 2.3.7 任意可枚举集合的有限子集集合也是可枚举的。

证明: 对可枚举集合 A , 其元素编码到自然数后的集合 A' , 令 $D = \{0, 1, \dots, |A| - 1\}$, 有部分可计算函数 f 满足 $\text{dom } f = D$, $\text{ran } f = A'$ 。按定理 2.3.6 证明 b 中正方向时给出的方式构造部分可计算的逆函数 f^{-1} 。

则我们可以将 A' 的有限子集按 $\begin{cases} 0 & A = \emptyset \\ \prod_{x \in A'} pr(f^{-1}(x) + 1) & A \neq \emptyset \end{cases}$ 编码到自然数上 (回顾 $pr(n)$ 等于第 n 个素数)。同时, A' 的有限子集集合经此编码后对应的集合是半可判定的, 我们可以采用类似判定哥德尔数的方法对其中的自然数进行判定, 注意这里不是可判定的是因为 A 的大小可以是未知的¹。这样, A 的有限子集集合是可枚举的。□

定理 2.3.5 的证明: 定义阿克曼函数如下, 令二元自然数函数 $\uparrow^0 = +$, 对 $n > 0$:

$$a \uparrow^n b = \overbrace{a \uparrow^{n-1} (a \uparrow^{n-1} \dots (a \uparrow^{n-1} a) \dots)}^{b \uparrow a}$$

其中要求 $b > 0$ 。则如对 $b > 0$, $a \uparrow^0 b = a + b$, $a \uparrow^1 b = a \cdot b$, $a \uparrow^2 b = a^b$ 。定义阿克曼函数:

$$A(x, y) = \begin{cases} y + 1 & x = 0 \\ 2 \uparrow^{x-1} (y + 3) - 3 & x \neq 0 \end{cases}$$

其中 $\text{dom } A = \mathbb{N} \times \mathbb{N}$ 。则有如 $A(0, 0) = 1$, $A(1, 1) = 3$, $A(2, 2) = 7$, $A(3, 2) = 29$, $A(4, 2) = 2^{65536} - 3$, $A(4, 3) = 2^{2^{65536}} - 3$ 等。可以看到阿克曼函数的增长非常快, 这个即是其无法用原始归 μ -递归函数计算的根本原因, 或者说没有原始递归函数跟得上阿克曼函数的增长速度。在进一步证明阿克曼函数不是原始递归的之前, 我们先证明其是全递归的。首先由数学归纳法有对任意自然数 i , 正整数 a, b 有 $a \uparrow^i b > 0$, 则可以看到阿克曼函数是全函数。同时, 其也是部分递归的, 令未定义时返回 0, 则:

$$\begin{aligned} \uparrow^1(a, b) &= (\uparrow^0(\uparrow^1(a, \uparrow^1 b), a) \text{ if } b \text{ else } 0) \\ \uparrow^2(a, b) &= (\uparrow^1(\uparrow^2(a, \uparrow^1 b), a) \text{ if } b \text{ else } 0) \\ &\vdots \\ \uparrow^n(a, b) &= (\uparrow^{n-1}(\uparrow^n(a, \uparrow^1 b), a) \text{ if } b \text{ else } 0) \quad \text{其中 } n > 0 \end{aligned} \quad (i)$$

¹即如下一小节所见, 我们不能通过部分可计算函数的计算方法确定其定义域或值域的大小。

令 $i_{\uparrow^n}$ 表示 $\uparrow^n$ 的哥德尔数, 则由 (i) 又有, 对 $n > 0$:

$$\uparrow^n(a, b) = (\mathcal{U}F'_2(i_{\uparrow^{n-1}}, \uparrow^n(a, \perp b), a), a) \text{ if } b \text{ else } 0)$$

即我们由 $i_{\uparrow^{n-1}}$ 的定义了 $\uparrow^n$, 进一步定义了 $i_{\uparrow^n}$, 即有 μ -递归函数 G 满足 $i_{\uparrow^n} = G(i_{\uparrow^{n-1}})$ 。令 $H(n) = (G(H(\perp n)) \text{ if } n \text{ else } i_{\uparrow^0})$, 则 H 为 μ -递归函数且 $H(n) = i_{\uparrow^n}$ 。则阿克曼函数 A 可由以下 μ -递归函数实现:

$$A(x, y) = \text{cond}(x, \perp (\mathcal{U}F'_2(H(\perp x), 2, +(y, 3)), 3), S(y))$$

这样我们就证明了阿克曼函数 A 是全递归的。同时注意其中 $\mathcal{U}F'_2$ 含最小化算子, 因此 A 不是原始递归的。接着, 我们通过以下两个引理证明¹ 对任意 n , n 元原始递归函数 f , 总是存在常数 q 满足对任意 $x_1 \dots x_n$, $f(x_1 \dots x_n) < A(q, x)$, 其中 $x = \max\{x_1 \dots x_n\}$ (即 $x_1 \dots x_n$ 中的最大值)。即对任意原始递归函数 f 总存在 q 有 $A(q, q)$ 不属于 $\text{ran } f$ 。□

引理 2.3.8

- a. 对任意自然数 i , 有 $2 \uparrow^i 2 = 4$ 。
- b. 对任意自然数 i , $1 \leq u < v$, 有 $2 \uparrow^i u < 2 \uparrow^i v$ 。
- c. 对任意自然数 i , 有 $2 \uparrow^{i+1} 3 > 2 \uparrow^i 3$ 。
- d. 对任意自然数 i , $u \geq 3$, 有 $2 \uparrow^i u > u + 1$ 。

对任意自然数 x, y :

- e. $A(0, y) = y + 1$ 注²。
- f. $A(x + 1, 0) = A(x, 1)$ 。
- g. $A(x + 1, y + 1) = A(x, A(x + 1, y))$ 。
- h. 对任意自然数 $z > y$, 由 $A(x, y) < A(x, z)$ 。
- i. 对任意自然数 $z > x$, 由 $A(x, y) < A(z, y)$ 。
- j. $A(x, 0) > x$ 。
- k. $A(x, y) > y$ 。
- l. $A(x, y + 1) \leq A(x + 1, y)$ 。
- m. 对任意自然数 z , $A(x, A(y, z)) < A(x + y + 2, z)$ 。

证明:

- a. 对 i 做数学归纳可得。
- b. 对 i 做数学归纳可得。其中当 $i \neq 0$ 时, 同样用数学归纳法证明有 $2 \uparrow^{i-1} (v - u + 1) > 2$ 。
- c. $2 \uparrow^{i+1} 3 - 2 \uparrow^i 3 = 2 \uparrow^i 4 - 2 \uparrow^i 3 > 0$ 。
- d. 对 i 做数学归纳, 当 $i = 0$ 时, 对任意 $u \geq 3$, 有 $2 + u > u + 1$ 。否则假设对任意 $u \geq 3$, 有 $2 \uparrow^{i-1} (u - 1) > u$, 继续对目标中的 u 做数学归纳, 当 $u = 3$, $2 \uparrow^i 3 > 2 \uparrow^0 3 > 4$, 否则假设 $2 \uparrow^i (u - 1) > u$, 则 $2 \uparrow^i u = 2 \uparrow^{i-1} (2 \uparrow^i (u - 1)) > 2 \uparrow^{i-1} u > u + 1$ 。
- e. 即定义。
- f. 当 $x = 0$ 时由 $A(1, y) = 2 + (y + 3) - 3 = 2 + y$ 直接可得, 否则由 a 有 $2 \uparrow^x 3 = 2 \uparrow^{x-1} 4$, 即得。
- g. 当 $x = 0$ 时由 $A(1, y) = 2 + y$ 直接可得, 否则由定义有 $2 \uparrow^x (y + 4) = 2 \uparrow^{x-1} (2 \uparrow^x (y + 3))$, 即得。
- h. 当 $x = 0$ 时由 $A(1, y) = 2 + y$ 直接可得, 否则由 b 可得。
- i. 当 $x = 0$ 或 $x = 1$ 时, $A(0, y) < A(1, y)$, 否则 $A(x, y) - A(x - 1, y) = 2 \uparrow^x y - 2 \uparrow^{x-1} y = 2 \uparrow^{x-1} (2 \uparrow^x (y - 1)) - 2 \uparrow^{x-1} y$, 由 d 有 $2 \uparrow^x (y - 1) > y$, 则 $2 \uparrow^x (2 \uparrow^x (y - 1)) - 2 \uparrow^{x-1} y > 0$ 。

¹ 跳过这两个引理的证明不影响后续阅读。其中引理 2.3.8 主要为引理 2.3.9 的证明作准备。

² ??? 为阿克曼函数在很多材料中的常用定义方法。阿克曼函数还有其它变体。

- j. 当 $x = 0$ 或 $x = 1$ 时成立, 否则对 x 做数学归纳, 其中 $A(x, 0) - A(x - 1, 0) = 2^{\uparrow^{x-1} 3} - 2^{\uparrow^{x-2} 3} > 0$ 。
- k. 由 d 有 $A(x, y) = 2^{\uparrow^x (y+3)} - 3 > y + 4 - 3 > y$ 。
- l. 对 y 做数学归纳, 当 $y = 0$, $A(x, 1) = A(x + 1, 0)$ 。否则假设 $A(x, y) = A(x + 1, y - 1)$, 则 $A(x, y + 1) \leq A(x, A(x, y)) \leq A(x, A(x + 1, y - 1)) = A(x + 1, y)$ 。
- m. $A(x, A(y, z)) < A(x + y, A(y, z)) < A(x + y, A(x + y + 1, z)) = A(x + y + 1, z + 1) \leq A(x + y + 2, z)$ 。 $\square$

引理 2.3.9 对任意 n , n 元原始递归函数 f , 总是存在常数 q 满足对任意 $x_{1\dots n}$, $f(x_{1\dots n}) < A(q, x)$, 其中 $x = \max\{x_{1\dots n}\}$ 。

证明¹: 我们对用于计算 f 的 μ -递归函数 $\mathbf{f}$ 做数学归纳:

1. 若 $\mathbf{f}$ 为后继函数, $f(x) = x + 1 < A(1, x)$, 即取 $q = 1$ 。常量函数 $f(x_{1\dots n}) = c < A(c, x)$, 取 $q = c$, 具体的, $A(c, x) \geq A(c, 0) > c$ 。投影函数 $f(x_{1\dots n}) = x_i < A(0, x)$ ($1 \leq i \leq n$), 取 $q = 0$ 。
2. 若 $\mathbf{f}(\mathbf{x}_{1\dots n}) = \mathbf{h}(\mathbf{g}_1(\mathbf{x}_{1\dots n}), \dots, \mathbf{g}_m(\mathbf{x}_{1\dots n}))$, 即由复合算子构成。其中 $g_i(x_{1\dots n}) < A(r_i, x)$ 且 $y(y_{1\dots m}) < A(s, y)$, 其中 $1 \leq i \leq m$ 且 $y = \max\{y_{1\dots m}\}$ 。令 $r = \{r_1, \dots, r_m\}$ 。令 $g(x_{1\dots n}) = \max\{g_1(x_{1\dots n}), \dots, g_m(x_{1\dots n})\}$ 。则 $f(x_{1\dots n}) < A(s, g(x_{1\dots n})) < A(s, A(r, x)) < A(s + r + 2, x)$, 即取 $q = s + r + 2$ 。
3. 若 $\mathbf{f}(\mathbf{y}, \mathbf{x}_{1\dots n}) = (\mathbf{h}(\mathbf{1}^{\perp} \mathbf{y}, \mathbf{f}(\mathbf{1}^{\perp} \mathbf{y}, \mathbf{x}_{1\dots n}), \mathbf{x}_{1\dots n}) \text{ if } \mathbf{y} \text{ else } \mathbf{g}(\mathbf{x}_{1\dots n}))$, 即由原始递归构成。其中, $g(x_{1\dots n}) < A(r, x)$ 且 $h(y_{1\dots n+2}) < A(s, y)$, $y = \max\{y_{1\dots n+2}\}$ 。我们首先证明存在常数 t 满足对任意 $x_{1\dots n}$ 和 y :

$$f(y, x_{1\dots n}) < A(t, y + x)$$

取 $t = 1 + \max\{r, s\}$, 对 y 做数学归纳。当 $y = 0$, 有 $f(0, x_{1\dots n}) = g(x_{1\dots n}) < A(r, x) < A(t, x)$ 。否则假设 $f(y - 1, x_{1\dots n}) < A(t, y - 1 + x)$, 首先有 $f(y, x_{1\dots n}) = g(y - 1, f(y - 1, x_{1\dots n}), x_{1\dots n}) < A(s, w)$, 其中 $w = \max\{y - 1, f(y - 1, x_{1\dots n}), x\}$ 。又由假设以及 $\max\{y - 1, x\} \leq y - 1 + x < A(t, y - 1 + x)$ 有 $z < A(t, y - 1 + x)$, 则 $f(y, x_{1\dots n}) < A(s, z) < A(s, A(t, y - 1 + x)) \leq A(t - 1, A(q, y - 1 + x)) = A(t, y + x)$, 即 t 满足条件。最后, 令 $z = \max\{y, x\}$, 则 $f(y, x_{1\dots n}) < A(t, y + x) \leq A(t, 2z) < A(t, 2z + 3) = A(t, A(2, z)) < A(t + 4, z)$, 即取 $q = t + 4$ 。 $\square$

2.3.8 停机问题

现在, 我们考虑不可解的问题, 即不可计算的函数。同时, 由以下定理, 我们进一步将讨论限制于特征函数之内, 既判定问题。

定理 2.3.10 对任意一元自然数函数 f :

- a. 若 f 是全函数, 其是全递归的当且仅当是可判定的。
- b. 若 f 是部分函数, 其是部分递归的当且仅当是半可判定的。

证明: 令 c 为 f 的特征函数, μ -递归函数 $\mathbf{f}$ 和 $\mathbf{c}$ 分别计算 f 和 c 。对 a 的正方向, $\mathbf{c}(\mathbf{x}, \mathbf{y}) = \mathbf{?}(\mathbf{f}(\mathbf{x}), \mathbf{y})$ 。对 a 的反方向, $\mathbf{f} = \mathbf{Min}_y\{\mathbf{c}(\mathbf{x}, \mathbf{y})\}$ 。对 b 的反方向, 类似定理 2.3.6 中的证明, 我们使用 $\mathcal{U}\mathbf{F}'_2$ 对 $f(x)$ 的结果进行广度优先搜索。即 $\mathbf{f}(\mathbf{x}) = 2\mathbf{nd}(\mathbf{Min}_y\{\mathbf{cond}(\mathbf{?}(1\mathbf{st}(\mathbf{y}), \mathbf{x}), \mathbf{?}(\mathcal{U}\mathbf{F}'_2(\mathbf{i}_c, \mathbf{x}, 2\mathbf{nd}(\mathbf{y}), 3\mathbf{rd}(\mathbf{y})), 2), 0)\})$, 其中 $\mathbf{i}_c$ 为 $\mathbf{c}$ 的哥德尔数。 $\square$

接着, 因一元 μ -递归函数的哥德尔数是无限可枚举的, 存在 $\mathbb{N}$ 到 (一元, 以下默认) μ -递归函数的哥德尔数集合的双射全递归函数 f , 即认为 $f(i)$ 表示第 i 个 μ -递归函数的哥德尔数。令 $\mathbf{f}$ 计算 f , $\mathcal{U}\mathbf{F}^+(\mathbf{i}, \mathbf{x}) = \mathcal{U}\mathbf{F}'_1(\mathbf{f}(\mathbf{i}), \mathbf{x})$ 。

¹证明自 <https://planetmath.org/ackermannfunctionisnotprimitiverecursive>. [Last modified: 2013-03-22]。

这样，我们首先证明存在半可判定（即可枚举）但不可判定的集合。具体的，令：

$$\mathcal{H}_0 = \{ gn(i, x) : \text{第 } i \text{ 个 } \mu\text{-递归函数在输入 } x \text{ 时返回} \}$$

显然 $\mathcal{H}_0$ 是半可判定的，因为我们可以直接用部分递归函数 $\mathcal{U}_{F^+}$ （和常量函数的组合）计算 $c' : \mathcal{H}_0 \mapsto \{1\}$ 。即我们总是可以通过执行计算得到其最终返回的结论。但：

定理 2.3.11（停机问题结论） $\mathcal{H}_0$ 是不可判定的。即不存在有效方法对任意 μ -递归函数和输入都能确定其不会返回。

证明¹：我们使用对角论证法证明不存在 μ -递归函数 c 可以计算 $\mathcal{H}_0$ 的特征函数 $c : \mathbb{N} \mapsto \{0, 1\}$ 。对任意全递归函数 $f : \mathbb{N} \mapsto \{0, 1\}$ ，其对应 μ -递归函数 f ，令部分 μ -递归函数 $g(x) = \text{Min}_y \{!f(gn_2(x, y))\}$ （注意其中条件满足与否不依赖 y ）。即 g 计算 $g : \{x : f(gn(x, x)) = 0\} \mapsto \{0\}$ ，或²。此处对角论证指 g 由 $f'(u, v) = f(gn_2(u, v))$ 的对角线生成，即 $g(u)$ 的计算结果依赖且仅依赖 $f'(u, u)$ 。

接着进一步在此基础上，我们构造 zz 有 $f(zz) \neq c(zz)$ 若 c 存在。即有（任意全递归函数） $f \neq c$ 。具体的，令 $z = i_g$ ，即 g 对应哥德尔数的下标， $zz = gn_2(z, z)$ 。这样若 c 存在，有要求 $c(zz) = \begin{cases} 1 & \text{若 } g(z) \text{ 返回} \\ 0 & \text{若 } g(z) \text{ 不返回} \end{cases}$ 。进而若 $f(zz)$ 返回 0，则 $g(z)$ 返回，则 $c(zz)$ 返回 1。若 $f(zz)$ 返回 1，则 $g(z)$ 不返回，则 $c(zz)$ 返回 0。即 $f(zz) \neq c(zz)$ 。□

推论 2.3.11.1 存在二元 μ -递归函数 F ，有集合 $\mathcal{H}_F$ 是不可判定的，其中：

$$\mathcal{H}_F = \{ gn(x_1, x_2) : F(x_1, x_2) \text{ 返回} \}$$

证明： $\mathcal{U}_{F^+}$ 对应的 $\mathcal{H}_{\mathcal{U}_{F^+}}$ 即是不可判定的。因为若 $\mathcal{H}_{\mathcal{U}_{F^+}}$ 是可判定的，则 $\mathcal{H}_0$ 是可判定的。□

撇开上述推论证明的简单性，其是证明集合不可判定的一般方法。具体的，对任意集合 $A \subseteq \mathbb{N}$, $B \subseteq \mathbb{N}$ ，若可以证明 B 是可判定的意味着 A 是可判定的，则我们称 A 可以图灵归约到 B ，写作 $A \leq_T B$ ，可以直观地认为集合 A 的判定难度是不大于 B 的。这里我们有 $\mathcal{H}_0 \leq_T \mathcal{H}_{\mathcal{U}_{F^+}}$ 。这样，（对任意 A ）若， $\mathcal{H}_0 \leq_T A$ ，则 A 是不可判定的。

技术上， $A \leq_T B$ 的定义等价于在假定存在任意计算方法 c_B 有对任意输入 x ， $c_B(x) = \begin{cases} 1 & x \in B \\ 0 & x \notin B \end{cases}$ （即其可用于计算 B 的特征函数），其中 x 为 μ -递归函数编码方式下任意自然数 x 的编码结果的情况下，若我们将其视为 μ -递归函数，则按照 μ -递归函数的组合方式，我们能够构造出用于计算 A 特征函数的 μ -递归函数 c_A 。特别的，当 c_B 不为可计算方法时，我们称 c_B 的执行设备为预言者，增加了预言者调用的机械设备为预言机，注意邱奇-图灵猜想又等价于指预言者在现实世界中不存在。这样，回到停机问题，假定存在 $c_{\mathcal{H}_{\mathcal{U}_{F^+}}}$ ，则可构造 $c_{\mathcal{H}_0} = c_{\mathcal{H}_{\mathcal{U}_{F^+}}}$ 。则即有 $\mathcal{H}_0 \leq_T \mathcal{H}_{\mathcal{U}_{F^+}}$ 。

同时，当 $A \leq_T B$ 且 $B \leq_T A$ ，我们称集合 A, B 图灵等价，写作 $A \equiv_T B$ 。如又同时因可构造 $c_{\mathcal{H}_{\mathcal{U}_{F^+}}} = c_{\mathcal{H}_0}$ ，我们有 $\mathcal{H}_{\mathcal{U}_{F^+}} \leq_T \mathcal{H}_0$ ，进而有 $\mathcal{H}_0 \equiv_T \mathcal{H}_{\mathcal{U}_{F^+}}$ 。类似的，有：

推论 2.3.11.2 令 $\mathcal{H} = \{ i : \text{第 } i \text{ 个 } \mu\text{-递归函数在输入 } i \text{ 时返回} \}$ ， $\mathcal{H} \equiv_T \mathcal{H}_0$ 。

证明：显然， $\mathcal{H} \leq_T \mathcal{H}_0$ 。另一方面，写 f_i 表示第 i 个 μ -递归函数，对任意 i ，任意输入 x ，存在 j 满足对任意 y ， $f_j(y) = f_i(x)$ 。且 i 到 j 的对应函数是可计算的。则 $gn(i, x) \in \mathcal{H}_0$ 当且仅当 $j \in \mathcal{H}$ 。□

¹证明自 Wikipedia contributors. Halting problem — Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/w/index.php?title=Halting_problem&oldid=1055955070. 2021, #Sketch_of_rigorous_proof.

²`int g(x) { if (!f(x, x)) return 0; else while (1) continue; }`

而在定义图灵归约的过程中，我们事实上采用了一种相对可计算关系。具体的，对任意集合 $A \subseteq \mathbb{N}$ ，函数 $f: A \mapsto \mathbb{N}$ ，若在假定给出计算任意集合 $B \subseteq \mathbb{N}$ 的特征函数的计算设备 c_B ，我们可构造 μ -递归函数 f 计算 f ，则称 f 是 B 部分可计算，或 B 部分递归的。当 f 是全函数时，称 f 是 B (全) 可计算，或 B (全) 递归的。同时，也称 f 为 B - μ -递归函数，其对应图灵机为 B -图灵机。如此，可以看到 $A \leq_T B$ 当且仅当 c_A 是 B (全) 可计算 / (全) 递归的。也可以看到部分可计算、部分递归即指 $\emptyset$ 部分可计算、 $\emptyset$ 部分递归，(全) 可计算、(全) 递归即指 $\emptyset$ (全) 可计算、 $\emptyset$ (全) 递归。

并且，显然对任意 $A \subseteq \mathbb{N}$ ， A 部分可计算函数是无限可枚举的，即我们同样可以通过下标 i 指定第 i 个 A - μ -递归函数。

此外，显然对任意自然数子集合 A ， $A \leq_T A$ 。对任意 $A B C$ ，若 $A \leq_T B$ 且 $B \leq_T C$ ，则 $A \leq_T C$ 。因此 $\leq_T$ 是一种 (非严格) 偏序关系。同时：

定理 2.3.12 存在 $A \subseteq \mathbb{N} B \subseteq \mathbb{N}$ 满足 $A \not\leq_T B$ 且 $B \not\leq_T A$ ，即 $\leq_T$ 不是全序的。

证明¹：写 f_i^A 和 f_i^B 分别表示第 i 个 A - μ -递归函数和第 i 个 B - μ -递归函数。写 $c_A c_B$ 分别表示 $A B$ 的特征函数。即只要证明存在 $A B$ ，对任意 i ， f_i^B 不计算 c_A 且 f_i^A 不计算 c_B 。我们通过自零起逐步有序增量指定自然数是否属于 $A B$ 来构造满足条件的集合 $A B$ 。

具体的，对任意 k ，构造条件集合 $\mathcal{A}_k$ 含且仅含任意自然数 $n < |\mathcal{A}_k|$ 是否属于 A 的条件要求，如 $\mathcal{A}_2$ 可以是 $\{0 \in A, 1 \notin A\}$ 。且有 $\mathcal{A}_k \subseteq \mathcal{A}_{k+1}$ ， $\mathcal{A}_k \neq \mathcal{A}_{k+2}$ ，则 A 可由条件集合 $\bigcup_k \mathcal{A}_k$ (即 $\mathcal{A}_{k \rightarrow \infty}$) 确定。对 B 类似构造 $\mathcal{B}_k$ 。则 $\mathcal{A}_0 = \mathcal{B}_0 = \emptyset$ 。接着，在 $k = 2i + 1$ 步，构造使对任意满足 $\mathcal{A}_k$ 的自然数子集合 A' ，满足 $\mathcal{B}_k$ 的 B' ，第 i 个 B' - μ -递归函数不计算 $c_{A'}$ ；在 $k = 2i + 2$ 步，同理使第 i 个任意 A' - μ -递归函数不计算任意 $c_{B'}$ 。

- $k = 2i + 1$ 。令 $x = |\mathcal{A}_{k-1}|$ 。尝试寻找条件集合 $\mathcal{B}$ 满足 $\mathcal{B}_{k-1} \subseteq \mathcal{B}$ ，并含且仅含任意自然数 $n < |\mathcal{B}|$ 是否属于 B 的条件要求，使对任意满足 $\mathcal{B}$ 的自然数子集合 B' 有 $f_i^{B'}(x)$ 返回且计算过程中对 $c_{B'}$ 的调用结果均已被 $\mathcal{B}$ 确定。
 - 若存在 $\mathcal{B}$ ， $\mathcal{B}$ 是有限的，因计算过程中只包含有限次对 $c_{B'}$ 的调用。且 $f_i^{B'}(x)$ 的返回结果是确定的，令结果对应 y 。则构造 $\mathcal{A}_k = \begin{cases} \mathcal{A}_{k-1}; (x \in A) & y = 0 \\ \mathcal{A}_{k-1}; (x \notin A) & y \neq 0 \end{cases}$ ， $\mathcal{B}_k = \mathcal{B}$ 。这样有 $c_A(x) \neq y$ ，即 f_i^B 不计算 A 。
 - 否则，可构造 $\mathcal{A}_k = \mathcal{A}_{k-1}; (x \in A)$ 。因对任意 B' 满足 $\mathcal{B}_{k-1}$ ， $f_i^{B'}(x)$ 不返回， f_i^B 不计算 A 。
- $k = 2i + 2$ 。同理。 □

接着，我们定义对任意 $A \subseteq \mathbb{N}$ ， $B \subseteq \mathbb{N}$ ， $A <_T B$ 表示 $A \leq_T B$ 且 $A \not\equiv_T B$ 。这样有 $\emptyset <_T \mathcal{H}$ ，现在通过以下定理，我们证明存在 $\mathcal{H}'$ 有 $\mathcal{H} <_T \mathcal{H}'$ ，即直观的说存在比停机问题更难的问题。

定理 2.3.13 对任意 $A \subseteq \mathbb{N}$ ，定义 $A' = \{i : \text{第 } i \text{ 个 } A\text{-}\mu\text{-递归函数在输入 } i \text{ 时返回}\}$ ，有 $A <_T A'$ 。

证明：首先证明 $A \leq_T A'$ 。对任意自然数 x 可构造 A - μ -递归函数 $f_x(y) = \text{Min}_y\{c_A(x)\}$ 。且存在 (全) 递归函数 g 满足第 $g(x)$ 个 A - μ -递归函数为 f_x 。则构造 $c_A(x) = c_{A'}(g(x))$ 。

然后证明 $A' \not\leq_T A$ 。类似定理 2.3.11 的证明，对任意 A 全递归函数 f ，构造部分 A - μ -递归函数 $g(x) = \text{Min}_y\{!f(x)\}$ ，则存在 i 有第 i 个 A - μ -递归函数等于 g 且 $f(i) \neq c_{A'}(i)$ 。 □

这样，对任意自然数 n ，令 $\emptyset^{(n)} = \begin{cases} \emptyset & n = 0 \\ (\emptyset^{(n-1)})' & n \neq 0 \end{cases}$ 。我们即能建立起一个序列有 $\emptyset^{(0)} <_T \emptyset^{(1)} <_T$

¹证明自 J. DeAntonio. The Turing degrees and their lack of linear order. 2010。

$\emptyset^{(2)} <_T \dots$ 。其中注意 $\mathcal{H} \equiv_T \emptyset^{(1)}$ 。进一步，我们对自然数子集到图灵度上满足对任意 $A \subseteq \mathbb{N}$, $B \subseteq \mathbb{N}$, $A \equiv_T B$ 当且仅当其图灵度相等，这样 $<_T$ 即也是图灵度上的偏序关系。而对任意 $A \subseteq \mathbb{N}$, 称 A' 为 A 的图灵跳跃，则可以看到自 $\emptyset$ 起由图灵跳跃构成的关系是全序的，因此其对应的图灵度为全部图灵度的真子集。

最后我们证明存在不可枚举的（自然数子）集合。如令：

$${}^A\mathcal{H} = \{ i : \text{第 } i \text{ 个 } \mu\text{-递归函数总是返回} \}$$

定理 2.3.14（总是停机问题结论） ${}^A\mathcal{H}$ 是不可枚举的。

证明¹：我们通过定理 2.3.6 c 证明。显然 ${}^A\mathcal{H} \neq \emptyset$ 。接着，同样，我们使用对角论证法证明不存在全递归函数 r 将 $\mathbb{N}$ 满射到 ${}^A\mathcal{H}$ 。对任意全递归函数 $f: \mathbb{N} \mapsto {}^A\mathcal{H}$, 对应 $\mathbf{f}$, 构造全递归函数 g 对应 $\mathbf{g}(\mathbf{x}) = S({}^u\mathbf{F}^+(\mathbf{f}(\mathbf{x}), \mathbf{x}))$ 。这里 g 是全递归的因第 $f(x)$ 个 μ -递归函数是全递归的。则不存在 $f(z) = i_g$ （即 g 的下标），因否则 $\mathbf{g}(z) = S(\mathbf{g}(z))$ 。这样有 $i_g \notin {}^A\mathcal{H}$, 即 $f \neq r$ 。□

推论 2.3.14.1 存在可数但不可枚举的集合。

事实上，我们还有 $\mathcal{H} <_T {}^A\mathcal{H}$ 。其中显然有 $\mathcal{H} \leq_T {}^A\mathcal{H}$ 。而对于 ${}^A\mathcal{H} \not\leq_T \mathcal{H}$, 直观的说，是因为 ${}^A\mathcal{H}$ 的判定问题可以看作是由对应不同输入的无限多个停机问题构成的。其严格证明我们将在章节 3.2.3 通过算术阶层给出。

2.3.9 莱斯定理

现在，我们讨论一个关于图灵机行为不可判定性的重要定理：

定理 2.3.15（莱斯定理）行为满足特定非平凡性质的图灵机集合是不可判定的。其中非平凡行为性质指存在但不是全部图灵机的行为（即包括是否停机及停机时的输入输出关系，亦或指所计算的函数）都具有该性质。

证明²：由计算能力的等价性，我们只需证明对任意行为性质 P , $A = \{ i : \text{第 } i \text{ 个 } \mu\text{-递归函数行为满足 } P \}$, 若 $A \neq \emptyset$ 且 $A \neq \mathbb{N}$, 则 A 是不可判定的。若 P 等于 μ -递归函数总是不返回，因 $A \neq \mathbb{N}$, 存在 $u \notin A$ ；否则因 $A \neq \emptyset$, 存在 $h \in A$ 。令第 h 个 μ -递归函数为 h 。这样，类似的，假定 A 可由 c_A 判定，则对任意自然数 x , 可构造 $\mathbf{f}_x(\mathbf{y}) = P_2^2({}^u\mathbf{F}^+(\mathbf{x}, \mathbf{x}), h(\mathbf{y}))$, 注意当 ${}^u\mathbf{F}^+(\mathbf{x}, \mathbf{x})$ 返回（即 x 个 μ -递归函数在输入 x 时返回）， $\mathbf{f}_x$ 和 h 的行为一致，其中由 h 的挑选方式， h 不总是不返回的。同时存在 g 计算 g 满足第 $g(x)$ 个 μ -递归函数为 $\mathbf{f}_x$ 。则可构造 $\mathbf{c}_H(\mathbf{x}) = \begin{cases} !c_A(\mathbf{g}(\mathbf{x})) & P = \text{总是不返回} \\ c_A(\mathbf{g}(\mathbf{x})) & P \neq \text{总是不返回} \end{cases}$ 。□

应用莱斯定理，我们有如：

- 停机问题不可判定。
- 至少存在一个输出结果为 0 的图灵机集合不可判定。
- 输出固定结果的图灵机集合不可判定。

¹证明自 Wikipedia contributors. Machine that always halts — Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/w/index.php?title=Machine_that_always_halts&oldid=1047443566. 2021, #Relationship_to_partial_Turing_machines.

²证明自 Wikipedia contributors. Rice's theorem — Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/w/index.php?title=Rice's_theorem&oldid=1047316403. 2021, #Proof_sketch.

d. 行为和特定图灵机相同的图灵机集合不可判定，意味着没有通用方法确定两个图灵机行为是否相同。

而由于大部分程序设计语言（不考虑运行时物理限制时）是图灵完备的，因此莱斯定理告诉我们没有通用的办法确定这些语言编写的程序是否具有某一特定（非平凡的）行为特点。

最后，我们明确莱斯定理的应用场景限制，并在下一小节中给出另一个常在证明不可判定性中使用的不可判定问题。对于应用限制，具体的说，一是图灵机的行为性质并不指图灵机本身的性质，如图灵机的指令数量，显然包含特定数量指令的图灵机集合是可判定的。二是不可判定性针对的是全部图灵机，或全部图灵完备计算模型下的计算设备集合。莱斯定理不适用于对全部图灵机的真子集讨论，即意味着不适用于图灵不完备的计算模型讨论。本质上，莱斯定理的证明在这里不适用的原因是可用于判定图灵机真子集中机器行为性质的计算设备并不一定能处理如证明中所构造的 f_x 所对应的图灵机。例如限制纸带最大长度的图灵机的行为性质均是可判定的（因其机器状态是有限的）。下面给出一个更具体的例子。

回顾 0-型文法由有限终结符集 T ，非空有限非终结符集 N 满足 $N \cap T = \emptyset$ ，开始符号 $S \in N$ 和产生式规则集 $P \subseteq (T \cup N)^+ \times (T \cup N)^+$ 。我们首先证明对任意可枚举集合，都存在 0-型文法其生成的语言与之对应，接着证明所有 0-型文法生成的语言都是可枚举的。直观的说，这样我们建立了可枚举集合和 0-型文法生成语言之间表达能力上的某种等价关系。事实上由 0-型文法生成的语言也称为可枚举语言或图灵可识别语言，因此，集合是可枚举的等于其是图灵可识别或简称可识别的。进一步在此基础上，我们举例说明何时可以，以及怎么应用莱斯定理证明可枚举语言的某些性质是不可判定的，并给出其中无法应用莱斯定理的性质的不可判定性证明方法。

引理 2.3.16 对任意可枚举集合 $A \subseteq \mathbb{N}$ ，构造对应字符序列集合 $A_L = \{ \epsilon \overbrace{t \dots t}^n \epsilon : n \in A \}$ ，存在 0-型文法 G 满足其生成语言 $\mathcal{L}(G) = A_L$ 。

证明：令结构化图灵机 c' 计算 $c' : A \mapsto \{1\}$ ，令 Q_0 表示 c' 的当前状态集合， m 等于 Q 中状态最大下标， $t = m + 1$ ， $Q = Q_0; q_t$ 。令 I_0 表示指令集合， $I_1 = \{ q_i x x q_t : I_0 \text{ 中不存在指令以 } q_i x \text{ 开始} \}$ ，令 $I = I_0 \cup I_1$ ，即为所有会停机的机器状态添加一条指令使最终停机时的当前状态等于 q_t 。

令终结符集 $T = \{ \epsilon, t \}$ ，非终结符集 $N = \{ s, x \} \cup \{ L, R, \underline{0}, \underline{1} \} \cup Q$ ，开始符号 $S = s$ 。构造产生式规则，令 $P_0 = \{ s \mapsto \epsilon x \epsilon, x \mapsto x \underline{1}, x \mapsto q_0 \}$ ，即可对任意输入 x 生成 $\epsilon q_0 \overbrace{\underline{1} \dots \underline{1}}^x \epsilon$ 。 $P_1 = \{ q_i \epsilon \mapsto q_i \underline{0} \epsilon : q_i \in Q \}$ ，即当前状态在纸带右边缘时可向外扩展 $\underline{0}$ ，注意结构化图灵机不向初始状态左侧扩展。令：

$$\begin{aligned} P_2 = & \{ q_i x \mapsto q_j y : q_i x y q_j \in I \} \\ & \cup \{ x q_i y \mapsto q_j x y : x \in \{ \underline{0}, \underline{1} \} \text{ 且 } q_i y L q_j \in I \} \\ & \cup \{ q_i x \mapsto x q_j : q_i x R q_j \in I \} \end{aligned}$$

即对应三种不同的指令。 $P_3 = \{ q_t \underline{1} \mapsto q_t t, \epsilon q_t \mapsto \epsilon, \underline{0} \epsilon \mapsto \epsilon \}$ ，即用于由 $\epsilon \underline{0} \dots \underline{0} q_t \overbrace{\underline{1} \dots \underline{1}}^y \underline{0} \dots \underline{0} \epsilon$ 生成 $\epsilon \overbrace{t \dots t}^y \epsilon$ 。令产生式规则 $P = P_0 \cup P_1 \cup P_2 \cup P_3$ ，则文法 $G = \langle T, N, S, P \rangle$ 满足 $\mathcal{L}(G) = A_L$ 。 $\square$

引理 2.3.17 对任意 0-型文法 G ， $\mathcal{L}(G)$ 是可枚举的。

证明：我们可以以类似处理指令集合，即时描述和运算描述的方式哥德尔编码产生式规则集合，生成过程中每一步上的终结符 / 非终结符序列，以及整个生成过程。则可以构造 μ -递归函数通过最小化算子对每一个语句搜索对应生成过程已尝试确定其为文法生成语句。 $\square$

称可由上下文无关文法生成的语言为上下文无关语言。

定理 2.3.18 枚举上下文无关语言的图灵机集合是不可判定的。

证明：可以证明¹由字符 abc 构成的可枚举语言 $\{a\}^n \times \{b\}^n \times \{c\}^n$ ($n \geq 1$) 不是上下文无关的。因此枚举上下文无关语言是非平凡的行为性质，因此由莱斯定理可得满足该性质的图灵机集合是不可判定的。□

换句话说，没有通用的方法确定任意图灵机枚举的语言是否是上下文无关的。类似的，有枚举上下文有关语言 / 正则语言的图灵机集合是不可判定的。

接着，我们考虑文法相关的莱斯定理不适用的场景。

如前所述，莱斯定理不能用来否定计算设备本身性质的可判定性，因此，比如对于上下文无关文法，尽管由莱斯定理定理可知上下文无关语言是不可判定的。但按照上下文无关文法构造的机械设备是可判定的，比如我们可以确定任意 μ -递归函数是否是按照引理 2.3.17 中给出的方式构造的。令 $\mathcal{F}$ 表示由这种特定方式构造的机械设备（即这里的 μ -递归函数）集合，则对任意上下文无关语言 $\mathcal{L}$ ，存在 $c \in \mathcal{F}$ 可枚举 $\mathcal{L}$ 。

造成 $\mathcal{F}$ 是可判定的而可枚举上下文无关语言的图灵机集合是不可判定的原因在于上下文无关语言不要求必须由上下文无关文法生成，即存在图灵机有其对应的 μ -递归函数不属于 $\mathcal{F}$ 且可枚举某上下文无关语言。

另一方面，可以看到 $\mathcal{F}$ 是 μ -递归函数的真子集，因此其构成的计算模型不是图灵完备的。进而我们同样不能应用莱斯定理对任意行为性质 P ，确定集合 $\{f \in \mathcal{F} : f \text{ 满足 } P\}$ 的可判定性。比如我们不能应用莱斯定理确定集合 $N_f = \{f \in \mathcal{F} : f \text{ 对应的上下文无关文法是无二义性的}\}$ 的不可判定性。

2.3.10 波斯特对应问题

事实上， N_f 是不可判定的。我们最后证明这一点，并在过程中给出另一个常被用于证明不可判定性的不可判定问题，波斯特对应问题（PCP）。简单的说，我们将归约停机问题到一种 PCP 变体 MPCP，再到 PCP，最后到 N 以证明其是不可判定的²。

首先我们定义 PCP 和 MPCP。一个 PCP 或 MPCP 实例均包括有限字符集 A ， $L = \{\langle x_i, y_i \rangle : 1 \leq i \leq n \text{ 且 } x_i, y_i \in A^+\} \text{ } (n \geq 1)$ ，其中 $x_i y_i$ 即为由 A 中元素构成的长度不为零的字符序列。PCP 实例要求确定是否存在 $k \geq 1, j_1, \dots, j_k \in \{1, \dots, n\}$ 满足 $x_{j_1} x_{j_2} \dots x_{j_k} = y_{j_1} y_{j_2} \dots y_{j_k}$ 。如对 $L_0 = \{\langle ab, b \rangle, \langle a, aa \rangle\}$ 存在 $j_1 = 2, j_2 = 1$ 有 $abab = aab$ 。而 MPCP 实例要求确定是否存在 $k \geq 1, j_2, \dots, j_k \in \{1, \dots, n\}$ 满足 $x_1 x_{j_2} \dots x_{j_k} = y_1 y_{j_2} \dots y_{j_k}$ ，即相当于要求 PCP 肯定答案中 $j_1 = 1$ 。则对 L_0 ，MPCP 的答案显然是否定的。

因 PCP / MPCP / 文法中的字符集（对于文法，为终结符集和非终结符集的并集，下同）可替换为任意基数相等的字符集，通过将其替换为某一固定格式，如由 $a_0, a_1, \dots$ 构成的字符集，我们可以对 PCP / MPCP 实例、上下文无关文法进行哥德尔编码。令：

$$\mathcal{P} = \{i : \text{第 } i \text{ 个 PCP 实例有肯定答案}\}$$

$$\mathcal{M} = \{i : \text{第 } i \text{ 个 MPCP 实例有肯定答案}\}$$

$$\mathcal{N} = \{i : \text{第 } i \text{ 个上下文无关文法是无二义性的}\}$$

同时回顾 $\mathcal{H} = \{i : \text{第 } i \text{ 个 } \mu\text{-递归函数在输入 } i \text{ 时返回}\}$ 。下面我们逐一证明归约。

¹可参考 Wikipedia contributors. Pumping lemma for context-free languages — Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/w/index.php?title=Pumping_lemma_for_context-free_languages&oldid=1048802267. 2021。

²证明自 <http://infolab.stanford.edu/~ullman/ialc/spr10/slides/tm4.pdf>。

引理 2.3.19 $\mathcal{H} \leq_{\mathcal{T}} M$ 。

证明：对任意自然数 i ， $\mathcal{T}$ 为其对应的结构化图灵机， $\varkappa = \overbrace{\underline{1} \dots \underline{1}}^i$ ，令 Q_0 等于 $\mathcal{T}$ 的当前状态集， $Q_1 = \{\#, \underline{0}, \underline{1}\}$ ， $Q = Q_0 \cup Q_1$ ； $\mathbf{q}_t$ ，其中 $t = \max\{i : \mathbf{q}_i \in Q_0\}$ ，令 I_0 等于 $\mathcal{T}$ 的指令集合， $I = I_0 \cup \{\mathbf{q}_i x x \mathbf{q}_t : I_0 \text{ 中不存在指令以 } \mathbf{q}_i x \text{ 开始}\}$ 。构造 MPCP 实例 M 有肯定答案当且仅当 $i \in \mathcal{H}$ 。具体的，首先采用字符集 $A = \{\#, \underline{0}, \underline{1}\} \cup Q$ 。

对二元组集合， $l_1 = \langle \#, \# \mathbf{q}_0 \varkappa \# \rangle$ ，其也为 MPCP 答案中要求的首位二元组，即 $x_1 = \#$ ， $y_1 = \# \mathbf{q}_0 \varkappa \#$ 。进一步，简单的说，这里构造的 M 将满足对指令执行过程中的第任意 i 步 ($i \geq 1$)，存在 $k \geq 1$ ， $j_2, \dots, j_k$ ，有 $x_1 x_{j_2} \dots x_{j_k}$ 等于由分隔符 $\#$ 连接的第 1 至 $i-1$ 步的即时描述的符号序列， $y_1 y_{j_2} \dots y_{j_k}$ 等于由 $\#$ 连接的第 1 至 i 步的即时描述的符号序列，这里第一步的即时描述即为初始描述，则对第一步取 $k = 1$ 即满足该条件。同时， M 满足在第 i 步的基础上，在没有停机的情况下，只有继续选择可对应到指令执行的二元组才能使第 i 步的即时描述出现在 $x_1 x_{j_2} \dots$ 序列上，从而使其继续保持为 $y_1 y_{j_2} \dots$ 序列的前缀。比如在第一步后，为使 $\# \dots$ 序列可以等于 $\# \mathbf{q}_0 \dots$ ，我们必须选择形如 $\langle \mathbf{q}_0 x, y \rangle$ 的二元组。

我们通过以下方式实现这两个要求。令 $L_0 = \{\langle x, x \rangle : x \in Q_1\}$ ，即用于拷贝即时描述中不受指令执行影响的纸带内容和分隔符。令：

$$\begin{aligned} L_1 = & \{ \langle \mathbf{q}_i x, \mathbf{q}_j y \rangle : \mathbf{q}_i x y \mathbf{q}_j \in I \} \\ & \cup \{ \langle x \mathbf{q}_i y, \mathbf{q}_j x y \rangle : x \in \{\underline{0}, \underline{1}\} \text{ 且 } \mathbf{q}_i y \underline{L} \mathbf{q}_j \in I \} \\ & \cup \{ \langle \mathbf{q}_i x, x \mathbf{q}_j \rangle : \mathbf{q}_i x \mathbf{R} \mathbf{q}_j \in I \} \cup \{ \langle \mathbf{q}_i \#, \mathbf{q}_j \underline{0} \# \rangle : \mathbf{q}_i \underline{0} \mathbf{R} \mathbf{q}_j \in I \} \end{aligned}$$

即对应三种指令，其中还包括对纸带向右的扩展。

最后，在到达停机状态，也就是 $\mathbf{q}_t$ 时， M 满足存在二元组可使两序列对齐。直观的说，我们以 $\#$ 分隔逐步缩短即时描述的长度直至为零，并最终对齐。具体的，令 $L_2 = \{\langle \mathbf{q}_t \underline{0}, \mathbf{q}_t \rangle, \langle \mathbf{q}_t \underline{1}, \mathbf{q}_t \rangle, \langle \mathbf{q}_t \#, \# \rangle\}$ ，则 $L = L_0 \cup L_1 \cup L_2$ 有 M 符合要求。□

写 $a_{x_1 \dots x_u}$ 表示 $a_{x_1} \dots a_{x_u}$ ， $a_{y_1 \dots y_v}$ 表示 $a_{y_1} \dots a_{y_v}$ 。

引理 2.3.20 $M \leq_{\mathcal{T}} \mathcal{P}$ 。

证明：对任意正整数 m, n ， j_{xy} (任意 xy)， k_{xy} (任意 xy)， $u_1 \dots u_n$ ， $v_1 \dots v_n$ ；任意字符集为 $A_M = \{a_1, \dots, a_m\}$ 的 MPCP 实例 M ，其二元组集合 $L_M = \{\langle a_{\mathcal{J}(i,1)} \dots a_{\mathcal{J}(i,u_i)}, a_{\mathcal{K}(i,1)} \dots a_{\mathcal{K}(i,v_i)} \rangle : i \geq 1\}$ (任意 $a_{xy} \in A_M$ 而下标 $\mathcal{J}(x,y) = x j_{xy}$ ， $\mathcal{K}(x,y) = x k_{xy}$ ；直观的说 L_M 即为任意 MPCP 实例二元组集合的一般表示)，其中首位二元组为 $\langle a_{\mathcal{J}(1,1)} \dots a_{\mathcal{J}(1,u_1)}, a_{\mathcal{K}(1,1)} \dots a_{\mathcal{K}(1,v_1)} \rangle$ 。

构造 PCP 实例 P 有肯定答案当且仅当 M 有肯定答案。具体的，采用字符集 $A_P = \{\star, \$\} \cup A_M$ 。令 $s = \langle \star a_{\mathcal{J}(1,1)} \star \dots a_{\mathcal{J}(1,u_1)} \star, \star a_{\mathcal{K}(1,1)} \star \dots a_{\mathcal{K}(1,v_1)} \star \rangle$ ， $e = \langle \$, \$ \rangle$ ；构造：

$$L_P = \{ \langle a_{x_1} \star \dots a_{x_u} \star, \star a_{y_1} \dots \star a_{y_v} \rangle : \langle a_{x_1 \dots u}, a_{y_1 \dots v} \rangle \in L_M \} \cup \{s, e\}$$

可以看到 P 的肯定答案必然以 s 开头， e 结尾。而通过添加和删除 $\star \$$ 我们可以在 M 和 P 的答案间建立严格的对应关系。□

引理 2.3.21 $\mathcal{P} \leq_{\mathcal{T}} N$ 。

证明：类似上述引理证明，对任意 PCP 实例 P ，其二元组集合 $L_P = \{\langle a_{\mathcal{J}(i,1)} \dots a_{\mathcal{J}(i,u_i)}, a_{\mathcal{K}(i,1)} \dots a_{\mathcal{K}(i,v_i)} \rangle : i \geq 1\}$ ，对其以任何固定方式排序使其中元素有对应下标。构造上下文无关文法 G 其产生式规则集为：

$$\{S \mapsto A \mid B\} \cup \bigcup \{ \{A \mapsto a_{x_1 \dots x_m} A b_i \mid a_{x_1 \dots x_m} b_i, B \mapsto a_{y_1 \dots y_n} B b_i \mid a_{y_1 \dots y_n} b_i\} : \langle a_{x_1 \dots x_m}, a_{y_1 \dots y_n} \rangle \in L_M \text{ 且下标为 } i \}$$

P 实例的肯定答案意为存在符合制定条件的 $x y$ 存在 $x = y$ 。而文法中由 $S \mapsto A$ 开始的路径可生成对应潜在 x 的所有语句，且所生产语句尾部的 b 序列确定了 x 生成过程中二元组的选择顺序（ b 序列实际对应该选择顺序的逆序），同时 b 序列又保证了仅沿 A 路径生成的语句不会造成 G 的二义性。类似的， B 路径对应所有潜在的 y ，并仅由 B 路径同样不会引起 G 二义。因此有 G 是二义性的当且仅当 P 有肯定答案。□

至此，我们完成了本书中大部分关于可计算性理论极限的介绍¹。而关于计算理论中更具现实意义的问题讨论将在后续章节给出。现在，让我们回到对形式逻辑的讨论上。

2.4 本章小节

- 若两集合内元素可建立相互的一一对应关系、即存在双射函数，则我们称这两个集合的基数大小相等。若一个集合只能和另一集合的真子集基数相等，我们称该集合基数小于另一集合。在集合元素有限时，基数严格对应集合内元素个数。在无限时，基数为集合大小的比较提供了一种确定方式。
- 由对角论证法，可以证明集合的幂集的基数严格大于该集合的基数。有无限集合的幂集严格大于该无限集合的基数，即存在不同大小或等级的无限集合。写自然数集合的基数 $|\mathbb{N}| = \aleph_0$ ，其幂集 $|\wp(\mathbb{N})| = 2^{\aleph_0}$ 。
- 对数学不同的认识方式形成了不同的数学哲学分支。如一些数学工作者认为数学对象应是可构造的，称这样的认识方式为构造主义数学。进一步，构造主义又可区分 i：要求数学对象是现实世界可构造的，并认为在现实中不存在真正无限的对象，即如不承认数学中有无限的集合，称这样的认识为有限主义。其中，又区分是否允许任意大对象的存在，称不允许的情况为极端有限主义。或 ii：要求数学对象是思维中可构造的，则承认无限的存在，但不承认不可构造的无限对象的存在，称这样的认识为直觉主义。
- 从实用主义的角度来说，从不同认识论发展而来的形式化方法仍主要在于不同领域实用性上的区别。尽管很难认为在全局上我们已经掌握了不同认识的优劣。但如当前以及接下去的几个章节所示，我们仍能给出一些较通用的研究方法，并已经了解了不少较通用的结论。
- 形式逻辑是形式化方法的核心，其研究区分目标逻辑和元逻辑。我们应用目标逻辑解决实际问题，应用元逻辑研究目标逻辑的性质，即如解决实际问题的能力。在讨论中，我们应清晰区分各概念所处的位置。一般情况下，我们使用集合论作为元逻辑。
- 形式逻辑的语法一般包括词汇和语句构成方法。其中，词汇分为逻辑符号和非逻辑符号，非逻辑符号为由用户给出的符号。语句由合法的符号序列，称表达式，构成。合法的表达式又可细分为（形式）合法的公式、简称合式/式，以及其它构成成分。其中，式和语句的区别在于式可能包含未指定语义内容的部分，即自由变量。称包含自由变量的构成成分为开放的，否则为封闭的。则封闭式即为语句。称由全部语句构成的集合为语言，则即使在同一种形式逻辑下，不同的非逻辑符号决定了不同的语言。

¹除章节 3.2.3 中关于算术阶层的相关内容外。

- 文法是描述形式化语言语句构成方式的一种系统性方法。其又可分为无限制文法，上下文有关文法，上下文无关文法以及正则文法，分别对应 0 到 3-型文法。有 0-型 $\subset$ 1-型 $\subset$ 2-型 $\subset$ 3-型，即其可描述的语言要求趋于严格，而相对的，处理难度趋于简单。
- 上下文无关文法使用语法树描述语法结构。文法决定了语法树中间节点的合法构成方式。（自左到右）依次的叶节点序列构成了语句。对语法树的先根遍历对应派生过程，即语句的生成过程；后根遍历对应规约过程，即语句的分析过程。存在语句可对应不同（合法）语法树的文法为有二义性的，即可能歧义的。显然，形式逻辑中我们要求语言（的文法）是没有二义性的，但没有可有效计算的方法确定上下文无关文法是否具有二义性。最后，二义性可通过如指定语法树的方式消除。
- 语言中语句在元逻辑下的含义称为语义。语言中非固定含义的符号称为参数，可见非逻辑符号一般均为参数。对各个参数的赋值称为解释。解释结合非参数符号的固定含义以及语句构成法决定了语言中所有语句的含义。这些语句（语义上）的含义被分为多个类别。如二值逻辑中，被分为真和假两类。则本质上，形式逻辑的目标是从一系列已知类别的语句确定未知类别语句的类别。如二值逻辑中，从已知为真的语句得到新的为真的语句，即从已知事实得到新的事实。
- 二值逻辑中，我们称令某语句为真的解释为该语句的模型，称令某语句集为真的解释为该语句集的模型。对任意语句 / 语句集，其可能没有模型，可能只有一个模型，也可能有多个（从目标逻辑角度看亦不相同的）模型。我们称没有模型的语句集为不可满足的、或不一致的。而在有如上述多个模型时，（目标逻辑中）存在语句在不同模型中有不同真假。对任意语句集 Γ ，若有语句 σ 在 Γ 的全部模型中均为真，则称 Γ 语义蕴含或恒真蕴含 σ ，写 $\Gamma \models \sigma$ 。当语句 σ 在任意模型中为真，即 Γ 为空时，称 σ 为有效的，或恒真的。当两语句间互相有语义蕴含关系时，称其语义等价。
- 我们称由可有效计算的方法自一个语句集 Γ 肯定另一个语句 σ 的过程为演绎或演算。称其中采用的方法为演绎规则，称 Γ 中的语句（在不同语境下）为前提、假设或公理，称 σ 为推论、引理或定理。称 Γ 的全部定理为由 Γ （及采用的形式逻辑）确定的理论。称其肯定的结果为 Γ 语法蕴含或逻辑蕴含 σ ，写作 $\Gamma \vdash \sigma$ ，也称由 Γ 证明 σ 。当两语句间互相有语法蕴含关系时，称其语法等价。演绎规则的另一个有效应用前提为 Γ 是可有效计算的，即存在有效计算方法确定任意语句是否属于 Γ ，我们称这样的 Γ 为可公理化的。 Γ 有限时，称是有限可公理化的。
- 若一个演绎规则对任意语句集 Γ ，语句 σ ，有若 $\Gamma \vdash \sigma$ ，则 $\Gamma \models \sigma$ ，称该规则为可靠的；有若 $\Gamma \models \sigma$ ，则 $\Gamma \vdash \sigma$ ，称为完备的。显然，可靠且完备的演绎规则使我们可以在目标逻辑中计算内容上的元逻辑中语句间的真假关系。同时，可靠且完备的演绎规则下，语法蕴含和语义蕴含、语法等价和语义等价是一致的，简称蕴含和等价。
- 由语句在不同模型下有不同真假可知，存在语句是无法肯定也无法否定的，即不可证明也不可证伪的，称这样的语句为独立的。而对任意语句均能肯定或否定的前提集，我们称其为完备的。则显然完备的前提集至多对应一个（目标逻辑下不同的）模型，有其中对应一个模型的前提集是最大一致的。尽管在一般情况下如 $\Gamma \not\models \sigma$ 不等于 $\Gamma \vdash \neg \sigma$ ，但在 Γ 一致且完备时，有 $\Gamma \not\models \sigma$ 等于 $\Gamma \vdash \neg \sigma$ 。
- 称语言中符合特定模式并可用于取代其它语句以简化处理的语句为范式。
- 命题逻辑是一种最小在语句粒度上处理蕴含关系的二值形式逻辑。其非逻辑符号可解释为任意语句，或本质上，这些语句对应的真假值。其可由逻辑联结词 $\neg \wedge \vee \rightarrow \leftrightarrow$ 组成更复杂的语句。其中的一些逻辑联结词可由另一些定义，我们称可定义全部逻辑联结词的联结词集合为功能完备的，如 $\{\neg, \rightarrow\}$ 即为功能完备的。则在分析中，我们仅需分类讨论功能完备集合中的联结词元素。
- 命题逻辑可用真值表作为演绎规则，在前提即有限的情况下，该方法不仅可以得到如 $\Gamma \vdash \sigma$ ，亦可以确定 $\Gamma \not\models \sigma$ 。

- 命题逻辑中常用的范式有析取范式和合取范式，析取范式由以语句符号及其否定构成的合取子式做析取联结得到，合取范式相反。我们可以在析取范式的基础上得到另一种演绎规则。
- 由邱奇-图灵假说，可有效计算的方法即为可由图灵机或等价机械设备执行的方法。称图灵机这样的机械设备为计算设备，称符合一定要求的一类计算设备为计算模型。则具体的图灵机为计算设备，而（泛指的）图灵机为一种计算模型。
- 具体的图灵机为由一根两端可任意扩展的、按固定间隔划分且其上可填写不同符号的单元格的纸带，指向单元格以在其上填写不同符号的指针，当前状态，以及一组由当前状态、当前指针所指单元格内容决定当前应执行动作、包括填写符号、左右移指针和 / 或改变当前状态的指令构成。
- 图灵机的计算运行过程由初始状态及初始纸带内容起按指令所确定动作执行直至停机，并由最后的纸带内容确定该次计算的结果；或永不停机。称纸带初始内容为输入，停机后的内容为输出。称当前状态、指针位置及纸带内容为即时描述，称运行前的即使描述为输入描述，停机后的为输出描述。称输入输出的具体含义到计算设备中具体表示形式的转换为编解码。
- 称两个计算设备是等价的若其对含义相同的输入有含义相同的输出。称计算模型 $\mathcal{A}$ 的计算能力小于等于计算模型 $\mathcal{B}$ ，若 $\mathcal{A}$ 中的任意计算设备在 $\mathcal{B}$ 中有等价的计算设备。当两个计算模型计算能力相等时，称其为等价的。如两个符号的图灵机（计算模型）和任意多个符号的图灵机（计算模型）是等价的。称和图灵机等价的计算模型为图灵完备的。
- μ -递归函数是另一种图灵完备的计算模型，其由后继、常量和投影三种基础函数，复合、原始递归和极小化三种算子构成。其中，不含极小化算子的 μ -递归函数是总是返回的、即有输出的，称原始 μ -递归函数。
- 相比于图灵机更着重于可实际被搭建， μ -递归函数更接近对数学操作的抽象表达。通过在 μ -递归函数中表达图灵机的计算过程，可以证明其计算能力是不小于图灵机的。同时，这种对计算过程的表达也意味着存在通用 μ -递归函数 / 通用图灵机可模拟任意图灵机的计算过程。
- 哥德尔编码可用于将任意有限长自然数序列编码为单个自然数。在如任意复杂的计算过程时，我们可以应用哥德尔编码使其复杂度可由单个自然数涵盖。通过进一步给定语句中符号到自然数的对应关系，我们可以将任意有限长语句编码为单个自然数。即我们可通过自然数编码任意输入输出，并在此基础上讨论可计算性。
- 对自然数上的函数 f ，若存在图灵机 / μ -递归函数有其输入输出符合该函数的映射关系，则称 f 是部分可计算 / 部分递归的。若 f 全函数，则称（全）可计算 / （全）递归的。若对应的 μ -递归函数为原始 μ -递归函数，则称原始递归的。存在阿克曼函数是全递归但不是原始递归的。
- 对自然数子集合 A ，若其特征函数 $c : \mathbb{N} \rightarrow \{0, 1\}$ 是全递归的，则称 A 是可判定的，（全）可计算，（全）递归的；若 c 是原始递归的，称 A 是原始递归的。若函数 $c' : A \mapsto \{1\}$ 是部分递归的，则称 A 是（计算 / 递归）可枚举的，半可判定的，半可计算的，或半递归的。可进一步扩展以上概念至编码前的函数和集合。
- 停机问题给出了半可判定（即可枚举）但不可判定的自然数子集合的具体例子。总是停机问题给出了不可枚举自然数子集合的具体例子。
- 预言者是可给出任意、尤其是不可判定问题答案的理想计算设备。预言机指包含对预言者查询的计算设备。若存在含预言者 A 的预言机部分 / （全）可计算函数 f （的对应问题），称 f （及对应问题）是 A 部分 / （全）可计算的，或 A 部分 / （全）递归的。可类似扩展以上及可判定性相关概念至集合。

- 若集合 A 是 B 可判定的，称 A 可图灵归约到 B 。即若可解决 B 相应问题，则可解决 A 相应问题。图灵归约是一种非严格偏序关系，我们使用图灵度标记这种偏序关系上的具体位置，可认为其代表了问题的难易程度。若 A B 可互相图灵归约，称 A B 等价。
- 任意集合 A 经图灵跳跃后产生相对于 A 的停机问题 A' 。 A 可图灵归约到 A' ，但不图灵等价于 A' ，即可认为 A' 是严格难于 A 的。序列 $\emptyset^n$ 表示自空集 ($n = 0$) 其经不断图灵跳跃得到的相对停机问题序列，其对应的图灵度集合为全部图灵度集合的真子集。
- 莱斯定理指出满足非平凡行为性质的图灵机集合是不可判定的。在不关于图灵机集合时，我们可通过尝试将未知可判定性的问题图灵规约至已知可判定性问题已确定其的可判定性。波斯特对应问题 PCP 即是另一种常用的已知不可判定的问题。通过图灵归约至 PCP，可证明上下文无关文法的二义性是不可判定的。

第三章 框架：一阶逻辑

本章主要讨论最常用的形式逻辑之一：一阶逻辑。其将作为我们下一章节的框架并支撑起集合论和元逻辑的形式化。3.1 主要介绍一阶逻辑的语法语义及演绎规则，证明其演绎规则的可靠性和完备性，并给出一些演绎过程相关的重要性质。3.2 节通过一阶算术一方面展示一阶逻辑的表达能力，另一方面以其为例给出形式逻辑系统的重要限制。特别的，一方面我们在一阶算术语句语义和集合可判定性间建立联系，另一方面又通过哥德尔不完备定理展示其本身语法和语义相关概念间的差异。3.3 节扩展一阶逻辑至高阶逻辑，我们会看到尽管高阶逻辑有更强的表达能力，其没有严格意义上可靠且完备的演绎规则。而在放宽语义要求后，其本质上与一阶逻辑等价。

3.1 一阶逻辑

一阶逻辑也被称为谓词逻辑。其原因在于一阶逻辑的语句在最基本的组成方式上也一般区分主语和谓语。其中主语可被认为是语句中（主要）的名词性成分，谓语是形容词/动词等。且一阶逻辑也允许一个语句中包含任意多个名词性成分。相比于命题逻辑，这种语句的构成方式带来了一阶逻辑的第一个好处，即当需要对多个不同的句子成分进行组合时，一阶逻辑所需的词汇量往往远小于命题逻辑。比如对于 10 以内自然数的小于关系描述，命题逻辑需要 $A_0 = 0 < 1, \dots, A_8 = 0 < 9, \dots, A_{24} = 3 < 4, \dots, A_{44} = 8 < 9$ 这样 45 个语句符号。而一阶逻辑只需要符号 $0 \dots 9$ 和 $<$ 。

此外，一阶逻辑引入了量词，可分为全称量词和存在量词。全称量词允许我们在纯形式的构成上表达关于任意的概念，比如可以构成语句表达 0 小于任意其它自然数 并只在形式构成上即可得 $0 < 1, 0 < 2, \dots$ ，或相当于 $\bigwedge_{i \neq 0} 0 < i$ 。从而可以通过有限长度的语句表达无限多的事实。这在命题逻辑中是做不到的¹。而对于存在量词，其允许我们构造语句等价于如 $\bigvee_i 0 \geq i$ 。即除可以使表达更加简练外，量词的存在还带来了表达能力上的提升。

然而，一阶逻辑之所以被称为一阶，是因为其可被量词量化的内容仅限于上述概念中的名词性内容，或者说具体的对象。换句话说，一阶逻辑允许表达如任意对象满足某一性质，但不能表达如任意性质被某一对象满足。相对的，二阶逻辑允许量化性质，同时量化性质也使我们允许对被量化的性质进行性质描述，或者说允许我们谈论（任意）性质的性质。自然的，以此类推，我们有三阶逻辑进一步允许量化性质的性质等等。我们统称二阶及以上逻辑为高阶逻辑。我们将在章节 3.3 中介绍以二阶逻辑为代表的高阶逻辑。同时，最后，在这样的命名法下，命题逻辑也可被称为零阶逻辑。

3.1.1 语法和词汇

一个具体的功能完备的一阶逻辑语言（可）包括如下逻辑符号：

¹ $\bigwedge_{i \neq 0} 0 < i$ 不是合法的命题逻辑语言，而直接令语句符号如 A 对应 0 小于任意其它自然数 并不能从形式构成上得到 $\bigwedge_{i \neq 0} 0 < i$ 。

- 逗号和括号, ()。
- 形式上（及语义功能上）和命题逻辑相同的逻辑联结词 $\neg \rightarrow$ 。
- 全称量词, 任意 $\forall$ 。
- 变量符号 $x y z x' x'' x_0 x_1 x_2 x'_0$ 等。

如下非逻辑符号：

- 常量符号 $a b c$ 等。我们使用粗体和变量符号加以区分。
- 函数符号, 并进一步对任意正整数 n , 区分 n 位函数符号。函数符号一般写作 $F G F_0 F_1$ 等, 但语句中的函数符号及其位数可通过语法构成确定。这里不允许零位函数, 但零位函数本质上即等于常量。同时, 我们总是可以通过忽略高位函数的中某些位内容而使其对应低位函数, 也就是通过允许如二位函数, 我们即涵盖了一位函数提供的所有表达能力。
- 谓词符号, 同样进一步对任意正整数 n , 区分 n 位谓词符号。一般写作 $P Q P_0 P_1$ 等, 同样由语法可在语句中确定谓词符号及其位数。同样这里也不允许零位谓词, 事实上, 若允许零位谓词, 其即对应命题逻辑中的语句符号。仍然同样的, 高位谓词涵盖了低位谓词的表达能力, 亦即一阶逻辑涵盖了零阶逻辑、即命题逻辑的表达能力。

接着, 我们给出一阶逻辑的语法。同样称符号序列为表达式。随定义我们区分表达式的开放和封闭两种情况。简单的说, 开放表达式含自由变量, 或者说其中有变量符号是自由的、即未指定具体内容的; 否则有其中的变量符号都是被约束的, 称为约束变量。具体我们通过在定义过程中给出表达式包含的变量符号情况阐明, 对表达式 x , $f(x)$ 表示 x 中所包含自由变量集合, $b(x)$ 表示约束变量集合。这样表达式 x 是开放的当且仅当 $f(x) = \emptyset$ 。定义中, 当省略被定义表达式的某变量集合时, 该变量集合为 $\emptyset$:

- 项
 - 任意变量符号 x 构成项, $f(x) = \{x\}$ 。
 - 任意变量符号 a 构成项。
 - 对 n 位函数符号 F , 若 $\tau_{1\dots n}$ 为项, $F(\tau_{1\dots n})$ 为项。令其为 τ , $f(\tau) = \bigcup_{1 \leq i \leq n} f(\tau_i)$ 。也称 $\tau_{1\dots n}$ 为 τ 中 F 的参数。
- 式
 - 对 n 位谓词符号 P , 若 $\tau_{1\dots n}$ 为项, $P(\tau_{1\dots n})$ 为式, 令其为 φ , $f(\varphi) = \bigcup_{1 \leq i \leq n} f(\tau_i)$ 。也称 $\tau_{1\dots n}$ 为参数。称 φ 为原子式。
 - 类似命题逻辑的, 若 α 为式, $\neg \alpha$ 为式。令其为 φ , $f(\varphi) = f(\alpha)$, $b(\varphi) = b(\alpha)$ 。
 - 若 $\alpha \beta$ 为式, 且 $f(\alpha) \cap b(\beta) = f(\beta) \cap b(\alpha) = \emptyset$, 则 $(\alpha \rightarrow \beta)$ 为式。令其为 φ , $f(\varphi) = f(\alpha) \cup f(\beta)$, $b(\varphi) = b(\alpha) \cup b(\beta)$ 。
 - 若 α 为式, x 为变量, 且 $x \notin b(\alpha)$, 则 $(\forall x \alpha)$ 为式。令其为 φ , $f(\varphi) = f(\alpha) \setminus x$, $b(\varphi) = b(\alpha); x$ 。注意, 这里和 $\rightarrow$ 中关于变量集合的要求仅为排除语句如 $\forall x \forall x P(x)$ 及 $(\forall x (P(x) \rightarrow (\forall x Q(x))))$ 的效果, 这样的语句不增加表达能力¹且（如在演绎规则中）处理繁琐²。但注意我们仍允许 $((\forall x P(x)) \rightarrow (\forall x Q(x)))$ 。

我们称封闭式为语句。其为一阶逻辑, 或更一般的, 大部分形式逻辑演绎的基本单位³。

¹如下一小节的语义定义可知, 我们总是可以替换任意约束变量的具体符号而不改变语句的含义。

²但尽管这种额外要求使如对演绎规则的描述变得简洁直观, 语句在执行组合过程中（如定义的展开中）仍可能需要替换约束变量以避免违反该要求。可以看到其实现是简单的, 因而我们在下文中将并不总是赘述这一点。

³在命题逻辑中, 式和语句的概念是重合的。

此外，我们以类似命题逻辑且满足 $\rightarrow$ 的额外要求的方式定义 $\wedge \vee \leftrightarrow \perp$ ，并以类似方式省略逻辑联结词间的括号使用。并定义：

- 存在量词 $\exists$ ，对任意式 α ，变量 x ， $(\exists x \alpha) = \neg (\forall x \neg \alpha)$ 。

且我们同样允许省略量词相关的括号使用，我们令 $\forall \exists$ 有相等的、低于 $\wedge \vee$ 高于 $\leftrightarrow \rightarrow$ 的优先级，且右结合，即 $\forall x \exists y \forall z \alpha \vee \beta \rightarrow \gamma = (\forall x (\exists y (\forall z (\alpha \vee \beta))) \rightarrow \gamma)$ 。

最后引入简写。对任意变量 xy ，式 α ，写 $\forall x, y \alpha$ 表示 $\forall x \forall y \alpha$ ，则对任意 n ，任意变量符号 $x_1 \dots x_n$ ，式 α ，有 $\forall x_1 \dots x_n \alpha$ 表示 $\forall x_1 \dots \forall x_n \alpha$ 。并以类似方式简写 $\exists$ 式。

3.1.2 语义

直观的说，项对应自然语言中的名词性概念，式对应可能带有自由变量留白的句子，而语句即不包含这些留白。并且同样，一阶逻辑的语句本质上对应真/假，即对应是否符合事实。因此显然带有留白的句子是无法判断其是否是符合事实的。例如，若谓词符号 P 代表下雨，则式 $P(x)$ 代表 x 下雨，或__下雨。而若常量符号 a 代表今天，则式 $P(a)$ 代表今天下雨，即 $P(a)$ 为语句。若函数符号 F 代表下一天，则 $P(F(a))$ 代表明天下雨。最后若我们关注每一天的情况， $\forall x P(x)$ 代表每天下雨，而 $\exists x P(x)$ 代表存在某些天下雨，即不是每一天都不下雨，即可以看到 $\exists x P(x) = \neg \forall x \neg P(x)$ 。

现在我们给出严格的定义。

一阶逻辑语言的表达能力由其变量符号和非逻辑符号数量决定，我们仍然将讨论限制在可枚举的范围内，且不限变量符号的数量。则在确定了非逻辑符号数量之后，一阶逻辑的语言参数、即可由用户解释的部分包括全称量词 $\forall$ 和全部的非逻辑符号。具体的：

- 全称量词 $\forall$ 的解释代表了用户希望讨论的具体对象集合 A ，称域，要求 $A \neq \emptyset$ 。
- 常量符号对应 A 中的元素，这里并不要求有严格的一一对应关系（函数/谓词符号同）。即可以有不同的常量符号对应相同的元素，也可以有元素没有常量符号对应。
- 函数符号对应 A 中元素之间函数关系， n 位函数符号对应函数 A^n 到 A 。
- n 位谓词符号对应 A^n 的某一子集，即可解释为 A 的符合某一性质的子集（ $n = 1$ ）或 A 上的多元关系（ $n > 1$ ）。

这样，我们就给出了任意给任一阶逻辑的解释组成。进一步，我们考虑组合和真值，给定解释 $\mathfrak{I}$ ，写 $\mathfrak{I}[x]$ 表示全称量词或封闭项或封闭式 x 在 $\mathfrak{I}$ 下对应的内容：

- 对 n 位函数符号 F ，封闭项 $\tau_{1\dots n}$ ，要求 $\mathfrak{I}[F(\tau_{1\dots n})] = \mathfrak{I}[F](\mathfrak{I}[\tau_1], \dots, \mathfrak{I}[\tau_n])$ 。
- 对 n 位谓词符号 P ，封闭项 $\tau_{1\dots n}$ ，要求 $P(\tau_{1\dots n})$ 为真当且仅当 $\langle \mathfrak{I}[\tau_1], \dots, \mathfrak{I}[\tau_n] \rangle \in \mathfrak{I}[P]$ 。
- 由逻辑联结词构成的式的真值定义与命题逻辑相同。
- 对任意变量符号 x ，式 α ， $\forall x \alpha$ 为真当且仅当在用域 $\mathfrak{I}[A]$ 中的每一个元素替换式 α 中的变量符号 x 后对应的语句均为真。

该语义定义被称为塔斯基真值定义。下面让我们来看几个例子：

- 如对含 $A B$ 两个语句符号的命题逻辑 $\mathcal{P}$ 的讨论，我们有一阶逻辑语言 $\mathcal{L}_{\mathcal{P}}$ ：

- 常量符号 a, b 。
- 一位函数符号 $F_{\neg}$ ，二位函数符号 $F_{\rightarrow}$ 。
- 一位谓词符号 T 。

采用解释 $\mathfrak{I}$:

- $\mathfrak{I}[\forall]$ 等于命题逻辑 $\mathcal{P}$ 下的所有式（即语句）集合 S 。
- $\mathfrak{I}[a] = A$, $\mathfrak{I}[b] = B$ 。
- $\mathfrak{I}[F_{\neg}]$ 等于函数 $f_{\neg}: S \mapsto S$ 有 $f_{\neg}(x) = \neg x$ 。 $\mathfrak{I}[F_{\rightarrow}]$ 等于函数 $f_{\rightarrow}: S^2 \mapsto S$ 有 $f_{\rightarrow}(x, y) = x \rightarrow y$ 。
- $\mathfrak{I}[T]$ 等于语句恒真，即对封闭 τ , $T(\tau)$ 为真当且仅当 $\mathfrak{I}[\tau]$ 恒真。

则对 $\mathcal{L}_p$ ，我们有如 $\models_{\mathfrak{I}} \{T(a) \leftrightarrow T(F_{\neg}(F_{\neg}(a))), \forall x T(x) \leftrightarrow T(F_{\neg}(F_{\neg}(x)))\}$ 。回顾章节 2.2.3，即有 $\mathfrak{I}$ 为右侧语句集合的模型。

- 仅含等于符号的一阶逻辑语言 $\mathcal{L}_E$:

- 二位谓词符号，等于 $=$ 。

则对任意解释 $\mathfrak{I}$ 满足 $\mathfrak{I}[=]$ 为等于关系，有:

1. $|\mathfrak{I}[\forall]| > 1$ 当且仅当 $\models_{\mathfrak{I}} \forall x \exists y x \neq y$ 。
2. $|\mathfrak{I}[\forall]| > 2$ 当且仅当 $\models_{\mathfrak{I}} \forall x_1, x_2 \exists y x_1 \neq y \wedge x_2 \neq y$ 。
3. 对任意 $n \geq 1$, $|\mathfrak{I}[\forall]| > n$ 当且仅当 $\models_{\mathfrak{I}} \forall x_1 \dots x_n \exists y \bigwedge_{1 \leq i \leq n} x_i \neq y$ 。
4. 对任意 $n \geq 1$, $|\mathfrak{I}[\forall]| \leq n$ 当且仅当 $\models_{\mathfrak{I}} \forall x_0 \dots x_n \bigvee_{0 \leq i \leq n} \bigvee_{0 \leq j \leq n \wedge j \neq i} x_i = x_j$ 。
5. 则对任意 $n \geq 1$ ，可构造语句 γ 有 $|\mathfrak{I}[\forall]| = n$ 当且仅当 $\models_{\mathfrak{I}} \gamma$ 。

- 用于讨论基本算术理论的一阶逻辑语言 $\mathcal{L}_A$ ，这里也称其为算术语言:

- 常量符号 0 。
- 一位函数符号，后继 S 。二位函数符号，加 $+$ ，乘 $\cdot$ 。
- 二位谓词符号，等于 $=$ 。

另我们可以在 $\exists$ 和 $+$ 的基础上定义 $<$ 符号¹。

1. 考虑解释 $\mathfrak{N}_{st}$:

- $\mathfrak{N}_{st}[\forall] = \mathbb{N}$ 。
- $\mathfrak{N}_{st}[0]$ 等于自然数 0 。
- $\mathfrak{N}_{st}[S]$ 等于函数 $s: \mathbb{N} \mapsto \mathbb{N}$ 有 $s(x) = x + 1$ 。即如 $\mathfrak{N}_{st}[S(0)] = 1$, $\mathfrak{N}_{st}[S(S(0))] = 2$ 。 $\mathfrak{N}_{st}[+]$ 等于自然数加法， $\mathfrak{N}_{st}[\cdot]$ 等于自然数乘法。
- $\mathfrak{N}_{st}[=]$ 为等于关系。

则如语句 $+(S(0), S(S(0))) = S(S(S(0)))$ 表示 $1+2=3$ ，即有 $\models_{\mathfrak{N}_{st}} +(S(0), S(S(0))) = S(S(S(0)))$ 。

此外，我们还有如 $\models_{\mathfrak{N}_{st}} \forall x \exists y y = S(x)$, $\models_{\mathfrak{N}_{st}} \forall x \forall y \exists z z = +(x, y)$ 等。

2. 解释 $\mathfrak{I}$:

- $\mathfrak{I}[\forall]$ 等于 $A = \{0, 1, 2\}$ 。
- $\mathfrak{I}[0] = 0$ 。
- $\mathfrak{I}[S]$ 等于函数 $s = \{\langle 0, 1 \rangle, \langle 1, 2 \rangle, \langle 2, 0 \rangle\}$ 。即仍有 $\mathfrak{I}[S(0)] = 1$, $\mathfrak{I}[S(S(0))] = 2$ ，但 $\mathfrak{I}[S(S(S(0)))] = 0$ 。
- $\mathfrak{I}[+]$ 和 $\mathfrak{I}[\cdot]$ 分别等于自然数加法和乘法的域等于 A 的最大子集。
- $\mathfrak{I}[=]$ 为等于关系。

¹见章节 3.2.2。

则我们仍然有 $\models_{\mathfrak{A}} \forall x \exists y y = S(x)$, 但 $\not\models_{\mathfrak{A}} \forall x \forall y \exists z z = +(x, y)$ 。由此可以看到一个 $\mathcal{L}_A$ 的语句（集）可以有多个模型，而增加语句会减少模型的数量。此外，我们称 $\mathfrak{A}_{st}$ 为基本算术语言 $\mathcal{L}_A$ 的标准解释。其也为任意基本算术理论的模型，因此也被称为标准模型。同时，对任意模型 $\mathfrak{A}$ ，写 $\text{Th}(\mathfrak{A})$ 表示理论 $\{\alpha : \models_{\mathfrak{A}} \alpha\}$ ，即 $\mathfrak{A}$ 下为真的语句集合。则我们称一阶逻辑下的算术理论 $\text{Th}(\mathfrak{A}_{st})$ 为真算术。可以看到真算术理论是一致且完备的¹。

最后，就如集合论中定义函数是一种特殊的关系，这里的函数符号也可被谓词符号替换，为完成这种替换，我们的逻辑系统首先需要包含（通过公理或演绎规则²）保证其可赋予的语义满足等于关系基本要求的谓词符号，如 $=$ 。写 $x = y$ 表示 $=(x, y)$ 并令其有最高优先级，写 $x \neq y$ 表示 $\neg x = y$ 。这样，对如一位函数符号 F ，我们可将其替换为谓词符号 R_F 并要求 $\forall x \exists y R_F(y, x)$ 及 $\forall x, y_1, y_2 R_F(y_1, x) \wedge R_F(y_2, x) \rightarrow y_1 = y_2$ ，即对任意 x 有且仅有唯一 y 满足 $R_F(y, x)$ ，即 $R_F(y, x)$ 表示 $y = F(x)$ 。

3.1.3 演绎规则

类似命题逻辑，一阶逻辑有多种不同的演绎规则。这里主要介绍根岑形式演绎系统和希尔伯特形式演绎系统，其中根岑形式演绎系统又分为自然演绎和相继式演绎。这些演绎系统都有着类似的框架，我们首先介绍最接近日常证明的自然演绎，并在过程中给出公共概念的定义。接着我们讨论希尔伯特演绎以示系统间的区别。在下一小节中，我们给出相继式演绎并在此基础上证明可靠和完备性。

自然演绎的命名源于其演绎过程符合人们证明时的自然习惯。这也反映了我们将证明定义为演绎过程的原因。自然演绎由一系列称为形成规则的基础推导规则构成。形式逻辑中，推导规则的一般形式为：

$$\frac{\text{输入}_1 \quad \dots \quad \text{输入}_n}{\text{输出}} \text{ 规则名}$$

也就是我们通过这些规则从已知的输入中得到输出³，其中 n 可以等于零。且这里的输入和输出不一定是语句，而是有一定结构的语句组合。我们通过自然演绎给出具体的例子。

在自然演绎中，输入和输出有形式 α^H ⁴，其中 α 为任意语句，为该表示法中的推论部分。 H 为有限语句集合，为假设集合，这些假设可以是证明目标中的前提，也可以是为证明额外引入的，见下方示例。同样如下面所见，输出 α^H 即表示有 $H \vdash \alpha$ ，这里的上标形式仅为强调推论部分。此外，自然演绎的形成规则基本可分为引入（ I ）和消除（ E ）两类，其对应引入和消除假设或特定的逻辑符号。最后，对任意项 τv ，语句 α ，写 $\alpha_{[\tau/v]}$ 表示用 v 替换 α 中的所有 τ 。下面我们给出这些规则：

- 假设，对任意语句 α ：

$$\frac{}{\alpha^{\{\alpha\}}} H_I \quad (i)$$

即表示在没有任何前提的情况下，我们可以通过假设 α 得到 α 。即总是有 $\alpha \vdash \alpha$ 。下面我们写 $[\alpha]$ 表示 (i)。

- 合取，对任意语句 $\alpha \beta$ ，假设集合 $H_0 H_1$ ：

¹若语句 α 在 $\mathfrak{A}_{st}$ 下为假，则 $\neg \alpha$ 为真， $T \vdash \neg \alpha$ 。

²细节见章节 3.2.1。

³Coq 输出的上下文（图 1.3）即采用了这种表示法，不过在 Coq 中我们需要填充的是由输入到输出所需的规则（序列），这也是一般证明要求我们给出的。

⁴这里给出的形式和一般形式略有区别，更明确了上下文，也更接近相继式演绎规则。这也是相继式演绎提出的原因之一。

$$\frac{\alpha^{H_0} \quad \beta^{H_1}}{\alpha \wedge \beta^{H_0 \cup H_1}} \wedge_I \qquad \frac{\alpha \wedge \beta^{H_0}}{\alpha^{H_0}} \wedge_{E1} \qquad \frac{\alpha \wedge \beta^{H_0}}{\beta^{H_0}} \wedge_{E2}$$

即如左侧，我们可以在已知由 H_0 有 α 和由 H_1 有 β 的情况下，得出由 $H_0 \cup H_1$ 有 $\alpha \wedge \beta$ 。事实上，通过这几条规则我们已经可以得到比如 $\alpha \wedge \beta \vdash \beta \wedge \alpha$ ，我们可以以如下形成树（或更一般的，称推导树）的形式表现演绎过程：

$$\frac{\frac{[\alpha \wedge \beta]}{\beta \{ \alpha \wedge \beta \}} \wedge_{E2} \quad \frac{[\alpha \wedge \beta]}{\alpha \{ \alpha \wedge \beta \}} \wedge_{E1}}{\beta \wedge \alpha \{ \alpha \wedge \beta \}} \wedge_I$$

- 析取，对任意语句 $\alpha \beta \gamma$ ，假设集合 $H_0 H_1 H_2$ ：

$$\frac{\frac{\alpha^{H_0}}{\alpha \vee \beta^{H_0}} \vee_{I1} \quad \frac{\alpha \vee \beta^{H_0} \quad \alpha \rightarrow \gamma^{H_1} \quad \beta \rightarrow \gamma^{H_2}}{\gamma^{H_0 \cup H_1 \cup H_2}} \vee_E}{\frac{\beta^{H_0}}{\alpha \vee \beta^{H_0}} \vee_{I2}} \vee_E$$

- 条件，对任意语句 $\alpha \beta$ ，假设集合 $H_0 H_1$ ：

$$\frac{\alpha^{H_0}}{\beta \rightarrow \alpha^{H_0 \setminus \beta}} \rightarrow_I \qquad \frac{\alpha \rightarrow \beta^{H_0} \quad \alpha^{H_1}}{\beta^{H_0 \cup H_1}} \rightarrow_E$$

让我们来看一个引入额外非前提假设的例子， $\alpha \rightarrow (\beta \rightarrow \gamma) \vdash \beta \rightarrow (\alpha \rightarrow \gamma)$ ，其中 $[\alpha] [\beta]$ 为引入额外的假设：

$$\frac{\frac{[\alpha \rightarrow (\beta \rightarrow \gamma)] \quad [\alpha]}{\beta \rightarrow \gamma \{ \alpha, \alpha \rightarrow (\beta \rightarrow \gamma) \}} \rightarrow_E \quad [\beta]}{\frac{\gamma \{ \alpha, \beta, \alpha \rightarrow (\beta \rightarrow \gamma) \}}{\alpha \rightarrow \gamma \{ \beta, \alpha \rightarrow (\beta \rightarrow \gamma) \}} \rightarrow_I} \rightarrow_E$$

$$\frac{\alpha \rightarrow \gamma \{ \beta, \alpha \rightarrow (\beta \rightarrow \gamma) \}}{\beta \rightarrow (\alpha \rightarrow \gamma) \{ \alpha \rightarrow (\beta \rightarrow \gamma) \}} \rightarrow_I$$

- 否定，对任意语句 α ，假设集合 $H_0 H_1$ ：

$$\frac{\perp^{H_0}}{\neg \alpha^{H_0 \setminus \alpha}} \neg_I \qquad \frac{\neg \alpha^{H_0} \quad \alpha^{H_1}}{\perp^{H_0 \cup H_1}} \neg_E$$

- 假值，对任意语句 α ，假值集合 H_0 ，假值集合 H_1 满足 $\neg \alpha \in H_1$ ：

$$\frac{\perp^{H_0}}{\alpha^{H_0}} EFQ \qquad \frac{\perp^{H_1}}{\alpha^{H_1 \setminus \neg \alpha}} RAA$$

其中 EFQ 代表 若假 则任意。 RAA 代表 归谬，也就是反证法。事实上，规则 RAA 比看起来更特殊。这是直觉主义逻辑形式上唯一不承认的一条形成规则。在没有这条规则以后，我们就得不到 $\vdash \alpha \vee \neg \alpha$ ，即得不到无前提条件的排中律。因为否则我们只能通过 $\vee_I$ 得到 $\alpha \vee \neg \alpha$ ，但显然在没有其它前提的情况下得不到 α 或 $\neg \alpha$ 。

于是，显然的，在当前的语义要求下，直觉主义逻辑是不完备的¹。而从认识论的角度说，无条件接受排中律可理解为我们认为客观上任意语句只能是真的或假的，即排除了真与假之间、或者说之

¹事实上，可以证明直觉主义逻辑是无限多值的，见 John P. Burgess. "Intuitions of three kinds in Gödel's views on the continuum". In: Jan. 2014, pp. 11–31。

外的情况；但可认为直觉主义逻辑要求我们主观上知道、即可以确定给定语句的真假，而从构造主义的角度来说，可以确定给定语句的真假即意为着我们可以构造该语句或其否定的证明（回顾直觉主义为构造主义的一个分支）。同样，如章节 2.1.4 所述，我们接受作为定理的排中结论，更直接的，若我们已经证明了语句 α ，则有 $\alpha \vee \neg \alpha$ 。

技术上，相对的，下面给出通过 *RAA* 证明排中律的形成树：

$$\frac{\frac{\frac{[\neg(\alpha \vee \neg \alpha)]}{\perp \{ \neg(\alpha \vee \neg \alpha), \alpha \}} \neg E}{\neg \alpha \{ \neg(\alpha \vee \neg \alpha) \}} \neg I}{\frac{\perp \{ \neg(\alpha \vee \neg \alpha) \}}{\alpha \vee \neg \alpha} \text{RAA}} \quad \frac{\frac{\frac{[\neg \alpha]}{\perp \{ \neg(\alpha \vee \neg \alpha), \neg \alpha \}} \neg E}{\alpha \{ \neg(\alpha \vee \neg \alpha) \}} \text{RAA}}{\frac{\perp \{ \neg(\alpha \vee \neg \alpha) \}}{\alpha \vee \neg \alpha} \text{RAA}} \neg E$$

注意到目前为止的讨论均适用于命题逻辑，即由以上形成规则构成的自然演绎也可以作为命题逻辑的演绎规则。同时注意这里未包含处理 $\leftrightarrow$ 的规则，可认为在我们规则适用的命题逻辑 / 一阶逻辑中， $\leftrightarrow$ 是被其它逻辑联结词定义的。由功能完备性，这并不带来实质上的影响，因此取舍完全取决于方便与否。在下面很多情况中，我们都会通过（广义的、即如涉及量词的）功能完备性省略可定义符号的处理。

相比于真值表，自然演绎在很多时候可以更紧凑；但同时，自然演绎搜索形成树的过程不一定返回，即对应得到的 $\vdash$ 关系是可枚举但不可判定的（这也是 $\vdash$ 关系的最小要求）。同样的，以上直觉主义相关的讨论也适用于命题逻辑，则由其当前语义下的不完备可以看出直觉主义逻辑不能使用真值表作为演绎规则。下文中，如无特别说明，总是假定讨论的是经典逻辑，即含无条件排中律的逻辑。接下来我们进一步给出量词相关的形成规则。

- 全称量词，对任意变量符号 x ，语句 α 满足 $x \notin b(\alpha)$ ，语句 β ，假设集合 H ，不出现在 H 中的额外增加的常量符号 c ，封闭项 τ ：

$$\frac{\alpha^H}{\forall x \alpha [c/x]^H} \forall I \quad \frac{\forall x \beta^H}{\beta [x/\tau]^H} \forall E$$

其中额外增加的常量符号指仅为完成证明增加的纯形式（即不需要语义解释）的常量符号，其在证明中的对待方式和原有的常量符号一致。见下方示例。

- 存在量词，对任意变量 x ，式 α 满足 $f(\alpha) \subseteq \{x\} \wedge x \notin b(\alpha)$ 、即 α 至多只含为自由变量的 x ，下文写作 $\alpha[x]$ ，进一步写 $\alpha[x_1 \dots x_n]$ 表示至多只含 $x_1 \dots x_n$ ，语句 β ，封闭项 τ ，假设集合 $H_0 H_1$ ，可能额外增加的常量符号 c 满足其不出现在 $\alpha \beta H_0 H_1$ 中：

$$\frac{\alpha [x/\tau]^{H_0}}{\exists x \alpha^{H_0}} \exists I \quad \frac{\exists x \alpha^{H_0} \quad \alpha [x/c] \rightarrow \beta^{H_1}}{\beta^{H_0 \cup H_1}} \exists E$$

如若系统不含常量符号，则需增加常量符号 $u v$ 以证明 $\exists x \forall y P(x, y) \vdash \forall y \exists x P(x, y)$ ：

$$\frac{\frac{\frac{[\forall y P(u, y)]}{P(u, v) \{ \forall y P(u, y) \}} \forall E}{\exists x P(x, v) \{ \forall y P(u, y) \}} \exists I}{\forall y \exists x P(x, y) \{ \forall y P(u, y) \}} \forall E \rightarrow I \quad \frac{[\exists x \forall y P(x, y)] \quad (\forall y P(u, y)) \rightarrow (\forall y \exists x P(x, y))^\emptyset}{\forall y \exists x P(x, y) \{ \exists x \forall y P(x, y) \}} \exists E$$

撇开形式上的复杂性，可以看到量词的规则其实也非常符合我们日常的证明习惯。比如对于规则 $\forall I$

和 $\exists c$ ，在应用时要求涉及的 c 具有一般性。同时也可以看到通过额外增加得到的常量符号更容易满足这种一般性。

以上就是这里给出的全部形成规则。注意形成规则的选择和组合方式并不是唯一的。最后，对无限前提集 Γ ，式 σ ，要求 $\Gamma \vdash \sigma$ 若存在 $\Delta \subset \Gamma$ 满足有形成树对应 $\Delta \vdash \sigma$ 。可以看出，意味着一阶逻辑也有紧性定理，其证明会在章节 3.1.5 给出。这样我们便完成了对自然演绎规则的基本讨论。

下面我们简单介绍一下希尔伯特形式演绎系统。可认为希尔伯特演绎一般由非常少的推导规则构成。如这里介绍的系统¹仅包含一条类似于 $\rightarrow_E$ 的推导规则，我们称其为肯定前件。而为保证演绎的完备性，系统提供了大量的逻辑公理。同时，逻辑公理和非逻辑公理（即用户提供的前提）的处理方式是统一的。这也使整个系统在概念上相对更加简单，比如肯定前件的输入输出直接对应语句。具体的，对任意语句 $\alpha\beta$ ，肯定前件有：

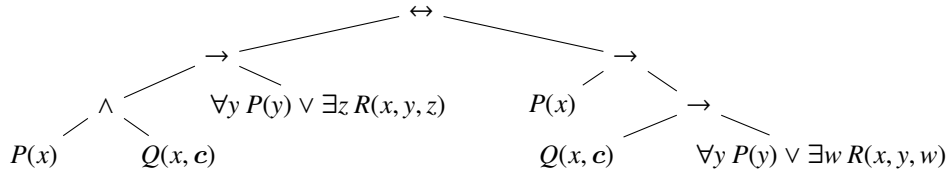
$$\frac{\alpha \quad \alpha \rightarrow \beta}{\beta}$$

其较复杂的部分为对逻辑公理的判定。首先对任意 n ，任意 $x_1 \dots x_n$ ，式 α ，我们称 $\forall x_1 \dots x_n \alpha$ 为 α 的泛化。如 $\forall x \forall y P(x, y)$ 是 $P(x, y)$ 的泛化， $P(c)$ 是 $P(c)$ 自身的泛化。则对任意语句 γ ，其是逻辑公理当且仅当其是以下形式的式²的泛化：

1. 命题恒真式，即若将式按以下方式转换为命题逻辑语句后其是恒真的。对任意式 α ，自顶向下展开语法树直至原子式或由全称量词构成的式。如对：

$$(P(x) \wedge Q(x, c) \rightarrow \forall y P(y) \vee \exists z R(x, y, z)) \leftrightarrow (P(x) \rightarrow (Q(x, c) \rightarrow \forall y P(y) \vee \exists w R(x, y, w)))$$

令其为 φ ，有展开树：



对展开树中的叶节点进行标记，两个叶节点标记相同当且仅当可通过约束变量的替换使节点式相等。如可标记 $P(x)$ 为 A ， $Q(x, c)$ 为 B ， $\forall y P(y) \vee \exists z R(x, y, z)$ 和 $\forall y P(y) \vee \exists w R(x, y, w)$ 均为 C 。继而将标记作为语句符号替换入语法树中得到转换后的命题逻辑语句，如 $(A \wedge B \rightarrow C) \leftrightarrow (A \rightarrow (B \rightarrow C))$ 。可以看到其为恒真语句，则 φ 是命题恒真的。因命题逻辑的恒真语句是可判定的，因此命题恒真是可判定的。

2. 任意 $x \alpha \tau$ 构成的式 $\forall x \alpha \rightarrow \alpha_{[x/\tau]}$ ，注意由我们的式要求有 $f(\tau) \cap b(\alpha) = \emptyset$ 。
3. 任意 $x \alpha \beta$ 构成的式 $\forall x (\alpha \rightarrow \beta) \rightarrow (\forall x \alpha \rightarrow \forall x \beta)$ 。
4. 任意 $x \alpha$ 构成的式 $\alpha \rightarrow \forall x \alpha$ ，其中额外要求 $x \notin f(\alpha)$ 。

可以看到希尔伯特系统包含有无限的逻辑公理。一般称一组符合特定语句模式的公理集合为公理模式，如上述规则 2 3 4 都可分别被认为描述了一个公理模式。带有公理模式也一般意味着公理数量是无限的。需要引入公理模式的理论一般也不是有限可公理化的。

¹Herbert B. Enderton. A Mathematical Introduction to Logic. New York: Academic Press, 1972, pp. 110–116.

²注意这里的式为当前逻辑系统所用语言下的式，即其不（需）包含额外添加的常量。结合演绎规则不引入额外常量，希尔伯特形式演绎系统不引入额外常量。

而在得到推导树后，有叶节点和逻辑公理的差集语法蕴含根节点。例如对：

$$\frac{\frac{P(c) \quad \frac{\forall x \neg P(x) \rightarrow \neg P(c) \quad (\forall x \neg P(x) \rightarrow \neg P(c)) \rightarrow (P(c) \rightarrow \neg \forall x \neg P(x))}{P(c) \rightarrow \neg \forall x \neg P(x)}}{\neg \forall x \neg P(x)}}$$

因 $\forall x \neg P(x) \rightarrow \neg P(c)$ 来自逻辑公理规则 2, $(\forall x \neg P(x) \rightarrow \neg P(c)) \rightarrow (P(c) \rightarrow \neg \forall x \neg P(x))$ 来自规则 3, 最终有 $P(c) \vdash \neg \forall x \neg P(x)$ 。进一步，我们以类似自然演绎的方式处理无限前提集的情况。

希尔伯特系统概念上的相对简单也导致了其直接应用时的相对复杂。

最后，在进入下一小节之前，另外简单讨论一下演绎规则（含逻辑公理，若有）间的等价性问题。对语句集 Γ ，语句 σ ，写 $\Gamma \vdash_A \sigma$ 表示由演绎规则 **A** 得到的 $\Gamma \vdash \sigma$ 。则称规则 **A B** 等价指对任意语句集合 Γ ，语句 σ ， $\Gamma \vdash_A \sigma$ 当且仅当 $\Gamma \vdash_B \sigma$ 。我们可以通过证明 **A B** 均为可靠且完备的证明它们之间是等价的。也可以通过纯形式的尝试证明它们的每一条推导规则及逻辑公理（若系统是以推导规则和逻辑公理形式给出的）可互相由对方的推导规则和逻辑公理生成；即给定一方的推导树，可递归的构造另一方的推导树。

3.1.4 相继式演绎

相继式演绎推导规则的输入输出形式为 $\Gamma \Rightarrow \Delta$ ， $\Gamma \Delta$ 为任意有限语句集合。 Γ 表示假设集合。事实上，自然演绎与相继式演绎含假定条件也可视为根岑形式演绎系统和希尔伯特形式演绎系统的主要区别之一。进一步，对有限语句集 $\Gamma \Delta$ ， $\Gamma \Rightarrow \Delta$ 表示 $\Gamma \vdash \bigvee \Delta$ ； $\perp$ 。可见 $\Gamma \Rightarrow \emptyset$ 即表示 Γ 不一致，而 $\emptyset \Rightarrow \emptyset$ 代表推导规则本身不一致。另一方面，相比于自然演绎，一般认为相继式演绎的规则更为规整，即如下文所见，更适用于关于证明过程性质、即证明论的相关研究。

下面给出一种相继式演绎的推导规则 **LK**^{注1 2}，对任意语句集合 $\Gamma \Delta$ ：

- 对任意语句 α ：

$$\frac{}{\{\alpha\} \Rightarrow \{\alpha\}} \text{L}_0$$

称其中 —— 上方的为上式，下方为下式。下同。当前规则没有上式。

- 对任意语句 α ：

$$\frac{\Gamma \Rightarrow \Delta}{\Gamma; \alpha \Rightarrow \Delta} \text{L}_{1a} \quad \frac{\Gamma \Rightarrow \Delta}{\Gamma \Rightarrow \Delta; \alpha} \text{L}_{1b}$$

- 对任意语句 α ：

$$\frac{\Gamma \Rightarrow \Delta; \alpha}{\Gamma; \neg \alpha \Rightarrow \Delta} \text{L}_{2a} \quad \frac{\Gamma; \alpha \Rightarrow \Delta}{\Gamma \Rightarrow \Delta; \neg \alpha} \text{L}_{2b}$$

- 对任意语句 $\alpha \beta$ ：

¹LK 代表经典逻辑，该套规则是由格哈德·根岑（Gerhard Gentzen）于 1935 年首次提出的。同时提出的还包括 NK、NJ，分别对应经典自然演绎规则和直觉主义自然演绎规则。其中采用了 J 而非 I 代表直觉主义。以及 LJ 对应直觉主义相继式演绎规则。

²同时注意这里我们使用了未严格形式化的集合的概念，而在根岑最初的规则中，其是更严格的基于机械式的字符串操作的，即可仍为基于一种更严格的计算模型的，如对由有限多个语句组成；分隔的有限字符串 $\Gamma \Delta$ 即语句 α 有规则：

$$\frac{\Gamma; \alpha; \alpha \Rightarrow \Delta}{\Gamma; \alpha \Rightarrow \Delta}$$

本质上该规则即表示集合中不含重复元素。根岑称该类规则为结构规则（其余为逻辑规则）。除该点外还略有其它改动。参考 G. Gentzen, “Investigations into Logical Deduction”. In: The Collected Papers of Gerhard Gentzen, Ed. by M.E. Szabo. Studies in logic and the foundations of mathematics. North-Holland Publishing Company, 1970, pp. 68–131。

$$\frac{\Gamma; \alpha \Rightarrow \Delta \quad \Gamma; \beta \Rightarrow \Delta}{\Gamma; \alpha \vee \beta \Rightarrow \Delta} \text{L}_{3a} \quad \frac{\Gamma \Rightarrow \Delta; \alpha}{\Gamma \Rightarrow \Delta; \alpha \vee \beta} \text{L}_{3b_1}$$

$$\frac{\Gamma \Rightarrow \Delta; \beta}{\Gamma \Rightarrow \Delta; \alpha \vee \beta} \text{L}_{3b_2}$$

- 对任意式 $\alpha[x]$, 封闭项 τ , 可能为额外增加的常量符号 c 满足不出现在 $\Gamma \Delta$ 以及 α 中:

$$\frac{\Gamma; \alpha[x/c] \Rightarrow \Delta}{\Gamma; \exists x \alpha \Rightarrow \Delta} \text{L}_{4a} \quad \frac{\Gamma \Rightarrow \Delta; \alpha[x/\tau]}{\Gamma \Rightarrow \Delta; \exists x \alpha} \text{L}_{4b}$$

- 对任意语句集合 $\Lambda \Sigma \Pi$:

$$\frac{\Gamma \Rightarrow \Delta \cup \Lambda \quad \Lambda \cup \Sigma \Rightarrow \Pi}{\Gamma \cup \Sigma \Rightarrow \Delta \cup \Pi} \text{L}_{cut}$$

其中最后一条规则 L_{cut} 为切规则, 可认为对应肯定前件。但其特殊性还在于其是唯一一条可能从前提中直接删除 (而不是修改或增加) 某些成分的规则。而同时其是可选的, 即该规则是否存在不影响可推导性, 或完备性。即意味着我们在不直接使用肯定前件这一最古老的逻辑规则的情况下, 可推导出所有应推导出的一阶逻辑语句。我们将在章节 3.1.6 证明这一点。同时注意, 这里仅包含了 $\neg \vee$ 和 $\exists$ 的处理, 即认为其它逻辑联结词和 $\forall$ 是被定义的¹。最后, 仍以类似自然演绎的方式处理在无限前提集下的情况。

下面我们给出的几个不含 L_{cut} 规则的示例, 其中 *c.f. LK* 即不含切规则。并省略 L_0 :

示例 3.1.1 对任意语句 α , $\vdash_{c.f. \text{LK}} \alpha \vee \neg \alpha$ 。

证明:

$$\frac{\frac{\frac{\{\alpha\} \Rightarrow \{\alpha\}}{\emptyset \Rightarrow \{\alpha, \neg \alpha\}} \text{L}_{2b}}{\emptyset \Rightarrow \{\alpha \vee \neg \alpha, \neg \alpha\}} \text{L}_{3b_1}}{\emptyset \Rightarrow \{\alpha \vee \neg \alpha\}} \text{L}_{3b_2} \quad (3.1.1) \quad \square$$

示例 3.1.2 对任意语句 $\alpha \beta$, $\alpha \vee \beta \vdash_{c.f. \text{LK}} \beta \vee \alpha$ 。

证明:

$$\frac{\frac{\frac{\{\alpha\} \Rightarrow \{\alpha\}}{\{\alpha\} \Rightarrow \{\beta \vee \alpha\}} \text{L}_{3b_2} \quad \frac{\frac{\{\beta\} \Rightarrow \{\beta\}}{\{\beta\} \Rightarrow \{\beta \vee \alpha\}} \text{L}_{3b_1}}{\{\alpha \vee \beta\} \Rightarrow \{\beta \vee \alpha\}} \text{L}_{3a} \quad \square$$

示例 3.1.3 (肯定前件) 对任意语句 $\alpha \beta$, $\alpha, \alpha \rightarrow \beta \vdash_{c.f. \text{LK}} \beta$ 。

证明: 即只需证明 $\{\alpha, \neg \alpha \vee \beta\} \Rightarrow \{\beta\}$:

$$\frac{\frac{\frac{\{\alpha\} \Rightarrow \{\alpha\}}{\{\alpha, \neg \alpha\} \Rightarrow \emptyset} \text{L}_{2a} \quad \frac{\{\beta\} \Rightarrow \{\beta\}}{\{\alpha, \beta\} \Rightarrow \{\beta\}} \text{L}_{1a}}{\{\alpha, \neg \alpha \vee \beta\} \Rightarrow \{\beta\}} \text{L}_{3a} \quad \square$$

我们将在下一小节的证明中看到切规则带来的便利。

¹ $\forall x \alpha \equiv \neg \exists x \neg \alpha$ 。

3.1.5 可靠性和完备性

令 LK 表示直接由上述（含 L_{cut} ）相继式演绎推导规则构成的演绎系统，令 LK' 表示扩展 LK 后可处理无限前提集情况的演绎系统。现在我们证明 LK' 的可靠性和完备性。

定理 3.1.4 LK' 是可靠的。即对任意语句集 Γ ，语句 σ ， $\Gamma \vdash_{LK'} \sigma \rightarrow \Gamma \models \sigma$ ¹。

证明：对演绎规则做数学归纳，显然每一条推导规则都是符合语义蕴含要求的，对无限前提集的处理也是符合语义蕴含要求的。因此 σ 可以由 Γ 经 LK' 证明意味着 σ 是 Γ 的结果。□

完备性的证明比较复杂。类似命题逻辑，我们区别有限前提集下的 LK 和可处理无限前提集的 LK' 的完备性证明。首先给出在 LK 是完备的以及紧性定理成立的情况下， LK' 的完备性证明。然后，我们分别证明 LK 的完备性和紧性定理，注意其中紧性证明将大量复用 LK 完备性证明²。具体的，我们通过：

引理 3.1.5 在有限前提集下， LK 是完备的。即对任意有限语句集 Γ ，语句 σ ， $\Gamma \models \sigma \rightarrow \Gamma \vdash_{LK} \sigma$ 。

引理 3.1.6（紧性定理）对任意语句集 Γ ，若 Γ 的每一个有限子语句集都是可满足的，则 Γ 是可满足的。

以及由语义蕴含定义直接可得引理：

引理 3.1.7 对任意语句集 $\Gamma \Delta$ 和语句 σ ：

- a. 若 $\Delta \subseteq \Gamma$ 且 $\Delta \models \sigma$ ，则 $\Gamma \models \sigma$ 。
- b. $\Gamma \models \sigma$ 当且仅当 $\Gamma; \neg \sigma$ 是不可满足的。
- c. 若 Γ 是不可满足的，则 $\Gamma \vdash \sigma$ 。

证明 LK' 的完备性：

定理 3.1.8（哥德尔完备性定理） LK' 是完备的。即对任意语句集 Γ ，语句 σ ， $\Gamma \models \sigma \rightarrow \Gamma \vdash_{LK'} \sigma$ 。

证明：同命题逻辑中的证明。若 $\Gamma \models \sigma$ ，则由引理 b 可得 $\Gamma; \neg \sigma$ 是不可满足的。同时，由紧性定理可得 $\Gamma; \neg \sigma$ 不可满足意味着存在有限 $\Delta \subseteq \Gamma$ 不可满足。因此，若 $\neg \sigma \notin \Delta$ ，则由引理 c 可得 $\Delta \vdash \sigma$ ，否则由引理 a 同样可得 $\Delta \vdash \sigma$ 。由有限场景下 LK 的可靠性可得 $\Delta \vdash_{LK} \sigma$ ，有 LK 返回，即 $\Gamma \vdash_{LK'} \sigma$ 。□

对于引理 3.1.5，即 LK 的完备性，写 $\Gamma \Rightarrow \Delta$ 表示在有限次数内由推导规则得到 $\Gamma \Rightarrow \Delta$ ：

引理 3.1.9 对任意有限语句集 Σ ， $\Sigma \Rightarrow \emptyset \leftrightarrow \Sigma \vdash_{LK} \perp$ 。

证明：正方向即定义。对于反方向，即若可推导出 $\Sigma \Rightarrow \sigma \wedge \neg \sigma$ ，则 $\Sigma \Rightarrow \emptyset$ ：

$$\frac{\begin{array}{c} \vdots \\ \Sigma \Rightarrow \neg(\neg \sigma \vee \sigma) \end{array} \quad \frac{\overline{\overline{\emptyset \Rightarrow \{\neg \sigma \vee \sigma\}}} \quad \neg(\neg \sigma \vee \sigma) \Rightarrow \emptyset}{\Sigma \Rightarrow \emptyset} \begin{array}{l} (3.1.1) \\ L_{2a} \\ L_{cut} \end{array}$$

□

引理 3.1.10 对任意有限语句集 Γ ，语句 σ ， $\Gamma \vdash_{LK} \sigma \leftrightarrow \Gamma; \neg \sigma \vdash_{LK} \perp$ ，右侧即为 $\Gamma; \neg \sigma$ 不一致。

¹我们开始在不混淆时，在元逻辑中逐步使用逻辑联结词，同时这也部分反映了一阶逻辑可以用于论证元逻辑。

²以下哥德尔完备性定理的证明首先是由哥德尔在 1930 年完成的。这里的证明方法由莱昂·亨金 (Leon Henkin) 于 1949 年发表。具体参考 Herbert B. Enderton. A Mathematical Introduction to Logic. New York: Academic Press, 1972, pp. 135–142 以及 C.C. Chang and H.J. Keisler. Model Theory: Third Edition. Dover Books on Mathematics. Dover Publications, 2013, pp. 61–66。证明过程可对比参照章节 2.2.6 中命题逻辑在无限前提集下的演绎规则的完备性证明。

证明：由引理 3.1.9，我们只需证明 $\Gamma \Rightarrow \{\sigma\} \leftrightarrow \Gamma; \neg \sigma \Rightarrow \emptyset$ ，正方向即规则 L_{2a} 。反方向：

$$\frac{\frac{\{\sigma\} \Rightarrow \{\sigma\}}{\emptyset \Rightarrow \{\sigma, \neg \sigma\}} 2b \quad \frac{\vdots}{\Gamma; \neg \sigma \Rightarrow \emptyset}}{\Gamma \Rightarrow \sigma} L_{cut}$$

□

这样，由引理 3.1.7 b 及 3.1.10，对 LK 的完备性，我们只需要证明：

引理 3.1.11 LK 下一致的语句集是可满足的。即对任意 Σ ，若 $\Sigma \not\models_{LK} \perp$ ，则 Σ 是可满足的。

因即有：

引理 3.1.5 的证明：令 $\Sigma = \Gamma; \neg \sigma$ ，有若 $\Gamma; \neg \sigma$ 是不可满足的，则 $\Gamma; \neg \sigma \vdash_{LK} \perp$ 。即 $\Gamma \models \sigma$ 。 □

为证明引理 3.1.11，对任意 LK 下一致的语句集合 Σ ，令其使用的一阶逻辑语言为 $\mathcal{L}$ ^{注1}。我们将扩展 $\mathcal{L}$ 至 $\mathcal{L}^*$ ，扩展 Σ 至 Σ^* 。并在其上构造解释 $\mathfrak{A}$ 为 Σ^* 的模型。最后由 $\Sigma \subset \Sigma^*$ 得到 $\mathfrak{A}$ 也为 Σ 的模型。即 Σ 是可满足的。

首先向 $\mathcal{L}$ 中添加令这些符号为（不和原有符号冲突的）常量符号 $c_1 c_2 \dots$ 。令新语言为 $\mathcal{L}^*$ 。可以看到 $\mathcal{L}^*$ 是可枚举的。我们首先在新语言上扩展 Σ 构造语句集 $\Sigma^\exists$ 有：其任意有限子语句集在 LK 下一致，且 $\Sigma^\exists$ 包含所有形如 $\exists x_i \varphi_i \rightarrow \varphi_i[x_i/c_i]$ 的语句。其中添加 $\exists x_i \varphi_i \rightarrow \varphi_i[x_i/c_i]$ 是为了使每一个 $\exists x_i \varphi_i$ 这样的语句可得到一个关于具体的常量符号 c_i 的语句 $\varphi_i[x_i/c_i]$ ，其中 c_i 不在集合 $\Sigma^\exists$ 其它任何地方出现，即不受其它语句影响。

具体的，首先：

引理 3.1.12 对任意 LK 下一致的有限语句集 Σ ， Σ 的任意有限子语句集 Δ 是一致的。

证明：若 Δ 不一致，即 $\Delta \Rightarrow \emptyset$ ，由规则 L_{1a} ，有 $\Sigma \Rightarrow \emptyset$ ，即 Σ 不一致。 □

进而我们构造 $\Sigma^\exists$ 。由 $\mathcal{L}^*$ 是可枚举的，枚举 $\exists x_1 \varphi_1, \exists x_2 \varphi_2, \dots$ 。令 $\Sigma_0^\exists = \Sigma$ ，对 $n > 0$ ，令 $\Sigma_n^\exists = \Sigma_{n-1}^\exists; \exists x_n \varphi_n \rightarrow \varphi_n[x_n/c_n]$ 。有：

引理 3.1.13 对任意 n ， $\Sigma_n^\exists$ 的任意有限子语句集是 LK 下一致的。

证明：因 Σ 是 LK 下一致的， $\Sigma_0^\exists$ 的任意有限子语句集是 LK 下一致的。对 $n > 0$ ，假设 $\Sigma_{n-1}^\exists$ 的任意子语句集是 LK 下一致的。若 $\Sigma_n^\exists$ 存在子语句集不一致，则存在有限的 $\Delta \subseteq \Sigma_{n-1}^\exists$ 有 $\Delta; \neg \exists x_n \varphi_n \vee \varphi_n[x_n/c_n] \Rightarrow \emptyset$ 。这样有：

$$\frac{\frac{\{\neg \exists x_n \varphi_n\} \Rightarrow \{\neg \exists x_n \varphi_n\}}{\{\neg \exists x_n \varphi_n\} \Rightarrow \{\neg \exists x_n \varphi_n \vee \varphi_n[x_n/c_n]\}} L_{3b_1} \quad \frac{\vdots}{\Delta; \neg \exists x_n \varphi_n \vee \varphi_n[x_n/c_n] \Rightarrow \emptyset}}{\Delta; \neg \exists x_n \varphi_n \Rightarrow \emptyset} L_{cut} \quad (i)$$

$$\frac{\frac{\{\varphi_n[x_n/c_n]\} \Rightarrow \{\varphi_n[x_n/c_n]\}}{\{\varphi_n[x_n/c_n]\} \Rightarrow \{\neg \exists x_n \varphi_n \vee \varphi_n[x_n/c_n]\}} L_{3b_2} \quad \frac{\vdots}{\Delta; \neg \exists x_n \varphi_n \vee \varphi_n[x_n/c_n] \Rightarrow \emptyset}}{\Delta; \varphi_n[x_n/c_n] \Rightarrow \emptyset} L_{cut} \quad (ii)$$

¹ 回顾我们将讨论限制在可枚举范围内，即 $\mathcal{L}$ 是可枚举的。

$$\begin{array}{c}
\vdots \\
\frac{\Delta; \neg \exists x_n \varphi_n \vee \varphi_n[x_n/c_n] \Rightarrow \emptyset}{\Delta; \neg \exists x_n \varphi_n \Rightarrow \emptyset} \text{ (i)} \\
\frac{\Delta; \neg \exists x_n \varphi_n \vee \varphi_n[x_n/c_n] \Rightarrow \emptyset}{\Delta; \neg \exists x_n \varphi_n \vee \exists x_n \varphi_n \Rightarrow \emptyset} \text{ (ii)} \\
\frac{\Delta; \neg \exists x_n \varphi_n \vee \exists x_n \varphi_n \Rightarrow \emptyset}{\Delta \Rightarrow \neg(\neg \exists x_n \varphi_n \vee \exists x_n \varphi_n)} \text{ L}_{2b}
\end{array}$$

即 Δ 不一致, 和假设矛盾。 $\square$

令 $\Sigma^\exists = \bigcup_n \Sigma_n^\exists$, 则 $\Sigma^\exists$ 符合我们的条件要求。进一步我们扩展 $\Sigma^\exists$ 至 Σ^* 有: 其任意有限子语句集在 LK 下一致, 且对任意语句 α 有 $\alpha \in \Sigma^*$ 或 $\neg \alpha \in \Sigma^*$ 。我们由以下引理经数学归纳证明存在这样的 Σ^* :

引理 3.1.14 对任意语句集 Σ 满足其任意有限子语句集是 LK 下一致的, 任意语句 α , 有 $\Sigma; \alpha$ 或 $\Sigma; \neg \alpha$ 满足其任意有限子语句集是 LK 下一致的。

证明: 若对 $\Sigma; \alpha$ 和 $\Sigma; \neg \alpha$, 均存在有限子语句集在 LK 下不一致。则存在有限的 $\Delta_1 \subseteq \Sigma$ 有 $\Delta_1; \alpha$ 在 LK 下不一致, 存在 $\Delta_2 \subseteq \Sigma$ 有 $\Delta_2; \neg \alpha$ 在 LK 下不一致。即 $\Delta_1; \alpha \Rightarrow \emptyset$ 且 $\Delta_2; \neg \alpha \Rightarrow \emptyset$, 令 $\Delta = \Delta_1 \cup \Delta_2$, 则由 L_{1a} 有 $\Delta; \alpha \Rightarrow \emptyset$ 且 $\Delta; \neg \alpha \Rightarrow \emptyset$, 继而:

$$\begin{array}{c}
\frac{\Delta; \alpha \Rightarrow \emptyset \quad \Delta; \neg \alpha \Rightarrow \emptyset}{\Delta; \alpha \vee \neg \alpha \Rightarrow \emptyset} \text{ L}_{3a} \\
\frac{\Delta; \alpha \vee \neg \alpha \Rightarrow \emptyset}{\Delta \Rightarrow \neg(\alpha \vee \neg \alpha)} \text{ L}_{2b}
\end{array}$$

即 Δ 是不一致的。但 Δ 是有限的且 $\Delta \subseteq \Sigma$, 矛盾。 $\square$

这样, 我们构造 Σ^* 。同样由 $\mathcal{L}^*$ 是可枚举的, 枚举其中所有的语句 $\alpha_1 \alpha_2 \dots$ 。构造 $\Sigma^*_0 = \Sigma^\exists$, 对 $n > 0$:

$$\Sigma^*_n = \begin{cases} \Sigma^*_{n-1}; \alpha_n & \text{若其任意有限子语句集是 LK 下一致的} \\ \Sigma^*_{n-1}; \neg \alpha_n & \text{若其它} \end{cases}$$

令 $\Sigma^* = \bigcup_n \Sigma^*_n$, 则其满足对任意 $\mathcal{L}^*$ 的语句 α 有 $\alpha \in \Sigma^*$ 或 $\neg \alpha \in \Sigma^*$, 且 Σ^* 的每一个有限子语句集都是 LK 下一致的。称 Σ^* 为最大有限一致的。

现在构造解释 $\mathfrak{A}$:

- 令 $\mathfrak{A}_{[\vee]}$ 等于 Σ^* 中出现的所有封闭项集合。
- 对 Σ^* 中的每一个封闭项 τ 令 $\mathfrak{A}_{[\tau]} = \tau$ 。
- 对 Σ^* 中的每一个 n 位谓词符号 P ($n > 0$), 构造集合 $\mathfrak{A}_{[P]} = \{ \langle \tau_1 \dots \tau_n \rangle \in \mathfrak{A}_{[\vee]}^n : P(\tau_1 \dots \tau_n) \in \Sigma^* \}$ 。即对任意项 $\tau_1 \dots \tau_n$, $P(\tau_1 \dots \tau_n) \in \Sigma^* \leftrightarrow \langle \tau_1 \dots \tau_n \rangle \in \mathfrak{A}_{[P]}$ 。

可以看到对 $\mathcal{L}^*$ 中的任意原子语句 φ_0 , $\varphi_0 \in \Sigma^* \leftrightarrow \models_{\mathfrak{A}} \varphi_0$ 。我们的目标是进一步证明对 $\mathcal{L}^*$ 中的任意语句 φ , $\varphi \in \Sigma^* \leftrightarrow \models_{\mathfrak{A}} \varphi$ 。其中的正方向即表明 $\mathfrak{A}$ 为 Σ^* 的模型。具体的, 我们对 $\mathcal{L}^*$ 中的语句按照语句结构进行数学归纳, 注意相继式演绎中我们使用的一阶逻辑语言仅包含逻辑联结词 $\neg \vee$ 和存在量词:

- 对任意语句 $\varphi = \neg \alpha$, 假设 $\alpha \in \Sigma^* \leftrightarrow \models_{\mathfrak{A}} \alpha$, 则:

$$\varphi \in \Sigma^* \leftrightarrow \neg \alpha \in \Sigma^* \leftrightarrow \alpha \notin \Sigma^* \leftrightarrow \not\models_{\mathfrak{A}} \alpha \leftrightarrow \models_{\mathfrak{A}} \neg \alpha \leftrightarrow \models_{\mathfrak{A}} \varphi$$

- 对任意语句 $\varphi = \alpha \vee \beta$, 假设 $\alpha \in \Sigma^* \leftrightarrow \models_{\mathfrak{A}} \alpha$ 且 $\beta \in \Sigma^* \leftrightarrow \models_{\mathfrak{A}} \beta$ 。若 $\varphi \in \Sigma^*$, 因:

$$\frac{\frac{\frac{\{\alpha\} \Rightarrow \{\alpha\}}{\{\alpha, \neg \alpha\} \Rightarrow \emptyset} L_{2a} \quad \frac{\{\beta\} \Rightarrow \{\beta\}}{\{\beta, \neg \alpha\} \Rightarrow \{\beta\}} L_{1a}}{\frac{\{\alpha \vee \beta, \neg \alpha\} \Rightarrow \{\beta\}}{\{\alpha \vee \beta, \neg \alpha, \neg \beta\} \Rightarrow \emptyset} L_{2a}} L_{3a}$$

即 $\{\alpha \vee \beta, \neg \alpha, \neg \beta\}$ 不一致, 由最大有限一致性有 $\alpha \in \Sigma^* \vee \beta \in \Sigma^*$, 有 $\models_{\mathfrak{A}} \alpha \vee \models_{\mathfrak{A}} \beta$, 即 $\models_{\mathfrak{A}} \varphi$ 。若 $\models_{\mathfrak{A}} \varphi$, 有 $\models_{\mathfrak{A}} \alpha \vee \models_{\mathfrak{A}} \beta$ 。有 $\alpha \in \Sigma^* \vee \beta \in \Sigma^*$ 。类似的, 由规则 L_{1b} 、 L_{2a} 有 $\{\alpha, \neg(\alpha \vee \beta)\}$ 和 $\{\beta, \neg(\alpha \vee \beta)\}$ 均不一致。则由最大有限一致有 $\varphi \in \Sigma^*$ 。

- 对任意语句 $\varphi = \exists x \alpha$, 假设对任意封闭项 τ , $\alpha[x/\tau] \in \Sigma^* \leftrightarrow \models_{\mathfrak{A}} \alpha[x/\tau]$ 。若 $\varphi \in \Sigma^*$, 由 Σ^* 中 $\Sigma^{\exists}$ 的构造过程有存在 i 满足 $\varphi \rightarrow \alpha[x/c_i]$, 由肯定前件和最大有限一致有 $\alpha[x/c_i] \in \Sigma^*$, 有 $\models_{\mathfrak{A}} \alpha[x/c_i]$, 有 $\models_{\mathfrak{A}} \varphi$ 。反方向逆否¹后有若 $\exists x \alpha \notin \Sigma^*$, 即 $\neg \exists x \alpha \in \Sigma^*$, 由规则 L_{4a} 、 L_{4b} 有对任意封闭项 τ , $\{\alpha[x/\tau], \neg \exists x \alpha\}$ 不一致, 则对任意 τ , $\models_{\mathfrak{A}} \neg \alpha[x/\tau]$, 则 $\models_{\mathfrak{A}} \neg \exists x \alpha$, 即 $\not\models_{\mathfrak{A}} \exists x \alpha$ 。

这样, 我们即完成了 $\mathfrak{A}$ 为 Σ^* 的模型的证明。由 $\Sigma \subset \Sigma^*$ 以及语义蕴含的定义有 $\models_{\mathfrak{A}} \Sigma$, 即 Σ 是可满足的。我们以类似的方式完成紧性定理的证明。

紧性定理 3.1.6 的证明: 若 Σ 的每一个有限子语句集都是可满足的。扩展其语言 $\mathcal{L}$ 至 $\mathcal{L}^*$, 扩展 Σ 至 $\Sigma^{\exists}$ (因每一次添加的 c_i 不出现在现有语句中) 有其每一个有限子语句集仍都是可满足的。由类似命题逻辑紧性定理中引理 2.2.5 b 的证明, 我们进一步扩展 $\Sigma^{\exists}$ 至 Σ^* 有其每一个有限子语句集依然是可满足的。以上述相同的方式构造解释 $\mathfrak{A}$, 由 Σ^* 有限子语句集的可满足性和任意语句 α 满足 $\alpha \in \Sigma^* \vee \neg \alpha \in \Sigma^*$, 可以以上述类似方式得到 $\mathfrak{A}$ 是 Σ^* 的模型, 即也为 Σ 的模型。□

定理 3.1.15 (勒文海姆-斯科伦定理) 对任意语句² 集 Σ , 若 Σ 是可满足的, 则 Σ 可被域可数的解释满足。

证明: 由 LK 是可靠的, 有 $\Sigma \not\models_{LK} \perp$ 。同时注意引理 3.1.11 在证明 $\Sigma \not\models_{LK} \perp$ 蕴含 Σ 可满足中构造的模型 $\mathfrak{A}$ 的域是可数的。□

尽管上述定理完整的证明过程看起来十分繁复, 但从语言的角度来看其还是较直观的。因可枚举目标语言语句集的模型域中不必要包含在该语言中不可区分的元素, 而显然可枚举语言中可区分的元素是可枚举的, 其任意子集是可数的。

3.1.6 切消定理

本小节将证明切规则 L_{cut} 是可选的³, 并在下一小节给出几个其所带来的结果。

定理 3.1.16 (切消定理) 对任意有限语句集 Γ , 语句 σ , $\Gamma \vdash_{LK} \sigma \rightarrow \Gamma \vdash_{c.f. LK} \sigma$ 。回顾其中 $c.f. LK$ 代表不含 L_{cut} 的 LK 演绎规则。

首先, 我们需要以下引理, 对任意语句集 Γ , 任意封闭项 τv , 写 $\Gamma[\tau/v]$ 表示将其中所有 τ 替换为 v :

¹对任意 $\alpha \beta$, $\vdash (\alpha \rightarrow \beta) \leftrightarrow (\neg \beta \rightarrow \neg \alpha)$, 称 $\leftrightarrow$ 两侧互为逆否命题。

²注意这里的讨论范围仍被限制为可枚举语言。

³证明自 G. Gentzen. "Investigations into Logical Deduction". In: The Collected Papers of Gerhard Gentzen. Ed. by M.E. Szabo. Studies in logic and the foundations of mathematics. North-Holland Publishing Company, 1970, pp. 68–131。

引理 3.1.17 对任意语句集 $\Gamma \Delta$, 封闭项 τv , $\Gamma \Rightarrow \Delta \rightarrow \Gamma[\tau/v] \Rightarrow \Delta[\tau/v]$ 。

证明: 对推导树高度做数学归纳, 当高度等于 1 时, $\Gamma = \Delta = \{\alpha\}$, 由规则 L_0 引入, 则同样可由 L_0 引入 $\{\alpha[\tau/v]\} \Rightarrow \{\alpha[\tau/v]\}$ 。假设高度小于等于 n ($n > 0$) 时成立, 对高度为 $n+1$ 的树, 按最后使用的规则分类讨论。以最后应用 $\frac{\Gamma' \Rightarrow \Delta'}{\Gamma'; \alpha \Rightarrow \Delta'} L_{1a}$ 为例, 由假设可得存在 $\Gamma'_{[\tau/v]} \Rightarrow \Delta'_{[\tau/v]}$, 则同样应用 L_{1a} 存在 $\Gamma'_{[\tau/v]}; \alpha[\tau/v] \Rightarrow \Delta'_{[\tau/v]}$ 。其它规则同理。其中注意对 $\frac{\Gamma'; \alpha[x/c] \Rightarrow \Delta'}{\Gamma'; \exists x \alpha \Rightarrow \Delta'} L_{4a}$ 的处理, 若 $\tau = c$, 因要求 c 不出现在 $\Gamma' \Delta' \alpha$ 中, $\Gamma'_{[\tau/v]}; \exists x \alpha[\tau/v] \Rightarrow \Delta'_{[\tau/v]}$ 等于 $\Gamma'; \exists x \alpha \Rightarrow \Delta'$ 。□

接着引入一条新的规则, 对任意有限语句集 $\Gamma \Delta \Sigma \Pi$, 语句 α , $\alpha \notin \Delta \wedge \alpha \notin \Sigma$:

$$\frac{\Gamma \Rightarrow \Delta; \alpha \quad \Sigma; \alpha \Rightarrow \Pi}{\Gamma \cup \Sigma \Rightarrow \Delta \cup \Pi} L_{mix}$$

称其中的语句 α 为混合语句, $\Gamma \Rightarrow \Delta; \alpha$ 为左上式, $\Sigma; \alpha \Rightarrow \Pi$ 为右上式。令 LK^* 为将 L_{cut} 替换为 L_{mix} 的 LK 演绎规则, $m.f.LK^*$ 表示不含 L_{mix} 的 LK^* 演绎规则, 注意 $c.f.LK = m.f.LK^*$ 。可以看到:

引理 3.1.18 对任意有限语句集 Γ , 语句 σ , $\Gamma \vdash_{LK} \sigma \leftrightarrow \Gamma \vdash_{LK^*} \sigma$ 。

则只需证明:

引理 3.1.19 对任意有限语句集 Γ , 语句 σ , $\Gamma \vdash_{LK^*} \sigma \rightarrow \Gamma \vdash_{m.f.LK^*} \sigma$ 。

进而只需证明:

引理 3.1.20 对任意有限语句集 $\Gamma \Delta$, 若 $\Gamma \Rightarrow_{LK^*} \Delta$ 且其推导过程中只有最后一步使用了 L_{mix} 规则, 则 $\Gamma \Rightarrow_{m.f.LK^*} \Delta$ 。

因即有:

引理 3.1.19 的证明: 由对推导树的数学归纳可得。□

对引理 3.1.20 的证明, 我们需要在两个维度上对推导树进行数学归纳。首先我们给出这两个维度相关概念的定义。令这里所关注的推导树 t 中最后的 L_{mix} 规则对应的混合语句为 α 。称任意式 φ 中所含的逻辑联结词和量词符号的数量为 φ 的度, 写作 $deg(\varphi)$ 。则第一个维度是 $deg(\alpha)$, 也称 t 的度, 写作 $deg(t)$ 。

称 $\Gamma \Rightarrow \Delta$ 中 Γ 为前项, Δ 为后项。对推导树, 称由叶节点向下到根结点为一条路径。对我们所关注的任意推导树 t , 令路径集合 L 表示所有经过左上式的路径, 对 $l \in L$, 令自左上式起向上连续后项中含语句 α 的节点个数为 $r_L(l)$, 令 $rank_L(t) = \max_{l \in L} r_L(l)$ 。可以看到由左上式后项含 α 有 $rank_L(t) \geq 1$ 。类似的定义 $rank_R(t)$, 注意其中采用的是连续前项中含 α 的节点个数。则第二个维度为 $rank(t) = rank_L(t) + rank_R(t)$, 称为 t 的秩, 有 $rank(t) \geq 2$ 。

引理 3.1.20 的证明: 令推导树 t 仅最后一步使用了 L_{mix} 规则。令其中混合语句为 α 。写上标 $^\circ$ 表示语句集不含 α 。分类讨论:

- L_{mix} 的左上式前项含 α 。即 t 为:

$$\frac{\begin{array}{c} \vdots \\ \Gamma^\circ; \alpha \Rightarrow \Delta^\circ; \alpha \end{array} \quad \begin{array}{c} \vdots \mathcal{D} \\ \Sigma^\circ; \alpha \Rightarrow \Pi \end{array}}{\Gamma^\circ; \alpha \cup \Sigma^\circ \Rightarrow \Delta^\circ \cup \Pi} L_{mix}$$

则在 $\mathcal{D}$ 得到 $\Sigma^\circ; \alpha \Rightarrow_{m.f. LK^*} \Pi$ 的基础上按需应用 L_{1a} 、 L_{1b} 可得 $\Sigma^\circ; \alpha \cup \Sigma \Rightarrow_{m.f. LK^*} \Delta^\circ \cup \Pi$ 。注意这里涵盖了左上式由 L_0 引入的情况。

- 否则 L_{mix} 的右上式后项含 α 。同理并涵盖了右上式由 L_0 引入的情况。

否则，进一步令 $d = \deg(t)$, $r = \text{rank}(t)$ 。归纳假设对任意 $d' < d \vee (d' = d \wedge r' < r)$ 时 L_{mix} 是可消去的。分类：

- $r = 2$ ，即 $\text{rank}_L(t) = \text{rank}_R(t) = 1$ 。由 $\text{rank}_L(t) = 1$ ， L_{mix} 的左上式不能由如 L_{1a} 引入。去除类似情况后，可区分以下几种场景：

1. L_{mix} 的左上式由 L_{1b} 引入。即 t 为：

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma^\circ \Rightarrow \Delta^\circ \end{array} \quad \begin{array}{c} \vdots \\ \Sigma^\circ; \alpha \Rightarrow \Pi^\circ \end{array} \quad L_{1b}}{\Gamma^\circ \Rightarrow \Delta^\circ; \alpha} \quad \frac{\Gamma^\circ \Rightarrow \Delta^\circ; \alpha \quad \Sigma^\circ; \alpha \Rightarrow \Pi^\circ}{\Gamma^\circ \cup \Sigma^\circ \Rightarrow \Delta^\circ \cup \Pi^\circ} L_{mix}$$

则在 $\mathcal{D}$ 得到 $\Gamma^\circ \Rightarrow_{m.f. LK^*} \Delta^\circ$ 的基础上按需应用 L_{1a} 、 L_{1b} 可得 $\Gamma^\circ \cup \Sigma^\circ \Rightarrow_{m.f. LK^*} \Delta^\circ \cup \Pi^\circ$ 。

2. 否则 L_{mix} 的右上式由 L_{1a} 引入。同理。
3. 否则 L_{mix} 的左右上式在引入的过程中还伴随了逻辑联结词 $\neg \vee$ 或量词 $\exists$ 的增加，即度的增加。注意至此已经完成了 $d = 0 \wedge r = 2$ 时的证明。对 $d > 0$ 的情况，我们通过 $d' < d$ 应用归纳假设：

- $\alpha = \neg \varphi$ 。则 t 为：

$$\frac{\begin{array}{c} \vdots \mathcal{D}_1 \\ \Gamma^\circ; \varphi \Rightarrow \Delta^\circ \end{array} \quad \begin{array}{c} \vdots \mathcal{D}_2 \\ \Sigma^\circ \Rightarrow \Pi^\circ; \varphi \end{array} \quad L_{2b} \quad \begin{array}{c} \vdots \mathcal{D}_2 \\ \Sigma^\circ \Rightarrow \Pi^\circ; \varphi \end{array} \quad L_{2a}}{\begin{array}{c} \Gamma^\circ \Rightarrow \Delta^\circ; \neg \varphi \quad \Sigma^\circ; \neg \varphi \Rightarrow \Pi^\circ \\ \Gamma^\circ \cup \Sigma^\circ \Rightarrow \Delta^\circ \cup \Pi^\circ \end{array}} L_{mix}$$

由归纳假设有存在不含 L_{mix} 的推导过程对应：

$$\frac{\begin{array}{c} \vdots \mathcal{D}_2 \\ \Sigma^\circ \Rightarrow \Pi^\circ; \varphi \end{array} \quad \begin{array}{c} \vdots \mathcal{D}_1 \\ \Gamma^\circ; \varphi \Rightarrow \Delta^\circ \end{array}}{\Sigma^\circ \cup \Gamma^\circ \Rightarrow \Pi^\circ \cup \Delta^\circ} L_{mix}$$

- $\alpha = \varphi \vee \psi$ 。以 L_{3b_1} 为例，则 t 为：

$$\frac{\begin{array}{c} \vdots \mathcal{D}_1 \\ \Gamma^\circ \Rightarrow \Delta^\circ; \varphi \end{array} \quad \begin{array}{c} \vdots \mathcal{D}_2 \\ \Sigma^\circ; \varphi \Rightarrow \Pi^\circ \end{array} \quad \begin{array}{c} \vdots \\ \Sigma^\circ; \psi \Rightarrow \Pi^\circ \end{array} \quad L_{3b_1} \quad \begin{array}{c} \vdots \mathcal{D}_2 \\ \Sigma^\circ; \varphi \Rightarrow \Pi^\circ \end{array} \quad \begin{array}{c} \vdots \\ \Sigma^\circ; \psi \Rightarrow \Pi^\circ \end{array}}{\begin{array}{c} \Gamma^\circ \Rightarrow \Delta^\circ; \varphi \vee \psi \quad \Sigma^\circ; \varphi \vee \psi \Rightarrow \Pi^\circ \\ \Gamma^\circ \cup \Sigma^\circ \Rightarrow \Delta^\circ \cup \Pi^\circ \end{array}} L_{mix}$$

类似的，由归纳假设有存在不含 L_{mix} 的推导过程对应：

$$\frac{\begin{array}{c} \vdots \mathcal{D}_1 \\ \Gamma^\circ \Rightarrow \Delta^\circ; \varphi \end{array} \quad \begin{array}{c} \vdots \mathcal{D}_2 \\ \Sigma^\circ; \varphi \Rightarrow \Pi^\circ \end{array}}{\Gamma^\circ \cup \Sigma^\circ \Rightarrow \Delta^\circ \cup \Pi^\circ} L_{mix}$$

L_{3b_2} 同理。

- $\alpha = \exists x \varphi$ 。则 t 为：

$$\frac{\frac{\frac{\vdots}{\Gamma^\circ \Rightarrow \Delta^\circ; \varphi[x/\tau]}{\Gamma^\circ \Rightarrow \Delta^\circ; \exists x \varphi} \text{L}_{4b} \quad \frac{\frac{\frac{\vdots}{\Sigma^\circ; \varphi[x/c] \Rightarrow \Pi^\circ}}{\Sigma^\circ; \exists x \varphi \Rightarrow \Pi^\circ} \text{L}_{4a}}{\Gamma^\circ \cup \Sigma^\circ \Rightarrow \Delta^\circ \cup \Pi^\circ} \text{L}_{mix}$$

由 c 不能出现在 $\Sigma^\circ \Pi^\circ$ 中以及引理 3.1.17 可得存在推导有 $\Sigma^\circ; \varphi[x/\tau] \Rightarrow \Pi^\circ$ 。则由归纳假设同理有 $\Gamma^\circ \cup \Sigma^\circ \Rightarrow \Delta^\circ \cup \Pi^\circ$ 。

• $rank_L(t) > 1$ 。注意左上式前项中不含 α ，考虑：

1. 左上式由 L_{1a} 引入。则 t 为：

$$\frac{\frac{\frac{\vdots \mathcal{D}_1}{\Gamma^\circ \Rightarrow \Delta^\circ; \alpha}}{\Gamma^\circ; \varphi \Rightarrow \Delta^\circ; \alpha} \text{L}_{1a} \quad \frac{\frac{\vdots \mathcal{D}_2}{\Sigma^\circ; \alpha \Rightarrow \Pi^\circ}}{\Gamma^\circ; \varphi \cup \Sigma^\circ \Rightarrow \Delta^\circ \cup \Pi^\circ} \text{L}_{mix}$$

则由归纳假设有存在不含 L_{mix} 的推导过程对应：

$$\frac{\frac{\vdots \mathcal{D}_1}{\Gamma^\circ \Rightarrow \Delta^\circ; \alpha} \quad \frac{\vdots \mathcal{D}_2}{\Sigma^\circ; \alpha \Rightarrow \Pi^\circ}}{\Gamma^\circ \cup \Sigma^\circ \Rightarrow \Delta^\circ \cup \Pi^\circ} \text{L}_{mix}$$

进一步应用 L_{1a} 可得 $\Gamma^\circ \cup \Sigma^\circ; \varphi \Rightarrow_{m.f. \text{LK}^*} \Delta^\circ \cup \Pi^\circ$ 。

2. 由 $\frac{\Gamma^\circ \Rightarrow \Delta^\circ; \alpha}{\Gamma^\circ \Rightarrow \Delta^\circ; \alpha; \varphi} \text{L}_{1b}$ 引入，其中 $\varphi \neq \alpha$ ，同 1。
3. 类似由 L_{1b} 引入，但 $\varphi = \alpha$ ，同 1，但不需要进一步应用 L_{1b} 。
4. 由 $\frac{\Gamma^\circ \Rightarrow \Delta^\circ; \alpha; \varphi}{\Gamma^\circ; \neg \varphi \Rightarrow \Delta^\circ; \alpha} \text{L}_{2a}$ 引入，其中 $\varphi \neq \alpha$ ，同 1。
5. 类似由 L_{2a} 引入，但 $\varphi = \alpha$ ，则等同于由 L_{1a} 引入相同，按 1 处理。
6. 由 $\frac{\Gamma^\circ; \varphi \Rightarrow \Delta^\circ; \alpha}{\Gamma^\circ \Rightarrow \Delta^\circ; \alpha; \neg \varphi} \text{L}_{2b}$ 引入，其中 $\neg \varphi \neq \alpha$ ，同 1。
7. 类似由 L_{2b} 引入，但 $\neg \varphi = \alpha$ ，则 t 为：

$$\frac{\frac{\frac{\vdots \mathcal{D}_1}{\Gamma^\circ; \varphi \Rightarrow \Delta^\circ; \neg \varphi}}{\Gamma^\circ \Rightarrow \Delta^\circ; \neg \varphi} \text{L}_{2b} \quad \frac{\frac{\vdots \mathcal{D}_2}{\Sigma^\circ; \neg \varphi \Rightarrow \Pi^\circ}}{\Gamma^\circ \cup \Sigma^\circ \Rightarrow \Delta^\circ \cup \Pi^\circ} \text{L}_{mix}$$

则由归纳假设有存在不含 L_{mix} 的推导 $\mathcal{D}_3$ 对应：

$$\frac{\frac{\frac{\vdots \mathcal{D}_1}{\Gamma^\circ; \varphi \Rightarrow \Delta^\circ; \neg \varphi} \quad \frac{\vdots \mathcal{D}_2}{\Sigma^\circ; \neg \varphi \Rightarrow \Pi^\circ}}{\Gamma^\circ; \varphi \cup \Sigma^\circ \Rightarrow \Delta^\circ \cup \Pi^\circ} \text{L}_{mix}$$

考虑推导：

$$\frac{\frac{\frac{\vdots \mathcal{D}_3}{\Gamma^\circ \cup \Sigma^\circ; \varphi \Rightarrow \Delta^\circ \cup \Pi^\circ}}{\Gamma^\circ \cup \Sigma^\circ \Rightarrow \Delta^\circ \cup \Pi^\circ; \neg \varphi} \text{L}_{2b} \quad \frac{\vdots \mathcal{D}_2}{\Sigma^\circ; \neg \varphi \Rightarrow \Pi^\circ}}{\Gamma^\circ \cup \Sigma^\circ \Rightarrow \Delta^\circ \cup \Pi^\circ} \text{L}_{mix}$$

观察其左秩等于 1，右秩等于 $\text{rank}_R(t)$ ，则其秩小于 r ，由归纳假设得存在不含 L_{mix} 的推导与之对应。

8. 由 L_{3a} 引入，同 1，注意对两条路径同时应用归纳假设。

9. 由 $\frac{\Gamma^\circ \Rightarrow \Delta^\circ; \alpha; \varphi}{\Gamma^\circ \Rightarrow \Delta^\circ; \alpha; \varphi \vee \psi} \text{L}_{3b_1}$ 引入，其中 $\varphi \neq \alpha$ 且 $\varphi \vee \psi \neq \alpha$ ，同 1。

10. 类似由 L_{3b_1} 引入，但 $\varphi = \alpha$ ，则等同于由 L_{1b} 引入。

11. 类似由 L_{3b_1} 引入，但 $\varphi \vee \psi = \alpha$ ，同 7。

12. 由 L_{3b_2} 引入，同 L_{3b_1} 。

13. 由 $\frac{\Gamma^\circ; \varphi[x/c] \Rightarrow \Delta^\circ; \alpha}{\Gamma^\circ; \exists x \varphi \Rightarrow \Delta^\circ; \alpha} \text{L}_{4a}$ 引入，同 1。注意若右上式为 $\Sigma^\circ; \alpha \Rightarrow \Pi^\circ$ ，由引理 3.1.17，对不出现在 $\Gamma^\circ \Delta^\circ \Sigma^\circ \Pi^\circ \varphi$ 中的常量 d 存在推导有 $\Gamma^\circ; \varphi[x/d] \Rightarrow_{m.f. \text{LK}^*} \Delta^\circ; \alpha$ 。

14. 由 L_{4b} 引入，同 L_{3b_1} 。

• 否则 $\text{rank}_R(t) > 1$ 。同理。 $\square$

3.1.7 切消定理的应用

仍考虑只含逻辑联结词 $\neg \vee$ 和量词 $\exists$ 的一阶逻辑语言，对任意语句 α ，写 $\text{sub}(\alpha)$ 表示 α 的子式¹，定义如下：

- α 为原子语句，则 $\text{sub}(\alpha) = \{\alpha\}$ 。
- $\alpha = \neg \varphi$ ，则 $\text{sub}(\alpha) = \text{sub}(\varphi); \alpha$ 。 $\alpha = \varphi \vee \psi$ ， $\text{sub}(\alpha) = \text{sub}(\varphi) \cup \text{sub}(\psi); \alpha$ 。
- $\alpha = \exists x \varphi$ ， $\text{sub}(\alpha) = \{\varphi[x/\tau] : \tau \text{ 为任意封闭项}\}; \alpha$ 。

定理 3.1.21 (子式性质) 对任意语句集 Γ ，语句 σ ，若 $\Gamma \vdash \sigma$ ，则存在相应推导树满足其包含的任意语句为 Γ 中语句或 σ 的子式。

证明：c.f. LK 中的所有规则都满足输入集合中语句为输出集合中语句的子式。 $\square$

推论 3.1.21.1 $\nVdash \perp$ ，即 LK 一致。

证明：显然不存在推导树对应 $\emptyset \Rightarrow_{c.f. \text{LK}} \emptyset$ 且其中语句为 $\emptyset$ 中语句的子式。 $\square$

对任意语句或语句集 x ，写 $\text{ns}(x)$ 表示其中包含的非逻辑符号，写 $\text{ps}(x)$ 表示其中的谓词符号。

定理 3.1.22 (克雷格插值定理) 对任意语句 $\gamma \sigma$ 满足 $\gamma \rightarrow \sigma$ ，若 $\text{ps}(\gamma) \cap \text{ps}(\sigma) \neq \emptyset$ ，则存在语句 ρ 满足 $\text{ns}(\rho) \subseteq \text{ns}(\gamma) \cap \text{ns}(\sigma)$ 有 $\gamma \rightarrow \rho$ 且 $\rho \rightarrow \sigma$ 。否则， $\neg \gamma$ 或 σ 。

我们通过以下引理证明²：

¹注意这里（即下述子式性质中所用的）的子式定义不完全同于式结构分析（如对结构归纳）时的所用的子式，后者不对式中自由变量进行替换。

²证明自 G. Takeuti. Proof Theory. Dover books on mathematics. Dover Publications, Incorporated, 2013, pp. 31–33。

引理 3.1.23 对任意语句集 $\Gamma \Delta$ 满足 $\Gamma \Rightarrow \Delta$, 任意 $\Gamma_1 \subseteq \Gamma$, $\Gamma_2 = \Gamma - \Gamma_1$, 任意 $\Delta_1 \subseteq \Delta$, $\Delta_2 = \Delta - \Delta_1$ 。扩展其语言 $\mathcal{L}$ 使存在恒真语句 $\top$ 有 $ns(\top) \cap ns(\mathcal{L}) = \emptyset$ 。则存在 ρ 满足 ρ 中除 $\top$ 外部分的非逻辑符号均属于 $ns(\Gamma_1 \cup \Delta_1) \cap ns(\Gamma_2 \cup \Delta_2)$ 并有 $\Gamma_1 \Rightarrow \Delta_1; \rho$ 且 $\Gamma_2; \rho \Rightarrow \Delta_2$ 。

定理 3.1.22 (克雷格插值定理) 的证明: 即 $\{\gamma\} \Rightarrow \{\sigma\}$, 以 $\Gamma_1 = \{\gamma\}$, $\Delta_1 = \emptyset$ 应用以上引理, 则存在 ρ 满足 $\{\gamma\} \Rightarrow \{\rho\}$ 且 $\{\rho\} \Rightarrow \{\sigma\}$ 。若 $ps(\gamma) \cap ps(\sigma) \neq \emptyset$, 取任意 $P \in ps(\gamma) \cap ps(\sigma)$, 令其位数为 n , 令 $\varphi = \exists x_{1 \dots n} P(x_{1 \dots n})$, 则以 $\varphi \vee \neg \varphi$ 替换 ρ 中 $\top$ 有结果满足其中非逻辑符号均属于 $ns(\gamma) \cap ns(\sigma)$ 。否则, ρ 由 $\top$ 和逻辑联结词/量词构成, 对 ρ 的组成方式做数学归纳有 $\{\gamma\} \Rightarrow \emptyset$ 或 $\emptyset \Rightarrow \{\sigma\}$ 。□

引理 3.1.23 的证明: 令推导树 t 对应 $\Gamma \Rightarrow_{c.f. \text{LK}} \Delta$, 对其高度做数学归纳:

- t 高度等于 1。则 $\Gamma = \Delta = \{\alpha\}$, $\Gamma_1 \Delta_1$ 的选择对应 $\{\alpha\}\{\alpha\}$ 、 $\{\alpha\} \emptyset$ 、 $\emptyset \{\alpha\}$ 和 $\emptyset \emptyset$ 四种情况。可分别取 ρ 为 $\neg \top$ 、 α 、 $\neg \alpha$ 和 $\top$ 满足要求。
- 假设对高度小于等于 n ($n \geq 1$) 的推导树对应结果成立。对任意语句集 $A_1 A_2 B_1 B_2$, 语句 π , 写 $P(A_1, A_2, B_1, B_2, \pi)$ 表示 π 中除 $\top$ 外部分的非逻辑符号均属于 $ns(A_1 \cup B_2) \cap ns(A_2 \cup B_1)$ 并有 $A_1 \Rightarrow B_1; \pi$ 且 $A_2; \pi \Rightarrow B_2$ 。考虑高度等于 $n+1$ 的 t 最后应用的规则, 以以下几种情况为例:

- $\frac{\Gamma' \Rightarrow \Delta}{\Gamma'; \alpha \Rightarrow \Delta} \text{L}_{1a}$ 。若 $\alpha \in \Gamma_1$, 则存在 $\Gamma'_1 \subseteq \Gamma'$ 有 $\Gamma_1 = \Gamma'_1; \alpha$, 令 $\Gamma'_2 = \Gamma' - \Gamma'_1$, 由归纳假设存在 φ 满足 $P(\Gamma'_1, \Gamma'_2, \Delta_1, \Delta_2, \varphi)$ 。则取 $\rho = \varphi$ 同样满足 $P(\Gamma_1, \Gamma_2, \Delta_1, \Delta_2, \rho)$ 。
- $\frac{\Gamma'; \alpha \Rightarrow \Delta \quad \Gamma'; \beta \Rightarrow \Delta}{\Gamma'; \alpha \vee \beta \Rightarrow \Delta} \text{L}_{3a}$ 。若 $\alpha \vee \beta \in \Gamma_1$, 则存在 $\Gamma'_1 \subseteq \Gamma'$ 有 $\Gamma_1 = \Gamma'_1; \alpha \vee \beta$, 令 $\Gamma'_2 = \Gamma' - \Gamma'_1$ 。由归纳假设存在 φ 满足 $P(\Gamma'_1; \alpha, \Gamma'_2, \Delta_1, \Delta_2, \varphi)$ 且存在 ψ 满足 $P(\Gamma'_1; \beta, \Gamma'_2, \Delta_1, \Delta_2, \psi)$ 。取 $\rho = \varphi \vee \psi$ 满足 $P(\Gamma_1, \Gamma_2, \Delta_1, \Delta_2, \rho)$ 。
- $\frac{\Gamma'; \alpha[x/c] \Rightarrow \Delta}{\Gamma'; \exists x \alpha \Rightarrow \Delta} \text{L}_{4a}$ 。若 $\exists x \alpha \in \Gamma_1$, 则存在 $\Gamma'_1 \subseteq \Gamma'$ 有 $\Gamma_1 = \Gamma'_1; \exists x \alpha$, 令 $\Gamma'_2 = \Gamma' - \Gamma'_1$ 。由归纳假设存在 φ 满足 $P(\Gamma'_1; \alpha[x/c], \Gamma'_2, \Delta_1, \Delta_2, \varphi)$ 。因 $ns(\varphi) \subseteq \Gamma'_2 \cap \Delta_2$ 有 $c \notin ns(\varphi)$ 。则取 $\rho = \varphi$ 同样满足 $P(\Gamma_1, \Gamma_2, \Delta_1, \Delta_2, \rho)$ 。
- $\frac{\Gamma \Rightarrow \Delta'; \alpha[x/\tau]}{\Gamma \Rightarrow \Delta'; \exists x \alpha} \text{L}_{4b}$ 。若 $\exists x \alpha \in \Delta_1$, 则存在 $\Delta'_1 \subseteq \Delta'$ 有 $\Delta_1 = \Delta'_1; \exists x \alpha$, 令 $\Delta'_2 = \Delta' - \Delta'_1$ 。由归纳假设存在 φ 满足 $P(\Gamma_1, \Gamma_2, \Delta'_1; \alpha[x/\tau], \Delta'_2, \varphi)$, 即 φ 除 $\top$ 外的非逻辑符号均属于 $ns(\Gamma_1 \cup \Delta'_1; \alpha[x/\tau]) \cap ns(\Gamma_2 \cup \Delta'_2)$ 并有 $\Gamma_1 \Rightarrow \Delta'_1; \alpha[x/\tau]; \varphi$ 且 $\Gamma_2; \varphi \Rightarrow \Delta'_2$ 。
令 $v_{1 \dots n}$ 对应 φ 中含逻辑符号不属于 $ns(\Gamma_1 \cup \Delta'_1; \exists x \alpha) \cap ns(\Gamma_2 \cup \Delta'_2)$ 的全部封闭项。则应用 L_{4b} 有 $\Gamma_1 \Rightarrow \Delta'_1; \exists x \alpha; \exists x_{1 \dots n} \varphi[v_1/x_1] \dots [v_n/x_n]$ 。对 $\Gamma_2; \varphi \Rightarrow \Delta'_2$, 添加不冲突的常量符号 $c_{1 \dots n}$, 由引理 3.1.17 有 $\Gamma_2; \varphi[v_1/c_1] \dots [v_n/c_n] \Rightarrow \Delta'_2$ 。则应用 L_{4a} 有 $\Gamma_2; \exists x_{1 \dots n} \varphi[v_1/x_1] \dots [v_n/x_n] \Rightarrow \Delta'_2$ 。即 $\rho = \exists x_{1 \dots n} \varphi[v_1/x_1] \dots [v_n/x_n]$ 满足 $P(\Gamma_1, \Gamma_2, \Delta_1, \Delta_2, \rho)$ 。

其它情况按类似方式处理。□

事实上, 切消定理、子式性质和克雷格插值定理对与 LK 相对的直觉主义相继式演绎规则 LJ 同样适用。可以看到其中推导规则的应用本质上是对无限推导树空间进行搜索。而这些性质可以为搜索方向提供指导。如若有限语句集 Γ 和语句 σ 中不含量词, 则子式性质可将可对应 $\Gamma \Rightarrow \sigma$ 的推导树的搜索空间限制于有限的范围内, 从而提供 Γ 是否蕴含 σ 的判定方法。即除为已介绍有其它判定方法(如真值表)的经典命题逻辑提供了新的判定方法, 也为直觉主义命题逻辑提供了判定方法。

3.1.8 演绎的实现

需明确以上演绎规则是可有效计算的。本小节以相继式演绎为例，给出在公理集可判定时，肯定任意语句可证明性的 μ -递归函数计算方法。其中我们将使用章节 2.3.4 和 2.3.5 定义的 μ -递归（子）函数。我们将哥德尔编码语言符号、项、式、语句、语句集及推导规则、推导树。

具体的，令可判定公理集 Γ 的特征函数（的 μ -递归函数）为 gamma ，即对任意编码结果为 c 的语句 γ ， $\text{gamma}(c)$ 返回 1 若 $\gamma \in \Gamma$ ，否则返回 0。我们将构造 μ -递归函数 prove ，对任意编码结果为 s 的语句 σ ， $\text{prove}(s)$ 返回 1 若 $\Gamma \vdash \sigma$ ，否则不返回。

最上层，我们搜索 $\Gamma \vdash \sigma$ 的推导树：

- $\text{prove}(s) = C_1^1(\text{Min}_t\{\text{proof}_?(s, t)\})$ 。

其中 $\text{proof}_?(s, t)$ 检查 t 是否是由 c 决定的公理集 Γ 到编码为 s 的语句 σ 的推导树。注意这里 Min_t 使用了最小化算子。我们将通过在 t 中编码入全部检查所需信息以避免进一步使用最小化算子，换句话说， $\text{proof}_?$ 是原始递归的，也就是总是返回的。

由切消定理，我们仅需实现 LK 中 L_0 到 L_4 ，其中我们仅需使用逻辑联结词 $\neg \vee$ 和量词 $\exists$ ¹。

考虑我们的数据结构。我们继续将递归局限²于原始递归的方式下，因此这里的数据结构被设计为满足原始递归的实现。本质上，从推导树到推导规则到语句、式、项、符号构成了最终以符号为叶节点的树，则以如后根遍历该树我们可将整棵树展开为一维数组，在该一维数组基础上，我们可以通过原始递归逐一检查各节点是否合法。 $\text{proof}_?(s)$ 的搜索即尝试找到这样合法的一维数组满足其推导结果为 s 。我们将在各节点上除记录子节点下标和当前节点数据外，还将记录足够的可能被父节点查询的中间数据以避免需要递归查询。技术上，数组中各元素将递归的哥德尔编码入这些数据。

现给出各节点记录的数据。对于不同的项：

- 变量为 $\langle \text{var}, id \rangle$ ，其中 $\text{var} = 0$ ， id 为符号标识。
- 常量为 $\langle \text{const}, id \rangle$ ， $\text{const} = 1$ 。
- 函数构成的项为 $\langle \text{fn}, id, \langle a_{1\dots n} \rangle, \text{form}, \text{free} \rangle$ ， $\text{fn} = 2$ 。 $a_{1\dots n}$ 为各参数子项在数组中的下标（引用）。 form 为对应符号序列的编码，如 $F(x)$ 对应 $\langle \langle \text{fn}, 1, F \rangle, \text{lp}, \langle \text{var}, x \rangle, \text{rp} \rangle$ 的编码结果（1为函数位数），可见我们还需要不冲突的 lp rp 及 cm 对应左右括号和逗号，这样我们即不需要查询子项即可得到完整的项表达式。 free 为该项所含的全部自由变量。见下方 $\text{fn}_?$ 实现。

可见其中首位为类型标识，递增。其中变量和常量为叶节点，函数构成的项为中间节点。类似对于式：

- 原子式、即直接由谓词构成的式 $\langle \text{pred}, id, \langle a_{1\dots n} \rangle, \text{form}, \text{free} \rangle$ 。
- 非 $\langle \text{neg}, a, \text{form}, \text{free}, \text{bound} \rangle$ ， a 为子式下标。 bound 为所含全部约束变量。
- 或 $\langle \text{or}, a, b, \text{form}, \text{free}, \text{bound} \rangle$ 。
- 存在 $\langle \text{exists}, x, a, \text{form}, \text{free}, \text{bound} \rangle$ ， x 为新约束的变量。

结合维护 free 和 bound 可完成式合法性的检查³，如下方 $\text{or}_?$ 。对各推导规则：

- $\langle L_0, a, \text{sequent} \rangle$ 。 a 为引入语句下标。 sequent 为 $\langle \langle c_{1\dots m} \rangle, \langle d_{1\dots n} \rangle \rangle$ 的编码结果，其中 $c_{1\dots m}$ 对应 $\Gamma \Rightarrow \Delta$ 中的 Γ （的 form ）， $d_{1\dots n}$ 对应 Δ ， $c_i d_i$ 为语句的 form 。

¹ 章节 3.1.4。

² 注意以更灵活的方式实现类似原始递归的函数是可行的，比如对于 $f(n) = n < 2 ? 1 : f(n - 1) + f(n - 2)$ ，我们可构造原始递归函数 h 有 $h(n)$ 返回 $f(0)$ 到 $f(n)$ 序列（哥德尔编码后）的值。

³ 除语法检查外，见 78 页注 2。章节 3.1.4 中相继式演绎规则的描述依赖该额外检查。

- $\langle L_{1a}, u, a, sequent \rangle$ 。 $\langle L_{1b}, u, a, sequent \rangle$ 。 u 为上式对应的子推导树下标。
- $\langle L_{2a}, u, a, sequent \rangle$ 。 $\langle L_{2b}, u, a, sequent \rangle$ 。
- $\langle L_{3a}, u_1, u_2, a, b, sequent \rangle$ 。 $\langle L_{3b_1}, u, a, b, sequent \rangle$ 。 $\langle L_{3b_2}, u, a, b, sequent \rangle$ 。
- $\langle L_{4a}, u, a, c, sequent \rangle$ 。 $\langle L_{4b}, u, a, t, sequent \rangle$ 。 其中 a 对应下式中的 $\exists x \alpha$ 。

写 $a[i]$ 表示 $ge(a, i)$ 。可实现：

- $proof_{\gamma}(s, t) = \cdot_3(gn_{\gamma}(t), proof'_{\gamma}(t, ge_{\#}(t)), yield_{\gamma}(t[ge_{\#}(t)], s))$ 。其中 $\cdot_n$ 表示 n 数相乘。 $proof'_{\gamma}$ 如下检查 t 中各节点是否合法。 $yield_{\gamma}$ 检查 t 最终确由 γ 推导出 s 。
- $proof'_{\gamma}(t, i) = (\cdot(valid_{\gamma}(t[i], t, i), proof'_{\gamma}(t, \perp i)) \text{ if } i \text{ else } 1)$ 。
- $valid_{\gamma}(c, t, i) = \cdot(gn_{\gamma}(c), \text{cond}(=_{\gamma}(c.id, var), var_{\gamma}(c), \dots \text{cond}(=_{\gamma}(c.id, L_{4b}), L_{4b?}(c, t, i), 0) \dots))$ 。
其中 $c.id$ 表示 $.id(c)$ ， $.id$ 从 c 中得到 id 字段、即类型标识，并按标识调用相应的检查函数。
- $yield_{\gamma}(e, s) = \cdot(axioms_{\gamma}(e.sequent_{\Gamma}), set_{\gamma}(e.sequent_{\Delta}, set_1(s)))$ 。其中， $.sequent_{\Gamma}$ 、 $.sequent_{\Delta}$ 分别得到 $sequent$ 字段中的前后项，为通过变长数组实现的集合的编码结果。 $set_1(s)$ 返回元素为单个 s 的集合。通过原始递归可实现集合运算，如这里的比较 set_{γ} 。
- $axioms_{\gamma}(as) = axioms'_{\gamma}(as, \perp ge_{\#}(as))$ 。可见实现集合的数组末位为无数据的尾（以简化空集处理）。并 $axioms'_{\gamma}(as, k) = (\cdot(axioms'_{\gamma}(as, \perp k), \gamma(as[k])) \text{ if } k \text{ else } 1)$ 。

下面给出检查具体类型的三个例子：

- $fn_{\gamma}(c, t, i) = \cdot_4(=_{\gamma}(ge_{\#}(c), 5), fn_{args?}(c.args, t, i), fn_{form?}(c.id, c.args, c.form, t), fn_{free?}(c.args, c.free, t))$ 。
其中 $.args$ 、 $.form$ 、 $.free$ 分别得到 $\langle a_{1\dots n} \rangle$ 、 $form$ 和 $free$ 字段。 $fn_{args?}$ 检查 $a_{1\dots n}$ 引用的类型为项，且均小于 i （以避免成环）。 $fn_{form?}$ 、 $fn_{free?}$ 如下检查 $form$ 和 $free$ 字段。
- $fn_{form?}(d, as, f, t) = =_{\gamma}(gnc_3(gn_2(gn_3(fn, ge_{\#}(as), d), lp), fn_{form}(t[as[ge_{\#}(as)]].form, as, t, \perp ge_{\#}(as)), gn_1(rp)), f)$ 。
其中 $gnc_n(g_{1\dots n})$ 连结哥德尔数 $g_{1\dots n}$ 对应的自然数序列，返回连结后序列的哥德尔数，见本小节末。 fn_{form} 调用返回参数对应的哥德尔编码，注意其中需扩展使 $c.form$ 返回任意子项 c 的编码结果，当 c 的类型标识为 var 或 $const$ 时返回 $gn_1(c)$ （否则仍返回 c 中的 $form$ 字段）。
- $fn_{form}(c, as, t, k) = (gnc_3(fn_{form}(t[as[k]].form, as, t, \perp k), gn_1(cm), c) \text{ if } k \text{ else } c)$ 。
- $fn_{free?}(as, f, t) = set_{\gamma}(fn_{free}(as, t, ge_{\#}(as)), f)$ 。 fn_{free} 返回子项自由变量的并集。
- $fn_{free}(as, t, k) = (set_{\cup}(fn_{free}(as, t, \perp k), as[k].free) \text{ if } k \text{ else } set_{\emptyset})$ 。 $set_{\cup}$ 对应并集运算， $set_{\emptyset}$ 对应空集。类似需扩展 $.free$ 以支持 var 和 $const$ 项。

及：

- $or_{\gamma}(c, t, i) = \cdot_5(=_{\gamma}(ge_{\#}(c), 6), ref_{formula?}(c.a, t, i), ref_{formula?}(c.b, t, i), or_{var?}(t[c.a], t[c.b], c.free, c.bound), or_{form?}(t[c.a], t[c.b], c.form))$ 。
其中 $ref_{formula?}$ 检查引用为式且下标小于 i 。 $or_{form?}$ 检查 $c.form$ 是否等于 $(a.form) \vee (b.form)$ 。
- $or_{var?}(a, b, f, d) = \cdot_4(set_{\gamma}(set_{\cap}(a.free, b.bound), set_{\emptyset}), set_{\gamma}(set_{\cap}(a.bound, b.free), set_{\emptyset}), set_{\gamma}(set_{\cup}(a.free, b.free), f), set_{\gamma}(set_{\cup}(a.bound, b.bound), d))$ 。

及：

- $L_{4b}?(c, t, i) = \cdot_6(=?(\text{ge}_{\#}(c, 5)), \text{ref}_{\text{sequent}}?(c.u, t, i), \text{ref}_{\exists}?(c.a, t, i), \text{ref}_{\text{term}}?(c.t, t, i), \text{set}_{=?}(t[c.u].\text{sequent}_{\Gamma}, c.\text{sequent}_{\Gamma}), \text{set}_{=?}(\text{set}_{\downarrow}(\text{set}_{\downarrow}(t[c.u].\text{sequent}_{\Delta}, \text{subst}(t[t[c.a].a].\text{form}, t[c.a].x, t[c.t].\text{form})), t[a].\text{form}), c.\text{sequent}_{\Delta}))$ 。
- $\text{subst}(a, x, t) = \text{subst}'(\text{st}(a[\text{ge}_{\#}(a)], x, t), a, x, t, \perp \text{ge}_{\#}(a))$ 。其中，
 $\text{st}(c, x, t) = \text{cond}(=?(\text{c}, x), t, \text{gn}_1(c))$
 $\text{subst}'(c, a, x, t, k) = (\text{gnc}(\text{subst}'(\text{st}(a[k], x, t), a, x, t, \perp k), c) \text{ if } k \text{ else } c)$

注意在通用 μ -递归函数的帮助下，我们可将 gamma 作为回调函数传入 prove 已得到一个和非逻辑公理集解藕的演绎规则实现。

最后我们实现 gnc_2 ，即：

- $\text{gnc}(a, b) = \text{gnc}'(a, b, \text{ge}_{\#}(b))$ 。其中，
 $\text{gnc}'(a, b, k) = (\text{gnc}''(\text{gnc}'(a, b, \perp k), a[k]) \text{ if } k \text{ else } a)$
 $\text{gnc}''(a, c) = \cdot(a, \wedge(\text{pr}(S(\text{ge}_{\#}(a))), c))$

3.2 一阶算术

3.2.1 等于

考虑在标准解释中被解释为等于关系的二位谓词符号 $=$ 。给出关于 $=$ 的（非逻辑）公理集合 A_E ：

1. $\forall x x = x$ 。
2. 对任意式 α 满足 $x \notin b(\alpha)$ ，令 $u_1 \dots u_n$ 对应 $f(\alpha) \setminus x$ 中元素， $\forall u_1 \dots u_n \forall y, z (y = z \rightarrow (\alpha[x/y] \rightarrow \alpha[x/z]))$ 。

可以看到由该公理集合可以证明如 $\forall x, y (x = y \rightarrow y = x)$ （因 $\forall x, y (x = y \rightarrow (x = x \rightarrow y = x))$ ）和 $\forall x, y, z (x = y \wedge y = z \rightarrow x = z)$ 等。

等于关系在一阶逻辑的应用中十分常见，因此在一些一阶逻辑的定义中，视 $=$ 为语言的一部分，即为语义为等于关系的逻辑符号。相应扩展原子式含 $\tau = v$ ，其中 τv 为任意项。

尽管在一阶逻辑的应用中这两种用法没有可见的区别，但概念上两点值得注意。一是作为谓词符号且给出 A_E 的 $=$ 仍是可解释的，其解释只要满足 A_E 即可。而严格的等于关系要求 $x = y$ 当且仅当 x, y 指代同一物体。二是如尽管对任意式 $\alpha[x_1, x_2]$ ， $\forall u, y, z (y = z \rightarrow (\alpha[x_1/u][x_2/y] \rightarrow \alpha[x_1/u][x_2/z]))$ ，但这和对任意二元关系 P 有 $\forall u, y, z (y = z \rightarrow (P(u, y) \rightarrow P(u, z)))$ 是有区别的。对解释 $\mathfrak{I}$ ，后者中二元关系属于 $\mathcal{P} = \wp(\mathfrak{I}[\mathfrak{V}]^2)$ ，在 $|\mathfrak{I}[\mathfrak{V}]| = \aleph_0$ 时，有 $\mathcal{P}$ 是不可数的。但前者 α 所属集合 $\mathcal{A}$ 是可数的。而这不影响其应用的原因在于 $\mathcal{P} - \mathcal{A}$ 中的元素在一阶逻辑中是不可表达的。我们将在如章节 3.3.2 中看到这种不可表达性带来的影响。

进一步可以为作为逻辑符号的 $=$ 引入推导规则，以相继式演绎为例，在 LK 的基础上可增加：

- 对任意封闭项 τ :

$$\frac{}{\emptyset \Rightarrow \{\tau = \tau\}} \text{L}_{=}$$

- 对任意式 $\alpha[x]$ ，封闭项 τv :

$$\frac{\Gamma \Rightarrow \Delta; \alpha[x/\tau]}{\Gamma; \tau = v \Rightarrow \Delta; \alpha[x/v]} \text{L}_{=a} \quad \frac{\Gamma; \alpha[x/\tau] \Rightarrow \Delta}{\Gamma; \tau = v; \alpha[x/v] \Rightarrow \Delta} \text{L}_{=b}$$

令其为 LK_E 。

定理 3.2.1 对任意有限语句集 Γ ，有限的 $A \subset A_E$ ，语句 σ ， $\Gamma \cup A \vdash_{\text{LK}} \sigma \leftrightarrow \Gamma \vdash_{\text{LK}_E} \sigma$ 。

证明：正方向即若 $\Gamma \cup A \Rightarrow_{\text{LK}} \sigma$ ，有 $\Gamma \cup A \Rightarrow_{\text{LK}_E} \sigma$ ，则通过证明有 $\emptyset \Rightarrow_{\text{LK}_E} A$ 并应用 L_{cut} 可得 $\Gamma \Rightarrow_{\text{LK}_E} \sigma$ 。
 如对 $\tau = v \rightarrow (\alpha[x/\tau] \rightarrow \alpha[x/v])$ 有：

$$\frac{\frac{\frac{\{\alpha[x/\tau]\} \Rightarrow \{\alpha[x/\tau]\}}{\{\alpha[x/\tau], \tau = v\} \Rightarrow \{\alpha[x/v]\}} \text{L}_{=a}}{\emptyset \Rightarrow \{\tau \neq v \vee (\neg \alpha[x/\tau] \vee \alpha[x/v])\}} \text{L}_{2b}, \text{L}_{3b}}$$

反方向可递归的替换推导树中 $\text{L}_{=}$ 等规则的应用。如将由 $\text{L}_{=}$ 引入的 $\emptyset \Rightarrow \{\tau = \tau\}$ 替换为：

$$\frac{\{\tau = \tau\} \Rightarrow \{\tau = \tau\}}{\{\neg \exists x x \neq x\} \Rightarrow \{\tau = \tau\}} \text{L}_2, \text{L}_{4b}$$

将：

$$\frac{\begin{array}{c} \vdots \\ \Gamma; \alpha[x/\tau] \Rightarrow \Delta \end{array}}{\Gamma; \tau = v; \alpha[x/v] \Rightarrow \Delta} \text{L}_{=b}$$

替换为：

$$\frac{\frac{\frac{\frac{\{\alpha_1\} \Rightarrow \{\tau = \tau\}}{\{\alpha_1, \alpha_2, \tau = v\} \Rightarrow \{\tau = \tau\}} \text{LK}}{\{\alpha_1, \alpha_2, \tau = v\} \Rightarrow \{\tau = \tau\}} \text{L}_{cut} \quad \frac{\frac{\frac{\{\alpha_2, \tau = v, \tau = \tau\} \Rightarrow \{\tau = \tau\}}{\{\alpha_3, v = \tau, \alpha[x/v]\} \Rightarrow \{\alpha[x/\tau]\}} \text{LK}}{\{\alpha_3, v = \tau, \alpha[x/v]\} \Rightarrow \{\alpha[x/\tau]\}} \text{L}_{cut}}{\{\alpha_1, \alpha_2, \alpha_3, \tau = v, \alpha[x/v]\} \Rightarrow \{\alpha[x/\tau]\}} \text{L}_{cut} \quad (i)$$

$$\frac{\frac{\frac{\frac{\{\alpha_1, \alpha_2, \alpha_3, \tau = v, \alpha[x/v]\} \Rightarrow \{\alpha[x/\tau]\}}{\Gamma; \alpha_1; \alpha_2; \alpha_3; \tau = v; \alpha[x/v] \Rightarrow \Delta} \text{L}_{cut}}{\Gamma; \alpha_1; \alpha_2; \alpha_3; \tau = v; \alpha[x/v] \Rightarrow \Delta} \text{L}_{cut} \quad \begin{array}{c} \vdots \\ \Gamma; \alpha[x/\tau] \Rightarrow \Delta \end{array}}{\Gamma; \alpha_1; \alpha_2; \alpha_3; \tau = v; \alpha[x/v] \Rightarrow \Delta} \text{L}_{cut}$$

其中 $\alpha_1 = \forall x x = x$ ， $\alpha_2 = \forall x, y (x = y \rightarrow (x = x \rightarrow y = x))$ ， $\alpha_3 = \forall y, z (y = z \rightarrow (\alpha[x/y] \rightarrow \alpha[x/z]))$ 。□

进一步可以证明¹ 切消定理同样适用于 LK_E 并有类似子式性质，即有如 LK_E 也是一致的。

3.2.2 算术语言

考虑数学中最为基本的部分、算术。本章节的主要目标是证明数理逻辑中非常重要的哥德尔第一第二不完备定理，更多关于数学对象的讨论见第四章。而在证明过程中，我们仍有足够的例子以说明一阶算术语言的表达能力，特别的，我们将在本小节和下一小节中建立一阶算术语句和可计算性间的对应联系。同时分别在章节 3.2.4 和章节 3.2.6 给出基础的算术公理系统：罗宾逊算术 **Q** 和皮亚诺算术 **P**，并进一步讨论其与理论不完备性之间的关系。

接下去我们总是将 $=$ 视为逻辑符号。回顾章节 3.1.2 中给出的算术语言 $\mathcal{L}_A$ ，其包括常量符号 **0**，函数符号后继 S ，加 $+$ ，乘 $\cdot$ ，并回顾标准解释 $\mathfrak{N}_\sigma$ 对应算术语言的常规解释。写 Sx 表示 $S(x)$ ， $x + y$ 表示 $+(x, y)$ ， $x \cdot y$ 表示 $\cdot(x, y)$ ，且按惯例处理多个符号同时出现的情况。

¹可参考 Franco Parmentolo and Flavio Previa. The Cut Elimination and the Nonlengthening Property for the Sequent Calculus with Equality. 2017. arXiv: [1705.00693 \[math.LO\]](https://arxiv.org/abs/1705.00693)，其中还包括相应的直觉主义演绎规则 LJ_E 并有更多性质讨论。

- 同时，对任意项 τv ，我们定义 $\tau < v$ 表示 $\exists x Sx + \tau = v$ ，其中 x 不出现在 τv 中。

回顾章节 2.3.4 给出的 μ -递归函数定义，其由三种基本 μ -递归函数，后继 S ，常量 C_c^n 和投射 P_m^n ，以及其上通过复合 $CN_m^n [H, G_1, \dots, G_m]$ ，原始递归 $PR^n [G, H]$ 和极小化 $MN^n [G]$ 三种方式组合得到的 μ -递归函数构成。

对 n 元 μ -递归函数 F ，若存在一阶算术式 $\rho_{[y, x_1 \dots x_n]}$ 对任意自然数 $x_1 \dots x_n$ 和 y 有语句 $\rho_{[y/y][x_1/x_1] \dots [x_n/x_n]}$ 在标准解释 $\mathfrak{N}_{st}$ 下为真当且仅当 $y = F(x_1 \dots x_n)$ ，则称 F 是算术可定义的，称 ρ 算术定义 F 。其中，回顾对任意自然数 c ， $c = \overbrace{S \dots S}^c 0$ ；同时，令 $c = \overbrace{S \dots S}^c 0$ ，称其为 c 的数码。即无衬线字体和粗体分别对应 μ -递归函数和一阶算术语言对自然数的编码方式。我们将证明：

定理 3.2.2 任意 μ -递归函数是算术可定义的。

为此，我们通过以下引理给出一种不同于哥德尔编码的但同样可用于表示任意有限长度自然数序列的编码方式。相对于哥德尔编码，其优势在于不需要使用幂运算，因此我们将看到其可被一阶算术语言直接描述。

引理 3.2.3 (β 函数引理) 对任意自然数序列 $a_0 \dots a_n$ ，存在 b, c 满足对任意 $0 \leq i \leq n$ ， $\beta(b, c, i) = a_i$ 。其中 $\beta(b, c, i) = b \text{ rem } (c * (i + 1) + 1)$ (rem 为取余运算，如 $5 \text{ rem } 2 = 1$)。

证明：由中国剩余定理可知对任意互质（最大公因数为 1）的序列 $t_0 \dots t_n$ ，若 $a_i < t_i$ ，则存在 b 满足 $b \text{ rem } t_i = a_i$ 。可以看到若 $t_i = c * (i + 1) + 1$ ， $t_0 \dots t_n$ 互质且可选取 c 使 $a_i < t_i$ 。□

定理 3.2.2 的证明：我们将递归的构造对应的一阶算术式 ρ ：

1. 对 $y = S(x)$ ，取 ρ 为 $y = Sx$ 。对 $y = C_c^n(x_1 \dots x_n)$ ，取 ρ 为 $y = c$ 。对 $y = P_m^n(x_1 \dots x_n)$ ，取 ρ 为 $y = x_m$ 。
2. 对 $y = CN_m^n [H, G_1, \dots, G_m] (x_1 \dots x_n)$ ，令 $\eta_{[y/y][x_1/x_1] \dots [x_m/x_m]}$ 对应 $y = H(x_1 \dots x_m)$ 。对 $1 \leq i \leq m$ ，令 $\gamma_i_{[y/y][x_1/x_1] \dots [x_n/x_n]}$ 对应 $y = G_i(x_1 \dots x_n)$ 。取 ρ 为：

$$\exists y_1 \dots y_m \eta_{[x_1/y_1] \dots [x_m/y_m]} \wedge \bigwedge_{1 \leq i \leq m} \gamma_i_{[y/y_i]}$$

3. 对 $y = PR^n [G, H] (x_1 \dots x_n)$ ，令 $\gamma_{[y/y][x_1/x_1] \dots [x_{n-1}/x_{n-1}]}$ 对应 $y = G(x_1 \dots x_{n-1})$ ， $\eta_{[y/y][x_1/x_1] \dots [x_{n+1}/x_{n+1}]}$ 对应 $y = H(x_1 \dots x_{n+1})$ 。取 ρ 为：

$$\begin{aligned} & \exists b, c (\exists a \beta[i/0] \wedge \gamma[y/a][x_1/x_2] \dots [x_{n-1}/x_n]) \\ & \wedge (\forall i < x_1 \exists a, a' \beta[a/a'][i/Si] \wedge \eta[y/a'][x_1/i][x_2/a][x_3/x_2] \dots [x_{n+1}/x_n]) \\ & \wedge \beta[a/y][i/x_1] \end{aligned} \quad (i)$$

其中 β 为 $\exists d b = (c * (i + 1) + 1) * d + a$ ，即 $\beta[a/a][b/b][c/c][i/i]$ 对应 β 函数的 μ -递归函数计算结果 $a = \text{beta}(b, c, i)$ 。同时有 $\forall x < \tau \varphi$ (项 τ 不含 x) 表示 $\forall x (x < \tau \rightarrow \varphi)$ ，称其中 $\forall x < \tau$ 为有界全称量词， $\exists x < \tau \varphi$ (τ 不含 x) 表示 $\exists x x < \tau \wedge \varphi$ ，称其中 $\exists x < \tau$ 为有界存在量词。则 (i) 意为存在 b, c 决定的自然数序列有其第零项等于 $G(x_2 \dots x_n)$ ，对任意 $i < x_1$ ，若令第 i 项等于 a ，第 $i + 1$ 项等于 $H(i, a, x_2 \dots x_n)$ 以及第 x_1 项等于 y 。

4. 对 $y = MN^n [G] (x_1 \dots x_n)$ ，令 $\gamma_{[y/y][x_0/x_0][x_1/x_1] \dots [x_n/x_n]}$ 对应 $y = G(x_0 \dots x_n)$ 。取 ρ 为：

$$(\forall x_0 < y \exists z \gamma[y/z] \wedge z \neq 0) \wedge \gamma[y/0][x_0/y] \quad \square$$

这样，任意部分递归函数对应的 μ -递归函数均是算术可定义的。因此也称部分递归函数是算术可定义的。类似的，称半可判定集合 $A \subseteq \mathbb{N}$ 是算术可定义的，令部分递归函数 $c' : A \mapsto \{1\}$ 可被 $\rho_{[x,y]}$ 定义，则 A 可被 $\rho_{[y/1]}$ 定义。我们即在判定问题和算术语句的正确性，即其是否在 $\mathfrak{N}_{st}$ 下为真，之间建立了联系。

扩展自然数子集合的算术可定义性，对任意自然数子集合 A ，称其为算术可定义的若存在算术式 $\rho_{[x]}$ 对任意自然数 x 有语句 $\rho_{[x/x]}$ 在 $\mathfrak{N}_{st}$ 下为真当且仅当 $x \in A$ ，称 ρ 算术定义 A 。注意因 $\wp(\mathbb{N})$ 不可数，存在子集合不可算术定义。我们可扩展定理 3.2.2 至相对可计算 μ -递归函数：

推论 3.2.2.1 对任意算术可定义的集合 $B \subseteq \mathbb{N}$ ，任意 B - μ -递归函数是算术可定义的。

证明：令 B 特征函数的 μ -递归函数 c_B 可被 $\gamma_{[y,x]}$ 定义。同理扩展定理 3.2.2 的证明，由 γ 替换 B - μ -递归函数中对 c_B 的调用。□

进一步，我们将用于定义 μ -递归函数 / 半可判定集合的算术语句重写为 $\exists$ 基本式。具体的，称任意仅由逻辑联结词和有界量词构成的式为基本式，对任意基本式 φ ，任意自由变量 x ，称式 $\exists x \varphi$ 为 $\exists$ 基本式。并称封闭的 $\exists$ 基本式为 $\exists$ 基本语句。

对任意式 $\alpha \beta$ ，称 α 等价于 β ，若令 $x_1 \dots x_n$ 为其中的自由变量符号， $u_1 \dots u_n$ 不出现在 $\alpha \beta$ 中，语句：

$$\forall u_1 \dots u_n (\alpha_{[x_1/u_1] \dots [x_n/u_n]} \leftrightarrow \beta_{[x_1/u_1] \dots [x_n/u_n]})$$

在 $\mathfrak{N}_{st}$ 下为真¹。写作 $\alpha \equiv \beta$ 。

可以看到用于定义 μ -递归函数的算术语句仅由 $=$ 、 $\neq$ 、 $\wedge$ 、有界全称量词和存在量词构成。因此我们仅需证明以下引理即可证明可重写，即任意 μ -递归函数 / 半可判定集合可被 $\exists$ 基本式定义。

引理 3.2.4 对任意仅由 $=$ 、 $\neq$ 、 $\wedge$ 、有界全称量词和存在量词构成的式 α ，存在算术等价于 α 的 $\exists$ 基本式 β 。

证明：对 α 的结构做数学归纳：

1. 若 α 为任意 $\tau = v$ 或 $\tau \neq v$ ，则取不冲突的自由变量符号 u ，取 β 为 $\exists u \alpha$ 。
2. 若 α 为任意 $\varphi \wedge \psi$ 。则存在 $\exists v_1 \delta_1 \equiv \varphi$ ， $\exists v_2 \delta_2 \equiv \psi$ ，取 β 为 $\exists u (\exists v_1 < u \delta_1 \wedge \exists v_2 < u \delta_2)$ 。
3. 若 α 为任意 $\forall x < \tau \varphi$ 。则由归纳假设存在 $\exists$ 基本式 $\exists v \delta \equiv \varphi$ ，取 β 为 $\exists u \forall x < \tau \exists v < u \delta$ 。
4. 若 α 为任意 $\exists x \varphi$ 。则存在 $\exists v \delta \equiv \varphi$ ，取 β 为 $\exists u \exists x < u, v < u \delta$ 。□

这样，我们即在 μ -递归函数 / 半可判定集合和 $\exists$ 基本式之间建立了对应关系。即：

推论 3.2.4.1 任意 μ -递归函数 / 半可判定集合可被 $\exists$ 基本式算术定义。

下面扩展以上结论至更复杂的算术可定义自然数子集合。

3.2.3 算术阶层

我们看到任意半可判定集合均可被 $\exists$ 基本式定义。事实上，任意算术可定义的集合都可被类似形式的式定义。定义：

¹引入 $u_1 \dots u_n$ 仅为保证语句符合我们对约束变量使用的额外要求

- $\Sigma'_0 = \Pi'_0 = \{ \varphi : \varphi \text{ 仅由逻辑联结词和有界量词构成} \}^{\text{注}1}$ 。
- $\Sigma'_{n+1} = \{ \exists x \varphi : \text{任意自由变量 } x \text{ 且 } \varphi \in \Pi'_n \}, \Pi'_{n+1} = \{ \forall x \varphi : \text{任意自由变量 } x \text{ 且 } \varphi \in \Sigma'_n \}$ 。
- $\Sigma_n = \{ \rho \in \Sigma'_n : |f(\rho)| \leq 1 \}, \Pi_n = \{ \rho \in \Pi'_n : |f(\rho)| \leq 1 \}$ 。即其中自由变量数不大于 1，从而可算术定义自然数子集合。并在无特别说明时，总是令其中的自由变量（若有）为 x 。
- $\Delta_n = \Sigma_n \cap \Pi_n$ 。

可以看到 $\exists$ 基本式集合等于 Σ_1 。而为证明对任意算术式 α ，存在 n 及式 β 满足 $\beta \in \Sigma_n \vee \beta \in \Pi_n$ ，我们首先引入前束范式。称逻辑联结词不出现在量词 $\forall \exists$ 之前的语句为前束范式，我们将证明对任意一阶逻辑语句，存在等价的前束范式。如对 $\forall x (\exists y P(y)) \vee (\exists z Q(z) \rightarrow R(x))$ ，有等价的 $\forall x \exists y \forall z P(y) \vee (Q(z) \rightarrow R(x))$ 。

定理 3.2.5 对任意一阶逻辑语句，存在等价的前束范式。

证明：由功能完备性，仅考虑逻辑联结词为 $\neg \rightarrow$ 的语句，则由以下事实可实现一阶逻辑语句到前束范式的重写：

1. $\neg \forall x \varphi \equiv \exists x \neg \varphi, \neg \exists x \varphi \equiv \forall x \neg \varphi$ 。
2. $\forall x \varphi \rightarrow \psi \equiv \exists x (\varphi \rightarrow \psi), \varphi \rightarrow \forall x \psi \equiv \forall x (\varphi \rightarrow \psi)$ 。
3. $\exists x \varphi \rightarrow \psi \equiv \forall x (\varphi \rightarrow \psi), \varphi \rightarrow \exists x \psi \equiv \exists x (\varphi \rightarrow \psi)$ 。

□

而显然 $\Pi_n \subseteq \Sigma_{n+1}$ （类似的 $\Sigma_n \subseteq \Pi_{n+1}$ ），因此 $\bigcup_n \Sigma_n$ （即 $\Sigma_{n \rightarrow \infty}$ ）包含了所有算术可定义集合的定义。

同时，在不引起歧义的情况下写 Σ_n / Π_n 表示算术可定义的自然数子集合的集合。注意²任意由 $\exists$ 基本式定义的集合 A 是半可判定的，即 $A \in \Sigma_1$ ，则 A 是半可判定的。直观的说，任意语句 $\alpha \in \Sigma_0$ 、即不含非有界量词，在 $\mathfrak{R}_M$ 下的真假是可判定的，因其中有界量词所量化的自然数个数是有限的。因此任意语句 $\alpha \in \Sigma_1$ 在 $\mathfrak{R}_M$ 下的真假是半可判定的。结合定理 3.2.2，我们即有：

引理 3.2.6 对任意集合 $A \subseteq \mathbb{N}$ ， $A \in \Sigma_1$ 当且仅当 A 为 $\emptyset^{(0)}$ 递归可枚举的。

其中回顾章节 2.3.8 中关于相对可计算性和图灵跳跃的定义， $\emptyset^{(0)}$ 递归可枚举集合即半可判定集合。我们同样可扩展该结论到任意 Σ_n ($n \geq 1$)。在扩展之前，我们首先引入映射归约关系。

对任意自然数子集合 A, B ，我们称 A 映射归约到 B ，写作 $A \leq_m B$ ，若存在全可计算函数 $f: \mathbb{N} \mapsto \mathbb{N}$ 满足对任意 x ， $x \in A \leftrightarrow f(x) \in B$ 。称 A, B 映射等价，写 $A \equiv_m B$ ，若 $A \leq_m B \wedge B \leq_m A$ 。

对任意自然数子集合 A ，显然有 $A \leq_m A$ 。并有对任意 A, B, C ， $A \leq_m B \wedge B \leq_m C \rightarrow A \leq_m C$ ，因若对任意 $x \in A \leftrightarrow f(x) \in B$ 且 $x \in B \leftrightarrow g(x) \in C$ ，则 $x \in A \leftrightarrow g(f(x)) \in C$ ，其中 f, g 为全可计算函数则 $y = g(f(x))$ 也为全可计算函数。同时，其与图灵归约之间有 $A \leq_m B \rightarrow A \leq_T B$ ，因若存在 μ -递归函数 c_B 可判定 B ，即计算 B 的特征函数 c_B ，则令 f 计算 f ，则 $c_A = \text{CN}_1^1[c_B, f]$ （对应 $y = c_B(f(x))$ ）可判定 A 。但 $A \leq_T B \not\rightarrow A \leq_m B$ ，因，直观的说，映射归约在构成判定 A 的 μ -递归函数时，对判定 B 的函数的调用方式有更严格的限制，我们将在后面引理 3.2.10 的证明中看到具体的例子。现在先证明几个关于映射归约和相对可计算性的引理。

令集合 $A'_0 = \{ gn(i, x) : \text{第 } i \text{ 个一元 } A\text{-}\mu\text{-递归函数在输入 } x \text{ 时返回} \}$ ，其中回顾章节 2.3.8， $gn(i, x)$ 表示代表 i, x 的哥德尔数。并回顾有 $\mathcal{H}_0 = \emptyset'_0$ 。并有 $A' = \{ i : \text{第 } i \text{ 个一元 } A\text{-}\mu\text{-递归函数在输入 } i \text{ 时返回} \}$ ， $\mathcal{H} = \emptyset'$ 。

引理 3.2.7 对任意 $A \subseteq \mathbb{N}$ ， $A' \equiv_m A'_0$ 。³

¹如下述，此处标注'仅为区别式中自由变量的数量，一般材料中不严格区分 Σ'_0 和 Σ_0 。同时，通过上标可进一步扩展至如 Σ_m^n ，其中 m 为约束变量的阶数， $m \neq 0$ 对应高阶逻辑式，见章节 3.3。此处 Σ_n 即对应 Σ_n^0 。

²可通过近本小节尾的图 3.1 对照理解以下对该算术可定义自然数子集合分类法性质的讨论。

³即蕴含 $\mathcal{H} \equiv_m \mathcal{H}_0$ 。

证明：因对任意 $i, i \in A' \leftrightarrow gn(i, i) \in A'_0$ 有 $A' \leq_m A'_0$ 。反方向，类似推论 2.3.11.2 的证明，存在全函数 M ，对任意第 i 个一元 A - μ 递归函数 f_i ，任意输入 x ，存在 $j = M(i, x)$ 满足对任意 y ， $f_j(y) = f_i(x)$ ，则对任意 $g \in A'_0 \leftrightarrow j \in A'$ ，其中 $j = \begin{cases} M(ge(g, 1), ge(g, 2)) & \text{若存在 } k, x \text{ 有 } g = gn(k, x) \\ l & \text{若其它} \end{cases}$ ，而 $ge(g, k)$ 计算哥德尔数 g 的第 k 项， l 为任意永不返回的一元 A - μ -递归函数下标，且可以看到其中的判断条件也是全可计算的。□

引理 3.2.8 对任意集合 $A \subseteq \mathbb{N}$ ， $B \subseteq \mathbb{N}$ ，若 A 为 B 递归可枚举的，则 $A \leq_m B'$ 。

证明：令第 i 个一元 B - μ -递归函数计算 $c' : A \mapsto \{1\}$ ，则对任意 $x \in A \leftrightarrow gn(i, x) \in B'_0$ ，即 $A \leq_m B'_0$ 。又因 $B'_0 \equiv_m B'$ 有 $A \leq_m B'$ 。□

最后，注意对任意自然数子集 A ， $A \in \Pi_n \leftrightarrow \mathbb{N} - A \in \Sigma_n$ ，因如对语句 $\forall x \varphi$ 在 $\mathfrak{N}_M$ 下为真当且仅当 $\exists x \varphi$ 为假。这样我们回到对引理 3.2.6 的扩展。

定理 3.2.9 (波斯特定理) 对任意 $n \geq 1$ ，i：对任意集合 $A \subseteq \mathbb{N}$ ， $A \in \Sigma_n$ 当且仅当 A 为 $\emptyset^{(n-1)}$ 递归可枚举的。ii： $\emptyset^{(n)}$ 是 Σ_n -完备的，即 $\emptyset^{(n)} \in \Sigma_n$ 且对任意集合 A ， $A \in \Sigma_n \rightarrow A \leq_m \emptyset^{(n)}$ 。

证明¹：由引理 3.2.8，这里的 i 蕴含了 ii。下面证明 i。我们对 n 进行数学归纳。引理 3.2.6 已经证明了 $n = 1$ 时的证明。假设 $n = k$ ($k \geq 1$) 时成立，需要证明 $n = k + 1$ 时 i 的两个方向。

对正方向，若 $A \in \Sigma_{k+1}$ 。则存在 φ 有 A 可被 $\exists y \forall z \varphi$ 算术定义且 $\forall z \varphi \in \Pi'_k$ ，即对任意自然数 y ， $\forall z \varphi[y/y] \in \Pi_k$ 。令 $\varphi[y/y]$ 定义集合 B_y ，则存在集合 $\overline{B}_y = \mathbb{N} - B_y$ 有 $\overline{B}_y \in \Sigma_k$ ，显然 $B_y \equiv_T \overline{B}_y$ 。由假设， $\overline{B}_y \leq_m \emptyset^{(k)}$ ，则 $\overline{B}_y \leq_T \emptyset^{(k)}$ ， $B_y \leq_T \emptyset^{(k)}$ 。而对任意 $x \in A$ 当且仅当存在 y 满足 $x \in B_y$ ，因此 $A \leq_T \emptyset^{(k)}$ 。

对反方向，若 A 为 $\emptyset^{(k)}$ 递归可枚举的。因 $\emptyset^{(k)}$ 是 $\emptyset^{(k-1)}$ 递归可枚举的，由假设有存在 $\delta \in \Sigma_k$ 算术定义 $\emptyset^{(k)}$ 。即对任意自然数 x ，若语句 $\delta[x/x]$ 为真，则计算集合 $\emptyset^{(k)}$ 特征函数 $c_{\emptyset^{(k)}}$ 的预言者 $c_{\emptyset^{(k)}}$ 返回 1，否则返回 0。令 $\gamma = (\delta \rightarrow y = 1) \wedge (\neg \delta \rightarrow y = 0)$ ，则显然对任意自然数 $x y$ ，语句 $\gamma[y/y][x/x]$ 为真当且仅当 $y = c_{\emptyset^{(k)}}(x)$ ，即 γ 算术定义 $c_{\emptyset^{(k)}}$ 。

与定理 3.2.2 证明中类似的方式构造定义一元 $\emptyset^{(k)}$ - μ -递归函数的算术式。与引理 3.2.4 类似的，我们归纳证明对任意这样构造的式 α 存在等价的 $\beta \in \Sigma'_{k+1}$ 。则存在 ρ 算术定义 A 且 $\rho \in \Sigma_{k+1}$ ：

1. 若 α 为任意 $\tau = v$ 或 $\tau \neq v$ ，存在 $\beta \in \Sigma'_{k+1}$ 有 $\alpha \equiv \beta$ 。
2. 若 α 为任意 $\gamma[y/v][x/\tau]$ 。以 $k = 2$ 为例，令 $\delta = \exists x_1 \forall x_2 \varphi$ ，则 $\gamma \equiv (\exists x_1 \forall x_2 \varphi \rightarrow y = 1) \wedge (\forall x'_1 \exists x'_2 \neg \varphi \rightarrow y = 0)$ ，按定理 3.2.5 中的规则可重写为 $\exists x'_1 \forall x_1 \forall x'_2 \exists x_2 \psi$ ，其中 ψ 为 $((\varphi \rightarrow y = 1) \wedge (\neg \varphi \rightarrow y = 0))$ 。进一步通过哥德尔编码 $x_1 x'_2$ 可重写为：

$$\exists x'_1 \forall g \exists u \exists x_1 < u, x'_2 < u, x_2 < u \zeta[y/x_1][k/0] \wedge \zeta[y/x'_2][k/1] \wedge \psi$$

其中 ζ 算术定义 $y = gn(g, k)$ 。则应用引理 3.2.4 有 $\beta \in \Sigma'_{k+1}$ 满足 $\alpha \equiv \beta$ 。

3. 若 α 为任意 $\varphi \wedge \psi$ 。由归纳假设存在 $\varphi_0 \in \Sigma_{k-1}$ ， $\psi_0 \in \Sigma_{k-1}$ ， $\alpha \equiv \exists x_1 \forall x_2 \varphi_0 \wedge \exists x'_1 \forall x'_2 \psi_0$ 。同 2。
4. 若 α 为任意 $\forall x < \tau \varphi$ 。由假设存在 $\varphi_0 \in \Sigma_{k-1}$ ， $\alpha \equiv \forall x < \tau \exists x_1 \forall x_2 \varphi_0$ 。以 $k = 2$ 为例，则：

¹证明参考 M. Davis. Computability and Unsolvability. Dover Books on Computer Science. Dover Publications, 2013, pp. 156–161 以及 Wikipedia contributors. Post's theorem — Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/w/index.php?title=Post%27s_theorem&oldid=1029346782. 2021。

$$\begin{aligned}
\alpha &\equiv \forall x < \tau \exists x_1 \forall x_2 \exists x_3 \psi \\
&\equiv \exists g \forall x < \tau \forall x_2 \exists u \exists x_1 < u, x_3 < u \zeta[y/x_1][k/x] \wedge \psi \\
&\equiv \exists g \forall x_2 \forall x < \tau \exists u \exists x_1 < u, x_3 < u \zeta[y/x_1][k/x] \wedge \psi
\end{aligned}$$

直观的说, g 尝试哥德尔编码了 x 等于 0 到 τ 时所有 x_1 的取值。应用引理 3.2.4。

5. 若 α 为任意 $\exists x \varphi$ 。由假设存在 $\varphi_0 \in \Sigma_{k-1}$, $\alpha \equiv \exists x \exists x_1 \forall x_2 \varphi_0$ 。同 2。 $\square$

推论 3.2.9.1 对任意 $n \geq 1$, 任意 $A \in \Delta_n$ 当且仅当 $A \leq_T \emptyset^{(n-1)}$ 。

证明: A 满足 A 且 $\mathbb{N} - A$ 是 $\emptyset^{(n-1)}$ 递归可枚举的当且仅当 A 是 $\emptyset^{(n-1)}$ 可判定的、即 $A \leq_T \emptyset^{(n-1)}$ 。 $\square$

推论 3.2.9.2 对任意 $n \geq 1$, 任意 $\Delta_n \subset \Sigma_n$, $\Pi_n \subset \Pi_n$ 。

证明: $\emptyset^{(n-1)}$ 递归可枚举的 $\emptyset^{(n)} \in \Sigma_n$, 但 $\emptyset^{(n-1)} <_T \emptyset^{(n)}$ 有 $\emptyset^{(n)} \notin \Delta_n$ 。又由任意 A , $A \in \Sigma_n \leftrightarrow \mathbb{N} - A \in \Pi_n$ 及 $A \in \Delta_n \leftrightarrow \mathbb{N} - A \in \Delta_n$, 有 $\Delta_n \subset \Pi_n$ 。 $\square$

进一步, 我们还有:

引理 3.2.10 对任意 $n \geq 2$, $\Sigma_{n-1} \subset \Delta_n$, $\Pi_{n-1} \subset \Delta_n$ 。

证明: 由任意 $A \in \Sigma_{n-1} \rightarrow A \leq_m \emptyset^{(n-1)}$ 及 $A \leq_m \emptyset^{(n-1)} \rightarrow A \leq_T \emptyset^{(n-1)}$ 有 $\Sigma_{n-1} \subseteq \Delta_n$ 。考虑集合:

$$A = \{ gn(m, n) : m \in \emptyset^{(n-1)} \wedge n \notin \emptyset^{(n-1)} \}$$

显然 $A \leq_T \emptyset^{(n-1)}$, 即 $A \in \Delta_n$ 。但若 $A \in \Sigma_{n-1}$, 即 A 是 $\emptyset^{(n-2)}$ 递归可枚举的。则对任意 x 我们可以通过判断 $gn(x, k) \in A$ 或 $gn(l, x) \in A$ 判定是否 $x \in \emptyset^{(n-1)}$, 其中 k 为永不返回的一元 $\emptyset^{(n-1)}$ - μ -递归函数下标, l 为总是返回的下标。即有 $\emptyset^{(n-1)} \leq_T \emptyset^{(n-2)}$, 矛盾。 $\square$

综上所述, 我们可以得到如下图 3.1 的算术可定义自然数子集合的层级结构¹, 称为算术阶层, 也称克莱恩-莫斯托夫斯基阶层。其中, 对任意 $A \subseteq \mathbb{N}$, 写 $\bar{A}$ 表示 A 在 $\mathbb{N}$ 下的补集。对任意集合 B , 集合 $\text{co-}B$ 满足任意 $A \in \text{co-}B \leftrightarrow \bar{A} \in B$ 。

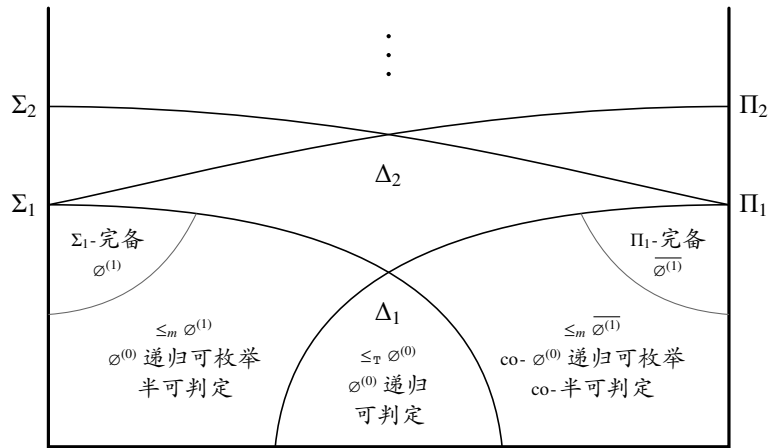


图 3.1: 算术阶层

¹其中还可看到对任意 $n \geq 1$, 存在集合 $A \in \Sigma_n - \Delta_n$ 且不是 Σ_n -完备的。其证明可参考 M. Davis. Computability and Unsolvability. Dover Books on Computer Science. Dover Publications, 2013 定义 5-5.2、定理 5-5.3 及定理 11-3.4。

在本小节的最后，我们通过以下事实证明章节 2.3.8 末宣称的 $\mathcal{A}\mathcal{H} \not\leq_T \mathcal{H}$ 。回顾 $\mathcal{A}\mathcal{H}$ 为总是返回的一元 μ -递归函数下标集合。

引理 3.2.11 $\mathcal{A}\mathcal{H}$ 是 Π_2 -完备的。

证明：由 $\overline{\mathcal{A}\mathcal{H}}$ 是 $\emptyset^{(1)}$ 递归可枚举的可以得到 $\mathcal{A}\mathcal{H} \in \Pi_2$ 。同时，对任意集合 $A \in \Pi_2$ ，令式 $\varphi \in \Pi_0'$ 满足 $x \in A \leftrightarrow \forall y \exists z \varphi[x/z]$ 。则存在全可计算函数 f 满足对任意 x 有下标为 $i = f(x)$ 的一元 μ -递归函数计算 $c' : \{y \in \mathbb{N} : \exists z \varphi[x/z][y/z]\} \rightarrow \{1\}$ ，则 $x \in A \leftrightarrow i \in \mathcal{A}\mathcal{H}$ 。□

推论 3.2.11.1 $\mathcal{A}\mathcal{H} \notin \Delta_2$ 。 $\mathcal{A}\mathcal{H} \not\leq_T \emptyset^{(1)}$ ，即 $\mathcal{A}\mathcal{H} \not\leq_T \mathcal{H}$ 。

证明： $\emptyset^{(1)} <_T \emptyset^{(2)} \equiv_T \overline{\emptyset^{(2)}} \leq_m \mathcal{A}\mathcal{H}$ 。□

又称 $\mathcal{A}\mathcal{H}$ 的判定问题为完全性问题。同时可以看到完全性问题可映射归约到确定两个 μ -递归函数在全部输入下是否均不返回或返回相同结果的问题，称后者为等价性问题，则等价性问题也是 Π_2 -完备的¹。尽管至此已经看到理论上完全性问题和等价性问题是严格难于停机问题的不可判定问题，在程序分析测试领域，我们仍然希望通过各种技术手段检查确定某些特定程序总是返回，或确定某两个程序对相同输入有相同输出。我们称这种程序分析为终止分析和等价性检测，我们将在后续章节讨论。

3.2.4 罗宾逊算术

下面，我们着手建立算术语句的正确性（即其是否在标准解释 $\mathfrak{R}_s$ 下为真）和罗宾逊算术系统 $\mathbf{Q}$ 下语句可证明性之间的联系。具体的说，罗宾逊算术 $\mathbf{Q}$ 的公理集 $A_{\mathbf{Q}}$ 由以下语句构成：

$$\forall x Sx \neq 0 \quad (\text{Q1})$$

$$\forall x, y Sx = Sy \rightarrow x = y \quad (\text{Q2})$$

$$\forall y y = 0 \vee \exists x y = Sx \quad (\text{Q3})$$

$$\forall x x + 0 = x \quad (\text{Q4})$$

$$\forall x, y x + Sy = S(x + y) \quad (\text{Q5})$$

$$\forall x x \cdot 0 = 0 \quad (\text{Q6})$$

$$\forall x, y x \cdot Sy = x \cdot y + x \quad (\text{Q7})$$

我们有：

定理 3.2.12 对任意 $\exists$ 基本语句 σ ， $A_{\mathbf{Q}} \vdash \sigma$ 当且仅当 σ 在 $\mathfrak{R}_s$ 下为真。

证明²：正方向由演绎规则的可靠性直接可得。因显然 $A_{\mathbf{Q}}$ 中的语句在 $\mathfrak{R}_s$ 下为真，由可靠性其推导得到的语句在 $\mathfrak{R}_s$ 下也为真，这也证明了 $\mathfrak{R}_s$ 是 $A_{\mathbf{Q}}$ 的模型。接着主要考虑反方向，对 σ 的结构做数学归纳，其中我们的归纳假设对任意基本语句 σ ，若 σ 在 $\mathfrak{R}_s$ 下为真，则 $A_{\mathbf{Q}} \vdash \sigma$ ，否则 $A_{\mathbf{Q}} \vdash \neg \sigma$ ：

1. 若 σ 为仅由 $S 0$ 构成的（原子）语句。若 σ 在 $\mathfrak{R}_s$ 下为真，则 σ 为对应某自然数 a 的 $a = a$ ³，由 $=$ 的规则直接可得 $A_{\mathbf{Q}} \vdash \sigma$ 。若 σ 为假，即若令 σ 为语句 $a = b$ ，有 $a \neq b$ 。以 $a > b$ 为例，令 $c = a - b$ ，则我们可以通过应用 b 次 (Q2) 从 $A_{\mathbf{Q}} \vdash c \neq 0$ 得到 $A_{\mathbf{Q}} \vdash a \neq b$ 。

¹ 见 E. Börger. Computability, Complexity, Logic. ISSN. Elsevier Science, 1989，其中还包括更多相关问题例子。

² 证明参考 George S. Boolos, John P. Burgess, and Richard C. Jeffrey. Computability and Logic. 5th ed. Cambridge University Press, 2007, pp. 208–210。

³ 回顾 $a = \overbrace{S \dots S}^a 0$ 。

2. 若 σ 为仅由 $=$ 、 S 、 0 和一个 $+$ 构成且为真，则对应 $c = a + b$ ，由 (Q4) (Q5) 有 $A_Q \vdash c = a + b$ 。若 σ 为仅由 $=$ 、 S 、 0 和一个 $\cdot$ 构成且为真，同理但应用 (Q6) (Q7)。
3. 若 σ 仅由 $=$ 、 S 、 0 和一个 $+$ 或 $\cdot$ 构成且为假。以 $=$ 为例，对应 $c \neq a + b$ ，令 $d = a + b$ ，由 1 和 2 有 $A_Q \vdash c \neq d$ 且 $A_Q \vdash d = a + b$ ，有 $A_Q \vdash c \neq a + b$ 。
4. 类似的，若 σ 仅由 $=$ 、 S 、 0 和总数超过一个的 $+$ 和 $\cdot$ 构成。通过对 $+$ 和 $\cdot$ 数量进一步做数学归纳有 σ 为真则 $A_Q \vdash \sigma$ 否则 $A_Q \vdash \neg \sigma$ 。例如对 $d = a + b + c$ ，令 $e = a + b$ ，则 $d = e + c$ 。
5. 这样即完成了 σ 为原子语句时 $\mathfrak{N}_M$ 下真假和 Q 下可证明性对应关系的证明，即 σ 在 $\mathfrak{N}_M$ 下为真，则 $A_Q \vdash \sigma$ ，否则 $A_Q \vdash \neg \sigma$ 。对更复杂的语句，若 σ 为 $\neg \varphi$ 或 $\varphi \Box \psi$ 、 $\Box \in \{\wedge, \vee, \rightarrow, \leftrightarrow\}$ 。由归纳假设和一阶逻辑基本演绎规则可得对应关系。
6. 若 σ 为 $\exists x < \tau \varphi$ ，令 t 为 τ 在 $\mathfrak{N}_M$ 下对应的自然数。我们对 t 做数学归纳，若 $t = 0$ ， σ 为假，由 (Q1) (Q4) (Q5) 及 $<$ 的定义¹ 可得 $\forall x \neg x < 0$ ，即 $A_Q \vdash \neg \sigma$ 。当 $t = u + 1$ ，结合归纳假设同样可得 $A_Q \vdash \exists x < t \varphi \leftrightarrow (\exists x x < u \varphi) \vee \varphi[x/u]$ ，进一步由归纳假设可得 σ 在 $\mathfrak{N}_M$ 下的真假和可证明性之间的关系。对 σ 为 $\forall x < \tau \varphi$ 同理。
7. 最后若 σ 为 $\exists x \varphi$ ，若 $\mathfrak{N}_M$ 下为真，即存在自然数 a 使 $\varphi[x/a]$ 为真，由归纳假设有 $A_Q \vdash \varphi[x/a]$ ，即有 $A_Q \vdash \sigma$ 。注意该构成方式为 $\exists$ 基本语句的最外层构成方式，因此不需要为归纳证明其为假时有 $A_Q \vdash \neg \sigma$ 。 $\square$

事实上，对上述证明中的最后一项，在 σ 为假时，我们并没有 $A_Q \vdash \neg \sigma$ 。因否则语句 $\neg \sigma$ 在 $\mathfrak{N}_M$ 下为真，可确定 σ 为假。即意味着对任意 $\exists$ 基本语句存在判定其是否在 $\mathfrak{N}_M$ 下为真的有效方法。再结合推论 3.2.4.1 指出的半可判定集合和 $\exists$ 基本语句之间的关系，有任意半可判定集合将是可判定的，而停机问题告诉我们存在半可判定但不可判定的集合。这其中的关键在于我们并不能保证 A_Q 模型的域中只有 $\mathfrak{N}_M$ 中的自然数。事实上，由紧性定理 3.1.6 我们可以很方便的构造这样的模型，向算术语言中增加常量 c ，考虑语句集 $\Gamma = A_Q \cup \{c \neq 0, c \neq 1, \dots\}$ ，可见（若 A_Q 可满足） Γ 的任意有限子集是可满足的，则由紧性定理 Γ 可满足，则满足 Γ 的模型有其满足 A_Q 但域中含 c （的对应元素）不等于任意自然数。

而同时，我们也得到了 $\exists$ 基本语句的正确性和其在 A_Q 下的可证明性之间的关系。同样结合半可判定集合和 $\exists$ 基本语句之间的关系，有：

推论 3.2.12.1 对任意 n 元 μ -递归函数 f ，存在 $\exists$ 基本式 $\alpha[y, x_1, \dots, x_n]$ 满足对任意自然数 $x_1, \dots, x_n$ 和 y ， $y = f(x_1, \dots, x_n)$ 当且仅当 $A_Q \vdash \alpha[y/y][x_1/x_1] \dots [x_n/x_n]$ 。

推论 3.2.12.2 对任意半可判定集合 $A \subseteq \mathbb{N}$ ，存在 $\exists$ 基本式 $\alpha[x]$ 满足对任意自然数 x ， $x \in A$ 当且仅当 $A_Q \vdash \alpha[x/x]$ 。

这样还有：

定理 3.2.13 (邱奇定理) 一阶逻辑的恒真语句集是不可判定的。

证明：注意 A_Q 仅含有限的公理集，则对任意语句 σ ， $A_Q \vdash \sigma$ 当且仅当语句 $\wedge A_Q \rightarrow \sigma$ 恒真。而由演绎规则的可靠和完备性， $A_Q \vdash \sigma \leftrightarrow \wedge A_Q \rightarrow \sigma$ 。结合上述推论，我们可以将停机问题规约到一阶逻辑恒真语句集的判定问题上。 $\square$

进一步，对特定语言上的公理集 A, B ，令其对应理论为 T_A, T_B ，称 T_A 弱于 T_B 若 $T_A \subset T_B$ ，称 A 弱于 B 若 T_A 弱于 T_B ，注意 $T_A \subset T_B$ 不代表一定有 $A \subset B$ 。可以认为较弱的公理集含有较少的假设，因此

¹ 回顾定义 $\tau < \nu$ 为 $\exists x Sx + \tau = \nu$ 。

也一般更容易被人们所接受。但其对应的模型也更多，可得到的定理更少。因而也可认为由更弱公理集得到的定理是更强的，既适用于更多模型的。则可认为罗宾逊算术系统 $\mathbf{Q}$ 是相对较弱的算术理论，很多时候其被认为是最弱的基本算术理论。

而由以上推论 3.2.12.2 还可以得到：

引理 3.2.14 一切包含（即强于或等于） $\mathbf{Q}$ 的可公理化的 $\mathfrak{R}_{st}$ 为其模型的理论都是不完备的¹。

而哥德尔第一不完备定理指出：一切包含 $\mathbf{Q}$ 的可公理化的一致理论都是不完备的。因此该引理弱于哥德尔第一不完备定理。事实上我们可以通过规约到其它不可解的问题证明² 哥德尔第一不完备定理。不过我们将在下一小节中给出不依赖计算理论的直接证明。

3.2.5 哥德尔第一不完备定理

为直接证明算术公理系统相关理论的不完备性³。首先我们扩展定理 3.2.12 和推论 3.2.12.1：

引理 3.2.15 对任意包含罗宾逊算术 $\mathbf{Q}$ 的理论 T ($\mathbf{Q} \subseteq T$)，任意 $\exists$ 基本语句 σ ，若 σ 在 $\mathfrak{R}_{st}$ 下为真，则 $T \vdash \sigma$ 。

证明：由公理集 $A_{\mathbf{Q}}$ 得到的理论 $\mathbf{Q} \subseteq T$ 。 □

引理 3.2.16 对任意包含 $\mathbf{Q}$ 的理论 T ，任意 n 元 μ -递归函数 f ，存在 $\exists$ 基本式 $\alpha_{[y/x_1 \dots x_n]}$ 满足对任意自然数 $x_1 \dots x_n$ 和 y ，若 $y = f(x_1 \dots x_n)$ ，则 $T \vdash \alpha_{[y/x_1 \dots x_n]}$ 。

接着，我们对一阶算术式进行哥德尔编码。令编码函数为 gn_A 。则对任意式 α ，有 $g = gn_A(\alpha)$ 为其（在只严格采用基本语法，即不省略括号并不使用简写时的）哥德尔数。进一步写 $\ulcorner \alpha \urcorner = g$ ，即再将自然数 g 编码成一阶算术语言下的符号序列 $\overbrace{S \dots S}^g 0$ ，称其为 α 的哥德尔数码。

在此基础上我们给出对角线引理，直观的说，对角线引理告诉我们对任意可被视为自然数属性判断谓词的式 $\beta_{[y]}$ ，存在语句 σ 为真当且仅当 σ 的哥德尔数具有该属性。即 σ 和 σ 自身相关的属性是等价的，因此也称对角线引理为自指引理。事实上，由以下证明可知其应用和自指悖论有密切联系。本质上可理解为在一个足够有潜力表达如说谎者悖论：我正在说谎 的系统中不应该存在这样的语句，否则无论肯定或否定这样的语句都将得到矛盾，即不一致。

¹回顾章节 2.2.3 末，可公理化理论指其存在公理集是全递归的。但可以看到，这里实际只要求理论是可枚举的，即公理集是可枚举、亦即半递归的。称这样的理论是半递归可公理化的（则也可称可公理化为递归可公理化）。然而，由克雷格定理有：半递归可公理化蕴含可公理化。具体的，对可枚举语句集合 $A = \{\alpha_1, \alpha_2, \dots\}$ ，存在等价的全递归集合 $A_t = \{\overbrace{\alpha_i \wedge \dots \wedge \alpha_i}^i : \text{正整数 } i\}$ 。 A_t 为全递归的因对任意语句 β ，可确定 β 是否等于 α_1 ，或存在 $j > 1$ 满足 $\beta = \overbrace{\alpha_j \wedge \dots \wedge \alpha_j}^j$ 。进一步，克雷格定理还告诉我们：半递归可公理化蕴含原始递归可公理化。具体的，对存在原始递归集合 $A_p = \{\overbrace{\alpha_i \wedge \dots \wedge \alpha_i}^{f(i)} : \text{正整数 } i\}$ ，其中 $f(i)$ 返回 α_i 在函数 $c' : A \mapsto \{1\}$ 对应图灵机 T 中最终返回 1 的计算过程，则对 $\beta = \overbrace{\gamma \wedge \dots \wedge \gamma}^c$ ，存在原始递归函数及其调用 ${}^uG(c, i_T, x)$ （见章节 2.3.6）确定其是否属于 A_p ，其中 i_T 和 x 分别为 T 和 γ 编码后结果。参考 Wikipedia contributors. Craig's theorem — Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/w/index.php?title=Craig%27s_theorem&oldid=1044755808. 2021。

²可参考 C. Smoryński. “The Incompleteness Theorems”. In: vol. 90. Dec. 1977, pp. 821–865。

³本小节证明主要参考 George S. Boolos, John P. Burgess, and Richard C. Jeffrey. Computability and Logic. 5th ed. Cambridge University Press, 2007, pp. 220–226。

同时，其也被称为不动点定理，即 σ 可被视为 β 决定的不动点。技术上，对任意函数 f ，我们称经 f 映射后不变的点为 f 的不动点，即 f 的不动点 x 满足 $x = f(x)$ 。则 σ 为真当且仅当 σ 经由 β 决定的函数映射后得到的语句为真。最后，我们同样将在以下证明中看到其采用了对角论证法。

而对任意式 $\alpha_{[x]}$ ，进一步考虑其与谓词的关系，可认为其定义了一位谓词符号 P_α 有对任意项 τ ， $P_\alpha(\tau) \equiv \alpha_{[x/\tau]}$ （特别的 $P_\alpha(x) \equiv \alpha$ ），称 P_α 是可定义的。则在式的构成中，我们写 $P_\alpha(\tau)$ 等于 $\alpha_{[x/\tau]}$ 。并在不会引起歧义时，我们也直接写 α 表示 P_α ，即 $\alpha(\tau)$ 等于 $\alpha_{[x/\tau]}$ 。以类似方式扩展有含 n 个自由变量的式可定义 $\geq \min\{n, 1\}$ 位谓词符号。

引理 3.2.17（对角线引理） 对任意包含 $\mathbf{Q}$ 的理论 T ，任意式 $\beta_{[y]}$ ，存在语句 σ 有 $T \vdash \sigma \leftrightarrow \beta(\ulcorner \sigma \urcorner)$ 。

证明：构造语句 σ 为：

$$\exists x x = \ulcorner \exists y \text{Diag}(y, x) \wedge \beta \urcorner \wedge (\exists y \text{Diag}(y, x) \wedge \beta)$$

其中构造 Diag 算术定义函数 $v = \text{diag}(u)$ 有若 u 为式 $\gamma_{[x]}$ 的哥德尔数，即若 $u = gn_A(\gamma)$ ，则 $v = gn_A(\exists x x = \ulcorner \gamma \urcorner \wedge \gamma)$ 。对 $v = \text{diag}(u)$ ，由 u 可确定式 $\gamma_{[x]}$ ，进而可确定 γ 的对角化结果 $\exists x x = \ulcorner \gamma \urcorner \wedge \gamma$ ¹，进而可确定 v ，即函数 diag 是可计算的，由推论 3.2.4.1 有 diag 可被 $\exists$ 基本式 $\delta_{[y, x]}$ 定义。取 Diag 等于 δ 。又因 $A_Q \subseteq T$ 及引理 3.2.16 可得若 $v = \text{diag}(u)$ 则 $T \vdash \text{Diag}(v, u)$ ，即对任意 $\gamma_{[x]}$ ：

$$T \vdash \text{Diag}(\ulcorner \exists x x = \ulcorner \gamma \urcorner \wedge \gamma \urcorner, \ulcorner \gamma \urcorner)$$

进一步注意对任意 τ ， $\varphi_{[x]}$ ，有 $\vdash \exists x x = \tau \wedge \varphi \leftrightarrow \varphi(\tau)$ ，则：

$$\begin{aligned} \sigma &\leftrightarrow \exists y \text{Diag}(y, \ulcorner \exists y \text{Diag}(y, x) \wedge \beta \urcorner) \wedge \beta \\ &\leftrightarrow \exists y y = \ulcorner \sigma \urcorner \wedge \beta \\ &\leftrightarrow \beta(\ulcorner \sigma \urcorner) \end{aligned}$$

□

进而有：

定理 3.2.18（塔斯基不可定义定理） 真算术理论 $T = \text{Th}(\mathfrak{N}_{st})$ 中的语句（即在 $\mathfrak{N}_{st}$ 下为真的算术语句）的哥德尔数集、即集合 $A = \{ gn_A(\alpha) : \alpha \in T \}$ 是不可算术定义的。

证明：若存在式 $\rho_{[x]}$ 算术定义 A ，则由对角引理存在语句 σ 有 $T \vdash \sigma \leftrightarrow \neg \rho(\ulcorner \sigma \urcorner)$ 。则：

$$\sigma \in T \leftrightarrow T \vdash \sigma \leftrightarrow T \vdash \neg \rho(\ulcorner \sigma \urcorner) \leftrightarrow gn_A(\sigma) \notin A \leftrightarrow \sigma \notin T$$

□

事实上我们还可以通过算术阶层证明该定理。即可以看到任意算术可定义集合可映射归约到 $T = \text{Th}(\mathfrak{N}_{st})$ ，因此由波斯特定理 T 不属于任意 Σ_n 。

推论 3.2.18.1 真算术理论不可判定。

而更一般的，我们将弱化如上述推论以及引理 3.2.14 中的前提使不要求 $\mathfrak{N}_{st}$ 为其中理论模型。而需要注意由于这种弱化，我们只能使用如引理 3.2.15 这样弱的关于 $\mathfrak{N}_{st}$ 下语句真假和可证明性之间的关系：

引理 3.2.19 任意包含 $\mathbf{Q}$ 的一致理论 T 是不可判定的。

证明：若集合 $A = \{ gn_A(\alpha) : \alpha \in T \}$ 是可判定的，则存在基本式 $\text{Prf}(x, y)$ 满足 $\exists y \text{Prf}(x, y)$ 算术定义集合 A ，或者说 $\exists y \text{Prf}(gn_A(\alpha))$ 在 $\mathbf{Q}$ 下可证即表示 α 可由 $\mathbf{Q}$ 的公理集 A_Q 推导得到（即 $\alpha \in T$ ）在元逻辑下可

¹ 可以看到对角化结果构成了函数 $f(\xi, \zeta) = \exists x x = \ulcorner \xi \urcorner \wedge \zeta$ 的对角线。同时注意 σ 为 $\exists y \text{Diag}(y, x) \wedge \beta$ 的对角化结果。

证：同样，存在基本式 $Disprf(x, y)$ 满足 $\exists y Disprf(x, y)$ 算术定义 $\mathbb{N} - A$ 。则由对角线引理有存在语句 σ ：

$$T \vdash \sigma \leftrightarrow \forall y (Prf(\ulcorner \sigma \urcorner, y) \rightarrow \exists z < y Disprf(\ulcorner \sigma \urcorner, z)) \quad (i)$$

直观的说，对任意语句 α ，自由变量 y ， $Prf(\ulcorner \alpha \urcorner, y) \rightarrow \exists z < y Disprf(\ulcorner \alpha \urcorner, z)$ 表示若语句 α 是可通过由 y 表示的证明过程证明的、即 $\alpha \in T$ ，则存在过程 $z < y$ 证伪 α ，即我们可以更快的证伪 α 、即 $\alpha \notin T$ 。

我们称这种构造下得到的语句 σ 为罗瑟语句。我们将在下面证明 σ 造成了自指悖论。同时，将在后续的讨论中给出一种更简单的构造法以及其得到的弱于当前引理的结论，而由这种更简单构造法得到的语句被称为哥德尔语句。

现指出矛盾所在：

- 若 $T \vdash \sigma$ ，即 $\sigma \in A$ ，则语句 $\exists y Prf(\ulcorner \sigma \urcorner, y)$ 在 $\mathfrak{N}_{st}$ 下为真，则存在自然数 n 有 $Prf(\ulcorner \sigma \urcorner, n)$ 在 $\mathfrak{N}_{st}$ 下为真。由引理 3.2.15 有：

$$T \vdash Prf(\ulcorner \sigma \urcorner, n)$$

且语句 $\neg \exists z < n Disprf(\ulcorner \sigma \urcorner, z)$ 在 $\mathfrak{N}_{st}$ 下为真，注意该语句仍为 $\exists$ 基本语句。则同样有：

$$T \vdash \neg \exists z < n Disprf(\ulcorner \sigma \urcorner, z)$$

但通过 (i) 结合逻辑演绎可得：

$$T \vdash \exists z < n Disprf(\ulcorner \sigma \urcorner, z)$$

即 T 不一致。

- 若 $T \nvdash \sigma$ ，即 $\sigma \in \mathbb{N} - A$ ，则语句 $\exists z Disprf(\ulcorner \sigma \urcorner, z)$ 在 $\mathfrak{N}_{st}$ 下为真，存在自然数 n 有 $Disprf(\ulcorner \sigma \urcorner, n)$ 在 $\mathfrak{N}_{st}$ 下为真，则同样：

$$T \vdash Disprf(\ulcorner \sigma \urcorner, n)$$

且 $\exists$ 基本语句 $\forall y (y < n \vee y = n \rightarrow \neg Prf(\ulcorner \sigma \urcorner, y))$ 在 $\mathfrak{N}_{st}$ 下为真，即有：

$$T \vdash \forall y (y < n \vee y = n \rightarrow \neg Prf(\ulcorner \sigma \urcorner, y))$$

又因：

$$A_Q \vdash \forall y y < n \vee y = n \vee n < y$$

$$A_Q \vdash Disprf(\ulcorner \sigma \urcorner, n) \rightarrow \forall y (n < y \rightarrow \exists z < y Disprf(\ulcorner \sigma \urcorner, z))$$

则：

$$T \vdash \forall y (Prf(\ulcorner \sigma \urcorner, y) \rightarrow \exists z < y Disprf(\ulcorner \sigma \urcorner, z))$$

即 $T \vdash \sigma$ ，即矛盾。 □

可以看到以上证明中如 $\exists y Prf(\ulcorner \sigma \urcorner, y)$ 在 $\mathfrak{N}_{st}$ 中为真当且仅当存在有效方法应用于 T 以得到 $T \vdash \sigma$ 、即 σ 在 T 中可证。需要注意这种有效方法存在于元逻辑层面，而 σ 在 T 中的属性位于目标逻辑层面，因此对角线引理（及引理 3.2.15）使我们可以将语句在目标逻辑中的属性与其在元逻辑中关于其在目标逻辑的可证明性之间建立联系，从而构造自指悖论。接着，我们即有：

定理 3.2.20（哥德尔第一不完备定理） 一切包含 Q 的可公理化的一致理论都是不完备的。

证明：因若理论是可公理化且一致且完备的，则其是可判定的。 □

推论 3.2.20.1 对任意包含 $\mathbf{Q}$ 的可公理化的一致理论 T ，存在语句 σ 既不能在该理论中被证明，也不能被证伪，即 $T \nvdash \sigma \wedge T \nvdash \neg \sigma$ 。

也就是说，我们并不能通过可计算方法扩展这样的理论 T 的公理集使其变得完备。特别的，虽然对上述语句 σ 、即 $T \nvdash \sigma \wedge T \nvdash \neg \sigma$ ，我们可以通过将要么 σ 要么 $\neg \sigma$ 添加进 T 的公理集中使其可被证明或可被证伪，且新的理论 T' 显然仍包含 $\mathbf{Q}$ 且可公理化且一致的。但我们仍能通过同样的方法构造出新的语句 σ' 有 $T' \nvdash \sigma' \wedge T' \nvdash \neg \sigma'$ 。

更具体的，尽管我们还没有开始形式化我们当前使用的元逻辑，但已经可以看到其应蕴含 $A_{\mathbf{Q}}$ ，且其应是可公理化且一致的，那么其无法是完备的。

下面介绍可得到哥德尔语句的更简单的构造方法。仍按引理 3.2.19 证明中的方式定义式 $\exists y \text{Prf}(x, y)$ 。注意在可公理化的一阶逻辑理论中，该式是存在的，其可通过逻辑演绎规则构造得到。则存在哥德尔语句 σ 有：

$$T \vdash \sigma \leftrightarrow \neg \exists y \text{Prf}(\ulcorner \sigma \urcorner, y)$$

直观的说，其意为 σ 可证当且仅当其不可证。然而仍需注意 $\mathfrak{M}_T$ 不一定是 T 的模型。这样，尽管对于假设 $T \vdash \sigma$ ，有 $T \vdash \exists y \text{Prf}(\ulcorner \sigma \urcorner, y)$ ，从而通过理论一致即可造成矛盾；但并没有假设 $T \nvdash \sigma$ 可造成矛盾。具体的，我们不能通过 $\exists y \text{Prf}(\ulcorner \sigma \urcorner, y)$ 在 $\mathfrak{M}_T$ 下为假、即对任意自然数 n 有 $\exists$ 基本语句 $\neg \text{Prf}(\ulcorner \sigma \urcorner, n)$ 为真、即对任意自然数 n 有：

$$T \vdash \neg \text{Prf}(\ulcorner \sigma \urcorner, n)$$

得到：

$$T \vdash \forall y \neg \text{Prf}(\ulcorner \sigma \urcorner, y)$$

即 $T \vdash \neg \exists y \text{Prf}(\ulcorner \sigma \urcorner, y)$ 。而这里可通过以下额外假设产生矛盾：

$$\text{对任意 } n \in \mathbb{N} \text{ 有 } T \vdash \neg \text{Prf}(\ulcorner \sigma \urcorner, n) \quad \text{蕴含} \quad T \nvdash \neg \forall y \neg \text{Prf}(\ulcorner \sigma \urcorner, y) \quad (\text{i})$$

$$T \nvdash \neg \forall y \neg \text{Prf}(\ulcorner \sigma \urcorner, y) \quad \text{蕴含} \quad T \vdash \neg \forall y \neg \text{Prf}(\ulcorner \sigma \urcorner, y) \quad (\text{ii})$$

称任意算术语言构成的理论 T 为 ω -一致¹ 的若对任意式 P 有若对任意自然数 n ， $T \vdash P(n)$ ，则有 $T \vdash \exists x P(x)$ 。这样不满足假设 (i) 意味着 T 非 ω -一致的，不满足假设 (ii) 意味着 T 不完备。意味着不产生矛盾要求 T 要么 ω -不一致，要么不完备。即由哥德尔语句的构造法可得：一切包含 $\mathbf{Q}$ 的可公理化的 ω -一致的理论都是不完备的。

进一步考虑上述哥德尔语句 σ ，通过以上讨论，可以看到若语句 $\exists y \text{Prf}(\ulcorner \sigma \urcorner, y)$ 在 $\mathfrak{M}_T$ 下为真则有矛盾，即 $\neg \exists y \text{Prf}(\ulcorner \sigma \urcorner, y)$ 为真。同时因 $A_{\mathbf{Q}} \vdash \sigma \leftrightarrow \neg \exists y \text{Prf}(\ulcorner \sigma \urcorner, y)$ ，语句 $\sigma \leftrightarrow \neg \exists y \text{Prf}(\ulcorner \sigma \urcorner, y)$ 在 $\mathfrak{M}_T$ 下也为真。可得 σ 也为真。这样我们即得到了在 $\mathfrak{M}_T$ 下为真但在包含 $\mathbf{Q}$ 的可公理化的一致理论 T 中不可证的语句。

事实上，结合演绎规则的可靠和完备性，可公理化的理论 T 中存在语句如 σ 不可证明且不可证伪意味着其存在 σ 为真和为假的不同模型。因此，如在章节 2.2.4 中所述，一致且完备的理论可以使我们通过演绎规则确定具体模型下语句的真假，特别的，可以帮助我们确定标准模型下语句的真假。而可公理化的前提集又使演绎规则的输入是可计算的，从而使语句的真假成为可判定的。然而哥德尔第一不完备定理告诉我们在保持理论可处理（即至少半可判定）的前提下，使语句可证等价于标准模型上为真的努力无法成功。或更一般的，也说明了一阶逻辑语句在一些模型中的真假无法严格对应到其在任意理论中的可证明性，即反映了语义和语法概念上的不一致。

¹ ω 表示第一个大于任意自然数的序数。不同于表示集合大小的基数，其用于表示集合中元素排序后所在的位置。可以看到其也可视为无限。我们将在下一章对其进行进一步讨论。

最后，由理论 T 的哥德尔语句 σ 在 $\mathfrak{N}_M$ 下为真但在 T 中不可证，我们可以将 $\neg\sigma$ 加入到 T 的公理集中得到理论 T' 。则其还是一致的，即可满足的，即存在解释 $\mathfrak{S}$ 为 T' 的模型。显然因 σ 在两个模型中的真假不同， $\mathfrak{S}$ 不是标准模型。进一步分析模型之间的区别，可以看到语句 $\exists y \text{Prf}(\ulcorner\sigma\urcorner, y)$ 在 $\mathfrak{N}_M$ 下为假，而在 $\mathfrak{S}$ 中为真，即在 $\mathfrak{S}$ 的域 $\mathfrak{S}[\mathbb{N}]$ 中存在对象 e 满足对任意自然数 n ， $\mathfrak{S}[n] \neq e$ 。若仍保持 $\mathfrak{S}[n] = n$ ，则可认为 $\mathbb{N}; e \subseteq \mathfrak{S}[\mathbb{N}]$ ，且后继 S ，加 $+$ ，乘 $\cdot$ 的解释都相应扩展以支持 e 及域中其它非自然数对象。比如若增加常量符号 e 对应 e ，则对任意自然数 n ，语句 $n \cdot S e = n \cdot e + n$ 应在 $\mathfrak{S}$ 中为真。

至此，对具有一定强度、即不弱于 Q 的算术理论，哥德尔第一不完备定理明确了其无法对应唯一的模型，即要么没有模型，要么有多于一个的模型。接着，我们尝试确定其并不是没有模型的，即其是可满足的、或一致的，进而可认为是值得讨论的、或非平凡的。然而，事实上，哥德尔第二不完备定理进一步明确了一定强度、如不弱于皮亚诺算术 P 的算术理论并不能确定其自身是一致的。为此，我们首先介绍 P ，并在过程中通过构造其它非标准模型证明其与 Q 的语句可证明性区别。

3.2.6 皮亚诺算术

皮亚诺算术 P 的公理集 A_P 可由在罗宾逊公理集 A_Q 基础上增加以下用于归纳证明的公理模式得到，对任意式 $\rho[u_1 \dots n, x]$ ：

$$\forall u_1 \dots n (\rho[x/0] \wedge \forall x (\rho \rightarrow \rho[x/Sx]) \rightarrow \forall x \rho) \quad (\text{Ind})$$

即直观的说，其含义为若在 0 时满足 ρ 且对任意 x ，在 x 时满足 ρ 意味着在 $x+1$ 时也满足，则任意 x 满足 ρ 。其即对应对自然数的数学归纳法。

由归纳公理模式 (Ind) 我们可以证明原本不能由 A_Q 证明的算术运算规则，如加法 / 乘法的交换律和结合律。我们首先给出几个 A_P 可证语句的例子，接着以示例 3.2.21 证明的语句为例，通过构造非标准模型证明其在 A_Q 下不可证。

示例 3.2.21 $A_P \vdash \forall x \, 0 + x = x$ 。

证明：对归纳公理 $0 + 0 = 0 \wedge \forall x (0 + x = x \rightarrow 0 + Sx = Sx) \rightarrow \forall x \, 0 + x = x$ 的两个条件：

1. 由 (Q4) 有 $0 + 0 = 0$ 。
2. 由 (Q5) 有 $0 + Sx = S(0 + x)$ ，则假设 $0 + x = x$ 有 $0 + Sx = Sx$ ，即 $\forall x (0 + x = x \rightarrow 0 + Sx = Sx)$ 。□

示例 3.2.22 $A_P \vdash \forall y, x \, Sy + x = S(y + x)$ 。

证明：对归纳公理的两个条件：

1. 由 (Q4) 有 $Sy + 0 = S(y + 0)$ 。
2. 由 (Q5) 有 $Sy + Sx = S(Sy + x)$ ，则假设 $Sy + x = S(y + x)$ 有 $Sy + Sx = SS(y + x)$ ，进而由 (Q5) 有 $Sy + Sx = S(y + Sx)$ ，即 $\forall x (Sy + x = S(y + x) \rightarrow Sy + Sx = S(y + Sx))$ 。□

示例 3.2.23 $A_P \vdash \forall x, y \, x + y = y + x$ 。

证明：对归纳公理的两个条件：

1. 由上面的示例 3.2.21 和 (Q4) 有 $0 + y = y + 0$ 。
2. 由示例 3.2.22 有 $Sx + y = S(x + y)$ ，同时由 (Q5) 有 $y + Sx = S(y + x)$ ，则结合假设 $x + y = y + x$ 有 $Sx + y = y + Sx$ 。即 $\forall x (x + y = y + x \rightarrow Sx + y = y + Sx)$ 。□

接着，我们通过构造 A_Q 的非标准模型证明语句 $\forall x \mathbf{0} + x = x$ 在 A_Q 下不可证¹。简单的说，该语句在我们构造的非标准模型下为假，因此由演绎规则的可靠性有其在 A_Q 下不可证。

示例 3.2.24 $A_Q \not\models \forall x \mathbf{0} + x = x$ 。

证明：构造解释 $\mathfrak{N}$ 在标准解释 $\mathfrak{N}_st$ 的基础上增加两个元素 a, b 满足：

$$Sa = a \quad \wedge \quad Sb = b$$

解释符号 $+$ 为如下的 $+$ 和 $\cdot$ ，其中 m, n 为任意满足表中要求的自然数，单元格中内容为所在行 $+$ / $\cdot$ 所在列的结果，如 $m + a = b$ ：

$+$	n	a	b
m	$m + n$	b	a
a	a	b	a
b	b	b	a

$\cdot$	0	$n \neq 0$	a	b
0	0	0	a	b
$m \neq 0$	0	$m \cdot n$	a	b
a	0	b	b	b
b	0	a	a	a

可以看到 $\mathfrak{N}$ 满足全部 A_Q 公理，但 $0 + a = b$ ，即语句 $\forall x \mathbf{0} + x = x$ 不为真。□

此为证明语句不可证的一般方法。同时，可以看到目前为止构造的非标准模型主要是为了说明逻辑学范畴中的问题，章节 5.4.3 我们将看到有非标准模型可以帮助我们处理其它领域，如数学分析中的问题。

3.2.7 哥德尔第二不完备定理²

我们首先给出哥德尔第二不完备定理：

定理 3.2.25（哥德尔第二不完备定理）一切包含 P 的可公理化的一致的理论都不能证明其是一致的。

具体的说，令理论为 T ，按引理 3.2.19 证明中的定义方式，我们有 $\exists$ 基本式 $Prv(x)$ 等于 $\exists y Prf(x, y)$ 满足对任意语句 σ ， $Prv(\ulcorner \sigma \urcorner)$ 在标准解释 $\mathfrak{N}_st$ 下为真当且仅当 σ 在 T 中可证。考虑语句 $\neg Prv(\ulcorner \perp \urcorner)$ ，我们可以同样结合引理 3.2.15 建立起其在 T 下的可证性与 T 的一致性之间的关系。具体的，若 $T \vdash \neg Prv(\ulcorner \perp \urcorner)$ ，则由 T 的一致性可得 $T \not\models Prv(\ulcorner \perp \urcorner)$ ，则由引理 3.2.15 有 $Prv(\ulcorner \perp \urcorner)$ 在 $\mathfrak{N}_st$ 下为假，即 $\perp$ 在 T 中不可证， T 一致。那么不能通过 T 证明 T 一致即指：

$$T \not\models \neg Prv(\ulcorner \perp \urcorner) \quad (3.2.1)$$

我们称其中 $\neg Prv(\ulcorner \perp \urcorner)$ 为理论 T 的一致性语句，写作 Con_T 。

类似第一不完备定理证明中的罗素语句和哥德尔语句，向 T 中添加 Con_T 得到的理论 T' 有新的（ T' 中不可证的）一致性语句 $Con_{T'}$ 。同时，可以看到 T' 可以证明 T 的一致性，则又有 T 不能证明 T' 的一致

¹证明自 P. Smith. Introduction to Gödel's Theorems. 2nd ed. Cambridge introductions to philosophy. Cambridge University Press, 2014, pp. 69–70。

²除章节 3.2.5 中所述哥德尔语句不同于第一不完备定理（正式）证明中使用的罗瑟语句，在哥德尔最初的证明中，其直接使用了（类） P （而非 Q ）并直接指出了其对第一不完备定理的证明过程是可在该 P 下可形式化的，从而直接得到了（之后所谓的）第二不完备定理，而没有给出详细的形式化过程。一方面如下所见形式化过程是繁琐的，另一方面，新的发展得到了更强的如 Q 或下述希尔伯特-博内斯可证明条件下的结果。然而其本质是一致的，其所带来的影响在不完备定理出现初始即是深刻的。进一步，除对第二不完备定理所涉形式化的省略外，哥德尔给出的证明在其它各方面都是简洁且自支持的，其定义了 μ -递归函数，并（依据希尔伯特式演绎规则）给出了（完整的）46 个 μ -递归函数以实现我们在章节 3.1.8 中（略概述的）证明过程计算的形式化，并关注到 μ -递归函数和算术语句之间的关系。可参考 Kurt Gödel. “Some metamathematical results on completeness and consistency, on formally undecidable propositions of principia mathematica and related systems i, and on completeness and consistency”. In: From Frege to Gödel. Ed. by Jean Van Heijenoort. Cambridge: Harvard University Press, 1967, pp. 592–617。

性。这样即有满足第二不完备要求的理论间一致性的可证明关系又是非对称的，又如第四章中将介绍的 ZFC 集合论可以证明 **P** 的一致性，但 **P** 不能证明 ZFC 的一致性。而这样的理论之间由此也可以形成某种树形关系，或强弱对比¹，也意味着可认为我们总是需要假设其中根理论或最强理论的一致性。

为证明 (3.2.1)，我们引入希尔伯特-博内斯可证明条件，或简称可证明条件。称满足以下三个条件的任意式 $P(x)$ 满足 T 的可证明条件，其中 φ, ψ 为任意语句：

$$\text{若 } T \vdash \varphi \text{ 则 } T \vdash P(\ulcorner \varphi \urcorner) \quad (\text{H1})$$

$$T \vdash P(\ulcorner \varphi \rightarrow \psi \urcorner) \rightarrow (P(\ulcorner \varphi \urcorner) \rightarrow P(\ulcorner \psi \urcorner)) \quad (\text{H2})$$

$$T \vdash P(\ulcorner \varphi \urcorner) \rightarrow P(\ulcorner P(\ulcorner \varphi \urcorner) \urcorner) \quad (\text{H3})$$

并引入：

定理 3.2.26（哥德尔第二不完备定理的希尔伯特-博内斯泛化）对任意包含 **P** 的可公理化的一致的理论 T ，任意式 $P(x)$ 满足 T 的可证明条件，则 $T \not\vdash \neg P(\ulcorner \perp \urcorner)$ 。

这样我们的证明被分成了两个部分，第一个部分证明（存在） $Prv(x)$ 满足 **P** 的可证明条件，第二个部分证明以上引理。我们首先证明第二部分，下一小节证明第一部分。为证明第二部分，我们进一步给出：

定理 3.2.27（洛布定理）对任意包含 **Q** 的理论 T ，任意式 $P(x)$ 满足 T 的可证明条件，则对任意语句 γ ，若 $T \vdash P(\ulcorner \gamma \urcorner) \rightarrow \gamma$ ，则 $T \vdash \gamma$ 。

证明²：假设：

$$T \vdash P(\ulcorner \gamma \urcorner) \rightarrow \gamma \quad (\text{i})$$

由对角线引理，存在语句 σ 满足：

$$T \vdash \sigma \rightarrow (P(\ulcorner \sigma \urcorner) \rightarrow \gamma) \quad (\text{ii})$$

$$T \vdash (P(\ulcorner \sigma \urcorner) \rightarrow \gamma) \rightarrow \sigma \quad (\text{iii})$$

应用 (H1) 于 (ii)：

$$T \vdash P(\ulcorner \sigma \rightarrow (P(\ulcorner \sigma \urcorner) \rightarrow \gamma) \urcorner) \quad (\text{iv})$$

应用 (H2)：

$$T \vdash P(\ulcorner \sigma \rightarrow (P(\ulcorner \sigma \urcorner) \rightarrow \gamma) \urcorner) \rightarrow (P(\ulcorner \sigma \urcorner) \rightarrow P(\ulcorner P(\ulcorner \sigma \urcorner) \urcorner) \rightarrow \gamma \urcorner)) \quad (\text{v})$$

由 (iv) 和 (v)：

$$T \vdash P(\ulcorner \sigma \urcorner) \rightarrow P(\ulcorner P(\ulcorner \sigma \urcorner) \urcorner) \rightarrow \gamma \quad (\text{vi})$$

再次应用 (H2)：

$$T \vdash P(\ulcorner P(\ulcorner \sigma \urcorner) \urcorner) \rightarrow \gamma \rightarrow (P(\ulcorner P(\ulcorner \sigma \urcorner) \urcorner) \rightarrow P(\ulcorner \gamma \urcorner)) \quad (\text{vii})$$

由 (vi) 和 (vii)：

$$T \vdash P(\ulcorner \sigma \urcorner) \rightarrow (P(\ulcorner P(\ulcorner \sigma \urcorner) \urcorner) \rightarrow P(\ulcorner \gamma \urcorner)) \quad (\text{viii})$$

应用 (H3)：

$$T \vdash P(\ulcorner \sigma \urcorner) \rightarrow P(\ulcorner P(\ulcorner \sigma \urcorner) \urcorner) \quad (\text{ix})$$

由 (viii) 和 (ix)：

¹这种强弱关系即使不考虑元逻辑和目标逻辑所处的层面不同，也不一定为前述的理论包含关系，如其使用的语言也可以是不同的。

²证明自 George S. Boolos, John P. Burgess, and Richard C. Jeffrey. Computability and Logic. 5th ed. Cambridge University Press, 2007, pp. 236–237。

$$T \vdash P(\ulcorner \sigma \urcorner) \rightarrow P(\ulcorner \gamma \urcorner) \quad (x)$$

由 (i) 和 (x):

$$T \vdash P(\ulcorner \sigma \urcorner) \rightarrow \gamma \quad (xi)$$

由 (iii) 和 (xi):

$$T \vdash \sigma \quad (xii)$$

再次应用 (H1) 于 (xii):

$$T \vdash P(\ulcorner \sigma \urcorner) \quad (xiii)$$

由 (xi) 和 (xiii):

$$T \vdash \gamma \quad \square$$

引理 3.2.26 的证明: 由 T 一致有 $T \not\vdash \perp$, 则应用洛布定理有 $T \not\vdash P(\ulcorner \perp \urcorner) \rightarrow \perp$, 则 $T \not\vdash \neg P(\ulcorner \perp \urcorner)$ 。 $\square$

3.2.8 可证明条件的推导

下面我们回到第一部分。

引理 3.2.28 对任意包含 $\mathbf{P}$ 的可公理化的理论 T , 存在算术定义 T 的 $Prv(x)$ 满足 T 的可证明条件。

其中条件 (H1) 若 $T \vdash \varphi$ 则 $T \vdash Prv(\ulcorner \varphi \urcorner)$ 由引理 3.2.15 可证, 即 $\mathbf{Q} \subseteq T$ 且 $Prv(x)$ 为 $\exists$ 基本式有若 $Prv(\ulcorner \varphi \urcorner)$ 在 $\mathfrak{N}_{st}$ 下为真, 则在 T 下可证。

而对于 (H2) 和 (H3), 因涉及大量 $\mathbf{P}$ 下的形式化证明工作¹, 这里仅给出证明概要²。

首先注意到因其中语句不为 $\exists$ 基本语句, 一直帮助我们建立语句在 $\mathfrak{N}_{st}$ 下的真假和可证明性之间关系的引理 3.2.15 在这里不再适用。具体的, 考虑条件 (H2): $T \vdash Prv(\ulcorner \varphi \rightarrow \psi \urcorner) \rightarrow (Prv(\ulcorner \varphi \urcorner) \rightarrow Prv(\ulcorner \psi \urcorner))$ 。对其中的语句, 展开 $Prv(x)$, 有其等价于前束范式:

$$\forall y_1, y_2 \exists y_3 (Prf(\ulcorner \varphi \rightarrow \psi \urcorner, y_1) \rightarrow (Prf(\ulcorner \varphi \urcorner, y_2) \rightarrow Prf(\ulcorner \psi \urcorner, y_3)))$$

由演绎规则的可靠和完备性有对任意自然数 y_1 对应语句 $\varphi \rightarrow \psi$ 的证明, y_2 对应 φ 的证明, 则存在自然数 y_3 对应 ψ 的证明。即对任意自然数 $y_1 y_2$, 存在自然数 y_3 满足:

$$Prf(\ulcorner \varphi \rightarrow \psi \urcorner, y_1) \rightarrow (Prf(\ulcorner \varphi \urcorner, y_2) \rightarrow Prf(\ulcorner \psi \urcorner, y_3))$$

则由引理 3.2.15, 对任意自然数 $y_1 y_2$:

$$T \vdash \exists y_3 Prf(\ulcorner \varphi \rightarrow \psi \urcorner, y_1) \rightarrow (Prf(\ulcorner \varphi \urcorner, y_2) \rightarrow Prf(\ulcorner \psi \urcorner, y_3))$$

但我们并不能因此得到 (H2), 因存在非标准模型其域中存在非自然数。

同时, 我们并不能通过向公理集中增加语句表示 若对任意自然数 n 有 $P(n)$, 则 $\forall x P(x)$ 。因为我们无法在算术语句中表达变量为自然数、即约束变量 $x \in \mathbb{N}$ 。因否则通过向如公理集 $A_{\mathbf{Q}}$ 中添加语句表示

¹ 可以看到条件 (H3) 是对条件 (H1) 的形式化表达。

² 参考 G. Takeuti. Proof Theory. Dover books on mathematics. Dover Publications, Incorporated, 2013, p. 83, W. Rautenberg. A Concise Introduction to Mathematical Logic. 10th ed. Universitext. Springer New York, 2009, pp. 269–279 以及 R.E. Grandy. Advanced Logic for Applications. Synthese Library. Springer Netherlands, 1979, pp. 74–80。

$\forall x x \in \mathbb{N}$ 可得到理论 Υ 满足其模型的域等于 $\mathbb{N}$, 由此可得其仅有唯一的标准模型¹, 则 Υ 包含 $\mathbf{Q}$ 且可公理化且一致且完备, 但这与哥德尔第一不完备定理矛盾。

但尽管我们不能通过这种方式直接得到 (H2) 和 (H3), 我们仍能在 $\mathbf{P}$ 中证明足够的算术性质以证明这两个条件。类比上述失败的步骤, 考虑更基本的例子。尽管由对任意自然数 x 有 $\mathbf{T} \vdash \exists y x + \mathbf{0} = y$ 不能得到 $\mathbf{i}: \mathbf{T} \vdash \forall x \exists y x + \mathbf{0} = y$ 。但通过 $\mathbf{A_Q}$ 我们可直接证明 $\mathbf{i}$ 。类似的, 尽管由对任意自然数 x 有 $x + \mathbf{0} = x + \mathbf{0}$ 不能得到 $\mathbf{ii}: \mathbf{T} \vdash \forall x x + \mathbf{0} = x + \mathbf{0}$ 。但通过演绎规则可以直接得到 $\mathbf{ii}$ 。直观的说, 这个例子表明在 $\mathbf{Q}$ 的所有模型中, 关系 $\{\langle x, x + \mathbf{0} \rangle : \text{任意 } x\}$ 仍是函数。

写 $\exists!$ 表示存在唯一, 即任意 $\exists!x P(x)$ 等价于 $\exists x P(x) \wedge \forall x_1, x_2 (P(x_1) \wedge P(x_2) \rightarrow x_1 = x_2)$ 。事实上, 我们有:

引理 3.2.29 对任意 n 元原始递归函数 f , 存在式 $\rho_{[x_1 \dots n, y]}$ 算术定义 f 且 $\mathbf{A_P} \vdash \forall x_1 \dots x_n \exists!y \rho$ 。

证明概要: 取 ρ 按定理 3.2.2 证明中的方式构造, 并按构造方法归纳证明 $\mathbf{i}: \mathbf{A_P} \vdash \forall x_1 \dots x_n \exists!y \rho$ 。如对基础 μ -递归函数后继对应的 $y = S(x)$, 显然 $\mathbf{A_P} \vdash \forall x \exists!y y = Sx$ 。需要注意在对原始递归算子对应的构造结果的归纳证明中, 我们需要额外在 $\mathbf{P}$ 中证明 β 函数的性质, 即在 $\mathbf{P}$ 中形式化证明引理 3.2.3。这进一步表示需要在 $\mathbf{P}$ 中形式化证明如中国剩余定理。并在 $\mathbf{P}$ 中应用归纳公理 (Ind) 证明对任意输入 x_1 有 $\mathbf{i}$, 回顾

$$\mathbf{F} = \mathbf{PR}^n [\mathbf{G}, \mathbf{H}] \text{ 要求 } \mathbf{F}(x_1 \dots x_n) = \begin{cases} \mathbf{G}(x_2 \dots x_n) & x_1 = \mathbf{0} \\ \mathbf{H}(y, \mathbf{F}(y, x_2 \dots x_n), x_2 \dots x_n) & x_1 = S(y) \end{cases} \quad \square$$

即在 $\mathbf{P}$ 的任意模型中, 关系 $\{\langle \langle x_1 \dots x_n \rangle, y \rangle : \rho \text{ 为真} \}$ 仍为函数。亦即可向包含 $\mathbf{P}$ 的理论 $\mathbf{T}$ 中增加函数符号 f 和公理 $\forall x_1 \dots x_n, y y = f(x_1 \dots x_n) \leftrightarrow \rho$ 以在相关语句中使用 f 替换 ρ 使其在可证明性不变的情况下变得更简洁²。因此, 我们将尽可能的以此方式替换算术定义原始递归函数的式。

接着, 为通过以上引理证明存在 $\text{Prv}(x)$ 满足 (H2) 和 (H3), 我们给出具体的由原始递归函数以特定有限方式组合而成的 $\text{Bew}(x, y)$ ^{注3} 对应由 y 对应的推导过程可证明 x 对应的语句。注意 Bew 不一定等于 Prf , 因 Prf 为基本式, 而基本式并不能算术定义所有原始递归函数⁴。尽管 Bew 不一定等于 Prf , 但 $\mathbf{P}$ 足以形式化证明类似引理 3.2.4 的结论⁵, 即当我们得到 $\text{Bew}(x, y)$, 我们可构造 $\text{Prf}(x, y)$ 且有 $\mathbf{A_P} \vdash \forall x, y \text{Bew}(x, y) \leftrightarrow \text{Prf}(x, y)$ 。

这样, 我们只需构造并证明 $\exists y \text{Bew}(x, y)$ 满足 (H2) 和 (H3)。通过希尔伯特演绎规则我们可构造这样的 $\text{Bew}(x, y)$ 。回顾章节 3.1.3, 希尔伯特演绎的推导树由唯一的肯定前件规则构成⁶, 则对任意自然数 x 对应语句 σ , 任意自然数 y , $\text{Bew}(x, y)$ 为真当且仅当 y 对应推导树且推导结果为 σ 。或更具体的, y 对应的语句序列 $\delta_{1 \dots n}$ 满足对任意 $1 \leq k < n$, δ_k 为公理或存在 $i < k, j < k$ 有 δ_i 等于 $\delta_j \rightarrow \delta_k$, 且 δ_n 等于 σ 。即 $\delta_{1 \dots n}$ 含 σ 推导树的某一后根遍历结果。或 $\text{Bew}(x, y)$ 等于:

$$G(y) \wedge (\forall k < \#y A(y_{[k]}) \vee \exists i < k, j < k y_{[i]} = y_{[j]} \rightarrow_g y_{[k]}) \wedge x = y_{[\#y-1]} \quad (\mathbf{i})$$

其中 $G(y)$ 为 $gn_\gamma(y) = 1$ 、即检查 y 为哥德尔数。 $\#y$ 为 $ge_\#(y)$ 、即长度。 $y_{[k]}$ 为 $ge(y, k)$ 、即第 k 项。 $A(y_{[k]})$ 检查 $y_{[k]}$ 是否为公理。 $\rightarrow_g$ 计算对应语句的哥德尔数。可以看到所有涉及的函数均为原始递归的⁷。因此

¹或更严格的说, 可以证明其所有模型都与标准模型同构。具体的, (在元逻辑中) 两个解释 ι_J 同构若存在双射函数 $\mathcal{F}$ 由 ι_V 到 ι_J , 且对常量符号 $\mathbf{c}$, $\iota_J[\mathbf{c}] = \mathcal{F}(\iota_V[\mathbf{c}])$ 。对任意 $n \geq 1$, 封闭项 $\tau_{1 \dots n}$, 若 $\bigwedge_{1 \leq i \leq n} \iota_J[\tau_i] = \mathcal{F}(\iota_V[\tau_i])$, 则对任意 n 位函数符号 F , $\iota_J[F(\tau_{1 \dots n})] = \mathcal{F}(\iota_V[F(\tau_{1 \dots n})])$, 对任意 n 位谓词符号 P , $\langle \iota_J[\tau_1], \dots, \iota_J[\tau_n] \rangle \in \iota_J[P]$ 当且仅当 $\langle \mathcal{F}(\iota_V[\tau_1]), \dots, \mathcal{F}(\iota_V[\tau_n]) \rangle \in \iota_V[P]$ 。

²更多关于由存在唯一性定义函数符号的讨论见第四章。

³ Bew 此处为德语 证明 beweis 的缩写, 即对应 Prf 。哥德尔最初使用 Bew 表示 可证明 beweisbar 的缩写, 即对应这里的 Prf 。

⁴见 Patrick Lutz. Primitive recursion and Δ_1^0 . Mathematics Stack Exchange. URL: <https://math.stackexchange.com/q/3700504>。

⁵更精确的, 这里的 $\text{Bew}(x, y)$ 构成除包括引理 3.2.4 中已含的 $=$ 、 $\neq$ 、 $\wedge$ 、有界全称量词、存在量词, 还包括 $\vee$ 。

⁶其可靠和完备性证明可参照相继式演绎, 或参考 Herbert B. Enderton. A Mathematical Introduction to Logic. New York: Academic Press, 1972, pp. 131–145。

⁷可公理化的理论也是原始递归可公理化的, 见 107 页注 1。

由引理 3.2.29, (i) 即为 (增加了相关函数符号后) T 下严格的形式化结果。

显然由语句 $\varphi \rightarrow \psi$ 的推导序列 $\delta_{1\dots n}$ 和 φ 的推导序列 $\pi_{1\dots m}$ 可以得到 ψ 的推导序列 $\delta_{1\dots n}, \pi_{1\dots m}, \psi$ 。则对任意自然数 $y_1 y_2$, 有:

$$T \vdash Bew(\ulcorner \varphi \rightarrow \psi \urcorner, y_1) \rightarrow (Bew(\ulcorner \varphi \urcorner, y_2) \rightarrow Bew(\ulcorner \psi \urcorner, y_1 * y_2 * 2^{S^{\ulcorner \psi \urcorner}}))$$

其中, $y_1 * y_2$ 为连结 $y_1 y_2$ 对应序列后得到的新序列的哥德尔数, 可以看到其也为原始递归的。而对 (H2), 我们需要证明即使在非标准模型中, 对应条件也为真。即需证:

$$T \vdash \forall y_1, y_2 Bew(\ulcorner \varphi \rightarrow \psi \urcorner, y_1) \rightarrow (Bew(\ulcorner \varphi \urcorner, y_2) \rightarrow Bew(\ulcorner \psi \urcorner, y_1 * y_2 * 2^{S^{\ulcorner \psi \urcorner}}))$$

事实上, 该结果可以通过以下事实结合 (i) 得到:

$$\begin{aligned} A_P \vdash \forall x, y (G(x) \wedge G(y) \rightarrow \#(x * y) = \#x + \#y \\ \wedge (\forall i < \#x (x * y)_{[i]} = x_{[i]}) \wedge (\forall i < \#y (x * y)_{[\#x+i]} = y_{[i]})) \end{aligned}$$

这样, 我们即完成了对条件 (H2) 的证明。

对于条件 (H3): $T \vdash Prv(\ulcorner \varphi \urcorner) \rightarrow Prv(\ulcorner Prv(\ulcorner \varphi \urcorner) \urcorner)$, 类似的, 首先考虑 $\mathfrak{N}_s$ 下的情况, 若 $Prv(\ulcorner \sigma \urcorner)$ 在 $\mathfrak{N}_s$ 下为真, 即存在 y 令语句 $Bew(\ulcorner \sigma \urcorner, y)$ 在 $\mathfrak{N}_s$ 下为真, 即 y 对应 $T \vdash \sigma$ 的推导树, 则按照定理 3.2.4 的方式递归构造 $f_0(y)$ 对应 $T \vdash Bew(\ulcorner \sigma \urcorner, y)$, 其中 f_0 为原始递归函数。进一步按照引理 3.2.4 注¹ 可构造 $f(y)$ 对应 $T \vdash Prv(\ulcorner \sigma \urcorner)$, 即 $Bew(\ulcorner Prv(\ulcorner \sigma \urcorner) \urcorner, f(y))$, 其中 f 为原始递归函数。即有语句 $Bew(\ulcorner \sigma \urcorner, y) \rightarrow Bew(\ulcorner Prv(\ulcorner \sigma \urcorner) \urcorner, f(y))$ 在 $\mathfrak{N}_s$ 下为真。则对任意自然数 y :

$$T \vdash Bew(\ulcorner \sigma \urcorner, y) \rightarrow Bew(\ulcorner Prv(\ulcorner \sigma \urcorner) \urcorner, f(y))$$

则同样类似的, 在一系列 P 下形式化证明之后可以得到:

$$T \vdash \forall y Bew(\ulcorner \sigma \urcorner, y) \rightarrow Bew(\ulcorner Prv(\ulcorner \sigma \urcorner) \urcorner, f(y))$$

即有 (H3)。

最后, 需要指出的是可以证明哥德尔第二不完备定理也适用于更弱的算术系统如 Q 注², 但注意通过 Q 并不能证明全部可证明条件, 因此这里的方法并不适用于 Q 。

这样, 我们即完成了对哥德尔不完备定理的讨论。也即基本宣告了希尔伯特计算的失败。该计划希望可以为全部数学提供一套严格的形式化基础, 在其中我们可以形式化的处理任意数学陈述并得到正确的事实判断。

3.2.9 更弱的算术

哥德尔不完备定理对所涉理论的强度要求意味着更弱的算术有望存在一定的完备性。本小节有详略的介绍三种这类不包含罗宾逊算术的可公理化的理论系统: 普雷斯伯格算术、斯科伦算术以及自验证公理系统。其中前两者是完备的, 因此是可判定的; 后者可证明自己的一致性。

¹ 同样需扩展以包含 $\vee$ 。

² 可参考 A. Bezboruah and J. C. Shepherdson. "Gödel's Second Incompleteness Theorem for Q ". in: The Journal of Symbolic Logic 41.2 (1976), pp. 503–512. ISSN: 00224812. URL: <http://www.jstor.org/stable/2272251>。

普雷斯伯格算术 P 的语言由常量符号 0 ，函数符号后继 S ，加 $+$ 组成。其公理集 A_P 包含：

$$\forall x Sx \neq 0 \quad (Q1)$$

$$\forall x, y Sx = Sy \rightarrow x = y \quad (Q2)$$

$$\forall x x + 0 = x \quad (Q4)$$

$$\forall x, y x + Sy = S(x + y) \quad (Q5)$$

$$\forall u_1 \dots u_n (\rho[x/0] \wedge \forall x (\rho \rightarrow \rho[x/Sx]) \rightarrow \forall x \rho) \quad (\text{Ind})$$

其中 ρ 为任意自由变量属于集合 $\{u_1, \dots, u_n, x\}$ 的式。可以看到其语言不包含函数符号乘 $\cdot$ ；相比 A_P ，其公理集不包含 (Q3)、(Q6) 和 (Q7)。

下面将直接通过构造其语句可证性的判定方法证明其是完备的¹。同时在此过程中，我们也可以看到 P 可用于比初看之下更为复杂的场景。具体的，我们将采用量词消去技术构造这里的判定方法。直观的说，可应用量词消去技术于理论 T 指存在可计算方法将 T 中任意带有量词的语句 α 转换为 T 下等价的且不出项量词的语句 $\alpha_{q.f.}$ ，即 $T \vdash \alpha \leftrightarrow \alpha_{q.f.}$ 。我们将看到在 P 中，存在证明或证伪任意经量词消去简化后语句 α 的可计算方法。

现在首先给出（更多）相关符号定义：

- 对任意自然数 n ，同样写 n 表示 $\overbrace{S \dots S}^n 0$ 。
- 对任意项 τ ，自然数 $n \geq 1$ ，写 $n\tau$ 表示 $\overbrace{\tau + \dots + \tau}^n$ 。即可以看到 P 语言可定义被乘数为（正）常数的乘法运算。并按惯例处理多种运算同时出现的情况。
- 对任意项 τ, ν ，自然数 $n \geq 1$ ，写 $\tau \equiv_n \nu$ 表示 $\exists x nx + \tau = \nu$ ，其中 x 不出现在 τ, ν 中。

本质上，我们的量词消去将量词隐藏到符号 $\equiv_n$ 中。以语句 $\exists x nx + a = b$ 为例，由 $\equiv_n$ 定义有 $\vdash \exists x nx + a = b \leftrightarrow a \equiv_n b$ 。而令 σ 为语句 $a \equiv_n b$ ，我们可以进一步检查 σ 在标准解释 $\mathfrak{N}_st$ 下是否为真。即若 $i: a \leq b$ 且 a, b 模 n 同余²，则 $A_P \vdash \sigma$ ，否则 $A_P \vdash \neg \sigma$ ^{注3}。而显然 i 是可判定的。

具体的，对任意语句 α ，令其中变量为 $x_1, \dots, x_n$ ，我们首先将其转换为前束范式 $\neg \exists x_1 \dots \neg \exists x_n \varphi$ ，其中 $\neg$ 表示可能存在的 $\neg$ 。接着，我们重复以下步骤直到 $\exists$ 符号不出现在转换后的结果中：

1. 将当前结果 $\neg \exists x_1 \dots \neg \exists x_k \psi$ 中的 ψ 转换为析取范式的形式，即得到：

$$\neg \exists x_1 \dots \neg \exists x_k (\neg \delta_{11} \wedge \dots \wedge \neg \delta_{1v_1}) \vee \dots \vee (\neg \delta_{u1} \wedge \dots \wedge \neg \delta_{uv_u})$$

其中任意 δ_{ij} 为由 $=$ 或 $\equiv_\square$ 构成的子式（由 $\equiv_\square$ 构成的子式将在步骤 3 中引入）。

2. 移入 $\exists x_k$ 至各个合取子式，即得到：

$$\neg \exists x_1 \dots \neg \exists x_{k-1} \neg ((\exists x_k \neg \delta_{11} \wedge \dots \wedge \neg \delta_{1v_1}) \vee \dots \vee (\exists x_k \neg \delta_{u1} \wedge \dots \wedge \neg \delta_{uv_u}))$$

3. 对每个 $1 \leq i \leq u$ ，将 $\exists x_k \neg \delta_{i1} \wedge \dots \wedge \neg \delta_{iv_i}$ （按后续方法）转换为 $\neg \delta'_{i1} \wedge \dots \wedge \neg \delta'_{iv_i}$ ，其中任意 δ'_{ij}

¹证明自 Ryan Stansifer. Presburger's Article on Integer Airthmetic: Remarks and Translation. Tech. rep. TR84-639. Cornell University, Computer Science Department, 1984。

² a, b 被 n 除后余数相同。

³若为真，则存在 c 有 $nc + a = b$ ，可以证明有 $A_P \vdash nc + a = b$ 。若为假，定义符号 $>$ ，有 $A_P \vdash \forall x x = 0 \vee \dots \vee x = b \vee x > b$ ，并有对任意 $i \leq b$ ， $A_P \vdash ni + a \neq b$ 以及 $A_P \vdash \forall x (x > b \rightarrow nx + a \neq b)$ 。参考 hmakholm left over Monica. Proving decidability of $(\mathbb{N}, +)$ with Quantifier elimination and evaluating basic formulas. Mathematics Stack Exchange. URL: <https://math.stackexchange.com/q/2732733>。

由 = 或 $\equiv_{\square}$ 构成。得到语句：

$$\neg ? \exists x_1 \dots \neg ? \exists x_{k-1} \psi'$$

为实现步骤 3 中对每个子式使用的转换方法，我们重复执行以下步骤直至在步骤中返回：

1. 对当前结果 $\exists x \theta_1 \wedge \dots \wedge \theta_k$ ，其中任意 θ_i 为 $\neg ? \delta_i$ ， δ_i 由 = 或 $\equiv_{\square}$ 构成。若全部 θ_i 不含 x ，则返回 $\theta_1 \wedge \dots \wedge \theta_k$ 。
2. 否则，若存在 θ_i 为任意 $\neg \tau \equiv_a v$ ，因：

$$\neg \tau \equiv_a v \quad \text{等价于} \quad \perp \vee \tau + 1 \equiv_a v \vee \dots \vee \tau + a - 1 \equiv_a v$$

结合任意 $\alpha \wedge (\beta \vee \gamma) \equiv \alpha \wedge \beta \vee \alpha \wedge \gamma$ 可得：

$$(\exists x \theta_{\neq i} \wedge \tau + 1 \equiv_a v) \vee \dots \vee (\exists x \theta_{\neq i} \wedge \tau + a - 1 \equiv_a v)$$

其中 $\theta_{\neq i}$ 等于 $\bigwedge_{1 \leq j \leq k \wedge j \neq i} \theta_j$ 。接着对每个 $1 \leq b \leq a - 1$ ，递归转换子式 $\exists x \theta_{\neq i} \wedge \tau + b \equiv_a v$ 。返回。

3. 否则，若存在 θ_i 为任意含 x 的 $\tau_0 = v_0$ ，将 θ_i 转换为等价的 $ax + \tau = v$ ，并考虑：
 - 若存在 θ_j 有 $j \neq i$ 为任意含 x 的 $\tau'_0 = v'_0$ ，将其转换为等价的 $a'x + \tau' = v'$ 。存在 b 等于 a 和 a' 的最小公倍数， $c = b/a$ ， $c' = b/a'$ ，有：

$$\begin{aligned} ax + \tau = v \wedge a'x + \tau' = v' & \quad \text{等价于} \quad bx + c\tau = cv \wedge bx + c'\tau' = c'v' \\ & \quad \text{等价于} \quad bx + c\tau = cv \wedge cv + c'\tau' = c'v' + c\tau \end{aligned}$$

- 否则，若存在 θ_j 为任意含 x 的 $\tau'_0 \neq v'_0$ ，转换为 $a' + \tau' \neq v'$ 。存在类似的 $b c c'$ 有：

$$ax + \tau = v \wedge a'x + \tau' \neq v' \quad \text{等价于} \quad bx + c\tau = cv \wedge cv + c'\tau' \neq c'v' + c\tau$$

- 否则，若存在 θ_j 为任意含 x 的 $\tau'_0 \equiv_p v'_0$ ，转换为 $a' + \tau' \equiv_p v'$ 。同样存在 $b c c'$ 有：

$$ax + \tau = v \wedge a'x + \tau' \equiv_p v' \quad \text{等价于} \quad bx + c\tau = cv \wedge cv + c'\tau' \equiv_{c'p} c'v' + c\tau$$

- 否则 $\exists x \theta_1 \wedge \dots \wedge \theta_n$ 等价于 $(\exists x ax + \tau = v) \wedge \theta_{\neq i}$ 等价于 $\tau \equiv_a v \wedge \theta_{\neq i}$ 。

4. 否则：

- 将所有 θ_i 为任意含 x 的 $\tau \neq v$ 转换为 $\top$ ，因这样的 θ_i 此时不会改变语句的可证性。
- 若存在 θ_i 为任意含 x 的 $\tau_0 \equiv_p v_0$ ，转换为 $ax + \tau \equiv_p v$ 。并考虑：
 - 若存在 θ_j 有 $j \neq i$ 且为任意含 x 的 $\tau'_0 \equiv_{p'} v'_0$ ，转换为 $a' + \tau' \equiv_{p'} v'$ 。存在 q 等于 p 和 p' 的最小公倍数， $r = q/p$ ， $r' = q/p'$ ，有：

$$ax + \tau \equiv_p v \wedge a' + \tau' \equiv_{p'} v' \quad \text{等价于} \quad rax + r\tau \equiv_q rv \wedge r'a'x + r'\tau \equiv_q r'v'$$

又因对任意自然数 $e e'$ 满足 $e > e'$ ，自然数 m ，项 $\xi \xi' \zeta \zeta'$ ，令 $e'' = e - e'$ ，有：

$$ex + \xi \equiv_m \zeta \wedge e'x + \xi' \equiv_m \zeta' \quad \text{等价于} \quad e''x + \xi + \zeta' \equiv_m \zeta + \xi' \wedge e'x + \xi' \equiv_m \zeta' \quad (i)$$

则重复应用 (i) 可得自然数 b 等于 ra 和 $r'a'$ 的最大公因数，项 $\tau_1 \tau'_1 v_1 v'_1$ 满足：

$$rax + r\tau \equiv_q rv \wedge r'a'x + r'\tau \equiv_q r'v' \quad \text{等价于} \quad bx + \tau_1 \equiv_q v_1 \wedge bx + \tau'_1 \equiv_q v'_1$$

并进一步等价于：

$$bx + \tau_1 \equiv_q v_1 \wedge v_1 + \tau'_1 \equiv_q v'_1 + \tau_1$$

- 否则 $\exists x \theta_1 \wedge \dots \wedge \theta_n$ 等价于 $(\exists x ax + \tau \equiv_p v) \wedge \theta_{\neq i}$ 等价于 $\tau \equiv_s v \wedge \theta_{\neq i}$, 其中 s 为 a 和 p 的最大公因数。

这样, 我们即完成了语句的量词消去。而转换结果为由 $= \neq \equiv \not\equiv$ 构成的子式的析取范式, 因而是可证明或证伪的。注意上述涉及的等价关系和算术性质都是在 P 下可证的。

事实上, 考虑标准解释 $\mathfrak{N}_{st}$ 是 $\mathbf{P}$ 的模型, 令 $\mathfrak{N}_1$ 为只含加法的标准解释, 则 $\mathfrak{N}_1$ 为 P 的模型, 即有 $P = \text{Th}(\mathfrak{N}_1)$, 即只含加法的算术语句在 $\mathfrak{N}_{st}$ 下是否为真是可判定的。类似的, 令 $\mathfrak{N}_2$ 为只含乘法的标准解释, 可以证明其语句 (即只含乘法的算术语句) 的真假 (通过量词消去) 也是可判定的, 则存在完备的理论 $S = \text{Th}(\mathfrak{N}_2)$, 我们称 S 为斯科伦算术¹。

最后, 我们简单介绍一下自验证公理系统²。自验证公理系统是一致的且可以证明其自身的一致性。在具体讨论这样的系统之前, 首先注意到以下事实: 对任意包含 $\mathbf{Q}$ 的算术公理集 A , 可扩展到 $B = A; \alpha$ 有其中语句 α 为 B 对应理论 T_B 的一致性语句 Con_{T_B} 。注意这不同于章节 3.2.7 关于一致性语句讨论中由理论 T 扩展至 T' 的方式, 这里 α 不是 A 对应理论 T_A 的一致性语句。在自然语言下, α 表示:

不存在由公理集 $A; \alpha$ 到语句 $\perp$ 的证明过程

可以看到这里的语句 α 是自指的。按照对角线引理 (引理 3.2.17) 中的方法我们可以构造这样的语句。具体的, 我们可以构造 μ -递归函数 $\text{Prf}_{with}^\perp(y, z)$ 返回 1 当且仅当 z 为由公理集 A 加语句 y 得到 $\perp$ 的证明过程, 则存在式 $\beta_{[y]}$ 等于 $\neg \exists z \text{Prf}_{with}^\perp(y, z)$, 其中 $\text{Prf}_{with}^\perp$ 算术定义 $\text{Prf}_{with}^\perp$ 。这样, 我们可构造语句 σ 有 $A \vdash \sigma \leftrightarrow \beta(\ulcorner \sigma \urcorner)$ 。取 $\alpha = \sigma$ 。

然而, 对于强至包含 $\mathbf{Q}$ 的理论 A , 尽管 $A; \alpha$ 可以证明自己的一致性, 但由哥德尔第二不完备定理 $A; \alpha$ 是不一致的。因此, 本质上, 自验证公理系统可以看作是公理集包含语句 (类似) α 的, 但又足够弱而使第二不完备定理不适用的, 且进一步其的一致性 (在元逻辑下亦) 可证的理论系统。技术上, 自验证公理系统为通过使用减法和除法替代加法和乘法, 由原算术语言下 (任意) 包含 $\mathbf{P}$ 的算术理论 T 构造而成的新理论 $\Upsilon = \text{IS}(T)$ 。一方面, 新算术语言也仍足够 (以新的方式) 算术定义证明所需的计算过程, 因此 Υ 可以包含一致性语句 Con_Υ 。另一方面, 可以证明 IS 映射是保留一致性的, 即若 T 是一致的, 则 Υ 也是一致的。而在新使用减法和除法的公理系统下, 其不能证明如对应 $\forall x, y \exists z z = x \cdot y$ 的语句, 即不能证明乘法为全函数, 因而也足够弱到可以避免哥德尔第二不完备定理带来的限制。

3.3 高阶逻辑

我们已经介绍了零阶逻辑 (命题逻辑) 和一阶逻辑 (谓词逻辑), 下面介绍高阶逻辑, 并以二阶逻辑为代表讨论有关性质。

¹可进一步参考 Alexis Bès. A Survey of Arithmetical Definability. 2002, 其中可以找到判定方法以及公理化 S 的相关材料。

²Dan E. Willard. "Self-Verifying Axiom Systems, the Incompleteness Theorem and Related Reflection Principles". In: The Journal of Symbolic Logic 66.2 (2001), pp. 536–596. ISSN: 00224812. URL: <http://www.jstor.org/stable/2695030>。

3.3.1 语法和语义

直观的说, 相比于一阶逻辑的量词仅能量化域中的单个元素, 如语句 $\forall x P(x)$ 表示对域中任意元素 x 有 $P(x)$; 二阶逻辑允许量词量化谓词和函数, 仅以谓词为例, 如二阶语句 $\forall X X(c)$ 表示对任意一位谓词 X 有 $X(c)$, 即元素 c 满足任意域上的一元关系。考虑域上的一元关系即为域的子集, 则二阶逻辑除可量化域中单个元素外还可量化元素的集合。进一步三阶逻辑可量化元素集合的集合, 以此类推。

具体的, 二阶逻辑首先包含全部一阶逻辑的表达能力, 因此二阶逻辑的语言包含一阶逻辑的语言的全部组成部分, 且这些部分和一阶逻辑下的语义解释相同。除此以外:

- 词汇上二阶逻辑还包括谓词变量符号和函数变量符号, 并改称原有的变量符号为个体变量符号, 统称三种变量符号为变量符号。并以 $X^P Y^P Z^P X_0^P X_1^P X_2^P$ 等表示谓词变量, $X^F Y^F Z^F X_0^F X_1^F X_2^F$ 等表示函数变量。
- 语法构成上:
 - 项将还包括由任意 n 位函数变量 X^F 及任意项 $\tau_{1\dots n}$ 构成的 $X^F(\tau_{1\dots n})$ 。
 - 原子式还将包括由任意 n 为谓词变量 X^P 及任意项 $\tau_{1\dots n}$ 构成的 $X^P(\tau_{1\dots n})$ 。
 - 式还包括由任意式 α , 任意谓词或函数变量 $X \notin b(\alpha)$ (同样仅为避免语句如 $\forall X \forall X X(c)$, 其中回顾并扩展 $b(\alpha)$ 表示二阶 α 中的约束变量) 构成的 $\forall X \alpha$ 。类似扩展 $\exists$ 的定义。
- 语义上, 考虑新增的语句构成方式 $\forall X \alpha$, X 为任意 n 位谓词或函数变量, $\forall X \alpha$ 为真当且仅当用域上任意 n 元关系或函数替换 α 中的 X 后所得语句均为真。

对于三阶逻辑, 类似的, 我们可以进一步通过增加 (常量和变量) 符号对应如谓词的谓词, 函数的谓词等, 则如对函数符号 F , 函数谓词符号 P , 有 $P(F)$ 为三阶式。我们可以构成三阶语句如:

$$\exists X^{PF} \forall Y^F X^{PF}(Y^F) \leftrightarrow \forall x, y Y^F(x) = Y^F(y)$$

这里 X^{PF} 即为函数谓词变量, 该语句意为存在常数函数的集合, 即存在 $X^{PF} = \{ Y^F : \forall x, y Y^F(x) = Y^F(y) \}$ 。就如二阶逻辑包含一阶逻辑, 可以看到三阶逻辑也包含二阶逻辑。并以此方式, 我们可以扩展得到更高级的逻辑。

写 $\wp^0(A) = A$, 对任意 $n > 1$, $\wp^n(A) = \wp(\wp^{n-1}(A))$, 即 $\wp^n(A)$ 表示集合 A 进行 n 次幂集运算后的结果。则可以看到任意 n 阶逻辑量词可量化的范围仍限于 $\bigcup_{1 \leq i \leq n} \wp^i(A)$, 其中集合 A 表示逻辑系统当前解释的域。同时随阶数的增加, 按以上方式扩展得到的常量变量类型数量也将以指数级增加。因此, 我们将在后续章节给出一种更通用的基于类型的高阶逻辑系统——类型论。

同时, 由语义的定义方式, 可以看到这里我们几乎只能以集合论作为元逻辑。且在集合论中又包含全部的 $\wp^n(A)$ 。因此又可认为如二阶逻辑是弱于集合论的。而如下一章所见, 集合论又是一种可以基于一阶逻辑构建的理论。因此, 事实上我们很难给一阶逻辑和二阶逻辑定义严格的强弱关系。尽管如此, 我们仍能给出一些例子阐述一阶逻辑和二阶 (及高阶) 逻辑的一些区别。

3.3.2 二阶皮亚诺算术

二阶皮亚诺算术 $\mathbf{P}_{II}$ 和 (一阶) 皮亚诺算术 $\mathbf{P}$ 的唯一区别在于对归纳公理的给出方式。即 $\mathbf{P}_{II}$ 公理集同样包含罗宾逊算术公理集 $\mathbf{A}_Q$ 中的全部语句; 同时, 相比于在 $\mathbf{P}$ 中归纳公理 (Ind) 需要以公理模式的方式

式给出^{注1}，在 $\mathbf{P}_{\Pi}$ 中，我们仅需语句：

$$\forall Y^P (Y^P(\mathbf{0}) \wedge \forall x (Y^P(x) \rightarrow Y^P(Sx)) \rightarrow \forall x Y^P(x)) \quad (\text{Ind}_{\Pi})$$

可以看到其中 Y^P 是一位谓词变量。即有 $\mathbf{P}_{\Pi}$ 是有限可公理化的。而我们将证明：

引理 3.3.1 有且仅有标准解释 $\mathfrak{N}_{st}$ 为 $\mathbf{P}_{\Pi}$ 的模型。

证明：由符号 $S + \cdot$ 符合 $\mathfrak{N}_{st}$ 下的定义，我们仅需证明 $\mathbf{P}_{\Pi}$ 的任意模型的域 A 等于 $\mathbb{N}$ 。令对任意自然数 n ， n 对应 n ，则 $\mathbb{N} \subseteq A$ 。添加谓词 $N(x)$ 对应 $x \in \mathbb{N}$ ，则由公理 (Ind_{Π}) 有 $N(\mathbf{0}) \wedge \forall x (N(x) \rightarrow N(Sx)) \rightarrow \forall x N(x)$ ，显然 $N(\mathbf{0}) \wedge \forall x (N(x) \rightarrow N(Sx))$ ，则有 $\forall x N(x)$ ，即域中的任意元素均属于 $\mathbb{N}$ 。□

事实上，二阶逻辑下的公理 (Ind_{Π}) 相比一阶逻辑的公理模式 (Ind) ，其表达能力的优势在于一阶逻辑式不能定义全部的关系。如考虑一位谓词，有任意一位谓词对应域 A 中的某个子集，则一位谓词的数量等于 $|\wp(A)|$ ，当 A 无限时， $\wp(A)$ 是不可数的，而显然一阶逻辑式的集合是可数的，即存在一位谓词如 $P(x)$ 有不存在一阶逻辑式 $\rho[x]$ 满足 $\forall x P(x) \leftrightarrow \rho$ 。或更一般的，存在（可被二阶逻辑量化的）谓词不可在一阶逻辑下定义。

这样 $\mathbf{P}_{\Pi}$ 是可公理化、一致且完备的， $\mathfrak{N}_{st}$ 为其唯一模型。结合推论 3.2.18.1：真算术理论是不可判定的，即 $\mathfrak{N}_{st}$ 下为真的一阶逻辑语句不可判定，及二阶逻辑语句包含一阶逻辑语句，我们有不存在可靠且完备的 $\mathbf{P}_{\Pi}$ 的演绎规则。或更一般的：

定理 3.3.2 不存在可靠且完备的高阶逻辑演绎规则。

向 $\mathbf{P}_{\Pi}$ 的公理集中增加公理模式 $\exists x x \neq n$ ，其中 n 为任意自然数，即意为域中存在不为自然数的元素。可以看到得到的新理论是不一致的。令新理论的公理集为 Γ ，即 Γ 是不可满足的，但显然 Γ 的任意有限子语句集都是可满足的，则可得如引理 3.1.6 这样的紧性定理对同样高阶逻辑不适用。

3.3.3 亨金语义

尽管高阶逻辑没有严格意义上可靠且完备的演绎规则，我们仍希望可以直接在高阶逻辑、包括类型论中进行有实际意义的逻辑推导。为此，我们可通过放宽语义要求从而得到新的更宽泛意义下的可靠且完备的演绎规则。下面还是以二阶逻辑为例²，考虑函数常变量符号可由谓词符号替换，这里（在大多数时候）仅考虑含谓词符号的二阶逻辑（并省略上标^P）。

具体的，对二阶逻辑中域为 A 的解释 $\mathfrak{I}$ ，有其中谓词可取值范围为 $\mathcal{P} = \bigcup_{1 \leq n} A^n$ ，即对任意谓词常变量符号 P 对应的解释 $\mathfrak{I}[P] \in \mathcal{P}$ ，任意谓词变量符号可量化范围等于 $\mathcal{P}$ 。若放宽这一要求使谓词可取值范围为由解释决定的任意 $\mathcal{P}$ 的子集，则我们即得到了泛通用语义。称在泛通用语义下的解释 / 模型为泛通用解释 / 模型，写作 $(\mathfrak{I}, \mathcal{G})$ ，其中 $\mathfrak{I}$ 即为原语义下的解释， $\mathcal{G} \subseteq \mathcal{P}$ ，并要求 $\mathfrak{I}$ 中任意谓词常变量 P 对应的解释 $\mathfrak{I}[P] \in \mathcal{G}$ 。当 $\mathcal{G} = \mathcal{P}$ 时，我们称 $(\mathfrak{I}, \mathcal{G})$ 为满解释 / 模型，可以看到满解释 $(\mathfrak{I}, \mathcal{G})$ 即为原解释 $\mathfrak{I}$ 。

则我们有：

引理 3.3.3 存在演绎规则 $\mathbf{F}$ ，对任意二阶逻辑语句集 Γ ，语句 σ ，可以通过 $\mathbf{F}$ 从 Γ 得到 σ 当且仅当 σ 在 Γ 的全部泛通用模型下为真。即 $\Gamma \vdash_{\mathbf{F}} \sigma$ 当且仅当 $\Gamma \vdash^{\mathcal{G}} \sigma$ ，其中 $\vdash^{\mathcal{G}}$ 表示泛通用语义下的 $\vdash$ 关系。

¹见章节 3.2.6。

²同时如后续章节所示，这里的讨论可类似扩展至类型论。

为得到上述引理，我们引入多类一阶逻辑。并证明泛通用语义下的二阶逻辑本质上与多类一阶逻辑等价，而多类一阶逻辑又和一阶逻辑等价。

简单的说，相比于一阶逻辑，多类一阶逻辑中的项是有类型的，称为类。而多类一阶逻辑的解释需要为每个类提供一个不同的域。严格的说，一个具体的多类一阶逻辑语言词汇包括除与一阶逻辑相同的， $() \neg \rightarrow$ ，变量符号，还有¹：

- 类 $\mathcal{T}_1 \mathcal{T}_2 \mathcal{T}_3$ 等。
- 任意 $i \geq 1$ ，类 $\mathcal{T}_i$ 下的全称量词 $\forall_{\mathcal{T}_i}$ 符号。
- 任意 $i \geq 1$ ，类 $\mathcal{T}_i$ 下的常量符号 $a \in \mathcal{T}_i$ 等。
- 任意 $n \geq 1$ ， $i_1 \geq 1$ 到 $i_n \geq 1$ ，表示要求任意第 k 位 ($1 \leq k \leq n$) 参数所属类为 $\mathcal{T}_{i_k}$ 的 n 位谓词符号 $P \in \mathcal{T}_{i_1} \times \dots \times \mathcal{T}_{i_n}$ 等。

其中类的域，常量符号，谓词符号为语言可解释的参数。相应的扩展语法构成和语义定义我们即得到了一个具体的多类一阶逻辑语言，其中 $\forall_{\mathcal{T}_i} x \alpha$ 表示任意类 $\mathcal{T}_i$ 下的变量 x 满足式 α 。

例如，取类 $\mathcal{T}_1 \mathcal{T}_2$ ，谓词 $P \in \mathcal{T}_1 \times \mathcal{T}_2$ ，语句 α 等于 $\forall_{\mathcal{T}_1} x \exists_{\mathcal{T}_2} y P(x, y)$ 。令解释 $\mathfrak{I}$ 中 $\mathcal{T}_1$ 的域 $\mathfrak{I}[\mathcal{T}_1]$ 等于小写英文字母， $\mathcal{T}_2$ 的域 $\mathfrak{I}[\mathcal{T}_2]$ 等于大写英文字母， $\mathfrak{I}[P](a, b)$ 表示 b 是 a 相应的大写字母，则 $\mathfrak{I}[\alpha]$ 表示对任意小写英文字母均存在相应的大写字母。

同时可以看到，一阶逻辑即为一类一阶逻辑。

进一步，我们有：

引理 3.3.4 对任意二阶逻辑语言 $\mathcal{L}_{\Pi}$ ，存在多类一阶逻辑语言 $\mathcal{L}_m$ ，存在函数映射 $F: \mathcal{L}_{\Pi} \mapsto \mathcal{L}_m$ ，有对任意语句 $\alpha_{\Pi} \in \mathcal{L}_{\Pi}$ ， $\models^{\mathcal{G}} \alpha_{\Pi}$ 当且仅当 $\models F(\alpha_{\Pi})$ 。

证明：构造 $\mathcal{L}_m$ 有无限多个类 $\mathcal{T}_0 \mathcal{T}_1 \mathcal{T}_2$ 等。其中 $\mathcal{T}_0$ 对应 $\mathcal{L}_{\Pi}$ 中个体常变量的取值集合，即如 $\mathcal{L}_{\Pi}$ 下的 $\forall x$ 可表示为 $\mathcal{L}_m$ 下的 $\forall_{\mathcal{T}_0} x$ ，并为 $\mathcal{L}_{\Pi}$ 中的全部个体常量符号在 $\mathcal{L}_m$ 中引入 $\mathcal{T}_0$ 下的常量符号；对任意 $n \geq 1$ ， $\mathcal{T}_n$ 对应 $\mathcal{L}_{\Pi}$ 中 n 为谓词常变量的取值集合，即如 $\forall X$ ， X 为 n 位谓词变量，可表示为 $\forall_{\mathcal{T}_n} X$ ，并为 $\mathcal{L}_{\Pi}$ 中的全部 n 为谓词常量引入 $\mathcal{T}_n$ 下的常量符号。同时，为任意 $n \geq 1$ 在 $\mathcal{L}_{\Pi}$ 中构造 $n+1$ 位谓词符号 $E^n \in \mathcal{T}_n \times \overbrace{\mathcal{T}_0 \times \dots \times \mathcal{T}_0}^n$ 使二阶式 $X_{\Pi}(x_{\Pi 1} \dots x_{\Pi n})$ (X_{Π} 为 n 位谓词常变量) 可表示为 $E^n(X_m, x_{m 1} \dots x_{m n})$ (X_m 为 X_{Π} 对应的类 $\mathcal{T}_n$ 下的一阶符号， $x_{m 1} \dots x_{m n}$ 为 $x_{\Pi 1} \dots x_{\Pi n}$ 对应的类 $\mathcal{T}_0$ 下的一阶符号序列)。则可以看到可以构造 $F: \mathcal{L}_{\Pi} \mapsto \mathcal{L}_m$ 满足上述的对应关系。且对任意二阶语句 α_{Π} ，令多类一阶语句 $\alpha_m = F(\alpha_{\Pi})$ ，有对任意 α_{Π} 的泛通用模型 $\mathfrak{M}_{\Pi}$ ，存在 α_m 的模型 $\mathfrak{M}_m$ ，以及对任意 α_m 的模型 $\mathfrak{M}_m$ 存在 α_{Π} 的泛通用模型 $\mathfrak{M}_{\Pi}$ 。□

类似的，我们还有：

引理 3.3.5 对任意多类一阶逻辑语言 $\mathcal{L}_m$ ，存在一阶逻辑语言 $\mathcal{L}$ ，存在映射函数 $F: \mathcal{L}_m \mapsto \mathcal{L}$ ，有对任意语句 $\alpha_m \in \mathcal{L}_m$ ， $\models \alpha_m$ 当且仅当 $\models F(\alpha_m)$ 。

证明：构造 $\mathcal{L}$ ，为 $\mathcal{L}_m$ 中任意类 $\mathcal{T}_i$ 增加一位谓词 T_i 有 $x_m \in \mathcal{T}_i$ 对应 $T_i(x)$ (x_m 对应 x)，为 $\mathcal{L}_m$ 中任意类 $\mathcal{T}_i$ 下的常量符号 c_m 增加常量符号 c 及前提 $T_i(c)$ 。改任意多类一阶式 $\forall_{\mathcal{T}_i} x \alpha$ 为 $\forall x T_i(x) \wedge \alpha$ 。则类似可构造满足条件的 F 。□

这样由一阶逻辑存在可靠且完备的演绎规则可得引理 3.3.3。事实上，我们可以通过下面的公理模式在一定程度上限制泛通用语义，从而去掉一些从二阶逻辑语义角度考虑明显不合理的模型。比如考虑一

¹ 不考虑可被谓词符号替代的函数符号。

谓词常量 P ，则显然有 $\exists X \forall y X(y) \leftrightarrow \neg P(x)$ ，即存在 P 对应集合的补集，但存在泛通用模型有 P 不存在补集，如存在模型其类 $\mathcal{T}_1$ 的域中仅有一个元素（ P 对应该元素）：

- 选择公理模式。对任意 n ：

$$\forall Y \forall x_{1 \dots n+1} Y(x_{1 \dots n+1}) \rightarrow \exists X^F \forall x_{1 \dots n} Y(x_{1 \dots n}, X^F(x_{1 \dots n}))$$

- 概括公理模式。对任意二阶式 $\rho[u_{1 \dots n}]$ ，其中 u_i 可以为（不等于 X 的）谓词变量：

$$\forall u_{1 \dots n} \exists X (\rho \leftrightarrow X(u_{1 \dots n}))$$

本质上，选择公理模式表示我们可以从任意集合（如 $\wp(\mathbb{N})$ 的任意子集）中选出一个元素，如这里我们可以通过函数 X^F 从由 Y 及 $x_{1 \dots n}$ 确定的集合中选出元素 $X^F(x_{1 \dots n})$ 。我们将在章节 4.3.9 给出更多关于选择公理的讨论。而概括公理模式意为总是存在可被定义的集合，即这里存在可被 ρ 定义的集合 Y 。显然，通过概括公理模式，我们可以避免上述集合不存在补集的情况。同时，对二阶算术理论，概括公理模式还使其模型包含了全部一阶算术可定义的集合，即进一步意味着包含了全部可枚举，可计算集合。

我们称解释需满足上述两项公理模式的语义为通用语义或亨金语义，称其解释 / 模型为通用解释 / 模型，或亨金解释 / 模型。显然，同样由亨金语义和一阶逻辑的关系，并推广至高阶逻辑，我们有：

定理 3.3.6 亨金语义下，存在可靠且完备的高阶逻辑演绎规则。

类似的，亨金语义下有高阶逻辑对应的紧性定理、哥德尔不完备定理等。

3.4 本章小节

- 一阶逻辑又称谓词逻辑。其逻辑符号除逻辑联结词外，还包括变量及量词 $\forall$ ，可由 $\forall$ 定义的 $\exists$ 。其非逻辑符号包括常量，各个位的函数和谓词。其原子式由对应名词成分的由常变量、函数构成的项和谓词构成。其式由子式经逻辑联结词联结或经量词约束构成。¹
- 一阶逻辑可解释的参数除非逻辑符号外，还包括对应域的 $\forall$ 。域表示解释中所有名词对象的可取集合 A 。则常量、约束变量以及自由变量的潜在可取值在解释中对应 A 中的元素。 n 位函数（ $n \geq 1$ ）对应 A^n 到 A 的函数。 n 位谓词（ $n \geq 1$ ）对应 A 上的 n 元关系。原子语句在解释中为真当且仅当其项序列对应的元素组属于谓词对应的关系。
- 一阶逻辑可靠且完备的演绎规则主要可分为根岑形式和希尔伯特形式演绎系统。其中根岑形式又可分为自然演绎和相继式演绎。这些演绎规则均需要使用由推导规则决定的可任意大的推导树完成演绎证明。其中自然演绎最接近日常（半）自然语言下的证明方式。相继式演绎相对更适用于证明过程的性质发掘，如切消定理可极大缩小推导树的搜索范围。相比于根岑形式，希尔伯特演绎使用更多的逻辑公理和更少的（可以少至一条的）推导规则。
- 可以通过构造模型和应用紧性定理证明如相继式演绎的完备性。这种完备性的存在即哥德尔完备性定理。紧性指我们可以通过研究更小（如有限）的场景得到更大（如无限）场景下的性质。而结合构造的模型的域是可数的，有勒文海姆-斯科伦定理，即任意可满足的语句集可被域可数的解释满足。

¹注意我们仍限制讨论于可枚举语言。

- 直觉主义逻辑形式上可看作缺少某些推导规则 / 逻辑公理的经典逻辑。即在非逻辑公理相同的情况下，直觉主义逻辑下的理论包含于经典逻辑下理论。
- 解释为等于关系的符号 $=$ 应用十分常见，因此也可视其为一阶逻辑语言的逻辑符号（而非谓词符号）。而其相关公理集可由推导规则替代（并保持切消性质）。
- 一阶算术语言为含 $=$ 的、有常量符号 0 函数符号 S （后继） $+$ $\cdot$ 的一阶逻辑语言。称 n 元 μ -递归函数 F 算术可定义若存在一阶算术式 $\rho[y, x_1 \dots x_n]$ 对任意 $x_1 \dots x_n$ 和 y 有语句 $\rho[y/y][x_1/x_1] \dots [x_n/x_n]$ 在标准解释 $\mathfrak{N}_{st}$ 下为真当且仅当 $y = F(x_1 \dots x_n)$ 。有任意 μ -递归函数算术可定义，特别的，其可被 $\exists$ 基本式定义。扩展定义有任意半可判定集合算术可定义。进一步扩展称集合 $A \subseteq \mathbb{N}$ 算术可定义若存在式 $\rho[x]$ 对任意 x 有 $\rho[x/x]$ 在 $\mathfrak{N}_{st}$ 下为真当且仅当 $x \in A$ 。
- 通过允许交替增加不同的非有界量词，可扩展 $\exists$ 基本式得到算术式的一种层级结构。由前束范式有对任意算术式存在等价式位于该结构中。称由该结构对应的算术定义的自然数子集合的层级结构为算术阶层。
- 称任意自然数子集合 A 可映射规约到 B 若存在全可计算函数 f 满足任意 $x \in A$ 当且仅当 $f(x) \in B$ 。则有算术阶层中层级 Δ_n ($n \geq 1$) 可映射规约到 $\emptyset^{(n-1)}$ ，为 $\emptyset^{(n-1)}$ 递归的；层级 Σ_n 可映射规约到 $\emptyset^{(n)}$ ，为 $\emptyset^{(n-1)}$ 递归可枚举的；层级 Π_n 可映射规约到 $\overline{\emptyset^{(n)}}$ ，为 $\text{co-}\emptyset^{(n-1)}$ 递归可枚举的。
- 称公理集 A 弱于 B 若 A 对应理论为 B 对应理论的真子集。称更弱公理集对应的理论更弱，得到的定理更强。
- 罗宾逊算术 $\mathbf{Q}$ 是一种相对较弱的有限可公理化的算术系统。可证明任意 $\exists$ 基本语句在 $\mathbf{Q}$ 中可证当且仅当该语句在 $\mathfrak{N}_{st}$ 下为真。结合 $\exists$ 基本式和半可判定集合的关系，有邱奇定理指出一阶逻辑的恒真语句集不可判定。
- 哥德尔第一不完备定理指出任意包含 $\mathbf{Q}$ 的可公理化且一致的一阶逻辑理论是不完备的，即存在语句在这样的系统中不可证明不可证伪。
- 皮亚诺算术 $\mathbf{P}$ 在 $\mathbf{Q}$ 的基础上增加了归纳公理。哥德尔第二不完备定理指出任意包含 $\mathbf{P}$ 的可公理化且一致的一阶逻辑理论不能证明其自身的一致性。通过希尔伯特-博内斯泛化我们可以得到更强的第二不完备定理。
- 普雷斯伯格算术和斯科伦算术是可公理化一致且完备的，但前者只含加法，后者只含乘法。自验证公理系统是可公理化且一致的，并可证明自身的一致性，但其不能证明如乘法为全函数。
- 二阶逻辑在一阶逻辑的基础上量词还可量化谓词和函数。三阶逻辑进一步可量化谓词的谓词、函数的谓词等。更高阶的逻辑以此类推。
- 二阶皮亚诺算术是可公理化且一致且完备的，标准模型为其唯一模型。则有二阶（及以上）逻辑没有可靠且完备的演绎规则。放宽语义要求，有高阶逻辑在亨金语义下本质上为多类一阶逻辑，进一步等价于一阶逻辑。即亨金语义下存在可靠且完备的演绎规则，但亦有对应的哥德尔不完备定理。

第四章 集合论组件

在第三章中，通过多种算术理论系统，我们介绍了不同强弱逻辑系统及它们的应用的具体例子。本章节我们讨论在数学和很多其它领域更一般也更常用的集合论的基本组成。在章节 4.1 中我们给出最常见从朴素集合论（因罗素悖论）发展而来的 ZFC 集合论的公理集，并在此基础上定义最基础的集合概念（如有序对、函数等）。在章节 4.2 中进一步构造各种基础数学对象（如自然数、实数等），并展开一定讨论，以示 ZFC 集合论（特别其中的 Z 部分）如何支持数学对象的建立、或可以作为数学的基础。最后在章节 4.3 中我们进一步泛化 4.2 中定义的对象，如扩展自然数和第一个无限从而得到序数和基数等，并开始着重讨论 ZFC（相较于 Z）余下部分公理在其中的作用。

4.1 公理集合论

区别于章节 2.1.1 中以（半）自然语言给出的集合论，称形式化、公理化的集合论为公理集合论。本节主要介绍公理集合论的基本概念、分类和部分历史，并从中看到公理集合论、特别是 ZFC 集合论出现的主要原因。

具体的，我们从最初的也是最基本的集合论、朴素集合论出发，将看到（公理）集合论成功的一大原因在于一方面其系统（基础）的简洁性，包括语言和公理集的简洁性；另一方面又能初步领略其表达能力的强大。同时也将看到朴素集合论过于简洁而缺乏约束，以至于在其上可以构造出如罗素悖论从而导致其自身的失败。而为解决罗素悖论，在朴素集合论的基础上，ZFC 集合论和类型论被相继发展起来。其中 ZFC 集合论将主要在本章节中讨论，针对类型论的详细讨论见后续章节。

一般情况下，集合论建立在一阶逻辑上。如章节 3.3 中关于一阶逻辑和高阶逻辑的关系讨论，以及本章中后续内容所示，可以相信一阶逻辑集合论并不相相比于高阶逻辑存在显见的劣势。当然，我们也可以如二阶逻辑的基础上建立集合论，但如相对于一阶逻辑可以非常自然的作为一阶逻辑的元逻辑¹，一个重要但不易处理的问题是应该使用什么作为二阶逻辑集合论的元逻辑²。

最后，本章节集合论中很大一部分内容是对我们已经应用至今的集合论概念的形式化，可对比结合章节 2.1 中的内容。

¹见第五章。

²可进一步参考 Jouko Väänänen. “Second-order and Higher-order Logic”. In: The Stanford Encyclopedia of Philosophy. Ed. by Edward N. Zalta. Fall 2021. Metaphysics Research Lab, Stanford University, 2021.

4.1.1 朴素集合论

公理集合论、包括（公理化后的）朴素集合论语言 $\mathcal{L}_S$ 除仍视为逻辑符号的 $=$ 符号以外¹ 仅包含二位谓词符号 $\in$ （并同样写 $a \in b$ 表示 $\in(a, b)$ ）。因而本质上，其它概念所使用的如集合表示法、多元组、函数等符号（序列）是在仅涉及 $\in$ 的一阶逻辑式基础上的定义，类似写 $a \notin b$ 表示 $\neg a \in b$ 。

对于朴素集合论的公理集，尽管康托尔在最初提出时并未给出严格形式化、公理化的定义。我们仍可以在其中识别出以下公理集 A_N ：

- 外延公理：

$$\forall a, b ((\forall x x \in a \leftrightarrow x \in b) \rightarrow a = b)$$

- 概括公理模式，对任意 n 、 $\rho[u_1, \dots, n, x]$ 有 u_i 不等于 b ²：

$$\forall u_1 \dots u_n \exists b \forall x (x \in b \leftrightarrow \rho)$$

这里的外延，相对于内涵，意指符号的内容而非形式。例如集合 $\{0, 1, 2\}$ 和 $\{x \in \mathbb{N} : x \leq 2\}$ 的内涵（即符号序列）是不同的，但其外延（即其所指的集合）是相同的。同时，注意外延公理的反方向 $\forall a, b (a = b \rightarrow \forall x x \in a \leftrightarrow x \in b)$ 可由逻辑符号 $=$ 的要求、即其公理 / 推导规则直接得到。进一步，由外延公理可以看到（这里介绍的）朴素集合论认为域中的元素均为集合，即如这里的 $\forall a \dots x \in a \dots$ 指出任意域中的元素支持原子式 $x \in a$ 。这一点对 ZFC 集合论同样适用。

而对于概括公理，直观的说，其意为任意（式定义的）一组物体可被视为一个集合。也可以看到这本质上涵盖了章节 3.3.3 中二阶逻辑的概括公理，而也将看到这里超越二阶逻辑概括公理的任意的部分最终导致了朴素集合论的失败。不过在此之前，首先让我们讨论其成功的地方。

例如，我们可以通过概括公理证明空集的存在，其中约束变量 b 对应空集，取 ρ 等于 $\perp$ ：

$$A_N \vdash \exists b \forall x (x \in b \leftrightarrow \perp) \quad (\text{空集})$$

因可见进一步有：

$$A_N \vdash \exists b \forall x x \notin b$$

同时，可以通过外延公理证明空集的唯一性，即：

$$A_N \vdash \forall a, b ((\forall x x \notin a) \wedge (\forall x x \notin b) \rightarrow a = b)$$

其证明过程中包含，具体的，为 a, b 引入额外常量 $\mathbf{a}, \mathbf{b}$ ，令 Γ 为语句集合 $\{(\forall x x \notin \mathbf{a}) \wedge (\forall x x \notin \mathbf{b})\}$ ：

$$\frac{\frac{\Gamma \Rightarrow \forall x x \notin \mathbf{a}}{\Gamma \Rightarrow x \notin \mathbf{a}}}{\frac{\Gamma \Rightarrow x \notin \mathbf{a} \quad \frac{\frac{\Gamma \Rightarrow \forall x x \notin \mathbf{b}}{\Gamma \Rightarrow x \notin \mathbf{b}}}{\Gamma \Rightarrow x \notin \mathbf{b}}}{\frac{\Gamma \Rightarrow x \notin \mathbf{a} \quad \Gamma \Rightarrow x \notin \mathbf{b}}{\Gamma \Rightarrow x \notin \mathbf{a} \wedge x \notin \mathbf{b}}}$$

$$\frac{\Gamma \Rightarrow x \notin \mathbf{a} \wedge x \notin \mathbf{b}}{\Gamma \Rightarrow x \in \mathbf{a} \leftrightarrow x \in \mathbf{b}}$$

$$\frac{\Gamma \Rightarrow x \in \mathbf{a} \leftrightarrow x \in \mathbf{b}}{\Gamma \Rightarrow \forall x (x \in \mathbf{a} \leftrightarrow x \in \mathbf{b})}$$

注意在不引起歧义的情况下，我们省略了相继式演绎表达法中集合的括号。仍可以看到尽管已略去了大量基本步骤，由形式化的演绎规则推导树表示的演绎规则仍显得十分笨重，而从另一方面讲，这种冗长

¹ 即默认包含相关的逻辑公理和 / 或推导规则。

² x 亦不应等于 b ，但其可由我们不允许语句如 $\forall x \forall x P(x)$ 蕴含，下同。

表述也部分源于其更为机械而精确，这也是其发展的主要目标。同时，这种笨重性也使得我们在讨论很多较复杂的实际问题时不会以这种方式给出证明。这里主要需要明确的是，在上下文假定的逻辑系统（如这里的朴素集合论及接下来的 ZFC 集合论）背景下，我们认为所有以（半）自然语言给出的证明应该也必须可以通过该系统下的对应形式化推导树给出。

回到对空集的讨论，在保证存在且唯一的前提下，我们即可以定义符号 $\emptyset$ 对应域中空集的唯一存在。类似的，通过概括公理我们有对任意两个集合 a, b ，存在并集 c ：

$$A_N \vdash \forall a, b \exists c \forall x (x \in c \leftrightarrow x \in a \vee x \in b) \quad (\text{并集})$$

还有，对任意 a, b ，存在交集 c ：

$$A_N \vdash \forall a, b \exists c \forall x (x \in c \leftrightarrow x \in a \wedge x \in b) \quad (\text{交集})$$

对任意 a ，存在幂集 b ：

$$A_N \vdash \forall a \exists b \forall x (x \in b \leftrightarrow \forall y (y \in x \rightarrow y \in a)) \quad (\text{幂集})$$

可以看到 $\forall y (y \in x \rightarrow y \in a)$ 意为 $x \subseteq a$ 。等等。且同样类似的，通过外延公理，我们可以证明集合的并集、交集和幂集等的唯一性，因而可定义相关的 $\cup, \cap$ 和 $\wp$ 等集合函数运算¹。从某种角度说，因存在唯一而可定义带来的形式上的便利性反映了存在唯一性的内在优势，如函数相比于关系的内在优势。由函数关系建立的元素序列是唯一的，如对（全）函数 f, g ，对任意 x ，有唯一 $y = f(x)$ ，唯一 $z = g(y) = g(f(x))$ ，这对任意包括无限长的序列都是有效的。但由关系可建立的元素序列数量是可能呈指数指数增加的，当序列长度为 $\aleph_0$ 时，其序列数量可能是 $2^{\aleph_0}$ 、即走向了不可数无限。

需要进一步指出的是，对这种额外符号定义的采用或弃用（展开）往往意味着对逻辑式的整体而非局部改写。如对式 $P(\emptyset)$ ，其展开后可以有 $\forall b \forall x (x \in b \leftrightarrow \perp) \rightarrow P(b)$ ²。

最后，在讨论朴素集合论的问题之前，还应指出因由朴素集合论可定义幂集等概念，我们事实上可以定义集合基数并完成其大小关系的性质论证³。

4.1.2 罗素悖论

通俗版的罗素悖论即理发师悖论，见章节 1.1.1。

在形式化后，朴素集合论中，罗素悖论指出我们有：

$$A_N \vdash \exists b \forall x (x \in b \leftrightarrow x \notin x)$$

即指存在集合 $R = \{x : x \notin x\}$ 。进而有 $A_N \vdash \perp$ ，即：

定理 4.1.1 朴素集合论不一致。

证明：额外引入常量 r ：

$$\begin{aligned} & \overline{\overline{\exists b \forall x (x \in b \leftrightarrow x \notin x) \Rightarrow \forall x (x \in r \leftrightarrow x \notin x)}} \\ & \overline{\overline{\exists b \forall x (x \in b \leftrightarrow x \notin x) \Rightarrow r \in r \leftrightarrow r \notin r}} \\ & \overline{\overline{\exists b \forall x (x \in b \leftrightarrow x \notin x) \Rightarrow \emptyset}} \end{aligned}$$

¹事实上，在章节 3.2.8 中，我们已经碰到过类似的函数符号定义，其被用以简化算术逻辑式。

²对该定义方式的说明见章节 4.1.3。同时，从整体改写的角度说，符号如 $\emptyset$ 即不能对应到上下文无关文法中的非终结符（因其展开依赖于上下文）。

³即章节 2.1.3。

而显然 $\exists b \forall x (x \in b \leftrightarrow x \notin x)$ 即为概括公理。有 $A_N \vdash \perp$ 。 $\square$

对比理发师悖论，可以看到这里域中的元素即对应不同的人， $x \notin x$ 表示 x 不给自己理发， b 即对应理发师，而 $b = \{x : x \notin x\}$ 要求 b 为所有不给自己理发的人理发。

尽管朴素集合论是简练直观且易用的，罗素悖论所指出的不一致使其显然不再能（直接）作为如数学的基础，因在这样的系统中所有语句都是可证的，这样的系统没有模型。

这里朴素集合论失败的本质原因是概括公理允许了本不应该被允许的集合存在，如罗素悖论中指出的 R 。这也意味着并不是任意（式定义）的一组物体都应可被视为集合。但注意如章节 4.3.1 将述，有时我们仍需要描述不能被视为集合的合集概念，我们将称这样的合集为类。

为解决罗素悖论，一种方法是注意到二阶逻辑中的概括公理并不会造成罗素悖论，其原因是对任意个体或谓词符号 x ，表达式 $x(x)$ 都不是合法的。或者说二阶逻辑只允许如 $X(x)$ 有其中 X 为谓词符号， x 为个体符号，意为 $x \in X$ 。这种形式上的要求可扩展至任意阶逻辑，即罗素悖论不适用于任意（此类）高阶逻辑。而这种避免悖论的方法本质上即为罗素最早提出类型论时的核心思想，即合法的属于关系 $x \in X$ 要求 x 满足一定的类型条件¹。

另一种解决罗素悖论的方法是限制概括公理，更具体的，限制集合的可构成方式。技术上，概括公理被细分为多条不同的集合构成公理，ZFC 集合论即为其中较为成功的一种方式，下面我们对其进行进一步介绍。

4.1.3 ZFC 集合论

整体来看，ZFC 集合论公理集 A_{ZFC} 包含以下公理²：

- 外延公理：
$$\forall a, b ((\forall x (x \in a \leftrightarrow x \in b) \rightarrow a = b) \quad (Ext)$$
- 空集公理：
$$\exists b \forall x (x \notin b) \quad (Empty)$$
- 配对公理：
$$\forall x, y \exists b \forall z (z \in b \leftrightarrow z = x \vee z = y) \quad (Pairing)$$
- 并集公理：
$$\forall a \exists b \forall x (x \in b \leftrightarrow \exists y (y \in a \wedge x \in y)) \quad (Union)$$

定义任意 $a \subseteq b$ 为 $\forall x (x \in a \rightarrow x \in b)$ 。

- 幂集公理：
$$\forall a \exists b \forall x (x \in b \leftrightarrow x \subseteq a) \quad (Power)$$
- 分类公理模式，对任意 n 、 $\rho_{[u_1 \dots n, x]}$ 有 u_i 不等于 b ^{注3}：
$$\forall u_1 \dots n, a \exists b \forall x (x \in b \leftrightarrow x \in a \wedge \rho) \quad (Sep(\rho))$$

¹详见后续章节。

²H.B. Enderton. Elements of Set Theory. Elsevier Science, 1977.

³如朴素集合论概括公理注释，语句构成亦蕴含了 x 及替换公理中的 y 不等于 b 。下替换公理模式同。

定义任意 $P(\emptyset)$ 为 $\forall b (\forall x x \notin b \rightarrow P(b))$ 。

- 无限公理，其中符号 $+$ 涵盖了后继操作，其定义见章节 4.1.7:

$$\exists b \emptyset \in b \wedge \forall x (x \in b \rightarrow x^+ \in b) \quad (Inf)$$

注意 $P(\emptyset)$ 的定义方式，上式展开后为 $\forall e (\forall x x \notin e \rightarrow \exists b e \in b \wedge \forall x (x \in b \rightarrow x^+ \in b))$ 。则通过额外引入常量 e 有:

$$\frac{\frac{Empty \Rightarrow \forall x x \notin e}{\quad} \quad \frac{Inf \Rightarrow \forall x x \in e \rightarrow \exists b e \in b \wedge \forall x (x \in b \rightarrow x^+ \in b)}{\quad}}{Empty, Inf \Rightarrow \exists e \forall x x \notin e \wedge \exists b e \in b \wedge \forall x (x \in b \rightarrow x^+ \in b)}$$

即存在空集 e 及包含 e 的集合 b 。事实上，在可证明存在唯一的情况下，选用全称和存在量词构成的定义是等价的，但选用全称所表达的语句证明和存在性是解耦的，如我们可仅通过外延公理证明通过全称量词定义的 $\emptyset = \emptyset$ 而不需要证明 $\emptyset$ 存在，而可见几乎全部其它公理引入是为了断言某类集合存在。因此这里选择采用全称量词的定义方式。

- 选择公理，其中谓词符号 $isRelation$ 、 $isFunction$ ，函数符号 dom 的定义见章节 4.1.6 ^{注1}:

$$\forall a (isRelation(a) \rightarrow \exists b isFunction(b) \wedge b \subseteq a \wedge dom(b) = dom(a)) \quad (Choice)$$

- 替换公理模式，对任意 n 、 $\rho[u_1 \dots n, x, y]$ 有 u_i 不等于 b :

$$\forall u_1 \dots n, a \left(\overbrace{\forall x (x \in a \rightarrow \forall y_1, y_2 (\rho[y/y_1] \wedge \rho[y/y_2] \rightarrow y_1 = y_2))}^{\beta} \right) \rightarrow \exists b \forall y (y \in b \leftrightarrow \exists x x \in a \wedge \rho) \quad (Rep(\rho))$$

注意其中 β 即为我们一直使用的一阶逻辑下函数符号的形式化定义，或者说可定义函数符号的展开结果。从这个角度说，这里的 ρ （及 $u_1 \dots n$ ）即决定了函数， a 包含其对应定义域， b 对应值域。而进一步注意这里采用了公理模式，而非 $i: \forall f isFunction(f) \rightarrow \exists b \forall y (y \in b \leftrightarrow \exists x \langle x, y \rangle \in f)$ ^{注2}。这是因为就如前提及存在不是集合的合集，同样存在不是集合下函数的但满足 β 的有序对合集 ³。而这样的合集的存在是我们需要替换公理模式的主要原因，而 i （即任意函数值域的存在）是由其它公理及相关定义直接可得 ⁴。

- 正则公理，其中 $\cap$ 的定义见下一小节:

$$\forall a a \neq \emptyset \rightarrow \exists x x \in a \wedge x \cap a = \emptyset \quad (Regularity)$$

观察 A_{ZFC} 中的公理，可以看到除外延公理和正则公理外，其它公理都致力于主张某类型集合（即其中的 b ）的存在、即为对概括公理的细分。其中由空集公理构造的集合可被视为最基础集合，即其构成不以正向的方式依赖其它集合。也可以看到分类公理模式和概括公理最为相似，其唯一区别在于其构造的 b 中元素除要求满足 $\rho[u_1 \dots n]$ 外，还要求其属于给定集合 a ，因此，其也被称为受限概括公理。也可见分类公理模式可构造的 b 均为所给定 a 的子集，因此也称其为子集公理模式，事实上，其是最直接的子集构造方法。

而通过 ZFC 集合论的命名由来，可以笼统的看到其发展的脉络。具体的，命名中的 Z 指恩斯特·策

¹很多材料中使用 AC 表示选择公理。

² $\langle x, y \rangle \in f$ 即意为 $y = f(x)$ ， $\langle \dots \rangle$ 的严格定义见章节 4.1.5。

³见章节 4.3.1。

⁴见章节 4.1.6。

梅洛。策梅洛于 1908 年最初提出的用以解决罗素悖论的 Z 集合论公理集 A_Z 等于¹ 以上除选择公理、替换公理模式和正则公理外的语句集合。即：

$$A_Z = \{ a \in A_{ZFC} : a \neq \text{Choice} \wedge a \neq \text{Rep} \wedge a \neq \text{Regularity} \}$$

命名中的 F 指亚伯拉罕·弗兰克尔。弗兰克尔和斯科伦于 1922 年分别各自提出了替换公理模式。 A_Z 加上替换公理模式，以及冯诺依曼在 1925 年提出的正则公理构成了 ZF 集合论^{2 3}，即其公理集 A_Z 有：

$$A_{ZF} = \{ a \in A_{ZFC} : a \neq \text{Choice} \}$$

命名中的 C 即指选择公理，事实上，选择公理亦是由策梅洛甚至在更早的 1904 年提出的⁴，但其被单独归类的历史根源在于认识论上⁵，当然这也最终足以影响其的适用性⁶。

本质上，公理集中公理的选择除有认识论意义上的区别，并有一致性要求⁷外，其被采纳的关键仍在于实用性，包括公理本身是否简练而又符合直觉、是否足够符合直觉的语句通过这些公理可证等等。朴素集合论的失败在于其不一致，Z 集合论及其衍生品的成功在于其在避免不一致的基础上，尽可能的保留了朴素集合论的简洁和直观特点，Z 基础上增加的其它公理也主要为解决部分符合直觉语句不可证的问题，同时附带进一步简化证明的过程。

最后，同样可以看到 Z、ZF、ZFC 集合论亦认为域中元素均为集合。

在本节的剩余部分，我们将看到 ZFC（或仅其中的 Z 部分）支持如集合运算等常见的相关基础概念。更多数学对象的定义见章节 4.2。通过这样的讨论，我们也将对 A_{ZFC} 中的公理有更深入的理解。

4.1.4 集合的基本运算

Z、ZF、ZFC 的细分 / 层级关系同样可以从理论 / 定理强弱的角度理解，即如三者之中，Z 集合论中的定理是最强的、或最易被接受的。因此，我们从最易被普遍接受的 Z 的公理集 A_Z 出发，并在需要的时候再进一步扩展添加层级结构中的其它公理。

上一小节中，我们已经看到空集和子集的可行定义：

- 空集。定义任意 $P(\emptyset)$ 为 $\forall b (\forall x (x \notin b \rightarrow P(b)))$ 。
- 子集。定义任意 $a \subseteq b$ 为 $\forall x (x \in a \rightarrow x \in b)$ 。

结合外延公理得到的唯一性，可以视 $\emptyset$ 为 A_Z 可证存在的第一个集合。接着我们证明第二个集合的存在，取第二个集合为 $b = \{\emptyset\}$ 。即：

¹更精确的，应为等价（即相互可证）而非完全等于，参考 Wikipedia contributors. Zermelo set theory — Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/w/index.php?title=Zermelo_set_theory&oldid=1075057155. 2022。但这里及下文不作此细节的区分。

²Wikipedia contributors. Zermelo–Fraenkel set theory — Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/w/index.php?title=Zermelo%E2%80%93Fraenkel_set_theory&oldid=1065328933. 2022。

³Wikipedia contributors. Axiom of regularity — Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/w/index.php?title=Axiom_of_regularity&oldid=1066585463. 2022。

⁴Wikipedia contributors. Axiom of choice — Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/w/index.php?title=Axiom_of_choice&oldid=1070052797. 2022。

⁵见章节 4.3.9。

⁶见定理 4.3.52。

⁷尽管哥德尔第二不完备定理告诉我们总是只能假设最终的一致性（见章节 3.2.7），但显然任意具有现实意义的公理集 A 不应有 $A \vdash \perp$ 。而对任意视为根逻辑系统的一致性的信心来自至今无法找到演绎过程在该系统下证明 $\perp$ 。但如停机问题所示，我们并不总能确定永远不会找到这样的演绎过程。

引理 4.1.2 $A_Z \vdash \exists b \forall x (x \in b \leftrightarrow x = \emptyset)$

证明：

$$\begin{array}{c} \text{Pairing} \Rightarrow \exists b \forall z (z \in b \leftrightarrow z = \emptyset \vee z = \emptyset) \\ \text{Pairing} \Rightarrow \exists b \forall x (x \in b \leftrightarrow x = \emptyset) \end{array}$$

□

类似的，可以同样由配对公理证明存在集合 $\{\emptyset, \{\emptyset\}\}$ 、 $\{\{\emptyset\}\}$ 等由任意至多两个指定元素构成的集合。并也可以通过外延公理证明这些集合的唯一性（下不赘述类似唯一性的可证明性）。可以看到我们已经可以构造无限多的集合了。而同样我们可以定义：

- 如任意 $P(\{\emptyset\})$ 为 $\forall b (\forall x (x \in b \leftrightarrow x = \emptyset) \rightarrow P(b))$ 等。

为证明含三个及以上元素集合的存在唯一性，首先需引入任意两集合并集的存在性，事实上，并集公理证明了对任意集合 A ，集合 $B = \bigcup A$ 的存在。则对任意 n, m ，任意集合 $X = \{x_{1\dots n}\}$ 、 $Y = \{y_{1\dots m}\}$ ，存在唯一的集合 $A = \{X, Y\}$ ，有存在唯一的 $B = \bigcup A = \{x_{1\dots n}, y_{1\dots m}\}$ 。这样我们即证明了由任意指定元素构成的有限集合的唯一存在，即这样的集合在 Z 下是可定义的。

考虑集合的另一种表达法 i ： $\{ _x, y, \dots _ : _ _x, y, \dots _ \}$ ，已经看到并不是所有这种形式的表达式都可以视为集合、即如罗素悖论中的 $R = \{x : x \notin x\}$ 不应被视为集合。但比如对任意集合 A ，可定义的谓词 $P(x)$ ，可以证明存在唯一的 $B = \{x \in A : P(x)\}$ ，其存在性由分类公理模式下的 $Sep(P)$ 直接可得。本质上，我们要求表达法 i 所示集合 B 中的所有元素首先属于某个已知存在的集合 A ，即要求 $B \subseteq A$ 。而对于罗素悖论，我们并不能看到 R 中 x 应首先属于的已知存在的集合，即看不到 R 对应的 A_R 。一个符合直觉的 A_R 应意为（域中）全部集合的集合，而在有分类公理的情况下，若可证明存在集合为全部集合的合集，则存在 R ，所在理论不一致¹。而为能够讨论这类不能是集合的合集，一种方法是如前述进一步扩展集合论以引入类的概念²。现首先回到表达法 i 本身，考虑我们需要为其所构造的 B 中的元素提供已属的集合 A 以及需进一步满足的谓词 P ，一般以：左侧 $_x, y, \dots _$ 对应属于 A ，右侧 $_ _x, y, \dots _$ 对应满足 P 。

这样我们可以定义：

- 任意两集合的交集 $a \cap b$ 为 $\{x \in a \cup b : x \in a \wedge x \in b\}$ 。
- 任意 $\bigcap a$ 为 $\{x \in \bigcup a : \forall b b \in a \rightarrow x \in b\}$ 。
- 任意 $a - b$ （仍）为 $\{x \in a : x \notin b\}$ 。

等等。

4.1.5 有序对

回顾定义：

- 任意有序对 $\langle x, y \rangle$ 为 $\{\{x\}, \{x, y\}\}$ 。

首先需要指出如大多数情况，满足有序对要求的定义方式不止一种。也如大多数情况，这里采用较为常见的定义。

然后，我们证明这样的定义满足要求、即有序对是有序的。注意在上下文清晰时，对任意 $\alpha_{[x_{1\dots n}]}$ ，我

¹参考 Asaf Karagila. Why is "the set of all sets" a paradox, in layman's terms? Mathematics Stack Exchange. URL: <https://math.stackexchange.com/q/909772>。

²见章节 4.3.1。同时需指出该概念不同于等价类中类的概念，我们使用的等价类总是指集合。

们简写语句 $\forall x_1 \dots x_n \alpha$ 为 α 。

定理 4.1.3 $A_Z \vdash \langle x_1, y_1 \rangle = \langle x_2, y_2 \rangle \rightarrow x_1 = x_2 \wedge y_1 = y_2$ 。

证明：由 = 的要求、有序对的定义及前提 $\langle x_1, y_1 \rangle = \langle x_2, y_2 \rangle$ ，有：

$$\overline{\overline{(\{x_1\} = \{x_2\} \vee \{x_1\} = \{x_2, y_2\})}}^{\alpha_1} \wedge \overline{\overline{(\{x_1, y_1\} = \{x_2\} \vee \{x_1, y_1\} = \{x_2, y_2\})}}^{\beta_1} \quad (i)$$

$$\overline{\overline{(\{x_2\} = \{x_1\} \vee \{x_2\} = \{x_1, y_1\})}}^{\alpha_2} \wedge \overline{\overline{(\{x_2, y_2\} = \{x_1\} \vee \{x_2, y_2\} = \{x_1, y_1\})}}^{\beta_2} \quad (ii)$$

分类讨论 (i) 中的四种情况：

1. $\alpha_1 \wedge \beta_1$ ，则可以看到 $x_1 = x_2 = y_1$ ，代入 (ii) 有 $x_1 = x_2 = y_1 = y_2$ 。
2. $\alpha_1 \wedge \beta_2$ ，代（由 α_1 得到的） $x_1 = x_2$ 入 β_2 有 $y_1 = x_1 \vee y_1 = y_2$ ，前者同上，后者即 $x_1 = x_2 \wedge y_1 = y_2$ 。
3. $\alpha_2 \wedge \beta_1$ ，则易见 $x_1 = x_2 = y_1 = y_2$ 。
4. $\alpha_2 \wedge \beta_2$ ，则代（由 α_2 得到的） $x_1 = x_2 = y_2$ 入 β_2 同样有 $x_1 = x_2 = y_1 = y_2$ 。 $\square$

接着，我们尝试给出一棵较为完整的推导树，从中将看到其描述精度和工作量的显著差异。首先我们对定义进行展开，令 $P(p, x, y)$ 为 $\forall z (z \in p \leftrightarrow z = x \vee z = y)$ 、即 $p = \{x, y\}$ 。目标展开后等价于：

$$\begin{aligned} A_Z \vdash & P(a_1, x_1, x_1) \wedge P(b_1, x_1, y_1) \wedge P(p_1, a_1, b_1) \wedge P(a_2, x_2, x_2) \wedge P(b_2, x_2, y_2) \wedge P(p_2, a_2, b_2) \\ & \rightarrow (p_1 = p_2 \rightarrow x_1 = x_2 \wedge y_1 = y_2) \end{aligned}$$

则对 $i \in \{1, 2\}$ ， a_i 对应 $\{x_i\}$ ， b_i 对应 $\{x_i, y_i\}$ ， p_i 对应 $\langle x_i, y_i \rangle = \{a_i, b_i\}$ 。

下一步我们得到 (i) 对应的四种情况：

$$\begin{aligned} & \overline{\overline{P(p_1, a_1, b_1) \Rightarrow a_1 \in p_1}}^{\overline{\overline{p_2 = p_1, P(p_2, a_2, b_2) \Rightarrow P(p_1, a_2, b_2)}}} \quad (\mathbf{T}_{ep}^{(p_1, a_1, b_1)}) \\ & \overline{\overline{P(p_1, a_1, b_1), p_2 = p_1, P(p_2, a_2, b_2) \Rightarrow a_1 = a_2, a_1 = b_2}} \\ & \overline{\overline{P(p_1, a_1, b_1), p_2 = p_1, P(p_2, a_2, b_2) \Rightarrow a_1 = a_2, a_1 = b_2}}^{\mathbf{T}_{ep}^{(p_1, a_1, b_1)}} \quad \overline{\overline{P(p_1, b_1, a_1), p_2 = p_1, P(p_2, a_2, b_2) \Rightarrow b_1 = a_2, b_1 = b_2}}^{\mathbf{T}_{ep}^{(p_1, b_1, a_1)}} \\ & \overline{\overline{p_1 = p_2, P(p_1, a_1, b_1), P(p_2, a_2, b_2) \Rightarrow (a_1 = a_2 \vee a_1 = b_2) \wedge (b_1 = a_2 \vee b_1 = b_2)}}^{\mathbf{T}_{i^+}} \\ & \overline{\overline{p_1 = p_2, P(p_1, a_1, b_1), P(p_2, a_2, b_2) \Rightarrow a_1 = a_2 \wedge b_1 = a_2, a_1 = a_2 \wedge b_1 = b_2, a_1 = b_2 \wedge b_1 = a_2, a_1 = b_2 \wedge b_1 = b_2}} \end{aligned}$$

对 (ii) 类似的：

$$\overline{\overline{p_1 = p_2, P(p_1, a_1, b_1), P(p_2, a_2, b_2) \Rightarrow (a_2 = a_1 \vee a_2 = b_1) \wedge (b_2 = a_1 \vee b_2 = b_1)}}^{\mathbf{T}_{ii}} \quad (\mathbf{T}_{ii})$$

考虑 $\mathbf{T}_{i^+}$ 后项中的四种情况：

1. $a_1 = a_2 \wedge b_1 = a_2$ ：

- 由 $a_1 = a_2$ 有：

$$\overline{\overline{a_1 = a_2, P(a_1, x_1, x_1), P(a_2, x_2, x_2) \Rightarrow x_1 = x_2}}^{\mathbf{T}_{ep}^{(a_1, x_1, x_1)}} \dots \quad (4.1.1)$$

- 由 $b_1 = a_2$ 有：

$$\overline{\overline{b_1 = a_2, P(b_1, x_1, y_1), P(a_2, x_2, x_2) \Rightarrow y_1 = x_2}}^{\mathbf{T}_{ep}^{(b_1, y_1, x_1)}} \dots \quad (4.1.2)$$

- 考虑 T_{ii} 后项, 由 $b_2 = a_1$ 有:

$$\overline{\overline{b_2 = a_1, P(b_2, x_2, y_2), P(a_1, x_1, x_1) \Rightarrow y_2 = x_1}} T_{ep}\left(\begin{smallmatrix} b_2, y_2, x_2 \\ a_1, x_1, x_1 \end{smallmatrix}\right) \dots \quad (4.1.3)$$

- 进一步有:

$$\begin{array}{ccc} \vdots (4.1.3) & \vdots (4.1.1) & \vdots (4.1.2) \\ b_2 = a_1, \dots & a_1 = a_2, \dots & b_1 = a_2, \dots \\ \Rightarrow y_2 = x_1 & \Rightarrow x_1 = x_2 & \Rightarrow y_1 = x_2 \\ \hline b_2 = a_1, a_1 = a_2, b_1 = a_2, \dots \Rightarrow x_1 = x_2 \wedge y_1 = y_2 \end{array} \quad (4.1.4)$$

- 由 $b_2 = b_1$ 有:

$$\overline{\overline{b_2 = b_1, P(b_2, x_2, y_2), P(b_1, x_1, y_1) \Rightarrow y_2 = x_1, y_2 = y_1}} T_{ep}\left(\begin{smallmatrix} b_2, y_2, y_1 \\ b_1, x_1, y_1 \end{smallmatrix}\right) \dots \quad (4.1.5)$$

- 进一步有:

$$\begin{array}{ccc} \vdots (4.1.5) & \vdots (4.1.1) & \vdots (4.1.2) \\ b_2 = b_1, \dots & a_1 = a_2, \dots & b_1 = a_2, \dots \\ \Rightarrow y_2 = x_1, y_2 = y_1 & \Rightarrow x_1 = x_2 & \Rightarrow y_1 = x_2 \\ \hline b_2 = b_1, a_1 = a_2, b_1 = a_2, \dots \Rightarrow x_1 = x_2 \wedge y_1 = y_2 \end{array} \quad (4.1.6)$$

- 则:

$$\begin{array}{ccc} \vdots T_{ii} \dots & \vdots (4.1.4) & \vdots (4.1.6) \\ p_1 = p_2, \dots & b_2 = a_1, a_1 = a_2, b_1 = a_2, \dots & b_2 = b_1, a_1 = a_2, b_1 = a_2 \\ \Rightarrow b_2 = a_1, b_2 = b_1 & \Rightarrow x_1 = x_2 \wedge y_1 = y_2 & \Rightarrow x_1 = x_2 \wedge y_1 = y_2 \\ \hline p_1 = p_2, a_1 = a_2, b_1 = a_2, \dots \Rightarrow x_1 = x_2 \wedge y_1 = y_2 \end{array} \quad (4.1.7)$$

2. $a_1 = a_2 \wedge b_1 = b_2$:

- 由 $b_1 = b_2$ 有:

$$\overline{\overline{b_1 = b_2, P(b_1, x_1, y_1), P(b_2, x_2, y_2) \Rightarrow y_1 = x_2, y_1 = y_2}} T_{ep}\left(\begin{smallmatrix} b_1, y_1, x_1 \\ b_2, x_2, y_2 \end{smallmatrix}\right) \quad (4.1.8)$$

- 还有 (对比 (4.1.6)):

$$\begin{array}{ccc} \vdots (4.1.1) & & \vdots (4.1.5) \\ a_1 = a_2, \dots \Rightarrow x_1 = x_2 & b_2 = b_1, \dots \Rightarrow y_2 = x_1, y_2 = y_1 & \\ \hline y_1 = x_2, a_1 = a_2, b_1 = b_2, \dots \Rightarrow x_1 = x_2 \wedge y_1 = y_2 \end{array} \quad (4.1.9)$$

- 则:

$$\begin{array}{ccc} \vdots (4.1.1) & \vdots (4.1.8) & \vdots (4.1.9) \\ a_1 = a_2, \dots & b_1 = b_2, \dots & y_1 = x_2, a_1 = a_2, b_1 = b_2, \dots \\ \Rightarrow x_1 = x_2 & \Rightarrow y_1 = x_2, y_1 = y_2 & \Rightarrow x_1 = x_2 \wedge y_1 = y_2 \\ \hline a_1 = a_2, b_1 = b_2, \dots \Rightarrow x_1 = x_2 \wedge y_1 = y_2 \end{array} \quad (4.1.10)$$

3. $a_1 = b_2 \wedge b_1 = a_2$:

- 由 $a_1 = b_2$ (除类 (4.1.3) 结果外还) 有:

$$\overline{\overline{a_1 = b_2, P(a_1, x_1, x_1), P(b_2, x_2, y_2) \Rightarrow x_1 = x_2}} T_{ep}\left(\begin{smallmatrix} b_2, x_2, y_2 \\ a_1, x_1, x_1 \end{smallmatrix}\right) \dots \quad (4.1.11)$$

- 由 $b_1 = a_2$ 有:

$$\frac{}{b_1 = a_2, P(b_1, x_1, y_1), P(a_2, x_2, x_2) \Rightarrow y_1 = x_2} \text{T}_{ep}\left(\frac{b_1, y_1, x_1}{a_2, x_2, x_2}\right) \dots \quad (4.1.12)$$

- 则:

$$\frac{\begin{array}{c} \vdots (4.1.3) \dots \\ a_1 = b_2, \dots \\ \Rightarrow x_1 = y_2 \end{array} \quad \begin{array}{c} \vdots (4.1.11) \\ a_1 = b_2, \dots \\ \Rightarrow x_1 = x_2 \end{array} \quad \begin{array}{c} \vdots (4.1.12) \\ b_1 = a_2, \dots \\ \Rightarrow y_1 = x_2 \end{array}}{a_1 = b_2, b_1 = a_2, \dots \Rightarrow x_1 = x_2 \wedge y_1 = y_2} \quad (4.1.13)$$

4. $a_1 = b_2 \wedge b_1 = b_2$:

- 则:

$$\frac{\begin{array}{c} \vdots (4.1.3) \dots \\ a_1 = b_2, \dots \\ \Rightarrow x_1 = y_2 \end{array} \quad \begin{array}{c} \vdots (4.1.11) \\ a_1 = b_2, \dots \\ \Rightarrow x_1 = x_2 \end{array} \quad \begin{array}{c} \vdots (4.1.8) \\ b_1 = b_2, \dots \\ \Rightarrow y_1 = x_2, y_1 = y_2 \end{array}}{a_1 = b_2, b_1 = b_2, \dots \Rightarrow x_1 = x_2 \wedge y_1 = y_2} \quad (4.1.14)$$

综上:

$$\frac{\begin{array}{c} \vdots \text{T}_{1+} \\ \vdots (4.1.7) \end{array} \quad \begin{array}{c} \vdots (4.1.10) \end{array} \quad \begin{array}{c} \vdots (4.1.13) \end{array} \quad \begin{array}{c} \vdots (4.1.14) \end{array}}{\frac{p_1 = p_2, \dots \Rightarrow x_1 = x_2 \wedge y_1 = y_2}{\dots \Rightarrow p_1 = p_2 \rightarrow x_1 = x_2 \wedge y_1 = y_2}}$$

这样，我们即完成了基本由推导树给出的证明。除精度与篇幅的差异外，从中我们至少还能看到两点。一是有推导过程采用相同的模式，如对任意 $p_1 \ x_1 \ y_1 \ p_2 \ x_2 \ y_2$ ，由 $\text{T}_{ep}\left(\frac{p_1, x_1, y_1}{p_2, x_2, y_2}\right)$ 均能得到：

$$p_1 = \{x_1, y_1\} \wedge p_2 = \{x_2, y_2\} \wedge p_1 = p_2 \rightarrow x_1 = x_2 \vee x_1 = y_2$$

二是有大量子推导树的直接复用。这两点都说明了推导的子过程同样是可被提取的，事实上，引理、定理、推论等类似概念的提出和区分即是不严格的子过程提取结果和层级结构。也应注意这种提取和复用和编程实践中对方法和框架的抽象非常相似。

同时我们仍需承认上面的推导树还有很多空隙，其子过程提取和复用机制也仍是不严格的，以及其形式亦是不易直接应用于解决实际问题的。为此，各种如 Coq 这样由形式化语言编写，可由机器检查，且设计目标即面向应用的工具被相继提出。在第一章中我们已经看到了应用 Coq 的具体例子，在后面的章节如 5.4.4 中我们将更详细的介绍更多不同的工具。而像上面这样几乎完全采用推导树给出证明过程的论证方式将暂告一段落。

回到对有序对的讨论，进一步可以定义：

- 有序对判定谓词。对任意 p ，可定义 $isOrderedPair(p)$ 为 $\exists x, y \ p = \{\{x\}, \{x, y\}\}$ 。
- 有序对前项判定谓词。对任意 $p \ x$ ，可定义 $isFst(p, x)$ 为 $\exists y \ \{\{x\}, \{x, y\}\}$ 。由定理 4.1.3 可见：

$$isFst(p, x_1) \wedge isFst(p, x_2) \rightarrow x_1 = x_2$$

即有序对前项具有唯一性，则可定义任意 $P(fst(p))$ 为 $\forall x (isFst(p, x) \rightarrow P(x))$ ，注意当 p 不为有序对时，其总为真。

- 有序对后项判定谓词。类似的，对任意 p, y ，可定义 $isSnd(p, y)$ 为 $\exists x \{ \{x\}, \{x, y\} \}$ 。及任意 $P(snd(p))$ 为 $\forall y (isSnd(p, y) \rightarrow P(y))$ 。

4.1.6 关系和函数

对任意集合 a 有存在唯一的幂集 $\wp(a)$ ，其存在性可由幂集公理直接得到。则可定义：

- 任意 $P(\wp(a))$ 为 $\forall b (\forall x (x \in b \leftrightarrow x \subseteq a) \rightarrow P(b))$ 。

回顾笛卡尔积、关系和函数的基本概念¹，可以看到可定义（并由分类公理、即子集公理可证明存在）：

- 任意笛卡尔积 $a \times b$ 为 $\{ \langle x, y \rangle \in \wp(\wp(a \cup b)) : x \in a \wedge y \in b \}$ 。写 a^n ($n \geq 1$) 指 $\overbrace{a \times \dots \times a}^n$ 。
- 任意定义域为 a 的子集、值域为 b 的子集且其中元素满足 P 的关系为 $\{ p \in a \times b : P(p) \}$ 。
- 关系判定谓词。对任意 r ，定义 $isRelation(r)$ 为 $\exists a, b \ r \subseteq a \times b$ 。
- 任意 $x r y$ 为 $\langle x, y \rangle \in r$ 。写 $x \not r y$ 表示 $\neg x r y$ 。
- 函数判定谓词。对任意 f ，定义 $isFunction(f)$ 为：

$$isFunction(f) \wedge \forall \langle x_1, y_1 \rangle \langle x_2, y_2 \rangle \in f (x_1 = x_2 \rightarrow y_1 = y_2)$$

注意类似算术中的有界量词，写任意 $\forall x \in b \ \alpha$ 表示 $\forall x (x \in b \rightarrow \alpha)$ ； $\exists x \in b \ \alpha$ 表示 $\exists x (x \in b \wedge \alpha)$ 。并写 $xy \in b$ 表示 $x \in b, y \in b$ 。写 $p = \langle x, y \rangle$ 表示其与 x, y 之间有 $x = fst(p) \wedge y = snd(p)$ ，即如任意 $\forall \langle x, y \rangle P(x, y)$ 表示 $\forall p P(fst(p), snd(p))$ ^{注2}。写 $f : d \mapsto r$ 表示 $isFunction(f) \wedge dom f = d \wedge ran f \subseteq r$ 。

- 任意 $P(f(x))$ 为 $\forall y (x f y \rightarrow P(y))$ 。则复合函数 $f \circ g = \{ \langle x, y \rangle \in (dom g) \times ran f : y = f(g(x)) \}$ 。
- 恒等函数判定谓词。对任意 f ，定义 $isIdentity_{\mathbb{F}}(f)$ 为 $isFunction(f) \wedge \forall \langle x, y \rangle \in f \ x = y$ ，即任意 x 有 $f(x) = x$ 。
- 单射函数判定谓词。对任意 f ，定义 $isInjective_{\mathbb{F}}(f)$ 为：

$$isFunction(f) \wedge \forall \langle x_1, y_1 \rangle \langle x_2, y_2 \rangle \in f (y_1 = y_2 \rightarrow x_1 = x_2)$$

- 双射函数判定谓词。对任意 f, a, b ，定义 $isBijective_{\mathbb{F}}(f, a, b)$ 表示 f 为 a 到 b 上的双射函数为：

$$isInjective_{\mathbb{F}}(f) \wedge dom f = a \wedge ran f = b$$

可类似的扩展定义多元组、多元关系以及多元函数。

又因 $x r y \rightarrow x \in \bigcup \bigcup r \wedge y \in \bigcup \bigcup r$ ，定义：

- 任意关系 r 的定义域 $dom(r)$ 为 $\{ x \in \bigcup \bigcup r : \exists y \ x r y \}$ 。写 $dom r = dom(r)$ 。
- 任意关系 r 的值域 $ran(r)$ 为 $\{ y \in \bigcup \bigcup r : \exists x \ x r y \}$ 。写 $ran r = ran(r)$ 。并写 $fld r = dom r \cup ran r$ 。

则可定义：

- 逆关系。对任意 r ，定义 r^{-1} 为 $\{ \langle x, y \rangle \in ran r \times dom r : y r x \}$ 。有：

¹ 章节 2.1.1。

² $\langle x, y \rangle$ 的写法会带来其是二元组的暗示，尽管在这里和下文的使用中一般也应如此，但从纯形式的角度其只是按上述方式替代了 $fst\ snd$ 的使用。即如语句 $P(fst(p))$ 在 p 不为二元组时也是合法的，其是否可证也应考虑 p 非二元组时的情况。

$$A_Z \vdash \text{isInjective}_{\mathbb{P}}(f) \rightarrow \text{isInjective}_{\mathbb{P}}(f^{-1})$$

及谓词¹:

- 自反关系: $\text{isReflexive}(a, r)$ 为 $r \subseteq a^2 \wedge \forall x \in a \ x r x$ 。
- 反自反关系: $\text{isAntiReflexive}(r)$ 为 $\forall x \ x \not r x$ 。
- 对称关系: $\text{isSymmetry}(r)$ 为 $\forall x, y \ x r y \rightarrow y r x$ 。
- 反对称关系: $\text{isAntiSymmetry}(r)$ 为 $\forall x, y \ x r y \wedge y r x \rightarrow x = y$ 。
- 非对称关系: $\text{isAsymmetry}(r)$ 为 $\forall x, y \ x r y \rightarrow y \not r x$ 。
- 传递关系: $\text{isTransitive}(r)$ 为 $\forall x, y, z \ x r y \wedge y r z \rightarrow x r z$ 。
- 强连通关系: $\text{isStrictConnected}(a, r)$ 为 $r \subseteq a^2 \wedge \forall x y \in a \ x r y \vee y r x$ 。
- 连通关系: $\text{isConnected}(a, r)$ 为 $r \subseteq a^2 \wedge \forall x y \in a \ x r y \vee y r x \vee x = y$ 。

及:

- 等价关系: $\text{isEquivalence}(a, r)$ 为 $\text{isReflexive}(a, r) \wedge \text{isSymmetry}(r) \wedge \text{isTransitive}(r)$ 。
- 偏序关系: $\text{isPartialOrder}(a, r)$ 为 $\text{isReflexive}(a, r) \wedge \text{isAntiSymmetry}(r) \wedge \text{isTransitive}(r)$ 。
- 严格偏序关系: $\text{isStrictPartialOrder}(a, r)$ 为 $r \subseteq a^2 \wedge \text{isAntiReflexive}(r) \wedge \text{isTransitive}(r)$ 。
- 全序关系: $\text{isTotalOrder}(a, r)$ 为 $\text{isPartialOrder}(a, r) \wedge \text{isStrictConnected}(a, r)$ 。
- 严格全序关系: $\text{isStrictTotalOrder}(a, r)$ 为 $\text{isStrictPartialOrder}(a, r) \wedge \text{isConnected}(a, r)$ 。

最后, 回顾集合的基数大小²比较, 可定义:

- 对任意 a, b , $|a| \leq |b|$ 为: $\exists f \text{ isInjective}_{\mathbb{P}}(f) \wedge \text{dom } f = a \wedge \text{ran } f \subseteq b$

定义 $|a| = |b|$ 为 $\exists f \text{ isBijection}_{\mathbb{P}}(f, a, b)$ 。定义 $|a| < |b|$ 为 $|a| \leq |b| \wedge |a| \neq |b|$ 。

则通过 A_Z 足以形式化证明康托尔通过对角论证法³得到的结论:

定理 4.1.4 (康托尔定理) $A_Z \vdash |a| < |\wp(a)|$ 。

证明: 对任意函数 $f \subset a \times \wp(a)$, 考虑集合 $b = \{ x \in a : x \notin f(x) \}$, 可见 $b \in \wp(a)$ 。我们证明 $b \notin \text{ran } f$, 即不存在 $x \in a$ 有 $b = f(x)$ 。假设这样的 x 存在, 若 $x \in b$, 则 $x \notin f(x) = b$, 矛盾, 否则 $x \notin b$ 则 $x \in f(x)$ 有 $x \in b$, 亦矛盾。□

更多相关定义和性质将在需要的时候给出。

4.1.7 无限

到目标为止我们已经应用了 A_Z 中除无限公理外的全部公理。也应注意到不应用无限公理我们无法证明任何无限集合的存在, 因对任意有限集合有限次的应用配对、并集或幂集公理只能在有限的范围内扩大被证明存在的集合内的元素个数。

例如, 回顾任意 $\wp^n(\emptyset)$ 表示对 $\emptyset$ 执行 n 次幂集运算后的结果, 尽管通过 $A_Z \setminus \text{Inf}$ 可以对任意 (自然数) m , 证明集合 $\{ \wp^0(\emptyset), \dots, \wp^m(\emptyset) \}$ 的存在; 但不能证明无限集合 $\{ \wp^0(\emptyset), \wp^1(\emptyset), \dots \}$ 的存在。

¹回顾章节 2.1.1。

²章节 2.1.2。

³章节 2.1.3。

但显然如自然数集的存在要求集合论可以证明无限集合的存在，因此 A_Z 中包含了无限公理：

$$\exists b \emptyset \in b \wedge \forall x \in b \ x^+ \in b$$

其中，定义：

- 任意 x^+ 为 $x \cup \{x\}$ (即 $x; x$)。则对任意 b 满足 $\emptyset \in b \wedge \forall x \in b \ x^+ \in b$ ，有 b 包含元素：

$$\emptyset, \quad \{\emptyset\}, \quad \{\emptyset, \{\emptyset\}\}, \quad \{\emptyset, \{\emptyset\}, \{\emptyset, \{\emptyset\}\}\}, \quad \dots$$

并定义：

- 任意 $isInductive(a)$ 为 $\emptyset \in a \wedge \forall x \in a \ x^+ \in a$ 。称任意 a 满足 $isInductive(a)$ 为归纳的。则无限公理即为存在归纳集。

也应指出，由增加无限公理到证明集合 $\{\wp^0(\emptyset), \wp^1(\emptyset), \dots\}$ 的存在仍有较大距离，其间我们还需要替换公理模式，详见章节 4.3.1。

4.2 数学对象

4.2.1 自然数

可定义自然数¹：

$$0 = \{\} = \emptyset = \emptyset \quad (4.2.1)$$

$$1 = \{0\} = \{\emptyset\} = \emptyset^+$$

$$2 = \{0, 1\} = \{\emptyset, \{\emptyset\}\} = \emptyset^{++}$$

$$3 = \{0, 1, 2\} = \{\emptyset, \{\emptyset\}, \{\emptyset, \{\emptyset\}\}\} = \emptyset^{+++}$$

⋮

则（标准）自然数集 $\mathbb{N}$ 即应为仅由这些元素组成的集合。然而，任意集合 B 为归纳集只要求 $\mathbb{N} \subseteq B$ 。事实上，我们并不能在（一阶）集合论中表达集合满足这样的要求 i ：即其为无限的且其中元素均为自然数。回顾在章节 3.2.4 中我们通过紧性定理构造了 $\mathbf{Q}$ 的非标准算术模型²。类似我们可构造集合论模型有其中最小归纳集包含非（标准）自然数的元素³。

尽管又一次遇到了语法和语义的不一致。我们仍能要求可证明存在某一可定义的集合有其在预期解释下为 $\mathbb{N}$ 。其中，预期解释即指（一般）期望的我们的理论需可处理的解释，如标准解释即可认为是算术理论的预期解释。而演绎规则的可靠性使我们的系统仍在大部分情况下有用，即其保证可证明的语句在预期解释下为真。回到这里对应 $\mathbb{N}$ 的可定义集合，可定义：

- 任意 $P(\omega)$ 为⁴：

¹将在下一小节看到这样定义自然数的优势。

²以及在章节 3.2.5 末（通过哥德尔语句）我们指出非标准算术模型的存在，以及在章节 3.2.8 中进一步指出无法在算术语句中表达变量为自然数。

³可参考 hmakholm left over Monica. Proof that the minimal inductive set does not contain any more elements than $\{\emptyset, s(\emptyset), s(s(\emptyset)), s(s(s(\emptyset))), \dots\}$? Mathematics Stack Exchange. URL: <https://math.stackexchange.com/q/1788794>。

⁴选择 ω 一方面因为如下述这里仍区分目标逻辑中的符号和元逻辑模型中的概念，即 $\mathbb{N}$ 用于表示预期解释下自然数集合；另一方面如章节 4.3.4 所示， ω 还为首个无限序数的一般符号。

$$\forall b (isInductive(b) \rightarrow \forall a (\overbrace{\forall x (x \in a \leftrightarrow x \in b \wedge \forall b (isInductive(b) \rightarrow x \in b))}^{\alpha} \rightarrow P(a)))$$

直观的说, 其中 ω 即为 (域中的) 最小归纳集。技术上, 由无限公理可以得到存在归纳集 b , 由分类公理可以得到存在最小归纳集 ω 对应上式 α 部分中的 a 为 $\{x \in b : \forall b (isInductive(b) \rightarrow x \in b)\}$ 。显然, 在含有 $\mathbb{N}$ 的解释中, $\mathbb{N}$ 即为最小归纳集。

这样, 在非预期模型 $\mathfrak{S}$ 注¹ 中, 可能存在非自然数 $e \in \mathfrak{S}_{[\omega]}$ 注²。集合表示法 $\{0, 1, 2, \dots\}$ 中的 $\dots$ 可能包含比我们认为的更多的内容。

事实上, 模型和模型之间存在着更大的差异。如由定理 4.1.4 有 $|\omega| < |\wp(\omega)|$, 即 i: 存在集合 $\wp(\omega)$ 是不可数的, 即其模型的域中存在不可数多个元素属于 $\wp(\omega)$ 。但由勒文海姆-斯科伦定理 3.1.15 有 ii: 存在模型其域是可数的。我们称这种看似矛盾的论述为斯科伦悖论。但斯科伦悖论并不是真正的悖论, 在非预期模型中, 类似 ω 可能含有更多的元素, $\wp(\omega)$ 也可能仅含少至可数的元素。而解释悖论的关键在于 i 中的不可数是目标逻辑下的概念, ii 中的可数是元逻辑下的概念。当域中 $\wp(\omega)$ 仅含元逻辑下可数的元素时, 其在目标逻辑看来即是不可数的。

从这个具体的例子进一步考察勒文海姆-斯科伦定理, 对照该定理相关的讨论³, 元逻辑 A_Z 下可证明存在的目标逻辑 $\wp(\omega)$ 的解释有其中元素是可数的, 而可见按预期方式可构造有 $\wp(\omega)$ 的解释不可数, 则其间有各种不同的元素均可以是 $\wp(\omega)$ 在模型中的对应解释。

接着我们讨论元逻辑下可数在目标逻辑下不可数的含义。首先要指出元逻辑下可数表示我们使用的元逻辑亦为集合论, 但目标逻辑的集合 A 在解释 $\mathfrak{S}$ 中对应的 $\mathfrak{S}_{[A]}$ 并不需要 (在形式上) 等于 A , 即 A 在元逻辑中可以是任意的, 目标逻辑的 $\in$ 关系亦不需等于元逻辑中的 $\in$, 即其解释也可以是元逻辑中的任意二元关系。这种任意的解释只要满足了由目标逻辑给出的语句的真假要求, 该解释即为目标逻辑的模型⁴。

尽管上述讨论中我们认为 $\mathbb{N}$ 为预期模型中的 ω , 但鉴于 (在目标逻辑中) 无法区分两者, 我们令 $\mathbb{N} = \omega$, 且在这两章节中我们一般使用后者表示自然数集合。

现更严格的定义:

- $\aleph_0$ 为 $|\omega|$ 。
- 可数判定谓词。对任意 a , 定义 $isCountable(a)$ 为 $|a| \leq \aleph_0$ 。

则部分展开 $\neg isCountable(\wp(\omega))$ 有:

$$\neg (\exists f isInjective_{\mathbb{F}}(f) \wedge dom f = \wp(\omega) \wedge ran f = \omega) \quad (4.2.2)$$

当 $\wp(\omega)$ 在模型 $\mathfrak{S}$ 中含有更少的元素时, $\mathfrak{S}$ 的域 $\mathfrak{S}_{[\mathbb{V}]}$ 中也将含有更少的元素。而定理 4.1.4 即通过在目标逻辑中证明上式 (即 $A_Z \vdash (4.2.2)$), 指出 $\mathfrak{S}_{[\mathbb{V}]}$ 中没有 f 满足语句 $isInjective_{\mathbb{F}}(f) \wedge dom f = \wp(\omega) \wedge ran f = \omega$ 。但这不代表在元逻辑中无法证明存在由 $\mathfrak{S}_{[\wp(\omega)]}$ 到 (元逻辑中的) ω 的单射函数、即可数。⁵

¹而类似的, 预期模型即对应算术理论如 $\mathbf{P}$ 的标准 (算术) 模型, 但在集合论下, 标准模型表示其它 (不相关的) 含义, 见 180 注 1。

²回顾 $\mathfrak{S}_{[x]}$ 表示 x 在 $\mathfrak{S}$ 中的对应解释。

³章节 3.1.5。

⁴如前述标准模型在模型论下有不同含义, 其中一个要求即为模型中的 $\in$ 关系为元逻辑中 $\in$ 关系的子集。见 180 注 1。

⁵参考 George S. Boolos, John P. Burgess, and Richard C. Jeffrey. Computability and Logic. 5th ed. Cambridge University Press, 2007, pp. 251–253 以及 Timothy Bays. “Skolem’s Paradox”. In: The Stanford Encyclopedia of Philosophy. Ed. by Edward N. Zalta. Winter 2014. Metaphysics Research Lab, Stanford University, 2014.

回到归纳集及 ω 的定义，我们有：

定理 4.2.1 (归纳原理) $A_Z \vdash p \subseteq \omega \wedge \text{isInductive}(p) \rightarrow p = \omega$ 。

证明： ω 为最小归纳集。 □

事实上，这就是我们一直使用的自然数上的数学归纳法。即意为对任意自然数属性（决定的子集合） p ，若可以证明 $0 \in p$ 且 $\forall x \in p, x^+ \in p$ ，则 $p = \omega$ 。可以看到这里 ω 在非预期模型中潜在的非自然数元素并不会影响该原理，因在任何模型中， ω 总是最小归纳集。

4.2.2 次序关系

以 (4.2.1) 方式定义，可以看到任意自然数 n 为由比其小的自然数构成的集合。则可定义：

- 自然数集上的小于关系 $<_\omega$ 为 $\{ \langle x, y \rangle \in \omega^2 : x \in y \}$ 。即任意 $x <_\omega y$ 当且仅当 $x \in y$ 。写任意 $x \leq_\omega y$ 表示 $x <_\omega y \vee x = y$ 。

我们将首先证明 $<_\omega$ 是 ω 上的严格全序关系。即需证 $\text{isStrictTotalOrder}(\omega, <_\omega)$ 。即：

$$\text{isAntiReflexive}(<_\omega) \wedge \text{isTransitive}(<_\omega) \wedge \text{isConnected}(\omega, <_\omega)$$

为此，定义：

- 传递集¹ 判定谓词。对任意 a ，定义 $\text{isTransitiveSet}(a)$ 为 $\forall x, u (x \in u \wedge u \in a \rightarrow x \in a)$ 。

可见：

引理 4.2.2

- $A_Z \vdash \text{isTransitiveSet}(a) \leftrightarrow \bigcup a \subseteq a$ 。
- $A_Z \vdash \text{isTransitiveSet}(a) \leftrightarrow \forall u \in a, u \subseteq a$ 。

以及：

引理 4.2.3 $A_Z \vdash \text{isTransitiveSet}(a) \rightarrow \bigcup a^+ = a$ 。

证明：

$$\begin{aligned} \bigcup a^+ &= \bigcup (a \cup \{a\}) \\ &= \bigcup a \cup \bigcup \{a\} && \text{由 } A_Z \vdash \bigcup (a \cup b) = \bigcup a \cup \bigcup b。 \\ &= \bigcup a \cup a \\ &= a && \text{由上述引理 } a。 \end{aligned}$$

□

引理 4.2.4 $A_Z \vdash \forall n \in \omega, \text{isTransitiveSet}(n)$ 。

证明：首先存在 $t = \{ n \in \omega : \text{isTransitiveSet}(n) \}$ 。然后我们应用归纳原理证明 $t = \omega$ 。显然 $0 \in t$ 。若 $n \in t$ ，由上述引理有 $\bigcup n^+ = n$ ，结合 $n \subseteq n^+$ 有 $\bigcup n^+ \subseteq n^+$ 。则同样由上述引理 a 有 $\text{isTransitiveSet}(n^+)$ 。则 $n^+ \in t$ 。由归纳原理有 $t = \omega$ 。 □

引理 4.2.5 $A_Z \vdash \text{isTransitiveSet}(\omega)$ 。

¹注意区别于传递关系。

证明：由上述引理 b ，即需证 $\forall n \in \omega \ n \subseteq \omega$ 。同样应用归纳原理证明 $t = \{ n \in \omega : n \subseteq \omega \}$ 有 $t = \omega$ 。有 $0 \in t$ 。若 $n \in t$ ，有 $n \subseteq t$ 且 $\{n\} \subseteq t$ ，则 $n^+ = n \cup \{n\} \subseteq t$ 。则 $n^+ \in t$ 。有 $t = \omega$ 。□

接着我们逐一证明严格全序的三个条件。

引理 4.2.6 $A_Z \vdash \forall m n \in \omega (m \in n \leftrightarrow m^+ \in n^+)$ 。

证明：正方向。应用归纳原理证明 $t = \{ n \in \omega : \forall m \in \omega (m \in n \rightarrow m^+ \in n^+) \}$ 有 $t = \omega$ 。显然 $0 \in t$ 。若 $n \in t$ ，需要证明 i：任意 $m \in n^+ \rightarrow m^+ \in n^{++}$ 。注意到：

$$A_Z \vdash x \in y^+ \leftrightarrow x \in y \vee x = y \quad (4.2.3)$$

则分类讨论 m ，若 $m \in n$ ，由归纳假设 $n \in t$ 有 $m^+ \in n^+$ ，结合 $n^+ \subseteq n^{++}$ 有 $m^+ \in n^{++}$ 。若 $m = n$ ，则 $m^+ = n^+ \in n^{++}$ 。即得 i。由归纳原理有 $t = \omega$ 。

反方向。若 $m^+ \in n^+$ ，由 (4.2.3) 有 $m^+ \in n \vee m^+ = n$ 。结合 $m \in m^+$ ，若 $m^+ \in n$ ，由 n 是传递集有 $m \in n$ ，否则 $m^+ = n$ ，则直接有 $m \in n$ 。□

引理 4.2.7 $A_Z \vdash \text{isAntiReflexive}(<_\omega)$ 。

证明：考虑 $\text{fld } <_\omega \subseteq \omega$ ，即仅需证明 $\forall n \in \omega \ n \notin n$ 。应用归纳原理证明 $t = \{ n \in \omega : n \notin n \}$ 有 $t = \omega$ 。显然 $0 \in t$ ，若 $n \in t$ ，则 $n \notin n$ ，由上述引理有 $n^+ \notin n^+$ ，有 $n^+ \in t$ 。则 $t = \omega$ 。□

引理 4.2.8 $A_Z \vdash \text{isTransitive}(<_\omega)$ 。

证明：对任意 $l <_\omega m \wedge m <_\omega n$ ，有 $l \in m \in n$ 且 $l \in \omega$ ， $n \in \omega$ ，有 n 为传递集，有 $l \in n$ ，有 $l <_\omega n$ 。□

引理 4.2.9 $A_Z \vdash \text{isConnected}(\omega, <_\omega)$ 。

证明：应用归纳原理证明 $t = \{ n \in \omega : \forall m \in \omega (m \in n \vee m = n \vee n \in m) \}$ 有 $t = \omega$ 。显然 $0 \in t$ 。若 $n \in t$ ，对任意 m ，若 $m \in n \vee m = n$ ，则由 (4.2.3) 有 $m \in n^+$ ，否则 $n \in m$ ，由引理 4.2.6 有 $n^+ \in m^+$ ，同样由 (4.2.3) 有 $n^+ \in m \vee n^+ = m$ 。则 $t = \omega$ 。□

即有：

引理 4.2.10 $A_Z \vdash \text{isStrictTotalOrder}(\omega, <_\omega)$ 。

进一步称 A 上的（严格）全序关系 R 为（严格）良序关系，若任意 A 的非空子集均有关于 R 的最小值。写：

- $\text{isMinimal}(x, a, r)$ 表示 $x \in a \wedge \forall y \in a \neg y r x$ 。

则任意 A 的非空子集 B 有关于 R 的最小值即指 $\exists x \text{isMinimal}(x, B, R)$ 。定义良序关系谓词：

- $\text{isWellOrder}(a, r)$ 为 $\text{isTotalOrder}(a, r) \wedge \forall b (b \subseteq a \wedge b \neq \emptyset \rightarrow \exists x \text{isMinimal}(x, b, r))$ 。
- $\text{isStrictWellOrder}(a, r)$ 为 $\text{isStrictTotalOrder}(a, r) \wedge \forall b (b \subseteq a \wedge b \neq \emptyset \rightarrow \exists x \text{isMinimal}(x, b, r))$ 。

注意由全序关系有此处的 $\text{isMinimal}(x, a, r)$ 等价于 $x \in a \wedge \forall y \in a \ x r y \vee x = y$ 。

有：

引理 4.2.11 $A_Z \vdash isStrictWellOrder(\omega, <_\omega)$ 。

证明：假设 $b \subseteq \omega$ 且 $\neg \exists x isMinimal(x, b, <_\omega)$ ，即需证 $b = \emptyset$ 。考虑集合：

$$t = \{ n \in \omega : \forall m \in b \ m \not<_\omega n \}$$

我们首先应用归纳原理证明 $t = \omega$ 。显然 $0 \in t$ 。若 $n \in t$ ，由 i: $\forall m \in b \ m \not<_\omega n$ 要证明 $\forall m \in b \ m \not<_\omega n^+$ ，即 (结合 (4.2.3)) 需证 $\forall m \in b \ m \neq n$ 、即 ii: $\neg \exists m \in b \ m = n$ 。假设 ii 不成立，则 $n \in b$ ，结合 $\forall m \in b \ m \not<_\omega n$ ，则 $isMinimal(n, b, <_\omega)$ ，矛盾。则 $n^+ \in t$ 。有 $t = \omega$ 。

然后证明 $b = \emptyset$ 。若存在 $m \in b$ ，由 $b \subseteq \omega$ 有 $m \in t$ ，则 $m^+ \in t$ ，则 $m \not<_\omega m^+$ ，矛盾。 $\square$

事实上，良序关系和严格良序关系是可互相构造的，因：

引理 4.2.12

a. $A_Z \vdash isTotalOrder(a, r) \rightarrow isStrictTotalOrder(a, \{ \langle x, y \rangle \in r : x r y \wedge x \neq y \})$ 。

b. $A_Z \vdash isStrictTotalOrder(a, r) \rightarrow isTotalOrder(a, r \cup \{ \langle x, y \rangle \in a^2 : x = y \})$ 。

因此，我们并不需要区分集合是否具有良序关系或严格良序关系。如：

推论 4.2.12.1 $A_Z \vdash isWellOrder(\omega, \leq_\omega)$ 。

一些时候，我们省略严格。

最后，写：

- 任意 $P(\min_r a)$ 表示 $\forall x (isMinimal(x, a, r) \rightarrow P(x))$ 。

定理 4.2.13 (强归纳原理) 若任意小于 n 的自然数有属性 p 则 n 有属性 p 蕴含任意自然数有属性 p ：

$$A_Z \vdash p \subseteq \omega \wedge \forall n (\forall m <_\omega n \ m \in p \rightarrow n \in p) \rightarrow p = \omega$$

证明：若 $p \neq \omega$ ，则 $\bar{p} = \omega - p \neq \emptyset$ ，由良序存在 $n = \min_{<_\omega} \bar{p}$ ，则任意 $m <_\omega n$ 有 $m \notin \bar{p}$ 有 $m \in p$ 、即 $\forall m <_\omega n \ m \in p$ ，则 $n \in p$ 。矛盾。 $\square$

强归纳原理是另一种常用的数学归纳证明方法，其使我们在做归纳时可以使用更强的归纳假设。¹

4.2.3 递归函数

很多时候，我们通过递归的方式定义函数运算。比如我们在罗宾逊算术²公理集中通过 (Q4) 和 (Q5) 递归定义了加法运算。本小节我们将证明这样的递归定义在 A_Z 中仍是可行的。即在给出了初始项的值 a 和通过前项计算后项的函数 f 后，存在唯一的函数 $h \subseteq \omega \times dom f$ 可计算任意项的值。或定义：

- $isRecursive(h, a, f)$ 为 $isFunction(h) \wedge h(0) = a \wedge \forall n \in \omega \ h(n^+) = f(h(n))$ 。

有：

定理 4.2.14 (递归定理) $A_Z \vdash isFunction(f) \wedge \{ a \} \cup ran f \subseteq dom f \rightarrow \exists! h isRecursive(h, a, f)$ 。

¹ 以上证明参考 H.B. Enderton. Elements of Set Theory. Elsevier Science, 1977, pp. 72–73, 83–87。

² 章节 3.2.4。

证明¹: 写 $P(v)$ 表示:

$$\begin{aligned} isFunction(v) \wedge dom v \subseteq \omega \wedge ran v \subseteq dom f \\ \wedge (0 \in dom v \rightarrow v(0) = a) \\ \wedge \forall n \in \omega (n^+ \in dom v \rightarrow n \in dom v \wedge v(n^+) = f(v(n))) \end{aligned} \quad \begin{matrix} (i) \\ (ii) \end{matrix}$$

称满足 P 的 v 为起始的, 可以看到任意起始的 v 应为 h 自 0 起的某一连续任意长度段。则构造 h 为:

$$\{ \langle n, y \rangle \in \omega \times dom f : \exists v P(v) \wedge y = v(n) \}$$

可以看到 h 是唯一的。同时可见通过 $P(h) \wedge dom h = \omega$ 可以证明 $isRecursive(h, a, f)$ 。接下来我们证明 $P(h) \wedge dom h = \omega$ 。

1. 证明 $P(h)$ 。首先证明 $isFunction(h)$ 。由 P 对定义域的要求有 $dom h \subseteq \omega$, 则通过证明对任意 $n \in \omega$ 至多有唯一 y 有 $n h y$ (即 $\langle n, y \rangle \in h$), 即证明对:

$$t = \{ n \in \omega : \forall y_1, y_2 n h y_1 \wedge n h y_2 \rightarrow y_1 = y_2 \}$$

有 $t = \omega$, 可以证明 $isFunction(h)$ 。应用归纳原则。因 P 要求 (i) 有 $0 \in t$ 。若 $n \in t$, 若对任意 y_1 , 任意起始的 v_1 有 $n^+ \in dom v_1 \wedge y_1 = v_1(n^+)$, 任意 $y_2 v_2$ 有 $n^+ \in dom v_1 \wedge y_1 = v_1(n^+)$, 则由 (ii) 有 $y_1 = f(v_1(n))$, $y_2 = f(v_2(n))$, 由归纳假设 $n \in t$ 有 $v_1(n) = v_2(n)$, 有 $y_1 = y_2$, 则 $n^+ \in t$ 。有 $t = \omega$ 。

进一步, 对条件 (i) 由若 $0 \in dom h$, 则存在起始的 v 有 $0 \in dom v$, 有 $v(0) = a$, 有 $h(0) = a$ 。类似可证明条件 (ii)。有 $P(h)$ 。

2. 证明 $dom h = \omega$ 。应用归纳原理。首先可见 $\{ \langle 0, a \rangle \}$ 是起始的, 则 $0 \in dom h$ 。若 $n \in dom h$, 则存在起始的 v 有 $n \in dom v$, 可见 $v \cup \{ \langle n^+, f(v(n)) \rangle \}$ 也是起始的, 有 $n^+ \in dom h$ 。□

进一步定义:

- 任意 $P(rec[a, f])$ 为 $\forall h (isRecursive(h, a, f) \rightarrow P(h))$ 。

4.2.4 算术模型

接下去我们讨论 Z 和皮亚诺算术 $\mathbf{P}$ ^{注2} 之间的关系。首先, 我们定义更多 Z 中的函数:

- S 为 $\{ \langle m, n \rangle \in \omega^2 : n = m^+ \}$ 。
- $+_\omega$ 为 $\{ \langle m, n, p \rangle \in \omega^3 : p = +_m(n) \}$, 其中 $+_m = rec[m, S]$ 。
- $\cdot_\omega$ 为 $\{ \langle m, n, p \rangle \in \omega^3 : p = \cdot_m(n) \}$, 其中 $\cdot_m = rec[0, +_m]$ 。

可以看到 $S +_\omega \cdot_\omega$ 即为 ω 上的后继、加法和乘法函数。细节上, $+_\omega[m]$ 对应被加数为 m 的加法运算, 可见 $+_m(0) = m$, $+_m(n^+) = S(+_m(n))$ 。类似有 $\cdot_m$ 为被乘数为 m 的乘法运算, $\cdot_m(0) = 0$, $\cdot_m(n^+) = +_m(\cdot_m(n))$ 。同样写 Sx 表示 $S(x)$, 并按惯例写 $m +_\omega n$ 表示 $+_\omega(m, n)$ 等。

回顾 $\mathbf{P}$ 的语言参数有域 $\forall$ 及符号 $0 S + \cdot$, 构造模型 $\mathfrak{N}$ 有其中解释为 $\mathfrak{N}[\forall] = \omega$, $\mathfrak{N}[0] = 0$, $\mathfrak{N}[S] = S$, $\mathfrak{N}[+] = +_\omega$, $\mathfrak{N}[\cdot] = \cdot_\omega$ 。接着我们通过证明 $\mathbf{P}$ 的全部公理³ 在 $\mathfrak{N}$ 下为真证明 $\mathfrak{N}$ 为 $\mathbf{P}$ 的模型:

¹ 参考 H.B. Enderton. Elements of Set Theory. Elsevier Science, 1977, pp. 73–75。

² 章节 3.2.6。

³ 回顾章节 3.2.4、章节 3.2.6。

- (Q1) 对任意 $x \in \omega$ 有 $x \in x^+ = Sx$ 有 $Sx \neq 0$ 。
- (Q2) 由引理 4.2.4、4.2.3 有任意 $n \in \omega$ 有 $\bigcup n^+ = n$ 。则若 $Sm = Sn$ 则 $m^+ = n^+$, $\bigcup m^+ = \bigcup n^+$, 有 $m = n$ 。
- (Q3) 考虑集合 $t = \{n \in \omega : n = 0 \vee \exists m n = Sm\}$ 。应用归纳原理。 $t = \omega$ 。
- (Q4) 由 $+_\omega$ 的定义可得。
- (Q5) 同上。
- (Q6) 由 $\cdot_\omega$ 的定义可得。
- (Q7) 同上。
- (Ind) 由归纳原理可得。

Z 证明了 $\omega \ 0 \ S \ +_\omega \ \cdot_\omega$ 的存在, 则 Z 证明了存在解释 $\mathfrak{N}$ 为 **P** 的模型, 即 Z 作为元逻辑证明了 **P** 的一致性¹。

另一方面, 上述讨论也证明了 **P** 的公理集包含于 Z 集合论。则 **P** 下的理论在 Z 下也可证。则也有定理 3.2.2, 即任意 μ -递归函数可被集合论语句定义。则通过相关 μ -递归函数的定义², 我们可以在 $+_\omega \cdot_\omega$ 的基础上进一步定义其它自然数上的运算符³, 以及可通过任意有限长的自然数序列表达的数学对象。如我们可以定义:

- 有理数集 $\mathbb{Q} = \{ \langle m, n \rangle \in (\omega \setminus 0)^2 : \gcd(m, n) = 1 \}; \langle 0, 1 \rangle$ 注⁴。其中 $\gcd(m, n)$ 计算 $m \ n$ 的最大公因素, 要求 $\gcd(m, n) = 1$ 以避免 $\mathbb{Q}$ 中含值相等的不同元素。则任意 $\langle m, n \rangle$ 为 m / n 。
- 整数集 $\mathbb{Z} = \{ \langle m, n \rangle \in \mathbb{Q} : n = 1 \}$, 即分母为 1 的有理数集。

并可进一步定义整数集和有理数集上的运算符。

4.2.5 实数

由勾股定理有腰长为 1 的等腰直角三角形底边长为 $\sqrt{2}$, 结合其不为有理数⁵, 我们需要比有理数集更大的集合有其中元素可用来如量化长度、时间和质量之类的连续量。这样的集合被定义为实数集。我们将看到按下面的(非有限主义的)定义, 实数集是不可数的, 即不是所有的实数都可被有限长的自然数序列定义, 则上述定义整数和有理数集的方式不适用。

事实上, 我们有很多不同的方法定义实数^{6 7}, 这些方法各有优缺点。这里给出一种常用的通过柯西序列的实数定义方法, 因我们将看到其是更直观的由十进制⁸ 小数表示实数的一般化, 且(如下)有更简单的操作定义, 但其完全性证明较复杂(见下一小节)。

具体的, 称一个无限序列为柯西序列若其序列元素间最终变得任意接近。更精确的, 可定义:

- 有理数集 $\mathbb{Q}$ 上的柯西序列判定谓词 $isCauchySequence_{\mathbb{Q}}(s)$ 为:

$$s : \omega \mapsto \mathbb{Q} \wedge \forall \varepsilon \in \mathbb{Q}_{>0} \exists k \in \omega \forall m n > k |s(m) - s(n)| < \varepsilon \quad (4.2.4)$$

¹ 尽管关于元逻辑的形式化要见下一章。

² 如幂运算, 回顾章节 2.3.4。

³ 显然由 μ -递归函数按定理 3.2.2 转换而来的定义是冗长且不易处理的, 但这里旨在说明其中的可行性, 并尝试得到一个更通用的结论。

⁴ 回顾 $a; b = a \cup \{b\}$, $a \setminus b = a - \{b\}$ 。

⁵ 章节 2.1.4。

⁶ Wikipedia contributors. Construction of the real numbers — Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/w/index.php?title=Construction_of_the_real_numbers&oldid=1068495750. 2022。

⁷ Ittay Weiss. “Survey Article: The real numbers—A survey of constructions”. In: Rocky Mountain Journal of Mathematics 45 (June 2015), pp. 737–762。

⁸ 或二进制。

即无限序列 s 对任意（小）的正有理数 ε ，存在自然数 k 有 k 项之后任意两元素差的绝对值小于 ε 。

则我们可以将实数表示为柯西序列，其值为该序列趋向的收敛结果。比如可以定义 π 为序列：

$$3, \quad 3.1, \quad 3.14, \quad 3.141, \quad 3.1415, \quad \dots$$

则可以看到以这种方式我们可以为任意十进制小数表示的实数构造一个对应的柯西序列。在进一步讨论十进制表示之前，首先需要说明可能有不同的柯西序列收敛结果相同。即如不能直接定义有理数集为底数不为 0 的自然数对，这里也不能直接将实数集定义为 $\mathbb{Q}$ 上的柯西序列的集合，否则其中将有不同的元素应在如预期解释中对应相同的数值。因此，我们需要将实数集定义为柯西序列集由数值相等关系得到的等价类划分¹。下面形式化的给出该定义。

可定义：

- 任意 a 由 a 上的等价关系 r 得到的等价类划分 a/r 为 $\{c \in \wp(a) : \exists x \forall y (y \in c \leftrightarrow x r y)\}$ 注²。

则可定义：

- 实数集 $\mathbb{R}$ 为 $\mathbb{Q}$ 上的柯西序列集合：

$$C_{\mathbb{Q}} = \{s \in \omega \times \mathbb{Q} : isCauchySequence_{\mathbb{Q}}(s)\}$$

由其上的等价关系：

$$\sim_{\mathbb{Q}} = \{ \langle a, b \rangle \in A^2 : \forall \varepsilon \in \mathbb{Q} \exists k \in \omega \forall n > k |a(n) - b(n)| < \varepsilon \}$$

得到的等价类划分 $C_{\mathbb{Q}} / \sim_{\mathbb{Q}}$ 。可以看到其中 $a \sim_{\mathbb{Q}} b$ 表示 a, b 序列中元素最终变得任意接近。

回到十进制表示，我们有 $0.999\dots = 1$ ，即对柯西序列 a 有 $a(n) = 0.\overbrace{9\dots 9}^n$ ， b 有 $b(n) = 1$ 有 $a \sim_{\mathbb{Q}} b$ 。而对于 $0.999\dots = 1$ ，一个更直观的证明方法是应用等比数列求和公式：

$$0.999\dots = \frac{9}{10} + \frac{9}{100} + \frac{9}{1000} + \dots = \frac{9}{10} \cdot \frac{1}{1-\frac{1}{10}} = 1$$

更一般的，任意有限小数有两种十进制表示方式，如 $1.4 = 1.3999\dots$ 。因此，即使不考虑小数点、如仅考虑区间 $[0, 1)$ 中的实数³，其也不直接对应由 0 到 9 构成的无限字符序列。则我们不能直接通过由 0 到 9 构成的序列字符序列集的基数等于 $|\wp(\omega)|$ 注⁴得到 $[0, 1)$ 中的实数集基数亦等于 $|\wp(\omega)|$ 。这里我们需要采用更细致的方法。

引理 4.2.15 $A_{\mathbb{Z}} \vdash |\mathbb{R}| = |\wp(\omega)|$ 。

证明⁵：由 $|\mathbb{R}|$ 小于等于由 0 到 9 及 $.$ 构成的无限字符序列集的基数，有 $|\mathbb{R}| \leq |\wp(\omega)|$ 。另一方面，考虑所有形如 $\sum_{n=1}^{\infty} a_n / 3^n$ 的实数，其中 $a_n \in \{0, 2\}$ ，令其所构成的集合为 A 。可以看到 $A \subset \mathbb{R}$ ，且 $|A|$ 等于由 0 2 构成的无限序列集的基数，则 $|\mathbb{R}| \geq |\wp(\omega)|$ 。□

¹回顾章节 2.1.1。

²Carl Mummert. Equivalence relations and the axiom of choice. Mathematics Stack Exchange. URL: <https://math.stackexchange.com/q/286611>。

³即仅考虑任意大于等于 0 小于 1 的实数。

⁴回顾章节 2.1.2，并注意由 0 到 9 构成的无限字符序列集基数等于由 0 1 构成的无限字符序列集基数，等于 $|\wp(\omega)|$ 。基数（而非基数大小比较）的严格定义见章节 4.3.8。

⁵部分参考 Thomas Jech. Set Theory: The Third Millennium Edition. Springer, 2003, p. 37。

而对于任意以 $\mathbb{Q}$ 上柯西序列表示的实数 s, t ，我们可以在其上方方便的执行加法、乘法和大小比较¹：

- 写柯西序列 $u = s + t$ ，若 $\forall n \in \omega \ u(n) = s(n) + t(n)$ 。
- 写柯西序列 $u = s \cdot t$ ，若 $\forall n \in \omega \ u(n) = s(n) \cdot t(n)$ 。
- 写柯西序列 $u > 0$ ，若 $\exists k \in \omega \exists q \in \mathbb{Q}_{>0} \forall n > k \ u(n) \geq q$ 。写 $s < t$ ，若 $t - s > 0$ ，其中 $u = t - s$ 指 $u + s = t$ 。

但这并不表示我们直接可以在此基础上定义 $\mathbb{R}$ 上的加法、乘法和大小比较，因 $\mathbb{R}$ 中实数定义为 $\mathbb{Q}$ 上柯西序列的 $\sim_{\mathbb{Q}}$ 等价类，即其为柯西序列集合。我们需要一个函数将这样的集合映射到其中的元素。或者说，对任意柯西序列 s ，需要函数 f 有 $f([s]_{\sim_{\mathbb{Q}}})$ 等于 s 或 $[s]_{\sim_{\mathbb{Q}}}$ 中的任意元素，其中定义：

- 任意 x 由等价关系 r 决定的等价类 $[x]_r$ 为 $\{y \in \text{ran } r : x r y\}$ 。

同时这里 $f([s]_{\sim_{\mathbb{Q}}})$ 可等于 $[s]_{\sim_{\mathbb{Q}}}$ 中的任意元素还意味着如：

示例 4.2.16 $A_Z \vdash \forall s t u v \in C_{\mathbb{Q}} \ [s]_{\sim_{\mathbb{Q}}} = [t]_{\sim_{\mathbb{Q}}} \wedge [u]_{\sim_{\mathbb{Q}}} = [v]_{\sim_{\mathbb{Q}}} \rightarrow [s + u]_{\sim_{\mathbb{Q}}} = [t + v]_{\sim_{\mathbb{Q}}}$ ，即由等价类中的元素得到的加法运算结果仍等价。

证明：对任意 $\varepsilon \in \mathbb{Q}$ ，存在 k 满足对任意 $n > k$ 有 $|s(n) - t(n)| < \frac{\varepsilon}{2} \wedge |u(n) - v(n)| < \frac{\varepsilon}{2}$ ，则：

$$|(s(n) + u(n)) - (t(n) + v(n))| < \varepsilon \quad \square$$

最后，我们需要证明这样的 f 存在。而十进制小数表示法为我们提供了一个在这样的柯西序列等价类中定位一个代表元素的方法。具体的，我们可以定义 f 为：

$$\{ \langle c, s \rangle \in \mathbb{R} \times C_{\mathbb{Q}} : s \in c \wedge I(s(0)) \wedge \forall n \in \omega \ E(s(n), s(n^+)) \wedge \exists m > n \ P(s(m)) \}$$

其中， $I(q)$ 表示有理数 q 为整数。 $E(q, r)$ 表示 r 可表示为比 q 可能多一最末非零位的十进制小数且除该位外数值相同，即 r 扩展了 q ，如（不严格的） $E(3.1, 3.14)$ 。 $P(q)$ 表示 q 小数中的最末非零位不等于 9。则上面的 s 为不以 9 循环结尾的十进制小数转换后的柯西序列。且可以看到 $I E P$ 均是可计算的，因此也是在集合论下可定义的。

则我们可定义实数加法、乘法和大小关系 $+_{\mathbb{R}}$ 、 $\cdot_{\mathbb{R}}$ 和 $<_{\mathbb{R}}$ ：

- $+_{\mathbb{R}} = \{ \langle x, y, z \rangle \in \mathbb{R}^3 : f(x) + f(y) \in z \}$ 。
- $\cdot_{\mathbb{R}} = \{ \langle x, y, z \rangle \in \mathbb{R}^3 : f(x) \cdot f(y) \in z \}$ 。
- $<_{\mathbb{R}} = \{ \langle x, y \rangle \in \mathbb{R}^2 : f(x) < f(y) \}$ 注^{2 3}。

4.2.6 实数完全性⁴

由 $\sqrt{2}$ 存在于 $\mathbb{R}$ 中但不存在于 $\mathbb{Q}$ 中，我们已经看到了实数和有理数的区别，现更一般化该区别⁵。为此，对任意 $t \subseteq \mathbb{R}$ ，我们写：

- 任意 $\text{isBound}_{\mathbb{R}}(r, t)$ 表示 $\forall x \in t \ t \leq_{\mathbb{R}} r$ 。写 $\text{isBounded}_{\mathbb{R}}(t)$ 表示 $\exists r \in \mathbb{R} \ \text{isBound}_{\mathbb{R}}(r, t)$ 即 t 有上界 (r) 。

¹Todd Kemp. “CAUCHY’S CONSTRUCTION OF $\mathbb{R}$ ”. in: 2016.

²注意其不是严格良序关系（因不存在该关系下的最小实数），事实上没有集合论语句可定义实数的良序关系。在章节 5.3.7 中我们将构造 ZF 的模型有不存在自然数幂集良序关系，则由实数和有理数和自然数的关系有其上同样没有良序关系。

³尽管下一小节有应用，这里省略以上比较和运算关系符合直观要求的具体证明。这些证明可参考 N.T. Hamilton and J. Landin. Set Theory: The Structure of Arithmetic. Dover Books on Mathematics. Dover Publications, 2018, pp. 187–192, 232–238.

⁴即完备性，这里采用完全性以区分理论完备性。

⁵一些材料中仅写 $\mathbb{R}$ 表示满足其中含该条件的条件集的集合并假定其存在，见 RAFFI GRINBERG. The Real Analysis Lifesaver: All the Tools You Need to Understand Proofs. Princeton University Press, 2017, p. 37. ISBN: 9780691172934.

- 任意 $isSup_{\mathbb{R}}(r, t)$ 表示 $isBound_{\mathbb{R}}(r, t) \wedge \forall r' \in \mathbb{R} (isBound_{\mathbb{R}}(r', t) \rightarrow r \leq_{\mathbb{R}} r')$, 即 r 为 t 的最小上界。

写 $B_{\mathbb{R}}(t)$ 表示 $t \subseteq \mathbb{R} \wedge t \neq \emptyset \wedge isBounded_{\mathbb{R}}(t)$ 、即 t 为非空有上界实数子集。则：

定理 4.2.17 (实数完全性定理) $A_Z \vdash B_{\mathbb{R}}(t) \rightarrow \exists r \in \mathbb{R} isSup_{\mathbb{R}}(r, t)$ 。

注意由 $\sqrt{2}$ 存在上述定理对 $\mathbb{Q}$ 不成立。

而为证明上述定理, 对任意 $s : \omega \mapsto \mathbb{Q}$ 注1:

- 任意 c , 写 $isConvTest_{\mathbb{Q}}(c, s)$ 表示 $c : \mathbb{Q}_{>0} \mapsto \omega \wedge \forall \langle \varepsilon, k \rangle \in c \forall m n > k |s(m) - s(n)| < \varepsilon$ 。称 c 为 s 的收敛测试 (函数)。显然 $\exists c isConvTest_{\mathbb{Q}}(c, s)$ 蕴含 $isCauchySequence_{\mathbb{Q}}(s)$ 。而对任意柯西序列 s , 通过对任意 ε , 从满足 $\rho(k)$ 为 $\forall m n > k |s(m) - s(n)| < \varepsilon$ 的集合 $z \subseteq \omega$ 中选择最小自然数, 我们可以构造 s 的收敛测试。则 $\exists c isConvTest_{\mathbb{Q}}(c, s)$ 等价于 $isCauchySequence_{\mathbb{Q}}(s)$ 。
- 写 $isBound_{S_{\mathbb{Q}}}^{\uparrow}(q, s)$ 表示 $\forall n \in \omega |s(n)| \leq q$, 写 $isBounded_{S_{\mathbb{Q}}}^{\uparrow}(s)$ 表示 $\exists q \in \mathbb{Q} isBound_{S_{\mathbb{Q}}}^{\uparrow}(q, s)$ 。称 s 有届 (q)。
- 写 $isEBound_{S_{\mathbb{Q}}}^{\uparrow}(q, k, s)$ 表示 $\forall n > k |s(n)| \leq q$, 写 $isEBounded_{S_{\mathbb{Q}}}^{\uparrow}(s)$ 表示 $\exists q \in \mathbb{Q}, k \in \omega isEBound_{S_{\mathbb{Q}}}^{\uparrow}(q, k, s)$ 。称 s 本质有界 (q 于 k)。

则：

引理 4.2.18 $A_Z \vdash s : \omega \mapsto \mathbb{Q} \wedge isEBounded_{S_{\mathbb{Q}}}^{\uparrow}(s) \rightarrow isBounded_{S_{\mathbb{Q}}}^{\uparrow}(s)$, 即任意本质有界的有理数序列有界。

证明：由定义存在有理数 q , 自然数 k 对任意 $n > k$ 有 $|s(n)| \leq q$ 。我们对 k 归纳证明 (若存在前述 q) 存在 q' 有对任意自然数 n 有 $|s(n)| \leq q'$ 。当 $k = 0$, $q' = \max\{|s(0)|, q\}$ 满足要求。对 k^+ , s 本质有界 $\max\{|s(k^+)|, q\}$ 于 k , 则由归纳假设有 q' 满足要求。□

引理 4.2.19 $A_Z \vdash s \in C_{\mathbb{Q}} \rightarrow isBounded_{S_{\mathbb{Q}}}^{\uparrow}(s)$, 即任意柯西序列有界。

证明：令 c 为 s 的收敛测试, 我们首先证明 s 本质有界 $|s(c(\frac{5}{9})^+)| + \frac{5}{9}$ 注2。令 $m = c(\frac{5}{9})^+$, 对任意 $n \geq m$:

$$\begin{aligned} |s(n)| &= |s(m) + (s(n) - s(m))| \leq |s(m)| + |s(n) - s(m)| = |s(c(\frac{5}{9})^+)| + |s(n) - s(m)| \\ &< |s(c(\frac{5}{9})^+)| + \frac{5}{9} \end{aligned}$$

其中第二行因 $mn > c(\frac{5}{9})$, 而 c 为 s 的收敛测试。则应用引理 4.2.18 有 s 有界。□

进一步, 我们需要定义几种数集元素之间的转换:

- 写下标 $\mathbb{Z}$ 表示到整数的转换结果。即写 $0_{\mathbb{Z}} = \langle 0, 1 \rangle$ (即 $0/1$), 对任意非零自然数 n , $n_{\mathbb{Z}} = \langle n, 1 \rangle$ 。
- 写下标 $\mathbb{Q}$ 表示到有理数, 可见在我们的定义下对任意自然数 n , $n_{\mathbb{Q}} = n_{\mathbb{Z}}$ 。
- 写下标 $C_{\mathbb{Q}}$ 表示到柯西序列, 对任意 $q \in \mathbb{Q}$, $q_{C_{\mathbb{Q}}}$ 表示 $\omega \times \{q\}$, 显然其为柯西序列 (因其有收敛测试 $\mathbb{Q}_{>0} \times \{0\}$)。对任意 $n \in \omega$, 写 $n_{C_{\mathbb{Q}}}$ 表示 $n_{q_{C_{\mathbb{Q}}}}$ 。
- 写下标 $\mathbb{R}$ 表示到实数, 对任意 $s \in C_{\mathbb{Q}}$, $s_{\mathbb{R}} = [s]_{\sim_{\mathbb{Q}}}$, 类似定义自然数、有理数到实数的转换。

当上下文清晰时, 我们 (一如既往的) 省略 (上述及关系和函数符号如 $<_{\omega}$ 中的) 下标。

引理 4.2.20 $A_Z \vdash m' \in \omega \wedge s \in C_{\mathbb{Q}} \wedge r \in \mathbb{R} \wedge \forall n > m' s(n)_{\mathbb{R}} \geq r \rightarrow s_{\mathbb{R}} \geq r$ 。

¹定理 4.2.17 的证明参考 N.T. Hamilton and J. Landin. Set Theory: The Structure of Arithmetic. Dover Books on Mathematics. Dover Publications, 2018, pp. 187–192, 232–238。

²注意这里 $\frac{5}{9}$ 完全是任意的, 这里选择 $\frac{5}{9}$ 因这里的参考资料 (注 1) 中使用的是 $\frac{5}{9}$ 。

证明：取柯西序列 $v \in r$ （可取 v 等于 r 的小数序列代表），即 $r = u_{\mathbb{R}}$ 。假设 $s_{\mathbb{R}} < r$ ，则 $s < v$ ，则由柯西数列上的比较和运算操作定义，存在自然数 k 、正有理数 q ，有对任意 $n > k$ ， $(v - s)(n) = v(n) - s(n) > q$ ¹。注意显然 i：对任意 i, j ， $s(i)_{C_{\mathbb{Q}}}(j) = s(i)$ 。则对 $n > \max\{k, m'\}$ ， $v(n) - s(n)_{C_{\mathbb{Q}}}(n) > q$ 。

取 s 的收敛测试 c ，由其定义有对任意 $n, m' > c(\frac{q}{2})$ ， $|s(n) - s(m')| < \frac{q}{2}$ ，即 $-\frac{q}{2} < s(n) - s(m')$ 。令 $m'' = \max\{k, m', c(\frac{q}{2})\}$ ，则对任意 $n, m > m''$ ，结合 i 有：

$$\begin{aligned} v(n) - s(m)_{C_{\mathbb{Q}}}(n) &= v(n) - s(n)_{C_{\mathbb{Q}}}(n) + s(n)_{C_{\mathbb{Q}}}(n) - s(m)_{C_{\mathbb{Q}}}(n) = v(n) - s(n)_{C_{\mathbb{Q}}}(n) + s(n) - s(m) \\ &> q - \frac{q}{2} = \frac{q}{2} \end{aligned}$$

即对任意 $n, m > m''$ ， $(v - s(m)_{C_{\mathbb{Q}}})(n) > \frac{q}{2}$ 。则由定义 $v - s(m)_{C_{\mathbb{Q}}} > 0$ ，则 $r - s(m)_{\mathbb{R}} > 0$ ， $s(m)_{\mathbb{R}} < r$ 。因 $m > m'$ ，其与引理前提矛盾。□

引理 4.2.21 $A_Z \vdash m' \in \omega \wedge s \in C_{\mathbb{Q}} \wedge r \in \mathbb{R} \wedge \forall n > m' s(n)_{\mathbb{R}} \leq r \rightarrow s_{\mathbb{R}} \leq r$ 。

证明：同上。□

引理 4.2.22 $A_Z \vdash B_{\mathbb{R}}(t) \rightarrow \forall n \in \omega \exists! j \in \mathbb{Z} \overbrace{(\forall x \in t x < (j / 2^n)_{\mathbb{R}}) \wedge \exists y \in t y \geq ((j - 1) / 2^n)_{\mathbb{R}}}$ 。

证明：令实数 u 为 t 的上界，柯西序列 $v \in u$ 。由引理 4.2.19 u 有界 q ，则有正整数界 z 。有对任意自然数 n ， $v(n) < z$ ，则 $v(n)_{\mathbb{R}} < z_{\mathbb{R}}$ 。应用引理 4.2.21 有 $u \leq z_{\mathbb{R}}$ 。而 $z < (2^n \cdot z + 1) / 2^n$ ，则存在整数 x 满足 $u < (x / 2^n)_{\mathbb{R}}$ ，即对任意 $x \in t$ ， $x < (x / 2^n)_{\mathbb{R}}$ 。而类似的，对任意 $y \in t$ ，由 y 有界应用引理 4.2.20 存在整数 w 满足 $y \geq (w / 2^n)_{\mathbb{R}}$ ，显然 $w < k$ 。而 w 的存在意味着自 k 递减可以找到最小（因而唯一的）整数 j 满足任意 $x \in t$ 有 $x < (j / 2^n)_{\mathbb{R}}$ 且（因 t 非空）存在 $y \in t$ 有 $y \geq ((j - 1) / 2^n)_{\mathbb{R}}$ 。□

写 $J_t(n)$ 等于上述（有关 ζ 的） j ，即 $P(J_t(n))$ 表示 $\forall j \in \mathbb{Z} (\zeta \rightarrow P(j))$ 。

引理 4.2.23 $A_Z \vdash B_{\mathbb{R}}(t) \rightarrow isCauchySequence_{\mathbb{Q}}(\overbrace{\{\langle n, j \rangle \in \omega \times \mathbb{Z} : j = J_t(n)\}}^{\xi})$ 。

证明：令 $s = \{\langle n, j \rangle \in \omega \times \mathbb{Z} : j = J_t(n)\}$ ，可见 $s : \omega \mapsto \mathbb{Q}$ 。我们证明以下 c 为其收敛测试：

$$c : \mathbb{Q}_{>0} \mapsto \omega = \{ \langle \varepsilon, k \rangle \in \mathbb{Q}_{>0} \times \omega : k = \min\{w \in \omega \setminus 0 : \frac{1}{2^w} < \varepsilon\} \}$$

对任意 $\langle \varepsilon, k \rangle \in c$ ，任意 $a \geq b \geq k$ ，有对任意 $x \in t$ ， $x < J_t(b) / 2^b = (2^{a-b} J_t(b)) / 2^a$ ²，也有 $x < J_t(a) / 2^a$ 且 $J_t(a)$ 为分母为 2^a 下的满足要求的最小整数。有 $J_t(a) \leq 2^{a-b} J_t(b)$ 。

结合存在 $y \geq (J_t(b) - 1) / 2^b = (2^{a-b} J_t(b) - 2^{a-b}) / 2^a$ ，并且 $y < J_t(a) / 2^a$ ，有 $J_t(a) \geq 2^{a-b} J_t(b) - 2^{a-b}$ 。则 $0 \leq 2^{a-b} J_t(b) - J_t(a) \leq 2^{a-b}$ 。同除 2^a ，有：

$$0 \leq \frac{J_t(b)}{2^b} - \frac{J_t(a)}{2^a} \leq \frac{1}{2^b} \quad (4.2.5)$$

即 $|s(b) - s(a)| \leq 1/2^b \leq 1/2^k < \varepsilon$ 。□

写 S_t 表示上述 ξ 。

引理 4.2.24 $A_Z \vdash B_{\mathbb{R}}(t) \rightarrow isBound_{\mathbb{R}}(S_t, t)$ 。

证明：由定义对任意 $x \in t$ ，任意自然数 n ， $x < J_t(n) = S_t(n)$ 。则应用引理 4.2.20， $S_t \geq x$ 。□

¹第一个减号对应柯西序列上的减法，第二个对应有理数。

²可见这里省略了转换下标 $\mathbb{R}$ 。

可见通过证明 S_t 小于等于 t 的任意上界, 可证明实数完全性定理 4.2.17, 为此还需:

引理 4.2.25 $A_Z \vdash B_{\mathbb{R}}(t) \rightarrow \forall m \in \omega S_{t\mathbb{R}} \leq (J_t(m) / 2^m)_{\mathbb{R}}$ 。

证明: 对任意 m , 考虑任意 $n \geq m$, 由 (4.2.5) 有 $S_t(n) = J_t(n) / 2^n \leq J_t(m) / 2^m$ 。即 $S_{t\mathbb{R}}(n) \leq (J_t(m) / 2^m)_{\mathbb{R}}$ 。应用引理 4.2.21, $S_{t\mathbb{R}} \leq (J_t(m) / 2^m)_{\mathbb{R}}$ 。□

引理 4.2.26 $A_Z \vdash x \in \mathbb{R} \wedge y \in \mathbb{R} \wedge x > y \rightarrow \exists m \in \omega x > y + (1/2^m)_{\mathbb{R}}$ 。

证明: 令 $u \in x$, $v \in y$, 则 $u - v > 0$, 则存在自然数 k 、正有理数 q , 对任意 $n > k$, $(u - v)(n) > q$ 。取自然数 m 有 $1/2^m < q$ ¹, 则 $u - v - (1/2^m)_{\mathbb{C}_Q} > 0$, 则 $x > y + (1/2^m)_{\mathbb{R}}$ 。□

定理 4.2.17 的证明: 引理 4.2.24 证明了 $r = S_{t\mathbb{R}}$ 为 t 的上界。则仅需证明任意实数 $p < S_{t\mathbb{R}}$, p 不为 t 的上界。应用引理 4.2.26 存在自然数 n 有 $p < r - (1/2^n)_{\mathbb{R}}$ 。由引理 4.2.25, $r \leq (J_t(n) / 2^n)_{\mathbb{R}}$ 。则:

$$p < r - (1/2^n)_{\mathbb{R}} \leq (J_t(n) / 2^n)_{\mathbb{R}} - (1/2^n)_{\mathbb{R}} = ((J_t(n) - 1) / 2^n)_{\mathbb{R}}$$

但由 $J_t(n)$ 的定义, 存在 $y \in t$ 有 $y > ((J_t(n) - 1) / 2^n)_{\mathbb{R}}$, 因此 p 不为 t 的上界。□

下面我们来看实数完全性的一个应用。我们首先定义:

- 对任意 $f : d \mapsto \mathbb{R}$ 有 $d \subseteq \mathbb{R}$ 、任意 $x \in d$, 定义 $isContinuousAt(f, x)$ 为:

$$\forall \varepsilon \in \mathbb{R}_{>0} \exists \delta \in \mathbb{R}_{>0} \forall y \in dom f (|x - y| < \delta \rightarrow |f(x) - f(y)| < \varepsilon) \quad (4.2.6)$$

其中 $\mathbb{R}_{>0}$ 为正实数。即对任意值差界 ε , 存在 (定义) 域差界 δ 有对任意 y 与 x 差小于 δ 时其值差小于 ε 。称 f 在 x 连续。写 $isContinuous(f)$ 表示 $\forall x \in dom f isContinuousAt(f, x)$, 称 f 连续。

有:

定理 4.2.27 (中值定理)

$$A_Z \vdash a \in \mathbb{R} \wedge b \in \mathbb{R} \wedge a < b \wedge f : [a, b] \mapsto \mathbb{R} \\ \wedge isContinuous(f) \wedge f(a) < 0 \wedge f(b) > 0 \rightarrow \exists c \in dom f f(c) = 0$$

其中 $[a, b]$ 表示 $\{x \in \mathbb{R} : a \leq x \wedge x \leq b\}$ 。

证明²: 对集合 $t = \{x \in [a, b] : f(x) \leq 0\}$, 因 $a \in t$, $t \neq \emptyset$, 而 b 为 t 上界, 则可应用实数完全性定理 4.2.17 有 c 为 t 的最小上界。显然 $c \in dom f$, 我们证明 $f(c) = 0$ 。由 f 连续, 对任意 ε , 存在 δ 满足对任意 $|y - c| < \delta$ 有 $|f(y) - f(c)| < \varepsilon$ 。即对任意 $c - \delta < y < c + \delta$, $f(y) - \varepsilon < f(c) < f(y) + \varepsilon$ 。因 c 为上界, 存在 $u \in t$ 有 $c - \delta < u \leq c$, 则 $f(c) < f(u) + \varepsilon \leq \varepsilon$ 。同时, 若 $c = b$, $f(c) > 0 > -\varepsilon$, 否则取 $v < b$ 且 $c < v < c + \delta$, 则 $f(c) > f(v) - \varepsilon > -\varepsilon$ 。则 $-\varepsilon < f(c) < \varepsilon$, 因 ε 为任意正实数, $f(c) \neq 0$ 且 $f(c) \neq 0$, $f(c) = 0$ 。□

我们将在章节 5.4.3 通过非标分析再次证明该定理以示区别。

¹如 $q = a / b$, 取 $m = 2^b$ 。

²参考 Wikipedia contributors. Intermediate value theorem — Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/w/index.php?title=Intermediate_value_theorem&oldid=1103996523. 2022。

4.3 集合结构

下面我们回到更特定于集合论范畴的讨论。尽管其更特定于集合论本身，但仍应注意其为上述适用于更一般数学讨论的自然扩展。同时也将看到这种自然扩展进一步引出的复杂度。并为下一章关于集合论元逻辑的讨论提供背景。

4.3.1 序数和替换公理模式

通过基数我们可以以类似配对的方式衡量集合的大小。而当我们总是能从集合中选出下一个元素时，我们可以通过点数 (shǔ) 的方式衡量集合（在该选择顺序下）的大小，我们称该方式为通过序数衡量集合的大小。可见我们总是可以用序数衡量具有（严格）良序关系 r 的集合 a 在 r 下的大小，因由良序关系可以得到该集合（若有）最小的元素 $x_1 = \min_r a$ 、次小的元素 $x_2 = \min_r (a \setminus x_1)$ ，并在任意剩余 a 的非空子集 b 时选出 $x = \min_r b$ 。

对任意 a ，令 $<_a = \{ \langle x, y \rangle \in a^2 : x \in y \}$ ，并按引理 4.2.12 方式定义 $\leq_a$ 。可证明任意 $b \subseteq \omega$ 有 $\leq_b$ 为其良序关系。为此，我们首先定义关系的限制，直观的说，即关系的定理域、值域经其它集合限制后得到的新的关系。具体的，可定义：

- 限制 $r \upharpoonright a$ 为 $\{ \langle x, y \rangle \in r : x \in a \}$ 。及 $r \upharpoonright a \times b$ 为 $r \cap a \times b$ 。
- 对任意关系 r ，任意 a ，写 $r \llbracket a \rrbracket$ 表示 $\text{ran}(r \upharpoonright a)$ ，即 r 定义域被 a 限制后的值域。

则：

引理 4.3.1 $A_Z \vdash \text{isWellOrder}(a, r) \wedge b \subseteq a \rightarrow \text{isWellOrder}(b, r \upharpoonright b^2)$ 。

证明：可见 $\text{isTotalOrder}(b, r \upharpoonright b^2)$ ，且 b 的任意非空子集有 $r \upharpoonright b^2$ 下的最小值。 $\square$

推论 4.3.1.1 $A_Z \vdash \forall b \subseteq \omega \text{ isWellOrder}(b, \leq_b)$ 。

证明：由外延公理有 $\leq_b = \leq_\omega \upharpoonright b^2$ ，结合推论 4.2.12.1 指出 $\text{isWellOrder}(\omega, \leq_\omega)$ 。 $\square$

结合对任意自然数 n 有 $n \subset \omega$ ¹，则其有良序关系 $<_n$ 。即直观的，其是可点数的，我们可以通过有限 n 次点数点数完 n 中的元素。类似的， ω 是可点数的，但其需要通过无限多次的点数才能点数完其中的元素。这是我们遇到的第一个这样的集合，事实上，我们有至少无限多个这样的集合。考虑 $\omega + 1 = \omega^+$ ，可见 $<_{\omega+1}$ 为其良序关系。类似有 $\omega + 2 = \omega^{++}$ 等等。直观的说，上述的小于关系令我们可以得到数轴：

$$0, 1, 2, \dots, \omega, \omega + 1, \omega + 2, \dots$$

然而，尽管对任意自然数 n ，我们可构造得到 $\omega + n$ ，但这种方法却不能使我们得到 $\omega \cdot 2 = \omega + \omega$ 。这里我们需要替换公理模式。具体的，类似将 ω 定义为最小归纳集，我们定义 $\omega \cdot 2$ 为还含有 ω 的最小归纳集。

示例 4.3.2 $A_Z \cup \text{Rep} \vdash \exists a \ a = \omega \cdot 2$ 。

¹见定义式 (4.2.1)。

¹自 Chris Eagle. Existence of $\{\omega, \omega + 1, \dots\}$ via the axiom of replacement. Mathematics Stack Exchange. URL: <https://math.stackexchange.com/q/305583>。

证明¹：取替换公理模式中 $\rho_{[x,y]}$ 表示 $y = \omega + x$ ($x \in \omega$)，则可证明存在 b 有 $\forall y (y \in b \leftrightarrow \exists x \in \omega y = \omega + x)$ ，则 $\omega \cdot 2 = \omega \cup b$ 。

现我们需要给出 ρ 。首先注意我们还没有 $y = \omega + x$ 为函数，因我们需要证明的即是其值域 b 存在。即尽管对任意自然数 n ，我们可以构造有序对 $\langle n, \omega + n \rangle$ ，但还并没有这些有序对的合集为集合。但我们可以取这些有序对的任意有限长的起始段为函数，即对任意自然数 n ， $f = \{\langle 0, \omega \rangle, \dots, \langle n, \omega + n \rangle\}$ 为函数，因由 A_Z 可以证明存在集合 $\{\omega, \dots, \omega + n\}$ 。即有 $A_Z \vdash \forall n \in \omega \exists f P(n, f)$ ，其中 $P(n, f)$ 为：

$$isFunction(f) \wedge n = dom f \wedge (0 \in n \rightarrow f(0) = \omega) \wedge \forall m (m^+ \in n \rightarrow f(m^+) = f(m)^+)$$

则可定义 ρ 为 $\forall f P(x^+, f) \rightarrow f(x) = y$ 。

□

我们将在章节 5.1.2 中通过构造不含 $\omega \cdot 2$ 的模型证明仅通过 A_Z 不能证明 $\omega \cdot 2$ 的存在，则在这样的模型中，有序对 $\langle n, \omega + n \rangle$ ($n \in \omega$) 的合集不为集合。非正式的²，我们称该合集为函数类。或更一般的，我们称域中元素合集为类若其可被一阶逻辑式定义，如全部集合的合集可被式 $x = x$ 定义，上述有序对合集可通过上面的 ρ 定义。显然，存在类可以由集合表示，称不可被集合表示的类为真类。且所有可表示的集合都有对应类。

类似的，因对任意自然数 n ，有函数 $\{\langle 0, \wp^0(\emptyset) \rangle, \dots, \langle n, \wp^n(\emptyset) \rangle\}$ ，有 $\{\wp^0(\emptyset), \wp^1(\emptyset), \dots\}$ 为集合³。

扩展表示法，对任意集合 a 、定义函数类的式 $\rho_{[x,y]}$ ，一般写 $\{y : \exists x \in a \rho\}$ 表示 a 经 ρ 替换后得到的集合。

同时注意替换公理模式蕴含了分类公理模式。定义逻辑联结词 $?$ ：为 $\alpha ? \beta : \gamma$ 表示 $(\alpha \wedge \beta) \vee (\neg \alpha \wedge \gamma)$ ，即若 α 则 β 否则 γ 。为得到 $\exists b \forall x (x \in b \leftrightarrow x \in a \wedge \rho)$ ，若 a 中没有 x 满足 ρ ，则取 b 为 $\emptyset$ ，否则存在 x_0 满足 ρ ，则取 b 为 $\{y : \exists x \in a (\rho ? y = x : y = x_0)\}$ 。

4.3.2 良序和良基结构

称有序对 $\langle a, r \rangle$ 满足 $isStrictWellOrder(a, r)$ 为良序结构。即可定义：

- 良序结构的判定谓词 $WO(p)$ 为 $isOrderedPair(p) \wedge isStrictWellOrder(fst(p), snd(p))$ 。写 $WO(a, r)$ 表示 $WO(\langle a, r \rangle)$ 。

接着，我们讨论良序结构之间的异同。

进一步，定义：

- 对任意 $p = \langle a, r \rangle$ 、 $q = \langle b, s \rangle$ 有 r 为 a 、 s 为 b 上的关系，定义任意 $isIsomorphism(f, p, q)$ 为：

$$isBijective_{\mathbb{P}}(f, a, b) \wedge \forall x, y (x r y \leftrightarrow f(x) s f(y))$$

即 f 为 a 到 b 上的双射函数且其保持映射前后分别在 r 和 s 上的关系。定义同构判定谓词 $p \cong q$ 表示 $\exists f isIsomorphism(f, p, q)$ ^{注4}。称 f 为 $a b$ 上的同构。

²这里非正式的表示其不为 ZFC 集合论下可定义的概念。

³显然我们的构造仍依赖替换公理模式，该依赖不可避免的证明仍见章节 5.1.2。

⁴可见该定义是和 115 页注 1 中解释之间的同构定义一致的，因此在一些材料中也称解释为结构。

- 任意关系 r 中小于 t 的起始段 $seg_r t$ 为 $\{x \in dom r : x r t\}$, 起始结构 $Seg_r t$ 为 $\langle seg_r t, r \upharpoonright (seg_r t)^2 \rangle$ 。

对任意自然数 n , 定义逻辑联结词 $\underline{\vee}^n$ 有语句 $\underline{\vee}^n(\alpha_{1...n})$ 表示 $\alpha_{1...n}$ 中有且仅有一个为真。则如 $\underline{\vee}^2$ 即为异或。具体的, 定义 $\underline{\vee}^n(\alpha_{1...n})$ 为 $(\bigvee_{i \leq n} \alpha_i) \wedge \bigwedge_{i < j \leq n} \neg(\alpha_i \wedge \alpha_j)$ 。

我们将证明:

定理 4.3.3

$$A_Z \vdash WO(a, <_a) \wedge WO(b, <_b) \rightarrow \underline{\vee}^3(\langle a, <_a \rangle \cong \langle b, <_b \rangle, (\exists t Seg_{<_a} t \cong \langle b, <_b \rangle), (\exists t \langle a, <_a \rangle \cong Seg_{<_b} t))$$

即对任意良序结构 $\langle a, <_a \rangle \langle b, <_b \rangle$, 要么两者同构, 要么其一的某起始结构与另一同构。

我们首先介绍另一种应用于良序结构的归纳和递归方法, 之后通过其中的递归部分证明上述定理。

定理 4.3.4 (超限归纳原理 ^{WO}) $A_Z \vdash WO(a, r) \wedge \forall t (seg_r t \subseteq b \rightarrow t \in b) \rightarrow b = a$ 。

证明: 类似定理 4.2.13 (强归纳原理) 的证明。若 $\bar{p} = a - b \neq \emptyset$, 其有最小 t 。则任意 $x r t$ 有 $x \in b$, 即 $seg_r t \subseteq b$ 。但这蕴含 $t \in b$ 。矛盾。 $\square$

写:

- 任意 $^{seg_r} b$ 表示 $\{f \in \wp((dom r) \times b) : isFunction(f) \wedge \exists t dom f = seg_r t\}$, 即任意 $seg_r t$ 到 b 的函数集合。

定理 4.3.5 (超限递归定理 ^{WO})

$$A_Z \vdash WO(a, r) \wedge \overbrace{g : (a \times ^{seg_r} b) \mapsto b}^{\alpha} \rightarrow \overbrace{\exists! h isFunction(h) \wedge dom h = a \wedge \forall t \in a h(t) = g(t, h \upharpoonright seg_r t)}^{\beta}$$

可见其中 α 要求 g 为 $a \times ^{seg_r} b$ 到 b 的函数。 β 指出存在且唯一的 h 为由 $a r g$ 决定的超限递归函数, 其中最后一部分指出任意 $t \in a$ 的对应值 (可) 由 t 和 h 中前项 (即自变量) 小于 t 的全部 (自变量及其应变量构成的) 有序对、即 h 中到 t 的起始函数段决定, 因此其是递归的。可见 α 中对 g 的要求保证了这些有序对集合属于 g 的定义域。

证明: 类似定理 4.2.14 (递归定理) 的证明, 写 $P(v)$ 表示:

$$isFunction(v) \wedge dom v \subseteq a \wedge ran v \subseteq b \wedge \forall u \in dom v seg_r u \subseteq dom v \wedge v(u) = g(u, v \upharpoonright seg_r u)$$

可以看到 v 对应 h 中某一起始函数段。则构造 h 为:

$$\{ \langle x, y \rangle \in a \times b : \exists v P(v) \wedge y = v(x) \}$$

超限归纳有:

$$\forall t \in a (\forall v_1, v_2 (P(v_1) \wedge P(v_2) \wedge t \in dom v_1 \wedge t \in dom v_2 \rightarrow v_1(t) = v_2(t))) \wedge (\exists v P(v) \wedge t \in dom a)$$

可以得到 h 为函数且 $dom h = a$ 。 $\square$

定义:

- 任意 $isTransfiniteRecursive(h, a, r, g)$ 表示 $h a r g$ 满足上述定理中的 β 。
- 任意 $P(trec[a, r, g])$ 为 $\forall h (isTransfiniteRecursive(h, a, r, g) \rightarrow P(h))$ 。

定理 4.3.3 的证明¹：取任意 e 满足 $e \notin a \wedge e \notin b$ (e 存在因 $a \cup b$ 无法包含全部集合)。通过超限递归定理构造由 a 到 b ; e 的函数 h 为：

$$trec [a, <_a, \{ \langle x, f, y \rangle \in a \times^{seg_{<_a}} b \times b : b \neq ran f \wedge isMinimal(y, b - ran f, <_b) : y = e \}] \quad \text{注}^2$$

即对任意 $t \in a$ ，写 $b_{\geq}(t)$ 表示 $b - h[seg_{<_a} t]$ ，有 $h(t) = \begin{cases} \min b_{\geq}(t) & b_{\geq}(t) \neq \emptyset \\ e & \text{其它} \end{cases}$ 。则有以下三种情况：

1. 若 $e \in ran h$ 。令 $h' = \{ \langle x, y \rangle \in h : y \neq e \}$ 。取 $t = \min_{<_a} (a - dom h')$ 。可证明 $Seg_{<_a} t \cong \langle b, <_b \rangle$ 。令 $Seg_{<_a} t = \langle a', <_{a'} \rangle$ 。可见 $dom h = a' \wedge ran h' = b$ 。且对任意 $x y$ 满足 $x \leq_a y <_a t$ 注³ 有：

$$x \leq_a y \Rightarrow h[seg_{<_a} x] \subseteq h[seg_{<_a} y] \Rightarrow b_{\geq}(y) \subseteq b_{\geq}(x) \Rightarrow h(x) \leq_b h(y)$$

而对 $x <_a y <_a t$ ，因 $h(x) \in h[seg_{<_a} y]$ 但 $h(y) \notin h[seg_{<_a} y]$ 有 $h(x) \neq h(y)$ 。则：

$$x <'_a y \rightarrow h'(x) <_b h'(y)$$

则 h' 满足 $isBijective_{\mathbb{P}}(h', a', b)$ 。且：

$$h'(x) <_b h'(y) \Rightarrow h'(y) \not\leq_b h'(x) \Rightarrow y \not\leq'_a x \Rightarrow x <'_a y$$

则 $seg_{<_a} t \cong \langle b, <_b \rangle$ 。

2. 否则若 $ran h = b$ 。类似有 $\langle a, <_a \rangle \cong \langle b, <_b \rangle$ 。
3. 否则，取 $t = \min_{<_b} (b - ran h)$ 。由 t 在 $b - ran h$ 中最小有 $seg_{<_b} t \subseteq ran h$ 。同时对任意 $x \in a$ 有 $t \not\leq_b h(x)$ ，因 $h(x)$ 为 $b_{\geq}(x)$ 中最小的，而 $t \in b - ran h$ 蕴含 $t \in b_{\geq}(x)$ 。则 $ran h = seg_{<_b} t$ 。进而类似有 $\langle a, <_a \rangle \cong Seg_{<_b} t$ 。□

更一般的，我们不仅可以通过函数，也可以通过逻辑式给出递归定义：

定理 4.3.6 (超限递归定理^{WO}_{II}) 对任意 (可能含其它自由变量的) 式 $\gamma(x, f, y)$ 注⁴：

$$\begin{aligned} A_Z \cup Rep \vdash WO(a, r) \wedge \overbrace{\forall x \in a, f (isFunction(f) \rightarrow \exists! y \gamma(x, f, y))}^{\alpha} \\ \rightarrow \exists! h \overbrace{isFunction(h) \wedge dom h = a \wedge \forall t \in a \gamma(h \upharpoonright_{seg_r t}, h(t))}^{\beta} \end{aligned}$$

可见其中 α 部分指出对任意 $x \in a$ 、函数 f 有唯一 y 满足 γ ，即 γ 定义了可认为定义域为 a 与所有函数笛卡尔积的函数类。 β 指出 h 为由 $a r \gamma$ 决定的超限递归函数，有任意 $t \in a$ 的对应值 (可) 由 t 和 h 中前项小于 t 的全部有序对决定。

证明⁵：类似示例 4.3.2 的证明。定义 $\rho_{[t, v]}$ 为：

$$isFunction(v) \wedge dom v = seg_r t; t \wedge \forall u \in dom v \gamma(u, v \upharpoonright_{seg_r u}, v(u))$$

可以看到 v 对应 h 中到 t 的起始函数段。则通过替换公理模式可取到 h 为 $\bigcup \{ v : \exists t \in a \rho \}$ 。对 a 中元素做超限归纳可证明 h 满足要求。□

¹证明自 H.B. Enderton. Elements of Set Theory. Elsevier Science, 1977, pp. 189–191。

²见定理 4.3.6 (超限递归定理^{WO}_{II}) 以回顾递归步骤中 $\gamma(x, f, y)$ 的参数含义。

³ $x \leq_a y$ 表示 $x <_a y \vee x = y$ 。

⁴可见其构成了定理模式。

⁵参考 H.B. Enderton. Elements of Set Theory. Elsevier Science, 1977, pp. 179–182。

定义：

- 任意 $isTransfiniteRecursive_{\Pi}(h, a, r, \gamma)$ 表示 $h \ a \ r \ \gamma$ 满足上述定理中的 β 。
- 任意 $P(trec_{\Pi}[a, r, \gamma])$ 为 $\forall h (isTransfiniteRecursive_{\Pi}(h, a, r, \gamma) \rightarrow P(h))$ 。

进一步观察超限归纳。可以发现其证明并不要求结构 $\langle a, r \rangle$ 为良序的。其仅要求 a 的任意非空子集有 r 下最小的元素，我们称这样的结构为良基结构。或具体的，定义：

- 良基结构的判定谓词 $WF(p)$ 为 $isOrderedPair(p) \wedge P(fst(p), snd(p))$ 。其中 $P(a, r)$ 为：

$$r \subseteq a^2 \wedge \forall b \subseteq a (b \neq \emptyset \rightarrow \exists x isMinimal(x, b, r))$$

写 $WF(a, r)$ 表示 $WF(\langle a, r \rangle)$ 。

显然，良序结构为良基结构。

则可得一般化的：

定理 4.3.7 (超限归纳原理) $A_Z \vdash WF(a, r) \wedge \forall t (seg_r t \subseteq b \rightarrow t \in b) \rightarrow b = a$ 。

类似的：

定理 4.3.8 (超限递归定理)

$$A_Z \vdash WF(a, r) \wedge g : (a \times^{seg_r} b) \mapsto b \rightarrow \exists! h isFunction(h) \wedge dom h = a \wedge \forall t \in a h(t) = g(t, h \upharpoonright_{seg_r t})$$

定理 4.3.9 (超限递归定理 Π) 对任意 (可能含其它自由变量的) 式 $\gamma(x, f, y)$ ：

$$A_Z \cup Rep \vdash WF(a, r) \wedge \forall x \in a, f (isFunction(f) \rightarrow \exists! y \gamma(x, f, y)) \\ \rightarrow \exists! h isFunction(h) \wedge dom h = a \wedge \forall t \in a \gamma(h \upharpoonright_{seg_r t}, h(t))$$

除良序结构的超限归纳递归为上述归纳递归的一种特殊情况外。对树和 (有向无环) 图等结构归纳、递归处理也是上述归纳递归的一种应用。比如对切消定理中对推导树性质的归纳证明即是超限归纳，即当我们证明了若对任意推导树其任意子树具有切消性质意味着该树也具有切消性质时，我们可以得到任意树具有切消性质。同时我们也可进一步推广应用到无限大的树和图上。

4.3.3 序数定义¹

回到对良序结构的讨论。

良序结构间的同构性质意味着我们可以为每个良序结构赋予一个特定的良序类型并比较它们之间的大小。注意这里我们不能定义良序类型为所有良序结构在同构下的等价类，因通过下述定理 4.3.20 可知由全部良序结构构成的类不为集合。则我们 (需) 定义良序类型为：

- 序数。写 $h_{ord}[p]$ 或 $h_{ord}[a, r]$ 表示任意良序结构 $p = \langle a, r \rangle$ 决定的递归函数 $trec_{\Pi}[a, r, y = ran f]$ 。定义该结构 p 的序数 $ord p$ 或 $ord(a, r)$ 为 $ran h_{ord}[a, r]$ ²。

事实上：

¹本小节和下一小节证明部分参考 H.B. Enderton. Elements of Set Theory. Elsevier Science, 1977, pp. 182–194。

²注意这里需要 $trec_{\Pi}$ 因我们正在通过该定义方式保证其值域、即序数的存在。

示例 4.3.10

1. $\text{ord}(0, <_0) = \emptyset = 0$ 。
2. $\text{ord}(1, <_1) = \{\emptyset\} = 1$ 。
3. $\text{ord}(2, <_2) = \{\emptyset, \{\emptyset\}\} = 2$ 。
4. $\text{ord}(3, <_3) = \{\emptyset, \{\emptyset\}, \{\emptyset, \{\emptyset\}\}\} = 3$ 。
- ...

以及如前注所述, ω 为序数:

示例 4.3.11 $A_Z \vdash \text{ord}(\omega, <_\omega) = \omega$ 。

证明: 令 $h = h_{\text{ord}}[\omega, <_\omega]$, 由归纳原理有 $\{n \in \omega : h(n) = n\} = \omega$, 有 $\text{ran } h = \omega$ 。 $\square$

可以看到在我们的定义下, 序数是符合直觉的。有限序数即为自然数。其为通过点数集合中元素得到的集合大小。下面我们更一般的讨论包括良序结构的大小关系的序数性质。

引理 4.3.12 $A_Z \vdash WO(a, <) \rightarrow \forall x \in a \ h_{\text{ord}}[a, <](x) \notin h_{\text{ord}}[a, <](x)$ 。

证明: 令 $h = h_{\text{ord}}[a, <]$ 。若存在 x 有 $h(x) \in h(x)$, 则有最小 v 有 $h(v) \in h(v)$, 结合 $h(v)$ 的递归定义 $h(v) = h[\text{seg}_{<} v]$, 存在 $u < v$ 有 $h(u) = h(v)$, 则 $h(u) \in h(u)$, 与 v 最小矛盾。 $\square$

定理 4.3.13 $A_Z \vdash WO(p) \rightarrow p \cong \langle \text{ord } p, <_{\text{ord } p} \rangle$ 。

证明: 令 $\langle a, < \rangle = p$, $b = \text{ord } p$ ^{注1}。我们将证明 $h = h_{\text{ord}}[a, <]$ 满足 $\text{isIsomorphism}(h, p, \langle b, <_b \rangle)$ 、即需证明 $\text{isBijective}_{\mathbb{F}}(h, a, b)$ 且 $x < y \leftrightarrow h(x) <_b h(y)$ 。

由定义有 $b = \text{ran } h$ 。则双射函数仅需证明 $x \neq y \rightarrow h(x) \neq h(y)$ 。由上述引理, 对任意 $x \in a$ 有 $h(x) \notin h(x)$ 。而对任意 $x < y$, 结合 $h(y)$ 的定义有 $h(x) \in h(y)$, 结合 $h(y) \notin h(y)$ 有 $h(x) \neq h(y)$ 。即有 $x \neq y \rightarrow h(x) \neq h(y)$ 。进一步若 $h(x) \in h(y)$, 由 $<_b$ 的定义有 $h(x) <_b h(y)$, 即有 $x < y \rightarrow h(x) <_b h(y)$ 。反方向若 $h(x) <_b h(y)$, 则存在 $u < y$ 有 $h(x) = h(u) \in h(y)$, 则 $x = u$, 即 $h(x) <_b h(y) \rightarrow x < y$ 。 $\square$

推论 4.3.13.1 $A_Z \vdash WO(p) \rightarrow WO(\text{ord } p, <_{\text{ord } p})$ 。

同时:

引理 4.3.14 $A_Z \vdash WO(p) \rightarrow \text{isTransitiveSet}(\text{ord } p)$ 。

证明: 令 $h = h_{\text{ord}}[p]$, $u \in v \in \text{ord } p = \text{ran } h$ 蕴含存在 y 有 $u \in h(y) \in \text{ord } p$, 则存在 $x < y$ 有 $h(x) = u$, 则 $u \in \text{ord } p$ 。 $\square$

事实上可定义:

- 序数判定谓词 $\text{isOrd}(a)$ 为 $\text{isTransitiveSet}(a) \wedge WO(a, <_a)$ 。

则可定义序数为满足 isOrd 的序数类中元素。为证明这两种定义等价, 除由以上推论和引理可得一个方向上的 $A_Z \vdash WO(p) \rightarrow \text{isOrd}(\text{ord } p)$ 外, 还需证明另一方向:

引理 4.3.15 $A_Z \vdash \text{isOrd}(a) \rightarrow \exists p \ WO(p) \wedge \text{ord } p = a$ 。

¹一般序数由小写希腊字母表示, 但为和其它部分尽可能一致, 本书使用 a, b, c 等。

证明：我们证明 $\text{ord}(a, <_a) = a$ 。令 $h = h_{\text{ord}}[a, <_a]$ 。将通过超限归纳原理证明 $b = \{x \in a : h(x) = x\} = a$ ，则 $\text{ran } h = a$ ，即 $\text{ord}(a, <_a) = a$ 。结合对任意 $t \in a$ 有 $\text{seg}_{<_a} t = t$ ，因由 a 为传递集，任意 $x \in t$ 有 $x \in a$ 有 $x <_a t$ 。且由 $<_a$ 的定义有 $x <_a t$ 有 $x \in t$ 。假设 $\text{seg}_{<_a} t \subseteq b$ ，即对任意 $x \in \text{seg}_{<_a} t$ 有 $h(x) = x$ ，则：

$$h(t) = h[\llbracket \text{seg}_{<_a} t \rrbracket] = \text{seg}_{<_a} t = t \quad \square$$

下面考虑良序结构的大小关系：

引理 4.3.16 $A_Z \vdash \text{WO}(p) \wedge \text{WO}(q) \wedge p \cong q \rightarrow \text{ord } p = \text{ord } q$ 。

证明：令 $\langle a, <_a \rangle = p$ ， $\langle b, <_b \rangle = q$ ， f 满足 $\text{isIsomorphism}(f, p, q)$ ， $g = h_{\text{ord}}[a, <_a]$ ， $h = h_{\text{ord}}[b, <_b]$ 。通过超限归纳我们证明 $c = \{x \in a : g(x) = h(f(x))\} = a$ ，则由 f 为 a 到 b 上的双射函数有 $\text{ran } g = \text{ran } h$ 。结合对任意 $t \in a$ 由外延公理有 $\{y \in \text{ran } h : \exists x y = f(x) \wedge x <_a t\} = h[\llbracket \text{seg}_{<_b} f(t) \rrbracket]$ 。假设 $\text{seg}_{<_a} t \subseteq c$ ，则：

$$g(t) = g[\llbracket \text{seg}_{<_a} t \rrbracket] = \{y \in \text{ran } h : \exists x y = f(x) \wedge x <_a t\} = h(f(t)) \quad \square$$

引理 4.3.17 $A_Z \vdash \text{WO}(p) \wedge \text{WO}(q) \wedge \exists t p \cong \text{Seg}_{\text{snd}(q)} t \rightarrow \text{ord } p <_{\text{ord } q} \text{ord } q$ 。

证明：令 $< = \text{snd}(q)$ 。由同构有 $\text{WO}(\text{Seg}_{<} t)$ 。令 $g = h_{\text{ord}}[\text{Seg}_{<} t]$ ， $h = h_{\text{ord}}[q]$ 。由超限归纳可见对任意 $x \in \text{seg}_{<} t$ ， $g(x) = h(x)$ 。则 $\text{ran } g = h(t) \in \text{ran } h$ 。令 $u = \text{ord } \text{Seg}_{<} t$ ， $v = \text{ord } q$ ，则 $u \in v$ ，即 $u <_v v$ 。结合由引理 4.3.16 有 $\text{ord } p = u$ ，有 $\text{ord } p <_{\text{ord } q} \text{ord } q$ 。 $\square$

我们可以得到关于序数顺序的以下性质：

定理 4.3.18

- a. $A_Z \vdash \text{isOrd}(a) \rightarrow a \notin a$ 。
- b. $A_Z \vdash \text{isOrd}(a) \wedge \text{isOrd}(b) \wedge \text{isOrd}(c) \wedge a \in b \wedge b \in c \rightarrow a \in c$ 。
- c. $A_Z \vdash \text{isOrd}(a) \wedge \text{isOrd}(b) \rightarrow \underline{\vee}^3(a \in b, a = b, b \in a)$ 。

证明：

- a. 由引理 4.3.15 证明有 $a = \text{ord}(a, <_a)$ 。令 $h = h_{\text{ord}}[a, <_a]$ 。假设 $a \in a$ ， $a \in \text{ran } h$ 有存在 $x \in a$ 有 $h(x) = a$ ，则 $h(x) \in h(x)$ 。与引理 4.3.12 矛盾。
- b. 由 c 为归纳集可得。
- c. 由定理 4.3.3 及引理 4.3.16 及 4.3.17 可得。 $\square$

由此可见，（不严格的） $\in$ 为序数类上的严格全序关系。

接着，我们给出更多关于序数的性质。并证明序数类为真类，即所有序数构成的类不为集合。

引理 4.3.19

- a. $A_Z \vdash \text{isOrd}(a) \rightarrow \forall t \in a \text{ isOrd}(t)$ 。
- b. $A_Z \vdash s \neq \emptyset \wedge \forall a \in s \text{ isOrd}(a) \rightarrow \exists u \text{ isMinimal}(u, s, <_s)$ 。即任意由序数构成的非空集合 s 有最小序数 u 。
- c. $A_Z \vdash \text{isTransitiveSet}(s) \wedge \forall a \in s \text{ isOrd}(a) \rightarrow \text{isOrd}(s)$ 。即任意由序数构成的传递集 s 为序数。

证明：

- a. 令 $p = \langle a, <_a \rangle$ 。类似有 $a = \text{ord } p$ 。令 $h = h_{\text{ord}}[p]$ ，由 $t \in a$ 有某 $u \in a$ 有 $h(u) = t$ ， $h(u) = h[\llbracket \text{seg}_{<_a} u \rrbracket]$ 。而 $\text{WO}(\text{Seg}_{<_a} u)$ 因可见：

$$A_Z \vdash \text{WO}(b, <) \rightarrow \forall v \in b \text{ WO}(\text{Seg}_{b, <} v)$$

令 $g = h_{\text{ord}}[\text{Seg}_{<_a} u]$ ，则由超限递归有任意 $x \in \text{seg}_{<_a} u$ ， $g(x) = h(x)$ 。则 $h(u) = \text{ran } g$ ，即 $\text{isOrd}(t)$ 。

- b. 考虑任意 $b \in s$, 若 $b \cap s = \emptyset$, 则任意 $a \in s$ 有 $a \notin b$, 由定理 4.3.18 c 有 $b \subseteq a$ ^{注1}, 则 $u = b = \min_{<_s} s$ 。否则令 $c = b \cap s \neq \emptyset$, 由 $c \subseteq b$ 有存在 $u = \min_{<_b} c$, 对任意 $a \in s$, 若 $a \in c$, $u \subseteq a$, 否则 $a \notin b$ 则 $b \subseteq a$, 结合 $u \in b$ 有 $u \in a$ 。有 $isMinimal(u, s, <_s)$ 。
- c. 由定理 4.3.18 有 $isStrictTotalOrder(<_s)$ 。由 b 有 $\forall t t \neq \emptyset \wedge t \subseteq s \rightarrow \exists u isMinimal(u, t, <_s)$ 。则由定义有 $isOrd(s)$ 。□

定理 4.3.20 (布拉利-福尔蒂定理) $A_Z \vdash \neg \exists s \forall a (a \in s \leftrightarrow isOrd(a))$ 。即序数类为真类。

证明：若序数类集合 s , 由以上引理 a 有 $isTransitiveSet(s)$ 。由以上 c 有 $isOrd(s)$, 则 $s \in s$, 与定理 4.3.18 a 矛盾。□

我们最后再给出另一个这样定义序数带来的有用性质：

定理 4.3.21 $A_Z \vdash isOrd(a) \wedge isOrd(b) \rightarrow (a \in b \leftrightarrow a \subset b)$ 。

证明：正方向由 $isTransitiveSet(a)$ 有 $a \subseteq b$ 但由定理 4.3.18 c 有 $a \neq b$, 则 $a \subset b$ 。反方向 $b - a \neq \emptyset$, 则由引理 4.3.19 b 有存在 $c = \min_{<_b} (b - a)$, 我们通过外延公理证明 $a = c$, 考虑任意 $x \in b$ 。若 $x \notin c$, 同样由 4.3.18 c 有 $c \subseteq x$, 假设 $x \in a$, $c \in a$, 矛盾。则 $x \in a \rightarrow x \in c$ 。若 $x \notin a$, $c \subseteq x$, $x \notin c$ 有 $x \in c \rightarrow x \in a$ 。□

这样, 我们首先以通过良序结构构造的方式定义了序数, 以保证我们定义的序数存在。接着, 我们引入了与之等价的更关注序数性质的通过序数判定谓词给出的序数定义。下面, 我们讨论更直观的类似通过递归定义得到的序数定义, 并在该基础上定义序数运算。

4.3.4 序数分类

注意到：

引理 4.3.22 $A_Z \vdash isOrd(a) \rightarrow isOrd(a^+)$ 。

证明：易见 $isTransitiveSet(a) \rightarrow isTransitiveSet(a^+)$ 。结合引理 4.3.19 a 有 $\forall t \in a^+ isOrd(t)$ 。则由 4.3.19 c 有 $isOrd(a^+)$ 。□

则以此我们可以把序数分为 0、后继序数和极限序数三类。具体要求：

- 任意后继序数 a 需满足 $isSuccessorOrdinal(a)$ 为 $\exists b isOrd(b) \wedge a = b^+$ 。
- 任意极限序数 a 需满足 $isLimitOrdinal(a)$ 为 $a \neq 0 \wedge \neg isSuccessorOrdinal(a)$ 。

引理 4.3.23 $A_Z \vdash isLimitOrdinal(\omega)$ 。

证明：显然 $\omega \neq 0$ 。假设存在 b 有 $isOrd(b) \wedge b^+ = \omega$ 。若 $b \in \omega$, 由 $isInductive(\omega)$, $b^+ \in \omega$, 则 $\omega \in \omega$, 与定理 4.3.18 a 矛盾。否则 $\omega \subseteq b \in b^+ = \omega$, 同样矛盾。□

我们即证明了存在极限序数。而如下示例 4.3.25 所示, 可认为 ω 为自然数趋向极限后的结果, 则可见该命名的原因。

引理 4.3.24 $A_Z \vdash isOrd(a) \rightarrow \neg \exists u a \in u \wedge u \in a^+$ 。

¹ $x \subseteq y$ 表示 $x \in y \vee x = y$ 。

证明：由 $isOrd(a)$ 有 $a \notin a$ 及 $isOrd(a^+)$ ，即 $a^+ \notin a^+$ ，而 $a^+ - a = \{a\}$ 。 $\square$

示例 4.3.25 $A_Z \vdash \forall b (isLimitOrdinal(b) \rightarrow \omega \in b)$ 。即 ω 为最小极限序数。

证明：假设存在 b 满足 $isLimitOrdinal(b) \wedge b \in \omega$ 。显然 $\emptyset \in b$ 。对任意 $a \in b$ ，由上一引理 $b \notin a^+$ ，结合 $isOrd(a^+)$ 有 $a^+ \in b$ ，但进一步 $isLimitOrdinal(b)$ 有 $a^+ \neq b$ ，则 $a^+ \in b$ 。我们有 $isInductive(b)$ 。由定理 4.3.21 有 $b \in \omega \rightarrow b \subset \omega$ ，与 ω 为最小归纳集矛盾。 $\square$

也可以看到，如前注所述，因小于 ω 的序数仅可以为 0 或某后继序数的后继， ω 为首个无限序数。

同时，在这样的分类下，我们有对序数的超限归纳。首先一并给出几个涉及处理序数子类的谓词定义：

- 对任意 $\rho_{[x]}$ ，定义由 ρ 确定的序数子类的最小元素判定谓词 $isLeast_{O_x}(a, \rho)$ 为：

$$isOrd(a) \wedge \rho_{[x/a]} \wedge \forall b (isOrd(b) \wedge \rho_{[x/b]} \rightarrow a \in b)$$

- 对任意 $\rho_{[x]}$ ，定义由 ρ 确定的序数子类的最大元素判定谓词 $isGreatest_{O_x}(a, \rho)$ 为：

$$isOrd(a) \wedge \rho_{[x/a]} \wedge \forall b (isOrd(b) \wedge \rho_{[x/b]} \rightarrow b \in a)$$

- 任意 $\rho_{[y]}$ 确定的序数子类最小上界判定谓词 $isSup_{O_y}(a, \rho)$ ，其中上界指任意大于等于该子类中任意序数的序数¹：

$$isLeast_{O_x}(a, \forall b (isOrd(b) \wedge \rho_{[y/b]} \rightarrow b \in x))$$

- 任意 $\rho_{[y]}$ 确定的序数子类最大下界判定谓词 $isInf_{O_y}(a, \rho)$ ，其中下界指任意小于等于该子类中任意序数的序数：

$$isGreatest_{O_x}(a, \forall b (isOrd(b) \wedge \rho_{[y/b]} \rightarrow x \in b))$$

- 写 $P(least_{O_x}(\rho))$ 表示 $\forall a (isLeast_{O_x}(a, \rho) \rightarrow P(a))$ 。等。

则²：

定理 4.3.26 (序数归纳原理) 对任意 $\rho_{[x]}$ ：

$$A_Z \vdash \rho_{[x/0]} \tag{i}$$

$$\wedge \forall a (isOrd(a) \wedge \rho_{[x/a]} \rightarrow \rho_{[x/a^+]}) \tag{ii}$$

$$\wedge \forall a (isLimitOrdinal(a) \wedge \forall b \in a \rho_{[x/b]} \rightarrow \rho_{[x/a]}) \tag{iii}$$

$$\rightarrow \forall a (isOrd(a) \rightarrow \rho_{[x/a]})$$

可见条件 (i) (ii) (iii) 分别对应序数的三种类别。

证明：若存在序数 u 有 $\neg \rho_{[x/u]}$ ，令 $s = \{x \in u^+ : \neg \rho\}$ ，存在 $v = \min_{<_s} s = least_{O_x}(\neg \rho)$ ，则 v 不能为 0 或后续序数或极限序数。矛盾。 $\square$

事实上同理我们有更强的序数归纳原理：

定理 4.3.27 对任意 $\rho_{[x]}$ ， $A_Z \vdash \forall a (isOrd(a) \wedge \forall b \in a \rho_{[x/b]} \rightarrow \rho_{[x/a]}) \rightarrow \forall a (isOrd(a) \rightarrow \rho_{[x/a]})$ 。

¹可见同章节 4.2.6 中 $isSup_{\mathbb{R}}$ 定义。

²对比参考定理 4.2.13 (强归纳原理) 和 4.3.4 (超限归纳原理)。

接着考虑序数、序数并集与边界的关系：

定理 4.3.28

- a. $A_Z \vdash \text{isOrd}(a) \rightarrow \text{isInf}_{O_Y}(a^+, a \in y)$ 。即 a^+ 为大于 a 的序数的最大下界。
- b. $A_Z \vdash \text{isOrd}(a) \rightarrow a = \bigcup a^+ \wedge \text{isSup}_{O_Y}(a, y \in a^+)$ 。
- c. $A_Z \vdash \text{isLimitOrdinal}(a) \rightarrow a = \bigcup a \wedge \text{isSup}_{O_Y}(a, y \in a)$ 。
- d. $A_Z \vdash \text{isOrd}(a) \rightarrow \text{isOrd}(\bigcup a) \wedge \text{isSup}_{O_Y}(\bigcup a, y \in a)$ 。

证明：

- a. 由引理 4.3.24 有不存序数 y 满足 $a \in y \in a^+$ 。则 $a \in y$ 有 $a^+ \subseteq y$, a^+ 为下界。同时不存在 z 有 $a^+ \in z$ 有 $z \subseteq a^+$, 因此最大。
- b. 由引理 4.2.3, $a = \bigcup a^+$ 。任意 $y \in a^+$ 有 $y \subseteq a$, a 为上界。不存在 $z \in a$ 有 $a \subseteq z$, 因此最小。
- c. 对任意 $b \in a$, $a \notin b^+$, 则 $b^+ \in a$, 则 $b \in \bigcup a$ 有 $a \subseteq \bigcup a$ 。由 $\text{isInductive}(a)$ 有 $\bigcup a \subseteq a$ 。有 $a = \bigcup a$ 。对后半部分, a 显然是任意 $y \in a$ 的上界。若存在 $z \in a$ 为任意 $y \in a$ 的上界, 则 $z^+ \in a$, 但 $z^+ \notin z$, 矛盾, 因此最小。
- d. 显然对 0 成立¹, 由 b 有对后继序数成立, 由 c 有对极限序数成立。□

更一般的：

定理 4.3.29 $A_Z \vdash \forall a \in s \text{isOrd}(a) \rightarrow \text{isOrd}(\bigcup s) \wedge \text{isSup}_{O_Y}(\bigcup s, y \in s)$ 。

证明：令 $b = \bigcup s$, 任意 $u \in b$, 存在 v 有 $u \in v \in s$, 由 $\text{isOrd}(v)$ 有 $x \in u \Rightarrow x \in v \Rightarrow x \in b$, 则 $\text{isTransitiveSet}(b)$ 。同时任意 $u \in b$ 有 $\text{isOrd}(u)$, 则由引理 4.3.19 a 有 $\text{isOrd}(b)$ 。对后半部分, 任意 $v \in s$, $v \subseteq b$, 由定理 4.3.21, $v \subseteq b$, b 为上界。若存在 $m \in b$ 为 s 的上界, 存在 $m \in n \in s$, 与 m 为上界矛盾。因此最小。□

4.3.5 序数运算²

下面我们定义序数运算。直观的, 可递归定义序数加法满足对任意序数 a, b :

$$a + b = \begin{cases} a & \text{若 } b = 0 \\ (a + c)^+ & \text{若 } b = c + \\ \bigcup \{y : \exists x \in b \ y = a + x\} & \text{其它} \end{cases}$$

可见分别对应序数的三种分类。注意由于全部序数不构成集合, 这里的 $+$ 不为函数。但其构成函数类, 令该类由 $\rho[x]$ 等于 $\exists a, b, d \ x = \langle a, b, c \rangle \wedge \bigwedge_{u \in \{a, b, d\}} \text{isOrd}(u) \wedge \theta_a$ 确定, 下面对任意序数 a 构造 $\theta_a[b, d]$ 可见满足以上要求且对任意 b 有唯一的序数 d 满足 θ_a ³。

采用类似示例 4.3.2 和定理 4.3.6 (超限递归定理) 证明中的方法。对任意 a , 由超限递归可以证明对任意序数 t , 存在函数 $+_{a,t} = \text{trec}_{\Pi}[t, <, \gamma_a]$, 其中 $\gamma_a(x, f, y)$ 为:

$$x = 0 ? y = a : (\text{isSuccessorOrdinal}(x) ? y = f(\bigcup x)^+ : y = \bigcup \text{ran } f)$$

¹一些材料定义 0 也为极限序数, 因其符合极限序数的一些性质, 如可见 0 满足 c 中 $0 = \bigcup 0 = \text{sup}_{O_Y}(0, y \in 0)$ 。但在接下去的序数运算中我们将看到其与极限序数的不同。

²本小节证明部分参考 Thomas Jech. Set Theory: The Third Millennium Edition. Springer, 2003, pp. 23–24。

³对比参考章节 4.2.4 中自然数上的加法函数 $+_{\omega}$ 的定义。

则可定义 $\theta_a[b, d]$ 为 $+_{a, b^+}(b) = d$ 。可见对任意序数 $u \in v$, $+_{a, u} \subset +_{a, v}$ 。且令 $h = +_{a, b^+}$, 对 $b = 0$ 有 $h(b) = a$ 。对任意后继序数 $b = c^+$ 有 $h[\text{seg}_{<_{b^+}} b]$ 存在最大元素等于 $h(c)$, 即 $h(b) = h(c)^+$ 。以及对任意极限序数 b 有 $h(b) = \bigcup h[\text{seg}_{<_{b^+}} b]$ 。

类似可定义序数乘法满足:

$$a \cdot b = \begin{cases} 0 & \text{若 } b = 0 \\ (a \cdot c) + a & \text{若 } b = c^+ \\ \bigcup \{y : \exists x \in b \ y = a \cdot x\} & \text{其它} \end{cases}$$

这样我们即正式定义了 $\omega \cdot 2$ 。而如示例 4.3.25 指出 ω 为最小极限序数:

示例 4.3.30 $A_Z \vdash \forall b (isLimitOrdinal(b) \wedge b \neq \omega \rightarrow \omega \cdot 2 \in b)$ 。即 $\omega \cdot 2$ 为除 ω 外的最小序数。

证明: 令 $h = +_{\omega, \omega^+}$, $\omega \cdot 2 = h(\omega) = \bigcup h[\text{seg}_{<_{\omega^+}} \omega]$ 。若存在极限序数 a 有 $\omega \in a \in \omega \cdot 2$, 则有 b 有 $a \in b \in h[\text{seg}_{<_{\omega^+}} \omega]$, 则 $a \in \text{ran } h$ 。而对任意 $x \in y$, $h(x) \in h(y)$, 则 $isInjective_{\mathbb{F}}(h)$ 。考虑 $v = h^{-1}(a)$, 因 $\omega \in a$, $\emptyset \in v$, 对任意 $u \in v$, $h(u) \in a$, $h(u)^+ \in a$, $u^+ \in v$, 但 $v \subset \omega$, 则与 ω 为最小归纳集矛盾。 $\square$

序数幂运算¹:

$$a^b = \begin{cases} 1 & \text{若 } b = 0 \\ a^c \cdot a & \text{若 } b = c^+ \\ \bigcup \{y : \exists x \in b \ y = a^x\} & \text{其它} \end{cases}$$

可见自然数运算结果和序数运算结果是一致的, 但注意序数运算性质不完全同于自然数运算, 如:

示例 4.3.31

- a. $A_Z \vdash 1 + \omega \neq \omega + 1$ 。
- b. $A_Z \vdash 2 \cdot \omega \neq \omega \cdot 2$ 。
- c. $A_Z \vdash (\omega \cdot 2)^2 \neq \omega^2 \cdot 2^2$ 。

证明: 对 $a \ b$ 由外延公理有 $1 + \omega = 2 \cdot \omega = \omega$ 。或直观的, $\bigcup \{1, 2, 3, \dots\} = \bigcup \{2, 4, 6, \dots\} = \omega$ 。对 c 有 $(\omega \cdot 2)^2 = \omega \cdot 2\omega \cdot 2 = \omega^2 \cdot 2 \neq \omega^2 \cdot 4$ 。 $\square$

但也有:

引理 4.3.32

- a. $A_Z \vdash isOrd(a) \wedge isOrd(b) \wedge isOrd(c) \rightarrow (a + b) + c = a + (b + c)$ 。
- b. $A_Z \vdash isOrd(a) \wedge isOrd(b) \wedge isOrd(c) \rightarrow (a \cdot b) \cdot c = a \cdot (b \cdot c)$ 。
- c. $A_Z \vdash isOrd(a) \wedge isOrd(b) \wedge isOrd(c) \rightarrow a^b \cdot a^c = a^{b+c}$ 。
- d. $A_Z \vdash isOrd(a) \wedge isOrd(b) \wedge isOrd(c) \rightarrow (a^b)^c = a^{b \cdot c}$ 。

证明: 对 c 做归纳, 如假设任意 $u \in c$ 满足 $(a + b) + u = a + (b + u)$, 则若 $c = v^+$:

$$(a + b) + c = (a + b) + v^+ = ((a + b) + v)^+ = (a + (b + v))^+ = a + (b + v)^+ = a + (b + v^+) = a + (b + c)$$

否则 (其中第三个等于由外延公理可证):

$$\begin{aligned} (a + b) + c &= \bigcup \{y : \exists x \in c \ y = (a + b) + x\} \\ &= \bigcup \{y : \exists x \in c \ y = a + (b + x)\} = \bigcup \{y : \exists x \in (b + c) \ y = a + x\} = a + (b + c) \end{aligned} \quad \square$$

¹注意由上下文应可明确该运算符表示序数幂运算还是集合的笛卡尔积。

4.3.6 康托尔范式定理

进一步, 扩展示例 4.3.25 及 4.3.30 关于序数表示和位次的讨论, 我们将证明:

定理 4.3.33 (康托尔范式定理) 任意非零序数 a 可被唯一的表示为 $\omega^{b_1} \cdot c_1 + \omega^{b_2} \cdot c_2 + \dots + \omega^{b_n} \cdot c_n$, 其中 n 为非零自然数, $b_{1\dots n}$ 为序数满足 $a \supseteq b_1 \supseteq b_2 \supseteq \dots \supseteq b_n$ ^{注1}, $c_{1\dots n}$ 为非零自然数。

首先:

引理 4.3.34

- a. $A_Z \vdash \text{isOrd}(a) \wedge \text{isOrd}(b) \wedge \text{isOrd}(c) \wedge b \in c \rightarrow a + b \in a + c$ 。
- b. $A_Z \vdash \text{isOrd}(a) \wedge \text{isOrd}(b) \wedge \text{isOrd}(c) \wedge 0 \in a \wedge b \in c \rightarrow a \cdot b \in a \cdot c$ 。
- c. $A_Z \vdash \text{isOrd}(a) \wedge \text{isOrd}(b) \wedge \text{isOrd}(c) \wedge 1 \in a \wedge b \in c \rightarrow a^b \in a^c$ 。

证明: 由函数类构造直接可得。如 $a + b = +_{a,b^+}(b) = +_{a,c^+}(b) \in +_{a,c^+}[\text{seg}_{<_{c^+}} c] = +_{a,c^+}(c) = a + c$ 。注意乘法要求 $0 \in a$ 、幂要求 $1 \in a$ 以保证在递归后继时仍有属于 (而非等于) 关系。 $\square$

引理 4.3.35

- a. $A_Z \vdash \text{isOrd}(a) \wedge \text{isOrd}(b) \wedge a \subseteq b \rightarrow \exists! c \ a + c = b$ 。
- b. $A_Z \vdash \text{isOrd}(a) \wedge \text{isOrd}(b) \wedge 0 \in b \rightarrow \exists! c, d \in b \ a = b \cdot c + d$ 。

证明:

- a. 对 b 归纳, 有 $a \subseteq b \rightarrow a + \text{ord}(b - a, <_{b-a}) = b$ 。
- b. 取 $c = \text{greatest}_{O_x}(b \cdot x \subseteq a)$, 取 d 满足 $a = b \cdot c + d$, 则 c, d 存在且唯一且 $d \in b$ 。 $\square$

定理 4.3.33 (康托尔范式定理) 的证明: 对 a 递归, 取 $b = \text{greatest}_{O_x}(\omega^x \subseteq a)$, 则由上述引理 b 有唯一 c 满足 $a = \omega^b \cdot c + d$ 有 $d \in \omega^b \subseteq a$, 若 $d \neq 0$, 则由归纳假设有 $b_{1\dots n} \ c_{1\dots n}$ 满足 $d = \omega^{b_1} \cdot c_1 + \dots + \omega^{b_n} \cdot c_n$, 则 $a = \omega^b \cdot c + \omega^{b_1} \cdot c_1 + \dots + \omega^{b_n} \cdot c_n$ 。可见 $\omega^{b_1} \subseteq d \in \omega^b$, 则 $b_1 \in b$ 。最后 $c \in \omega$ 因否则有 $\omega^{b^+} \subseteq a$ 。 $\square$

对于康托尔范式, 我们可以递归展开其中的 $b_{1\dots n}$ 。如有序数可展开为²:

$$\omega^{\left(\omega^{(\omega^7 \cdot 6 + \omega + 42) \cdot 1729 + \omega^9 + 88}\right) \cdot 3 + \omega^{(\omega^\omega)} \cdot 5 + 65537} \quad (4.3.1)$$

同时注意康托尔范式定理中可令 $b_1 = a$, 即指存在序数 $a = \omega^a$ 。我们称这样 a 为 ε 数。其中, 对任意非零序数 u 写 $u \uparrow\uparrow 0 = 1$, 对任意 $n \in \omega$, $u \uparrow\uparrow n^+ = u^{\uparrow\uparrow n}$, 有序数 $\varepsilon_0 = \omega^{\omega^{\omega^{\dots}}} = \bigcup \{y : \exists x \in \omega \ \omega \uparrow\uparrow x \subseteq y\}$ 。称其中 $\uparrow\uparrow$ 为迭代幂运算。

引理 4.3.36 $A_Z \vdash \text{isLeast}_{O_x}(\varepsilon_0, x = \omega^x)$ 。即 ε_0 为最小 ε 数。

证明: 假设存在 $a \in \varepsilon \wedge a = \omega^a$, 则存在 $n \in \omega$ 有 $a \in \omega \uparrow\uparrow n$, 则 $\omega \uparrow\uparrow (n + a) \subseteq \omega \uparrow\uparrow n$ 。矛盾。 $\square$

虽然可以看到 ε_0 已是很大的序数, 但事实上其仍是可数的。首先以更小的序数为例:

示例 4.3.37 $A_Z \vdash \text{isCountable}(\omega \cdot 2)$ 。

¹ $x \supseteq y$ 表示 $y \in x$, $x \supsetneq y$ 表示 $x \supseteq y \vee x = y$ 。

² 例自 Wikipedia contributors. Ordinal arithmetic — Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/w/index.php?title=Ordinal_arithmetic&oldid=1069193877. 2022。

证明：通过 $\omega \cdot 2 = \omega$ 可得。也可显式构造 $f = \{ \langle x, y \rangle \in \omega \cdot 2 \times \omega : x \in \omega \vee y = 2 \cdot x : y + \omega = x^+ \}$ 有 f 为 $\omega \cdot 2$ 到 ω 上的单射函数。直观的， f 将 $x \in \omega$ 映射到偶数，否则奇数。 $\square$

示例 4.3.38 $A_Z \vdash \text{isCountable}(\omega^2)$ 。

证明：任意 $a \in \omega^2$ 有唯一自然数 $c_1 c_2$ 满足 $a = \omega \cdot c_1 + c_2$ ，则存在函数 f 由 ω^2 到 $\omega \times \omega$ ，而自然数二元组集是可数的，其元素可通过哥德尔编码映射到自然数上。 $\square$

更直观的，如 $\omega \cdot 2$ 、 ω^ω 为可数的意味着即使我们有两种或无限种不同的方式尝试遍历如无限长字符序列，我们仍不能遍历到其中的每个元素。

回到 ε_0 ，则类似的：

引理 4.3.39 $A_Z \vdash \text{isCountable}(\varepsilon_0)$ 。

证明：对任意非零 $n \in \varepsilon_0$ ，可展开为如式 (4.3.1) 的形式，有其为有限长的，且其中除 ω 外涉及的序数均为（非零）自然数。则对该形式进行哥德尔编码可映射至自然数上。 $\square$

显然我们可以继续在 ε_0 的基础上执行运算，如存在序数等于 $\varepsilon_0 + \omega$ 。或进一步构造 ε 数满足对任意序数 a ：

$$\varepsilon_a = \begin{cases} \bigcup \{ y : \exists x \in \omega \omega \uparrow \uparrow x \} & \text{若 } a = 0 \\ \bigcup \{ y : \exists x \in \omega y = \varepsilon_b \uparrow \uparrow x \} & \text{若 } a = b^+ \\ \bigcup \{ y : \exists x \in a y = \varepsilon_x \} & \text{其它} \end{cases}$$

其为 ε 数因如对 ε_1 有 $\varepsilon_0^\omega = (\omega^{\varepsilon_0})^{\varepsilon_0^\omega} = \omega^{(\varepsilon_0 \cdot \varepsilon_0^\omega)} = \omega^{\varepsilon_0^{1+\omega}} = \omega^{\varepsilon_0^\omega}$ 注 1。

事实上：

定理 4.3.40 $A_Z \vdash \text{isCountable}(a) \rightarrow \text{isCountable}(\varepsilon_a)$ 。

证明：对任意 ε_a ，对任意 $x \in \varepsilon_a$ ，类似对 ε_0 可数的证明，由康托尔范式定理， x 有仅含符号 ω 、 ε_i ($i \in a$) 和自然数的唯一表示，该符号集是可枚举的，则该表示是可哥德尔编码的。 $\square$

下面引入不可数序数。

4.3.7 哈托格数

通过替换公理模式可以证明不可数序数的存在：

定理 4.3.41（哈托格数） $A_Z \cup \text{Rep} \vdash \exists b \text{ isOrd}(b) \wedge |b| \not\leq |s|$ 。即对任意集合 s ，存在基数更大的序数 b 。

证明²：构造 b 。首先构造 s 任意子集的良好序结构集 w 为：

$$\{ p \in \wp(s) \times \wp(s^2) : WO(p) \}$$

则通过替换公理模式，可构造 b 为：

$$\{ y : \exists x \in w y = \text{ord } x \}$$

¹参考 Wikipedia contributors. Epsilon numbers (mathematics) — Wikipedia, The Free Encyclopedia. [https://en.wikipedia.org/w/index.php?title=Epsilon_numbers_\(mathematics\)&oldid=1085026924](https://en.wikipedia.org/w/index.php?title=Epsilon_numbers_(mathematics)&oldid=1085026924). 2022。

²证明自 H.B. Enderton. Elements of Set Theory. Elsevier Science, 1977, pp. 195–196。

可见 b 为 w 中良序结构的序数集合。现证明 $isOrd(b)$ 。通过引理 4.3.19 c 仅需证明 $isTransitiveSet(b)$ ，若 $u \in v \in b$ ，则存在 p, q 有 $ord p = u \wedge ord q = v \wedge q \in w$ 且满足 p 为 q 的起始结构，则 $p \in w$ ，有 $v \in b$ 。还需证明 $|b| \not\leq |s|$ 。i：对任意序数 u 满足 $|u| \leq |s|$ ，存在 u 到 s 的单射函数 f ，则有：

$$\langle ran f, \{ \langle x, y \rangle \in s^2 : f^{-1}(x) <_u f^{-1}(y) \} \rangle \in w$$

则 $u \in b$ 。若 $|b| \leq |s|$ ，则 $b \in b$ 。矛盾。□

事实上，上面构造的 b 还满足 $b = least_{O_x}(|x| \not\leq |s|)$ ，因 ii：对任意 $u \in b$ ，存在 p 满足 $p \in w \wedge ord p = u$ ，则 $|u| \leq |s|$ 。而结合 i 和 ii，可见 b 被构造为满足任意 $u \in b \leftrightarrow isOrd(u) \wedge |u| \leq |s|$ 。最后称：

- 上述 b 为 s 的哈托格数。写作 $\mathcal{H}(s)$ 。

4.3.8 基数

在我们开始尝试通过序数定义基数之前，让我们再给出几个关于基数大小的性质：

引理 4.3.42 $A_Z \vdash |a| = |a|$ 。

证明：函数 $\{ \langle x, y \rangle \in a^2 : x = y \}$ 即为 a 到 a 的双射函数。□

引理 4.3.43 $A_Z \vdash |a| \leq |b| \wedge |b| \leq |c| \rightarrow |a| \leq |c|$ 。

证明：令 f 为 a 到 b 上的单射函数， g 为 b 到 c 上的单射函数，则 $\{ \langle x, y \rangle \in a \times c : \exists z z = f(x) \wedge y = g(z) \}$ 为 a 到 c 上的单射函数。□

定理 4.3.44 (康托尔-伯恩斯坦-施罗德定理) $A_Z \vdash |a| \leq |b| \wedge |b| \leq |a| \rightarrow |a| = |b|$ 。

证明¹：令 f 为 a 到 b 上的单射函数， g 为 b 到 c 上的单射函数。定义递归函数 $c \subset \omega \times \wp(a)$ 为：

$$rec[a - ran g, \{ \langle u, v \rangle \in \wp(a)^2 : v = \{ y \in a : \exists x \in u y = g(f(x)) \} \}]$$

可见 $c(0) = a - ran g$ 。对任意 $n \in \omega$ ， $c(n^+) = \{ y \in a : \exists x \in c(n) y = g(f(x)) \}$ ，即 $c(n^+)$ 为 $c(n)$ 中元素经 f, g 两次映射回 a 后的元素集。令函数 h 为：

$$\{ \langle x, y \rangle \in a \times b : \exists n \in \omega x \in c(n) \wedge y = f(x) \}$$

我们证明 h 为 a 到 b 的双射函数。首先证明 h 为单射函数，由 f, g 均为单射函数，仅需要证明对任意 x_1, x_2 ，当存在 u 有 $x_1 \in c(u)$ 而 $\neg \exists n \in \omega x_2 \in c(n)$ 时 $f(x_1) \neq g^{-1}(x_2)$ ，若 $f(x_1) = g^{-1}(x_2)$ ，则 $x_2 = g(f(x_1)) \in c(u^+)$ ，矛盾。

接着我们证明 $ran h = b$ ，对 $y \in b$ 有存在 x, n 满足 $x \in c(n) \wedge y = f(x)$ ， $y \in ran h$ 。即只需考虑 y 不存在 x, n 满足 $x \in c(n) \wedge y = f(x)$ 的情况。对这样的 y ，考虑 $g(y)$ ，显然 $g(y) \notin c(0) = a - ran g$ 。而对任意 u ， $g(y) \notin c(u^+)$ ，因否则存在 x 有 $x \in c(u)$ 。则 $g(y)$ 满足 $\neg \exists n \in \omega g(y) \in c(n)$ ，则 $h(g(y)) = g^{-1}(g(y)) = y$ ，有 $y \in ran h$ 。□

即有（不严格的）集合的基数的小于等于关系为全部集合上的偏序关系。

现在我们通过序数定义基数：

¹证明自 H.B. Enderton. Elements of Set Theory. Elsevier Science, 1977, pp. 147–148。

- 任意 $P(|a|)$ 为 $\forall b (isLeast_{Ox}(b, |x| = |a|) \rightarrow P(b))$ 。即 a 的基数为与其存在双射函数的最小序数。

让我们首先来看几个例子，从中可见其符合基数的直观定义：

示例 4.3.45 $A_Z \vdash n \in \omega \rightarrow |n| = n$ 。

证明：对 n 归纳。 □

示例 4.3.46 $A_Z \vdash |\omega| = \omega$ 。

证明：否则存在 $n \in \omega$ 有 $|n| = |\omega| = |n^+|$ ，则 $|n^+| = n$ ，与上一示例矛盾。 □

示例 4.3.47 $A_Z \vdash |\mathcal{H}(\omega)| = \mathcal{H}(\omega)$ 。即 ω 的哈托格数为自身的基数。

证明：因 $isLeast_{Ox}(\mathcal{H}(\omega), |x| \not\leq \omega)$ 。 □

然而，尽管由 $A_Z \cup Rep$ 可以证明不可数基数 $\mathcal{H}(\omega)$ 的存在。我们的定义仍是有问题的，因我们并不能由 $A_Z \cup Rep$ 证明对任意 a 有 $\exists b |b| = |a|$ ，因可能不存在序数 b 满足 $|b| = |a|$ 。这里需要选择公理。本质上，通过选择公理可以证明：

定理 4.3.48 (良序定理) $A_Z; Choice \vdash \exists r WO(s, r)$ 。即对任意集合 s 存在其上的 (严格) 良序关系 r 。

我们将在下一小节讨论选择公理并给出该定理的证明。但通过尝试为实数集 $\mathbb{R}$ 的任意子集赋予最小元素可以直观地感受到良序排序 $\mathbb{R}$ 的难度。

首先可以看到：

引理 4.3.49 $A_Z \vdash \exists b isOrd(b) \wedge |b| = |a| \rightarrow \exists r isStrictWellOrder(a, r)$ 。

证明：令 f 为 b 到 a 的双射函数。 r 为 $\{ \langle x, y \rangle \in a^2 : f^{-1}(x) <_b f^{-1}(y) \}$ 有 $isStrictWellOrder(a, r)$ 。 □

则若我们不能证明任意集合有其上的良序关系，则有模型域中存在元素 a 不存在其上的良序关系，则不存在 $b = |a|$ ，即上述的定义问题。

而回顾定理 4.3.41 (哈托格数) 的证明，应注意在没有选择公理的情况下，集合 s 并不一定有良序关系 r 使 $\langle s, r \rangle \in w$ 。但其构造的 w (域中) 包括了所有具有良序关系的 s 的子集，因此 b 中包含了所有基数小于等于 $|s|$ 的序数。

在解决了定义问题之后，集合的基数即为一部分特殊的序数。结合序数属于等于 / 包含于关系的全序性，有基数的小于等于关系为 (不严格的) 全序关系。则可写 $|a| > |b|$ 表示 $|a| \not\leq |b|$ 等价于 $|b| < |a|$ 。

定义 $\aleph$ 数为：

- $\aleph_0 = |\omega| = \omega$ 。
- 任意后继序数 a^+ ， $\aleph_{a^+}$ 为基数大于 $\aleph_a$ 的最小序数，即 $P(\aleph_{a^+})$ 为 $\forall b (isLeast_{Ox}(b, \aleph_a < |x|) \rightarrow P(b))$ 。
- 任意极限序数 a ， $\aleph_a = \bigcup \{ y : \exists x \in a y = \aleph_x \}$ 。

则 $|\mathcal{H}(\omega)| = \aleph_1$ 。而任意无限基数为某 $\aleph$ 数。以及对任意序数 a ， $|\aleph_a| = \aleph_a$ 。另：

- 对任意序数 a ，写 $\omega_a = \aleph_a$ 。

有如 $\omega_1 = \omega^{\omega_1} = \varepsilon_{\omega_1}$ 。

最后，写：

- ${}^a b$ 表示 $\{f \in \wp(a \times b) : \text{isFunction}(f) \wedge \text{dom } f = a\}$ 。即 a 到 b 的函数集合。

-我们定义序数运算，对任意基数 κ, λ ：

$$\begin{aligned} \kappa + \lambda &= |A \cup B| & \text{其中 } |A| = \kappa \wedge |B| = \lambda \wedge A \cap B = \emptyset \\ \kappa \cdot \lambda &= |A \times B| & \text{其中 } |A| = \kappa \wedge |B| = \lambda \\ \kappa^\lambda &= |{}^B A| & \text{其中 } |A| = \kappa \wedge |B| = \lambda \end{aligned}$$

可见在有限的情况下，基数运算和序数 / 自然数运算是一致的。同时：

引理 4.3.50 $A_Z \vdash 2^{|a|} = |\wp(a)|$ 。

证明：可构造 ${}^a\{0, 1\}$ 到 $\wp(a)$ 的双射函数 h 为：

$$\{\langle f, s \rangle \in {}^a\{0, 1\} \times \wp(a) : \forall x (x \in s \leftrightarrow f(x) = 1)\} \quad \square$$

这样我们正式定义了连续统假设 $2^{\aleph_0} = \aleph_1$ 。我们将不证明连续统假设在 ZFC 下的独立性，但将在章节 5.3.7 证明选择公理独立于 ZF，类似方法也可用于证明连续统假设的独立性。

4.3.9 选择公理

对任意集合 A 满足 $\emptyset \notin A$ ，称函数 F 满足 $\text{dom } F = A \wedge \forall \langle x, y \rangle \in F (y \in x)$ 为 A 的选择函数。直观的说，选择函数从 A 的各个元素 x 中选择 $y \in x$ 。如对 $A = \{\{0, 1\}, \{1\}, \{2, 3\}\}$ ，选择函数 F 可等于 $\{\langle \{0, 1\}, 0 \rangle, \langle \{1\}, 1 \rangle, \langle \{2, 3\}, 3 \rangle\}$ 或 $\{\langle \{0, 1\}, 1 \rangle, \langle \{1\}, 1 \rangle, \langle \{2, 3\}, 3 \rangle\}$ 等。

选择公理断言了任意不含空集集合的选择函数存在：

引理 4.3.51 $A_Z; \text{Choice} \vdash \emptyset \notin s \rightarrow \exists f \text{isFunction}(f) \wedge \text{dom } f = s \wedge \forall x \in s (f(x) \in x)$ 。

证明：令 $r = \{\langle u, v \rangle \in s \times \bigcup s : v \in u\}$ 。则由选择公理有函数 f 满足 $\text{dom } f = s$ 且 $f \subseteq r$ ，即对任意 $x \in s$ ， $f(x) \in x$ 。 $\square$

尽管从某种角度看从非空集合中选出一个元素应是可行的，但事实上在 Z 下我们并不总能证明这样的函数存在，如我们不能证明 $\mathbb{R}$ 的选择函数存在。同时，从认识论的角度上说，选择公理也在很长一段时间内（若非至今）具有争议（因而 ZFC 集合论的命名中专门使用了 C 指选择公理）。

下面我们讨论 Z 中可证明存在的选择函数，并说明其它无法证明存在的选择函数的问题所在。从中可以看到增加替换公理模式并不能在这种情况下帮助我们，也可以看到其中的争议所在。而如前所述，我们将在章节 5.3.7 通过证明选择公理的独立性，一方面证明所有不含空集的集合的选择函数是可存在的，另一方面为保证选择函数总是存在，选择公理是必要的。

显然，Z 下，对于非空集合 a ，我们可以证明存在有序对 $\langle a, x \rangle$ 有 $x \in a$ ，即：

$$A_Z \vdash a \neq \emptyset \rightarrow \exists x, p (p = \langle a, x \rangle \wedge x \in a)$$

则：

$$A_Z \vdash a \neq \emptyset \rightarrow \exists x, f (f = \{\langle a, x \rangle\} \wedge x \in a)$$

可见 f 为 $\{a\}$ 的选择函数。则通过并集公理：

$$A_Z \vdash a_1 \neq \emptyset \wedge a_2 \neq \emptyset \rightarrow \exists x_1, x_2, f \ f = \{ \langle a_1, x_1 \rangle, \langle a_2, x_2 \rangle \} \wedge x_1 \in a_1 \wedge x_2 \in a_2$$

则通过任意有限次的应用并集公理，有对任意（元逻辑下的）正整数 n ：

$$A_Z \vdash a_1 \neq \emptyset \wedge \dots \wedge a_n \neq \emptyset \rightarrow \exists x_1, \dots, x_n, f \ f = \{ \langle a_1, x_1 \rangle, \dots, \langle a_n, x_n \rangle \} \wedge x_1 \in a_1 \wedge \dots \wedge x_n \in a_n$$

即存在任意可通过有限次并集得到的集合 $\{a_1, \dots, a_n\}$ 的选择函数 f 。然而以此我们并不能一定能得到其它方式构造的集合的选择函数，因我们需要（要求）证明过程是有限的。但这也不代表我们不能处理全部其它集合，比如我们可以通过分类公理模式证明任意（二元）有序对的集合存在选择函数，如：

$$A_Z \vdash a \subseteq b \times c \rightarrow \exists f \ f = \{ \langle x, y \rangle \in a \times b : y = fst(x) \}$$

即 f 总是选择 a 中任意有序对 x 的前项。事实上，在章节 4.2.5 末，我们即是按此方式从柯西序列等价类中选出的实数运算中所用的代表序列。

应注意分类公理模式不能证明任意集合 a 存在选择函数 f 并不是因为我们不能找到 b 有 $f \subseteq b$ ，因可以取 $b = \wp(a) \times a$ 。这里的问题在于我们无法总是找到从 b 分类出 f 的式 $\rho_{[x]}$ ，因此替换公理模式并不能在更大程度上帮助我们。

本质上，无法找到定义 f 的 ρ 也是选择公理引起争议的主要原因之一。即选择公理断言了某些（其中元素）无法被定义的集合存在。因此其可被认为是非构造性的。比如：

定理 4.3.52（迪亚科内斯库-古德曼-迈希尔定理）在直觉主义 Z 集合论中，选择公理蕴含排中律¹。

证明²：对任意语句 α 。令： $u = \{ x \in \{0, 1\} : x = 0 \vee \alpha \}$

$$v = \{ x \in \{0, 1\} : x = 1 \vee \alpha \}$$

则 $u \neq \emptyset \wedge v \neq \emptyset$ 因 $0 \in u \wedge 1 \in v$ 。则 $0 \notin \{u, v\}$ ，由选择公理有 $\{u, v\}$ 的选择函数 f 满足 $f(u) \in u \wedge f(v) \in v$ 。则 $(\alpha \vee f(u) = 0) \wedge (\alpha \vee f(v) = 1)$ ，蕴含 $\alpha \vee f(u) \neq f(v)$ 。而有 $\alpha \rightarrow u = v$ ，蕴含 $\alpha \rightarrow f(u) = f(v)$ ，则 $f(u) \neq f(v) \rightarrow \neg \alpha$ 。可得 $\alpha \vee \neg \alpha$ 。□

最后，选择公理有多种等价形式，我们这里给出两个例子：

定理 4.3.53 $A_Z \vdash \text{Choice} \leftrightarrow \forall s (\emptyset \notin s \rightarrow \exists f \text{ isFunction}(f) \wedge \text{dom } f = s \wedge \forall x \in s \ f(x) \in x)$ 。

证明³：引理 4.3.51 为正方向。反方向对任意关系 a 存在等价关系 $r = \{ \langle p, q \rangle \in a^2 : fst(p) = fst(q) \}$ ，则存在等价类划分 $s = a/r$ ，则有 s 的选择函数 f ，取 $b = \text{ran } f$ 可见 b 满足要求。□

定理 4.3.54 $A_Z \vdash \text{Choice} \leftrightarrow \forall s \exists r \text{ WO}(s, r)$ 。即选择公理与良序定理等价。

证明：

1. 正方向由选择公理至良序定理⁴。有 $\wp(s) \setminus \emptyset$ 的选择函数 f 。构造 w 为：

¹如章节 3.1.3 所述，直觉主义逻辑不承认（无条件的）排中律。

²证明自 Wikipedia contributors. Diaconescu's theorem — Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/w/index.php?title=Diaconescu%27s_theorem&oldid=1018521870. 2021。注意该证明是在直觉主义推导规则下完成的。如其中用到的 $(\alpha \rightarrow \beta) \rightarrow (\neg \beta \rightarrow \neg \alpha)$ 在直觉主义逻辑下亦是可证的（章节 3.1.3）：

$$\frac{\frac{\frac{[\alpha \rightarrow \beta] \quad [\alpha] \quad [\neg \beta]}{\perp \{ \alpha \rightarrow \beta, \alpha, \neg \beta \}}}{(\alpha \rightarrow \beta) \rightarrow (\neg \beta \rightarrow \neg \alpha)} \varnothing}{\neg I \rightarrow I} \rightarrow E \neg E$$

⁴参考 Ernst Zermelo. “Proof that every set can be well-ordered”. In: From Frege to Gödel. Ed. by Jean Van Heijenoort. Cambridge: Harvard University Press, 1967, pp. 139–141。

$$\{ \langle m, < \rangle \in \wp(s) \times \wp(s^2) : WO(m, <) \wedge \forall x \in m \ x = f(s - \text{seg}_{<} x) \}$$

取 $t = \bigcup \text{dom } w$, $<_t = \bigcup \text{ran } w$ 。我们通过证明 $WO(t, <_t)$ 且 $t = s$ 证明可取 $r = <_t$ 满足要求:

- 对任意 $\langle m_1, <_1 \rangle \in w$ 、 $\langle m_2, <_2 \rangle \in w$ ，首先证明:

$$\bigvee^3 (\langle m_1, <_1 \rangle = \langle m_2, <_2 \rangle, (\exists t \text{ Seg}_{<_1} t = \langle m_2, <_2 \rangle), (\exists t \langle m_1, <_1 \rangle = \text{Seg}_{<_2} t)) \quad (4.3.2)$$

即两者要么相等，要么其一为另一的起始结构。通过定理 4.3.3 我们已有两者要么同构，要么其一与另一的起始段同构。只需证明同构的双射函数 h 为恒等函数。以 $\langle m_1, <_1 \rangle = \text{Seg}_{<_2} t$ 为例，对 m_1 中元素归纳。对 $v \in m_1$ ，归纳假设 $\forall u <_1 v \ u = h(u)$ ，则 $\text{seg}_{<_1} v = \text{seg}_{<_2} h(v)$ ，结合由 w 的定义有 $v = f(\text{seg}_{<_1} v)$ ， $h(v) = f(\text{seg}_{<_2} h(v))$ ，则 $v = h(v)$ 。

- 分别证明 $WO(t, <_t)$ 的几个条件。显然 $<_t$ 是反自反的。考虑 $x <_t y \wedge y <_t z$ ，若 $x <_1 y$ 而 $y <_2 z$ ，结合由 (4.3.2) 有 $<_1 \subseteq <_2 \vee <_2 \subseteq <_1$ ， $x <_1 z \vee x <_2 z$ ，则 $x <_t z$ ，即是传递的。类似有连通的。而对 t 的任意非空子集 u ，存在 $x \in u$ ，存在 $\langle m', <' \rangle \in w$ 有 $x \in m'$ ，取 $y = \min_{<' } (u \cap m') = \min_{<_t } u$ 。
- 最后证明 $t = s$ 。显然 $t \subseteq s$ 。若 $t \neq s$ ，则存在 $u = f(s - t)$ ，构造良序结构 $p = \langle t; u, <_t \cup (t \times \{u\}) \rangle$ 有 $p \in w$ ，则 $u \in t$ ，但 $u \in s - t$ 。矛盾。

2. 反方向由良序定理至选择公理¹。我们证明任意不含空集的集合 s 存在选择函数 f 。令 $t = \bigcup s$ ，则由良序定理存在 $<$ 有 $WO(t, <)$ 。则可构造 f 为 $\{ \langle x, y \rangle \in s \times t : \text{isMinimal}(y, x, <) \}$ 。□

定理 4.3.48 (良序定理) 的证明：即上述定理正方向。□

以及，选择公理有一些更弱的形式。如可数选择公理：

$$\text{isRelation}(a) \wedge \text{isCountable}(\text{dom } a) \rightarrow \exists b \text{ isFunction}(b) \wedge b \subseteq a \wedge \text{dom } b = \text{dom } a \quad (\text{Choice}_\omega)$$

或等价²的 $\emptyset \notin s \wedge \text{isCountable}(s) \rightarrow \exists f \text{ isFunction}(f) \wedge \text{dom } f = s \wedge \forall x \in s \ f(x) \in x$ 。

由可数选择公理，我们可以证明：

引理 4.3.55 $A_Z; \text{Choice}_\omega \vdash \text{isCountable}(s) \wedge \forall a \in s \ \text{isCountable}(a) \rightarrow \text{isCountable}(\bigcup s)$ 。即可数个可数集合的并集是可数的。

证明：我们通过 s 和其中元素到 ω 的单射函数构造 $\bigcup s$ 到 ω 的单射函数。直观的说，我们需要（可数）选择公理是因为任意 $a \in s$ 可能有多个 a 到 ω 的单射函数，我们需要从中选出一个单射函数。具体的，令 r 为：

$$\{ \langle a, f \rangle \in s \times (\bigcup s \times \omega) : \text{isInjective}_\mathbb{F}(f) \wedge a = \text{dom } f \}$$

即指 r 不一定为函数，但我们可以由选择公理得到函数 g 满足 $g \subseteq r \wedge \text{dom } g = \text{dom } r$ 。令 s 到 ω 的单射函数为 h 。则可构造 $\bigcup s$ 到 ω^2 的单射函数：

$$\{ \langle x, y, z \rangle \in \omega^2 \times \bigcup s : x \in \text{ran } h \wedge y \in \text{ran } g(h^{-1}(x)) \wedge z = g(h^{-1})^{-1}(x) \\ \wedge \forall n <_\omega x \ (n \notin \text{ran } h \vee z \notin \text{ran } g(h^{-1}(n))) \}^{-1} \quad \square$$

通过上述引理，我们可以证明可数序数经序数加法、乘法或幂运算后得到的序数是可数的，如 ε_0 是可数的。

¹证明自 Wikipedia contributors. Well-ordering theorem — Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/w/index.php?title=Well-ordering_theorem&oldid=1076165806. 2022.

²证明同定理 4.3.53。

注意可数选择公理严格弱于选择公理，即存在模型满足可数选择公理但不满足选择公理。

我们将在下一章节讨论唯一剩下的正则公理。

4.4 本章小节

- 公理集合论为形式化、公理化后的集合论，其语言一般为含 $=$ 符号的一阶逻辑语言加上二位谓词符号 $\in$ 。
- 罗素悖论指出了朴素集合论的不一致。ZFC 集合论为解决罗素悖论的一种常用的公理集合论。其中 Z 一般指外延、空集、配对、并集、幂集、无限公理和分类公理模式， F 增加了替换公理模式和正则公理， C 指选择公理。
- 集合论中出现的其它符号如 $\emptyset \subseteq$ 等均为集合论语言下被定义的符号，通过集合论公理我们可证明大多常见符号所代表集合的存在唯一性，其中我们主要通过外延公理证明唯一性。
- 集合论下的证明仍为演绎规则指导下的推导计算过程，尽管详尽的推导过程一般是繁复而琐碎的。
- ZFC 集合论（模型域）中全部元素均为集合。
- 通过配对和并集公理我们可以定义有序对。通过有序对我们可以定义笛卡尔积、关系、函数以及其上的一系列性质判定谓词，从而定义如良序、良基结构等。
- 无限公理指出了归纳集的存在。通过定义自然数集为最小归纳集可以得到归纳原理，在此基础上可以定义递归函数，具体的，可定义自然数加法、乘法等。则可认为任意 Z 模型中包含对应 P 模型的子模型。
- 同时也意味着 Z 下可进行 P 下可进行的全部运算，包括（通过哥德尔编码）操作任意有限长度自然数序列等，如可定义整数、实数及其上的相关运算。
- 实数的定义有多种常见方式，这里主要介绍的是基于无限长有理数序列构成的柯西序列的定义方式。实数和有理数的主要区别在于其具有完全性。
- 扩展自然数我们可以得到序数，自然数集 ω 为第一个无限序数。但 $\omega \cdot 2$ 的存在需要替换公理模式。
- 称可被式定义的集合合集为类，称不为集合的类为真类。罗素悖论指出了真类的存在，其在 ZFC 下不为集合因而避免了悖论的存在。称性质类似函数的类为函数类。
- 扩展归纳和递归原理我们可以得到良序和良基结构上的超限归纳递归原理。并可进一步扩展得到函数类的超限归纳递归原理。在此基础上我们可以定义序数，全部序数的合集为真类。
- 我们还可以按照序数性质或分类定义序数，在序数分类的基础上有序数归纳原理，在其上我们可以定义序数运算。康托尔范式定理指出在可数次迭代幂运算下得到的大序数如 ε 数仍可数。
- 哈托格数证明了不可数序数的存在， ω 的哈托格数为最小不可数序数。
- 则可扩展这种不可数关系（即不存在双射函数关系）区分序数以定义基数。即集合的基数为与其存在双射函数的最小序数。则可见如自然数为基数， ω 为基数 $\aleph_0$ ， ω 的哈托格数为 $\aleph_1$ 。
- 但任意集合的基数存在需要选择公理以保证良序关系的存在。而选择公理在认识论上有其不可构造性的争议，如技术上选择公理蕴含（直觉主义逻辑不接受的无条件）排中律。

第五章 集合论与元逻辑

在章节 4.2.4 中，我们看到了集合论 $\mathcal{Z}$ 的模型中包含了 $\mathbf{P}$ 的模型，同时指出了这里的集合论并不为元逻辑，因此也并未涉及元逻辑的形式化。本节将补全这一点，从而完成第二章首图 2.1 中的全部内容。特别的，需注意其中不含循环定义和论证。具体我们将在章节中形式化元逻辑的基础概念并通过不同的方法构造集合论的模型（如前述未提及的 $\mathcal{Z}$ 模型本身）。在章节 5.1 中我们将首先（在以集合论作为的元逻辑中）形式化证明和模型的概念，并介绍被称为冯·诺依曼全集的集合层级结构，而（第四章中未讨论到的）正则公理将全部集合限制于该结构中（从而剔除了难以处理且被认为不符合直觉的集合）。章节 5.2 将介绍如何通过内模型（在假定存在的集合论模型基础上）构造（另一个）集合论模型，其中我们需要用到另一种被称为可构造全集的集合层级结构。章节 5.3 介绍通过力迫法构造模型，相对于内模型向内寻找模型，可认为力迫法向外寻找模型。最后章节 5.4 在更一般的角度上尝试对元逻辑进行讨论。

5.1 证明与模型

本节在关于集合论模型的目标上是建立冯·诺依曼全集，在其基础上，我们将看到符合直觉的不同级别的集合论模型。为此以及后续关于元逻辑的讨论，我们需首先形式化证明和模型相关的概念，这也为之前章节的论证提供了依据。

5.1.1 证明与模型的形式化

我们将在集合论元逻辑中定义目标逻辑的语法（演绎规则）和语义概念。

首先对于语法部分。对于可枚举语言，类似哥德尔编码对符号的处理，其中全部符号可被定义为自然数，则项、式和语句可被定义为符号序列（多元组）的哥德尔编码结果。为重用章节 3.1.8 中演绎规则的实现。我们定义这些概念表示和 3.1.8 中使用的数据结构一致¹。如定义：

- 变量符号为 $\langle t_{var}, id \rangle$ 的哥德尔编码结果， $t_{var} = 0$ 。函数符号为 $\langle t_{fn}, n, id \rangle$ 的编码结果， $t_{fn} = 1$ 等。
- 项和式（语句）为相应树的编码结果（注意不是树根节点的 *form* 字段）。

对目标逻辑中任意以上概念的具体对象 a ，写 $\ulcorner a \urcorner$ 表示 a 的编码结果。对任意对象集合 $A = \{a, b, c, \dots\}$ ，写 $\ulcorner A \urcorner$ 表示 $\{\ulcorner a \urcorner, \ulcorner b \urcorner, \ulcorner c \urcorner, \dots\}$ 。比如 $\ulcorner A_Z \urcorner = \{\ulcorner Ext \urcorner, \dots, \ulcorner Inf \urcorner\}$ 。

结合章节 4.2.4 末关于 μ -递归函数可被集合论语句定义。我们有章节 3.1.8 中用于（通过 *form* 字段）判断语句是否属于公理集 Γ 的 $\text{gamma}(c)$ 可被 $\gamma_g[c] = c \in a_g$ 定义，其中 $g = \ulcorner \Gamma \urcorner$ ， a_g 为：

$$\{ x \in \omega : \exists b \in g \ x = b[ge_{\#}(b)].form \}$$

¹可以将 $=$ 视为二位谓词。或按当前作为逻辑符号处理，扩展 3.1.8 中的数据结构和实现。

其中 $b[ge_{\#}(b)].form$ 返回 b 树根节点的 $form$ 字段，写其为 $form'(b)$ ，其是可计算的，因而是可定义的。

可见章节 3.1.8 中的 $prove$ 是 a_g - μ -可计算的，结合推论 3.2.2.1 有存在 $\rho_g[s]$ 定义 $prove(s)$ 。可定义：

- $\vdash'$ 关系为 $\{ \langle g, s \rangle \in \wp(\omega) \times \omega : \rho_g[s/form'(s)] \}$ 。写 $\Gamma \vdash \sigma$ 表示 $\ulcorner \Gamma \urcorner \vdash' \ulcorner \sigma \urcorner$ 。

对任意（可枚举）一阶逻辑语言 $\mathcal{L}$ ，令 V 表示其变量符号集合， C 表示常量符号集合，任意正整数 n ， F_n 表示 n 位函数符号， P_n 表示 n 位谓词符号，则还要求上面的项和式（语句）中涉及符号的编码结果属于：

$$\ulcorner \{ (,), \cdot, \neg, \vee, \exists, = \} \cup V \cup C \cup \bigcup_{n \in \omega_{\neq 0}} (F_n \cup P_n) \urcorner$$

其中 $\omega_{\neq 0}$ 表示 $\omega \setminus 0$ 。写：

- $isSentence_{\mathcal{L}}(x)$ 表示 x 为 $\mathcal{L}$ 下的语句（编码结果）。 $isFormula_{\mathcal{L}}(x)$ 表示 x 为 $\mathcal{L}$ 下的式。

下面考虑语义部分。仍以 $\mathcal{L}$ 为例，可见语言的参数为：

$$p = \ulcorner \{ \forall \} \cup C \cup \bigcup_{n \in \omega_{\neq 0}} (F_n \cup P_n) \urcorner$$

则可定义：

- 解释 $\Im$ 为 $\langle d, f \rangle$ 有 d 为任意集合对应 $\Im$ 的域， f 为符号解释的映射函数，即要求满足：

$$isFunction(f) \wedge f \in \{ \langle x, y \rangle \in (p \setminus \ulcorner \forall \urcorner) \times (d \cup ran r) : (x \in \ulcorner C \urcorner \rightarrow y \in d) \\ \wedge \forall n \in \omega_{\neq 0} (x \in \ulcorner F_n \urcorner \rightarrow isFunction(y) \wedge y \in r(n)) \wedge (x \in \ulcorner P_n \urcorner \rightarrow y \in r(n)) \}$$

其中 r 为 $n \in \omega_{\neq 0}$ 到 d^n 集合的映射函数。注意其存在需要替换公理模式，或为：

$$trec_{II}[\omega_{\neq 0}, <_{\omega_{\neq 0}}, x = 1? d : y = f(x-1) \times d] \text{ 注 } ^1$$

定义 $isInterp_{\mathcal{L}}(x)$ 表示 x 为 $\mathcal{L}$ 的解释。

最后，我们考虑 $\vdash$ 。其有两种用法，一为如 $\vdash_{\Im} \Sigma$ 有 $\Im$ 为解释、 Σ 为语句集，表示 $\Im$ 为 Σ 的模型（满足 Σ ）。二为 $\Gamma \vdash \sigma$ 表示语句集 Γ 语义蕴含语句 σ 。其中可定义：

- $\Gamma \vdash \sigma$ 为 $\forall x (isInterp_{\mathcal{L};\sigma}(x) \wedge \vdash_x \Gamma \rightarrow \vdash_x \sigma)$ 。其中 $\mathcal{L}_{\Gamma;\sigma}$ 为由 $\Gamma; \sigma$ 所含符号构成的语言。

则我们仅需讨论 $\vdash_{\Im} \Sigma$ 或对语句 $\vdash_{\Im} \sigma$ 。我们将看到其是可（依据塔斯基真值语义）定义的。对 σ 的构成做归纳：

1. σ 为原子语句 $P(c_{1...n})$ 有 $c_{1...n}$ 均为常量。则 $\vdash_{\langle d, f \rangle} \sigma$ 等价于 i: $\langle f(\ulcorner c_1 \urcorner), \dots, f(\ulcorner c_n \urcorner) \rangle \in f(\ulcorner P \urcorner)$ 。
2. σ 为 $y = F(c_{1...n})$ 有 $c_{1...n}$ 为常量。则 $\vdash_{\langle d, f \rangle} \sigma$ 等价于 ii: $\langle f(\ulcorner c_1 \urcorner), \dots, f(\ulcorner c_n \urcorner), f(\ulcorner y \urcorner) \rangle \in f(\ulcorner F \urcorner)$ 。
3. σ 为 $\neg \alpha$ 。则 $\vdash_{\Im} \sigma$ 等价于 $\neg \vdash_{\Im} \alpha$ 。对 $\vee$ 同理。
4. σ 为 $\exists x \alpha$ 。扩展当前语言，增加额外的常量符号 e ，则 $\vdash_{\langle d, f \rangle} \sigma$ 等价于 $\exists x \in d \vdash_{\langle d, f; \langle \ulcorner e \urcorner, x \rangle \rangle} \alpha[x/e]$ 。
5. σ 为含嵌套函数符号的语句，则可引入中间变量展开嵌套。如 $P(F(c))$ 可展开为 $\exists y y = F(c) \wedge P(y)$ 。

显然：

- 该归纳可被转换为 B - μ -递归函数对任意 $\Im \sigma$ 返回是否有 $\vdash_{\Im} \sigma$ 。其中预言者 B 回答 i 和 ii，可由 i 和 ii 定义。则 $\vdash_{\Im} \sigma$ 也是可定义的。写 $\vdash'$ 表示处理编码结果的关系（式），如 $\vdash_{\Im} \sigma$ 即为 $\vdash'_{\Im} \ulcorner \sigma \urcorner$ 。

¹ $x-1$ 即为 $\bigcup x$ 。

在以上定义的基础上，我们可形式化前三章中关于证明和模型的全部内容。

下面考虑集合的层级结构并在此基础上构造集合论的具体模型。

5.1.2 冯·诺伊曼全集¹

不管是集合直观的物体合集概念，还是形式化的构造方法都暗示了一种全体集合的层级结构的存在。从形式化的角度，可认为空集公理为该结构提供了起点，幂集、无限和替换公理证明了层级的存在，配对、并集和分类公理在层级中填充了元素。

具体的，该结构有序数层级，定义：

- 对任意序数 a ，层级 $V_a = \bigcup \{ y : \exists b \in a \ y = \wp(V_b) \}$ 。

整个结构 V 由全部 V_a 构成，称 V 为冯·诺伊曼全集。或等价的，可定义：

- $V_0 = \emptyset$ 。
- 对任意后继序数 $a = b^+$ ， $V_a = \wp(V_b)$ 。如 $V_5 = \wp^5(\emptyset)$ 。
- 对任意极限序数 a ， $V_a = \bigcup \{ y : \exists b \in a \ y = V_b \}$ 。如 $V_\omega = \bigcup \{ \wp^0(\emptyset), \wp^1(\emptyset), \dots \}$ 。

其中，对二三项后继和极限序数定义的简化可由以下至引理 5.1.3 可见。但首先，我们在第一种定义方式的基础上给出各 V_a 的更严格的定义：

- 对任意序数 a ，可定义 V_a 为 $\text{trc}_{\Pi}[a^+, <_{a^+}, y = \bigcup \{ z : \exists u \in \text{ran } f \ z = \wp(u) \}](a)$ 。

该定义暗示了如 V_ω 的存在依赖替换公理模式。事实上，我们可构造一个 Z 的模型有 V_ω 不在其域中，见下述引理 5.1.6。同时在替换公理的帮助下，任意序数 a 的存在意味着集合 V_a 的存在，即我们有如 $V_{\omega \cdot 2}$ 、 V_{ϵ_0} 、 V_{ω_1} 以及 V_{ω_ω} 等的存在。

现考虑关于 V 的一些性质：

引理 5.1.1 $A_Z \vdash \text{isOrd}(a) \rightarrow \text{isTransitiveSet}(V_a)$ 。

证明：应用定理 4.3.27 对序数做归纳。对序数 a ，假设任意 $b \in a$ 满足 $\text{isTransitiveSet}(V_b)$ ，我们需要证明 $\text{isTransitiveSet}(V_a)$ ，即 $\text{isTransitiveSet}(\bigcup \{ y : \exists b \in a \ y = \wp(V_b) \})$ 。该目标通过以下两个事实可直接得到：

$$A_Z \vdash \text{isTransitiveSet}(s) \rightarrow \text{isTransitiveSet}(\wp(s)) \quad (5.1.1)$$

$$A_Z \vdash \forall t \in s \ \text{isTransitiveSet}(t) \rightarrow \text{isTransitiveSet}(\bigcup s) \quad (5.1.2)$$

对 (5.1.1)，考虑任意 $x \in \wp(s)$ ， $x \subseteq s$ ，即任意 $y \in x \rightarrow y \in s$ ，结合引理 4.2.2 b 有 $y \subseteq s$ 有 $y \in \wp(s)$ 。而对 (5.1.2)，任意 $x \in \bigcup s$ ，有 $x \in t \in s$ ，而任意 $y \in x \in t$ 有 $y \in t$ ，则 $y \in \bigcup s$ 。□

引理 5.1.2

- $A_Z \vdash \text{isOrd}(a) \wedge \text{isOrd}(b) \wedge a \in b \rightarrow V_a \in V_b$ 。
- $A_Z \vdash \text{isOrd}(a) \wedge \text{isOrd}(b) \wedge a \in b \rightarrow V_a \subseteq V_b$ 。

证明：对 a ，可见 $V_a \in \wp(V_a) \subseteq V_b$ ，则 $V_a \in V_b$ 。而 b 由 a 及 $\text{isTransitiveSet}(V_b)$ 可得。□

¹据 Wikipedia contributors. Von Neumann universe — Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/w/index.php?title=Von_Neumann_universe&oldid=1102797999. 2022，尽管如此命名，其最早是策梅洛于 1930 年提出的。

引理 5.1.3

- a. $A_Z \vdash \text{isOrd}(a) \rightarrow V_{a^+} = \wp(V_a)$ 。
 b. $A_Z \vdash \text{isLimitOrdinal}(a) \rightarrow V_a = \bigcup \{y : \exists b \in a \ y = V_b\}$ 。即 $V_a = \bigcup_{b \in a} V_b$ 。

证明：

- a. 任意 $b \in a$ 有 $V_b \subseteq V_a$ 有 $\wp(V_b) \subseteq \wp(V_a)$ ，则 $V_{a^+} = \bigcup_{b \in a} \wp(V_b) = \wp(V_a)$ 。
 b. 任意 x ，若 $x \in V_a = \bigcup_{b \in a} V_b$ ，存在 b 有 $x \in V_b \subseteq V_{b^+} = \wp(V_b)$ ，则 $x \in \bigcup_{b \in a} \wp(V_b)$ 。另一方面，若存在 b 有 $x \in \wp(V_b)$ ，则 $x \in V_{b^+}$ ，结合 $b^+ \in a$ 有 $x \in V_a = \bigcup_{b \in a} V_b$ 。□

综合上述讨论我们有如图 5.1 所示的 V 的结构。

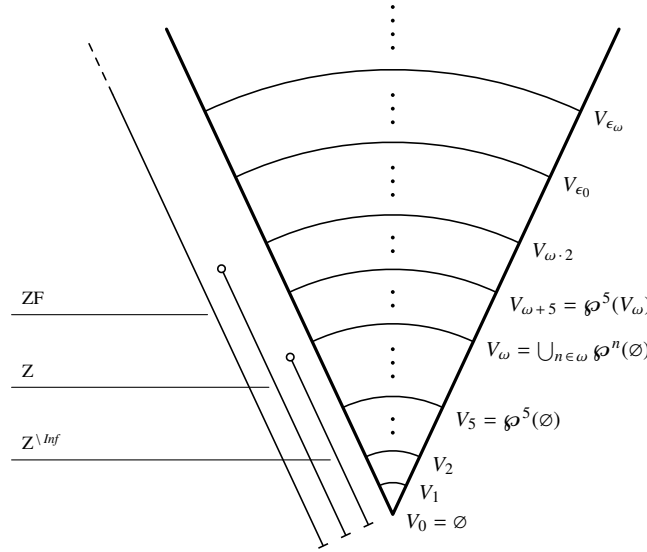


图 5.1: 冯·诺伊曼全集

图中还标注了一些层级可以构成的模型，比如 V_{ω} （注意不含 V_{ω} ）构成了由公理集 $A_Z \setminus \text{Inf}$ 决定的理论 $Z \setminus \text{Inf}$ 的模型。下面我们对此展开更详细的讨论。

具体的，我们认为 V 中集合所在的集合论逻辑为元逻辑，这里要求该集合论元逻辑可证明讨论所需层级（如 V_{ω} ）的存在¹。我们称 V_{ω} 可构成 $Z \setminus \text{Inf}$ 的模型指存在 $Z \setminus \text{Inf}$ 的模型 $\mathfrak{M}$ 有域解释为（元逻辑中的） V_{ω} ，符号 $\in$ 解释为（元逻辑中 V_{ω} 上的）属于关系。对集合论的模型，写 (v, e) 表示 $\langle v, \{\langle \ulcorner \in \urcorner, e \rangle\} \rangle$ 。并写 $\in_v$ 表示 v 上的属于关系（即等于 $<_v$ ），写 (v) 表示 $(v, \in_v)$ ，则 $\mathfrak{M} = (V_{\omega})$ ，有：

引理 5.1.4 $A_Z \vdash \models_{(V_{\omega})} A_Z \setminus \text{Inf}$ 。

证明：分别考虑 (V_{ω}) 满足 $A_Z \setminus \text{Inf}$ 的六条公理。写 $\in'$ 表示 $\in_{V_{\omega}}$ 。如对外延公理，元逻辑中的集合满足外延公理，域 V_{ω} 中的元素满足外延公理。空集公理， $\emptyset \in' V_1 \in' V_{\omega}$ 。配对公理，对任意 V_{ω} 中的元素 x, y ，存在自然数 a, b 有 $x \in' V_a, y \in' V_b$ ，令 $c = \max(a, b)$ ，则 $x \in' V_c \wedge y \in' V_c$ ，有 $\{x, y\} \in' V_{c^+} \in' V_{\omega}$ 。并集公理，任意 $x \in' y \in' V_a$ 有 $x \in' V_a$ ，有 $\bigcup y \in' V_{a^+}$ ²。等。□

引理 5.1.5 $A_Z \vdash \models_{(V_{\omega \cdot 2})} A_Z$ 。

¹元逻辑的选择取决于我们需要讨论构造何种目标逻辑的模型。比如显然证明更高层级的存在需要更强的集合论。

²如章节 5.1.3 所示，更精确的，这里存在 $b \in' a$ 有 $x \in' V_b$ ，则 $\bigcup y \in' V_a$ 。

证明：其它公理同上。考虑无限公理，由归纳有对任意序数 $a \in' V_{a^+}$ ，即 $\omega \in' V_{\omega+1} \in' V_{\omega \cdot 2}$ ($\in'$ 为 $\in_{V_{\omega \cdot 2}}$)。□

这里有两点值得注意。一是对于 $Z \setminus Inf$ 的模型，元逻辑中 V_ω 的存在依赖 ω 的存在，继而依赖无限公理。类似对于 Z ， $V_{\omega \cdot 2}$ 的存在依赖替换公理模式。或更精确的， A_Z 作为元逻辑（公理集）可证明 $Z \setminus Inf$ 的一致性， $A_Z \cup Rep$ 可证明 Z 的一致性¹。这符合任意足够强的公理集不能证明自身的一致性，即哥德尔第二不完备定理。二， (V_ω) 是公理集 $A_Z \setminus Inf; \neg Inf$ 的模型（因在有限集合上有限次的使用幂集运算不能得到无限集合），而 $(V_{\omega \cdot 2})$ 是 A_Z 的模型，这证明了无限公理相对于剩下的公理是独立的。类似的， $(V_{\omega \cdot 2})$ 为 $A_Z; \exists \alpha \in Rep \neg \alpha$ （因归纳可见任意序数 $a \notin V_a$ ，即 $\omega \cdot 2$ 不属于 $V_{\omega \cdot 2}$ ），而 ZF 的模型是 $A_Z \cup Rep$ 的模型，从而在如下更强的元逻辑下，证明了替换公理模式的独立性。

对此，对于 ZF 的模型，图中没有给出上界。但类似 $Z \setminus Inf$ 模型的存在需要无限公理，Z 模型的存在需要替换公理模式，ZF 模型的存在需要可证明更大序数存在的公理。这类用于证明更大序数 / 基数存在的公理被称为大基数公理。

最后， V 的直观性还为承认替换公理模式提供了支持。其中，对任意集合 s ，写 $\bigcup^0 s = s$ ，对任意自然数 n ， $\bigcup^{n+1} s = \bigcup \bigcup^n s$ ：

引理 5.1.6 $A_Z \cup Rep \vdash \exists m \models_{(m)} A_Z \wedge V_\omega \notin m$ 。

证明²：写 $\in'$ 为 $\in_m$ 。令 $m = \bigcup_{n \in' \omega} \wp^n(\omega)$ 。类似引理 5.1.4，结合由 (5.1.1) 可得任意 $\wp^n(\omega)$ 为传递集，可证模型满足 $A_Z \setminus Inf$ 。而 $\omega \in' \wp(\omega) \in' m$ 。还需证明 $V_\omega \notin m$ 。由 V_ω 定义有 $\bigcup V_\omega = V_\omega$ ，则任意 $\bigcup^n V_\omega = V_\omega$ 。但对任意 $x \in' m$ ，存在 n 有要么 $\bigcup^n x = \emptyset$ 要么 $\bigcup^n x = \omega$ 。有 $x \neq V_\omega$ 。□

则 A_Z 不能证伪上述 (m) 的存在。类似的，存在 A_Z 的模型有 $\{\wp^0(\emptyset), \wp^1(\emptyset), \dots\}$ 不在其域中，即该集合亦不可被 A_Z 证明存在。

5.1.3 正则公理³

本质上，正则公理限制任意集合在结构 V 中，或全部集合构成的类等于 V ，或：

定理 5.1.7 $A_{ZF} \vdash \forall x \exists a \text{ isOrd}(a) \wedge x \in V_a$ 。

注意不同于（除外延公理外的）其它公理为证明某类集合的存在，正则公理限制被认为无用且反直觉的集合存在。直观的说，这里指两方面。一是 V 中已含有足够的元素使我们可以描述几乎全部（数学）对象，如章节 4.2 所见， $V_{\omega+1}$ 有足够元素对应整数（及主要运算）， $V_{\omega+2}$ 足够处理实数⁴。二，从物体和物体合集的角度认识属于关系可见其不应成环。即一个物体所在的合集不应构成该物体（的一部分）。如最基本的：

引理 5.1.8 $A_{ZF} \vdash \forall x x \notin x$ 。

证明：假设存在 $x \in x$ ，存在 $a = \{x\}$ ，则由选择公理有 $x \cap a = \emptyset$ ，但由假设有 $x \in x \cap a$ 。矛盾。□

¹即在引理 5.1.5 基础上 $A_Z \cup Rep \vdash (\models_{(V_{\omega \cdot 2})} A_Z) \wedge \exists b b = V_{\omega \cdot 2}$ 。

²参考 Thoralf Skolem. “Some remarks on axiomatized set theory”. In: From Frege to Gödel. Ed. by Jean Van Heijenoort. Cambridge: Harvard University Press, 1967, pp. 290–301。

³证明部分参考 H.B. Enderton. Elements of Set Theory. Elsevier Science, 1977, pp. 200–206。

⁴Wikipedia contributors. Von Neumann universe — Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/w/index.php?title=Von_Neumann_universe&oldid=1068449716. 2022。

注意区别上述引理与罗素悖论¹。解决罗素悖论要求不存在集合 $\{x : x \notin x\}$ ，而非要求任意集合 x 有或没有 $x \notin x$ 。特别的，增加公理不能解决原公理集的悖论，即不能使不一致的公理集变得一致。

更一般的：

引理 5.1.9 $A_{ZF} \vdash \neg \exists f \text{ isFunction}(f) \wedge \text{dom } f = \omega \wedge \forall n \in \omega f(n^+) \in f(n)$ 。即不存在无限递减链。

证明：若存在这样的 f ，考虑 $a = \text{ran } f$ ，对任意 $x \in a$ 存在 n 满足 $x = f(n)$ ，而 $f(n^+) \in x$ ，有 $f(n^+) \in x \cap a$ 。与正则公理矛盾。 $\square$

现考虑定理 5.1.7。定义：

- 对任意 y ， $\text{rank}(y) = \text{least}_{O_X}(y \subseteq V_x)$ ，即也为最小序数 x 有 $y \in V_{x^+}$ 。称其为 y 的秩（若存在）²。
- 写 $\text{isGrounded}(y)$ 表示 $\exists r = \text{rank}(y)$ 。定理 5.1.7 即 A_{ZF} 下 $\forall y \text{ isGrounded}(y)$ 。
- 对任意 x ， $TC_0(x) = x$ ，对任意 $n \in \omega$ ， $TC_{n^+}(x) = \bigcup TC_n(x)$ 。 $TC(x) = \{y : \exists n \in \omega y = TC_n(x)\}$ ，即 $\bigcup_{n \in \omega} TC_n(x)$ 。称其为 x 的传递闭包。可见 $x \subseteq TC(x)$ 且 $TC(x)$ 为传递集。

我们将证明：

定理 5.1.10 $A_Z \cup \text{Rep} \vdash \text{Regularity} \leftrightarrow \forall y \text{ isGrounded}(y)$ 。

首先：

引理 5.1.11

- $A_Z \vdash a \in b \wedge \text{isGrounded}(b) \rightarrow \text{isGrounded}(a) \wedge \text{rank}(a) \in \text{rank}(b)$ 。
- $A_Z \vdash \forall x \in a \text{ isGrounded}(x) \rightarrow \text{isGrounded}(a) \wedge \text{rank}(a) = \bigcup \{y : \exists x \in a y = \text{rank}(x)^+\}$ 。

证明：

- 由 $\text{isGrounded}(b)$ 有存在 y 满足 $b \in V_{y^+}$ ，由 $a \in b$ 有存在最小 $x \in y \wedge a \in V_{x^+}$ 。
- 令 $u = \bigcup \{y : \exists x \in a y = \text{rank}(x)^+\}$ ，由对任意 $x \in a$ 有 $\text{rank}(x) \in u$ 有 $x \in V_u$ ，有 $a \in V_{u+1}$ ，即 $\text{isGrounded}(a) \wedge \text{rank}(a) \subseteq u$ 。而由 a 部分有任意 $x \in a$ 有 $\text{rank}(x) \in \text{rank}(a)$ ，即 $\text{rank}(x)^+ \subseteq \text{rank}(a)$ ，则 $u \subseteq \text{rank}(a)$ 。 $\square$

定理 5.1.10 的证明：

- 正方向。若存在 y 有 $\neg \text{isGrounded}(y)$ ，存在 $b = TC(\{y\})$ ，存在 $a = \{u \in b : \neg \text{isGrounded}(u)\}$ 。而 $y \in a$ 有 $a \neq \emptyset$ ，则由正则公理存在 $x \in a$ 有 $x \cap a = \emptyset$ ，由 b 为传递集有对任意 $z \in x$ 有 $z \in b$ ，则 $\text{isGrounded}(z)$ 因否则 $z \in x \cap a$ ，但由上述引理 b 有 $\text{isGrounded}(x)$ 。矛盾。
- 反方向。对任意 $a \neq \emptyset$ ，存在 $u \in a$ 满足 $\text{rank}(u) = \text{least}_{O_X}(\exists v \in a x = \text{rank}(v))$ 。则对任意 $z \in u$ ，由上述引理 a 有 $\text{rank}(z) \in \text{rank}(u)$ ，则 $z \notin a$ ，即 $u \cap a = \emptyset$ 。 $\square$

定理 5.1.7 的证明：即定理 5.1.10 正方向。 $\square$

写 $\min_{\in a}$ 表示 $\min_{\in_a} a$ 。有：

¹ 章节 4.1.2。

² 可对比章节 3.1.6 中对秩的定义。

引理 5.1.12 $A_{ZF} \vdash s \neq \emptyset \rightarrow \exists x x = \min_{\in} s$ 。

证明¹：类似引理 4.3.19 c 证明。考虑任意 $b \in s$ ，若 $b \cap s = \emptyset$ ，则 $b = \min_{\in} s$ 。否则令 $c = TC(b) \cap s$ ， $c \neq \emptyset$ ，则由正则公理存在 $x \cap c = \emptyset$ 。则 $x \cap s = \emptyset$ ，因否则令 $y \in x \wedge y \in s$ ，有 $y \in TC(b)$ 有 $y \in x \cap c$ ，矛盾。即 $x = \min_{\in} s$ 。□

推论 5.1.12.1 $A_{ZF} \vdash WF(s, \in_s)$ 。

最后，如章节首所述，我们将最终证明选择公理的独立性，这里首先给出一个与选择公理否定形式相关（但反面）的引理。定义：

- 对任意 s, t, f ， $isIsomorphism_{\in}(f, s, t)$ 表示 $isIsomorphism(f, \langle s, \in_s \rangle, \langle t, \in_t \rangle)$ ，称 f 为 s, t 上的 $\in$ 同构。
- 对任意 s, f ， $isAutomorphism_{\in}(f, s)$ 表示 $isIsomorphism_{\in}(f, s, s)$ ，称 f 为 s 上的 $\in$ 自同构。

则：

引理 5.1.13 $A_Z \vdash isTransitiveSet(s) \wedge isTransitiveSet(t) \wedge isIsomorphism_{\in}(f, s, t) \rightarrow isIdentity_{\mathbb{F}}(f)$ 。亦可见可得 $s = t$ 。

证明：对 s 中元素 x （在 $\in_s$ ）上做归纳。令 $y = f(x)$ 。归纳假设任意 $z \in x$ 有 $f(z) = z$ 。结合 $\in$ 同构有 $z = f(z) \in f(x) = y$ ，则 $x \subseteq y$ 。同时对任意 $w \in y$ 有存在 $z \in x$ 有 $f(z) = w$ ， $f(z) \in f(x)$ 有 $z \in x$ 有 $f(z) = z$ ，即 $w = z \in x$ ，则 $y \subseteq x$ 。□

现简述上述引理和选择公理否定形式的关系。由正则公理有 A_{ZF} 的任意模型 (m) 域 m 是传递集（ $\in$ 为元逻辑中的 $\in_m$ 关系）。代入上述引理取 $t = s = m$ ，则 m 上的任意 $\in$ 自同构均为恒等函数。若非如此，则存在 $u, v \in m$ 有 $v = f(u)$ 且 $u \neq v$ ，但 u, v 在 m 中（和任意元素）的任意属于关系均一致，则 u, v 在 (m) 中是不可分辨的。则可见没有选择函数可从 $\{u, v\}$ 中选出唯一元素（因若 $h(\{u, v\}) = u$ ，则 $u \in \{h(\{u, v\})\}$ 而 $v \notin \{h(\{u, v\})\}$ ）。而尝试找到类似不可分辨性是我们证明选择公理否定形式相对一致性的关键²。

5.2 内模型

在讨论选择公理否定形式之前，我们首先来看另一种集合层级结构，并在此基础上证明 ZF 不能证伪选择公理。

5.2.1 可构造全集

定义：

- 对任意序数 a ，层级 $L_a = \bigcup \{y : \exists b \in a y = \text{def}(L_b)\}$ ，或 $L_a = \bigcup_{b \in a} \text{def}(L_b)$ 。其中：

$$\text{def}(v) = \{w \in \wp(v) : \exists s, c_f \text{ isSentence}_{\mathcal{L}_{S, x, c_1 \dots}}(s) \wedge c_f : \ulcorner c_1 \dots \urcorner \mapsto v \wedge w = \{x \in v : \models'_{(v, \in_v, x, c_1, \dots)} s\}\}$$

有语言 $\mathcal{L}_{S, x, c_1 \dots}$ 为增加了常量符号 x 和符号集 $c_1 \dots = \{c_1, c_2, \dots\}$ 的集合论语言。上面的约束变量 c_f 表示由 $\ulcorner c_1 \dots \urcorner$ （中 s 涉及的常量，我们的编码满足 s 大于任意 s 中涉及的符号编码）到 v 的函数

¹证明自 Thomas Jech. Set Theory: The Third Millennium Edition. Springer, 2003, p. 64。

²见章节 5.3.7。

映射。这里 $(v, \in_v, x, c_1, \dots)$ 表示解释 $\langle v, \{\langle \ulcorner \in \urcorner, \in_v \rangle, \langle \ulcorner x \urcorner, x \rangle\} \cup c_f \rangle$ 。注意我们可按需选择常量符号以使含义清晰，并保持解释中的粗体对应关系。最后，我们（总是）进一步简写解释如 $(v, \in_v, x, c_1, \dots)$ 为 $(v, \in_v)$ （继而 (v) ）并改所满足语句中的 $x, c_1, \dots$ 为 $x, c_1, \dots$ 以表达相同含义，例见引理 5.2.1 证明。

直观的说， $w \in \text{def}(v)$ 表示 $w \subseteq v$ 且 w 可由某 s 和 c_f 定义。或对增加了常量符号集 $c_1, \dots$ 的集合论语言，给定由 v 为域， w 为域 v 中满足式 $\sigma[x]$ 的元素集合。有这里的 $s = \ulcorner \sigma[x/x] \urcorner$ ， c_f 决定 σ 中 $c_1, \dots$ 在 v 中的解释。当 σ 中不含 $(c_1, \dots)$ 中常量符号时，可见 w 即为（无扩展的集合论语言下） v 由式 σ 定义的子集。可见该结构中的任意非空集合都是由处于更底层的集合定义 / 构造的，因此称由全部 L_a 构成的结构 L 为可构造全集。

类似有等价的：

- $L_0 = \emptyset$ 。
- 对任意后继序数 $a = b^+$ ， $L_a = \text{def}(L_b)$ 。
- 对任意极限序数 a ， $L_a = \bigcup \{y : \exists b \in a, y = L_b\}$ 。

也有：

引理 5.2.1 $A_Z \vdash \text{isOrd}(a) \rightarrow \text{isTransitiveSet}(L_a)$ 。

证明：同引理 5.1.1 证明。其中相对式 (5.1.1)，这里需要 $\text{isTransitiveSet}(s) \rightarrow \text{isTransitiveSet}(\text{def}(s))$ 。因对任意 $u \in \text{def}(s)$ 有 $u \subseteq s$ ，对任意 $v \in u$ 有 $v \in s$ 有 $v \subseteq s$ ，则 $\sigma[x]$ 为 $x \in v$ 定义 $v \in \text{def}(s)$ 。或可见 $v = \{x \in s : \vdash_{(s)} x \in v\} \in \text{def}(s)$ 。□

需注意为有 $v = \{x \in s : \vdash_{(s)} x \in v\}$ ，对任意 $x \in s$ ，要求 $(\vdash_{(s)} x \in v) \leftrightarrow x \in v$ ，此处成立因任意 $x \in v$ 有 $x \in s$ 。我们称这样的语句 $y \in x$ 为绝对的。

更具体的：

- 对任意 m ，式 $\phi_{[x_1, \dots, x_n]}$ 称 ϕ 在 m 上是绝对的若 $\forall x_1, \dots, x_n \in m ((\vdash_{(m)} \phi) \leftrightarrow \phi)$ 。可见直观的，绝对性指在 (m) 中得到的语句 $\phi(x_1, \dots, x_n)$ 的真值不受 m 外元素的影响。比如语句 $\forall x, x = \emptyset$ 在 $(\{\emptyset\})$ 上不是绝对的，因其在 $(\{\emptyset\})$ 中为真，外为假。

另一个有用的例子：

引理 5.2.2

- $A_Z \vdash \text{isOrd}(a) \wedge \text{isOrd}(b) \wedge a \in b \rightarrow L_a \in L_b$ 。
- $A_Z \vdash \text{isOrd}(a) \wedge \text{isOrd}(b) \wedge a \in b \rightarrow L_a \subseteq L_b$ 。

证明：类似引理 5.1.2，其中任意 s 有 $s \in \text{def}(s)$ 可由绝对的 $\top$ 定义。□

同时可见：

引理 5.2.3 $A_Z \vdash \forall a (\text{isOrd}(a) \rightarrow L_a \subseteq V_a)$ 。特别的， $A_Z \vdash \forall a \in \omega, L_a = V_a$ 。

证明：对 a 归纳。对后者 $a \in \omega$ ，因任意 $x \in V_a$ 有 x 含有限个元素 $y_1, \dots, y_n$ ，而对任意 $1 \leq i \leq n$ ， $y_i \in x$ 有 $y_i \in V_a$ 有 $\text{rank}(y_i) < \text{rank}(x)$ 有存在 $b_i \in a$ 而 $y_i \in V_{b_i}$ 。则存在 $b \in a$ 对任意 $1 \leq i \leq n$ ， $y_i \in V_b$ ，由归纳 $y_i \in L_b$ ，则因 n 有限存在绝对的 $\phi[x]$ 为 $\bigvee_{1 \leq i \leq n} x = y_i$ （若 $n \neq 0$ 否则 $\perp$ ）定义 x 。□

本质上，我们将证明由可构造全集构成的模型满足 A_{ZFC} ，而如上可构造全集为冯·诺伊曼全集的一部分。又可见绝对性为相互确定语句在部分和整体上的真值提供了有效的工具，因此我们将给出更一般的绝对性性质。

- 递归定义 Δ_0 式，仍仅（需）考虑仅由 $\neg \vee \exists$ 组成的式。原子式、即任意 $x \in y$ 、 $x = y$ 为 Δ_0 式。若 φ 为 Δ_0 式， $\neg \varphi$ 、 $\varphi \vee \psi$ 为 Δ_0 式。若 $\varphi_{[u_1 \dots n, x]}$ 为 Δ_0 式， $\exists x \in y \varphi$ 为 Δ_0 式。¹

定理 5.2.4 对任意 Δ_0 式 ϕ ， $A_Z \vdash \text{isTransitiveSet}(m) \rightarrow (\phi \leftrightarrow (\vdash_{(m)} \phi))$ 。即 Δ_0 式在传递集上是绝对的。

证明：对式 ϕ 的结构归纳。显然原子式是绝对的。归纳假设 $\varphi \psi$ 是绝对的，也可见 $\neg \varphi$ ， $\varphi \vee \psi$ 是绝对的。最后对 ϕ 为 $\exists x \in y \varphi(u_1 \dots n, x)$ ，若 $\vdash_m \phi$ ，则存在 $v \in m$ 有 $\vdash_m \varphi(u_1 \dots n, v)$ ，则由归纳假设有 $\varphi(u_1 \dots n, v)$ ，有 ϕ 。若 ϕ ，则存在 $v \in y$ 有 $\varphi(u_1 \dots n, v)$ ，有 $\vdash_{(m)} \varphi(u_1 \dots n, v)$ ，由 m 是传递的， $v \in m$ ，则 $\vdash_{(m)} \phi$ 。□

直观的说， Δ_0 式保证了给定模型中参数后的语句真值不因量化受模型外元素的影响。同时注意上述定义可扩展到类上，即 Δ_0 式在传递的子类上也是绝对的。但直接的扩展需同时扩展域为类的模型及其下的 $\vdash$ 关系，我们将通过抬升使工作于元逻辑中规避这些额外的形式化定义，见 (5.2.3)。

我们来看几个（可）等价于 Δ_0 式而绝对的例子：

示例 5.2.5

- Z 下 $\phi[s]$ 等于 $s = \emptyset$ ，因 $A_Z \vdash \phi \leftrightarrow \forall x \in s x \neq x$ ，其中 $\leftrightarrow$ 右侧为 Δ_0 式。
- Z 下 $\phi[p, s, t]$ 等于 $p = \{s, t\}$ ，因 $A_Z \vdash \phi \leftrightarrow (\forall x \in p x = s \vee x = t) \wedge s \in p \wedge t \in p$ 。
- Z 下 $\phi[s, t]$ 等于 $s \subseteq t$ ，因 $A_Z \vdash \phi \leftrightarrow \forall x \in s x \in t$ 。
- Z 下 $\phi[s]$ 等于 $\text{isTransitiveSet}(s)$ ，因 $A_Z \vdash \phi \leftrightarrow \forall x \in s x \subseteq s$ 。
- ZF 下 $\phi[s]$ 等于 $\text{isOrd}(s)$ ，因 $A_{ZF} \vdash \phi \leftrightarrow \text{isTransitiveSet}(s) \wedge (\forall x y \in s x \in y \vee x = y \vee y \in x) \wedge \forall x y z \in s (x \in y \wedge y \in z \rightarrow x \in z)$ 。注意这里需通过正则公理保证良序关系中的反自反性和任意子集 $\in_s$ 上最小元素存在。
- ZF 下 $\phi[s]$ 等于 $\text{isLimitOrdinal}(s)$ ，因 $A_Z \vdash \phi \leftrightarrow \text{isOrd}(s) \wedge s \neq 0 \wedge \forall x \in s \exists y \in s x \in y$ 。
- ZF 下 $\phi[s]$ 等于 $s = \omega$ ，因 $A_Z \vdash \phi \leftrightarrow \text{isLimitOrdinal}(s) \wedge \forall x \in s \neg \text{isLimitOrdinal}(x) \wedge \forall y \in x \neg \text{isLimitOrdinal}(y)$ 。其中对 x 的条件表示要求 x 为自然数。
- Z 下 $\phi[u, s]$ 等于 $u = \bigcup s$ ，因 $A_Z \vdash \phi \leftrightarrow (\forall x \in u \exists y \in s x \in y) \wedge (\forall y \in s \forall x \in y x \in u)$ 。
- Z 下 $\phi[i, s, t]$ 等于 $i = s \cap t$ ，因 $A_Z \vdash \phi \leftrightarrow (\forall x \in i x \in s \wedge x \in t) \wedge \forall x \in s (x \in t \rightarrow x \in i)$ 。
- Z 下 $\phi[p, s, t]$ 等于 $p = \langle s, t \rangle$ ，因 $A_Z \vdash \phi \leftrightarrow (\forall x \in p x = \{s\} \vee \{s, t\}) \wedge (\exists x y \in p x = \{s\} \wedge y = \{s, t\})$ 。
- Z 下 $\phi[p, s, t]$ 等于 $p = s \times t$ ，因 $A_Z \vdash \phi \leftrightarrow (\forall z \in p \exists x \in s, y \in t z = \langle x, y \rangle) \wedge (\forall x \in s, y \in t \exists z \in p z = \langle x, y \rangle)$ 。
- Z 下 $\phi[x, r]$ 等于 $x \in \text{dom } r$ ，因 $A_Z \vdash \phi \leftrightarrow \exists p \in r \exists u \in p \exists y \in u p = \langle x, y \rangle$ 。同理对 $\text{ran } r$ 。
- Z 下 $\phi[x, r]$ 等于 $\forall x \in \text{dom } r \varphi$ 若 φ 为 Δ_0 ，因 $A_Z \vdash \phi \leftrightarrow \forall p \in r \forall u \in p \forall y \in u (p = \langle x, y \rangle \rightarrow \varphi)$ 。同理对 $\exists$ 及 $\text{ran } r$ 。
- Z 下 $\phi[d, r]$ 等于 $d = \text{dom } r$ ，因 $A_Z \vdash \phi \leftrightarrow (\forall x \in d x \in \text{dom } r) \wedge (\forall x \in \text{dom } r x \in d)$ 。同理对 $\text{ran } r$ 。
- Z 下 $\phi[r]$ 等于 $\text{isRelation}(r)$ ，因 $A_Z \vdash \phi \leftrightarrow \forall p \in r \exists x \in \text{dom } r, y \in \text{ran } r p = \langle x, y \rangle$ 。
- Z 下 $\phi[f]$ 等于 $\text{isFunction}(f)$ ，因 $A_Z \vdash \phi \leftrightarrow \text{isRelation}(f) \wedge \forall x \in \text{dom } f \forall y_1 y_2 \in \text{ran } f (\langle x, y_1 \rangle \in f \wedge \langle x, y_2 \rangle \in f \rightarrow y_1 = y_2)$ 。
- Z 下 $\phi[g, f, s]$ 等于 $g = f \upharpoonright s$ ，因 $A_Z \vdash \phi \leftrightarrow \text{isFunction}(g) \wedge g \subseteq f \wedge \forall x \in s (x \in \text{dom } f \rightarrow x \in \text{dom } g) \wedge \forall x \in \text{dom } g x \in s$ 。

但注意 $p = \wp(s)$ 不是 Δ_0 的，因此如 $\exists x \subseteq s \varphi$ 不是 Δ_0 的。事实上 $\wp(s)$ 不是绝对的，直观的说，其非绝对性使我们不同模型间的层级有了宽窄的区别，如 L 可认为是窄于 V 的，更具体的可见引理 5.2.9 中关于幂集公理的证明。同时在章节 5.3 中由迫力法构造的模型一般是宽于原模型的。

以此：

¹对比可见其定义与章节 3.2.3 的算术 Δ_0 式（即算术基本式）统一。

引理 5.2.6 $A_{ZF} \vdash isOrd(a) \rightarrow a = \{x \in L_a : isOrd(x)\}$ 。

证明：对 a 归纳。即需分别证明 $a \subseteq L_a$ 及 $\forall u (isOrd(u) \wedge u \in L_a \rightarrow u \in a)$ 。因对任意 $b \in a$ 有 $b \subseteq L_b$ 有 $b \in def(L_b)$ 由在 L_b 上绝对的 $isOrd(x)$ 定义，即 $b = \{x \in L_b : isOrd(x)\} = \{x \in L_b : \models_{(L_b)} isOrd(x)\}$ 。有 $a \subseteq L_a$ 。另一方面若存在序数 u 有 $a \subseteq u \wedge u \in L_a$ ，由传递集有 $a \in L_a$ ，则存在序数 $b \in a$ 有 $a \in def(L_b)$ 则 $a \subseteq L_b$ ，则 $b \in L_b$ ，与归纳假设矛盾。□

推论 5.2.6.1 $A_{ZF} \vdash isOrd(a) \rightarrow a \in L_{a^+}$ 。

证明：由上述引理 $a \subseteq L_a$ ，同上述引理 $a \in def(L_a) = L_{a^+}$ 。□

另一方面：

- 对式 $\phi_{[x_{1...n}]}$ ，对于 $\forall x_{1...n} \in m ((\models_{(m)} \phi) \leftrightarrow \phi)$ ，从模型的角度说，我们也称 (m) 反射 ϕ 。如当 ϕ 为语句时，其真值可直接在 (m) 下确定，或 (m) 反映 / 反射了 ϕ 的真值。

同样作为确定真值的一个有效手段，关于反射性，我们有：

定理 5.2.7 (反射原理) 对任意式 $\phi_{[x_{1...n}]}$ ， $A_{ZF} \vdash \exists m \supseteq m_0 \forall x_{1...n} \in m ((\models_{(m)} \phi) \leftrightarrow \phi)$ ^{注1}。

为此：

引理 5.2.8 对任意至多含 v 个自由变量的有限谓词式组 $\pi_{1...n}$ ：

$$A_{ZF} \vdash \exists m \supseteq m_0 (\bigwedge_{0 < i \leq n} \bigwedge_{1 \leq j < v} \forall u_{1...v-1} \in m (\exists x \pi_i(u_{1...j-1}, x, u_{j...v-1}) \rightarrow \exists x \in m \pi_i(u_{1...j-1}, x, u_{j...v-1})))$$

注意这里的定理模式是以式组的形式给出的。

证明：构造 m 。已有 m_0 ，对任意 $k \in \omega$ ，在 m_k 的基础上，我们递归定义 m_{k+1} 。直观的说，为任意 $0 < i \leq n$ ， $0 < j < v$ ，任意 $u_{1...m-1} \in m_k$ ，我们将满足 $i: \pi_i(u_{1...j-1}, x, u_{j...v-1})$ 的 x 加入到 m_{k+1} 中，则可见 $m = \bigcup_{k \in \omega} m_k$ 满足定理要求。

然而具体的形式化要更复杂一些。其中最主要的问题在于满足 $\xi[x] = \pi_i(u_{1...j-1}, x, u_{j...v-1})$ 的元素类不一定构成集合，通过替换公理模式构造 m_k 序列要求函数类的值是集合，这里需要通过斯科特技巧得到集合 s 满足任意 $x \in s$ 有 ξ 。

对任意 $\xi[y]$ 可定义式 $\zeta[x]$ 为 $isOrd(x) \wedge \exists y x = rank(y) \wedge \xi$ ， ζ 定义了满足 ξ 的元素的秩的类。而由正则公理，满足 ξ 的元素总是有秩的，因此存在满足元素满足 ξ 则存在序数满足 ζ 。而类似引理 4.3.19 b (及其证明)，有对任意 $\rho[x]$ ：

$$A_Z \vdash \exists a isOrd(a) \wedge \rho[x/a] \rightarrow \exists a a = least_{O_x}(\rho)$$

即非空的序数类存在最小序数。则可构造 $s = (\exists y \xi)? \{y \in V_{least_{O_x}(\zeta)} : \xi\} : \emptyset$ ，有 $x \in s$ 满足 ξ 。

写 $s_{i,j,u_{1...m-1}}$ 表示在 $i j u_{1...m-1}$ 下的上述 s 。 $t_{k,i,j} = \{y : \exists u_{1...m-1} \in v_k y = s_{i,j,u_{1...m-1}}\}$ 。则：

$$m_{k+1} = m'_{k+1} = \bigcup_{0 < i \leq n} \bigcup_{0 < j < m} \bigcup t_{k,i,j} \quad (5.2.1)$$

则可定义 m 为 ω 上以该递归方式定义的函数的值域。□

¹写 $x \supseteq y$ 表示 $y \subseteq x$ ， $x \supset y$ 表示 $y \subset x$ 。

若在 (5.2.1) 中选择 $m_{k+1} = TC(m'_{k+1})$, 则得到的 m 为传递集。而若选择 m_{k+1} 为最小 $V_a \supseteq m_{k+1}$, 则得到的 m 为某 V_b 有 b 为极限序数。另外, 若同时有选择公理, 我们可以将满足 i 的一个 x 加入到 m_{k+1} 中, 则因 $v \cdot n$ 有限, 若 m_0 可数, 归纳可见 m_{k+1} 可数, m 可数。技术上, 对于 (5.2.1) 中的 $t_{k,i,j}$ 有选择函数 $f_{k,i,j}$, 则替换 $\bigcup t_{k,i,j}$ 为 $ran f_{k,i,j}$ 可实现上述仅将一个 x 加入到 m_{k+1} 中。

定理 5.2.7 (反射原理) 的证明: 构造 $\pi_{1\dots n}$ 有 $\phi \in \pi_{1\dots n}$ 。且有 (仍考虑由 $\neg \vee \exists$ 构成的语言) 若 $\neg \phi \in \pi_{1\dots n}$, $\phi \in \pi_{1\dots n}$, $\phi \vee \psi \in \pi_{1\dots n}$ 有 $\phi \in \pi_{1\dots n}$ 及 $\psi \in \pi_{1\dots n}$, $\exists x \phi \in \pi_{1\dots n}$ 有 $\phi \in \pi_{1\dots n}$ 。则可见有最小 $n \in \omega$ 有 $\pi_{1\dots n}$ 满足以上条件。应用上述引理, 存在 $m \supseteq m_0$ 满足:

$$\forall u_{1\dots v-1} \in m (\exists x \pi_i(u_{1\dots j-1}, x, u_{j\dots v-1}) \rightarrow \exists x \in m \pi_i(u_{1\dots j-1}, x, u_{j\dots v-1})) \quad (5.2.2)$$

其中 v 为 $\pi_{1\dots n}$ 中语句最大自由变量数。

我们将对式的结构做归纳证明 m 反射任意 $\pi_i \in \pi_{1\dots n}$, 则由 $\phi \in \pi_{1\dots n}$ 有 m 反射 ϕ 。显然 m 反射原子语句。同样由归纳假设易见若 m 反射 φ , m 反射 $\neg \varphi$, 对 $\vee$ 同理。最后归纳假设 m 反射 $\varphi[x_{1\dots v}]$, 证明对任意 $1 \leq j \leq v$, m 反射 $\exists x \varphi[x_j/x]$ 。即对任意 $u_{1\dots v-1} \in m$, 写 $\varphi' = \varphi[x_1/u_1] \dots [x_{j-1}/u_{j-1}][x_j/x][x_{j+1}/u_{j+1}] \dots [x_v/u_{v-1}]$:

$$\begin{aligned} \exists x \varphi' &\leftrightarrow \exists x \in m \varphi' && \text{正方向即 (5.2.2)} \\ &\leftrightarrow \exists x \in m \models_{(m)} \varphi' && \text{由归纳假设} \\ &\leftrightarrow \models_{(m)} \exists x \varphi' && \text{由 } \models \text{ 定义} \quad \square \end{aligned}$$

同样, 若在 (5.2.1) 中选择 m_{k+1} 为传递集 / 最小 V_a :

推论 5.2.8.1 对任意式 $\phi[x_{1\dots n}]$, $A_{ZFC} \vdash \exists m \supseteq m_0 \wedge isTransitiveSet(m) \wedge \forall x_{1\dots n} \in m ((\models_{(m)} \phi) \leftrightarrow \phi)$ 。

推论 5.2.8.2 对任意式 $\phi[x_{1\dots n}]$, $A_{ZFC} \vdash \exists b isLimitOrdinal(b) \wedge m_0 \subseteq V_b \wedge \forall x_{1\dots n} \in V_b ((\models_{(V_b)} \phi) \leftrightarrow \phi)$ 。

而若有选择公理, 上述 m 可以在 m_0 可数时可数^{注1}。即:

推论 5.2.8.3 对任意式 $\phi[x_{1\dots n}]$:

$$A_{ZFC} \vdash isCountable(m_0) \rightarrow \exists m \supseteq m_0 isCountable(m) \wedge \forall x_{1\dots n} \in m ((\models_{(m)} \phi) \leftrightarrow \phi)$$

我们的目标是证明可构造全集构成的模型满足 A_{ZFC} 。显然, 如之前, 我们可以采用更强的集合论元逻辑以证明其中存在某 (L_κ) 为这样的模型²。但这意味着我们需要强调该更强理论的一致性, 而更强的理论意味着更多 (可能存疑) 的公理。因此这里证明另一种被称为相对一致性的结果, 即若 A_{ZF} 有模型 m_V , 则可构造 A_{ZF} 的模型 m_L 同时满足选择公理。或借助以上概念证明:

$$A_Z \vdash \exists m_V \models_{m_V} A_{ZF} \rightarrow \exists m_L \models_{m_L} A_{ZFC}$$

本质上, Z 为我们的元逻辑, m_L 为 m_V 中的可构造全集。我们将首先 (在元逻辑中) 构造 m_L , 然后证明 m_L 满足 A_{ZF} 。继而证明其满足可构造公理:

$$\forall x \exists a isOrd(a) \wedge x \in L_a \quad (constructibility)$$

即类似正则公理将集合限制在 V 中, 可构造公理将集合限制在 L 中。注意这里尽管 m_L 为 m_V 中的可构造全集, 我们不能直接得到其亦为 m_L 自身的可构造全集, 或者说, 我们需证明可构造性也是绝对的。最

¹事实上, 在可数的基础上, m 也可以进一步是传递的, 见定理 5.2.20。

²其中, κ 为其存在性独立于 ZF 的大基数 / 基数。见 Wikipedia contributors. Large cardinal — Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/w/index.php?title=Large_cardinal&oldid=1077380690. 2022。

后，我们由满足可构造公理可到满足选择公理。

其中，我们将看到称 m_L 为 ZF 在 m_V 中的内模型。

5.2.2 ZF 的内模型

- 在元逻辑中，对任意 ZF 的模型 $m_V = (v, \epsilon)$ ，令 $l = \{x \in v : \models_{m_V} \exists a \text{ isOrd}(a) \wedge x \in L_a\}$ ，写 ϵ_l 表示 $\epsilon|_l$ ，则可定义 / 构造 $\mathfrak{Q}(m_V)$ 为模型 (l, ϵ_l) 。写 $\mathfrak{Q}(m_V)[\forall]$ 表示其中的 l ，写 $\mathfrak{Q}(m_V)[\epsilon]$ 表示其中的 ϵ_l 。最后令（元逻辑中的） $l_a \in v$ 对应 m_V 中的 L_a 。

则首先我们给出元逻辑 Z 中模型 m_V 和 $\mathfrak{Q}(m_V)$ 间类似定理 5.2.4（及其证明）的绝对性性质：

$$A_Z \vdash \text{is}\Delta_0(\ulcorner \phi_{[x_1 \dots x_n]} \urcorner) \rightarrow \forall x_{1 \dots n} \in \mathfrak{Q}(m_V)[\forall] ((\models_{m_V} \phi) \leftrightarrow (\models_{\mathfrak{Q}(m_V)} \phi)) \quad (5.2.3)$$

这里有两点值得注意，一是 $\text{is}\Delta_0$ 判断其参数是否为 Δ_0 式的编码结果，这可行因我们工作于元逻辑中，如后面 $\models_{\mathfrak{Q}(m_V)} \phi$ 其处理的也为（可能扩展集合论语言后）语句的编码结果¹。二是上式按定义已隐含了 $\mathfrak{Q}(m_V)$ 的域在 m_V 中是传递的，即任意 $x \in y \in l_a$ 有 $x \in l_a$ 。

结合该绝对性性质：

引理 5.2.9 $A_Z \vdash (\models_{m_V} A_{ZF}) \rightarrow (\models_{\mathfrak{Q}(m_V)} A_{ZF})$ 。

证明：令 $m_L = \mathfrak{Q}(m_V)$ ，令 $v \in l_a$ $l \in l$ 按 $\mathfrak{Q}(m_V)$ 定义时给出的定义定义。逐一证明 m_L 满足 A_{ZF} 中的公理：

1. 外延公理。对任意 $st \in l$ ， $\models_{m_L} \forall x (x \in s \leftrightarrow x \in t)$ 。有 $\forall x \in l (x \in_l s \leftrightarrow x \in_l t)$ ，而由任意 L_a 传递， $x \notin_l s$ 有 $x \neq l$ ， t 同。结合 ϵ_l 的定义，则 $\forall x \in v (x \in x \leftrightarrow x \in t)$ ，即 $\models_{m_V} \forall x (x \in s \leftrightarrow x \in t)$ ，有 $\models_{m_V} s = t$ ，因 $s = t$ 为 Δ_0 绝对，由 (5.2.3) 有 $\models_{m_L} s = t$ 。
2. 空集公理。存在 e 满足 $\models_{m_V} e = \emptyset$ ，而 $\emptyset = 0 \in L_1$ （推论 5.2.6.1），有 $e \in l$ ，因 $e = \emptyset$ 绝对，由 (5.2.3) 有 $\models_{m_L} e = \emptyset$ 。
3. 配对公理。对任意 $st \in l$ ，存在 m_V 中的序数 a 有 $\models_{m_V} s \in L_a \wedge t \in L_a$ ，且存在 p 有 $\models_{m_V} p = \{s, t\}$ ，或 $\models_{m_V} p = \{x \in L_a : x = s \vee x = t\}$ 。因 $x = s \vee x = t$ 为 Δ_0 式（在 m_V 中的） L_a 上绝对，应用定理 5.2.4 有 $\models_{m_V} p = \{x \in L_a : \models_{(L_a)} x = s \vee x = t\}$ 。有 $\models_{m_V} p \in L_{a^+}$ ，则 $p \in l$ 。而因 $p = \{s, t\}$ 绝对（见示例 5.2.5），由 (5.2.3) 有 $\models_{m_L} p = \{s, t\}$ 。
4. 并集公理。对任意 $s \in l$ ，存在序数 a 有 $\models_{m_V} s \in L_a$ ，且存在 u 有 $\models_{m_V} u = \bigcup s$ ，或等价于 $\models_{m_V} u = \{x \in L_a : \exists y \in s x \in y\}$ ，则 $\models_{m_V} u \in L_{a^+}$ ， $u \in l$ ，则因 $u = \bigcup s$ 绝对，有 $\models_{m_L} u = \bigcup s$ 。
5. 幂集公理。对任意 $s \in l$ ，存在 p 有 $\models_{m_V} \{t \in \wp(s) : \exists x \text{ isOrd}(x) \wedge t \in L_x\}$ ，即有 p 对应 m_V 中（不严格）的 $\wp(s) \cap L$ 。则存在序数 a 有 $\models_{m_V} a = \bigcup \{least_{Ox}(t \in L_x) : t \in p\}$ 。则 $\models_{m_V} p \subseteq L_a$ 。即 $\models_{m_V} p = \{t \in L_a : t \subseteq s\}$ ，因 $t \subseteq s$ 绝对 $\models_{m_V} p \in L_{a^+}$ ， $p \in l$ 。而 $t \in p \leftrightarrow t \subseteq s$ 绝对有 $\forall t \in l \models_{m_L} t \in p \leftrightarrow t \subseteq s$ ，有 $\models_{m_L} \forall t (t \in p \leftrightarrow t \subseteq s)$ ，有 $\models_{m_L} p = \wp(s)$ 。
6. 分类公理模式。对任意 $s \in l$ ， $u_{1 \dots n} \in l$ ，式 $\rho[u_{1 \dots n}, x]$ ，我们需要证明存在 $t \in l$ 有 $\models_{m_L} t = \{x \in s : \rho\}$ 。而类似定理 5.2.7（反射原理）（及其证明），对任意式 $\phi_{[x_1 \dots x_n]}$ ，写 m_{L_a} 表示模型 (l_a, ϵ_{l_a}) ，有：

$$(\models_{m_V} \text{isOrd}(a_0)) \rightarrow \exists a (\models_{m_V} \text{isOrd}(a) \wedge a_0 \in a) \\ \wedge \forall x_{1 \dots n'} ((\models_{m_V} \langle x_{1 \dots n'} \rangle \in L_a^{n'}) \rightarrow ((\models_{m_L} \phi) \leftrightarrow (\models_{m_{L_a}} \phi)))$$

¹而注意定理 5.2.4 给出的可认为是定理模式，其需要采用定理模式因我们没有语句的有效性判定谓词，即不能定义 $P(x)$ 有 $A_Z \vdash \phi \leftrightarrow P(\ulcorner \phi \urcorner)$ 。此即为定理 3.2.18（塔斯基不可定义定理）。

- 即 m_{L_a} 反映了 ϕ 在 m_L 中的真值。取 a_0 有 $\models_{m_V} s \in L_{a_0} \wedge \bigwedge_{1 \leq i \leq n} u_i \in L_{a_0}$ 。则由上式取 ϕ 为 ρ , 存在 a 有 $(\models_{m_L} \rho) \leftrightarrow (\models_{m_{L_a}} \rho)$ 。而 $\models_{m_{L_a}} \rho$ 又等价于 $\models_{m_V} \models_{(L_a)} \rho$ 。则存在 $t \in v$ 有 $\models_{m_V} t = \{x \in s : \models_{(L_a)} \rho\}$, 即 $\models_{m_V} t = \{x \in L_a : x \in s \wedge \models_{(L_a)} \rho\}$ 。又因 $x \in s$ 绝对, 有 $\models_{m_V} t = \{x \in L_a : (\models_{(L_a)} x \in s) \wedge \models_{(L_a)} \rho\}$, 即 $\models_{m_V} t = \{x \in L_a : \models_{(L_a)} x \in s \wedge \rho\}$, 则 $\models_{m_V} t = L_{a^+}$ 。又由 t 的构造有 $\models_{m_L} \forall x (x \in t \leftrightarrow x \in s \wedge \rho)$, 即 $\models_{m_L} t = \{x \in s : \rho\}$ 。
7. 无限公理。存在 o 有 $\models_{m_V} o = \omega$, 而 $\omega \in L_{\omega^+}$, 因此 $o \in l$ 。结合归纳性绝对有 $\models_{m_L} isInductive(o)$ 。
8. 替换公理模式。对任意 $s \in l$, $u_1 \dots u_n \in l$, 式 $\rho[u_1 \dots n, y, z]$ 有 $\models_{m_L} \forall y \in s \exists! z \rho$ 。即对任意 $\models_{m_L} y \in s$, 即 $\models_{m_V} y \in s$, 存在 (唯一) z 满足 $\models_{m_L} \rho$ 。取 a_0 满足 $\models_{m_V} s \in L_{a_0} \wedge \bigwedge_{1 \leq i \leq n} u_i \in L_{a_0}$, 可以看到 $\models_{m_V} \exists a_z \exists a_0 isOrd(a_z) \wedge z \in L_{a_z}$ 。则存在 a 有 $\models_{m_V} a = \bigcup \{least_{O_X}(a_0 \subseteq x \wedge \exists z \in L_x \models_{(L_x)} \rho) : y \in s\}$, 则任意 z 有 $\models_{m_V} z \in L_a$ 。而 $l_a \in l_{a^+}$ 即 $l_a \in l$, 则 $\models_{m_L} z \in l_a$, 则应用 m_L 满足分类公理模式有 m_L 满足替换公理模式。
9. 正则公理。任意 $s \in l$, 存在 a 有 $\models_{m_V} s \in L_a$, 且由 m_V 满足正则公理存在 t 有 $\models_{m_V} t \in s \wedge t \cap s = \emptyset$ 。则由 L_a 传递, 有 $t \in l$ 。由 $s \cap t = \emptyset$ 绝对有 $\models_{m_L} t \cap s = \emptyset$ 。□

我们称上述的 $m_L = \mathfrak{M}(m_V)$ 为 ZF 的内模型。具体的, 称模型 $\mathfrak{M} = (n, \epsilon_n)$ 为集合论 T 在 $\mathfrak{M} = (m, \epsilon_m)$ 中的内模型若 i: $n \subseteq m$, $\epsilon_n = \epsilon_m \upharpoonright n^2$, 称 $\mathfrak{M}$ 为 $\mathfrak{M}$ 的子模型; ii: $\models_{\mathfrak{M}} T$; iii: $x \in y \in n \rightarrow x \in n$, 即可认为 n 是传递的, 或为 m 中的传递类; iv: 对任意 a , $(\models_{\mathfrak{M}} isOrd(a)) \leftrightarrow (\models_{\mathfrak{M}} isOrd(a))$ 。对于 iv 上述 m_L 成立因由推论 5.2.6.1 有任意 a 满足 $\models_{m_V} isOrd(a)$ 有 $a \in l$, 同时因 m_L 满足正则公理有 $isOrd(a)$ (在 m_L 和 m_V 之间) 是绝对的 (示例 5.2.5)¹。

最后, 结合哥德尔第二不完备定理, 我们有 ZFC 作为元逻辑不能证明 ZF 的一致性。注意这不同于 ZFC 的一致性蕴含 ZF 的一致性。其关键在于 ZF 的模型 (和 ZFC 的一样) 其存在性不可在 ZFC 中被证明。

5.2.3 可构造性的绝对性

我们的目标是:

引理 5.2.10 $A_Z \vdash (\models_{m_V} A_{ZF}) \rightarrow (\models_{\mathfrak{M}(m_V)} constructibility)$ 。

证明: 仍写 $m_L = \mathfrak{M}(m_V)$ 等。首先如上一小节末指出:

$$A_Z \vdash (\models_{m_V} isOrd(a)) \leftrightarrow (\models_{m_L} isOrd(a))$$

即 m_V 和 m_L 有相同序数。同时, 由可构造的递归定义有函数类 $y = L_x$ 表示对任意 m_V / m_L 中的序数 x , $y = L_x$ 。我们需要证明 $isOrd(x) \rightarrow y = L_x$ 是绝对的。但这里 $y = L_x$ 不是 Δ_0 式, 因此我们无法通过 (5.2.3) 证明其绝对性。

我们需对 Δ_0 式这种分类 / 分层方法进行扩展, 这里将不进行完整扩展而只引入证明所需的 Σ_1 Π_1 和 Δ_1 式。称 ϕ 为 Σ_1 式若其在 ZF 下等价于 $\exists x \phi$ 有 ϕ 为 Δ_0 式。称 ϕ 为 Π_1 式若其等价于 $\forall x \phi$ 有 ϕ 为 Δ_0 式。称 ϕ 为 Δ_1 式若其即为 Σ_1 式也为 Π_1 式 (即其即等价于 $\exists x \phi_1$ 也等价于 $\forall x \phi_2$ 有 $\phi_1 \phi_2$ 均为 Δ_0 式)²。

可见仍以传递集为例 (类比定理 5.2.4), 对任意 Σ_1 式 ϕ , $A_Z \vdash isTransitiveSet(m) \rightarrow ((\models_{(m)} \phi) \rightarrow \phi)$, 因如

¹ 称满足 i 和 ii 的模型 $\mathfrak{M}$ 为 $\mathfrak{M}$ 在 T 下的标准模型。

² 同样可对比章节 3.2.3 算术阶层。同时对其的进一步泛化也类似算术阶层中采用的方式。称泛化后的结果为莱维阶层。

$\exists x \in m \varphi$ 有 $\exists x \varphi$ 。称这样的 ϕ 向上绝对。类似对 Π_1 式 ϕ , $A_Z \vdash isTransitiveSet(m) \rightarrow ((\models_{(m)} \phi) \leftarrow \phi)$ ^{注1}, 称其向下绝对。则可见 Δ_1 式是绝对的。注意这里的等价关系工作于 ZF 下, 而我们又已有 $m_V m_L$ 满足 A_{ZF} , 因此我们可以对 Δ_1 式得到类似 (5.2.3) 这样的结果。

有了这些背景, 我们将证明 $\phi_{[x,y]}$ 等于 $isOrd(x) \rightarrow y = L_x$ 为 Δ_1 式。自顶向下的, 我们首先可以看到若函数类 $y = L_x$ 为 Σ_1 式, 由定理 3.2.5 中关于前束范式的讨论, 结合 $isOrd(x)$ 在 A_{ZF} 下为 Δ_0 式, 我们有 ϕ 为 Σ_1 式。同时可见 ϕ 等价于 $\pi_{[x,y]}$ 有 $isOrd(x) \rightarrow \forall z (z = L_x \rightarrow y = z)$, 同样由前束范式——并结合 (同样类似于算术式性质的) 若 φ 为 Δ_0 式 $\forall x, y \varphi$ 为 Π_1 式, 因其等价于 $\forall p \exists u \in p \exists x y \in u (p = \langle x, y \rangle \rightarrow \varphi)$ (对 $\exists$ 同理)——我们有 π 为 Π_1 式。即 ϕ 为 Δ_1 式。则我们的目标是证明 $y = L_x$ 、即 $y = L_a$ 对任意序数 a 为 Σ_1 式。令对任意序数 a , 递归定义的 (见章节 5.2.1 首) 函数类 $y = L_a$ 中递归步骤为式 $\gamma[a, f, y]$, 我们可定义 $y = L_a$ 为:

$$\exists f isFunction(f) \wedge dom f = a \wedge (\forall b \in a \gamma[a/b][f \upharpoonright b][y/f(b)]) \wedge \gamma$$

由示例 5.2.5 有其中如 $isFunction(f)$, $dom f = a$, $f \upharpoonright b$ 等都是 Δ_0 的。事实上可见若 γ 是 Σ_1 的, 则 $y = L_a$ 是 Σ_1 的, 因 $\forall u \in x \exists v \varphi \leftrightarrow \exists y \forall u \in x \exists v \in y \varphi$ ^{注2}。

回顾章节 5.2.1 中 L 的层级定义, 对于 $\gamma[a, f, y]$, 可见 $L_a = a = 0? \emptyset : (\bigcup a \in a? def(L_b) : \bigcup_{b \in a} L_b)$ 有其中 L_a 为 y 而 L_b 为 $f(b)$, 而 $\bigcup a \in a$ 判断 a 是否为后继序数。则若任意 $y = def(v)$ 是 Σ_1 的, 有 γ 是 Σ_1 的。下述引理 5.2.14 将给出 $y = def(v)$ 是 Σ_1 的。□

我们可以通过按照章节 5.2.1 定义的 $def(v)$ 证明其是 Σ_1 式, 这其中需要用到序数加法和乘法为 Δ_1 式。同时尽管其中的 $c_f \in \wp(\ulcorner c_1 \dots \urcorner \times v)$ 不是绝对的, 但注意到 s 编码的语句需为有限长的, 其中涉及的常量是有限的, 而集合的有限子集关系是 Δ_1 的。以此完成证明。

但这里我们采用另一种 $def(v)$ 的 (等价) 定义方法 (从而为选择公理否定形式一致性证明作准备)。我们首先引入相对化的概念。具体的说:

- 对任意项 m , 不含 m 的集合论式 ϕ , 写 $\phi|_m$ 表示递归改写 ϕ 中全部量词以限制其范围于 m 内——如改写 $\exists x \varphi$ 为 $\exists x \in m (\varphi|_m)$ ——后的新式。称该过程为相对化, 其结果为 ϕ 相对化于 m 后的结果 ³。

显然, 对任意语句:

$$A_Z \vdash (\phi|_m) \leftrightarrow (\models_{(m)} \phi) \quad (5.2.4)$$

对任意式 $\phi(x_1 \dots x_n)$, 额外添加的 (但在下式被简写省略的) 常量 $c_{1 \dots n}$ 有其解释 $c_{1 \dots n} \in m$, 有:

$$A_Z \vdash \langle c_{1 \dots n} \rangle \in m^n \rightarrow ((\phi(c_{1 \dots n})|_m) \leftrightarrow (\models_{(m)} \phi(c_{1 \dots n}))) \quad (5.2.5)$$

可见相对化在语句和语句满足模型间建立了对应关系。使我们对模型的处理可以不需要直接引入模型的概念。同时可见相对化后的语句为 Δ_0 的, 此处我们将首先应用这一点。

接着, 还需定义:

- 哥德尔运算包括 ⁴:
 - $G_1(x, y) = \{x, y\}$ 。
 - $G_2(x, y) = x \times y$ 。
 - $G_3(x, y) = \{ \langle u, v \rangle : u \in x \wedge v \in y \wedge u \in v \}$ 。

¹写 $q \leftarrow p$ 表示 $p \rightarrow q$ 。

²该等价关系将用于引理 5.2.14 的证明。

³同样类似定理 5.2.4, 我们可以对 m 进行到类的泛化, 同样我们不做此扩展。

⁴这里参考 Thomas Jech. Set Theory: The Third Millennium Edition. Springer, 2003, p. 177, 其略不同于哥德尔给出的形式, 见 Kurt Gödel. Consistency of the Continuum Hypothesis. Princeton University Press; 1951。

- $G_4(x, y) = x - y$ 。
- $G_5(x, y) = x \cap y$ 。
- $G_6(x) = \bigcup x$ 。
- $G_7(x) = \text{dom } x$ 。
- $G_8(x) = x^{-1}$ 。
- $G_9(x) = \{ \langle u, v, w \rangle : \langle u, w, v \rangle \in x \}$ 。
- $G_{10}(x) = \{ \langle u, v, w \rangle : \langle v, w, u \rangle \in x \}$ 。

- 对任意 m ，写 $cl(m)$ 表示 m 在哥德尔运算下的闭包。即如当任意 $xy \in m$ 时要求 $G_1(x, y) \in m$ 、 $G_2(x, y) \in m$ 等。具体的，定义对 ω 递归中的函数类 $\gamma[x, f, y]$ 为：

$$\begin{aligned} x = 0 \ ? \ y = m \\ : \ y = f(x-1) \cup \{ G_1(u_1, u_2) : \langle u_1, u_2 \rangle \in f(x-1)^2 \} \cup \dots \cup \{ G_5(u_1, u_2) : \langle u_1, u_2 \rangle \in f(x-1)^2 \} \\ \cup \{ G_6(u) : u \in f(x-1) \} \cup \dots \cup \{ G_{10}(u) : u \in f(x-1) \} \end{aligned}$$

$cl(m)$ 为 $\bigcup \text{ran } h$ 有 $h = \text{trec}_{\Pi}[\omega, <_{\omega}, \gamma]$ 。

我们将证明如下引理并在其基础上证明 $y = \text{def}(v)$ 为 Σ_1 式：

引理 5.2.11 $A_Z \vdash (\models_m A_{ZF}) \rightarrow (\models_m \forall v (\text{isTransitiveSet}(v) \rightarrow \text{def}(v) = cl(v; v) \cap \wp(v)))$ 。即对 ZF 模型 m 中的任意传递集 v 有 $\text{def}(v) = cl(v; v) \cap \wp(v)$ 。

为直观的可见上述引理成立，回顾 $\text{def}(v)$ （简写后）可表示为 $\{ \{ x \in v : \models_{(v)} \phi(x, c_{1\dots n}) \} : c_{1\dots n} \in v \}$ 。由 (5.2.5) 有任意 $w \in \text{def}(v)$ 当且仅当存在式 ϕ 、 $c_{1\dots n} \in v$ 有：

$$w = \{ x \in v : \phi(x, c_{1\dots n}) \mid v \} \quad (5.2.6)$$

如前所述其中的 $\phi(x, c_{1\dots n}) \mid v$ 为 Δ_0 式。本质上，我们将证明由 Δ_0 式定义的集合和 cl 得到的集合式可互相表示。为此我们通过推论 5.2.12.1 和引理 5.2.13 分别证明两个方向。

定理 5.2.12（哥德尔范式定理）

$$A_Z \vdash \text{is}\Delta_0(\ulcorner \phi \urcorner) \wedge (\models_m A_{ZF}) \rightarrow (\models_m \forall x_{1\dots n} \{ \langle u_{1\dots n} \rangle \in \prod_{1 \leq i \leq n} x_i : \phi \} \in cl(\{ x_{1\dots n} \}))$$

即在 $x_{1\dots n}$ 内由 Δ_0 式 ϕ 定义的（多元组）集合属于 $x_{1\dots n}$ 的哥德尔闭包。

证明：对 ϕ 的结构归纳。我们只（需）考虑由 $\neg \wedge \exists$ 构成的语句，只考虑 u_i （ i 为自然数）为自由变量，即如 $u_1 \in u_2$ 定义了集合 i ： $\{ \langle u_1, u_2 \rangle \in x_1 \times x_2 : u_1 \in u_2 \}$ 。不考虑 = 符号，因 $u_1 = u_2$ 等价于 $v \in u_1 \leftrightarrow v \in u_2$ 。则：

1. ϕ 为原子式 $u_i \in u_j$ （ $i \neq j$ ¹）。对 n 归纳：

- a. $u_1 \in u_2$ 为集合 $i = G_3(x_1, x_2)$ 。 $u_2 \in u_1$ 为 $i^{-1} = G_8(G_3(x_1, x_2))$ 。
- b. $n > 2$ ， $i j \neq n$ 。由归纳假设有由哥德尔运算组合而成的 $G(x_{1\dots n-1})$ 有：

$$\{ \langle u_{1\dots n-1} \rangle \in \prod_{1 \leq i < n} x_i : u_i \in u_j \} = G(x_{1\dots n-1})$$

则：

$$\{ \langle u_{1\dots n} \rangle \in \prod_{1 \leq i \leq n} x_i : u_i \in u_j \} = G(x_{1\dots n-1}) \times x_n = G_2(G(x_{1\dots n-1}))$$

¹我们假设了正则公理。

c. $n > 2$, $i, j \neq n-1$ 。由归纳假设 (即 b) 有 $G(x_{1\dots n})$ 有:

$$\{ \langle u_{1\dots n-2}, u_n, u_{n-1} \rangle \in (\prod_{1 \leq i \leq n-2} x_i) \times x_n \times x_{n-1} : u_i \in u_j \} = G(x_{1\dots n})$$

注意 $\langle u_{1\dots n-2}, u_n, u_{n-1} \rangle$ 即为 $\langle \langle u_{1\dots n-2} \rangle, u_n, u_{n-1} \rangle$, 有:

$$\{ \langle u_{1\dots n} \rangle \in \prod_{1 \leq i \leq n} x_i : u_i \in u_j \} = G_9(G(x_{1\dots n}))$$

d. $n > 2$, $i = n-1 \wedge j = n$ 或 $i = n \wedge j = n-1$ 。以前者为例。由 a 有:

$$\{ \langle u_{n-1}, u_n \rangle \in x_{n-1} \times x_n : u_{n-1} \in u_n \} = G_3(x_{n-1}, x_n)$$

则:

$$\{ \langle \langle u_{n-1}, u_n \rangle, \langle u_{1\dots n-2} \rangle \rangle \in x_{n-1} \times x_n \times \prod_{1 \leq i \leq n-2} x_i : u_{n-1} \in u_n \} = G_3(x_{n-1}, x_n) \times (x_1 \times \dots \times x_{n-2}) = G(x_{1\dots n})$$

注意 $\langle \langle u_{n-1}, u_n \rangle, \langle u_{1\dots n-2} \rangle \rangle$ 即为 $\langle u_{n-1}, u_n, \langle u_{1\dots n-2} \rangle \rangle$, 有:

$$\{ \langle u_{1\dots n} \rangle \in \prod_{1 \leq i \leq n} x_i : u_{n-1} \in u_n \} = G_{10}(G(x_{1\dots n}))$$

2. ϕ 为 $\neg \varphi(u_{1\dots n})$, 由归纳假设有 $G(x_{1\dots n})$ 对应 φ 即:

$$\{ \langle u_{1\dots n} \rangle \in \prod_{1 \leq i \leq n} x_i : \varphi \} = G(x_{1\dots n})$$

则:

$$\{ \langle u_{1\dots n} \rangle \in \prod_{1 \leq i \leq n} x_i : \neg \varphi \} = (x_1 \times \dots \times x_n) - G(x_{1\dots n})$$

3. ϕ 为 $\varphi(u_{1\dots n}) \wedge \psi(u_{1\dots n})$, 由归纳假设有 $G'(x_{1\dots n})$ 、 $G''(x_{1\dots n})$ 分别对应 φ 及 ψ , 则:

$$\{ \langle u_{1\dots n} \rangle \in \prod_{1 \leq i \leq n} x_i : \varphi \wedge \psi \} = G'(x_{1\dots n}) \cap G''(x_{1\dots n})$$

4. ϕ 为 $\exists u_{n+1} \in u_i \varphi(u_{1\dots n+1})$ ($1 \leq i \leq n$)。考虑 π 为 $\varphi(u_{1\dots n+1}) \wedge u_{n+1} \in u_n$, 则存在 $G(x_{1\dots n+1})$ 有:

$$\{ \langle u_{1\dots n+1} \rangle \in \prod_{1 \leq i \leq n+1} x_i : \pi \} = G(x_{1\dots n+1})$$

则:

$$\{ \langle u_{1\dots n} \rangle \in \prod_{1 \leq i \leq n} x_i : \exists u_{n+1} \in u_i \varphi \} = (x_1 \times \dots \times x_n) \cap \text{dom } G(x_{1\dots n}, \bigcup x_i)$$

因:

$$\begin{aligned} \langle u_{1\dots n} \rangle \in \prod_{1 \leq i \leq n} x_i \wedge \phi(u_{1\dots n}) &\leftrightarrow \langle u_{1\dots n} \rangle \in \prod_{1 \leq i \leq n} x_i \wedge \exists u_{n+1} \in u_i \varphi(u_{1\dots n+1}) \\ &\leftrightarrow \exists u_{n+1} (\langle u_{1\dots n} \rangle \in \prod_{1 \leq i \leq n} x_i \wedge u_{n+1} \in \bigcup x_i \wedge \varphi(u_{1\dots n+1}) \wedge u_{n+1} \in u_i) \\ &\leftrightarrow \langle u_{1\dots n} \rangle \in \text{dom} \{ \langle \langle u_{1\dots n} \rangle, u_{n+1} \rangle \in (\prod_{1 \leq i \leq n} x_i) \times \bigcup x_i : \pi \} \quad \square \end{aligned}$$

关注 (5.2.6):

推论 5.2.12.1 $A_Z \vdash (\models_m A_{ZF}) \wedge (\models_m n \in \omega) \rightarrow (\models_m \forall v \forall c_{1\dots n} \in v \{ x \in v : \phi(x, c_{1\dots n}) \mid v \} \in cl(v; v))$ 。

证明: 令 φ 等于 $\phi(x, c_{1\dots n}) \mid v$ 有其为 Δ_0 式, 则由哥德尔范式定理存在:

$$G(v, \{c_1\}, \dots, \{c_n\}) = \{ \langle x, c_{1\dots n} \rangle \in v \times \prod_{1 \leq i \leq n} \{c_i\} : \varphi \}$$

则:

$$\{ x \in v : \phi(x, c_{1\dots n}) \mid v \} = \overbrace{\text{dom} \dots \text{dom}}^{n \text{ 次}} G(v, G_1(c_1, c_1), \dots, G_1(c_n, c_n))$$

由 $c_{1\dots n} \in v$, 可见上式 = 右侧属于 $cl(v; v)$ 。 $\square$

下面我们处理反方向。

引理 5.2.13 $A_Z \vdash (\models_m y \in cl(\{x_{1\dots n}\})) \rightarrow \exists^\Gamma \phi(x_{1\dots n}, y)^\Gamma is \Delta_0(\Gamma \phi^\Gamma) \wedge (\models_m \phi)$ 。即对任意哥德尔闭包中的集合 $y = G(x_{1\dots n})$ 都存在 Δ_0 式 ϕ 与之等价。

证明: 对 G 的组成归纳:

1. 对 G 为 G_i ($1 \leq i \leq 10$), 在示例 5.2.5 中我们已处理了其中的大多数情况。我们可以以类似的方式处理余下的 G_i , 如:

$$\begin{aligned} A_Z \vdash y = G_8(x) &\leftrightarrow (\forall t \in y \exists s \in x \exists u \in \text{dom } x, v \in \text{ran } x \ s = \langle u, v \rangle \wedge t = \langle v, u \rangle) \\ &\wedge (\forall s \in x, u \in \text{dom } x, v \in \text{ran } x \exists t \in y (s = \langle u, v \rangle \rightarrow t = \langle v, u \rangle)) \end{aligned}$$

2. 对 $y = G(x_{\dots})$ 。其等价于 $(\forall s \in y \ s \in G(x_{\dots})) \wedge (\forall s \in G(x_{\dots}) \ s \in y)$ 。为证明其为 Δ_0 式, 我们需要证明:

- a. $y \in G(x_{\dots})$ 为 Δ_0 式。分类讨论 G 的构成, 以 G_2 为例:

$$A_Z \vdash y \in G'(x_{\dots}) \times G''(x_{\dots}) \leftrightarrow \exists s \in G'(x_{\dots}), t \in G''(x_{\dots}) y = \langle s, t \rangle$$

- b. (需) 且对任意 Δ_0 式 $\varphi(y)$, $\exists y \in G(x_{\dots}) \varphi(y)$ 为 Δ_0 式。同样讨论 G 的构成, 以 G_1 为例:

$$A_Z \vdash \exists y \in G(x_{\dots}) \varphi(y) \leftrightarrow \varphi(G'(x_{\dots})) \vee \varphi(G''(x_{\dots}))$$

- c. 且对任意 Δ_0 式 $\varphi(y)$, $\varphi(G(x_{\dots}))$ 为 Δ_0 式。因 $G(x_{\dots})$ 在 $\varphi(G(x_{\dots}))$ 可出现的形式有 $s = G(x_{\dots})$ 、 $s \in G(x_{\dots})$ 、 $G(x_{1\dots n}) \in s$ 、 $\exists s \in G(x_{\dots}) \psi$ 和 $\forall s \in G(x_{\dots}) \psi$, 其为 Δ_0 式可由归纳假设得到, 可见我们已在归纳证明其中的第一、第二和第四种情况。而 $G(x_{\dots}) \in s$ 等价于 $\exists u \in s \ u = G(x_{\dots})$ 。对 $\forall s \in G(x_{\dots}) \psi$ 同 b。 $\square$

引理 5.2.11 的证明: 推论 5.2.12.1 已证明了其中的正方向, 即 $w \in \text{def}(v)$ 有 $w \in cl(v; v)$ 。我们通过以上引理证明反方向。对任意 $w \in cl(v; v)$, 存在 G 有 $w = G(v, c_{1\dots n})$ 有 $c_{1\dots n} \in v$, 则由上述引理有 Δ_0 式 $\phi'(v, x, c_{1\dots n})$ 满足 $w = \{ x : \phi'(v, x, c_{1\dots n}) \}$, 进一步要求 $w \in \wp(v)$, 则 $w = \{ x \in v : \phi(v, x, c_{1\dots n}) \}$ 。在 v 为传递集的前提下, 由 $x \in v$ 且 ϕ 为 Δ_0 式我们可以确定 $\phi(v, x, c_{1\dots n})$ 中全部关于 v 的 (Δ_0) 原子式的真值, 如由传递性有任意新引入的约束变量 $z \in v$ 。由 $\top \perp$ 替换这些原子式, 我们有 $\varphi(v, x, c_{1\dots n})$ 有 v 只出现在如 $\exists z \in v \delta$ 中, 则有 $\psi(x, c_{1\dots n}) \mid v$ 与之等价, 则 $w = \{ x \in v : \models_{(v)} \psi(x, c_{1\dots n}) \}$ 。即 $w \in \text{def}(v)$ 。 $\square$

引理 5.2.14 ZF 下, $y = \text{def}(v)$ 为 Σ_1 式。

证明: 由上述引理仅需证 $cl(v; v)$ 为 Σ_1 式, 因 $y = \text{def}(v)$ 等价于:

$$(\forall x \in y \ x \in cl(v; v) \wedge x \subseteq v) \wedge (\forall x \in cl(v; v) (x \subseteq v \rightarrow x \in y))$$

其为 Σ_1 式, 因对任意 Σ_1 式 $\varphi_1 \varphi_2$, 显然 $\varphi_1 \wedge \varphi_2$ 为 Σ_1 式¹, 并有 181 页注 2。而对任意 m , 由 $cl(m)$ 的定义 (182 页) 有任意 $y = cl(m)$ 等价于:

$$\begin{aligned} \exists h \text{ isFunction}(h) \wedge \text{dom } h = \omega \wedge y = \bigcup \text{ran } h \wedge h(0) = m \\ \wedge \forall n \in \omega \ h(n^+) = h(n) \cup \{ G_1(u_1, u_2) : \langle u_1, u_2 \rangle \in h(n)^2 \} \cup \dots \cup \{ G_5(u_1, u_2) : \langle u_1, u_2 \rangle \in h(n)^2 \} \\ \cup \{ G_6(u) : u \in h(n) \} \cup \dots \cup \dots \cup \{ G_{10}(u) : u \in h(n) \} \quad \square \end{aligned}$$

引理 5.2.15 $A_Z \vdash (\models_{m_V} A_{ZF}) \rightarrow (\models_{\mathfrak{L}(m_V)} A_{ZF}; \text{constructibility})$ 。

证明: 即引理 5.2.9 和 5.2.10。 □

5.2.4 可构造全集的全局良序性

结合引理 5.2.10, 我们通过以下引理最终证明选择公理的相对一致性:

引理 5.2.16 $A_Z \vdash (\models_m A_{ZF}; \text{constructibility}) \rightarrow (\models_m \text{Choice})$ 。即可构造公理蕴含选择公理。

直观的说, 由可构造全集的定义即可见上述引理的成立, 因可递归定义良序 $<$ 对任意层级 $L_a L_b$, x 第一次出现于 L_a 中, y 第一次出现于 L_b 中有 $a \in b$ 则 $x < y$ 。若 $a = b$, 则可首先比较用于定义 x 的最小式编码 s_x 和 s_y 之间的大小, 即 $s_x < s_y$ 则 $x < y$ 。若 $s_x = s_y$ 则可递归按字典排序比较 c_f 中涉及的参数大小。而以上定义可在 L 中完成因可构造性是绝对的。同时显然 $<$ 是全局的。

但这里 (同样为了下一节中选择公理否定形式相对一致性证明), 我们采取按哥德尔运算 / 闭包方式定义的可构造全集的构成方式构造其良序 (类), 尽管本质上这两种证明方式是相同的。

引理 5.2.16 的证明: 不涉及类, 我们证明对任意序数 a , (L 中) 存在 L_a 上的良序关系 $<_a$ (则对任意 $x \in L_a$, 有 $<_a \upharpoonright x^2$ 为其上的良序关系)。本质上, 我们仍按字典排序对 L_a 中的元素进行排序, 首先我们比较元素第一次出现的可构造层级序数 (即秩), 对此, 我们对 a 归纳 (即递归构造 $<_a$)。由此 (我们会看到) 我们构造的 $<_a$ 将是全局的, 或者说对任意序数 $b \in a$ ($b < a$) $<_a$ 为 $<_b$ 的尾扩展, 或 $<_b \subset <_a$ 且 $L_b \times (L_a - L_b) \subset <_a$, 或 $\dots <_a \dots \underbrace{<_a \dots <_a}_{L_b} \dots \underbrace{<_a \dots <_a}_{L_a - L_b} \dots$, 即我们接下去的归纳 (递归) 过程仅需考虑 $L_a - L_b$ 中元素的排列顺序。具体的:

1. $<_0 = \emptyset$ 。对极限序数 a , 由尾扩展 $<_a = \bigcup_{b \in a} <_b$ 。
2. 而对后继序数 a^+ , 由引理 5.2.11:

$$L_{a^+} = \wp(L_a) \cap cl(L_a; L_a) = \wp(L_a) \cap \bigcup_{n \in \omega} h_{a^+}(n)$$

其中 $h_{a^+}(0) = L_a; L_a$ 。而:

$$\begin{aligned} h_{a^+}(n^+) = h_{a^+}(n) \cup \{ G_1(u_1, u_2) : \langle u_1, u_2 \rangle \in h_{a^+}(n)^2 \} \cup \dots \cup \{ G_5(u_1, u_2) : \langle u_1, u_2 \rangle \in h_{a^+}(n)^2 \} \\ \cup \{ G_6(u) : u \in h_{a^+}(n) \} \cup \dots \cup \{ G_{10}(u) : u \in h_{a^+}(n) \} \end{aligned}$$

继续字典排序, 本质上, 我们首先比较元素第一次出现的 n , 然后 i , 最后 $u_1 u_2$ 。可见对 $u_1 u_2$ 的比较将是对 n 归纳 (递归) 的。具体的, 我们定义函数 $h_{<_{a^+}}$ 有 $h_{<_{a^+}}(n)$ 为 $h_{a^+}(n)$ 上的良序关系。则:

¹ $A_Z \vdash (\exists x_1 \delta_1) \wedge (\exists x_2 \delta_2) \leftrightarrow \exists y \exists x_1 \in y, x_2 \in y \delta_1 \wedge \delta_2$ 。

$$h_{<_{a^+}}(0) = <_a \cup (L_a \times \{L_a\}) \cup (h_{a^+}(0) \times (L_{a^+} - h_{a^+}(0)))$$

而:

$$\begin{aligned} h_{<_{a^+}}(n^+) &= h_{<_{a^+}}(n) \\ &\cup \bigcup_{1 \leq i \leq 5} (\{ \langle G_i(u_1, u_2), G_i(u_1, u'_2) \rangle : u_1 \in h_{a^+}(n) \wedge u_2 \in h_{<_{a^+}}(n) \wedge u'_2 \in h_{a^+}(n) \} \\ &\quad \cup \bigcup \{ \{ \langle G_i(u_1, u_2), g \rangle : g \in (\{ G_i(u'_1, u'_2) : \langle u'_1, u'_2 \rangle \in h_{a^+}(n)^2 \} \\ &\quad \quad - \{ G_i(u'_1, u'_2) : (u'_1 \in h_{<_{a^+}}(n) \wedge u'_1 = u_1) \wedge u'_2 \in h_{a^+}(n) \}) \} : \langle u_1, u_2 \rangle \in h_{a^+}(n)^2 \} \\ &\quad \cup (\{ G_i(u_1, u_2) : \langle u_1, u_2 \rangle \in h_{a^+}(n)^2 \} \times (h_{a^+}(n^+) - \bigcup_{1 \leq j \leq i} \{ G_j(u_1, u_2) : \langle u_1, u_2 \rangle \in h_{a^+}(n)^2 \}))) \\ &\cup \bigcup_{6 \leq i \leq 10} (\{ \langle G_i(u), G_i(u') \rangle : u \in h_{<_{a^+}}(n) \wedge u' \in h_{a^+}(n) \} \\ &\quad \cup (\{ G_i(u) : u \in h_{a^+}(n) \} \times (h_{a^+}(n^+) - \bigcup_{1 \leq j \leq 5} \{ G_j(u_1, u_2) : \langle u_1, u_2 \rangle \in h_{a^+}(n)^2 \} \\ &\quad \quad - \bigcup_{6 \leq j \leq i} \{ G_j(u) : u \in h_{a^+}(n) \}))) \\ &\cup (h_{a^+}(n^+) \times (L_{a^+} - h_{a^+}(n^+))) \end{aligned}$$

撇开其复杂度，其即定义了上述字典排序。则 $<_{a^+} = (ran h_{<_{a^+}}) \cap \wp(L_a)^2$ 。

□

由上述证明可见以下推论，其将被用于章节 5.3.5 对序数可定义集合的讨论，后者将被用于选择公理否定形式相对一致性的证明：

推论 5.2.16.1 $A_{ZF} \vdash WO(m, <_m) \rightarrow \exists <_{cl(m)} WO(cl(m), <_{cl(m)})$ 。且 $<_{cl(m)}$ 是可定义的。

更主要的：

定理 5.2.17 $A_Z \vdash (\models_{m_V} A_{ZF}) \rightarrow (\models_{(m_V)} A_{ZFC})$ 。

证明：引理 5.2.15、5.2.16。

□

最后，结合哥德尔第二不完备定理，我们有 ZFC 作为元逻辑不能证明 ZF 的一致性。注意这不同于 ZFC 的一致性蕴含 ZF 的一致性。其关键在于 ZF 的模型（和 ZFC 的一样）其存在性不可在 ZFC 中被证明。

这样，我们即完成了 ZF 不可证伪选择公理的证明。在给出 ZF 不可证明选择公理的证明之前，让我们进一步给出一种相对一致性证明的元逻辑框架¹。

5.2.5 相对一致性的形式化

注意我们将在弱如 A_P 的元逻辑中给出该框架。具体的，对任意可公理化的理论 T ，对其公理集 A_T ，写 $Con(A_T)$ 表示 Con_T ^{注2}。则如这里对新语句 ϕ ，我们将尝试得到 $A_P \vdash Con(A_{ZFC}) \rightarrow Con(A_{ZF}; \phi)$ 或 $A_P \vdash Con(A_{ZFC}) \rightarrow Con(A_{ZFC}; \phi)$ ^{注3}。

¹ 后续介绍的力迫法和选择公理否定形式相对一致性证明将需要应用该框架。

² 回顾章节 3.2.7， Con_T 为算术语句表示 T 一致。

³ 关于以 A_P 作为基础元逻辑完成相对一致性证明参考自 N. Weaver. Forcing For Mathematicians. World Scientific Publishing Company, 2014。

回顾相对化，我们可以在 $\mathbf{P}$ 中完成相对一致性证明的关键点之一在于相对化使对（最外层）模型的讨论可转换到对语句的讨论上，并通过证明论演绎处理这些语句。同时注意 $(\varphi \wedge \psi) \mid m \equiv \varphi \mid m \wedge \psi \mid m$ 等。并可见相对化过程是可计算的。进一步：

- 对任意语句集 Γ ，项 m ，写 $\Gamma \mid m$ 表示 $\{(\gamma \mid m) : \gamma \in \Gamma\}$ ，即其中各语句相对化后得到的新集合。

定义：

- $isSentence_{\mathcal{L}_S}(\ulcorner \phi \urcorner)$ 表示 ϕ 为集合论语句。

扩展集合论语言 $\mathcal{L}_S$ ，增加常量符号 $\mathbf{m}$ 得到新语言 $\mathcal{L}_{S,m}$ 。在新语言下扩展 ZFC 集合论至 ZFC_m 有其公理集：

$$A_{ZFC_m} = A_{ZFC} \cup A_{ZFC} \mid m ; isCountable(\mathbf{m}) ; isTransitiveSet(\mathbf{m}) \quad (5.2.7)$$

可以看到以 ZFC_m 作为元逻辑，由定义有存在目标逻辑 ZFC 的模型 $(\mathbf{m})$ 、即 $\vdash_{(\mathbf{m})} A_{ZFC}$ 。我们将首先证明：

定理 5.2.18

$$A_{\mathbf{P}} \vdash isSentence_{\mathcal{L}_S}(\ulcorner \phi \urcorner) \wedge (A_{ZFC_m} \vdash \exists m^\dagger (\vdash_{(m^\dagger)} A_{ZF} ; \phi)) \rightarrow (Con(A_{ZFC}) \rightarrow Con(A_{ZF} ; \phi)) \quad \text{注}^1$$

注意其中 $A_{ZFC_m} \vdash \exists m^\dagger (\vdash_{(m^\dagger)} A_{ZF} ; \phi)$ 表示 A_{ZFC_m} 语义蕴含（即可演绎推导得到） $\exists m^\dagger (\vdash_{(m^\dagger)} A_{ZF} ; \phi)$ ，语义蕴含关系是算术可定义的，因此其为算术式。结合其它判定谓词均是算术可定义的， $A_{\mathbf{P}} \vdash$ 右侧的部分为算术语句。 $A_{ZFC_m} \vdash \exists m^\dagger (\vdash_{(m^\dagger)} A_{ZF} ; \phi)$ 表示在 ZFC_m 中可证明存在解释 $(m^\dagger)$ 有 $\vdash_{(m^\dagger)} A_{ZF} ; \phi$ ，可见这里的 $\vdash$ 关系定义于 A_{ZFC_m} 而非最外层元逻辑 $A_{\mathbf{P}}$ 中²（因此不影响 $A_{\mathbf{P}} \vdash$ 右侧算术语句的定义）。可见本质上，通过在 ZFC_m 中构造 $m^\dagger$ 我们可以证明相对一致性的结果。

证明：注意结合 (5.2.4) 由 $\vdash_{(m^\dagger)} A_{ZF} ; \phi$ 有 i：对任意 A_{ZF} 中的语句 α 有 $\alpha \mid m^\dagger$ ，且有 $\phi \mid m^\dagger$ 。

首先证明 ii： $A_{\mathbf{P}} \vdash Con(A_{ZFC_m}) \rightarrow Con(A_{ZF} ; \phi)$ 。若 $A_{ZF} ; \phi \vdash \perp$ ，考虑涉及的公理 $\Gamma \subset A_{ZF}$ ，考虑相继式演绎³，最终有 $\Gamma ; \phi \Rightarrow \emptyset$ 。我们下一段落证明由此有 $(\Gamma ; \phi) \mid m^\dagger \Rightarrow \emptyset$ 。现因结合由 i 我们可以证明该式前项，即 $A_{ZFC_m} \Rightarrow \{ \wedge (\Gamma ; \phi) \mid m^\dagger \}$ 。则我们可以证明 ii。

现在证明 iii：由 $\Gamma ; \phi \Rightarrow \emptyset$ 可得 $(\Gamma ; \phi) \mid m^\dagger \Rightarrow \emptyset$ 。我们对推导树做归纳，归纳假设对任意上式 $\Sigma \Rightarrow \Pi$ 有 $\Sigma \mid m^\dagger \Rightarrow \Pi \mid m^\dagger$ ，证明下式同样满足该条件。这里唯一值得关注的为规则 L_3 ，但注意到有归纳假设有其中的 c 和 τ 均属于 $m^\dagger$ ，则可得 $(\exists x \alpha) \mid m^\dagger$ 。进一步注意因任意推导树是有限的，其可被编码为自然数，而 $\mathbf{P}$ 支持自然数上的归纳，因此 iii 由 $A_{\mathbf{P}}$ 可证。

最后证明 $A_{\mathbf{P}} \vdash Con(A_{ZFC}) \rightarrow Con(A_{ZFC_m})$ 。若 $A_{ZFC_m} \vdash \perp$ ，考虑其中涉及的公理包括 $\Gamma \subset A_{ZFC}$ 、 $\Sigma \mid m \subset A_{ZFC} \mid m$ 有 $\Sigma \subset A_{ZFC}$ 、以及 θ 断言 $\mathbf{m}$ 传递。因 Σ 有限，令 $\sigma = \wedge \Sigma$ 为语句，有 $\Gamma ; \sigma \mid m \wedge \theta \Rightarrow \emptyset$ 。又因 $\mathbf{m}$ 不出现在 Γ 中，可进一步应用规则 L_{4a} 有 $\Gamma ; \exists w \sigma \mid w \wedge \theta[m/w] \Rightarrow \emptyset$ 。

则若可以证明 iv：存在 $\Lambda \subset A_{ZFC}$ 有 $\Lambda \Rightarrow \exists w \sigma \mid w \wedge \theta[m/w]$ ，通过切规则 L_{cut} 有 $\Lambda \cup \Gamma \Rightarrow \emptyset$ ，即 $A_{ZFC} \vdash \perp$ 。又因 $\Sigma \subset A_{ZFC}$ ，则若可证 v：存在 Λ' 有 $\Sigma \subseteq \Lambda' \subset A_{ZFC}$ （即已有 $\Lambda' \Rightarrow \sigma$ ）有 $\Lambda' \Rightarrow \exists w(\sigma \leftrightarrow \sigma \mid w) \wedge \theta[m/w]$ ，我们可以证明 iv。事实上 v 即为下述定理 5.2.20。□

定理 5.2.19

$$A_{\mathbf{P}} \vdash isSentence_{\mathcal{L}_S}(\ulcorner \phi \urcorner) \wedge (A_{ZFC_m} \vdash \exists m^\dagger (\vdash_{(m^\dagger)} A_{ZFC} ; \phi)) \rightarrow (Con(A_{ZFC}) \rightarrow Con(A_{ZFC} ; \phi))$$

¹其中 $m^\dagger$ 和（下文中） $m^\ddagger$ 为普通变量名。

²事实上，仅通过 $A_{\mathbf{P}}$ 并不能完整定义 $\vdash$ ，见章节 5.4.1。

³章节 3.1.4。

证明：同上。仅需令 ii 为 $A_P \vdash \text{Con}(A_{ZFC_m}) \rightarrow \text{Con}(A_{ZFC}; \phi)$ 。 $\square$

定理 5.2.20 对任意语句 σ , $A_{ZFC} \vdash \exists w \overbrace{(\sigma \leftrightarrow \sigma|_w) \wedge \text{isCountable}(w) \wedge \text{isTransitiveSet}(w)}^\alpha$ 。

可见 α 部分即为反射原理推论 5.2.8.3。我们仅需证明 w 可为传递集（并保持 α ）。为此我们应用下述的引理 5.2.21（莫斯托夫斯基塌陷引理）和引理 5.2.22。

- 称任意集合 v 为外延集若 $\forall x, y \in v, x \neq y \rightarrow x \cap v \neq y \cap v$, 写作 $\text{isExtensional}(v)$ 。

有：

引理 5.2.21（莫斯托夫斯基塌陷引理）

$$A_{ZFC} \vdash \text{isExtensional}(v) \rightarrow \exists! w, f \text{ isIsomorphism}_\epsilon(f, v, w) \wedge \text{isTransitiveSet}(w)$$

证明：由对良基关系 $\langle v, \in_v \rangle$ 的超限递归定理，令：

$$f = \text{trec}_\Pi[v, \in_v, y = \{z : \exists u \in (x \cap v) z = f(u)\}]$$

可见对任意 $x \in v$, 递归的, $f(x) = \{f(z) : z \in x \cap v\}$ 。可见 $w = \text{ran } f$ 为传递集。且对任意 $z, x \in v$ 有 $z \in x \leftrightarrow f(z) \in f(x)$ 。

进一步通过外延集证明 f 为双射函数。可见我们仅需证明对任意 $x, y \in v$, $x \neq y \rightarrow f(x) \neq f(y)$ 。假设不成立，取最小 z 有 $z = f(x) = f(y)$ 但 $x \neq y$, 或 $z = \min_\epsilon \{z \in w : \exists x, y \in v, x \neq y \wedge z = f(x) \wedge z = f(y)\}$ 。由 v 为外延集有对 $x \neq y$, 存在 $u_1 \in x \cap v \wedge u_1 \notin y \cap v$ 且 / 或 $u_2 \notin x \cap v \wedge u_2 \in y \cap v$ 。以且为例，由 z 最小，意味着任意这样的 $u_1 \neq u_2$ 有 $f(u_1) \neq f(u_2)$, 则 $f(x) \neq f(y)$, 矛盾。

最后对于唯一性。假设不唯一，即有满足要求的传递集 w_1, w_2 存在 g 满足 $\text{isIsomorphism}_\epsilon(g, w_1, w_2)$ 有存在 $x \in w_1 \wedge x \neq g(x)$ 。则由引理 5.1.12 存在 $y = \min_\epsilon \{x \in w_1 : x \neq g(x)\}$ 。则任意 $u \in y$ 有 $u = g(u) \in g(y)$, 且任意 $g(v) \in g(y)$ 有 $v \in y$ 有 $v = g(v) \in y$, 则 $y = g(y)$, 矛盾。 $\square$

注意 f 为双射函数，因此 $|v| = |w|$ 。这里塌陷裁剪的是 v 中的元素使其不含多余的元素破坏传递性。

及：

引理 5.2.22 对任意式 $\phi_{[x_1 \dots x_n]}$ ：

$$A_{ZFC} \vdash \text{isIsomorphism}_\epsilon(f, v, w) \rightarrow \forall u_1 \dots u_n \in v (\phi_{[x_1/u_1] \dots [x_n/u_n]}|_v \leftrightarrow \phi_{[x_1/f(u_1)] \dots [x_n/f(u_n)]}|_w)$$

证明：对 ϕ 的结果做归纳。可见对原子式 $x_1 \in x_2$ 和 $x_1 = x_2$, 由同构显然成立。在归纳假设子式满足条件的情况下，可见由 $\neg \vee \exists$ 构成的式满足条件。 $\square$

定理 5.2.20 的证明：令 ϕ 为 $\sigma \wedge \text{Ext}$, 应用定理 5.2.7（反射原理），我们有模型 (v) 满足 σ 和外延公理。由满足外延公理有 v 是外延集。则应用引理 5.2.21（莫斯托夫斯基塌陷引理）可将 v 裁剪为传递集 w , 并因同构由上述引理及同构的双射性保证了 (w) 保持满足定理的 α 部分。 $\square$

这样我们即完成了定理 5.2.18 及 5.2.19 的证明。下面的章节仅涉及这两个定理的应用¹。

¹事实上，我们正文给出的关于选择公理否定形式相对一致性的证明仅应用前者。但对力迫法的介绍将构造 $m^\dagger$ 满足选择公理。

5.3 力迫法

直观的说，相对于内模型通过裁切假设存在的理论如 ZF（或如下 199 页注 1 中提及的 ZFC）的模型得到新的模型，力迫法通过系统性的扩展原模型得到新模型满足更多的语句。另一方面，我们将看到力迫法（和很多其它构造、定义、证明一样）其当下的发展结果很难说是易于理解的，本节末也将尝试通过历史对此直觉上和技术上的相互关系进行简单的讨论。

5.3.1 力迫法构成

对于章节 5.2.5 中的 ZFC_m ，我们将通过力迫法扩展其中 m 构造 $m^\dagger = m[g]$ 有其由 g 的选择满足特定的 $\models_{(m[g])} \phi$ 。并可见该构造证明将全部在 ZFC_m 中完成。而不管采用何种 g 用以证明何种 ϕ 的相对一致性，我们首先证明 $\models_{(m[g])} A_{ZFC}$ 。本小节将介绍对 $p \in g$ 的要求， $m[g]$ 的构造法。并在接下去的两个小节中证明该点。

本质上，构造令特定（元逻辑 ZFC_m 中） p 相关的集合 $g \in m[g]$ ，我们可通过 g 证明 $m[g]$ 满足特定 ϕ 注¹。注意通常 $g \notin m$ ，当 $g \in m$ 时， m 应已满足 ϕ ，通过力迫法我们只是证明了这一点。而当 $g \notin m$ ，为维持 $m[g]$ 满足 A_{ZFC} ，我们需要向 m 中（除加入 g 外）进一步加入一系列和 g 相关的元素。现在我们来查看 $m[g]$ 的定义及相关的技术细节。

相对于 m ，我们将对更一般的场景进行讨论：

- 对任意 m ，写 $CTM(m)$ 表示 $isCountable(m) \wedge isTransitiveSet(m) \wedge (\models_{(m)} A_{ZFC})$ ，称 (m) 为可数传递模型。可见 ZFC_m 下有 $CTM(m)$ 。

也即在力迫法的应用中，我们不一定要使用 (m) 作为 $(m[g])$ 的基准模型，我们下面的讨论基于以任意可数传递模型 (m) 作为基准模型的场景²。

对任意 p 及其上的偏序关系 $\leq$ ，写 $p_\leq$ 表示 $\langle p, \leq \rangle$ ：

- 任意 $s, t \in p$ ，对 $s \leq t$ 。称 s 为 t 的扩展。
- 任意 $t_1, t_2 \in p$ ，写 $isCompatible(t_1, t_2, p_\leq)$ 表示 $\exists s \in p \ s \leq t_1 \wedge s \leq t_2$ 。称 t_1, t_2 是相容的。
- 任意 $d \subseteq p$ ，写 $isDense(d, p_\leq)$ 表示 $\forall t \in p \ \exists s \in d \ s \leq t$ ，称 d 为稠密的。
- 任意 $f \subseteq p$ ，写 $isFilter(f, p_\leq)$ 表示：

$$f \neq \emptyset \wedge \overbrace{\forall s \in f, t \in p (s \leq t \rightarrow t \in f)}^\alpha \wedge \overbrace{\forall t_1, t_2 \in f \ isCompatible(t_1, t_2, f_\leq)}^\beta$$

其中 $f_\leq$ 为 $\langle f, \leq|_f \rangle$ 。并称其中条件 α 为向上闭合性，条件 β 为向下有向性（即 f 中任意 t_1, t_2 可指向同一个扩展 $s \in f$ ，或 $s \leq t_1 \wedge s \leq t_2$ ）。称 f 为 p 上的滤子集。

类似 $\in_p$ ，定义 $\supseteq_p = \{ \langle x, y \rangle \in p^2 : x \supseteq y \}$ 。写 $p_\supseteq$ 表示 $p_{\supseteq_p}$ （即 $\langle p, \supseteq_p \rangle$ ）。来看一个具体的例子。对：

$$p = \{ \{1\}, \{2\}, \{3\}, \{1, 2\} \}$$

¹如连续统假设， $2^{\aleph_0} = \aleph_1$ ，在定理 5.2.19 基础上，我们通过 g 构造（如 $\bigcup g$ ）模型 $m[g]$ 下的 $\wp(\omega)$ 到 $\aleph_1$ 之间的双射函数。对连续统假设的否定，我们构造（更多的） ω 的子集使 $\wp(\omega)$ 有到如 $\aleph_2$ 的双射函数，但没有到 $\aleph_1$ 的双射函数。本书将不进一步介绍连续统假设独立性的证明，具体可参考 N. Weaver. Forcing For Mathematicians. World Scientific Publishing Company, 2014 及 Thomas Jech. Set Theory: The Third Millennium Edition. Springer, 2003, pp. 219–220。

²事实上，该前提可被进一步弱化，见 Kris. Does forcing need a countable transitive model. Mathematics Stack Exchange. URL: <https://math.stackexchange.com/q/378438>。

对于 $\supseteq_p$, 可见 $\{1, 2\}$ 为 $\{1\}$ 的扩展, $isCompatible(\{1\}, \{2\}, p_\supseteq)$ 等, 但没有 $isCompatible(\{1\}, \{3\}, p_\supseteq)$ 等。还有 $isDense(\{\{3\}, \{1, 2\}\}, p_\supseteq)$ 。还可以看到 $isFilter(\{\{1\}, \{2\}, \{1, 2\}\}, p_\supseteq)$, 但没有 $isFilter(\{\{1\}, \{1, 2\}\}, p_\supseteq)$ 因其不是向上闭合的, 也没有 $isFilter(\{\{1\}, \{2\}\}, p_\supseteq)$ 因其不是向下有向的。

对于滤子集, 另一个有用的事实是当在 $p_\supseteq$ 下, 若滤子集 f 中的元素均为函数, 因这些元素均是相容的, 其并集也为函数。即我们可以通过 f 中全部有限大的函数片段取并集构造完整函数¹。

进一步, 写 $p_{\leq, \nabla}$ 表示 $\langle p, \leq, \nabla \rangle$, 对任意 m :

- 写任意 $isForcingNotion_m(p_{\leq, \nabla})$ 表示:

$$p \neq \emptyset \wedge p_{\leq, \nabla} \in m \wedge isPartialOrder(p, \leq) \wedge \nabla \in p \wedge \forall x \in p \ x \leq \nabla$$

注意这里要求 $\nabla \in p$ 大于等于 p 中的任意元素 (即为 p 中唯一的最大值)。称 $p_{\leq, \nabla}$ 为 (m 的) 力迫概念, 称 p 中的元素为力迫条件。我们将看到力迫概念的选择决定了 g 可证明的语句 (即要证明不同的 ϕ , 我们需选择合适的 p)²。

- 写任意 $isGeneric_m(g, p_{\leq, \nabla})$ 表示:

$$g \subseteq p$$

$$\wedge \nabla \in g \tag{G1}$$

$$\wedge isFilter(g, p_\leq) \tag{G2}$$

$$\wedge \forall d \in m (d \subseteq p \wedge isDense(d, p_\leq) \rightarrow d \cap g \neq \emptyset) \tag{G3}$$

称 g 为 (m 上) $p_{\leq, \nabla}$ 的通集。注意 m 是传递的仅表示 p 中元素属于 m , 即 g 不一定属于 m ^{注3}。

可见到目前为止的定义都很难说是直观的, 现做初步解释, 并将在章节 5.3.8 提供更详细的关于这些概念的历史和讨论。本质上说, p 包含了已存在于 m 中的关于新模型可满足的一定特性的条件信息碎片, 这些碎片存在于 (一致的) 模型 (m) 中, 因此每一碎片本身显然不与该原模型矛盾。对 $st \in p$, s 为 t 的扩展指 s 相比于 t 提供了更多信息, 其中表示法 $s \leq t$ 表示满足 s 的模型少于满足 t 的模型。通集 g 的作用是从 p 中挑选整合这些信息碎片以得到更完整的信息, 可见 g 条件 (G2) 中的向上闭合性即指当 g 含某一条件时, 其也含这一条件的子条件碎片。同时 g 还需满足以下两个要求。一是其整合的条件信息结果应是不 (会在新模型中引入) 矛盾的, 这对应了 g 的向下闭合性。二是其整合的信息是足够多的, 或者说足以确定新模型中各语句真假性的 (而非只含原模型或部分新模型语句真假性的, 这在一定程度上反映了通集名字的由来⁴), 该要求对应了这里的 (G3)。而对于 (G1), 如上注述, 其是非必需的, 但其存在可以简化我们的讨论。最后, 我们将在章节 5.3.4 第一次具体应用力迫法而看到 $p_{\leq, \nabla} g$ 的具体例子。

有:

引理 5.3.1 $A_{ZFC} \vdash CTM(m) \wedge isForcingNotion_m(p_{\leq, \nabla}) \wedge u \in p \rightarrow \exists g \ u \in g \wedge isGeneric_m(g, p_{\leq, \nabla})$ 。

证明: 存在集合 $c = \{d \in m : d \subseteq p \wedge isDense(d, p_\leq)\}$, 由 m 可数存在 r 为其上的良序关系。令:

$$h = trec_\Pi [c, r, \overbrace{y = \min_\leq \{t \in p : t \leq u \wedge (\forall s \in ran f \ t \leq s) \wedge t \in x\}}^y]$$

¹如 189 页注 1。

²这里的力迫概念 (及下述选择公理否定形式独立性证明主要) 参考 J. R. Shoenfield. Unramified Forcing. 1971。具体我们要求 ∇ 的存在 (以简化处理), 而一些材料如 Thomas Jech. Set Theory: The Third Millennium Edition. Springer, 2003, pp. 201–204 及 N. Weaver. Forcing For Mathematicians. World Scientific Publishing Company, 2014, pp. 29–32 中并不做此要求, 但如 S. Shelah. Proper and Improper Forcing. Perspectives in Logic. Cambridge University Press, 2017, p. 3 所述对任意 $p_\leq$ 我们总是可以得到含 ∇ 的等价力迫概念。事实上力迫法在发展过程中对其中涉及的概念有不断抽象和泛化的过程, 见章节 5.3.8。

³当 $g \notin m$ 而 m 仍为 ZFC 模型的情况类似于章节 4.2.1 中介绍的斯科伦悖论中可数模型存在的情况。

⁴<https://zhuanlan.zhihu.com/p/93294818>. [编辑于 2020-02-28]。

可见如令 $d_0 = \min_r c$, $d_1 = \min_r (c \setminus d_0)$, 有 $h(d_0) \leq u \wedge h(d_0) \in d_0$, $h(d_1) \leq h(d_0) \wedge h(d_1) \in d_1$ 等。即对任意稠密集 $d \subseteq p$, $t = h(d)$ 扩展了任意 $s \in h[\text{seg}_r d]$; u 且 $t \in d$ 则 $\text{ran } h \cap d \neq \emptyset$ 。注意这里 d 稠密及 m 可数从而 c 可数保证理 γ 中 y 即 $h(d)$ 的存在¹。接着令:

$$g = \{ t \in p : \exists s \in \text{ran } h \ s \leq t \}$$

显然满足 isGeneric_m 的条件 (G1)。对 (G2)。上述定义直接满足了 (G2) 的向上闭合性。对向下有向性, 可见对任意 $t_1 t_2 \in g$, 存在 $s = h(d)$ 有 $s \leq t_1 \wedge s \leq t_2$ 且 $s \in g$, 即 s 同时为 $t_1 t_2$ 的扩展, 因此 $t_1 t_2$ 是相容的。□

当 $g \notin m$ 时, (为维持满足 A_{ZFC}) 向 m 中增加 g 意味着可能需要向 m 中增加别的相关集合, 直观上这是可行的因 $p \in m$ 而 $g \subset m$ 。而增加这些相关集合可能进一步引入新的集合, 为处理这种可能无限的迭代过程。我们通过下面的概念对关于 m 和 p 的元素作预处理, 使其可以一次性的解决引入任意 $g \subseteq p$ 所增加的新集合。同时我们仍需要注意区分元逻辑 ZFC 中的概念和其中 ZFC 模型的 (m) 中的概念, 如 $\text{isOrd}(a)$ 表示 a 为元逻辑中的序数, 而 $\text{isOrd}(a) \upharpoonright m$ 或 $\vdash_m \text{isOrd}(a)$ 表示 a 为 m 中的序数。注意这两者不一定相同, 如因 m 是可数的, 其中的 $\aleph_1$ 在元逻辑中也是可数的, 因此不等于元逻辑中的 $\aleph_1$ 。类似的, 我们可写 m 中 $\aleph_1$ 等为 $\aleph_1 \upharpoonright m$ 。

直观的说, 我们将以 m 中的元素作为名, 以下述方式确定名的值, 并由全部值的集合构成 $m[g]$ ^{注2}。具体的:

- 称 m 中的元素为名。
- 定义 m 上由 g 确定的名属于关系 $\epsilon_{m,g} = \{ \langle x, y \rangle \in m^2 : \exists s \in g \ \langle x, s \rangle \in y \}$ 。
- 对任意名 $y \in m$, 简写 y^* 表示其值 $\text{value}_{m,g}(y)$, 在 m 中在秩上递归定义 $y^* = \{ x^* : x \epsilon_{m,g} y \}$ 。该递归定义成立因任意 $x \epsilon_{m,g} y$ 有 $x \in \text{dom } y$ 有 $\text{rank}(x) < \text{rank}(y)$ 。注意不同的名可能有相同的值。
- 定义 $m[g] = \{ x^* : x \in m \}$ 。

写 $G_{m,p_{\leq \nabla}}(g)$ 表示 $CTM(m) \wedge \text{isForcingNotion}_m(p_{\leq \nabla}) \wedge \text{isGeneric}_m(g, p_{\leq \nabla})$ 。我们将证明:

定理 5.3.2 写 $\rho(u)$ 表示 $\text{isCountable}(u) \wedge \text{isTransitiveSet}(u) \wedge m \subseteq u \wedge g \in u \wedge (\vdash_{m[g]} A_{ZFC})$:

$$A_{ZFC} \vdash G_{m,p_{\leq \nabla}}(g) \rightarrow \rho(m[g]) \wedge \forall m' (\rho(m') \rightarrow m[g] \subseteq m')$$

即 $m[g]$ 满足 ρ 中的条件且为最小满足该条件的集合。

现逐一证明上述定理中的各个结论。

引理 5.3.3 $A_{ZFC} \vdash G_{m,p_{\leq \nabla}}(g) \rightarrow \text{isCountable}(m[g])$ 。

证明: $\text{value}_{m,g}$ 由 m 满射到 $m[g]$ 上而 m 可数。□

引理 5.3.4 $A_{ZFC} \vdash G_{m,p_{\leq \nabla}}(g) \rightarrow \text{isTransitiveSet}(m[g])$ 。

证明: 因名的值中的元素亦是名的值。□

¹当 m 不可数时, 如 $h(\omega)$ 可能不存在。可进一步参考 Asaf Karagila. Rasiowa-Sikorski Lemma. Mathematics Stack Exchange. URL: <https://math.stackexchange.com/q/1986032>。

²同样注意此处名的定义参考 J. R. Shoenfield. Unramified Forcing. 1971, 其不同于 N. Weaver. Forcing For Mathematicians. World Scientific Publishing Company, 2014, pp. 33–36 及 Wikipedia contributors. Forcing (mathematics) — Wikipedia, The Free Encyclopedia. [https://en.wikipedia.org/w/index.php?title=Forcing_\(mathematics\)&oldid=1096932150](https://en.wikipedia.org/w/index.php?title=Forcing_(mathematics)&oldid=1096932150)。2022 中关于 P-名的定义, 但其角色应是相同的。

为证明 $m \subseteq m[g]$:

- 对任意 $y \in m$, 简写 $\check{y}$ 表示 $\text{canon}_\nabla(y)$, 在 m 中在秩上递归定义 $\check{y} = \{ \langle \check{x}, \nabla \rangle : x \in y \}$, 即有 $\check{y} \in m$ 。同时归纳可证在 $G_{m, p_{\leq, \nabla}}(g)$ 时, 任意 $x \in m$ 有 $\check{x}^* = x$, 我们称 $\check{x}$ 为 x 的典范名。

有:

引理 5.3.5 $A_{ZFC} \vdash G_{m, p_{\leq, \nabla}}(g) \rightarrow m \subseteq m[g]$ 。

证明: 如上任意 $x \in m$ 的典范名 $\check{x} \in m$, 其值 $x \in m[g]$ 。 $\square$

引理 5.3.6 $A_{ZFC} \vdash G_{m, p_{\leq, \nabla}}(g) \rightarrow g \in m[g]$ 。

证明: 令 $h = \{ \langle \check{s}, \nabla \rangle : s \in p \}$ 。可见 $h \in m$ 且 $h^* = g$ 。 $\square$

至此我们可以得到 $m[g]$ 满足部分 A_{ZFC} 中的部分公理:

引理 5.3.7 $A_{ZFC} \vdash G_{m, p_{\leq, \nabla}}(g) \rightarrow (\models_{(m[g])} \text{Ext} \wedge \text{Empty} \wedge \text{Pairing} \wedge \text{Union} \wedge \text{Inf} \wedge \text{Regularity})$ 。

证明: 外延公理由 $m[g]$ 为传递集可得。空集公理由 $\emptyset \in m \subseteq m[g]$ 。配对公理对任意 $x^* y^* \in m[g]$, 构造名 $z = \{x, y\} \times \{\nabla\} \in m$ 有其值 $z^* = \{x^*, y^*\}$ 。并集公理对任意 $x^* \in m[g]$, 构造名 $z = (\bigcup x) \times \{\nabla\} \in m$ 有其值 $z^* = \bigcup x^*$ 。无限公理由 $m \subseteq m[g]$ 。正则公理由 (元逻辑) ZFC 满足正则公理。 $\square$

$m[g]$ 满足余下公理的关键在于力迫关系。

5.3.2 力迫关系

- 对任意 (m 上的) 力迫概念 $p_{\leq, \nabla}, s \in p$, 集合论语言 $\mathcal{L}_S$ 的式 $\ulcorner \phi(x_{1 \dots n}) \urcorner, v : [1, n] \mapsto m$, 其中 $[1, n] = n^+ \setminus 0$ (即 1 到 n 含, 并注意这里 n 是可由 $\ulcorner \phi \urcorner$ 计算的)。简写 $\Vdash$ 表示 (在 $s \phi v$ 上的) 力迫关系 $\Vdash_{m, p_{\leq, \nabla}}$, 写 $v_{[x]}$ 表示 $v(x)$, 在元逻辑 ZFC 中定义 $s \Vdash \phi(v_{[1] \dots [n]})$ 为¹:

$$\forall g (isGeneric_m(g, p_{\leq, \nabla}) \wedge s \in g \rightarrow \models_{(m[g])} \phi(v_{[1] \dots [n]}^*))$$

即定义 $s \Vdash \phi(v_{[1] \dots [n]})$ 若对任意 $p_{\leq, \nabla}$ 的通集 $g \ni s$ 有 $\models_{(m[g])} \phi(v_{[1] \dots [n]}^*)$ 。而类似章节 5.1.1 中对 $\vdash'$ 及 $\vdash' \vdash$ 的处理, 有 $s \Vdash' \ulcorner \phi \dots \urcorner$ 为 $s \Vdash \phi \dots$ 。同时注意这里 $v_{[1] \dots [n]}^*$ 的引入如章节 5.2.1 可构造全集的定义扩展了语言 $\mathcal{L}_S$ 使之含 $c_{1 \dots n}$ 在满足关系 $\models$ 中被解释为 $v_{[1] \dots [n]}^*$, 我们也称 $v_{[1] \dots [n]}^*$ 为语句 $\phi(v_{[1] \dots [n]})$ 中引入的值。而 $v_{[1] \dots [n]}$ 在力迫关系 $\Vdash$ 中也可通过该机制引入, 称其为引入的名。最后注意也因通集 g 不一定属于 m , 该定义需在元逻辑下完成。

尽管上述定义是在元逻辑中完成的, 但事实上力迫关系可仅由 (m) 和 $p_{\leq, \nabla}$ 决定:

引理 5.3.8

$$A_{ZFC} \vdash CTM(m) \wedge isForcingNotion_m(p_{\leq, \nabla})$$

$$\rightarrow \forall s \in p, \ulcorner \phi(x_{1 \dots n}) \urcorner, v : [1, n] \mapsto m \exists \ulcorner \varphi(s, x_{1 \dots n}) \urcorner (s \Vdash \phi(v_{[1] \dots [n]}) \leftrightarrow \models_{(m)} \varphi(s, v_{[1] \dots [n]})) \quad \text{注}^2$$

由此可见对 ϕ 的力迫关系可在 (m) 内由 φ 定义。即 $\{ \langle s, v_{[1] \dots [n]} \rangle : s \Vdash \phi(v_{[1] \dots [n]}) \}$ 为在 (m) 中的类。

¹其中 $v_{[1] \dots [n]}$ 表示 $v_{[1]}, \dots, v_{[n]}$ 。

²写 $\forall f : d \mapsto r \alpha$ 表示 $\forall f (f : d \mapsto r \rightarrow \alpha)$, 写 $\exists f : d \mapsto r \alpha$ 表示 $\exists f f : d \mapsto r \wedge \alpha$ 。

并有：

引理 5.3.9

$$A_{ZFC} \vdash CTM(m) \wedge isForcingNotion_m(p_{\leq, \nabla}) \\ \rightarrow \forall s t \in p, \ulcorner \phi(x_{1 \dots n}) \urcorner, v : [1, n] \mapsto m (s \leq t \wedge (t \Vdash \phi(v_{[1] \dots [n]})) \rightarrow s \Vdash \phi(v_{[1] \dots [n]}))$$

及：

引理 5.3.10 (力迫引理)

$$A_{ZFC} \vdash G_{m, p_{\leq, \nabla}}(g) \rightarrow \forall \ulcorner \phi(x_{1 \dots n}) \urcorner, v : [1, n] \mapsto m ((\models_{(m[g])} \phi(v_{[1] \dots [n]}^*)) \leftrightarrow \exists s \in g s \Vdash \phi(v_{[1] \dots [n]}))$$

由引理 5.3.8 和 5.3.10 可见在 (m) 中可定义的由 m 和力迫概念 $p_{\leq, \nabla}$ 决定的力迫关系可以决定以任意 $p_{\leq, \nabla}$ 的通集 g 扩展得到的解释 (m[g]) 中语句的真假。

而为证明上述三个引理，我们引入强力迫关系 $\Vdash_{m, p_{\leq, \nabla}}^*$ 或简写为 $\Vdash^*$ 。我们将证明上述三个引理由 $\Vdash^*$ 替换 $\Vdash$ 后对应的：

引理 5.3.11

$$A_{ZFC} \vdash CTM(m) \wedge isForcingNotion_m(p_{\leq, \nabla}) \\ \rightarrow \forall s \in p, \ulcorner \phi(x_{1 \dots n}) \urcorner, v : [1, n] \mapsto m \exists \ulcorner \varphi(s, x_{1 \dots n}) \urcorner (s \Vdash^* \phi(v_{[1] \dots [n]}) \leftrightarrow \models_{(m)} \varphi(s, v_{[1] \dots [n]}))$$

引理 5.3.12

$$A_{ZFC} \vdash CTM(m) \wedge isForcingNotion_m(p_{\leq, \nabla}) \\ \rightarrow \forall s t \in p, \ulcorner \phi(x_{1 \dots n}) \urcorner, v : [1, n] \mapsto m (s \leq t \wedge (t \Vdash^* \phi(v_{[1] \dots [n]})) \rightarrow s \Vdash^* \phi(v_{[1] \dots [n]}))$$

引理 5.3.13

$$A_{ZFC} \vdash G_{m, p_{\leq, \nabla}}(g) \rightarrow \forall \ulcorner \phi(x_{1 \dots n}) \urcorner, v : [1, n] \mapsto m ((\models_{(m[g])} \phi(v_{[1] \dots [n]}^*)) \leftrightarrow \exists s \in g s \Vdash^* \phi(v_{[1] \dots [n]}))$$

并证明：

引理 5.3.14

$$A_{ZFC} \vdash CTM(m) \wedge isForcingNotion_m(p_{\leq, \nabla}) \\ \rightarrow \forall s \in p, \ulcorner \phi(x_{1 \dots n}) \urcorner, v : [1, n] \mapsto m (s \Vdash \phi(v_{[1] \dots [n]}) \leftrightarrow s \Vdash^* \neg \neg \phi(v_{[1] \dots [n]}))$$

引理 5.3.8 至 5.3.10 的证明：上述四个引理结合任意 $(\models_m \phi) \leftrightarrow (\models_m \neg \neg \phi)$ 。

□

现首先定义 $\Vdash^*$ ：

- 取语言仅由逻辑联结词 $\neg \vee$ 及量词 $\exists$ ，符号 $\in$ 和 $\neq$ （则可定义 $x = y$ 为 $\neg x \neq y$ ）。同样对 $p_{\leq, \nabla}$ 、 $s \in p$ 、 $\ulcorner \phi(x_{1 \dots n}) \urcorner$ 、 $v : [1, n] \mapsto m$ ，递归定义¹：

¹注意如 (a) 中写 $v_{[1]} \in v_{[2]}$ 表示 $(x_1 \in x_2)(v_{[1]}, v_{[2]})$ ，或 $(x_1 \in x_2)(x_1/v_{[1]}][x_2/v_{[2]})$ 。

- (a) $s \Vdash^* v_{[1]} \in v_{[2]}$ 若 $\exists u \in m, t \geq s \langle u, t \rangle \in v_{[2]} \wedge s \Vdash^* u = v_{[1]}$ 。
- (b) $s \Vdash^* v_{[1]} \neq v_{[2]}$ 若 $(\exists u \in m, t \geq s \langle u, t \rangle \in v_{[1]} \wedge s \Vdash^* u \neq v_{[2]}) \vee (\exists u \in m, t \geq s \langle u, t \rangle \in v_{[2]} \wedge s \Vdash^* u \neq v_{[1]})$ 。
- (c) $s \Vdash^* \neg \phi(v_{[1] \dots [n]})$ 若 $\forall r \leq s \neg (r \Vdash^* \phi(v_{[1] \dots [n]}))$ 。
- (d) $s \Vdash^* \phi(v_{[1] \dots [n]}) \vee \varphi(v_{[1] \dots [n]})$ 若 $(s \Vdash^* \phi(v_{[1] \dots [n]})) \vee (s \Vdash^* \varphi(v_{[1] \dots [n]}))$ 。
- (e) $s \Vdash^* \exists x \phi(x, v_{[1] \dots [n]})$ 若 $\exists u \in m s \Vdash^* \phi(u, v_{[1] \dots [n]})$ 。

需指出该定义是非循环的，因此可以是递归的。具体的，如 (a) 和 (b) 定义互相依赖时，其中 u 的秩是小于 $v_{[1]}$ 或 $v_{[2]}$ 的秩的。同时可见其中涉及的变量均有界于 m ，因此其可在 m 中定义。

引理 5.3.11 的证明：如上所述，对上述定义归纳可证。 $\square$

引理 5.3.12 的证明：对 (a) 至 (e) 进行归纳可见。如对 (a)，归纳假设对任意 $r \leq s$ ， $s \Vdash^* u = v_{[1]}$ 蕴含 $r \Vdash^* u = v_{[1]}$ ，则显然 $\exists u \in m, t \geq s \langle u, t \rangle \in v_{[2]} \wedge s \Vdash^* u = v_{[1]}$ 有 $\exists u \in m, t \geq r \langle u, t \rangle \in v_{[2]} \wedge r \Vdash^* u = v_{[1]}$ ，即 $r \Vdash^* v_{[1]} \in v_{[2]}$ 。 $\square$

引理 5.3.13 的证明：首先证明若对 $\phi(x_{1 \dots n})$ 有对任意 $v : [1, n] \mapsto m$ 满足当前引理结论：

$$\models_{(m[g])} \phi(v_{[1] \dots [n]}^*) \leftrightarrow \exists s \in p s \Vdash^* \phi(v_{[1] \dots [n]}) \quad (5.3.1)$$

则：
$$v_{[1]} \in_{m,g} v_{[2]} \wedge (\models_{(m[g])} \phi(v_{[1] \dots [n]}^*)) \leftrightarrow \exists s \in g \exists t \geq s \langle v_{[1]}, t \rangle \in v_{[2]} \wedge s \Vdash^* \phi(v_{[1] \dots [n]}) \quad (5.3.2)$$

正方向若 $\leftrightarrow$ 左侧成立，由 $\in_{m,g}$ 的定义存在 $t \in g$ 有 $\langle v_{[1]}, t \rangle \in v_{[2]}$ ，由 (5.3.1) 存在 $r \in g$ 有 $r \Vdash^* \phi(v_{[1] \dots [n]})$ 。则由 g 的向下有向性有 $s \in g$ 满足 $s \leq t \wedge s \leq r$ 。且由引理 5.3.12 有 $s \Vdash^* \phi(v_{[1] \dots [n]})$ 。既得 $\leftrightarrow$ 右侧。反方向，由 (5.3.1) 有 $\models_{(m[g])} \phi(v_{[1] \dots [n]}^*)$ 。同时由 g 的向上闭合性有 $t \in g$ 有 $v_{[1]} \in_{m,g} v_{[2]}$ 。

接着同样对 (a) 至 (e) 进行归纳：

- (a) 由 $v_{[2]}^*$ 定义 $\models_{(m[g])} v_{[1]}^* \in v_{[2]}^*$ 等价于 $\exists u \in_{m,g} v_{[2]} \models_{(m[g])} u^* = v_{[1]}^*$ 。由归纳假设及 (5.3.2) 有其等价于 $\exists s \in g \exists u \in m, t \geq s \langle u, t \rangle \in v_{[2]} \wedge s \Vdash^* u = v_{[1]}$ ，即 $\exists s \in g s \Vdash^* v_{[1]} \in v_{[2]}$ 。
- (b) 类似 $\models_{(m[g])} v_{[1]}^* \neq v_{[2]}^*$ 等价于 $(\exists u \in_{m,g} v_{[1]} \models_{(m[g])} u^* \notin v_{[2]}) \vee (\exists u \in_{m,g} v_{[2]} \models_{(m[g])} u^* \notin v_{[1]})$ 。同样有 $(\exists s \in g \exists u \in m, t \geq s \langle u, t \rangle \in v_{[1]} \wedge s \Vdash^* u \neq v_{[2]}) \vee (\exists s \in g \exists u \in m, t \geq s \langle u, t \rangle \in v_{[2]} \wedge s \Vdash^* u \neq v_{[1]})$ ，即 $\exists s \in g s \Vdash^* v_{[1]} \neq v_{[2]}$ 。
- (c) 由归纳假设， $\models_{(m[g])} \neg \phi(v_{[1] \dots [n]}^*)$ ^{注1} 等价于 $\neg \xi$ 有 ξ 为 $\exists s \in g s \Vdash^* \phi(v_{[1] \dots [n]})$ 。则为证明其等价于 ζ 为 $\exists s \in g s \Vdash^* \neg \phi(v_{[1] \dots [n]})$ ，我们需要证明 $\xi \oplus \zeta$ （即其中有且仅有一个成立）。为证明 $\xi \vee \zeta$ ，由通集 g 定义条件 (G3) 仅需证 $d = \{ s \in p : (s \Vdash^* \phi(v_{[1] \dots [n]})) \vee (s \Vdash^* \neg \phi(v_{[1] \dots [n]})) \} \in m$ 且稠密。由引理 5.3.11 有 $d \in m$ 。对于 d 稠密，需证对任意 $s \in p$ 存在 $r' \leq s$ 有 $r' \in d$ 。由当前 $s \Vdash^* \neg \phi(v_{[1] \dots [n]})$ 的递归定义，若任意 $r \leq s$ 没有 $r \Vdash^* \phi(v_{[1] \dots [n]})$ 则 $s \in d$ 可取 $r' = s$ ，否则存在 $r \Vdash^* \phi(v_{[1] \dots [n]})$ 即 $r \in d$ 可取 $r' = r$ 。对于 $\neg(\xi \wedge \zeta)$ ，假设存在 $s_1 s_2 \in g$ 有 $s_1 \Vdash^* \phi(v_{[1] \dots [n]})$ 且 $s_2 \Vdash^* \neg \phi(v_{[1] \dots [n]})$ ，则存在 $r \leq s_1 \wedge r \leq s_2$ 由引理 5.3.12 有 $r \Vdash^* \phi(v_{[1] \dots [n]})$ ，但这与 $s_2 \Vdash^* \neg \phi(v_{[1] \dots [n]})$ 矛盾。
- (d) 由 $\models$ 定义有 $\models_{(m[g])} \phi(v_{[1] \dots [n]}^*) \vee \varphi(v_{[1] \dots [n]}^*)$ 等价于 $(\models_{(m[g])} \phi(v_{[1] \dots [n]}^*)) \vee (\models_{(m[g])} \varphi(v_{[1] \dots [n]}^*))$ ，由归纳假设其进一步等价于 $(\exists s_1 \in g s_1 \Vdash^* \phi(v_{[1] \dots [n]})) \vee (\exists s_2 \in g s_2 \Vdash^* \varphi(v_{[1] \dots [n]}))$ ，同样有存在 $r \in g$ 有 $r \leq s_1 \wedge r \leq s_2$ 有 $(r \Vdash^* \phi(v_{[1] \dots [n]})) \vee (r \Vdash^* \varphi(v_{[1] \dots [n]}))$ ，即 $\exists r \in g r \Vdash^* \phi(v_{[1] \dots [n]}) \vee \varphi(v_{[1] \dots [n]})$ 。
- (e) 如上同理。 $\square$

¹其中 $\dots$ 表示 $[1] \dots [n]$ 。

引理 5.3.14 的证明：正方向我们证明逆否成立，即若 $\neg(s \Vdash^* \neg\neg\phi(v\dots))$ ，由 (c) 有存在 $r \leq s$ 有 $r \Vdash^* \neg\phi(v\dots)$ ，由引理 5.3.1 有通集 g 满足 $r \in g$ ，则由引理 5.3.13 有 $\models_{(m[g])} \neg\phi(v\dots)$ ，即 $\neg \models_{(m[g])} \phi(v\dots)$ 。而由 g 的向上闭合性有 $s \in g$ ，则有 $\Vdash$ 定义有 $\neg(s \Vdash \phi(v\dots))$ 。

反方向若 $s \Vdash^* \neg\neg\phi(v\dots)$ ，对任意通集 $g \ni s$ 由引理 5.3.13 有 $\models_{(m[g])} \neg\neg\phi(v\dots)$ ，即 $\models_{(m[g])} \phi(v\dots)$ ，则 $s \Vdash \phi(v\dots)$ 。□

推论 5.3.14.1

$$A_{ZFC} \vdash CTM(m) \wedge isForcingNotion_m(p_{\leq, \nabla})$$

$$\rightarrow \forall s \in p, \ulcorner \phi(x_{1\dots n}) \urcorner, v : [1, n] \mapsto m (s \Vdash \neg\phi(v_{[1]\dots[n]}) \leftrightarrow \forall r \leq s \neg (r \Vdash \phi(v_{[1]\dots[n]}))) \quad (5.3.3)$$

$$\wedge \forall s \in p, \ulcorner \phi(x_{1\dots n}) \urcorner, v : [1, n] \mapsto m (s \Vdash \phi(v_{[1]\dots[n]}) \leftrightarrow \forall r \leq s \neg (r \Vdash \neg\phi(v_{[1]\dots[n]}))) \quad (5.3.4)$$

证明：对 (5.3.3)，令 (c) 中的 ϕ 为 (当前的) $\neg\neg\phi$ 并应用引理 5.3.14。对 (5.3.4)，替换 (5.3.3) 中的 ϕ 为 $\neg\phi$ 并注意由定义有 $s \Vdash \neg\neg\phi(v\dots)$ 等价于 $s \Vdash \phi(v\dots)$ 。□

5.3.3 力迫关系的应用

本小节将应用力迫关系证明 $m[g]$ 满足分类、替换公理模式，幂集公理和选择公理从而完成 $m[g]$ 为 ZFC 模型的证明。同时我们也将证明 $m[g]$ 为包含 m 和 g 的最小 ZF 模型。如此，我们即可完成定理 5.3.2 的证明。

引理 5.3.15

$$A_{ZFC} \vdash G_{m, p_{\leq, \nabla}}(g) \rightarrow \forall \ulcorner \rho(u_{1\dots n}, x) \urcorner, v : [1, n] \mapsto m, a \in m \exists b \in m (\models_{(m[g])} b^* = \{x \in a^* : \rho(v_{[1]\dots[n]}, x)\})$$

即对 $(m[g])$ 中任意 a^* 有进一步满足 ρ 的子集 b^* 注¹。

证明：令 $b = \{\langle x, s \rangle \in (dom a) \times p : (\exists t \geq s \langle x, t \rangle \in a) \wedge s \Vdash \rho(v\dots, x)\}$ 。由引理 5.3.8 有 $b \in m$ ，还需证对任意 $x \in b^* \leftrightarrow x \in a^* \wedge (\models_{(m[g])} \rho(v\dots, x))$ 。

正方向，对任意 $x \in b^*$ 由值定义存在 $y \in m$ 有 $y^* = x$ 且 $y \in_{m, g} b$ ，则由 $\in_{m, g}$ 定义存在 $s \in g$ 有 $\langle y, s \rangle \in b$ ，则存在 $t \geq s$ 有 $\langle y, t \rangle \in a$ ，由 g 的向上闭合性有 $t \in g$ 有 $x \in a^*$ 。同时有 $s \Vdash \rho(v\dots, x)$ ，由力迫引理 5.3.10 有 $\models_{(m[g])} \rho(v\dots, x)$ 。

反方向， $x \in a^*$ 有存在 $y \in m$ 有 $y^* = x$ 且 $y \in_{m, g} a$ ，即存在 $t \in g$ 有 $\langle y, t \rangle \in a$ ，而由 $\models_{(m[g])} \rho(v\dots, x)$ 有存在 $s \in g$ 有 $s \Vdash \rho(v\dots, y)$ ，由 g 的向下有向性存在 $r \in g$ 有 $r \leq s \wedge r \leq t$ ，则结合引理 5.3.9 有 $\langle y, r \rangle \in b$ 有 $x \in b^*$ 。□

推论 5.3.15.1 $A_{ZFC} \vdash G_{m, p_{\leq, \nabla}}(g) \rightarrow (\models_{(m[g])} Sep)$ 。

接着我们证明 $m[g]$ 满足替换公理模式：

引理 5.3.16 $A_{ZFC} \vdash G_{m, p_{\leq, \nabla}}(g) \rightarrow \forall w \subseteq m[g] (\exists z \in m \forall x \in w \exists y \in z x = y^* \rightarrow \exists k \in m[g] w \subseteq k)$ 。即任意 $m[g]$ 的子集 w 若满足其中任意元素 x 有存在其名 $y \in z \in m$ ，则 $w \in k \in m[g]$ 。

证明：令 $k = z \times \{\nabla\}$ ，可见 $k \in m \subseteq m[g]$ 。而对任意 $x \in w$ 有 $x = y^* \in k^*$ ，即 $w \in k \in m[g]$ 。□

¹可见这里的 $a b c$ 不为序数。

引理 5.3.17

$$A_{ZFC} \vdash G_{m,p \leq \gamma}(g) \rightarrow \forall \rho(u_{1 \dots n}, x, y)^\top, v : [1, n] \mapsto m, a \in m \exists k \in m[g] \{ y \in m[g] : \exists x \in a^* (\models_{(m[g])} \rho(v_{[1] \dots [n]}^*, x, y)) \} \subseteq k$$

即存在 $k \in m[g]$ 有 ρ 由 a^* 决定的值域的并集 $w = \{ y \in m[g] : \exists x \in a^* (\models_{(m[g])} \rho(v_{[1] \dots [n]}^*, x, y)) \} \subseteq k$ 。

证明：应用引理 5.3.16 需构造 $z \in m$ 对任意 $y \in w$ 存在名 $d \in z$ 。令：

$$z_0 = \{ e : \exists \langle c, s \rangle \in (\text{dom } a) \times p \forall d \in e \ s \Vdash \rho(v_{\dots}, c, d) \}$$

令 f 为 $z_0 \setminus \emptyset$ 的选择函数， $z = \text{ran } f$ 。可见 $z \in m$ 。而对任意 $y \in w$ ，存在 $x \in a^*$ 存在 $c \in \text{dom } a$ 有 $x = c^*$ 。且 $\models_{(m[g])} \rho(v_{\dots}^*, x, y)$ ，即存在 $s \in g \subseteq p$ ， $d \in m$ 有 $y = d^*$ 有 $s \Vdash \rho(v_{\dots}, c, d)$ 。则存在 y 的名 $d \in z$ 。□

推论 5.3.17.1 $A_{ZFC} \vdash G_{m,p \leq \gamma}(g) \rightarrow (\models_{(m[g])} \text{Rep})$ 。

证明：上述引理证明了 ρ 的值域 $w \subseteq k \in m[g]$ ^{注1}，应用推论 5.3.15.1 证明的分类公理模式。□

幂集公理：

引理 5.3.18 $A_{ZFC} \vdash G_{m,p \leq \gamma}(g) \rightarrow (\models_{(m[g])} \text{Power})$

证明²：对任意 $c^* \in m[g]$ ，我们将构造 a^* 有 $i : \forall x \in m (x^* \subseteq c^* \rightarrow x^* \in a^*)$ ，则应用分类公理模式可得在 $m[g]$ 下存在 $b^* = \wp(c^*)$ 。具体的，令 $a = d \times \{\gamma\}$ 有 $d = \{ e \in \wp(m \times p) : \text{dom } e \subseteq \text{dom } c \}$ 。可见 $d \in m$ ， $a \in m$ 。现需证明 i 。对任意 $x \in m$ ，构造 $y = \{ \langle w, s \rangle \in m \times p : w \in \text{dom } c \wedge s \Vdash w \in x \}$ 。可见 $y \in d$ 有 $y^* \in a^*$ 。则需证 $x^* = y^*$ 。

为证 $x^* \subseteq y^*$ ，对任意 $w^* \in x^*$ ，因 $x^* \subseteq c^*$ ， $\text{dom } w \in \text{dom } c$ 。而由力迫引理存在 $s \in g$ 满足 $s \Vdash w \in x$ 。则 $\langle w, s \rangle \in y$ 有 $w^* \in y^*$ 。为证 $y^* \subseteq x^*$ ，对任意 $w^* \in y^*$ ，存在 $s \in g$ 有 $\langle w, s \rangle \in y$ ，即 $s \Vdash w \in x$ ，则 $w^* \in x^*$ 。□

我们应用以下引理证明 $m[g]$ 为最小模型以及 $m[g]$ 满足选择公理：

引理 5.3.19

$$A_{ZFC} \vdash G_{m,p \leq \gamma}(g) \rightarrow (m \subseteq m' \wedge g \in m' \wedge (\models_{(m')} A_{ZF}) \rightarrow \exists f \in m' \text{ isFunction}(f) \wedge f \restriction m = \text{value}_{m,g})$$

证明：类似 $\epsilon_{m,g}$ 令 $\epsilon_{m',g} = \{ \langle x, y \rangle \in m'^2 : \exists s \in g \langle x, s \rangle \in y \}$ 。注意 $g \in m'$ 有 $\epsilon_{m',g} \in m'$ 。递归定义定义域为 m' 的函数 f 使其满足 $f(y) = \{ f(x) : x \in \epsilon_{m',g} y \}$ 。由 m 为传递集可见对任意 $x \in m$ ， $f(x) = \text{value}_{m,g}(x)$ (即 x^*)。□

推论 5.3.19.1 $A_{ZFC} \vdash G_{m,p \leq \gamma}(g) \rightarrow (m \subseteq m' \wedge g \in m' \wedge (\models_{(m')} A_{ZF}) \rightarrow m[g] \subseteq m')$ 。

证明：任意 $x^* \in m[g]$ 有 $x^* = f(x) \in m'$ 。□

引理 5.3.20 $A_{ZFC} \vdash G_{m,p \leq \gamma}(g) \rightarrow (\models_{(m[g])} \text{Choice})$ 。

¹注引理 5.3.17 不要求 ρ (在 a^* 内) 为函数类，即其强于这里所需的结论。但对应更强的版本事实上是 ZF 下可证的，见 K. Kunen. Set Theory An Introduction To Independence Proofs. ISSN. Elsevier Science, 2014, pp. 201–203, 108。

²参考 K. Kunen. Set Theory An Introduction To Independence Proofs. ISSN. Elsevier Science, 2014, pp. 201–203，其相较于 J. R. Shoenfield. Unramified Forcing. 1971 中的证明更为直接。

证明：应用引理 5.3.19 取 $m^\dagger = m[g]$ ，则存在 $f \in m[g]$ 有 $f \upharpoonright m = \text{value}_{m,g}$ 。同时对任意 $x \in m$ ，由 m 满足选择公理存在序数 a 有函数 $g \in m$ 满足 $\text{isBijection}_{\mathbb{F}}(g, a, \text{dom } x)$ 。则 $h = f \circ g$ 满足 $h \in m[g]$ 且 $\text{dom } h = a$ 而 $\text{ran } h = \{f(y) : y \in \text{dom } x\} = y^*y \in \text{dom } x$ 。结合值定义有 $x^* \subseteq \text{ran } h$ ，则通过 h 可定义 x^* 上的良序关系。 $\square$

定理 5.3.2 的证明¹：引理 5.3.3、5.3.4、5.3.5、5.3.6、5.3.7、推论 5.3.15.1、5.3.17.1、引理 5.3.18、推论 5.3.19.1 和引理 5.3.20。 $\square$

然而注意定理 5.3.2 证明了 $\vdash_{(m[g])} \text{A}_{ZFC}$ ，这使得显然 $m[g]$ 不能直接用于证明选择公理否定形式的相对一致性。我们需要进一步通过内模型的方法由 $(m[g])$ 构造 ZF 的模型 $(m^\dagger)$ 有 $m^\dagger$ 适用于定理 5.2.18 且其 ϕ 指出选择公理不成立²。

具体的，接下去几个小节的路线图如下。我们将首先应用力迫法完成可构造公理的独立性证明，在章节 5.2.3 及 5.2.4 中我们已经证明了可构造公理和 ZF 一致，因此我们仅需证明其否定形式也是一致的。而该证明除了为力迫法本身提供一个相对较简单的例子外，其也将被复用于选择公理否定形式的证明。对此，技术上，我们通过内模型的方法构造域相对可构造的特定 $m[g]$ 的内模型 $(m^\dagger)$ ，并证明其中含集合 z 不存在相对可构造的其上的良序关系。我们将在下一小节完成可构造公理的独立性证明，在章节 5.3.5 介绍上述扩展可构造概念后得到的相对可构造集合。

5.3.4 可构造公理的独立性

在章节 5.2.3 中，通过内模型的应用，引理 5.2.15 指出了可构造公理和 ZF 一致，结合章节 5.2.4 中引理 5.2.16 有可构造公理和 ZFC 一致。因此我们仅需证明其否定形式亦与 ZFC 一致。通过定理 5.2.19，我们仅需证明如下引理，证明过程中我们将应用力迫法构造 $m^\dagger$ ：

引理 5.3.21 $\text{A}_{ZFC_m} \vdash \exists m^\dagger \vdash_{(m^\dagger)} \text{A}_{ZFC} ; \exists z \forall a (\text{isOrd}(a) \rightarrow z \notin L_a)$ 。即模型 $(m^\dagger)$ 中存在 z 不属于任意 L_a 。

为此，我们首先证明：

引理 5.3.22 $\text{A}_{ZFC} \vdash G_{m,p \leq \gamma}(g) \rightarrow \forall a ((\vdash_m \text{isOrd}(a)) \leftrightarrow (\vdash_{(m[g])} \text{isOrd}(a)))$ 。即 (m) 和 $(m[g])$ 中序数相同。

证明：由 (m) 和 $(m[g])$ 均满足 A_{ZF} ， isOrd 为 Δ_0 式。则正方向由定理 5.3.2 指出的 $m \subseteq m[g]$ 直接可得。反方向，我们仅需证 $(m[g])$ 中的任意序数 $a \in m$ 。由值的定义可见对任意 x ， $\text{rank}(x^*) \in \text{rank}(x)$ ，则对任意 $(m[g])$ 中的序数 a ，令 $w \in m$ 为其名（即 $w^* = a$ ），则 $a = \text{rank}(a) \in \text{rank}(w) \in m$ ，由传递集有 $a \in m$ 。 $\square$

推论 5.3.22.1 写 $\rho(x)$ 表示 $\exists a \text{isOrd}(a) \wedge x \in L_a$ ：

$$\text{A}_{ZFC} \vdash G_{m,p \leq \gamma}(g) \rightarrow \{x \in m : \vdash_m \rho(x)\} = \{x \in m[g] : \vdash_{m[g]} \rho(x)\}$$

即 (m) 和 $(m[g])$ 中可构造全集相同。

¹其中除引理 5.3.18 外，主要参考 J. R. Shoenfield. Unramified Forcing. 1971。

²事实上，若要直接应用 $m[g]$ 于定理 5.2.19，则需 ϕ 指出 $(m[g])$ 下有内模型 $(m^\dagger)$ 满足 A_{ZF} 以及选择公理不成立。但注意这样的 $m^\dagger$ 为 $(m[g])$ 下的类（而非集合），因此若以此需扩展 $\vdash$ 等的定义以适用于类。而采用 5.2.18 我们即类似选择公理相对一致性证明，将域为类的模型推至了更基础的元逻辑中使其域可作为（该元逻辑中的）集合处理。

引理 5.3.23

$$A_{ZFC} \vdash CTM(m) \rightarrow \exists p_{\leq, \nabla} isForcingNotion_m(p_{\leq, \nabla}) \wedge \forall g (isGeneric_m(g, p_{\leq, \nabla}) \rightarrow \exists z \in m[g] z \notin m)$$

证明：令力迫概念 $p = \{ (w : u \mapsto 2) \in m : u \subset \omega \wedge |u| \in \omega \}$ ，即 (m) 中全部有限大小的自然数集到 $\{0, 1\}$ 的函数集合。注意这里我们通过有限大小（即 $|u| \in \omega$ ）保证 $p \in m$ ，需要以此保证因 p 的定义是在元逻辑中完成的，即 $u \subset \omega$ 不保证 $u \in m$ （进一步因 $x = \wp(\omega)$ 不是绝对的）。而可以以此保证因有限次的应用配对和并集公理我们可以在 (m) 中证明任意有限大小的 w 存在，有 $p \subseteq m$ ，则 $p = p \upharpoonright m \in m$ 。令 $\leq = \supseteq_p$ ^{注1}，同样 $\leq \in m$ 。则可见 $\nabla = \emptyset$ 。取 g 为 (m) 上 $p_{\leq, \nabla}$ 的通集。由 $g \in m[g]$ 有 $f = \bigcup g \in m[g]$ 。我们证明 $f \notin m$ ，或对任意 $h : \omega \mapsto 2 \in m$ 有 $h \neq f$ 。对任意这样的 h ，令：

$$d = \{ (w : u \mapsto 2) \in m : u \subseteq \omega \wedge \exists k \in \omega k \in u \wedge w(k) \neq h(k) \}$$

可见 d 稠密。由 (G3) 存在 $w \in g$ 且 $w \in d$ ，则存在 k 有 $f(k) \neq h(k)$ 。取 $z = f$ 。□

引理 5.3.21 的证明：令 $m = \mathbf{m}$ ，结合引理 5.3.23 存在 p 对任意 g ，存在 $z \in m[g]$ 且 $z \notin m$ 。取 $m^\dagger = m[g]$ ，结合推论 5.3.22.1 有模型 $(m^\dagger)$ 中 z 不属于任意 L_α 。□

鉴于 Z 已可形式化解释和模型的相关概念，并可证明如可靠和完备性等相关性^{2,3}，我们有：

定理 5.3.24

$$\begin{aligned} A_Z \vdash Con(A_{ZF}) &\rightarrow Con(A_{ZF}; constructibility) \wedge Con(A_{ZF}; \neg constructibility) \\ \wedge \exists m (\models_m A_{ZF}) &\rightarrow \exists m_1^\dagger (\models_{m_1^\dagger} A_{ZF}; constructibility) \wedge \exists m_2^\dagger (\models_{m_2^\dagger} A_{ZF}; \neg constructibility) \end{aligned}$$

可见（由可靠和完备性）这两行等价。

证明：引理 5.2.15、定理 5.2.17、5.2.19 和引理 5.3.21。具体的，可见其中 5.2.15 保证了可构造公理对 ZF 的一致性，而 5.2.17 保证了 ZFC 相对于 ZF 的一致性，则可以应用 5.2.19 并结合 5.3.21 以保证公理集为 $A_{ZFC}; \neg constructibility$ 的理论 T 相对于 ZF 的一致性⁴，即有 $A_{ZF}; \neg constructibility$ 理论相对于 ZF 一致。□

下面我们介绍选择公理否定形式相对一致性证明所需的相对可构造概念。

5.3.5 序数可定义集合

不严格的说，集合 w 为序数可定义集合若存在任意式 $\phi(x, u_{1...n})$ 、序数 $a_{1...n}$ 满足任意 $x \in w$ 当且仅当 $\phi(x, a_{1...n})$ 成立。显然序数可定义集合的合集为类（因序数合集为类）。而事实上我们仍无法直接照此构造式对序数可定义集合进行定义，因由塔斯基不可定义定理有 $\phi(x, a_{1...n})$ 成立是不可被式定义的。我们需借助哥德尔运算对其进行定义，并将（在元逻辑中）证明其与上述条件等价：

¹任意 $\supseteq_s$ 表示 $\{ \langle x, y \rangle \in s^2 : x \supseteq y \}$ 。

²见下文 210 页注 4。

³特别的，在可数语言下，如紧性定理的证明不需要选择公理，见 Wikipedia contributors. Gödel's completeness theorem — Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/w/index.php?title=G%C3%B6del%27s_completeness_theorem&oldid=1106103619. 2022。

⁴另注意这里相对一致的 T 中同时含选择公理和可构造公理的否定形式，结合 5.2.16 可见可构造公理是严格强于选择公理的。

- 定义序数可定义集合判定谓词 $isOD(w)$ 为 $\exists a isOrd(a) \wedge w \in cl(\{V_b : b \in a\})$ 。

有 $isOD$ 与前述条件等价：

定理 5.3.25

$$\begin{aligned} A_P \vdash \forall^\Gamma \phi(x, u_{1\dots n})^\Gamma A_{ZFC} \vdash \forall w, a (isOrd(a) \rightarrow \forall b : [1, n] \mapsto a (\forall x (x \in w \leftrightarrow \phi(x, b_{[1]\dots[n]})) \rightarrow isOD(w))) \\ \wedge \exists^\Gamma \phi(x, u)^\Gamma A_{ZFC} \vdash \forall w (isOD(w) \rightarrow \exists a isOrd(a) \wedge (\forall x x \in w \leftrightarrow \phi(x, a))) \end{aligned}$$

注意第二行表示有统一的 ϕ 用于定义满足 $isOD$ 的 w 。

证明：对第一行，对任意 ϕ ，由反射原理推论 5.2.8.2 存在极限序数 c 有 $x \subseteq V_c$ 且 $ran b \subseteq c$ 且：

$$\phi(x, b_{[1]\dots[n]}) \leftrightarrow (\phi(x, b_{[1]\dots[n]}) \upharpoonright V_c)$$

$\phi(x, b_{[1]\dots[n]}) \upharpoonright V_c$ 为 Δ_0 式，应用哥德尔范式定理 5.2.12 存在（由哥德尔运算组合而成的） $G(V_c, b_{[1]\dots[n]}) = w$ 。结合任意序数 $a = \{x \in V_a : isOrd(x)\}$ ，其中 $isOrd(x)$ （在 ZF 下）为 Δ_0 式，有 a 可通过哥德尔运算由 V_a 得到。则存在（由哥德尔运算组合而成的） $H(V_c, V_{b_{[1]}}, \dots, V_{b_{[n]}}) = w$ ，可见 $isOrd(w)$ 。

对第二行，由推论 5.2.16.1 有任意 $cl(\{V_b : b \in a\})$ 存在可定义的良好序关系，则按字典排序存在式 φ 定义 $cl(\{V_b : b \in a\})$ 合集上的良好序关系类。则存在式 ϕ 定义序数一对一映射到该合集的函数类。□

接着：

- 定义遗传序数可定义集合判定谓词 $isHOD(w)$ 为 $\forall x \in TC(w) isOD(x)$ 。

进一步我们对这两个概念进行一定的泛化（以用于选择公理否定形式相对一致性的证明）。同样不严格的，称集合 w 为在集合 q 上相对可定义的若存在任意式 $\phi(x, u_{1\dots n}, q, u'_1 \dots k)$ 、序数 $a_{1\dots n}, e_{1\dots k} \in q$ 满足任意 $x \in w$ 当且仅当 $\phi(x, a_{1\dots n}, q, e_{1\dots k})$ 成立。同样（同理）有等价的严格定义：

- 定义 q 上序数可定义集合判定谓词 $isOD(w, q)$ 为 $\exists a isOrd(a) \wedge w \in cl(\{V_b : b \in a\} \cup q; q)$ 。
- 定义 q 上遗传序数可定义集合判定谓词 $isHOD(w, q)$ 为 $\forall x \in TC(w) isOD(x, q)$ 。

类似章节 5.2.2 中对可构造全集构造对应的内模型的讨论，我们需要：

- 在元逻辑中，对任意 ZF 的模型 $m_V = (v, \epsilon)$ ，任意 q ，令 $h_q = \{x \in v : \models_{m_V} isHOD(w, q)\}$ 。写 ϵ_{h_q} 表示 $\epsilon \upharpoonright h_q$ 。写 $\mathfrak{H}_q(m_V)$ 表示解释 (h_q, ϵ_{h_q}) 。

则：

引理 5.3.26 $A_Z \vdash (\models_{m_V} A_{ZF}) \wedge q \in m_V[\forall] \rightarrow (\models_{\mathfrak{H}_q(m_V)} A_{ZF})$ 。即任意 ZF 模型中任意 q 上遗传序数可定义集合全集构成的解释为 ZF 的模型¹。

为此，我们首先证明以下更一般的引理：

引理 5.3.27

¹事实上，我们还有（未扩展的）遗传序数可定义集合全集为 ZFC 的模型，见 Thomas Jech. Set Theory: The Third Millennium Edition. Springer, 2003, pp. 194–195。

$$\begin{aligned}
& A_Z \vdash (\models_{(v,\epsilon)} A_{ZF}) \wedge \overbrace{w \subseteq v \wedge \exists \rho(x) \forall x \in w (\models_{(v,\epsilon)} \rho(x))}^{\alpha} \wedge \overbrace{(x \in y \wedge y \in w \rightarrow x \in w)}^{\beta} \\
& \wedge \overbrace{(\bigwedge_{1 \leq i \leq 5} \forall x y \in w \exists z \in w (\models_{(v,\epsilon)} z = G_i(x, y))) \wedge (\bigwedge_{6 \leq i \leq 10} \forall x \in w \exists z \in w (\models_{(v,\epsilon)} z = G_i(x)))}^{\gamma} \\
& \wedge \overbrace{\forall y \in v (\forall x \in y x \in w \rightarrow \exists z \in w \forall x \in y x \in z)}^{\delta} \rightarrow (\models_{(w,\epsilon_w)} A_{ZF})
\end{aligned}$$

其中首先 ϵ_w 表示 $\epsilon|_{w^2}$ 。条件 α 指出 w 为 $m_V = (v, \epsilon)$ 中的类, β 指出其是传递的, γ 指出 w 在哥德尔运算下是闭合的, δ 指出 m_V 下类 w 的任意子集 y 包含于某 $z \in w$ 。

证明: 同样类似引理 5.2.9 或定理 5.3.2 (包括引理 5.3.7 等) 我们逐一证明 $m_W = (w, \epsilon_w)$ 满足 A_{ZF} 中的各公理 / 公理模式。外延公理由 w 传递可得。空集公理由存在 $e \in v$ 在 m_V 下 $e = \emptyset$ 且由条件 δ 存在 $z \in w$ 有 $\forall x \in e x \in z$, 在 m_W 下由 (下方将证明的) 分类公理模式有 $\models_{m_W} e = \{x \in z : x \neq x\}$ 注¹。配对和并集公理即哥德尔运算 G_1 和 G_6 。

幂集公理类似空集公理的证明, 对任意 $s \in w$ 有 $s \in v$ 有 p 满足 $\models_{m_V} p = \wp(s)$, 写任意 $s \cap_\epsilon t$ 表示 $\{x \in s : x \in t\}$, 则存在 $q = w \cap_\epsilon p$, 存在 $z \in w$ 有 $\forall x \in q x \in z$, 有 $\models_{m_W} q = \{x \in z : x \subseteq s\}$, 可见 $\models_{m_W} q = \wp(s)$ 。无限公理类似, 对 $s \in v$ 有 $\models_{m_V} isInductive(s)$, 取 $t = w \cap_\epsilon s$, 存在 $z \in w$, 而上述 $e \in w$ 且 $e \in s$ 有 $e \in z$, 而由哥德尔运算闭合有任意 $n \in w$ 且 $n \in s$ 有 $n^+ \in w$ 有 $n^+ \in z$, 则 $\models_{m_W} isInductive(z)$ 。替换公理模式同样类似, 对任意集合 $s \in w$ 有 $s \in v$, 对任意函数类 ρ , 有 $t \in v$ 为由 s 经 ρ 映射所得, 取 $r = w \cap_\epsilon t$, 则存在 $z \in w$ 有 r 在 m_W 中可由 z 经分类公理模式可得。

我们现在证明分类公理模式。对任意 $s \in w$, $u_{1..n} \in w$, 式 $\rho[u_{1..n}](x)$ 注², 我们需要证明存在 $t \in w$ 满足 $\models_{m_W} t = \{x \in s : \rho(x)\}$ 。对任意式 ϕ , 写 ϕ_ϵ 表示将 ϕ 中的 $\in$ 关系 / 符号全部替换为 ϵ 后的新式。则在元逻辑中, 我们需要证明 i: 存在 $t \in w$ 满足 $t = \{x \in w : x \in s \wedge (\rho(x)|_w)_\epsilon\}$ 。

我们将证明 ii: 对任意自然数 k , 含 k 个量词的 $\forall$ 由 $\exists$ 定义的式 $\varphi(x_{1..n})$, 任意 $s \in w$, 任意 $x_{1..n} \in s$, 存在 $y_{1..k} \in w$ 有 $(\varphi(x_{1..n})|_w)_\epsilon$ 当且仅当 $\bar{\varphi}(x_{1..n}, y_{1..k})_\epsilon$, 其中 $\bar{\varphi}$ 为将 φ 中 k 个量词约束如 $\exists b_i$ 替换为 $\exists b_i \in y_i$ 后的新式。结合亦将证明的 iii: 对任意 Δ_0 式 $\psi(x, y_{1..k})$ 有 $\models_{m_W} \forall s, y_{1..k} \exists t \forall x (x \in t \leftrightarrow x \in s \wedge \psi(x, y_{1..k}))$ 。有 i 因由 ii 有 $\bar{\rho}(x, y_{1..k'})$ (k' 为 ρ 中量词个数) 对应 $\rho(x)|_w$ 且其为 Δ_0 式, 应用 iii 可得 $t \in w$ 满足相应条件。

我们首先对 k 归纳证明 ii。对 $k = 0$, $\bar{\varphi}$ 等于 φ , 成立。对 $k \geq 1$, 考虑 $\varphi(x_{1..n})$ 为 $\exists b \theta(x_{1..n}, b)$ 有 θ 含 $k-1$ 个量词, 则 $\bar{\varphi}$ 为 $\exists b \in y_k \bar{\theta}(x_{1..n}, b, y_{1..k-1})$ 。需证明 iv: 对任意 $s \in w$, 任意 $x_{1..n} \in s$, 存在 $y_{1..k} \in w$ 有:

$$(\exists b \theta(x_{1..n}, b)|_w)_\epsilon \leftrightarrow (\exists b \in y_k \bar{\theta}(x_{1..n}, b, y_{1..k-1}))_\epsilon$$

左侧等价于 ξ 为 $(\exists b (\rho(b)|_v) \wedge (\theta(x_{1..n}, b)|_w))_\epsilon$ 。我们证明 v: 有 $y_k \in w$ 有 ξ 等价于 $(\exists b \in y_k (\theta(x_{1..n}, b)|_w))_\epsilon$ 。为此, 我们仅需证明:

$$\models_{m_V} \exists y \forall x_{1..n} \in s (\exists b \rho(b) \wedge \vartheta(x_{1..n}, b) \leftrightarrow \exists b \in y \rho(b) \wedge \vartheta(x_{1..n}, b)) \quad (5.3.5)$$

其中要求 ϑ (在元逻辑中) 被解释为 $\theta|_w$, 显然 ϑ 是可在 ρ 和 θ 的基础上构造的。其可证明 v 因可见 $y \in v$, 而右侧 $\exists b \in y \rho(b)$ 表示 $b \in y' = w \cap_\epsilon y$, 则由条件 δ 有 $z \in w$ 有 $b \in z$, 则取 $y_k = s \cup z$ (以使 $s \subseteq y_k$, 见下述)。为证明 (5.3.5) 的正方向 (显然其反方向成立), 我们证明更一般的, 对任意式 π :

$$A_{ZF} \vdash \exists y \forall x_{1..n} \in s (\exists b \pi(x_{1..n}, b) \rightarrow \exists b \in y \pi(x_{1..n}, b))$$

¹ 可见若假设域中至少含一个元素 (该假设也为一般的域定义要求), 则分类公理模式蕴含空集公理。

² 即 $\rho[u_{1..n}, x]$ 或 $\rho(u_{1..n}, x)$, 如此下文我们强调 x 但 (可) 省略 $u_{1..n}$ 。

本质上，我们应用引理 5.2.8 中介绍的斯科特技巧从类中得到集合。具体的，对任意 $x_{1..n} \in s$ ，令 $\zeta_{[b]}$ 为 $\pi(x_{1..n}, b)$ 定义了类 b 满足 ζ ，则由斯科特技巧可（以一致方式）得到集合 c 有其中任意 $b \in c$ 有 ζ 。则由此可得定义域包含 s^n 的函数类，由替换公理模式存在 t 为其值域。则可取 $y = \bigcup t$ 。

在完成 v 的证明后，结合归纳假设及 $s \subseteq y_k$ 可得对任意 $x_{1..n} b \in y_k$ 有存在 $y_{1..k-1} \in w$ 满足 $(\theta(x_{1..n}, b) | w)_\epsilon$ 等价于 $\bar{\theta}(x_{1..n}, b,)$ 。我们即完成了对 iv 的证明，也就是完成了对 ii 的证明。

最后我们证明 iii 。因 ψ 为 Δ_0 式，由哥德尔范式定理 5.2.12 有：

$$\models_{m_W} \forall s, y_{1..k} \{ \langle x, y_{1..k} \rangle \in s \times \prod_{1 \leq i \leq k} \{y_i\} : \psi(x, y_{1..k}) \} \in cl(\{s, \{y_1\}, \dots, \{y_k\}\})$$

结合条件 γ 指出 w 在哥德尔运算下闭合：

$$\models_{m_W} \forall s, y_{1..k} \exists t t = \overbrace{dom \dots dom}^{k \text{ 次}} \{ \langle x, y_{1..k} \rangle \in s \times \prod_{1 \leq i \leq k} \{y_i\} : \psi(x, y_{1..k}) \}$$

可见 t 满足 iii 。 □

引理 5.3.26 的证明：为应用以上引理可见 $\mathfrak{H}_q(m_V)$ 由定义已满足条件 $\alpha\beta$ 和 γ 。而为证明 δ ，令 $m_V = (v, \epsilon)$ ， $\mathfrak{H}_q(m_V) = (w, \epsilon_w)$ ，令任意 $v_a \in v$ 对应 m_V 中的冯·诺伊曼层级 V_a ，对任意 $y \in v$ ，存在 v_a 有 $\forall x \in y x \in v_a$ （即在 m_V 下 $y \subseteq V_a$ ）。若 y 进一步满足 $\forall x \in y x \in w$ ，则 $\forall x \in y x \in w \cap_\epsilon v_a$ 因 $z = w \cap_\epsilon v_a$ （由分类公理模式）有 $z \in v$ 。则我们仅需证明 $z \in w$ 。为此，我们证明 $\models_{m_V} isOD(z, q)$ ，而事实上：

$$\models_{m_V} z = \{ x \in V_a : \forall y \in TC(\{x\}) \exists b z \in cl(\{V_c : c \in b\} \cup q; q) \} \quad \square$$

5.3.6 迭代力迫

迭代力迫即指多次应用力迫法构造的模型。在元逻辑下，由定理 5.3.2 可见由一次力迫法应用得到的 $m[g_1]$ 仍满足 $\models_{m[g_1]} A_{ZFC}$ 且可数且为传递集，意味着我们可以再次应用力迫法得到 $m' = m[g_1][g_2]$ ，依此类推。

若仅考虑有限次应用力迫法迭代，我们仅需讨论两次应用时的情况。简化表示法。写 $p_{\leq \square}$ 表示 $p_{\square \leq \square}$ ，写 $p_{\leq, \nabla \square}$ 表示 $p_{\square \leq \square, \nabla \square}$ ，如下 $p_{\leq, \nabla 1}$ 表示 $p_{1 \leq 1, \nabla 1}$ ：

定理 5.3.28

$$\begin{aligned} A_{ZFC} \vdash & CTM(m) \wedge isForcingNotion_m(p_{\leq, \nabla 1}) \wedge isForcingNotion_m(p_{\leq, \nabla 2}) \\ \wedge p = p_1 \times p_2 \wedge \leq = & \{ \langle \langle s_1, s_2 \rangle, \langle t_1, t_2 \rangle \rangle \in p^2 : s_1 \leq_1 t_1 \wedge s_2 \leq_2 t_2 \} \wedge \nabla = \langle \nabla_1, \nabla_2 \rangle \\ \rightarrow & \forall g_1, g_2 (isGeneric_m(g_1, p_{\leq, \nabla 1}) \wedge isGeneric_{m[g_1]}(g_2, p_{\leq, \nabla 2}) \\ & \rightarrow isGeneric_m(g_1 \times g_2, p_{\leq, \nabla}) \wedge m[g_1 \times g_2] = m[g_1][g_2]) \\ & \wedge \forall g (isGeneric_m(g, p_{\leq, \nabla}) \\ & \rightarrow g = dom g \times ran g \wedge isGeneric_m(dom g, p_{\leq, \nabla 1}) \wedge isGeneric_{m[g_1]}(ran g, p_{\leq, \nabla 2})) \end{aligned}$$

即对任意 $p_1 p_2$ 相关通集 $g_1 g_2$ 有 $g_1 \times g_2$ 为 $p_1 \times p_2$ 相关力迫概念 $p_{\leq, \nabla}$ 的通集，且 $m[g_1 \times g_2] = m[g_1][g_2]$ 。且以此可得任意 $p_{\leq, \nabla}$ 的通集 g 。

为此：

- 对任意 p 及其上偏序关系 $\leq$ 、任意 $e \subseteq p$ 、 $t \in p$ ，写 $isDenseBelow(e, p_{\leq}, t)$ 表示 $\forall s \leq t \exists r \in e \ r \leq s$ 。称 e 为 t 下稠密的。

引理 5.3.29

$$A_{ZFC} \vdash G_{m, p_{\leq, \nabla}}(g) \rightarrow \forall e \in m, t \in g (e \subseteq p \wedge isDenseBelow(e, p_{\leq}, t) \rightarrow e \cap g \neq \emptyset)$$

证明：我们首先证明由 $e \in m \wedge e \subseteq p$ 有 i: $e \cap g \neq \emptyset \vee \exists t' \in g \forall s \in e \neg isCompatible(s, t', p_{\leq})$ 。令：

$$d = \{ r \in p : \exists s \in e \ r \leq s \} \cup \{ t' \in p : \forall s \in e \neg isCompatible(s, t', p_{\leq}) \}$$

我们证明 d 是稠密的，考虑 $t' \in p$ 且 $t' \notin d$ （否则 $d \ni t' \leq t'$ ），则存在 $s \in e$ 有 $isCompatible(s, t', p_{\leq})$ ，即存在 $r \leq s \wedge r \leq t'$ ，可见 $r \in d$ 。则由 d 稠密有 $g \cap d \neq \emptyset$ ，分类讨论 $u \in g \cap d$ 时 u 所在 d 中的位置，结合 g 的向上闭合性，有其蕴含 i。

结合 $t \in g$ 且 $isDenseBelow(e, p_{\leq}, t)$ 。若 $e \cap g = \emptyset$ ，则由 i 存在 $t' \in g$ 有 $\forall s \in e \neg isCompatible(s, t', p_{\leq})$ 。由 g 的向下有向性有 $s' \leq t' \wedge s' \leq t$ ，因 e 在 t 下稠密，存在 $s \leq s'$ 且 $s \in e$ ，则 $s \leq t'$ ，与任意 s 和 t' 不相容矛盾。□

定理 5.3.28 的证明：首先注意由 $p_{\leq, \nabla}$ 定义显然有 $isForcingNotion_m(p_{\leq, \nabla})$ 。对于 $g = g_1 \times g_2$ 为 (m) 上 $p_{\leq, \nabla}$ 的通集，易见其满足通集定义条件 $g \subseteq p$ 及（由 $g_1 \ g_2$ 满足条件有）(G1)、(G2)。下面考虑 (G3)。对任意 $d \in m$ 满足 $d \subseteq p$ 且 $isDense(d, p_{\leq})$ ，要证明 $d \cap g \neq \emptyset$ ，即需证明对 $d_2 = \{ s_2 \in p_2 : \exists s_1 \in g_1 \langle s_1, s_2 \rangle \in d \}$ 有 $d_2 \cap g_2 \neq \emptyset$ 。可见 $d_2 \in m[g_1]$ 且 $d_2 \subseteq p_2$ ，则仅需证明 $isDense(d_2, p_{\leq_2})$ 。即需证对任意 $t_2 \in p_2$ 存在 $s_2 \leq_2 t_2$ 及 $s_1 \in g_1$ 有 $\langle s_1, s_2 \rangle \in d$ 。即对 $d_1 = \{ s_1 \in p_1 : \exists s_2 \leq_2 t_2 \langle s_1, s_2 \rangle \in d \}$ 有 $d_1 \cap g_1 \neq \emptyset$ 。同样 $d_1 \in m$ 且 $d_1 \subseteq p_1$ ，仅需证 $isDense(d_1, p_{\leq_1})$ 。即对任意 $t_1 \in p_1$ ，存在 $s_1 \leq_1 t_1$ 满足 $s_1 \in d_1$ ，而因 d 稠密，可取到 s_1 满足 $\langle s_1, s_2 \rangle \in d$ 且 $\langle s_1, s_2 \rangle \leq \langle t_1, t_2 \rangle$ 。

对 $m[g_1 \times g_2] = m[g_1][g_2]$ ，因由定理 5.3.2 或更具体的推论 5.3.19.1 有其均为对应模型满足 A_{ZF} 且包含 m 并含元素 $g_1 \ g_2$ 的最小集合。

现考虑结论的后半部分。对任意 (m) 上 $p_{\leq, \nabla}$ 的通集 g ，写 $g_1 = dom \ g, g_2 = ran \ g$ 。首先证明 $g = g_1 \times g_2$ ，显然 $g \subseteq g_1 \times g_2$ ，还需 $g_1 \times g_2 \subseteq g$ ，对任意 $\langle s_1, s_2 \rangle \in g_1 \times g_2$ ，由 $g_1 \ g_2$ 分别为 g 的定义域值域，存在 $t_1 \ t_2$ 有 $\langle s_1, t_2 \rangle \langle t_1, s_2 \rangle \in g$ ，由 g 的向下有向性存在共同扩展 $\langle r_1, r_2 \rangle \in g$ ，则由向上闭合性有 $\langle s_1, s_2 \rangle \in g$ 。

对于 $g_1 \ g_2$ 为 $p_1 \ p_2$ 相应的通集。显然 $g_1 \subseteq p_1, g_2 \subseteq p_2$ 。且易见 $g_1 \ g_2$ 满足条件 (G1)、(G2)。为证明 g_1 满足 (G3)，考虑 p_1 的子稠密集 $d_1 \in m$ ，可见 $d_1 \times p_2 \in m$ 为 p 的子稠密集，则 $d_1 \times p_2 \cap g \neq \emptyset$ ，则 $d_1 \cap g_1 \neq \emptyset$ 。

为证明 g_2 满足 (G3)，考虑 p_2 的子稠密集 $d_2 \in m[g_1]$ ，即 $\models_{(m[g_1])} isDense(d_2, p_{\leq_2})$ 。令 $c_2 = canon_{\nabla_1}(d_2)$ ， $q_2 = canon_{\nabla_1}(p_{\leq_2})$ 。写 $\phi(x)$ 表示 $t_1 \Vdash_{m, p_{\leq, \nabla_1}} isDense(c_2, q_2)$ 。则由力迫引理 5.3.10 存在 $t_1 \in g_1$ 有 $\phi(t_1)$ 。则令：

$$e = \{ \langle s_1, s_2 \rangle \in p_1 \times p_2 : s_1 \leq_1 t_1 \wedge s_1 \Vdash_{m, p_{\leq, \nabla_1}} canon_{\nabla_1}(s_2) \in c_2 \}$$

我们证明 $isDenseBelow(e, p_{\leq}, \langle t_1, \nabla_2 \rangle)$ 。对任意 $\langle s_1, s_2 \rangle \leq \langle t_1, \nabla_2 \rangle$ ，因 $s_1 \leq_1 t_1$ ，由引理 5.3.9 有 $\phi(s_1)$ 。则由力迫关系和稠密集的定义有 $s_1 \Vdash_{m, p_{\leq, \nabla_1}} \exists r_2 \in q_2 (x \in c_2 \wedge x \leq_2 canon_{\nabla_1}(s_2))$ 。这意味着存在 $r_1 \leq_1 s_1$ 及 $r_2 \in p_2$ 有 $r_1 \Vdash_{m, p_{\leq, \nabla_1}} canon_{\nabla_1}(r_2) \in c_2 \wedge canon_{\nabla_1}(r_2) \leq_2 canon_{\nabla_1}(s_2)$ 。则 $\langle r_1, r_2 \rangle \leq \langle s_1, s_2 \rangle$ 有 $\langle r_1, r_2 \rangle \in e$ 。

结合 $\langle t_1, \nabla_2 \rangle \in g$ ，应用引理 5.3.29 存在 $\langle s_1, s_2 \rangle \in e \cap g$ ，有 $s_1 \Vdash_{m, p_{\leq, \nabla_1}} canon_{\nabla_1}(s_2) \in c_2$ 。因 $s_1 \in g_1$ 由力迫引理有 $(m[g_1])$ 下 $s_2 \in d_2$ 。即 $d_2 \cap g_2 \neq \emptyset$ 。□

现我们考虑特定形式的无限迭代。本质上多元组可被视为索引集为有限序数的函数，则我们对力迫关系 $p_{\leq, \nabla_0}$ 迭代（可能无限的）索引集 j 次后的力迫关系为：

$$p_{\leq, \nabla_0}^{(j)} = \langle q, \{ \langle s, t \rangle \in q^2 : \forall i \in j : s^\circ(i) < t^\circ(i) \}, \emptyset \rangle$$

其中 $q = \{ s \in {}^j p : |dom s| \in \omega \}$ 。任意 $P(x^\circ(i))$ 表示 $\forall y ((i \in dom x \rightarrow y = x(i) : y = \nabla) \rightarrow P(y))$ 。可见（同样由 q 中任意元素有限大¹⁾） $p_{\leq, \nabla_0}^{(j)} \in m$ 。我们将证明：

定理 5.3.30

$$\begin{aligned} A_{ZFC} \vdash & isForcingNotion_m(p_{\leq, \nabla_0}) \wedge k \cap l = \emptyset \wedge k \cup l = j \rightarrow \forall g', g_1, g_2 (isGeneric_m(g', p_{\leq, \nabla_0}^{(j)}) \\ & \wedge g_1 = g' \upharpoonright k \wedge g_2 = g' \upharpoonright l \rightarrow isGeneric_m(g_1, p_{\leq, \nabla_0}^{(k)}) \wedge isGeneric_m(g_2, p_{\leq, \nabla_0}^{(l)}) \wedge m[g'] = m[g_1][g_2]) \end{aligned}$$

为此：

- 对任意 f ，力迫概念 $p_{\leq, \nabla_1}, p_{\leq, \nabla_2}$ ，写 $isCEmbedding(f, p_{\leq_1}, p_{\leq_2})$ 表示²⁾：

$$\begin{aligned} f : p_1 &\mapsto p_2 \\ \wedge \forall s_1 t_1 \in p (s_1 \leq_1 t_1 &\rightarrow f(s_1) \leq_2 f(t_1)) \end{aligned} \quad (C1)$$

$$\wedge \forall s_1 t_1 \in p (isCompatible(s_1, t_1, p_{\leq_1}) \leftrightarrow isCompatible(f(s_1), f(t_1), p_{\leq_2})) \quad (C2)$$

$$\wedge \forall t_2 \in q \exists t_1 \in p \forall s_1 \in p (s_1 \leq_1 t_1 \rightarrow isCompatible(f(s_1), t_2, p_{\leq_2})) \quad (C3)$$

显然：

引理 5.3.31

$$\begin{aligned} A_{ZFC} \vdash & isForcingNotion_m(p_{\leq, \nabla_1}) \wedge isForcingNotion_m(p_{\leq, \nabla_2}) \\ & \wedge isIsomorphism(f, p_{\leq_1}, p_{\leq_2}) \rightarrow isCEmbedding(f, p_{\leq_1}, p_{\leq_2}) \end{aligned}$$

证明：如对 (C3) 因 $\forall t_2 \in q \exists t_1 \in p \forall s_1 \in p (s_1 \leq_1 t_1 \rightarrow isCompatible(s_1, t_2, p_{\leq_2}))$ 。□

引理 5.3.32

$$\begin{aligned} A_{ZFC} \vdash & CTM(m) \wedge isForcingNotion_m(p_{\leq, \nabla}) \rightarrow \\ & (isGeneric_m(g, p_{\leq, \nabla}) \leftrightarrow g \subseteq p \wedge \nabla \in g \\ & \wedge (\forall s \in g, t \in p (s \leq t \rightarrow t \in g)) \quad (i) \\ & \wedge (\forall s t \in g \exists r \in p r \leq s \wedge r \leq t) \quad (ii) \\ & \wedge \forall d \in m (d \subseteq p \wedge isDense(d, p_{\leq}) \rightarrow d \cap g \neq \emptyset) \quad (iii) \end{aligned}$$

证明：对比通集定义³⁾， $\leftrightarrow$ 左右两侧仅 (ii) 与通集定义中向下有向性不同，且可见向下有向性蕴含 (ii)。则仅需证明反方向。假设右侧成立，考虑任意 $st \in g$ 存在 $r \in p$ 满足 $r \subseteq s \wedge r \subseteq t$ ，取：

$$d = \{ r \in p : \neg isCompatible(r, s, p_{\leq}) \vee \neg isCompatible(r, t, p_{\leq}) \vee (r \subseteq s \wedge r \subseteq t) \}$$

¹⁾ 见引理 5.3.21 的证明。

²⁾ (可进一步) 参考 K. Kunen. Set Theory An Introduction To Independence Proofs. ISSN. Elsevier Science, 2014, pp. 217–220。这里 $isCEmbedding$ 中的 C 表示 complete。

³⁾ 章节 5.3.1。

可见 $d \in m$, $d \subseteq p$ 且分类讨论有 $isDense(d, p_{\leq})$ 。则由 (iii) 有 $d \cap g \neq \emptyset$ 。取 $r' \in d \cap g$, 由 (ii) 有 r' 和 s, t 均相容, 则 $r' \in g$ 有 $r' \leq s \wedge r' \leq t$, 即向下有向。 $\square$

引理 5.3.33

$$\begin{aligned} A_{ZFC} \vdash & CTM(m) \wedge isForcingNotion_m(p_{\leq, \nabla_1}) \wedge isForcingNotion_m(p_{\leq, \nabla_2}) \\ & \wedge f \in m \wedge isCEmbedding(f, p_{\leq_1}, p_{\leq_2}) \\ & \rightarrow \forall g_2 (isGeneric_m(g_2, p_{\leq, \nabla_2}) \rightarrow isGeneric_m(f^{-1} \llbracket g_2 \rrbracket, p_{\leq, \nabla_1}) \wedge m[f^{-1} \llbracket g_2 \rrbracket] \subseteq m[g_2]) \end{aligned}$$

证明: 令 $g_1 = f^{-1} \llbracket g_2 \rrbracket$ 。我们首先应用引理 5.3.32 证明 g_1 是 $p_{\leq, \nabla_1}$ 的通集。对引理中要求的条件 (i), 假设 $s_1 \in g_1$ 且 $t_1 \in p_1$ 且 $s_1 \leq_1 t_1$ 但 $t_1 \notin g_1$, 则 $f(s_1) \in f(g_1)$ 且 $f(t_1) \in p_2$ 且由 (C1) 有 $f(s_1) \leq_2 f(t_1)$ 但 $f(t_1) \notin g_2$, 与 g_2 为通集矛盾。同理 (ii) 可由 (C2) 得到。对条件 (iii), 假设存在 p_1 的子稠密集 $d_1 \in m$ 有 $d_1 \cap g_1 = \emptyset$ 。令 $e_2 = f \llbracket d_1 \rrbracket$, 则 $e_2 \cap g_2 = \emptyset$ 。同时可见若存在 z_2 有任意 $x_2 \in e_2$ 有 $x_2 \leq_2 z_2$, 则 (因 d_1 稠密) e_2 在 z_2 下稠密, 否则 (同样) e_2 在 ∇_2 下稠密。应用引理 5.3.29 证明中的 i, 存在 $t'_2 \in g_2$ 对任意 $s'_2 \in e_2$ 有 $\neg isCompatible(s'_2, t'_2, p_{\leq_2})$, 即任意 $s'_1 \in d_1$ 有 $\neg isCompatible(f(s'_1), t'_2, p_{\leq_2})$ 。令 $t'_1 \in p_1$ 对 t'_2 满足 (C3) 中的任意 $s'_1 \in p$ 有 $s'_1 \leq_1 t'_1$ 蕴含 $isCompatible(f(s'_1), t'_2, p_{\leq_2})$, 则任意 $s'_1 \leq_1 t'_1$ 有 $s'_1 \notin d_1$, 这与 d_1 (对于 $p_{\leq_1}$) 稠密矛盾。

对于 $m[g_1] \subseteq m[g_2]$, 有 $f \in m \subseteq m[g_2]$, $g_2 \in m[g_2]$, 则 $g_1 = f^{-1} \llbracket g_2 \rrbracket \in m[g_2]$ 。而由推论 5.3.19.1 有 $m[g_1]$ 为包含 m 且有元素 g_1 且 $(m[g_1])$ 为 ZF 模型的最小集, 则 $m[g_1] \subseteq m[g_2]$ 。 $\square$

引理 5.3.34

$$\begin{aligned} A_{ZFC} \vdash & CTM(m) \wedge isForcingNotion_m(p_{\leq, \nabla_1}) \wedge isForcingNotion_m(p_{\leq, \nabla_2}) \\ & \wedge f \in m \wedge isIsomorphism(f, p_{\leq_1}, p_{\leq_2}) \\ & \rightarrow \forall g_1 (isGeneric_m(g_1, p_{\leq, \nabla_1}) \leftrightarrow isGeneric_m(f \llbracket g_1 \rrbracket, p_{\leq, \nabla_2})) \\ & \wedge (isGeneric_m(g_1, p_{\leq, \nabla_1}) \rightarrow m[g_1] = m[f \llbracket g_1 \rrbracket]) \end{aligned}$$

证明: 结合引理 5.3.31, 在两个方向上应用引理 5.3.33。 $\square$

定理 5.3.30 的证明: 令 $p_{\leq, \nabla'} = p_{\leq, \nabla_0}^{(j)}$, $p_{\leq, \nabla_1} = p_{\leq, \nabla_0}^{(k)}$, $p_{\leq, \nabla_2} = p_{\leq, \nabla_0}^{(l)}$ 。令 $p = p_1 \times p_2$:

$$p_{\leq, \nabla} = \langle p, \{ \langle \langle s_1, s_2 \rangle, \langle t_1, t_2 \rangle \rangle \in p^2 : s_1 \leq_1 t_1 \wedge s_2 \leq_2 t_2 \}, \langle \nabla_1, \nabla_2 \rangle \rangle$$

我们构造 $p_{\leq'}$ 和 $p_{\leq}$ 上的同构 $f: p' \mapsto p$ 有 $f(s) = \langle s \upharpoonright k, s \upharpoonright l \rangle$, 同样由其中元素均有限有 $f \in m$ 。则应用引理 5.3.34 对任意 g' 为 $p_{\leq, \nabla'}$ 的通集, 取 $g = f \llbracket g' \rrbracket$ 为 $p_{\leq, \nabla}$ 的通集且有 $m[g'] = m[g]$ 。同时可见 $g = g_1 \times g_2$, 则进一步应用定理 5.3.28 g_1, g_2 分别为 $p_{\leq, \nabla_1}$ 和 $p_{\leq, \nabla_2}$ 的通集且 $m[g'] = m[g] = m[g_1 \times g_2] = m[g_1] \llbracket g_2 \rrbracket$ 。 $\square$

至此, 我们基本完成了选择公理否定形式相对一致性证明的外部准备工作。

5.3.7 选择公理的独立性

我们在章节 5.2.4 中已经完成了选择公理相对一致性的证明。且至此, 我们有了大部分证明选择公理否定形式相对一致性所需的定义和性质。下面给出选择公理否定形式相对一致性的证明。本小节将是偏

技术的¹，下一小节会进一步（统一）补充关于直观性的讨论。

然而，仍需指出证明的核心在于尽管引理 5.1.13 表明任意 $\in$ 自同构均为恒等函数、即不同元素间总有不同 $\in$ 关系使其可区分，我们仍能在一定程度上构造元素间的自同构关系使其虽非 $\in$ 自同构但也无法在模型中被有效区分，从而无法被有效选择。

首先类似 $\in$ 自同构，定义：

- 任意 $isAutomorphism(f, p)$ 为 $isIsomorphism(f, p, p)$ 。称 f 为 p 的自同构。

引理 5.3.35

$$A_{ZFC} \vdash G_{m, p_{\leq, \nabla}}(g) \rightarrow \forall h \in m (isAutomorphism(h, p_{\leq}) \rightarrow isGeneric_m(h \llbracket g \rrbracket, p_{\leq, \nabla}) \wedge m[g] = m[h \llbracket g \rrbracket])$$

证明： $h \llbracket g \rrbracket$ 为 $p_{\leq, \nabla}$ 的通集因 h 在 $p_{\leq}$ 上自同构，如可见 $h(\nabla) = \nabla$ 。下面证明 $m[g] = m[h \llbracket g \rrbracket]$ 。由 $h \in m$ 有 $h \in m[g]$ ，结合 $g \in m[g]$ ， $h \llbracket g \rrbracket \in m[g]$ ，则由定理 5.3.2 (引理 5.3.19.1) 有 $m[h \llbracket g \rrbracket] \subseteq m[g]$ 。同理由 $h^{-1} \in m$ 有 $h^{-1} \in m[h \llbracket g \rrbracket]$ ， $h \llbracket g \rrbracket \in m[h \llbracket g \rrbracket]$ ，有 $m[g] = m[h^{-1} \llbracket h \llbracket g \rrbracket \rrbracket] \subseteq m[h \llbracket g \rrbracket]$ 。□

进一步对任意可数传递模型域 m 上力迫概念 $p_{\leq, \nabla}$ ，定义：

- 对任意 $w \in m$ 为 $p_{\leq}$ 上自同构的集合，任意 $x \in m$ ，写 $isInvariant_{m, p_{\leq, \nabla}}(x, w)$ 表示：

$$\forall g, h \in w (isGeneric_m(g, p_{\leq, \nabla}) \rightarrow value_{m, g}(x) = value_{m, h \llbracket g \rrbracket}(x))$$

称 x 是 w 不变量。如任意典范名在任意通集下值相同，因此是任意 w 不变量。若对于力迫关系中的语句 $\phi(v_{[1] \dots [n]})$ 有其中引入的名 $v_{[1] \dots [n]}$ 均是 w 不变量，称语句 $\phi(v_{[1] \dots [n]})$ 也 w 不变。

- 对任意 w 为 $p_{\leq}$ 上自同构的集合，写 $isHomogeneous_m(p_{\leq}, w)$ 表示：

$$\forall t_1 t_2 \in p \exists h \in w isCompatible(h^{-1}(t_1), t_2, p_{\leq})$$

称 $p_{\leq}$ 是 w 齐次的。简写 $isHomogeneous_m(p_{\leq})$ 表示：

$$isHomogeneous_m(p_{\leq}, \{ h \in p^2 : isAutomorphism(h, p_{\leq}) \})$$

即当 w 为 $p_{\leq}$ 上的全部自同构的集合。简称 $p_{\leq}$ 是齐次的。

例如 $p = \{ s : \omega \mapsto 2 : |s| \in \omega \}$ ， $p_{\leq} = \langle p, \supseteq_p \rangle$ 是齐次的。因对任意 $t_1 t_2 \in p$ ，取双射函数 $f : \omega \mapsto \omega$ 满足 $f \llbracket dom t_1 \rrbracket \cap dom t_2 = \emptyset$ ，取自同构 h 有 $h(t) = t \circ f$ ，则 $h^{-1}(t_1) \cup t_2 \in p$ 同时为 $h^{-1}(t_1)$ 和 t_2 的扩展。

则：

引理 5.3.36

$$\begin{aligned} A_{ZFC} \vdash G_{m, p_{\leq, \nabla}}(g) \rightarrow \forall w \in m, \ulcorner \phi(x_{1 \dots n}) \urcorner, v : [1, n] \mapsto m (\\ (\forall h \in w isAutomorphism(h, p_{\leq})) \wedge isHomogeneous_m(p_{\leq}, w) \wedge \forall i \in [1, n] isInvariant_{m, p_{\leq, \nabla}}(v[i], w) \\ \rightarrow ((\models_{(m[g])} \phi(v_{[1] \dots [n]}^{\star})) \leftrightarrow \nabla \Vdash \phi(v_{[1] \dots [n]}))) \end{aligned}$$

其中第二行指出 $p_{\leq}$ 满足 w 齐次， $\phi(v_{[1] \dots [n]})$ 满足 w 不变。可见 w 不变的语句在 w 齐次的力迫概念下其在模型 $(m[g])$ 中的真假不受 g 的选择影响。

¹ 仍参考 J. R. Shoenfield. Unramified Forcing. 1971。

证明：正方向考虑 $\models_{(m[g])} \phi(v_{[1] \dots [n]}^*)$ 但 $\neg(\nabla \Vdash \phi(v_{[1] \dots [n]}))$ 。前者由力迫引理 5.3.10 蕴含存在 $t_1 \in g \subseteq p$ 有 $t_1 \Vdash \phi(v_{[1] \dots [n]})$ 。后者由推论 5.3.14.1 中的 (5.3.4) 存在 $t_2 \leq \nabla$ 有 $t_2 \Vdash \neg \phi(v_{[1] \dots [n]})$ 。取 $h \in w$ 满足 $h^{-1}(t_1)$ 和 t_2 相容。令 $s \in p$ 同时为 $h^{-1}(t_1)$ 和 t_2 的扩展。由引理 5.3.1 存在 $p_{\leq, \nabla}$ 的通集 $g' \ni s$ ，由 g' 的向上闭包性， $h^{-1}(t_1) \in g'$ 且 $t_2 \in g'$ 。由引理 5.3.35 有 $h[g']$ 也为 $p_{\leq, \nabla}$ 的通集。则分别由 $t_1 \in h[g']$ 及 $t_2 \in g'$ ，我们有 $\models_{(m[h[g']])} \phi(v_{[1] \dots [n]}^*)$ 而 $\models_{(m[g'])} \neg \phi(v_{[1] \dots [n]}^*)$ 。但亦由引理 5.3.35 有 $m[h[g']] = m[g']$ 且由 $\phi(v_{[1] \dots [n]})$ 满足 w 不变 $v_{[1] \dots [n]}^*$ 在两模型中的解释相同，则应有 $\models_{(m[h[g']])} \phi(v_{[1] \dots [n]}^*)$ 当且仅当 $\models_{(m[g'])} \phi(v_{[1] \dots [n]}^*)$ 。矛盾。

反方向由引理 5.3.9 和 5.3.10（力迫引理）直接可得。 $\square$

引理 5.3.37

$$\begin{aligned} \text{A}_{ZFC} \vdash G_{m, p_{\leq, \nabla}}(g) \rightarrow \forall w \in m, z \in m[g], v : [1, k] \mapsto m \big(\\ (\forall h \in w \text{ isAutomorphism}(h, p_{\leq})) \wedge \text{isHomogeneous}_m(p_{\leq}, w) \wedge \forall i \in [1, k] \text{ isInvariant}_{m, p_{\leq, \nabla}}(v_{[i]}, w) \\ \wedge (\models_{(m[g])} \text{isOD}(z, \{v_{[1] \dots [k]}^*\})) \rightarrow z \cap m \in m) \end{aligned}$$

证明：回顾章节 5.3.5 序数可定义集合判定谓词 isOD 的定义，存在式 $\phi(x, u_{1 \dots n})$ ($n \geq k$) 有任意 $x \in z$ 当且仅当 $\models_{(m[g])} \phi(x, v_{[1] \dots [k]}^*, a_{1 \dots n-k})$ ，其中 $a_{1 \dots n-k}$ 为 $(m[g])$ 中的序数。由引理 5.3.22， $a_{1 \dots n-k} \in m$ ，则 $\check{a}_{1 \dots n-k}$ 皆为 w 不变的。对 $x \in z \cap m$ ，即 $x \in m$ ， $\check{x}$ 也为 w 不变的，则 $\phi(\check{x}, v_{[1] \dots [k]}, \check{a}_{1 \dots k})$ 满足 w 不变，应用引理 5.3.36 有：

$$x \in z \cap m \leftrightarrow (\models_{(m[g])} \phi(x, v_{[1] \dots [k]}^*, a_{1 \dots n-k})) \leftrightarrow \nabla \Vdash \phi(\check{x}, v_{[1] \dots [k]}, \check{a}_{1 \dots k})$$

由引理 5.3.8 可见 x 是否满足 $\nabla \Vdash \phi(\check{x}, v_{[1] \dots [k]}, \check{a}_{1 \dots k})$ 可在 (m) 中定义，写 $\varphi(x)$ 表示该定义。进一步取 b 为 z 在 $(m[g])$ 中的秩，则 $z \cap m \subseteq (V_b \restriction m[g]) \cap m = V_b \restriction m$ ，其中第二个等于由对 b 归纳可得。则 $z \cap m = \{x \in V_b \restriction m : \varphi(x)\} \in m$ 。 $\square$

写 A_{ZF-C} 表示 A_{ZF} ； $\neg \text{Choice}$ ：

引理 5.3.38 $\text{A}_{ZFC_m} \vdash \exists m^\ddagger (\models_{(m^\ddagger)} \text{A}_{ZF-C})$ 。

证明：由定理 4.3.54 选择公理与良序定理等价，及定理 4.3.13 存在序数及其上大小关系与良序结构同构，我们可通过构造 $(m^\ddagger)$ 有其中不存在序数 b 有满射函数 $f : b \mapsto \wp(\omega)$ 证明选择公理的否定形式在 $(m^\ddagger)$ 中为真。取 $m = \mathbf{m}$ 。令 $p_0 = \{w : u \mapsto 2 : u \subseteq \omega \wedge |u| \in \omega\}$ ， $p_{\leq, \nabla_0} = \langle p_0, \subseteq_{p_0}, \emptyset \rangle$ ，如引理 5.3.23 证明， $p_{\leq, \nabla_0}$ 为 m 的力迫概念。 $p_{\leq, \nabla'} = p_{\leq, \nabla_0}^{(\omega)}$ 为 m 的力迫概念。令 g' 为 $p_{\leq, \nabla'}$ 的通集。对 $i \in \omega$ ，令 $q_i = g' \restriction \{i\}$ 。结合定理 5.3.30 可见 q_i 为 $p_{\leq, \nabla_0}^{(i)}$ 的通集， $q = \{q_i : i \in \omega\}$ ，构造 $(m^\ddagger) = \mathfrak{S}_q((m[g']))$ 。即 $m^\ddagger$ 由模型 $(m[g'])$ 中 q 上的遗传序数可定义集合构成。我们证明 $(m^\ddagger)$ 中不存在上述序数 b 。

假设存在序数 b 存在满射函数 $f : b \mapsto \wp(\omega)$ ，则 $\models_{(m^\ddagger)} \text{isOD}(f, q)$ ，即在 $(m^\ddagger)$ 中存在 $n \in \omega$ ， f 可自（有限数量的）序数及 $q_0 \dots q_{n-1}, q$ 定义。则 $(m^\ddagger)$ 中任意自然数集合可由 f 和某序数 $a \in b$ 定义，即同样可自序数及 $q_0 \dots q_{n-1}, q$ 定义。

考虑 $g_1 = g' \restriction n$ ， $g_2 = g' \restriction \omega - n$ 。仍由定理 5.3.30 可见 $m[g'] = m[g_1][g_2]$ 。写 $m' = m[g_1]$ ，我们将构造集合 $z \cap m' \notin m'$ ，并在下面的段落证明 $z \cap m' \in m'$ 从而得到矛盾。再次由定理 5.3.30 有 q_n 为 m' 上 $p_{\leq, \nabla_0}^{(n)}$ 的通集。同时由 q_n 定义存在 q'_n 有 $q_n = \{n\} \times q'_n$ ，且 $\{n\}$ 为力迫概念 $\langle \{n\}, \{\langle n, n \rangle\}, n \rangle$ 的通集而 q'_n 为 $p_{\leq, \nabla_0}$ 的通集，即 $m'[q_n] = m'[\{n\}][q'_n]$ 。又因 $\{n\} \in m[g_1]$ 由引理 5.3.19.1 最小性 $m' = m'[\{n\}]$ 。即仅需考虑 $m'[q'_n]$ 。则由引理 5.3.23（及其证明）取 $z_f = \bigcup q'_n$ 有 $z_f \notin m'$ 。进一步取 $z = \{x \in \omega : z_f(x) = 1\} \notin m'$ （因否则 $z_f = \{\langle x, y \rangle \in \omega \times 2 : x \in z? y = 1 : y = 0\}$ ）。而 $z \subset \omega \subset m'$ 有 $z \cap m' = z$ ，则 $z \cap m' \notin m'$ 。

最后由引理 5.3.37 证明 $z \cap m' \in m$ 。对任意双射函数 $h_0 : \omega - n \mapsto \omega - n$ ，取 $h : p_2 \mapsto p_2$ （注意 $p_2 = p_0^{(\omega-n)}$ ）满足对任意 $s \in p_2$ 、 $i \in \omega - n$ ， $h(s)(i) = s(h_0(i))$ ，可见 h 为 $p_{\leq 2}$ 上的自同构，其在 s 中用 $h_0(i)$ 对应的值替

换了 i 对应的值。取 $w \subset m'$ 由全部这样的 h 构成。则 $w = w|_{m'} \in m'$ 。同时可见所有执行有限次替换的 h 均属于 m' ，类似齐次定义中的例，有 $p_{\leq 2}$ 为 w 齐次的。进一步看到 $z \subset \omega$ 可自 q_n 定义，其是 q 上序数可定义的，结合其中元素自然数均属于 $m^\dagger$ ， $z \in m^\dagger$ 。则由归纳假设有其可自序数和 $q_{0 \dots n-1}, q$ 定义，则要应用引理 5.3.37，我们仅需证明 m' 中含 $q_{0 \dots n-1}, q$ 的 w 不变名。

由 $q_{0 \dots n-1} \in m'$ ，其典范名 $\check{q}_{0 \dots n-1}$ 即 w 不变名。我们还需构造 q 的 w 不变名 y 。对任意 $i \in \omega$ ，取 $y_i = \check{q}_i$ 若 $i < n$ 否则 $\{\langle \check{s}_i, s \rangle : s \in p_2\}$ ，取 $y = \{\langle y_i, \nabla_2 \rangle : i \in \omega\}$ 。可见 y_i 均为 q_i 的名，有 y 为 q 的名。对任意 $m'[h[g_2]]$ ，若 $i < n$ ， y_i 仍为 q_i 的名，否则 y_i 为 $g_2 \upharpoonright_{\{h_0(i)\}}$ 的名。由 h_0 为双射函数， y 仍包含全部 q_i 的名， y 仍为 q 的名。则 h 也是 w 不变的。□

定理 5.3.39 $A_Z \vdash \text{Con}(A_{ZF}) \rightarrow \text{Con}(A_{ZFC}) \wedge \text{Con}(A_{ZF-C})$ 。

证明：定理 5.2.17，5.2.18 和引理 5.3.38。□

5.3.8 力迫法的起源与发展

本书到目前为止很多内容都是偏技术性的，然而本书的最终目的在于阐明存粹的技术性内容（在本书所采用的认识论下）尽管繁琐但应是绝对可知的。形式化方法的本质在于其令存粹技术性的内容可被机器识别、检查甚至生成。因此并不应过分强调技术性内容的重要性。更重要的部分应是技术性内容对真实世界的适用性，这种适用性可体现在其应是符合直觉的，因此其生成也应依赖直觉。我们将在后续章节看到机器生成仍需指明较为清晰的目标且其过程很多时候仍是十分低效的。而如若采用机械式的方式，人类显然更易混淆目标¹且更为低效。

更直白的，技术性的内容即使被检查确认是符合规则的，有时我们也很难建立起对其的直观感受。在感叹于证明之巧妙时，我们看到更多的只是技巧而非原因。比如仅通过阅读最终的结果，我们有时很难理解很多抽象概念定义、性质定理等的背后出发点。这种感叹事实上反映了直观理解的难以复制性。但这种直觉本质上为我们的探索（搜索）提供了尽管模糊的方向，其即是我们的对应于人工智能的启发函数²，其难以复制性即对应了启发函数的难以解释性。

而对应于软件工程，最现代的理论成果的技术性汇总可认为是最新版本的代码，其发展同样经历了不断的抽象、模式的引入和重构。然而事实上抽象本身在更多的时候并不是以使代码更易理解为目的的，甚至我们并不总能判断抽象后的代码是否更易于理解。而当我们不理解一段代码的初衷时，对其的改动显然更容易对整个系统造成破坏。在难以对最新版本下手的时候，我们在代码的版本更替中寻找答案。在这方向，软件工程显然是更具优势的，其各类版本管理工具是相对更为成熟的，从而其历史是相对集中的。这是其内生的形式化形式的一个好处，亦是可借鉴的部分。但显然这种对历史的梳理仍不能从根本上解决理解的问题，但至少提供了又一个模糊的方向。

¹如学童可能犯的一种典型错误（计算方法）：

$$\begin{array}{r} 54 \\ - 23 \\ \hline 31 \end{array} \quad \checkmark \quad \begin{array}{r} 54 \\ - 28 \\ \hline 34 \end{array} \quad \times \quad \begin{array}{r} 612 \\ - 39 \\ \hline 627 \end{array} \quad \times \quad \begin{array}{r} 317 \\ - 81 \\ \hline 376 \end{array} \quad \times$$

可见其显然同样是符合严格计算规则的。引自 Stanislas. Dehaene. The number sense : how the mind creates mathematics / Stanislas Dehaene. English. Oxford University Press New York, 1997, xi, 274 p. .。

²见后续章节。

抛开工具上的技术性细节，我们在本小节中对力迫法的起源和发展¹做一定的讨论，作为理论发展的一个实例，从中尝试看到其发展的一点脉络。我们将看到其技术上的最初形式与本书介绍的有显见的不同²，且其显然是更贴近问题而更易理解的（并甚至是在某种意义上更统一的）。

我们的重点将放在保罗·寇恩（Paul Cohen）最初发明的于1963年发表的力迫法形式上³。将尝试向前梳理其发明力迫法的基础，向后梳理力迫法的发展过程。我们仍将主要以可构造公理（对比 章节 5.3.4）和选择公理（5.3.7）否定形式的相对一致性证明为例。

在经历了一系列 ZFC 集合论下的讨论，现应更明确的注意到朴素集合论公理集的简洁。然而罗素悖论指出了其的无效性。也因而策梅洛于1908年提出了 Z 集合论⁴。然而新公理系统自提出起其一致性就成为了当时新待解决的问题，直至哥德尔于1931年证明了哥德尔（第二）不完备定理⁵，即其指出一定强度的如 Z 集合论无法证明其自身的一致性。自此，探索的主要方向来到了相对一致性上，即由含较直观较无异议假设的较弱理论证明较强理论的一致性。一个发展是哥德尔于1938年通过可构造全集和内模型的方法证明了选择公理（和连续统假设）的相对一致性⁶。

寇恩发明力迫法的关键在于他发现了最小内模型的存在，这意味着内模型证明相对一致性的适用性极限的存在。因而寇恩转而看向模型外，即尝试在元逻辑中构造更大的模型，或用他的话说⁷：

“这并不代表我们不能通过外部于 ZF 理论的手段、如通过基于对理论中全部语句真假的遍历构造模型。”

显而易见这（其中后半部分）和上文介绍的方法有不小的差异，下面对此做一定展开。

如上引用所述为通过对语句真假遍历构造模型，这里的出发点应是由力迫条件集（条件集的集合即后发展为这里的力迫概念和通集）到语句真假的力迫关系应反映 ZF 模型构造的基本要求，并同时具有一定的灵活性。由他的论文⁸可见，寇恩依据哥德尔运算⁹设计了力迫关系，使不矛盾但又足够确定全部语句真假性的条件集所对应的由哥德尔闭包得到的相对特定集合的可构造全集（即对应相对序数可定义集合类）满足 A_{ZF} 。而其中的相对性和可构造集合的构造方法提供了所需的灵活性。

首先以可构造公理否定形式的相对一致性为例。这里的目标是构造不可构造的自然数子集合 o ，其采用的条件是自然数 x 是否属于 o ，其在该条件上建立了与哥德尔运算相对应的力迫关系，而最终构造的条件集将不同时有 $x \in o$ 且 $x \notin o$ 以避免矛盾，且将覆盖全部自然数。现自 $\emptyset$ 起逐次向条件集增加条件（视每次增加得到一个条件集，则最终我们得到的即是条件集的集合，即这里的通集）。注意这里需要遍历全部语句，也需要遍历全部可构造集合，寇恩在奇数次处理语句的真假以使最终结果为 ZF 的模型，在偶数次处理可构造集合以获得不可构造的集合。具体的，在 $2n+1$ 次时向条件集中添加最小 x 是否属于 o 以使新条件集中不含矛盾（即没有 x 同时属于且不属于 o ），且要么力迫第 n 个语句要么力迫该语句的否定。在 $2n$ 次时取条件集中还未出现的最小 x ，并依据 x 属于或不属于第 n 个可构造集合向条件集中加入 x 不属于或属于 o ，由此可见最后由条件集确定的集合不等于任意可构造集合。在 o 确定后相对 o 的

¹Gregory H. Moore. “The Origins of Forcing”. In: Logic Colloquium '86. Ed. by F.R. Drake and J.K. Truss. Vol. 124. Studies in Logic and the Foundations of Mathematics. Elsevier, 1987, pp. 143–173.

²尽管本书中仍主要选择参考了较折中的版本，即 J. R. Shoenfield. Unramified Forcing. 1971。

³Paul J. Cohen. The Independence of the Axiom of Choice. 1963。

⁴Ernst Zermelo. “Investigations in the foundations of set theory I”. in: From Frege to Gödel. Ed. by Jean Van Heijenoort. Cambridge: Harvard University Press, 1967, pp. 199–215。

⁵Kurt Gödel. “Some metamathematical results on completeness and consistency, on formally undecidable propositions of principia mathematica and related systems i, and on completeness and consistency”. In: From Frege to Gödel. Ed. by Jean Van Heijenoort. Cambridge: Harvard University Press, 1967, pp. 592–617。

⁶Kurt Gödel. “The Consistency of the Axiom of Choice and of the Generalized Continuum-Hypothesis”. In: Proceedings of the National Academy of Sciences 24.12 (1938), pp. 556–557。

⁷Paul J. Cohen. A minimal model for set theory. 1963。

⁸注 3。

⁹其略不同于本书介绍的哥德尔运算。见 181 页注 4。

可构造全集结合力迫关系的定义为 ZF 的模型（且其中的 ω 不可构造）。

对于选择公理的否定形式。对任意自然数 i, j ，令 $o_{ij} \subseteq \omega$ ，任意 i ， $u_i = \{o_{ij} : j \in \omega\}$ ， $v = \{u_i : i \in \omega\}$ 。目标是证明对任意相对可构造集合 t 存在 i ， $|t \cap u_i| \neq 1$ （因令 $u'_i = \{i\} \times u_i$ ， $v' = \{u'_i : i \in \omega\}$ ，若 v' 的选择函数 f 存在，对任意 i ， $|f \cap u'_i| = 1$ ）。这里的条件是对自然数 x, i, j ，是否有 $x \in o_{ij}$ 。这里对应于 $o_{ij} u_i v$ 的引入和条件的改动，力迫关系的定义也需进行相应的（但仍符合哥德尔运算给出的可构造关系）的改动。同时，为使任意 u_i 中有不可区分的元素，这里同样采用了双射函数 h 重排 u_i 中的元素。同时 h 被扩展为可操作任意（新模型中）的集合，力迫条件和语句以保证 i ：若条件集 s 力迫 ϕ ，则 $h(s)$ 力迫 $h(\phi)$ ，为实现该保证，寇恩进一步修改了可构造全集的构造方法使在构造中有对应的对称性。在此基础上，寇恩在偶数次处理所遍历的语句真假，在奇数次保证存在 i 使 u_i 和当前遍历到的可构造集合 z 的交集元素个数大于 1。具体的，若 z 还未满足存在 u_i 有不可由其区分的元素，这里找到最小未出现在条件集中的 $n, j \neq 1$ 有存在 h 重排 a_{n1} 和 a_{nj} ，但 $h(z) = z$ 且对当前条件集 s ， $h(s) = s$ 。则由 i 有 $a_{n1} \in z$ 且 $a_{nj} \in z$ 。并以当前 a_{n1} 的条件集作为新 a_{nj} 的条件集。

需要指出的是寇恩在对连续统假设的否定形式相对一致性证明中，为得到所需结果的全部灵活性来自对可构造集合的构造方法的修改，即全部条件增加均只（需）考虑语句的真假性。

可见尽管概念上是类似的，但每次应用于不同场景时需改动多处（包括力迫条件、力迫关系和可构造集合构造方法）以完成证明，这提供了抽象和泛化的可能。本书介绍的概念即其抽象和泛化后的结果^{1 2 3 4 5 6}。具体的，力迫条件的集合被首先视为函数，力迫条件集合的可扩展顺序即函数间的包含（于）关系（即前述的 $\leq = \supseteq$ ），进一步力迫条件集合的集合及其上的包含关系被泛化为偏序结构（即 $p \leq$ ，并注意如 190 页注 2 所述， ∇ 的存在主要为简化讨论）。同时，最终的力迫条件集合（即扩展序列极限）对应于前文的通集⁷，其中条件 (G2) 的滤子向上有向性对应条件序列的扩展关系，向下闭合性对应条件集合不含矛盾，(G3) 对应条件足够确定全部语句的真假。也因此力迫关系也仅需出现在关于新模型性质的论证而非新模型的构造中。进一步，新模型的构造也被发现不需要依赖基于哥德尔运算且构造方法可变的相对可构造集合，因此仅对力迫法而言，哥德尔运算也是非必要的。但从选择公理否定形式的相对一致性证明中额外的内模型构造方法的应用来看，这里的抽象可见并不很能认为是无缺陷的。而更复杂的问题在于这种抽象带来的结果的潜在的难以理解性。

至此我们完成了本书关于集合论模型构造方法的讨论，也即完成了集合论公理集间相对一致性的讨论。下面我们从更基础的视角考察元逻辑中的这些以及相关概念。

¹S. Feferman. "Some applications of the notions of forcing and generic sets". eng. In: Fundamenta Mathematicae 56.3 (1964), pp. 325–345.

²William B. Easton. "Powers of regular cardinals". In: Annals of Mathematical Logic 1.2 (1970), pp. 139–178. issn: 0003-4843, 下注 3 引用了还未发表的该论文（中的定义）。

³Azriel Lévy. "Definability in axiomatic set theory I". in: Logic, methodology, and philosophy of science. Proceedings of the 1964 International Congress. Ed. by Y. Bar-Hillel. Amsterdam: North-Holland, 1965, pp. 127–151.

⁴Paul J. Cohen. Set Theory and the Continuum Hypothesis. New York: W. A. Benjamin, 1966, pp. 126–127.

⁵Robert M. Solovay. "A Model of Set-Theory in Which Every Set of Reals is Lebesgue Measurable". In: Annals of Mathematics 92.1 (1970), pp. 1–56. issn: 0003486X. URL: <http://www.jstor.org/stable/1970696> (visited on 08/24/2022).

⁶J. R. Shoenfield. Unramified Forcing. 1971.

⁷通集概念及其字面 (generic set) 最早已在寇恩 1963 年的论文 (208 页注 3) 中 (非正式的) 出现，其后来被固定为术语。Timothy Chow. Origin of the term "generic" in set theory. MathOverflow. URL: <https://mathoverflow.net/q/169271> 中包含关于其命名来源的讨论。

5.4 基础的基础

5.4.1 逆向数学

应已注意到我们常工作在不同理论下，特别的，本章中大部分定理等由蕴含关系 $\vdash$ 给出。可见我们会工作在 Z 、 ZF 、 ZFC 集合论下，对这三者的区分意为一些定理可以在更少的公理下完成¹，如前述，更少的公理意为着更弱的假设，意为更易接受的结果。更进一步，如定理 5.2.18 等工作在 P 下，这意味着公理集的选择并不一定只是数量上的，而 P 下可证的 Z 下亦可证，而 P 又是弱于 Z 的²，即可认为定理 5.2.18 是更可靠的。

可见由此引出了对特定定理、或为建立特定理论，哪些公理必要的问题。我们称这类问题为逆向数学。比如选择公理否定形式的相对一致性表明在一些场景下选择公理是必要的，因其不可由 A_{ZF} 得到。而如定理 5.2.18 所示，证明相关的问题应是可在 P 中形式化的，因本质上，演绎推导即总是处理的有限长字符串，或其即可被视为一种计算模型，其显然是不强于图灵机或 μ -递归函数的。

然而对于模型相关的讨论， P 是不足够的，因解释（中的关系）内在被定义为与集合相关的。然而我们仍可以尝试在 P 中（或算术语言中）解释模型。特别的，由勒文海姆-斯科伦定理 3.1.15 有理论存在模型时，存在域可数的模型，即其域是可被算术语言编码的。这里的问题在于关系（和函数），当全部关系可被算术定义时，我们才可以在 P 下编码该模型³。

然而，事实上，相比使用 A_Z ，我们可以进一步采用更弱的但可以谈论自然数集的算术理论完成大部分对模型的讨论⁴。

5.4.2 形式主义数学

关于证明可在 P 中形式化显然是可理解的，因演绎推导本质即是计算过程，计算过程本质就是操作（有限长）字符串，而其显然是可编码的。而对构建模型除了为逻辑系统提供语义外，其还是为证明逻辑系统的一致性（不完备的逻辑系统在一些场景下失效，而不一致的逻辑系统则总是无效的），这种证明是在元逻辑中发生的，则在元逻辑的元逻辑中是可被 P 进一步形式化的。而哥德尔第二不完备定理指出超过一定强度的逻辑系统（包括 P ）不能自证其一致性，而需在更强的系统中证明其一致性，则显然对于这种系统我们无法最终确定语义所指。事实上，由 $A \vdash \gamma$ 有 $\vdash \alpha \rightarrow \gamma$ ，其中语句 α 为语句集 A 的有限子集析取后的结果，从语义蕴含的角度，公理集是非必要的⁵（而公理集的有限可公理化为 α 是否可由公理集 A 得到提供了确认方式）。因此，当我们不再关注语义时，一切定理及其证明过程都可看作是对字符串形式上的操作。此可认为是形式主义数学的核心观念⁶。

在这种核心观念下，对公理的选择和演绎推导可仅被视为一种在一定规则下的游戏，其目标是尝试得到各种不同的定理。如何得到是游戏过程的技术性部分，而游戏目标（即目标定理）的不同将影响游

¹也应注意本书中对定理所需公理集的选择并不总是精确的而（应）可能趋于保守的、即含更多公理的。

²章节 4.2.4。

³参考 Noah Schweber. The arithmetized completeness theorem. Mathematics Stack Exchange. URL: <https://math.stackexchange.com/q/3802941>。

⁴可进一步参考 Noah Schweber. Ideal treatment of set theory as a meta theory for developing first-order logic. Mathematics Stack Exchange. URL: <https://math.stackexchange.com/q/3259546>。

⁵对比邱奇定理 3.2.13。

⁶Wikipedia contributors. Formalism (philosophy of mathematics) — Wikipedia, The Free Encyclopedia. [https://en.wikipedia.org/w/index.php?title=Formalism_\(philosophy_of_mathematics\)&oldid=1100232718](https://en.wikipedia.org/w/index.php?title=Formalism_(philosophy_of_mathematics)&oldid=1100232718). 2022。

戏本身的价值。一个好的目标应能反映物理世界的一个侧面，同时也应易被我们的意识所理解¹（尽管这里我们无法形式化理解的含义），而这两者应是在一定范围内一致的，因我们的意识可更好的理解我们身处的物理世界是我们作为生物得以延续和扩张的基础。

而区分这两者，集合论中的一些公理事实上更多的是为了简化集合论从而使我们更易理解和操控集合论。正则公理即是一个显见的例子，其通过显式的去掉不那么规则²的集合使域中任意元素满足如属于某一冯·诺依曼层级（定理 5.1.7），从而使模型更为规整；或均可以和属于关系构成良基结构（推论 5.1.12.1），从而可以在其上超限归纳/递归。选择公理是另一个例子，因在大部分情况下（如计算机科学）存在但无法描述的对象是无法处理的³。类似的，实数概念的最基本形式（即可能无限的小数）也是易于理解的，无理数的发现并不代表实数需要被如此定义，如在构造主义数学中实数即一般是以其计算方法定义的（则可见是可数的）⁴。最后，无限的概念（在最初）也应是简化讨论的，因我们难以有效的找到最后的自然数。

然而对简洁性的追求并没有带来可认为是简洁的结果。当我们希望将数学对象限制于正整数时，我们发现我们需要零和负数；而在整数时我们需要有理数、无理数等等。当我们希望将公理系统限制于朴素集合论时，我们又遇到了罗素悖论。当采用 Z 集合论时又发现一些符合直觉的集合存在性的缺失（引理 5.1.6）和不符合直觉的（即非正则的）集合存在的可能。而无限的引入同样引入了如何处理无限的问题，如欧氏几何和非欧几何公理集的唯一区别就在于前者认为平行线不相交而后者认为平行线在无限远处相交。而回到本书的例子，不完备性本质在于无法解决（如）停机问题（引理 3.2.14），即是否可能会执行无限次计算的问题。而一致性问题，在对集合论，我们使用了内模型和力迫法等方法尝试解决后，尽管这些方法在集合论下有一定的通用性，但其本质仍是我们无法解决的总是停机问题，即没有通用方法确定推导是否总是返回（即不一致）。在这一意义上，特定于集合论的方法显然又很难称之为通用的。然而注意计算理论仍是一种理论，其应是可在如算术语言中形式化的，这里提到如停机问题（的结论）等应是可在如 $\mathbf{P}$ 中形式化证明的。我们可以选择其它理论有其中停机问题可解，但这样的理论并不反映真实物理世界⁵。

下面我们来看不同的理论选择可带来潜在正面效果的例子。

5.4.3 非标分析^{6 7 8}

考虑章节 4.2.6 中关于函数连续性定义。直观的说，实数函数连续即指当函数自变量任意接近时其因变量（值）也任意接近。但显然我们定义任意接近的方式是曲折的，我们通过对任意因变量（函数值）差可找到一个非零的自变量区间有其中值的范围变化小于该差。而可见更直接的表达方式当自变量差任意小时因变量差也任意小。称这种任意小量为无穷小，我们事实上可构造这种无穷小量。

类似我们在章节 3.2.4 中构建域中含非（标准）自然数的算术模型，（在集合论语言中）添加常量 c ，考虑语句集 $\Gamma = A_{ZFC} \cup \{c >_{\omega} 0, c >_{\omega} 1, \dots\}$ ，可见同样由紧性定理 3.1.6（因其任意子有限集可满足） Γ 可满足。令其模型为 $\mathfrak{M}$ ，可见 $\mathfrak{M}$ 的 ω 中含非（标准）自然数 c （对应的元素）。而因 $\mathfrak{M}$ 同样满足 A_{ZFC} ，

¹对比章节 2.1.4。

²正则性即 regularity。

³后续章节将更进一步讨论选择公理在更贴近计算机科学的类型论中出现的形式。

⁴我们在后续章节处理浮点数时做进一步讨论。

⁵我们将在后续章节给出计算理论和如 $\mathbf{P}$ 之间关系的例子。

⁶Herbert B. Enderton. A Mathematical Introduction to Logic. New York: Academic Press, 1972, pp. 173–180。

⁷A. Robinson. Non-standard Analysis. Studies in logic and the foundations of mathematics. North-Holland Publishing Company, 1966。

⁸Edward Nelson. “Internal Set Theory: A New Approach to Nonstandard Analysis”. In: Journal of Symbolic Logic 48.4 (1977), pp. 1203–1204。

则 ω 中还含 $c+1$ 等；更相关的，其 $\mathbb{Q}$ 中含 $\frac{1}{c}$ 等，仍因其满足 A_{ZFC} ， $\frac{1}{c}$ 小于任意正（标准）有理数，如 $\frac{1}{10^{100}}$ 、 $\frac{1}{10^{1000}}$ 等。则进一步有实数 $\iota = \frac{1}{c_{\mathbb{R}}} = \{\omega \times \{\frac{1}{c}\}, \dots\}$ 小于任意正（标准）实数。 ι 即为无穷小量。可见直观的， ι 不等于但任意接近 0，同样存在 $\iota+1$ 不等于但任意接近 1，等等。我们称不考虑非标准自然数的实数分析为标准分析，则通过由非标准自然数得到的无穷小量完成的实数分析为非标分析。

上面的讨论是基于语义的，事实上，非标分析也可以基于语法、或者说在目标逻辑中（仅）通过额外的公理实现。具体的，下面介绍 ZFC 的保守扩展 IST 集合论。在给出保守的含义前，我们（需）先给出 IST 集合论的相关概念。IST 使用语言为集合论语言 $\mathcal{L}_S$ 基础上增加一个一元谓词符号 st ，称 $st(x)$ 为 x 是标准的。即如 $x \in \omega$ 且 $st(x)$ 表示 x 为标准自然数。称不含 st 的式 / 语句 α 为内部的，否则为外部的。同时，（补上内部）定义：

- $isFinite(x)$ 表示 $\neg \exists f isBijjective_{\mathbb{P}}(f, x, x)$ ，或等价的 $\exists a, f isOrd(a) \wedge a \in \omega \wedge isBijjective_{\mathbb{P}}(f, x, a)$ ，即 x 是有限的。

写 $\forall^{st} x \alpha$ 表示 $\forall x (st(x) \rightarrow \alpha)$ ， $\forall^{fin} x \alpha$ 表示 $\forall x (isFinite(x) \rightarrow \alpha)$ ， $\forall^{stfin} x \alpha$ 表示 $\forall x (st(x) \wedge isFinite(x) \rightarrow \alpha)$ 。类似的，写 $\exists^{st} x \alpha$ 等表示 $\exists x st(x) \wedge \alpha$ 等。修改 A_{ZFC} 中的分类和替换公理模式只可作用于内部式（显然其原只可作用于内部式，因此这里的改动并没有弱化这两个公理模式）。对此，注意若允许外部式作为如分类公理模式的条件，考虑 $\{x \in \omega : st(x)\}$ ，可见其应等于 ω ，则 ω 中不含非标准自然数，结合章节 4.2.1 对非标准自然数必然可存在的讨论，这里的 st 谓词显然不能是我们期望的含义¹。此外，增加以下三个公理模式：

- 理想化公理模式，对任意内部式 $\rho[u_1 \dots n, x, y]$ ：

$$\forall u_1 \dots n (\forall^{stfin} z \exists x \forall y \in z \rho \leftrightarrow \exists x \forall^{st} y \rho) \quad (I)$$

可见这即对应了紧性定理 3.1.6 的形式化，即若对任意标准有限集合 z ，存在 x 满足 z 中任意元素 y 的 ρ （给出的）性质，则存在（全局的） x 满足任意标准元素 y 的 ρ 性质。如我们代 $x >_{\omega} y$ 入 ρ 即可证明存在大于任意标准自然数的非标准自然数。

- 标准化公理模式，对任意（内部或外部）式 $\rho[u_1 \dots n, z]$ ：

$$\forall u_1 \dots n \forall^{st} x \exists^{st} y \forall^{st} z (z \in y \leftrightarrow z \in x \wedge \rho) \quad (S)$$

这条公理作为对分类和替换公理模式缺失对外部式处理的补偿，使我们可以通过可含标准谓词 st 的式 ρ 在标准集合 x 的基础上证明标准集合 y 的存在。直观的说，其表明我们只关注标准集合中标准元素的性质。

- 传递公理模式，对任意内部式 $\rho[u_1 \dots n, x]$ ：

$$\forall^{st} u_1 \dots n (\forall^{st} x \rho \rightarrow \forall x \rho) \quad (T)$$

直观的，即对应应在标准模型中为真的语句在非标准模型下也应为真。

写 IST 的公理集为 A_{IST} 。

现说明 IST 为保守 ZFC 保守扩展的含义，并做一定讨论。这里称 IST 为 ZFC 的保守扩展因在 IST 下可证明的内部语句在 ZFC 下亦可证明。同时反方向显然成立。即对任意内部语句 α ， $A_{IST} \vdash \alpha$ 等价于 $A_{ZFC} \vdash \alpha$ 。对正方向 $(A_{IST} \vdash \alpha) \rightarrow (A_{ZFC} \vdash \alpha)$ ，其证明可概述为等价于证明 $\neg(A_{ZFC} \models \alpha) \rightarrow \neg(A_{IST} \models \alpha)$

¹事实上，其也与 IST 公理矛盾，见定理 5.4.1。

α), 等价于存在 A_{ZFC} 的模型 $\mathfrak{M}$ 满足 $\neg\alpha$ 蕴含 A_{IST} 的模型 $\mathfrak{N}$ 同样满足 $\neg\alpha$, 而 $\mathfrak{N}$ 是可基于 $\mathfrak{M}$ 构造的。可见元逻辑下保守性的证明仍是基于语义的。同时这种保守性意味着 IST 从形式化角度看更多的可被视为一种证明技巧 (而非新的理论)¹。

下面我们进一步对非标分析的性质进行展开, 并最终证明在章节 4.2.6 中已通过标准分析证明的中值定理以示区别。

定理 5.4.1 $A_{IST} \vdash st(s) \wedge isFinite(s) \leftrightarrow \forall x \in s st(x)$ 。即 s 为标准有限集当且仅当其中元素均是标准的。

证明: 应用理想化公理 (I) 取其 $\rho_{[u_1, x, y]}$ 为 $x \in s \wedge x \neq y$, 我们有 $\forall^{st} \dot{z} \exists x \forall y \in z x \in s \wedge x \neq y$ 等价于 $\exists x \forall^{st} y x \in s \wedge x \neq y$, 即等价于 $\exists x \in s \neg st(x)$ 。两侧取否有 $\exists^{st} \dot{z} \forall x \exists y \in z x \notin s \vee x = y$ 等价于 $\forall x \in s st(x)$, 前者等价于 $\exists^{st} \dot{z} z \subseteq s$ 。对于正方向, 即 s 为标准有限集, 取 $z = s$, 有 $\forall x \in s st(x)$ 。对于反方向, 有存在标准有限 z 有 $s \subseteq z$, 考虑 $p = \wp(z)$, $s \in p$ 且因 z 有限 p 是有限的, 则若可以证明 p 还是标准的, 则应用正方向有 s 是标准有限的。

我们证明 p 是标准的。结合传递公理模式 (T), 对任意 $\rho_{[u_1 \dots n, x, y]}$, 显然:

$$\forall^{st} u_1 \dots n (\forall^{st} x \rho \leftrightarrow \forall x \rho) \quad \forall^{st} u_1 \dots n (\exists^{st} x \rho \leftrightarrow \exists x \rho) \quad (5.4.1)$$

而由 A_{ZFC} 可得 $\forall z \exists! p \forall x (x \in p \leftrightarrow x \subseteq z)$ 。则展开 $\exists!$ 并应用 (5.4.1) 有 $\forall^{st} z \exists!^{st} p \forall x (x \in p \leftrightarrow x \subseteq z)$ 。即对标准 z 存在唯一的标准 p 为其幂集。□

类似应用 (5.4.1), 对任意内语句 α 相对 (标准) 化, 写 $\alpha|_{st}$ 表示将其中 $\forall x \varphi$ 均改为 $\forall^{st} x \varphi$, $\exists x \varphi$ 为 $\exists^{st} x \varphi$, 有:

$$A_{IST} \vdash \alpha|_{st} \leftrightarrow \alpha \quad (5.4.2)$$

回到定理:

推论 5.4.1.1 $A_{IST} \vdash \neg isFinite(s) \rightarrow \exists x \in s \neg st(x)$ 。即任意无限集含非标准元素。

则 ω 含非标准元素。这里有限判定谓词的定义需考虑这些非标准元素, 但同时, (类似其它内部语句) ω 在 ZFC 中可证明的性质在 IST 中亦可证。如 (因否则在 ZFC 中可构造更小的归纳集):

$$A_{IST} \vdash \forall n \in \omega n = 0 \vee \exists m \in \omega n = m^+ \quad (5.4.3)$$

$$A_{IST} \vdash isFinite(s) \wedge isStrictTotalOrder(s, r) \rightarrow \exists x \in s isMinimal(x, s, r) \quad (5.4.4)$$

结合下述定理, 非标分析带来的便利性将可 (能) 超过其中如非标准自然数带来的不直观性。

定理 5.4.2 $A_{IST} \vdash \exists^{fin} w \forall^{st} y y \in w$ 。即存在有限集 w 包含全部标准集合。

证明: 应用 (I) 取 $\rho_{[w, y]}$ 为 $y \in w \wedge isFinite(w)$ 。□

这个反直觉的定理成立的原因在于存在非标准自然数。然而也可以从其中看出, 我们可以按照有限集合的方式处理含某类标准元素的集合, 如类似有 $\exists^{fin} w \forall^{st} y y \in \mathbb{R} y \in w$ ², 则结合 (5.4.4) 有其中含最小实数。我们将借助这种方式证明以下非标分析下的连续性定义等价于 (章节 4.2.6 中) 标准分析下的定义:

- 对任意实数 $x y$, 写 $x \simeq y$ 表示 $\forall^{st} \varepsilon \in \mathbb{R}_{>0} |x - y| < \varepsilon$, 称 $x y$ 无限接近。

¹保守性的完整证明见 211 页注 8。

²取 $\rho_{[w, y]}$ 为 $(y \in \mathbb{R} \rightarrow y \in w) \wedge isFinite(w)$ 。

- 对任意标准 $f : d \mapsto \mathbb{R}$ ($d \subseteq \mathbb{R}$), 标准 $x \in d$, 写 $isS\text{-ContinuousAt}(f, x)$ 表示:

$$\forall y \in \text{dom } f (x \simeq y \rightarrow f(x) \simeq f(y))$$

称 f 在 x S -连续。写 $isS\text{-Continuous}(f)$ 表示 $\forall^{st} x \in \text{dom } f \text{ } isS\text{-ContinuousAt}(f, x)$, 称 f S -连续。

即:

引理 5.4.3 $A_{IST} \vdash d \subseteq \mathbb{R} \rightarrow \forall^{st} f : d \mapsto \mathbb{R} (isS\text{-Continuous}(f) \leftrightarrow isContinuous(f))$ 。

证明: 展开连续性定义, 我们需要证明 $\forall^{st} x \in d \text{ } isS\text{-ContinuousAt}(f, x) \leftrightarrow \forall x \in d \text{ } isContinuousAt(f, x)$ 。结合 (5.4.2), 仅需证明 $\forall^{st} x \in d \text{ } isS\text{-ContinuousAt}(f, x) \leftrightarrow \forall^{st} x \in d \text{ } isContinuousAt(f, x)$ 。或对任意 (标准 f)、标准 $x \in d$:

$$isS\text{-ContinuousAt}(f, x) \leftrightarrow isContinuousAt(f, x) \quad (5.4.5)$$

完全展开 S -连续定义, 有上式左侧等价于:

$$\forall y \in d (\forall^{st} \delta \in \mathbb{R}_{>0} |x - y| < \delta \rightarrow \forall^{st} \varepsilon \in \mathbb{R}_{>0} |f(x) - f(y)| < \varepsilon)$$

由类似前束范式转换规则 (见定理 3.2.5 证明), 有进一步等价于:

$$\forall y \in d \forall^{st} \varepsilon \in \mathbb{R}_{>0} \exists^{st} \delta \in \mathbb{R}_{>0} \overbrace{(|x - y| < \delta \rightarrow |f(x) - f(y)| < \varepsilon)}^{\varphi}$$

等价于 i: $\forall^{st} \varepsilon \in \mathbb{R}_{>0} \forall y \in d \exists^{st} \delta \in \mathbb{R}_{>0} \varphi$ 。其中 $\forall y \in d \exists^{st} \delta \in \mathbb{R}_{>0} \varphi$ 等价于 $\forall y \exists^{st} \delta \overbrace{(y \in d \rightarrow \delta \in \mathbb{R}_{>0} \wedge \varphi)}^{\phi}$ 。取 $\rho_{[y, \delta]}$ 为 $\neg \phi$ 应用 (I) 有等价于 $\exists^{st \text{fin}} z \forall y \exists \delta \in z \phi$ 。等价于 $\exists^{st \text{fin}} z \forall y \in d \exists \delta \in z \cap \mathbb{R}_{>0} \varphi$ 。反向应用前束范式转换规则有 i 等价于:

$$\forall^{st} \varepsilon \in \mathbb{R}_{>0} \exists^{st \text{fin}} z \forall y \in d \overbrace{(\forall \delta \in z \cap \mathbb{R}_{>0} |x - y| < \delta \rightarrow |x - y| < \varepsilon)}^{\psi}$$

则由 (5.4.2) 有其等价于 $\forall \varepsilon \in \mathbb{R}_{>0} \exists^{st \text{fin}} z \forall y \in d \psi$ 。由 z 有限 $z \cap \mathbb{R}_{>0}$ 中含最小实数 δ' , 使其等价于:

$$\forall \varepsilon \in \mathbb{R}_{>0} \exists \delta' \in \mathbb{R}_{>0} \forall y \in d (|x - y| < \delta' \rightarrow |x - y| < \varepsilon)$$

其即为 (5.4.5) 右侧的定义。¹

□

而对于标准化:

引理 5.4.4 对任意式 $\rho_{[u_1, \dots, n, z]}$, $A_{IST} \vdash \forall u_1, \dots, n \forall^{st} x \exists!^{st} y \forall^{st} z (z \in y \leftrightarrow z \in x \wedge \rho)$ 。

证明: 对比标准化公理模式 (S), 这里仅需证明证明唯一性。若 $y_1 y_2$ 均满足上述 y 的要求, 显然可以看到 $\forall^{st} z \in y_1 z \in y_2$, 即 $y_1 \subseteq y_2 \upharpoonright_{st}$, 由 (5.4.2) 有 $y_1 \subseteq y_2$ 。类似的 $y_2 \subseteq y_1$, 有 $y_1 = y_2$ 。□

则可:

- 对任意 x , 条件 $\rho_{[x]}$, 写 $^S\{z \in x : \rho\}$ 表示由标准化自 x 和 ρ 得到的 (唯一且标准) 集合。或如前, 写 $P(^S\{z \in x : \rho\})$ 表示 $\forall^{st} y (\forall^{st} z (z \in y \leftrightarrow z \in x \wedge \rho) \rightarrow P(y))$ 。

在此基础上, 我们考虑非标准实数和标准实数之间的关系, 定义:

- 对任意实数 x , 写 $isLimited(x)$ 表示 $\exists^{st} r \in \mathbb{R} |x| < r$ 。称 x 是有限的。

¹完整外部语句到内部语句的转换方法见 211 页注 8。

有：

定理 5.4.5 $A_{IST} \vdash x \in \mathbb{R} \wedge isLimited(x) \rightarrow \exists !^{st} y \in \mathbb{R} x \simeq y$ 。即任意有限实数存在唯一的与之任意接近的标准实数¹。

证明：由 x 为有限实数，存在 r 有 $|x| < r$ 。考虑集合 $t = {}^S\{u \in \mathbb{R} : u \leq x\}$ ， t 有上界 r 因其中标准元素均小于标准 r 注²，且 t 非空因 $-r \in t$ 。则由实数完全性定理 4.2.17 存在唯一实数 y 为 t 的最小上界。则（类似定理 5.4.1 中标准集合的幂集是标准的）因 t 是标准的， y 是标准的。我们证明 $x \simeq y$ 。

若存在标准正实数 ε 有 $x - y \geq \varepsilon$ ，即 $y + \varepsilon \leq x$ ，结合 $y + \varepsilon$ 标准， $y + \varepsilon \in t$ ，与 y 为 t 上界矛盾。若存在标准正实数 ε 有 $y - x \geq \varepsilon$ ，即 $x \leq y - \varepsilon$ ，则由 $y - \varepsilon$ 标准有任意标准 $u \in t$ 有 $u \leq y - \varepsilon$ ，即 $y - \varepsilon$ 为 t 上界，与 y 为最小上界矛盾。

最后对唯一性，考虑任意标准 $y' \simeq x$ ，有 $y \simeq y'$ 注³，而 $|y - y'|$ 标准，有 $y - y' = 0$ ， $y = y'$ 。□

则可：

- 对任意有限实数 x ，写 $st x$ 表示与之任意接近的标准实数。或 $P(st x)$ 表示 $\forall^{st} y \in \mathbb{R} (x \simeq y \rightarrow P(y))$ 。称 $st x$ 为 x 的标准部分。

至此，我们可在非标分析下证明中值定理：

定理 5.4.6 (中值定理 IST)

$$A_{IST} \vdash a \in \mathbb{R} \wedge b \in \mathbb{R} \wedge a < b \wedge f : [a, b] \mapsto \mathbb{R} \\ \wedge isContinuous(f) \wedge f(a) < 0 \wedge f(b) > 0 \rightarrow \exists c \in dom f f(c) = 0$$

证明：令 $d = [a, b]$ 。结合引理 5.4.3 有前提蕴含对任意标准实数 a, b 满足 $a < b$ ，任意标准函数 $f : d \mapsto \mathbb{R}$ 满足 f S -连续且 $f(a) < 0$ 且 $f(b) > 0$ 。由 (5.4.1) 仅需证存在标准 $c \in d$ 有 $f(c) = 0$ 。

类似定理 5.4.2 (及讨论)，存在有限的 w 包含全部 d 中标准实数。考虑 $v = \{u \in w : f(u) \geq 0\}$ ，有 $v \subseteq w$ ，有 v 有限，结合 (5.4.4)，其中有最小 x 。而 $f(x) \geq 0$ 。令 $c = st x$ ，有 $x \simeq c$ ，由 S -连续有 $f(x) \simeq f(c)$ ，则因 $f(c)$ 标准， $f(c) \geq 0$ 。

因标准 $a \in d$ 且 $f(a) < 0$ ， x 不为 w 中的最小实数。而 w 有限，由 (5.4.3) 有 w 中存在 x 的前一个实数 y (即 y 为比 x 小的最大实数)。由 $y \notin v$ ， $f(y) < 0$ 。而因 w 中含全部标准实数，有 $y \simeq x$ (否则有标准实数大于 y 小于 x 不属于 w)，有 $y \simeq c$ ，有 $f(y) \simeq f(c)$ ，则同上 $f(c) \not\geq 0$ 。则 $f(c) = 0$ 。□

则结合 IST 的保守性，我们有章节 4.2.6 中的中值定理 4.2.27。同时可见在非标分析中，我们可以以类似离散的方式操作实数，而这种离散性以无穷小量作为间隔，如可视上面的 d 区间被间隔为（大于任意标准自然数的）非标准自然数段。

5.4.4 形式化数学数据库

到目前为止，除去直观性的讨论外，我们一再强调的是技术上的可形式化性。事实上，第一章中引入后续章节中会详细讨论的 Coq 亦可作为数学理论形式化的工具。在当前形式化百大数学定理的网站⁴中我们可以看到 Coq 解决了其中 78 个，同时我们可以看到还有其它工具共完成了 98 个定理的形式化（证

¹如任意无穷小量任意接近且只任意接近 0。

²但不一定有上界 x 因若 x 非标准时，不能由 (T) 得到 t 中任意标准元素均小于 x 蕴含任意元素均小于 x 。下面非空因 $-r \in t$ 同理。

³对任意标准正实数 ε ， $|y - y'| \leq |y - x| + |y' - x| < \frac{\varepsilon}{2} + \frac{\varepsilon}{2} = \varepsilon$ 。

⁴<http://www.cs.ru.nl/~freek/100/>. [Last modified: 2021-06-22]。

明)。本小节将介绍的是其中排第四的 Metamath¹。相对于 Coq 趋向于使用直觉主义逻辑，Metamath 趋向于使用经典逻辑。相对于 Coq 可同时用于验证软件正确性，Metamath 的基本目标在于建立数学理论（公理、定理及其证明）的数据库。其主数据库（经典逻辑下的 ZFC 理论）目前包含了 23,000 条证明，被认为是当前最大的数学形式化数据库²。

下面我们首先给出一个其作为数据库被检索的例子。在 Metamath 安装目录下³，运行 `./metamath set.mm` 加载经典逻辑下的 ZFC 集合论数据库（即 `set.mm`）进入交互界面。我们将检索定理 $2 + 2 = 4$ 的相关内容。如下通过 `SEARCH * "2 + 2" /COMMENTS` 搜索注释中含 $2 + 2$ 的公理 / 定理：

```

1 [demo] /metamath-exe-0.198/ $ ./metamath set.mm
2 Metamath - Version 0.198 7-Aug-2021          Type HELP for help, EXIT to exit.
3 MM> READ "set.mm"
4 Reading source file "set.mm"... 42546663 bytes
5 42546663 bytes were read into the source buffer.
6 The source has 202888 statements; 2691 are $a and 40431 are $p.
7 No errors were found. However, proofs were not checked. Type VERIFY PROOF *
8 if you want to check them.
9 MM> SEARCH * "2 + 2" /COMMENTS
10 27248 o2p2e4 $p "2 + 2 = 4 for ordinal numbers. Ordinal numbers are modeled..."
11 37346 2p2e4 $p "...2p2e4 is similar but proves `2 + 2 = 4` using ordinal n..."
12 50723 6gcd4e2 $p "...d ` ( 2 gcd 4 ) = ( 2 gcd ( 2 + 2 ) ) = ( 2 gcd 2 ) = ..."
13 111593 conventions $p "...xample, the ` + ` in ` ( 2 + 2 ) ` ; see ~ 2p2e4 ..."
14 202819 2p2ne5 $p "Prove that ` 2 + 2 /= 5 ` . In George Orwell's "1984", P..."
15 MM> █

```

图 5.2: (交互界面下) 运行 `SEARCH * "2 + 2" /COMMENTS` 后的结果

如图示，`MM>` 为交互界面的提示符，第 10 至 14 行为搜索结果。其中第二列为名 / 标签；第三列 `$p` 表示定理，有其它类型如 `$a` 表示公理；最后一列表示注释上下文。如我们在自然数、整数等下有不同 2 的定义方式⁴，注意其 `o2p2e4`（第 10 行）为序数 $2 + 2 = 4$ 的定理，其显然是区别于 `2p2e4`（第 11 行）的。现我们来看 `2p2e4` 具体指什么：

```

1 MM> SHOW STATEMENT 2p2e4 /COMMENT
2 "Two plus two equals four. For more information, see "2+2=4 Trivia" on the
3 Metamath Proof Explorer Home Page: ~ mmset.html#trivia . This proof is
4 simple, but it depends on many other proof steps because 2 and 4 are
5 complex numbers and thus it depends on our construction of complex
6 numbers. The proof ~ o2p2e4 is similar but proves ` 2 + 2 = 4 ` using
7 ordinal natural numbers (finite integers starting at 0), so that proof
8 depends on fewer intermediate steps. (Contributed by NM, 27-May-1999.)"
9 37346 2p2e4 $p |- ( 2 + 2 ) = 4 $= ... $. █

```

图 5.3: `SHOW STATEMENT 2p2e4 /COMMENTS`

¹<http://metamath.tirix.org/index.html>. [Last modified: 2020-11-32].

²Wikipedia contributors. Metamath — Wikipedia, The Free Encyclopedia. <https://en.wikipedia.org/w/index.php?title=Metamath&oldid=1104552483>. 2022.

³可通过: `docker run -it gcc:12.2.0`

```

wget https://codeload.github.com/metamath/metamath-exe/zip/refs/tags/v0.198
wget https://github.com/metamath/set.mm/archive/f49c1f59a3ba988631ba9d23722255d418b0543b.zip
unzip v0.198
unzip f49c1f59a3ba988631ba9d23722255d418b0543b
cp set.mm-f49c1f59a3ba988631ba9d23722255d418b0543b/*.mm metamath-exe-0.198
cd metamath-exe-0.198
gcc *.c -o metamath

```

注意 Metamath 官网有最新版源码安装包，这里从 github 上下载仅为固定版本。同时可见两个仓库分别是 metamath 可执行文件源码和数据库文件。

⁴章节 4.2.1、章节 4.2.4。

注释中可以关注到两方面信息，首先是 i: 2p2e4 中的 2 (和 4) 是复数¹。其次是 ii: 这里简化的证明依赖于这里复数所谓的相关构造。通过检查其证明我们将除看到其证明方式外，并最终理清这两点的含义：

```

1 MM> SHOW PROOF 2p2e4
2 17   oveq1i.1=df-2   $a |- 2 = ( 1 + 1 )
3 18   eqtr4i.1=oveq2i $p |- ( 2 + 2 ) = ( 2 + ( 1 + 1 ) )
4 32   3eqtri.1=df-4   $a |- 4 = ( 3 + 1 )
5 37   oveq1i.1=df-3   $a |- 3 = ( 2 + 1 )
6 38   3eqtri.2=oveq1i $p |- ( 3 + 1 ) = ( ( 2 + 1 ) + 1 )
7 42   axi.1=2cn       $p |- 2 e. CC
8 43   axi.2=ax-1cn    $a |- 1 e. CC
9 44   axi.3=ax-1cn    $a |- 1 e. CC
10 45   3eqtri.3=addassi $p |- ( ( 2 + 1 ) + 1 ) = ( 2 + ( 1 + 1 ) )
11 46   eqtr4i.2=3eqtri $p |- 4 = ( 2 + ( 1 + 1 ) )
12 47 2p2e4=eqtr4i     $p |- ( 2 + 2 ) = 4

```

图 5.4: SHOW PROOF 2p2e4

行号外，我们称自第二行起第一个数字为步骤编号。显而易见的部分在于其为证明树，根节点第 12 行、即第 47 步即我们所需的结果，其子节点为第 3 行和第 11 行。其含义为由 $(2 + 2) = (2 + (1 + 1))$ 和 $4 = (2 + (1 + 1))$ 可得 $(2 + 2) = 4$ 。我们进一步对该步骤展开，我们也将逐步看到这里步骤编号不全的原因：

```

1 Proof step 47: 2p2e4=eqtr4i $p |- ( 2 + 2 ) = 4
2 This step assigns source "eqtr4i" ($p) to the final assertion being proved.
3 The source assertion requires the hypotheses "cA" ($f, step 4), "cB" ($f, step
4 11), "cC" ($f, step 12), "eqtr4i.1" ($e, step 18), and "eqtr4i.2" ($e, step
5 46). The target has no parent because it is the assertion being proved.
6 The source assertion before substitution was:
7   eqtr4i $p |- A = C
8 The following substitutions were made to the source assertion:
9   Variable Substituted with
10  A         ( 2 + 2 )
11  C         4

```

图 5.5: SHOW PROOF 2p2e4 /DETAILED_STEP 47

其指出 47 步应用了定理 eqtr4i 在有第 4、11、12、18 和 46 步提供的前提 cA、cB、cC、eqtr4i.1 和 eqtr4i.2 下可得到定理 2p2e4。我们现检查这些前提：

```

1 MM> SHOW STATEMENT cA
2 7797 cA $f class A $.
3 MM> SHOW STATEMENT eqtr4i.1
4 8044 eqtr4i.1 $e |- A = B $.
5 MM> SHOW STATEMENT eqtr4i.2
6 8045 eqtr4i.2 $e |- C = B $.

```

图 5.6: SHOW STATEMENT ...

可见对 eqtr4i 的应用即对其中的 A B C 进行替换。直观的说前提 cA 即指出了替换的类型要求、即 A 需为 class，其本质上即对应章节 4.3 中定义的类型。可见这里的前提开始涉及元的部分了（这其中的基础

¹即由实部和虚部两部分组成的扩展了实数概念的另一种数集，写作 $a+bi$ ，其中 a, b 为实数， a 为实部， bi 为虚部。当 $b=0$ 时，其即等于实数 a 。这里不会用到 $b \neq 0$ 的情况，但简单的说虚部的存在是为解决负数没有平方根的问题，其中虚部单位 $i = \sqrt{-1}$ 。

性也是其被隐藏的原因)，而事实上，class 仍不是内建的：

```

1 MM> SHOW STATEMENT cab /FULL
2 Statement 7796 is located on line 24131 of the file "set.mm". Its statement
3 number for HTML pages is 2792.
4 "Introduce the class builder or class abstraction notation ("the class of
5 sets `x` such that `ph` is true"). Our class variables `A`,
6 `B`, etc. range over class builders (implicitly in the case of defined
7 class terms such as  $\sim$ df-nul). Note that a setvar variable can be
8 expressed as a class builder per theorem  $\sim$ cvjust, justifying the
9 assignment of setvar variables to class variables via the use of  $\sim$ cv."
10 7796 cab $a class { x | ph } $.
11 Its mandatory hypotheses in RPN order are:
12 wph $f wff ph $.
13 vx $f setvar x $.
14 The statement and its hypotheses require the variables: x ph
15 The variables it contains are: x ph

```

图 5.7: SHOW STATEMENT cab /FULL...

可见 iii：其形式为 $\$a \text{ class } \{ x \mid ph \}$ ，其中 iv：要求 wff ph，setvar x。结合前述 \$a 表示公理，可见 iii 事实上为语法形式（而非专门类型）的定义，即若这里写 $\$a \dots \text{class}$ ，则 cA 应写作 A class。即在 Metamath 引擎仅有按语法替换语句中的词汇，而没有类型的概念（尽管上层构建了类型并在如网页等浏览工具中标注了这些类型）。类似的， $\{ x \mid ph \}$ 也是语法上的，其性质基于如下定义，e. 表示 $\in$ ：

```

1 MM> SHOW STATEMENT df-clab
2 7901 df-clab $a |- ( x e. { y | ph } <-> [ x / y ] ph ) $.

```

图 5.8: SHOW STATEMENT df-clab ^{注1}

对于 iv，其引用了另外两种（上层）类型，前者 setvar 对应自由/约束变量，但 Metamath（在引擎）不区分自由/约束变量，其采用了不同于我们采用的自由约束变量替换规则²。而后者 wff 即对应（合）式。而 wff 的合法性也是通过和上述 class 相同的方式给出的，如：

```

1 MM> SHOW STATEMENT wn
2 42 wn $a wff -. ph $.

```

图 5.9: SHOW STATEMENT wn

其中 $\neg$ 表示 $\neg$ 。当有 /FULL 时可见其前提要求 wff ph ^{注3}。类似的，也可以想见 2 亦是如此定义的，事实上图 5.4 第 2 行即指出 df-2 为 $\$a \mid - 2 = (1 + 1)$ 。这样定义方式的一个好处在于统一，而需要注意的是公理的增加意味着不一致风险的增加，如若定义 $\$a \mid - 2 = (1 + 2)$ ，显然语法上（即引擎层面）其完全是合法的，但其即会导致理论的不一致。不过有额外工具可以检查符合一定惯例的定义公理（其命名以 df- 起），而几乎全部定义可以按照该惯例定义。

现在回到图 5.4 下的两个问题。对于 i，已经看到 2 被定义为 $(1 + 1)$ ，而：

¹使用 SHOW STATEMENT df-sb 进一步查询 $[x / y] ph$ 的定义。

²回顾章节 3.1.1，我们为使得推导规则相对简单，在式的构成上添加了自由/约束变量之间的区分要求，而在需要的时候允许任意的替换变量以满足构成要求。虽是技术上繁琐的细节，但仍显然我们的方式很难被认为是一个好解决方案。这里提供了另一种解决方式（也带来了推导细节上的一些区别）。

³注意我们在章节 5.1.1 中形式化证明所采用的树同样有最终的子节点为语言符号。

```

1 MM> SHOW STATEMENT df-1
2 35023 df-1 $a |- 1 = <. 1R , 0R >. $.

```

图 5.10: SHOW STATEMENT df-1

对于 ii，图 5.4 中第 8 行 `ax-1cn` 指出 $\$a \vdash 1 \in \text{CC}$ ，其中 CC 表示复数集合。即第 10 行应用 `addassi` 为 $\$p \vdash ((A + B) + C) = (A + (B + C))$ 要求 ABC 为复数。但注意到 `ax-1cn` 为公理，此情况下其属于复数和 `df-1` 是完全无关的。事实上：

```

1 MM> SHOW STATEMENT ax-1cn /COMMENT
2 "1 is a complex number. Axiom 2 of 22 for real and complex numbers,
3   justified by theorem ~ ax1cn . (Contributed by NM, 1-Mar-1995.)"
4 35161 ax-1cn $a |- 1 e. CC $.

```

图 5.11: SHOW STATEMENT ax-1cn /FULL

即其虽为公理，但 `ax1cn` 给出了其证明。因而 `2p2e4` 的证明得到了简化。

另一方面，Metamath 交互界面还提供了一定的定理证明逆向信息的查询，如用到的公理（和定义）：

```

1 MM> SHOW TRACE_BACK 2p2e4 /ESSENTIAL /AXIOMS
2 Statement "2p2e4" assumes the following axioms ($a statements):
3   ax-mp ax-1 ax-2 ax-3 df-bi df-an df-or df-3an df-tru df-ex df-nf ax-gen ax-4
4   ax-5 df-sb ax-6 ax-7 ax-9 ax-10 ax-11 ax-12 ax-13 ax-ext df-clab df-cleq
5   df-clel df-nfc df-ne df-ral df-rex df-rab df-v df-dif df-un df-in df-ss
6   df-nul df-if df-sn df-pr df-op df-uni df-br df-iota df-fv df-ov ax-resscn
7   ax-1cn ax-icn ax-addcl ax-addrcl ax-mulcl ax-mulrcl ax-addass ax-i2m1 ax-1ne0
8   ax-rrecex ax-cnre df-2 df-3 df-4

```

图 5.12: SHOW TRACE_BACK 2p2e4 /ESSENTIAL /AXIOMS

进一步检查 `ax-mp ax-1 ax-4 ax-ext` 等可以看到熟悉的公理，其分别对应肯定前件、命题逻辑中的某逻辑公理、一阶逻辑中的逻辑公理和 ZFC 中的外延公理等。而在没有 `/ESSENTIAL` 时，其将同时列出语法结构（类型）相关的定义，其中可以看到 `wn cab` 等。注意这里包含了如 `ax-1cn` 等简化，因此更应可以看到 `2p2e4` 背后证明的复杂性。

最后，Metamath 有其它数据库对应不同的理论系统，这也是其引擎工作在相当基础的规则下带来的好处。同时，Metamath 的数据库文件（如 `set.mm`）即为文本文件，除证明树的保存（一般会）采用最基本的压缩算法外，其是完全可读的¹。

5.5 本章小节

- Z 集合论可形式化定义证明与模型，塔斯基真值语义即是以集合论为元逻辑定义的。

¹在其中甚至可以找到如 `latexdef "e." as "\in ";`，即 `e.` 为 $\in$ 。

- 冯·诺依曼全集为全体集合提供了一个直观的层级结构，其不同层级下有对应不同 ZFC 集合论公理子集的模型。这些子集的模型同时可被用于证明其它公理的独立性。
- 哥德尔第二不完备定理指出了一定强度理论绝对一致性的不可证明性，因此我们更多关注相对一致性，即在假设较弱理论一致性的前提下证明较强理论的一致性。
- 正则公理保证了全部集合位于冯·诺依曼全集中，其排除了直观上不合理的集合。
- 可构造全集为冯·诺依曼全集中可由式递归可构造的集合构成的层级结构。由可构造全集构成的模型满足 A_Z 以及可构造公理，可构造公理蕴含选择公理。
- 可构造全集构成的模型为其所在冯·诺依曼全集构成模型的内模型，内模型的定义可认为是对它的泛化。
- 绝对性和相对化是我们处理内模型中语句真假性的有效方法，如以此我们可以证明内模型满足所要求的公理。
- 算术理论如 P 也可形式化定义证明过程。因此相对一致性证明也可以 P 作为元逻辑。
- 相比于内模型向内寻找新模型满足额外语句。力迫法向外构造新模型满足额外语句。其构造过程通过系统的向原模型中添加相关集合以保证新集合除满足额外语句外，仍满足原公理集。
- 力迫法的关键在于力迫关系，其在我们的构造法和新模型中语句的真假之间建立了有效的关联以使我们的构造法可以保证额外语句和原公理集。
- 通过（更为现代的、本书中主要介绍的）力迫法我们可以直接证明如可构造公理相对于 A_Z 的独立性。
- 但我们需要进一步在由迭代力迫得到的模型基础上应用内模型构造由全部特定遗传序数可定义集合构成的模型以证明选择公理的独立性。其关键在于构造使其中有集合总是含相对不可区分的元素。
- 对力迫法发展过程的讨论（应）在一定程度上反映了方法等的直观性逐渐模糊的过程。
- P 可形式化证明，但其不足以形式化模型。但我们也并非一定要在强如 Z 的理论中讨论模型。对元逻辑的强度要求，或更一般的，对逻辑定理的公理要求构成了逆向数学。
- 而对哥德尔第二不完备定理，换一句话说，当理论有一定强度时，我们总是无法找到其最终的元逻辑，或其最终的语义。因此撇开语义，我们可以以纯形式的方式认识数学，该认识论被称为形式主义数学。但从应用考虑，形式主义下，理论仍应是符合客观物理世界且易于理解的。
- 非标分析为处理实数分析提供了另一套（可能更易理解或处理的）解决方案。其从纯形式证明的角度有扩展的集合论理论如 IST。
- 已有工具如 Metamath 尝试以形式化的方式为（不同逻辑系统下的）数学建立数据库。